

Kubernetes

Complete Interview Study Notes

Beginner → Advanced · 83 Topics & Capstones · GKE + EKS

Foundations & Workloads | **Config, Networking, Storage**

Scheduling & Security | **Observability & Scaling**

Advanced & Managed K8s | **Interview Deep Dive + Capstones**

13 Modules · Pre-Module + Topics 1–73 + 10 Capstone Labs

Prepared for candidates with 1–4 years of experience

Compiled 27 September 2026

Table of Contents

Table of Contents

🏗 Pre-Module — Lab Environment Setup
- 📋 What You Will Set Up
- 🛠 Section 1 — Local Tools to Install (All OS)
- 💻 Section 2 — Local Cluster Setup
- ☁ Section 3 — GKE Setup (Google Cloud)
- ▩ Section 4 — EKS Setup (AWS)
- ☠ Section 5 — Cost-Safety Checklist (READ BEFORE EVERY CLOUD LAB)
- 🧪 Section 6 — Pre-Module Verification Lab
- 🏗 Section 7 — Workflow Diagram
- 🎯 Interview Deliverable — 60-second Answer

📋 Topic 1 — What is Kubernetes? History & Why Kubernetes
- 🔹 Definition
- 🔹 About
- 🔹 How It Integrates With Other Kubernetes Components
- 🏗 Architecture / Flow Diagram
- 💻 Real-World Examples
- 💻 Related Commands
- ☠ Scenarios
- ☁ GCP GKE Perspective
- ▩ AWS EKS Perspective
- 🧪 Hands-On Lab Exercises
- 🎯 Interview Deliverable Answer (60–90 s)
- ? Possible Interview Questions

📋 Topic 2 — Containers vs VMs vs Kubernetes
- 🔹 Definition
- 🔹 About
- 🔹 How It Integrates With Other Kubernetes Components
- 🏗 Architecture / Comparison Diagrams
- 🔹 Comparison Tables
- 💻 Real-World Examples
- 💻 Related Commands
- ☠ Scenarios
- ☁ GCP GKE Perspective
- ▩ AWS EKS Perspective
- 🧪 Hands-On Lab Exercises
- 🎯 Interview Deliverable Answer (60–90 s)
- ? Possible Interview Questions

📋 Topic 3 — Kubernetes Architecture
- 🔹 Definition
- 🔹 About
- 🔹 How It Integrates With Other Kubernetes Components
- 🏗 Architecture Diagrams
- 💻 Real-World Examples
- 💻 Related Commands
- ☠ Scenarios
- ☁ GCP GKE Perspective
- ▩ AWS EKS Perspective
- 🧪 Hands-On Lab Exercises
- 🎯 Interview Deliverable Answer (60–90 s)
- ? Possible Interview Questions

📋 Topic 4 — Cluster Setup Options
- 🔹 Definition
- 🔹 About
- 🔹 How It Integrates With Other Kubernetes Components
- 🏗 Setup Decision Flow
- 💻 Real-World Examples
- 💻 Related Commands
- ☠ Scenarios
- ☁ GCP GKE Perspective
- ▩ AWS EKS Perspective
- 🧪 Hands-On Lab Exercises
- 🎯 Interview Deliverable Answer (60–90 s)
- ? Possible Interview Questions

- **Topic 5 — kubectl Basics and kubeconfig**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 6 — Pods**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 7 — ReplicaSets**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 8 — Deployments (Rolling Updates, Rollbacks, Strategies)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 9 — StatefulSets**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 10 — DaemonSets**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions

- **Topic 11 — Jobs and CronJobs**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 12 — ReplicationControllers (Legacy)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 13 — Labels, Selectors, Annotations**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 14 — ConfigMaps**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 15 — Secrets (Including External Secret Managers)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 16 — Resource Requests and Limits**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions

- **Topic 17 — Namespaces**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 18 — Environment Variables and Downward API**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 19 — Kubernetes Networking Model**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 20 — Services (ClusterIP, NodePort, LoadBalancer, ExternalName, Headless)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 21 — Ingress and Ingress Controllers**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 22 — Network Policies**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions

- **Topic 23 — CNI Plugins (Calico, Cilium, VPC CNI, GKE Dataplane V2)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 24 — DNS in Kubernetes (CoreDNS)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 25 — Service Mesh Overview (Istio, Linkerd)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 26 — Volumes (emptyDir, hostPath, etc.)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 27 — Persistent Volumes (PV)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions
- **Topic 28 — Persistent Volume Claims (PVC)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - ? Possible Interview Questions

Topic 29 — Storage Classes and Dynamic Provisioning
- Definition
- About
- How It Integrates With Other Kubernetes Components
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

Topic 30 — CSI Drivers (GCE PD, EBS, EFS, Filestore)
- Definition
- About
- How It Integrates With Other Kubernetes Components
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

Topic 31 — StatefulSet Storage Patterns
- Definition
- About
- How It Integrates With Other Kubernetes Components
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

Topic 32 — Scheduler Deep Dive
- Definition
- About
- How It Integrates With Other Kubernetes Components
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

Topic 33 — Node Selectors, Node Affinity, Anti-Affinity
- Definition
- About
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

Topic 34 — Taints and Tolerations
- Definition
- About
- Diagrams
- Real-World Examples
- Related Commands
- Scenarios
- GCP GKE Perspective
- AWS EKS Perspective
- Hands-On Lab Exercises
- Interview Deliverable Answer (60–90 s)
- Possible Interview Questions

- **Topic 35 — Topology Spread Constraints**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 36 — Pod Priority and Preemption**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 37 — Custom Schedulers**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 38 — Authentication and Authorization**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 39 — RBAC (Roles, ClusterRoles, RoleBindings, ClusterRoleBindings)**
  - Definition
  - About
  - How It Integrates With Other Kubernetes Components
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions
- **Topic 40 — Service Accounts**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Possible Interview Questions

- **Topic 41 — Pod Security Standards / Pod Security Admission**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 42 — Security Contexts**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 43 — Secrets Management Deep Dive (Sealed Secrets, External Secrets, Vault, Workload Identity, IRSA)**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 44 — Network Policies (Deep Dive)**
  - Definition
  - About — Advanced Patterns
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 45 — Image Security and Admission Controllers (OPA/Gatekeeper, Kyverno)**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 46 — Logging (kubectl logs, EFK, Cloud Logging, CloudWatch)**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Hands-On Lab Exercises
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions

- **Topic 47 — Monitoring (Metrics Server, Prometheus, Grafana, GKE Managed Prometheus, CloudWatch Container Insights)**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GCP GKE Perspective
  - AWS EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 48 — Events and Debugging**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 49 — Liveness, Readiness, Startup Probes**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 50 — Distributed Tracing Basics**
  - Definition
  - About
  - How It Integrates
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 51 — Horizontal Pod Autoscaler (HPA)**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 52 — Vertical Pod Autoscaler (VPA)**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions

- **Topic 53 — Cluster Autoscaler**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 54 — Karpenter (EKS) and Node Auto-Provisioning (GKE)**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 55 — Pod Disruption Budgets (PDB)**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions
- **Topic 56 — Multi-Zone / Multi-Region HA**
  - Definition
  - About
  - Diagrams
  - Real-World Examples
  - Related Commands
  - Scenarios
  - GKE Perspective
  - EKS Perspective
  - Labs
  - Interview Deliverable Answer (60–90 s)
  - Interview Questions

Module 10 — Advanced Topics (Part 1)

- Topic 57 — Custom Resource Definitions (CRDs)
- Topic 58 — Operators and the Operator Pattern
- Topic 59 — Helm (Charts, Templating, Releases)
- Topic 60 — Kustomize
- Topic 61 — GitOps (ArgoCD, Flux)
- Pause Point — End of Batch 1 of 4

Module 10 — Advanced Topics (Part 2)

- Topic 62 — Admission Controllers (Mutating & Validating)
- Topic 63 — API Aggregation Layer
- Topic 64 — etcd Deep Dive (Backup, Restore, Disaster Recovery)
- Topic 65 — Multi-Cluster Management (GKE Fleet, EKS Anywhere)
- Topic 66 — Service Catalog & API Priority and Fairness (APF)
- Pause Point — End of Batch 2 of 5

Module 11 — Managed Kubernetes Deep Dive

- Topic 67 — GKE Standard vs GKE Autopilot — Full Comparison
- Topic 68 — EKS Managed Node Groups vs Self-Managed Nodes vs Fargate
- Topic 69 — GKE Workload Identity vs EKS IRSA (& EKS Pod Identity)
- Topic 70 — Upgrades and Version Lifecycle on GKE and EKS
- Pause Point — End of Batch 3 of 5

Module 12 — Interview-Only Content

- Topic 71 — Top 50 Real-World Kubernetes Troubleshooting Scenarios
- Topic 72 — Top 30 System Design Questions Using Kubernetes
- Topic 73 — Behavioral + Kubernetes Hybrid Questions
- Pause Point — End of Batch 4 of 5

Module 13 — Capstone Hands-On Lab Projects

- 🏆 Capstone 1 — Deploy a 3-Tier Web Application (Frontend + Backend + MySQL) on Minikube → GKE Autopilot → EKS Fargate
- 🏆 Capstone 2 — CI/CD Pipeline: GitHub Actions → Docker → GKE and EKS with Rolling Updates + Rollback
- 🏆 Capstone 3 — HPA + Cluster Autoscaler on GKE and Karpenter on EKS Under Load Testing (hey / k6)
- 🏆 Capstone 4 — Ingress with TLS: cert-manager + Let's Encrypt on GKE (GCE Ingress) and EKS (ALB Ingress)
- 🏆 Capstone 5 — Full Observability Stack: Prometheus + Grafana + Loki + Alertmanager on GKE and EKS
- 🏆 Capstone 6 — StatefulSet PostgreSQL with PVC on GKE Filestore + EKS EBS/EFS, Including Backup and Restore
- 🏆 Capstone 7 — Enforce Security: RBAC + NetworkPolicy + Pod Security Admission + WLI (GKE) + IRSA (EKS)
- 🏆 Capstone 8 — GitOps Deployment Using ArgoCD on GKE and EKS with Multi-Environment (dev/stage/prod) Sync
- 🏆 Capstone 9 — Build and Deploy a Custom Operator Using Kubebuilder on GKE and EKS
- 🏆 Capstone 10 — Disaster Recovery: etcd Backup/Restore on kubeadm + Backup for GKE + Velero Cross-Region on EKS
- 📋 Capstone Lab Index
- ⏸ Pause Point — End of Batch 5 of 5

MODULE

Pre-Module — Lab Environment Setup

🧰 Pre-Module — Lab Environment Setup

> Everything you need to install, configure, and secure your local + cloud Kubernetes lab environments.
> Complete this pre-module **before** starting Topic 1.

Official References - Kubernetes install tools: https://kubernetes.io/docs/tasks/tools/ - Minikube: https://minikube.sigs.k8s.io/docs/start/ - Kind: https://kind.sigs.k8s.io/docs/user/quick-start/ - Google Kubernetes Engine (GKE): https://cloud.google.com/kubernetes-engine/docs/quickstarts - Amazon Elastic Kubernetes Service (EKS): https://docs.aws.amazon.com/eks/latest/userguide/getting-started.html

---

📋 What You Will Set Up

Environment	Purpose	Free Tier?
Docker Desktop / Podman	Local container runtime	Yes
Minikube	Single-node local cluster	Yes
Kind (Kubernetes-in-Docker)	Multi-node local cluster	Yes
kubeadm	Bare-metal / VM install (production-like)	Yes
GKE Autopilot & Standard	Managed cluster on Google Cloud	~$300 free credit (90 days)
EKS Managed Node Groups & Fargate	Managed cluster on AWS	No free tier — costs apply

---

🛠 Section 1 — Local Tools to Install (All OS)

Install these **once** — every lab uses them.

Tool	Purpose	Install (macOS)	Install (Linux)	Install (Windows)
`kubectl`	K8s CLI	`brew install kubectl`	`curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl" && sudo install kubectl /usr/local/bin/`	`choco install kubernetes-cli`
`minikube`	Local single-node cluster	`brew install minikube`	`curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64 && sudo install minikube-linux-amd64 /usr/local/bin/minikube`	`choco install minikube`
`kind`	Multi-node in Docker	`brew install kind`	`go install sigs.k8s.io/kind@latest`	`choco install kind`
`docker`	Container runtime	Docker Desktop	`sudo apt install docker.io`	Docker Desktop
`helm`	Package manager	`brew install helm`	`curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash`	`choco install kubernetes-helm`
`kustomize`	Overlay/templating	`brew install kustomize`	`curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash`	`choco install kustomize`
`k9s`	Terminal UI for K8s	`brew install k9s`	`curl -sS https://webinstall.dev/k9s | bash`	`choco install k9s`
`stern`	Multi-pod log tailing	`brew install stern`	`go install github.com/stern/stern@latest`	`choco install stern`
`kubectx` / `kubens`	Fast context/namespace switcher	`brew install kubectx`	`sudo git clone https://github.com/ahmetb/kubectx /opt/kubectx`	`choco install kubectx`
`hey`	HTTP load generator	`brew install hey`	`go install github.com/rakyll/hey@latest`	`choco install hey`
`k6`	Modern load generator	`brew install k6`	`sudo apt install k6` (add repo)	`choco install k6`
`jq`	JSON processor	`brew install jq`	`sudo apt install jq`	`choco install jq`
`yq`	YAML processor	`brew install yq`	`snap install yq`	`choco install yq`
`gcloud`	GCP CLI	`brew install --cask google-cloud-sdk`	See https://cloud.google.com/sdk/docs/install	Installer from https://cloud.google.com/sdk
`aws` CLI v2	AWS CLI	`brew install awscli`	`curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip && unzip && sudo ./aws/install`	MSI from AWS
`eksctl`	EKS CLI	`brew install eksctl`	`curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" | tar xz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin`	`choco install eksctl`

Verify Installation

```
kubectl version --client
minikube version
kind version
docker version
helm version
kustomize version
gcloud version
aws --version
eksctl version
```

💻 Section 2 — Local Cluster Setup

2.1 Docker Desktop Kubernetes (Simplest)

1. Install Docker Desktop.
2. Open Docker Desktop → **Settings** → **Kubernetes** → check **Enable Kubernetes** → **Apply & Restart**.
3. Verify:

```
kubectl config use-context docker-desktop
kubectl get nodes
```

2.2 Minikube (Recommended for beginners)

```
# Start a single-node cluster with 4GB RAM, 2 CPUs
minikube start --cpus=2 --memory=4096 --driver=docker

# Verify
kubectl get nodes
minikube status

# Enable useful addons
minikube addons enable ingress
minikube addons enable metrics-server
minikube addons enable dashboard

# Open dashboard
minikube dashboard
```

2.3 Kind — Multi-node Local (Recommended for intermediate labs)

Save as `kind-3node.yaml`:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: k8s-lab
nodes:
- role: control-plane
- role: worker
- role: worker
```

```
kind create cluster --config kind-3node.yaml
kubectl cluster-info --context kind-k8s-lab
kubectl get nodes -o wide
```

2.4 kubeadm (Production-style — VMs)

Use two Ubuntu 22.04 VMs (2 vCPU, 4GB each). On both nodes:

```
sudo swapoff -a
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
sudo apt update && sudo apt install -y containerd apt-transport-https curl
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.l
sudo apt update && sudo apt install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
```

On control-plane node:

```
sudo kubeadm init --pod-network-cidr=192.168.0.0/16
mkdir -p $HOME/.kube && sudo cp /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
```

Join worker (paste command printed by `kubeadm init`):

```
sudo kubeadm join <cp-ip>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

☁ Section 3 — GKE Setup (Google Cloud)

3.1 One-time GCP Project + Billing

```
# Log in
gcloud auth login

# Create a project (id must be globally unique)
gcloud projects create k8s-lab-$USER --set-as-default

# Link billing (get BILLING_ACCOUNT_ID from console → Billing)
gcloud billing projects link k8s-lab-$USER --billing-account=<BILLING_ACCOUNT_ID>

# Enable required APIs
gcloud services enable container.googleapis.com \
  compute.googleapis.com \
  cloudresourcemanager.googleapis.com \
  artifactregistry.googleapis.com \
  monitoring.googleapis.com \
  logging.googleapis.com
```

3.2 GKE Autopilot (fully managed — recommended for labs)

```
gcloud container clusters create-auto autopilot-lab \
  --region=asia-south1 \
  --release-channel=regular

# Get credentials
gcloud container clusters get-credentials autopilot-lab --region=asia-south1

kubectl get nodes
```

3.3 GKE Standard (control over node pools)

```
gcloud container clusters create std-lab \
  --zone=asia-south1-a \
  --num-nodes=2 \
  --machine-type=e2-medium \
  --disk-size=50 \
  --release-channel=regular \
  --enable-ip-alias

gcloud container clusters get-credentials std-lab --zone=asia-south1-a
```

3.4 Free-Tier Cost Guardrails on GCP

- Use `e2-small` or `e2-medium` — not `n2-standard` .
- Pick a **single zone** (not multi-region) for labs.
- Set a **budget alert** at $10, $25, $50 (Billing → Budgets).
- **Always** run cleanup at the end of a lab session.

🏁 Section 4 — EKS Setup (AWS)

4.1 AWS CLI + IAM

```
aws configure   # provide access key + secret + region (e.g., ap-south-1)
aws sts get-caller-identity   # verify
```

Required IAM permissions: `AdministratorAccess` (for lab account only — never a production identity).

4.2 EKS with Managed Node Groups

```
eksctl create cluster \
  --name eks-lab \
  --region ap-south-1 \
  --version 1.31 \
  --nodegroup-name workers \
  --node-type t3.medium \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 3 \
  --managed \
  --with-oidc

# Verify
kubectl get nodes -o wide
```

4.3 EKS with Fargate

```
eksctl create cluster \
  --name eks-fargate-lab \
  --region ap-south-1 \
  --version 1.31 \
  --fargate
```

Fargate runs pods without you managing nodes. Charged per pod vCPU/memory-second.

4.4 Cost Guardrails on AWS

- **Do NOT** leave clusters running overnight — control-plane is $0.10/hr × 24 × 30 = **~$72/month** per cluster.
- Use **AWS Budgets** → alert at $10 and $50.
- Add tags `Purpose=lab` and `Owner=<you>` for tracking.

🚨 Section 5 — Cost-Safety Checklist (READ BEFORE EVERY CLOUD LAB)

Before starting - [ ] Budget alert set (GCP Billing / AWS Budgets) - [ ] Region selected (single zone/region) - [ ] Cluster size minimal (2 nodes, small type) - [ ] `Purpose=lab` tag/label applied - [ ] Timer set for 90 min

After finishing (MANDATORY) - [ ] Delete workloads: `kubectl delete -f <manifests>` - [ ] Delete cluster: - GKE: `gcloud container clusters delete <name> --region=<r> --quiet` - EKS: `eksctl delete cluster --name=<name> --region=<r>` - [ ] Delete leftover LoadBalancers (they cost money even without a cluster): - GCP: `gcloud compute forwarding-rules list` → delete each - AWS: `aws elbv2 describe-load-balancers` → delete each - [ ] Delete leftover disks / PVs: - GCP: `gcloud compute disks list` - AWS: `aws ec2 describe-volumes --filters Name=status,Values=available` - [ ] Confirm billing page shows no active resources

---

🚦 Section 6 — Pre-Module Verification Lab

Lab 0.1 — Sanity Check (Beginner, 15 min)

Objective: Confirm every tool works end-to-end on your local cluster.

Prerequisites: Minikube installed, Docker running.

Steps

```
# 1. Start a fresh cluster
minikube delete && minikube start --cpus=2 --memory=4096

# 2. Deploy nginx
kubectl create deployment sanity-nginx --image=nginx:1.27

# 3. Wait until Ready
kubectl rollout status deployment/sanity-nginx

# 4. Expose it
kubectl expose deployment sanity-nginx --port=80 --type=NodePort

# 5. Curl the service
minikube service sanity-nginx --url | xargs curl -s | head -n 5

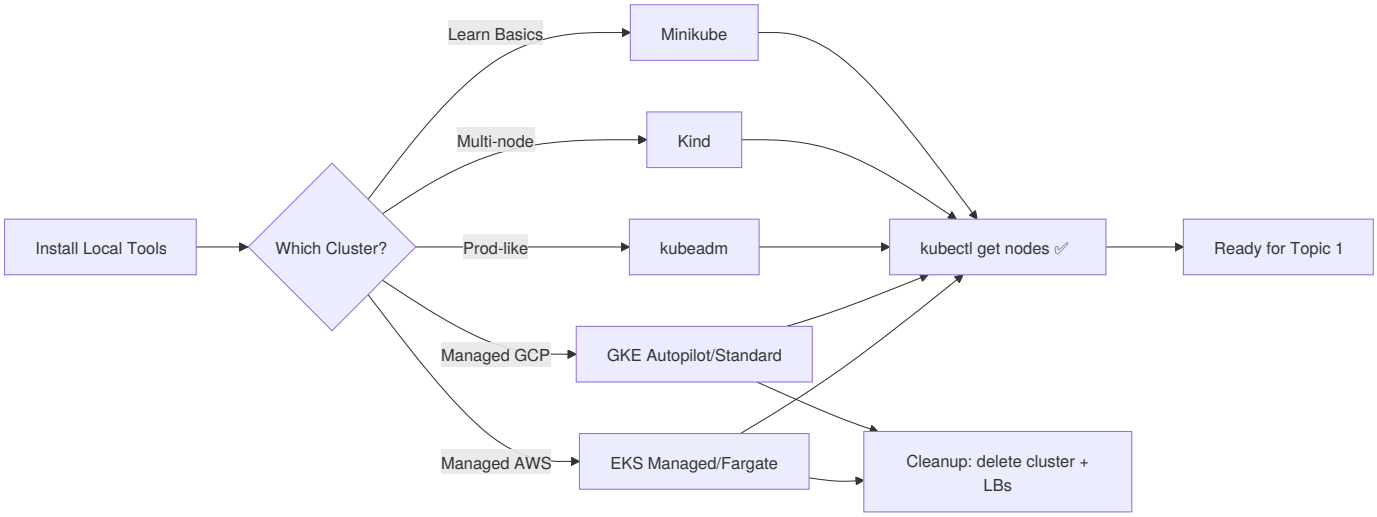
# 6. Cleanup
kubectl delete deployment,svc sanity-nginx
```

Expected output: HTML `<title>Welcome to nginx!</title>` in the curl.

Troubleshoot | Error | Fix | |---|---| | `docker: command not found` | Start Docker Desktop | | `error: You must be logged in to the server` | `minikube start` again | | Curl hangs | `minikube tunnel` in another shell |

Bonus: Repeat all 6 steps on Kind. Then on GKE Autopilot. Then delete both cloud clusters.

🏗 Section 7 — Workflow Diagram



🎯 Interview Deliverable — 60-second Answer

"In my lab I use a layered approach: Minikube for quick local experimentation, Kind when I need multi-node scenarios like DaemonSets or scheduler behavior, and I match that with GKE Autopilot on Google Cloud and EKS with managed node groups on AWS. I standardize the toolchain — kubectl, helm, kustomize, k9s, stern, kubectx — so muscle memory is identical everywhere. Before touching a cloud cluster I set a budget alert and label everything Purpose=lab, and my definition of done is always the cleanup step: cluster deleted, orphan load balancers checked, disks released. That discipline keeps the lab under a dollar a day."

MODULE

Module 1 — Foundations

🗐 Topic 1 — What is Kubernetes? History & Why Kubernetes

Official Documentation Reference - Overview: https://kubernetes.io/docs/concepts/overview/ - Historical context (Borg → Omega → K8s): https://kubernetes.io/blog/2015/04/borg-predecessor-to-kubernetes/ - GKE overview: https://cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview - EKS overview: https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

🔹 Definition

Kubernetes (K8s) is an open-source, extensible platform that automates deployment, scaling, healing, and operations of containerized applications across a cluster of machines.

🔹 About

What problem does it solve?

Running one container on a laptop is trivial. Running **thousands** of containers, on **hundreds** of servers, under real-world stress — hardware failures, traffic spikes, rolling releases, security patches, secret rotation — is not. Before Kubernetes, teams wrote fragile shell scripts or paid vendors for closed platforms to handle this.

Kubernetes provides a **declarative control loop**: you describe the desired state (e.g., "I want 5 replicas of my API running with 2 GB memory each behind a load balancer"), and Kubernetes continuously reconciles reality to match that state.

Origin

- **Google Borg (2003)**: internal cluster manager running Search, Gmail, YouTube.
- **Google Omega (2013)**: next-gen scheduler research.
- **Kubernetes (2014)**: Google open-sourced a Go-based re-implementation.
- **CNCF (2015)**: Google donated Kubernetes to the Cloud Native Computing Foundation.
- **v1.0**: July 2015.
- **v1.37**: current stable line as of 2026.
- The name comes from Greek κυβερνήτης — *helmsman*. "K8s" = K + 8 letters + s.

Why it dominates

Feature	What it means for you
Self-healing	Failed containers/pods restart automatically
Declarative API	Describe *what*, not *how*
Horizontal scaling	Add replicas by changing a number or based on CPU/RAM/custom metrics
Service discovery + load balancing	Pods find each other by DNS; traffic distributed automatically
Rolling updates + rollback	No-downtime deploys, one command to roll back
Storage orchestration	Attach local, EBS, GCE PD, EFS, Filestore, NFS transparently
Secret + config management	Decouple sensitive data from images
Extensibility	CRDs, operators, admission webhooks, custom schedulers
Portability	Same manifests run on laptop, GKE, EKS, AKS, on-prem

What Kubernetes is NOT

- Not a PaaS (does not build/deploy source code).
- Not middleware (no database, message bus, cache).
- Not a CI/CD tool (integrates with Jenkins, GitHub Actions, ArgoCD).
- Not a logging/monitoring stack (integrates with Prometheus, EFK, Cloud Logging).
- Not an orchestration engine in the workflow sense — it is a **set of independent control loops** converging on desired state.

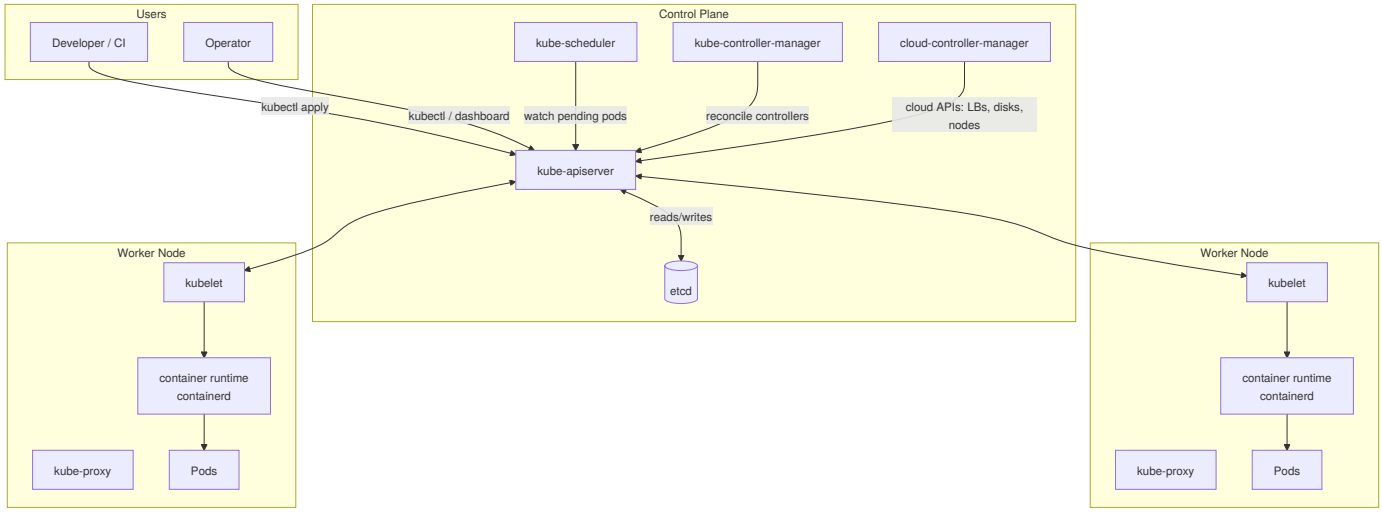
🔹 How It Integrates With Other Kubernetes Components

Every layer of the K8s stack depends on Kubernetes primitives being present:

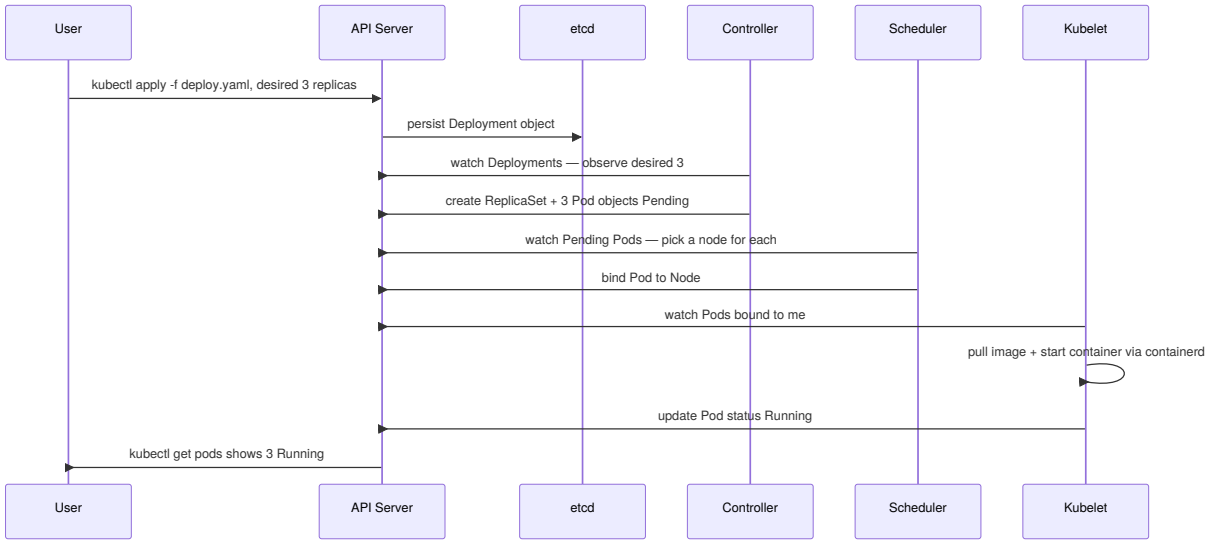
Consumer	How it uses Kubernetes
Developers	Push image → kubectl apply → Deployment
CI/CD (Jenkins, GH Actions, ArgoCD)	Update manifests → apply → rollout
Service Mesh (Istio, Linkerd)	Inject sidecars via mutating admission webhooks
Observability (Prometheus, Grafana Loki)	Scrape metrics/logs via K8s service discovery
Storage (CSI drivers)	Dynamically provision PVs from PVC requests
Networking (CNI plugins)	Give each pod an IP; enforce NetworkPolicy
Security (OPA/Gatekeeper, Kyverno)	Intercept API requests via admission webhooks

🏗 Architecture / Flow Diagram

High-level Component Diagram



Reconciliation Loop



💻 Real-World Examples

Example 1 — Deploy nginx and expose it

```
# nginx-example.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports: [{ containerPort: 80 }]
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits:   { cpu: 250m, memory: 256Mi }
---
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
  type: ClusterIP
```

```
kubectl apply -f nginx-example.yaml
kubectl get pods -l app=web
kubectl port-forward svc/web 8080:80
# open http://localhost:8080
```

Example 2 — Rolling update + rollback

```
# Update image → K8s replaces pods 1 at a time
kubectl set image deployment/web nginx=nginx:1.28
kubectl rollout status deployment/web

# Oops — rollback
kubectl rollout undo deployment/web
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl version`	`--client`	Show only client version (skip server)	`kubectl version --client`
`kubectl cluster-info`	—	Confirm API and DNS endpoints	`kubectl cluster-info`
`kubectl get nodes`	`-o wide`	See node IPs and OS/kernel	`kubectl get nodes -o wide`
`kubectl get pods`	`-A`	All namespaces	`kubectl get pods -A`
`kubectl apply`	`-f`	Apply a manifest file	`kubectl apply -f deploy.yaml`
`kubectl apply`	`-k`	Apply a kustomize overlay	`kubectl apply -k overlays/prod`
`kubectl describe`	`pod/<n>`	Full event + status detail	`kubectl describe pod web-1`
`kubectl logs`	`-f`	Follow logs live	`kubectl logs -f pod/web-1`
`kubectl explain`	`--recursive`	Explain object fields	`kubectl explain pod.spec --recursive`
`kubectl api-resources`	`--namespaced=true`	List namespaced kinds	`kubectl api-resources --namespaced=true`

🚨 Scenarios

Scenario 1 — "Our monolith goes down under Black-Friday load"

- **Symptom**: single VM saturates at 500 req/s, cascading errors.
- **K8s fix**: containerize → Deployment with 10 replicas → HorizontalPodAutoscaler on CPU 70% → LoadBalancer service.
- **Result**: scale-out in ~60 s, zero-downtime during spikes.

Scenario 2 — "Every deploy takes 2 hours of manual SSH"

- **Symptom**: engineers SSH into servers, `git pull`, restart supervisord.
- **K8s fix**: `kubectl set image` or ArgoCD auto-sync. Rolling update controlled by strategy `maxSurge/maxUnavailable`.
- **Result**: deploy = 90 s, rollback = 5 s.

Scenario 3 — "A node died at 3 AM; my pager fired"

- **Symptom**: node crashes → traffic errors → oncall reboots node.
- **K8s fix**: Deployment controller re-creates pods on healthy nodes automatically; PodDisruptionBudget guarantees availability; multi-zone node pool means the incident is silent.

☁ GCP GKE Perspective

- GKE runs the **control plane for free (per project)** on Autopilot; charges per pod resources.
- **Autopilot** = Google runs the nodes; you cannot SSH into them; pay per pod-second.
- **Standard** = you choose machine types and manage node pools.
- Deep GCP integrations: Workload Identity → GCP IAM, Cloud Logging, Cloud Monitoring, Backup for GKE, Cloud DNS.
- Release channels (Rapid / Regular / Stable) dictate upgrade cadence.

```
gcloud container clusters create-auto k8s-lab --region=asia-south1
gcloud container clusters get-credentials k8s-lab --region=asia-south1
gcloud container operations list
```

▩ AWS EKS Perspective

- EKS charges **$0.10/hour** per cluster control plane, plus EC2/Fargate for workers.
- **Managed Node Groups** = EKS provisions EC2 in your VPC; you patch AMIs.
- **Self-Managed** = you fully own the ASG.
- **Fargate** = serverless pods; pay per pod-second, no nodes to manage; limitations on privileged, DaemonSets, hostPath.
- Native integrations: IRSA (IAM roles for service accounts), VPC CNI (pod = ENI), CloudWatch Container Insights, AWS Load Balancer Controller.

```
eksctl create cluster --name k8s-lab --region ap-south-1 --managed
aws eks update-kubeconfig --name k8s-lab --region ap-south-1
aws eks describe-cluster --name k8s-lab
```

🧪 Hands-On Lab Exercises

Lab 1.1 — Your first Deployment (Beginner, 20 min)

Objective: Deploy nginx and see the reconciliation loop work.

Prerequisites: Minikube running, `kubectl` on PATH.

Setup

```
minikube start --cpus=2 --memory=4096
kubectl get nodes
```

Steps 1. Create the manifest `hello.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: hello }
spec:
  replicas: 3
  selector: { matchLabels: { app: hello } }
  template:
    metadata: { labels: { app: hello } }
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports: [{ containerPort: 80 }]
---
apiVersion: v1
kind: Service
metadata: { name: hello }
spec:
  selector: { app: hello }
  ports: [{ port: 80, targetPort: 80 }]
  type: NodePort
```

2. Apply: `kubectl apply -f hello.yaml` 3. Watch reconciliation: `kubectl get pods -w` (Ctrl-C after they're Running) 4. **Kill a pod** and observe self-healing:

```
kubectl delete pod $(kubectl get pods -l app=hello -o jsonpath='{.items[0].metadata.name}')
kubectl get pods -l app=hello       # new pod appears
```

5. Access: `minikube service hello --url`

Verify: `kubectl get deploy hello` shows `3/3 READY`.

Expected Output

```
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
hello   3/3     3            3           45s
```

Cleanup: `kubectl delete -f hello.yaml`

Troubleshoot | Error | Fix | |---|---| | `ImagePullBackOff` | `docker pull nginx:1.27` first / check DNS | | Service not reachable | `minikube tunnel` in another shell |

Bonus: Scale to 10, then to 1. Watch `kubectl get pods -w`.

Lab 1.2 — Kill a node, see K8s heal (Intermediate, 30 min)

Objective: Prove Kubernetes reschedules pods after a node failure.

Prerequisites: Kind cluster with 1 control-plane + 2 workers.

Setup

```
cat > kind-3.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
EOF
kind create cluster --config kind-3.yaml --name heal-lab
```

Steps 1. Deploy 6 pods with anti-affinity so they spread across nodes:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: spread }
spec:
  replicas: 6
  selector: { matchLabels: { app: spread } }
  template:
    metadata: { labels: { app: spread } }
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector: { matchLabels: { app: spread } }
      containers:
      - { name: web, image: nginx:1.27 }
```

2. `kubectl apply -f spread.yaml` 3. Confirm pods land on both workers: `kubectl get pods -o wide` 4. Simulate node death:

```
docker stop heal-lab-worker
kubectl get nodes -w   # node shows NotReady
```

5. After 5 min (default eviction), pods on that node get rescheduled to the surviving worker.

Verify: `kubectl get pods -o wide` shows all 6 pods running.

Cleanup: `kind delete cluster --name heal-lab`

Bonus: Restart the stopped node — `docker start heal-lab-worker` — then observe that already-running pods are NOT moved back (K8s does not preempt for balance without a rebalancer add-on).

Lab 1.3 — Compare local vs GKE Autopilot vs EKS (Advanced, 60 min)

Objective: Deploy the same app in three environments and note the differences.

Prerequisites: gcloud + eksctl + credentials + budgets set.

Steps 1. Deploy `hello.yaml` (from Lab 1.1) on Minikube. Note pod IP range, Service type behavior. 2. Create GKE Autopilot: `gcloud container clusters create-auto trio --region=asia-south1` 3. `kubectl apply -f hello.yaml` on Autopilot. Change Service to `type: LoadBalancer`. Note that GKE provisions a real Google LB. 4. Create EKS: `eksctl create cluster --name trio --region ap-south-1 --managed` 5. Apply same manifest on EKS. Note the AWS Classic ELB provisioned. 6. Compare `kubectl get nodes -o wide` on each.

Cleanup (mandatory)

```
kubectl delete svc hello
gcloud container clusters delete trio --region=asia-south1 --quiet
eksctl delete cluster --name trio --region ap-south-1
```

Bonus: Time each step, tabulate cluster-creation duration.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Kubernetes is an open-source, declarative platform originally built at Google — evolved from Borg and Omega — that automates deploying, scaling, healing, and operating containerized applications on a cluster.
Why: Before Kubernetes, teams glued together bash scripts and Nagios to keep dozens of servers alive. Kubernetes replaces that with a set of control loops that continuously drive the actual state of the cluster toward the state you declared in YAML. That gives us self-healing, zero-downtime rollouts, autoscaling, service discovery, and secret management — for free.
How: I describe workloads as Deployments, expose them through Services and Ingress, keep configuration in ConfigMaps and Secrets, and let controllers do the rest. In production I run Kubernetes on GKE Autopilot or EKS with managed node groups, tie identity to GCP Workload Identity or IRSA, and monitor with Prometheus and Cloud Logging.
Example: In my last project we migrated a monolithic API to Kubernetes; a bad release used to take 2 hours to roll back — with `kubectl rollout undo` it now takes 5 seconds, and our uptime went from 99.5% to 99.95%."

? Possible Interview Questions

Basic (0–1 yr)

Q1. What is Kubernetes?
Open-source container orchestration platform that automates deployment, scaling, and management of containerized apps.

Q2. Why was Kubernetes created and by whom?
Google open-sourced it in 2014, based on 15+ years of experience running Borg internally; donated to CNCF in 2015.

Q3. What does K8s stand for?
"K" + 8 letters between K and s + "s" — an abbreviation of Kubernetes.

Q4. Name three problems Kubernetes solves.
Self-healing failed containers; automated rollouts/rollbacks; horizontal autoscaling and service discovery/load balancing.

Intermediate (2 yr)

Q5. What is the difference between imperative and declarative Kubernetes?
Imperative = `kubectl run/create/edit` telling K8s exact actions. Declarative = `kubectl apply -f` giving desired state; controllers reconcile. Declarative is safer, versionable, GitOps-friendly.

Q6. Kubernetes is not a PaaS — explain.
K8s is a **container** platform: it does not build source code, provide databases, mandate a language/framework, or ship logging. It gives primitives you build a PaaS on (e.g., OpenShift, Cloud Foundry Korifi).

Q7. What is a controller in Kubernetes?

A control loop watching a resource kind, comparing current state to desired state stored in etcd, and taking corrective action (e.g., Deployment controller ensures ReplicaSet count matches spec.replicas).

Advanced (3–4 yr)

Q8. Why does K8s use etcd and not a SQL database?
etcd is a distributed, strongly consistent (Raft) key-value store optimized for watch semantics — controllers need to be notified of every change instantly. SQL doesn't provide efficient long-lived watches at scale, and eventual consistency would break reconciliation guarantees.

Q9. How does Kubernetes achieve extensibility without forking?
Through CRDs (custom resources), admission webhooks (mutating + validating), the API aggregation layer, custom schedulers, and device plugins/CSI/CNI interfaces.

Q10. Give one thing Kubernetes explicitly does NOT do, and why that matters.
It does not run CI/CD or build code. That means teams choose Jenkins, GitHub Actions, or ArgoCD independently — preserving flexibility and letting K8s remain a stable execution substrate.

Scenario-based

Q11. A team wants to move a stateful Java app that stores files on the local disk to Kubernetes. Would you recommend it?
Yes, but with PersistentVolumes backed by network storage (EBS/PD/EFS/Filestore) and a StatefulSet (not Deployment) so pods keep stable identities. Warn about the migration complexity for stateful data.

GKE-specific

Q12. Why choose GKE Autopilot over Standard?
Autopilot removes node management — Google patches and scales nodes — and bills per pod resources. Choose Autopilot when you don't need custom OS, DaemonSets on system nodes, or specific GPU types.

EKS-specific

Q13. Why does EKS cost more than a self-hosted kubeadm cluster?
$0.10/hour per control plane (24×365 = ~$876/year) for AWS-managed HA control plane, integrated IAM, VPC CNI, automatic patching, and SLA. It's usually cheaper than the engineering time to run kubeadm HA yourself.

Hands-on / Lab-based

Q14. In your lab, what happens when you delete a pod under a Deployment?
The Deployment → ReplicaSet controller notices `current=2, desired=3`, creates a new Pod object, scheduler binds it to a node, kubelet pulls the image and starts the container. Usually within 5–10 s.

Q15. Walk me through what `kubectl apply -f deploy.yaml` actually does end-to-end.
kubectl reads YAML → authenticates via kubeconfig → sends POST/PUT to kube-apiserver → apiserver runs authN/authZ + admission controllers → validates schema → persists to etcd → responds to kubectl → controllers watch the change → reconciliation begins.

🗐 Topic 2 — Containers vs VMs vs Kubernetes

Official Documentation Reference - Concepts overview (Container era): https://kubernetes.io/docs/concepts/overview/ - Containers: https://kubernetes.io/docs/concepts/containers/ - Images: https://kubernetes.io/docs/concepts/containers/images/ - GKE overview: https://cloud.google.com/kubernetes-engine/docs/concepts/kubernetes-engine-overview - EKS overview: https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

🔹 Definition

- **VM (Virtual Machine)** — a full guest OS running on virtualized hardware via a hypervisor.
- **Container** — an isolated, OS-level process bundled with its dependencies, sharing the host kernel.
- **Kubernetes** — an orchestrator that schedules, scales, heals, and networks many containers across many machines.

🔹 About

Physical → VMs → Containers → Orchestrated Containers

1. **Bare metal**: one OS per box; wasted CPU when app idle; app conflicts.
2. **VMs (Xen/KVM/VMware)**: many guest OSes per host; strong isolation but each VM is ~1 GB RAM + minutes to boot.
3. **Containers (Docker/containerd/CRI-O)**: share kernel; ~10 MB overhead; start in milliseconds; portable image format.
4. **Kubernetes**: containers don't schedule, heal, network, or scale themselves. K8s handles that at fleet scale.

Key mechanism: containers use Linux namespaces (PID, NET, MNT, UTS, IPC, USER, CGROUP) for isolation and cgroups for resource limits — no hypervisor.

When to use which

Situation	Best pick
Multi-tenant untrusted code	VM (stronger isolation) or Firecracker microVM
Windows-only enterprise app	VM (or Windows nodes on K8s)
Microservices, CI/CD, scaling	Containers + Kubernetes
One-off script	Docker container, no K8s
Regulatory heavy DB with static workload	VM often still wins

🔺 How It Integrates With Other Kubernetes Components

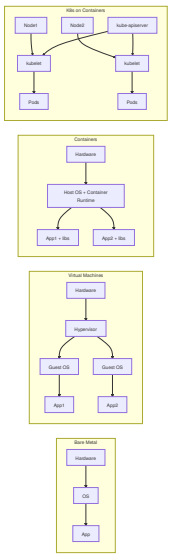
Every Kubernetes primitive assumes the container abstraction:

K8s Component	Relies on containers because...
kubelet	Uses CRI (containerd/CRI-O) to run containers
Pod	Group of 1+ containers sharing net/IPC/storage namespaces
Image pull	Container registry (Docker Hub, Artifact Registry, ECR)
CNI plugin	Attaches network namespace to pod
CSI driver	Mounts volumes into container filesystem
Security context	Sets Linux capabilities, UID, seccomp, AppArmor on the container

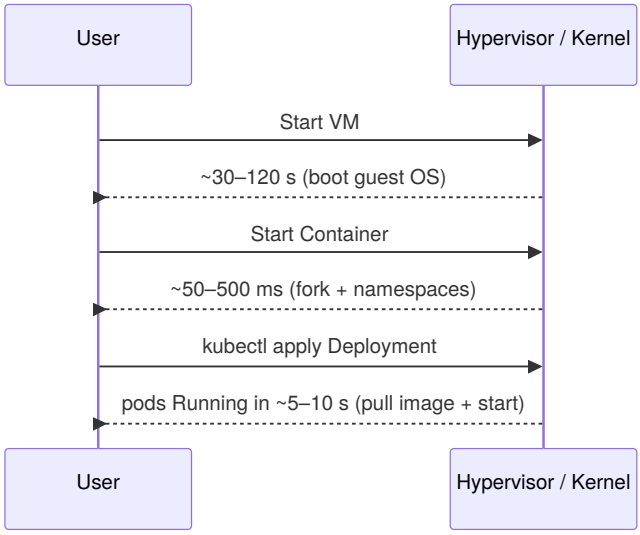
VMs still show up: **GKE nodes = Compute Engine VMs**; **EKS nodes = EC2 VMs**; **kata-containers** or **Firecracker** wrap a container in a lightweight VM for stronger isolation.

🏗 Architecture / Comparison Diagrams

Layer Comparison



Startup Time Sequence



🔷 Comparison Tables

VM vs Container

Attribute	VM	Container
Isolation	Hypervisor level (strong)	Kernel level (weaker)
Boot time	30 s – 2 min	50 ms – few seconds
Overhead	100s of MB per VM (guest OS)	~10 MB per container
Density	10–20/host	100s/host
Portability	Image + hypervisor	OCI image → any Linux
Kernel	Own	Shared with host
Security boundary	Very strong	Depends on kernel + seccomp/AppArmor

Docker vs Kubernetes (common interview trap)

	Docker	Kubernetes
Layer	Runs one container	Runs many containers across nodes
Scope	Single machine	Cluster
Scaling	Manual (`docker run` again)	Declarative Deployments, HPA
Healing	None	Restarts pods, reschedules on failed nodes
Networking	Bridge/host per machine	Cluster-wide flat network, Services, DNS
Storage	Volumes on host	PV/PVC across nodes

💻 Real-World Examples

Example 1 — Same nginx: VM way vs container way

VM way (Bash):

```
# On an EC2 instance
sudo apt update && sudo apt install -y nginx
sudo systemctl start nginx
# to run 3 copies → 3 EC2 instances, ELB in front, manual patching
```

Container way:

```
docker run -d --name web -p 80:80 nginx:1.27
```

Kubernetes way (`nginx-k8s.yaml`):

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - { name: nginx, image: nginx:1.27, ports: [{ containerPort: 80 }] }
```

`kubectl apply -f nginx-k8s.yaml` → 3 self-healing replicas across nodes with 1 command.

Example 2 — Minimal multi-stage build → tiny container

```
FROM golang:1.22 AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/api

FROM gcr.io/distroless/static:nonroot
COPY --from=build /out/app /app
USER nonroot
ENTRYPOINT ["/app"]
```

Result: ~10 MB image, no shell, no package manager. Compare to a VM AMI ~1–4 GB.

💻 Related Commands

Command	Flag	Purpose	Example
`docker ps`	`-a`	Include stopped containers	`docker ps -a`
`docker run`	`-d`	Detach	`docker run -d nginx`
`docker run`	`--rm`	Remove after exit	`docker run --rm alpine echo hi`
`docker exec`	`-it`	Interactive shell	`docker exec -it web sh`
`docker build`	`-t`	Tag image	`docker build -t me/app:1 .`
`docker push`	—	Push to registry	`docker push me/app:1`
`kubectl run`	`--image`	Create ad-hoc pod	`kubectl run tmp --image=busybox --rm -it -- sh`
`kubectl exec`	`-it`	Shell in container	`kubectl exec -it web-0 -- sh`
`crictl`	`ps`	Container list on a node (CRI)	`sudo crictl ps`
`nerdctl`	`ps`	Docker-like CLI for containerd	`nerdctl ps`

🚨 Scenarios

Scenario 1 — "Container works locally, breaks in K8s"

- **Root cause**: local Docker had `--privileged`, K8s pod runs as non-root with dropped capabilities.
- **Fix**: add securityContext explicitly:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  capabilities: { drop: ["ALL"] }
```

Scenario 2 — "Legacy Windows service must live"

- **Root cause**: can't containerize legacy .NET Framework app.
- **Fix**: run it in a VM in the same VPC; expose it via a K8s `ExternalName` or headless Service so pods can reach it by DNS.

Scenario 3 — "Pod restarted mid-request"

- **Root cause**: memory limit exceeded → OOMKilled → container restart.
- **Fix**: raise `resources.limits.memory` or fix the leak. Diagnose:

```
kubectl describe pod web-0 | grep -A2 "Last State"   # OOMKilled
kubectl top pod web-0
```

☁ GCP GKE Perspective

- GKE **nodes are Compute Engine VMs** (Container-Optimized OS by default).
- **Autopilot** hides the VM; you only see pods. Google uses gVisor/sandboxed nodes for isolation on request (`--enable-gvisor` node pool in Standard).
- Artifact Registry replaces the deprecated Container Registry (`gcr.io`).

```
# Push an image
gcloud auth configure-docker asia-south1-docker.pkg.dev
docker tag me/app:1 asia-south1-docker.pkg.dev/$PROJECT/repo/app:1
docker push  asia-south1-docker.pkg.dev/$PROJECT/repo/app:1
```

▩ AWS EKS Perspective

- EKS **nodes are EC2 instances** (Amazon Linux 2023 / Bottlerocket AMIs) OR **Fargate microVMs** (Firecracker).
- **Fargate** gives you a serverless-container experience — each pod runs in its own microVM for strong isolation, but no DaemonSets, no privileged, no hostPath.
- ECR is the container registry.

```
aws ecr get-login-password --region ap-south-1 | \
  docker login --username AWS --password-stdin <acct>.dkr.ecr.ap-south-1.amazonaws.com
docker tag me/app:1 <acct>.dkr.ecr.ap-south-1.amazonaws.com/app:1
docker push       <acct>.dkr.ecr.ap-south-1.amazonaws.com/app:1
```

Hands-On Lab Exercises

Lab 2.1 — Build → Run → Kubernetes (Beginner, 30 min)

Objective: Take one Go/Python service through Docker then Kubernetes.

Prerequisites: Minikube, Docker.

Steps 1. `app.py`:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200); self.end_headers()
        self.wfile.write(b"hello from container\n")
HTTPServer(("", 8080), H).serve_forever()
```

2. `Dockerfile`:

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
CMD ["python", "app.py"]
EXPOSE 8080
```

3. Build: `docker build -t hello:1 .` 4. Run in Docker: `docker run --rm -p 8080:8080 hello:1` → `curl localhost:8080` 5. Load into Minikube: `minikube image load hello:1` 6. Deploy:

```
kubectl create deployment hello --image=hello:1
kubectl expose deployment hello --port=8080 --type=NodePort
minikube service hello --url | xargs curl
```

Verify: response = `hello from container`.

Cleanup: `kubectl delete deploy,svc hello`

Bonus: Slim image to `python:3.12-alpine`; measure size delta.

Lab 2.2 — VM vs Container startup benchmark (Intermediate, 45 min)

Objective: Empirically compare boot times.

Prerequisites: Multipass / VirtualBox and Docker.

Steps 1. `time multipass launch --name vm-test 22.04` → note wall time (~30–60 s). 2. `time docker run --rm hello-world` → note wall time (~1 s). 3. `time kubectl run tmp --image=busybox --restart=Never --rm -it -- sh -c "echo hi"` → note wall time (~5 s including scheduling + pull).

Record in a table: | Env | Boot to ready | |---|---| | VM (Ubuntu 22.04) | ~30–60 s | | Docker container | ~1 s | | K8s pod (cached image) | ~2–5 s |

Cleanup: `multipass delete vm-test && multipass purge`.

Lab 2.3 — Firecracker microVMs on EKS Fargate (Advanced, 60 min)

Objective: Deploy a pod on Fargate (Firecracker microVM) and observe isolation.

Prerequisites: EKS Fargate cluster (`eksctl create cluster --fargate`).

Steps 1. `kubectl get nodes -o wide` — see `fargate-ip-…` nodes. 2. Deploy a pod; it runs in its own microVM. 3. Try `kubectl exec` — works, but `hostPath` / `privileged` will fail (Fargate limitations).

```
apiVersion: v1
kind: Pod
metadata: { name: fg-demo, labels: { app: fg } }
spec:
  containers:
  - { name: c, image: nginx:1.27 }
```

4. Verify: `kubectl describe pod fg-demo` shows `Node: fargate-...`.

Cleanup: `eksctl delete cluster --name=<name> --region=ap-south-1`

Bonus: Try to add `hostPath` volume; observe rejection by Fargate admission.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: VMs virtualize hardware — each VM ships its own OS. Containers virtualize the OS — they share the host kernel via Linux namespaces and cgroups, so they're an order of magnitude lighter and start in milliseconds. Kubernetes is the layer above: it orchestrates thousands of those containers across many machines.
Why: We pick containers when we need portability, density, and fast iteration. We pick VMs when we need hard isolation, a different kernel, or Windows-only workloads. Kubernetes is what turns containers from a laptop tool into a production platform — scaling, healing, service discovery, rolling updates.
How: In practice I containerize services using multi-stage Docker builds targeting distroless images, push to ECR or Artifact Registry, and deploy them as Deployments on EKS or GKE. Where regulation demands stronger isolation I'll pair it with Fargate microVMs on AWS or gVisor sandboxes on GKE.
Example: We had a monolithic Java app on VMs — 40 minutes to deploy, 12 servers, 30% CPU utilization. Containerized on EKS with HPA, that dropped to 5 minutes to deploy, 6 nodes, 70% average utilization — with better reliability."

? Possible Interview Questions

Basic

Q1. Container vs VM in one line?
VM = own OS on virtualized hardware; container = isolated process on shared kernel.

Q2. Why are containers faster to start?
No guest OS to boot — just a process with namespaces/cgroups; typically ~ms.

Q3. What is an OCI image?
An Open Container Initiative-standard image format (layers + manifest + config). Docker images are OCI-compliant.

Intermediate

Q4. Which Linux features enable containers?
Namespaces (PID/NET/MNT/UTS/IPC/USER/CGROUP) for isolation; cgroups for CPU/memory limits; capabilities/seccomp/AppArmor/SELinux for security.

Q5. Docker vs Kubernetes?
Docker runs one container per machine; Kubernetes orchestrates many across a cluster with scheduling, service discovery, healing, and scaling.

Q6. Why is a container "not a small VM"?
Because it shares the host kernel — a kernel exploit escapes the container. That's why runtimes like gVisor/Kata/Firecracker exist for stronger isolation.

Advanced

Q7. When would you still pick a VM over a container in 2026?
Untrusted multi-tenant code without sandboxing; Windows-only legacy without Windows containers; kernel-module needs (e.g., DKMS drivers); strict regulatory boundary.

Q8. Explain the CRI. Why did K8s deprecate dockershim?
CRI (Container Runtime Interface) is the gRPC contract kubelet uses to talk to runtimes. Dockershim was an adapter to Docker; removed in 1.24 so kubelet talks directly to containerd/CRI-O — simpler, less code.

Q9. What's the difference between containerd and Docker?
containerd is the low-level runtime (image pull, container lifecycle); Docker is a toolchain (CLI, build, compose) that used to embed containerd. Kubernetes uses containerd directly.

Scenario-based

Q10. A container works on Docker Compose but fails on Kubernetes — where do you look?
readOnlyRootFilesystem/securityContext differences; missing volume mounts; DNS (K8s uses CoreDNS with different search); resource limits triggering OOM; env vars in Compose not translated to ConfigMap/Secret.

GKE-specific

Q11. How does GKE Autopilot handle the container isolation boundary?
Autopilot runs pods on multi-tenant nodes but enforces strict Pod Security Standards, no privileged, and per-pod Workload Identity. For stronger boundary use `--enable-sandbox` (gVisor).

EKS-specific

Q12. What isolation model does EKS Fargate use, and why does it matter?
Each pod runs in its own Firecracker microVM. This gives near-VM isolation with container ergonomics — ideal for multi-tenant SaaS, at the cost of no DaemonSets/privileged/hostPath.

Hands-on / Lab-based

Q13. In your lab, how did you measure that a container starts faster than a VM?
Timed `docker run` vs `multipass launch` on the same host. Containers: ~1 s; VM: ~45 s. K8s pods came in at ~3–5 s including scheduling and image cache lookup.

Q14. You built a multi-stage Dockerfile — walk me through it.
First stage compiles with the SDK image; second stage copies the binary into a distroless/nonroot image. Smaller final image, no compiler, no shell — reduces attack surface and pull time.

Q15. Show me how you would convert a docker-compose file to Kubernetes.
Each service → a Deployment (or StatefulSet for stateful); its ports → a Service; env → ConfigMap/Secret; volumes → PV/PVC. Or use `kompose convert` and refine the output.

📘 Topic 3 — Kubernetes Architecture

Official Documentation Reference - Cluster architecture: https://kubernetes.io/docs/concepts/architecture/ - Components: https://kubernetes.io/docs/concepts/overview/components/ - Nodes: https://kubernetes.io/docs/concepts/architecture/nodes/ - Control-plane ↔ node communication: https://kubernetes.io/docs/concepts/architecture/control-plane-node-communication/ - Cloud controller manager: https://kubernetes.io/docs/concepts/architecture/cloud-controller/ - GKE cluster architecture: https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-architecture - EKS cluster architecture: https://docs.aws.amazon.com/eks/latest/userguide/clusters.html

🔷 Definition

A Kubernetes cluster consists of a **control plane** (brain) that makes global decisions and a set of **worker nodes** (muscles) that run the containerized workloads. All state lives in **etcd**, and every component talks through the **API server**.

🔷 About

Control Plane Components

Component	Role	Talks to
kube-apiserver	REST front door for the cluster (only door to etcd)	Everyone
etcd	Consistent key-value store; source of truth	Only kube-apiserver
kube-scheduler	Assigns Pending pods to nodes	apiserver
kube-controller-manager	Runs built-in controllers (Deployment, ReplicaSet, Node, EndpointSlice, ServiceAccount…)	apiserver
cloud-controller-manager	Cloud-specific glue (LoadBalancers, disks, nodes)	apiserver + cloud API

Worker Node Components

Component	Role
kubelet	Node agent: watches API for pods bound to it; talks to container runtime; reports status
kube-proxy	Implements Services (iptables/IPVS/nftables) — routes cluster IPs to pods
container runtime	containerd / CRI-O / (Docker via cri-dockerd, legacy) — actually runs containers via OCI
CNI plugin	Assigns pod IPs + wires the network (Calico, Cilium, VPC CNI, GKE DP-v2)
CSI driver	Attaches persistent storage into pods

Key contracts

- **CRI** — Container Runtime Interface (kubelet ↔ runtime).
- **CNI** — Container Network Interface (kubelet ↔ network plugin).
- **CSI** — Container Storage Interface (kubelet ↔ storage).
- **OCI** — image + runtime standards.

High Availability

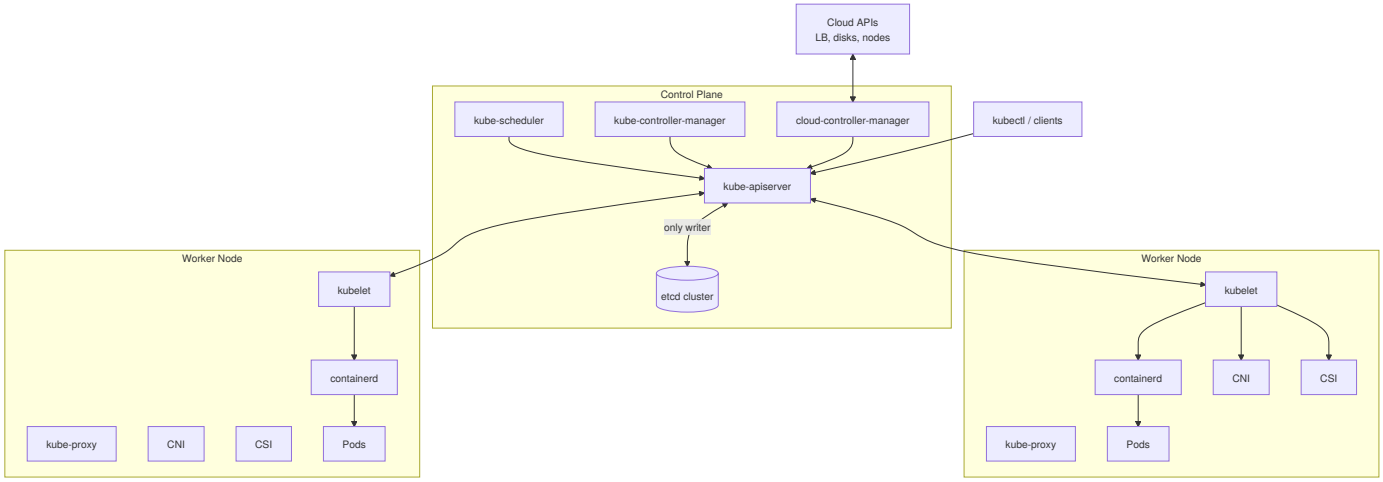
Production runs 3 or 5 control-plane replicas (etcd needs an odd number for Raft quorum), each with API server + scheduler + controller-manager. Only one scheduler / one controller-manager is "leader" at a time via leader election.

🔷 How It Integrates With Other Kubernetes Components

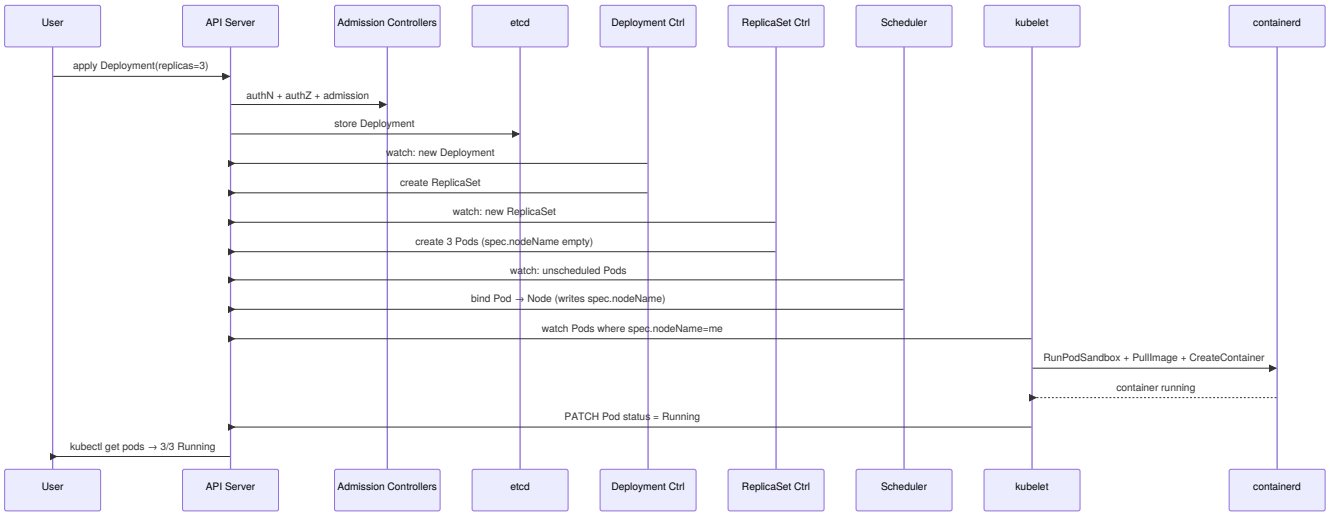
Workflow step	Component sequence
`kubectl apply -f deploy.yaml`	kubectl → API server → admission → etcd
ReplicaSet creates pods	Deployment ctrl → ReplicaSet ctrl → API → etcd
Pod scheduling	Scheduler watches Pending pods → binds → apiserver
Pod start	kubelet watches → CRI pulls image → runs container → reports back
Service traffic	kube-proxy programs iptables/IPVS on every node
Autoscaling	HPA controller queries metrics-server → patches replicas
Cloud LB	cloud-controller-manager creates GCLB/ELB → updates Service status

🏗 Architecture Diagrams

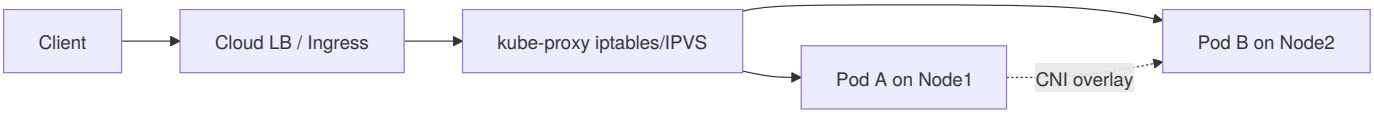
Component View



Pod Creation Sequence



Networking Overview



💻 Real-World Examples

Example 1 — Check every component's health

```
kubectl get componentstatuses   # legacy (may be empty in v1.19+)
kubectl get --raw='/livez?verbose'
kubectl get --raw='/readyz?verbose'
kubectl get nodes
kubectl get pods -n kube-system
```

Example 2 — Inspect etcd (on kubeadm cluster)

```
# On control-plane node
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list
```

Example 3 — Static pod manifest for control plane (kubeadm)

```
# /etc/kubernetes/manifests/kube-apiserver.yaml   (managed by kubeadm)
apiVersion: v1
kind: Pod
metadata: { name: kube-apiserver, namespace: kube-system }
spec:
  containers:
  - name: kube-apiserver
    image: registry.k8s.io/kube-apiserver:v1.31.0
    command:
    - kube-apiserver
    - --advertise-address=10.0.0.10
    - --etcd-servers=https://127.0.0.1:2379
    - --service-cluster-ip-range=10.96.0.0/12
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl cluster-info`	`dump`	Deep dump for support	`kubectl cluster-info dump > cluster.log`
`kubectl get nodes`	`-o wide`	See internal/external IPs, kernel	`kubectl get nodes -o wide`
`kubectl describe node`	—	Capacity, taints, events	`kubectl describe node worker-1`
`kubectl get pods`	`-n kube-system`	See control-plane pods	`kubectl get pods -n kube-system`
`kubectl logs`	`-n kube-system kube-apiserver-cp1`	Read apiserver logs	(kubeadm only)
`kubectl top`	`nodes`	Live CPU/mem (needs metrics-server)	`kubectl top nodes`
`kubectl get --raw`	`/metrics`	Prometheus metrics	`kubectl get --raw /metrics`
`crictl`	`pods`	See pods on this node	`sudo crictl pods`
`journalctl`	`-u kubelet -f`	kubelet logs	`sudo journalctl -u kubelet -f`
`etcdctl`	`snapshot save`	Back up etcd	`etcdctl snapshot save snap.db`

🚨 Scenarios

Scenario 1 — "kubectl commands hang"

- **Root cause**: API server down or etcd unreachable.
- **Diagnose**: `curl -k https://<cp-ip>:6443/livez` → check response. On control-plane node: `sudo systemctl status kubelet` and `sudo crictl ps | grep apiserver`.
- **Fix**: restart kubelet (which restarts static pods) or restore etcd from snapshot.

Scenario 2 — "Pods stuck Pending forever"

- **Root cause**: scheduler cannot find a fitting node — insufficient CPU/mem, or taints without tolerations, or nodeAffinity mismatch.
- **Diagnose**:

```
kubectl describe pod <name> | tail -20    # events
kubectl get nodes -o custom-columns=NAME:.metadata.name,ALLOCATABLE:.status.allocatable
```

- **Fix**: scale nodes, reduce requests, or add tolerations.

Scenario 3 — "Nodes show NotReady"

- **Root cause**: kubelet crashed, disk pressure, PID pressure, or CNI plugin failure.
- **Diagnose**:

```
kubectl describe node <n>       # conditions
ssh node && sudo journalctl -u kubelet -n 100
```

- **Fix**: address the specific condition (clear /var/log, restart kubelet, reinstall CNI).

Scenario 4 — "Split-brain risk with 2 controller-managers"

- **Root cause**: leader election broken (e.g., clock skew, lease expired).
- **Fix**: sync time (chrony/ntpd), verify `kubectl get leases -n kube-system`.

☁ GCP GKE Perspective

- **You do not see** kube-apiserver / etcd / scheduler / controller-manager — Google runs them in Google's project, behind your regional endpoint.
- **Regional clusters** = HA control plane across 3 zones.
- **GKE Dataplane V2** replaces kube-proxy with an eBPF-based Cilium equivalent.
- **Autopilot** hides nodes too — you only see Pods and PodSpecs.
- **cloud-controller-manager** is managed and integrates with Compute Engine LBs, PD CSI, Filestore CSI.
- Health / diagnostics:

```
gcloud container clusters describe autopilot-lab --region=asia-south1
gcloud container operations list
gcloud logging read "resource.type=k8s_control_plane_component" --limit=20
```

▩ AWS EKS Perspective

- Control plane is **fully AWS-managed** — you never SSH into apiserver/etcd. AWS runs multi-AZ HA and patches automatically.
- **VPC CNI** gives every pod a real ENI IP from your subnet — no overlay by default.
- **kube-proxy** and **CoreDNS** deploy as EKS add-ons you version.
- **Fargate** replaces both node and kubelet — a Firecracker microVM is spun up per pod by an AWS-managed data plane.
- Diagnostics:

```
aws eks describe-cluster --name eks-lab --region ap-south-1
aws eks list-addons --cluster-name eks-lab --region ap-south-1
aws logs tail /aws/eks/eks-lab/cluster --follow    # if control-plane logging enabled
```

- **Control-plane logging** must be enabled to get apiserver/audit/scheduler logs to CloudWatch — it's off by default.

🧪 Hands-On Lab Exercises

Lab 3.1 — Explore control-plane on kubeadm (Beginner, 30 min)

Objective: See every component running as a static Pod and hit their /livez endpoints.

Prerequisites: kubeadm cluster from pre-module (Section 2.4).

Steps 1. `kubectl get pods -n kube-system` — spot `kube-apiserver-*`, `etcd-*`, `kube-scheduler-*`, `kube-controller-manager-*`. 2. On the control-plane node: `ls /etc/kubernetes/manifests/`. 3. `curl -k https://localhost:6443/livez?verbose | head`. 4. `sudo crictl ps | grep etcd`.

Verify: at least 4 static pods, `/livez` returns 200.

Cleanup: nothing (part of the cluster).

Bonus: Move `kube-scheduler.yaml` out of `/etc/kubernetes/manifests/` for 30 s; observe that the pod disappears; move it back.

Lab 3.2 — Cause a Pending pod, diagnose the scheduler (Intermediate, 45 min)

Objective: Prove how the scheduler makes decisions using resource requests.

Prerequisites: Kind 3-node cluster.

Steps 1. Run:

```
apiVersion: v1
kind: Pod
metadata: { name: gluttony }
spec:
  containers:
  - { name: c, image: pause:3.9, resources: { requests: { memory: "100Gi" } } }
```

2. `kubectl apply -f gluttony.yaml` 3. `kubectl get pod gluttony` → Pending 4. `kubectl describe pod gluttony | tail -10` → event: `0/3 nodes are available: 3 Insufficient memory`. 5. Reduce request to `100Mi`, apply, watch it schedule.

Bonus: Add a toleration for a taint you invent, verify scheduling to a tainted node.

Lab 3.3 — Simulate API-server outage on Kind (Advanced, 60 min)

Objective: See what still works when apiserver is down.

Prerequisites: Kind cluster.

Steps 1. `docker exec -it kind-control-plane sh -c "mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/"` (kubelet detects removal, kills apiserver). 2. `kubectl get pods` → hangs / fails. 3. Confirm existing pods keep running (kubelet keeps them alive from local cache). 4. Restore: `docker exec -it kind-control-plane sh -c "mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/"`. 5. Wait ~30 s → `kubectl get pods` works again.

Cleanup: `kind delete cluster`.

Bonus: While apiserver is down, kill a pod on a worker (via `crictl stop`). Observe kubelet restarts it locally (no rescheduling until apiserver returns).

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Kubernetes cluster has two halves — a control plane and worker nodes. The control plane is the kube-apiserver, etcd, scheduler, controller-manager, and on a cloud the cloud-controller-manager. Workers run the kubelet, kube-proxy, a container runtime like containerd, plus CNI and CSI drivers.
Why: This split gives us a single source of truth in etcd, one API front door, and horizontally scalable data plane. Every component follows a controller pattern: watch desired state, act, report actual state.
How: A `kubectl apply` hits the apiserver, goes through authN/Z and admission, lands in etcd. Controllers see the change and create child objects; the scheduler picks a node; kubelet on that node pulls the image via CRI and starts the container; kube-proxy programs iptables so a Service IP reaches it.
Managed twist: On GKE and EKS the control plane is opaque and highly available across zones; on Autopilot even the nodes are hidden. So in production I focus on what I can see — pod scheduling, resource requests, Service and Ingress routing — and trust the cloud provider's SLA on the control plane."

? Possible Interview Questions

Basic

Q1. Name the control-plane components.
kube-apiserver, etcd, kube-scheduler, kube-controller-manager, (cloud-)controller-manager.

Q2. What runs on every worker node?
kubelet, kube-proxy, container runtime, CNI plugin.

Q3. Why is etcd important?
It stores the entire cluster state — objects, configuration, secrets. Losing etcd = losing the cluster's memory.

Intermediate

Q4. Why is only the apiserver allowed to talk to etcd?
To centralize authentication, authorization, admission, validation, and audit — and to keep etcd's client fan-out small.

Q5. What does the scheduler actually do?
Filters nodes by predicates (fit, affinity, taints, resources) then scores them (spread, image locality, NodeResourcesFit) and binds the pod to the highest-scoring node.

Q6. Difference between kube-controller-manager and cloud-controller-manager?
kube-controller-manager runs cloud-agnostic controllers (Deployment, ReplicaSet, Node, EndpointSlice…). cloud-controller-manager runs cloud-specific ones (RouteController, ServiceController for LBs, NodeController that queries the cloud provider).

Advanced

Q7. Explain leader election in the control plane.
Multiple replicas of scheduler/controller-manager run for HA, but only the leader acts. They contend for a Lease object; the winner renews it; if it dies the lease expires and another wins.

Q8. Walk me through what happens if etcd loses quorum.
Writes stop (reads may still work); apiserver becomes read-only; controllers cannot reconcile changes; workloads keep running because kubelets cache their pods. Fix: restore quorum (bring back members) or restore from snapshot.

Q9. What is a static pod and where would you see one?
A pod defined by a manifest file (usually `/etc/kubernetes/manifests/`) that kubelet runs directly without going through apiserver. Control-plane components on kubeadm are static pods.

Scenario-based

Q10. `kubectl` returns "unable to connect to server: EOF". What now?
Check apiserver: `curl -k https://cp:6443/livez`. If down, check `kubelet` on control-plane, check etcd health, check the manifests directory, check disk pressure. On managed K8s, check cloud provider status page.

GKE-specific

Q11. Why can't I SSH into the GKE control plane?
Google runs it inside their tenant project; you only get the API endpoint. Diagnostics come via Cloud Logging and `gcloud container operations`.

EKS-specific

Q12. How do you get audit logs from an EKS control plane?
Enable control-plane logging (api, audit, authenticator, controllerManager, scheduler) on the cluster; logs land in CloudWatch under `/aws/eks/<name>/cluster`.

Hands-on / Lab-based

Q13. In your lab you stopped kube-apiserver. What still worked?
Existing pods kept running because kubelet had cached PodSpecs. New `kubectl` commands failed. When apiserver returned, everything reconciled without data loss because etcd was untouched.

Q14. How would you back up an etcd cluster?
`ETCDCTL_API=3 etcdctl snapshot save snap.db` on one member, plus PKI directory. Store off-cluster (S3/GCS). To restore: stop apiservers, `etcdctl snapshot restore` per member, re-attach data dirs, restart.

Q15. Show me how you would prove kube-proxy is doing its job.
Deploy 2-replica nginx behind a ClusterIP Service. From another pod: `for i in {1..10}; do curl -s <svc>:80 | grep Server; done`. See both pod hostnames — proof kube-proxy load-balances. Confirm rules via `sudo iptables-save | grep <cluster-ip>` on a node.

🗐 Topic 4 — Cluster Setup Options

Official Documentation Reference - Getting started: https://kubernetes.io/docs/setup/ - Learning environment: https://kubernetes.io/docs/setup/learning-environment/ - Production environment: https://kubernetes.io/docs/setup/production-environment/ - kubeadm: https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/create-cluster-kubeadm/ - Minikube: https://minikube.sigs.k8s.io/docs/ - Kind: https://kind.sigs.k8s.io/ - GKE quickstart: https://cloud.google.com/kubernetes-engine/docs/quickstarts - EKS getting-started: https://docs.aws.amazon.com/eks/latest/userguide/getting-started.html

⏚ Definition

Kubernetes can be installed in many shapes: for learning (Minikube, Kind, Docker Desktop), for production on your own hardware (kubeadm, Kubespray, Rancher RKE2), or as a fully managed cloud service (GKE, EKS, AKS, DOKS, LKE).

⏚ About

What choice actually maps to what use case?

Setup	Best for	Nodes	HA control plane	Time to bring up
Docker Desktop K8s	quick localhost tests	1	No	seconds
Minikube	learn kubectl, addons, dashboard	1 (multi-node possible)	No	1 min
Kind	multi-node local, CI pipelines	many (in Docker)	Optional (`--control-plane-nodes 3`)	1–2 min
kubeadm	on-prem/production or CKAD/CKA prep	many	Yes (manual HA)	20–60 min
Kubespray	Ansible-based prod install	many	Yes	60 min+
RKE2 / k3s	edge + IoT + hardened prod	many	Yes	5–15 min
GKE Autopilot	GCP-managed, hands-off	Google manages	Yes (regional)	4–7 min
GKE Standard	GCP-managed, node control	You size pools	Yes (regional)	4–7 min
EKS + Managed NG	AWS-managed, EC2 workers	You choose types	Yes	15–20 min
EKS + Fargate	serverless pods	AWS manages microVMs	Yes	15–20 min
AKS / DOKS / LKE	Azure / DigitalOcean / Linode	Provider-managed	Yes	5–10 min

Rules of thumb

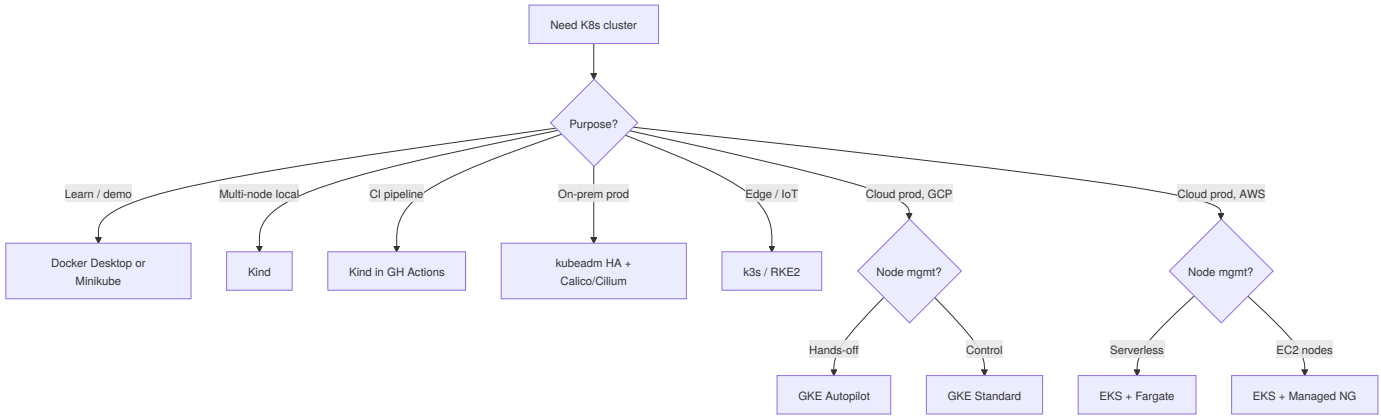
- **Learning path**: Docker Desktop or Minikube → Kind → kubeadm.
- **Career interviews**: know Minikube, Kind, kubeadm, GKE, EKS.
- **Production**: managed (GKE/EKS/AKS) unless a hard reason forces on-prem (regulatory, air-gapped, existing hardware).

🔗 How It Integrates With Other Kubernetes Components

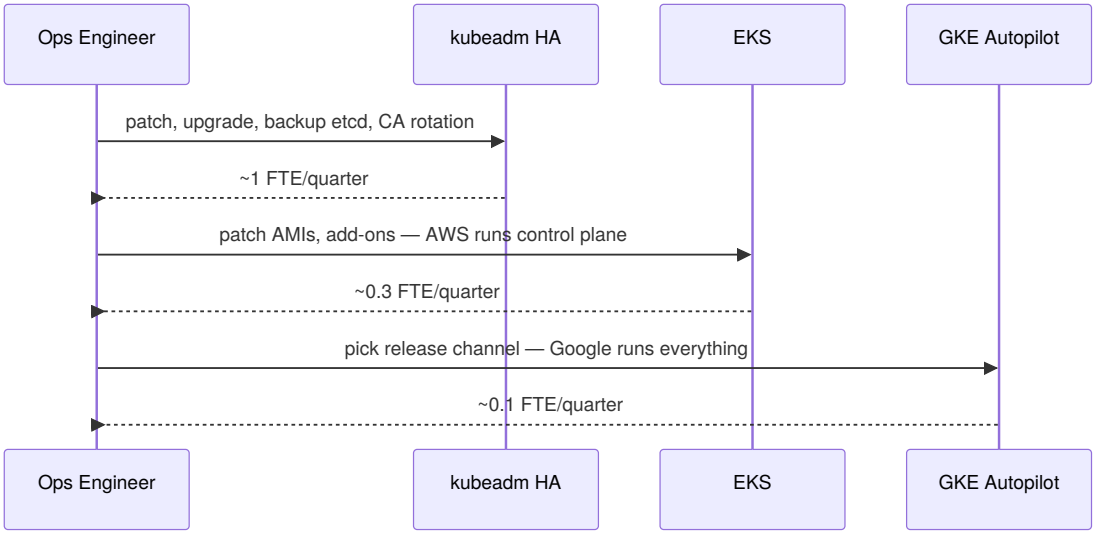
Every setup option pre-installs a specific stack:

Setup	CNI default	Ingress default	LoadBalancer	Storage
Docker Desktop	Docker bridge	none	localhost	local hostPath
Minikube	bridge/kindnet	addon (`ingress`)	`minikube tunnel`	hostPath
Kind	kindnet	none	needs MetalLB	local-path-provisioner
kubeadm	user-chosen (Calico/Cilium/Flannel)	user-chosen	needs MetalLB or cloud	user-chosen
GKE Standard	GKE Dataplane V2 (eBPF)	GCE Ingress / NGINX	Google Cloud LB	PD-CSI / Filestore-CSI
GKE Autopilot	GKE DP-V2 (locked)	GCE Ingress	Google Cloud LB	PD-CSI (managed)
EKS Managed NG	VPC CNI (default) or Calico/Cilium	AWS Load Balancer Controller	NLB/ALB	EBS-CSI / EFS-CSI
EKS Fargate	VPC CNI	AWS LB Ctrl (ALB required)	ALB	EFS only

🏗 Setup Decision Flow



Managed vs Self-Managed Cost/Effort



💻 Real-World Examples

Example 1 — Minikube multi-node

```
minikube start --nodes=3 --cpus=2 --memory=2048
kubectl get nodes
```

Example 2 — Kind HA (3 CP + 3 workers)

```
# kind-ha.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: control-plane
- role: control-plane
- role: worker
- role: worker
- role: worker
```

```
kind create cluster --config kind-ha.yaml --name ha
```

Example 3 — kubeadm HA (stacked etcd)

1. Load balancer (HAProxy/keepalived) fronting 3 control-plane IPs on TCP 6443.
2. `sudo kubeadm init --control-plane-endpoint "lb.example.com:6443" --upload-certs --pod-network-cidr=192.168.0.0/16`.
3. On CP2 + CP3: `sudo kubeadm join lb.example.com:6443 --control-plane --certificate-key <k> --token <t> --discovery-token-ca-cert-hash sha256:<h>`.

Example 4 — GKE Autopilot in one command

```
gcloud container clusters create-auto prod \
  --region=asia-south1 \
  --release-channel=regular
```

Example 5 — EKS with eksctl

```
# cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata: { name: prod, region: ap-south-1, version: "1.31" }
managedNodeGroups:
- name: general
  instanceType: t3.medium
  desiredCapacity: 2
  minSize: 1
  maxSize: 5
  volumeSize: 50
```

```
eksctl create cluster -f cluster.yaml
```

💻 Related Commands

Command	Flag	Purpose	Example
`minikube start`	`--nodes 3 --cpus 2 --memory 4g`	Multi-node local	`minikube start --nodes=3 --cpus=2 --memory=4g`
`minikube addons enable`	`<addon>`	Enable ingress/metrics-server	`minikube addons enable ingress`
`minikube service`	`--url`	Print URL of a Service	`minikube service web --url`
`kind create cluster`	`--config`	Use YAML config	`kind create cluster --config kind.yaml`
`kind get clusters`	—	List Kind clusters	`kind get clusters`
`kubeadm init`	`--pod-network-cidr`	Set pod CIDR for CNI	`sudo kubeadm init --pod-network-cidr=192.168.0.0/16`
`kubeadm join`	`--control-plane`	Join as CP node	see Example 3
`kubeadm reset`	`-f`	Undo an init/join	`sudo kubeadm reset -f`
`gcloud container clusters create-auto`	`--region`	GKE Autopilot	see Example 4
`gcloud container clusters get-credentials`	`--region`	Merge kubeconfig	`gcloud container clusters get-credentials prod --region=asia-south1`
`eksctl create cluster`	`-f`	Declarative EKS	`eksctl create cluster -f cluster.yaml`
`aws eks update-kubeconfig`	`--name --region`	Merge kubeconfig	`aws eks update-kubeconfig --name prod --region ap-south-1`

🚨 Scenarios

Scenario 1 — "Minikube: image pull is slow on flaky Wi-Fi"

- **Fix**: `minikube cache add nginx:1.27` warms an in-cluster cache; then `minikube image load <img>` uses local Docker images without a registry.

Scenario 2 — "Kind cluster works locally but fails in CI"

- **Root cause**: CI runners often lack `/dev/kmsg` or run without kernel modules.
- **Fix**: use official kind GH Action `helm/kind-action`; increase Docker file limits; pre-load images with `kind load docker-image`.

Scenario 3 — "kubeadm cluster: nodes stuck NotReady after init"

- **Root cause**: no CNI installed.
- **Fix**: `kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml`.

Scenario 4 — "GKE cluster creation fails with quota error"

- **Root cause**: default `SSD_TOTAL_GB` or `IN_USE_ADDRESSES` quota exhausted.
- **Fix**: request quota increase in the region, or pick a smaller machine type / no PD.

Scenario 5 — "EKS Fargate pod stuck Pending"

- **Root cause**: pod's namespace/label doesn't match any Fargate profile selector.
- **Fix**: add the profile:

```
eksctl create fargateprofile --cluster prod --name default --namespace default
```

☁ GCP GKE Perspective

Autopilot vs Standard

Feature	Autopilot	Standard
Node management	Google	You
Billing	Per pod resources	Per node VM
Node customization	Very limited	Full
DaemonSets	Allowed (Google-approved)	Free
Best for	Teams that want zero ops	Custom OS, GPUs, special workloads

Handy commands

```
# List versions available in a channel
gcloud container get-server-config --region asia-south1

# Upgrade control plane
gcloud container clusters upgrade prod --master --cluster-version=1.31.3 --region=asia-south1

# Upgrade a node pool
gcloud container clusters upgrade prod --node-pool=default --cluster-version=1.31.3 --region=asia-south1

# Delete cluster
gcloud container clusters delete prod --region=asia-south1 --quiet
```

Notable GKE-only features: **release channels** (Rapid/Regular/Stable/No-channel), **maintenance windows/exclusions**, **Autopilot**, **GKE Dataplane V2** (eBPF), **Backup for GKE**, **GKE Enterprise (Fleet)**, **Workload Identity Federation**.

▩ AWS EKS Perspective

Managed NG vs Self-Managed vs Fargate

Feature	Managed NG	Self-Managed	Fargate
Node OS	AL2023/Bottlerocket	Any	AWS microVM
Patching	AWS launches new AMI	You	AWS
Scaling	ASG	ASG	per-pod
Cost	EC2 hourly	EC2 hourly	vCPU/GB-second
Limits	450 nodes/NG	Cluster-wide	4 vCPU / 30 GB per pod
DaemonSets	Yes	Yes	No

Handy commands

```
# List clusters
aws eks list-clusters --region ap-south-1

# Upgrade cluster version
aws eks update-cluster-version --name prod --kubernetes-version 1.31 --region ap-south-1

# Upgrade a managed NG
aws eks update-nodegroup-version --cluster-name prod --nodegroup-name general --region ap-south-1

# Add-on lifecycle
aws eks describe-addon-versions --kubernetes-version 1.31 --addon-name vpc-cni --region ap-south-1
aws eks update-addon --cluster-name prod --addon-name vpc-cni --addon-version <v> --region ap-south-1

# Delete
eksctl delete cluster --name prod --region ap-south-1
```

EKS-only integrations: **IRSA** (Pod IAM), **VPC CNI** (real ENI IPs), **AWS Load Balancer Controller** (ALB/NLB), **EBS/EFS CSI**, **CloudWatch Container Insights**, **EKS Access Entries** (2024+), **EKS Auto Mode** (2024+).

Hands-On Lab Exercises

Lab 4.1 — Multi-node Kind cluster (Beginner, 30 min)

Objective: Create a 3-worker Kind cluster and confirm scheduling across workers.

Prerequisites: Docker, kind.

Setup + Steps

```
cat > kind-3.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: multi
nodes:
- role: control-plane
- role: worker
- role: worker
- role: worker
EOF
kind create cluster --config kind-3.yaml
kubectl get nodes -o wide
kubectl create deployment web --image=nginx:1.27 --replicas=6
kubectl get pods -o wide     # confirm spread
```

Cleanup: `kind delete cluster --name multi`

Bonus: Load a local Docker image without a registry: `kind load docker-image me/app:1 --name multi`.

Lab 4.2 — kubeadm HA control plane on VMs (Advanced, 90 min)

Objective: Build a 3-CP, 2-worker HA cluster.

Prerequisites: 5 Ubuntu 22.04 VMs, HAProxy VM in front of CPs, network reachability.

Steps 1. Install prereqs on all nodes (see pre-module section 2.4). 2. HAProxy config on LB VM:

```
frontend k8s
  bind *:6443
  mode tcp
  default_backend cps
backend cps
  mode tcp
  balance roundrobin
  server cp1 10.0.0.11:6443 check
  server cp2 10.0.0.12:6443 check
  server cp3 10.0.0.13:6443 check
```

3. On CP1: `sudo kubeadm init --control-plane-endpoint "lb.local:6443" --upload-certs --pod-network-cidr=192.168.0.0/16`. 4. Save the two join commands printed. 5. Install Calico: `kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml`. 6. Join CP2, CP3 with `--control-plane --certificate-key ...`. 7. Join workers with plain join command. 8. Verify: `kubectl get nodes` shows 3 CP + 2 workers Ready.

Cleanup: `sudo kubeadm reset -f && sudo rm -rf /etc/cni/net.d $HOME/.kube` on each node; destroy VMs.

Bonus: Kill CP1; verify LB still gives access and workloads keep running.

Lab 4.3 — GKE Autopilot vs EKS Fargate side-by-side (Intermediate, 60 min)

Objective: Bring up managed clusters on both clouds, deploy the same app, measure wall-clock time and quirks.

Prerequisites: gcloud, eksctl, budgets set.

Steps 1. Time GKE creation:

```
time gcloud container clusters create-auto side-by-side --region=asia-south1
```

2. Time EKS creation:

```
time eksctl create cluster --name side-by-side --region ap-south-1 --fargate
```

3. Apply the same nginx Deployment + LoadBalancer Service to both. 4. Compare: cluster creation time, LB provisioning time, pod scheduling time, cost estimate.

Cleanup (mandatory)

```
gcloud container clusters delete side-by-side --region=asia-south1 --quiet
eksctl delete cluster --name side-by-side --region ap-south-1
```

Bonus: Try to run a DaemonSet on both. On Autopilot Google may reject non-approved DaemonSets; on Fargate DaemonSets are outright unsupported.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: I match the cluster setup to the goal. For learning I run Minikube; for realistic multi-node scenarios and CI I use Kind; for on-prem or production-like practice I use kubeadm; and for real workloads I run either GKE — Autopilot when we want zero node operations, Standard when we need custom node pools — or EKS with managed node groups when we're standardized on AWS.
Why: Managed clusters bury the boring parts — control-plane HA, patching, etcd backups — so my team focuses on workloads. Self-hosted with kubeadm is invaluable for understanding failure modes and passing CKA, but it's a real time sink in production.
How: My default recipe is: pick a region, use a release channel or LTS version, keep node pools small (`e2-medium` / `t3.medium`), enable Workload Identity (GKE) or IRSA (EKS), set budget alerts, then tear down at the end of the lab. I always add a maintenance window and a PodDisruptionBudget so upgrades don't hurt uptime.
Example: On my last project we moved from a self-managed kubeadm cluster to GKE Autopilot; our ops toil dropped ~70% and our monthly cloud spend actually went down because Autopilot bills per pod, not per node."

? Possible Interview Questions

Basic

Q1. Difference between Minikube and Kind?
Minikube runs Kubernetes in a VM or container; single-cluster focus, addon ecosystem. Kind runs multi-node clusters entirely inside Docker containers, ideal for CI.

Q2. Is Docker Desktop's Kubernetes production-grade?
No — single-node, no HA, mostly for developer inner loop.

Q3. What tool is used to install Kubernetes on VMs manually?
`kubeadm`, plus Kubespray or RKE2 for automation.

Intermediate

Q4. What is a release channel in GKE?
A tier (Rapid/Regular/Stable/No-channel) controlling how quickly your cluster gets new K8s versions. Rapid is closest to upstream; Stable is most conservative.

Q5. How does an EKS control plane differ from a kubeadm one?
EKS is fully managed and HA across AZs; you never touch the CP. kubeadm gives you every knob but you own patching, HA, backup, and audit.

Q6. What CNI does GKE use by default?
GKE Dataplane V2 (eBPF, based on Cilium) is the recommended default.

Advanced

Q7. How would you build an HA control plane with kubeadm?
Three CP nodes with stacked etcd (or three external etcd), a TCP LB in front of 6443, use `--control-plane-endpoint`, join CPs with `--upload-certs` / `--certificate-key`, install a CNI, and put workers behind the same LB.

Q8. Trade-offs of Fargate vs managed NG on EKS?
Fargate: no node ops, per-pod billing, strong isolation, but no DaemonSets, no privileged, no hostPath, limited pod sizes, higher pod cost. Managed NG: full pod features, cheaper per pod at scale, but you patch nodes.

Q9. When would you pick Autopilot over Standard on GKE?
When teams don't need node-level customization, want per-pod billing, and want Google to enforce best-practice defaults (PSS, no privileged).

Scenario-based

Q10. Your Minikube pods can't reach the internet — where do you look?
Docker network, VM driver DNS, `--dns` flag, kube-proxy status, CoreDNS pod logs, MTU issues on Kind (`--containerd-config-patches`).

GKE-specific

Q11. What's the fastest way to give an app pod access to a GCS bucket on GKE?
Workload Identity: bind a Kubernetes ServiceAccount to a GCP IAM service account, no keys in the pod.

EKS-specific

Q12. How do you tell EKS to use a specific IAM role for a namespace's pods?
Enable OIDC provider, create IAM role trusting the OIDC issuer with a StringEquals condition on the ServiceAccount, annotate the SA with `eks.amazonaws.com/role-arn` .

Hands-on / Lab-based

Q13. Walk me through your Kind multi-node lab.
Declared 1 CP + 3 workers in kind-3.yaml, created the cluster, ran `nginx` with 6 replicas, confirmed pods spread across workers with `-o wide` . Then loaded a local image via `kind load docker-image` and redeployed.

Q14. In your kubeadm HA lab, what did you do to survive CP1 failure?
LB continued routing to CP2/CP3, kubectl kept working, workloads unaffected. Verified etcd quorum (2/3) via `etcdctl member list` .

Q15. On EKS you saw a Fargate pod pending — what next?
Check Fargate profile selectors, ensure namespace + labels match, check subnet capacity, check pod resources don't exceed Fargate max.

🗐 Topic 5 — kubectl Basics and kubeconfig

Official Documentation Reference - kubectl overview: https://kubernetes.io/docs/reference/kubectl/ - Introduction: https://kubernetes.io/docs/reference/kubectl/introduction/ - Cheat sheet: https://kubernetes.io/docs/reference/kubectl/quick-reference/ - kubeconfig: https://kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/ - Configure access to multiple clusters: https://kubernetes.io/docs/tasks/access-application-cluster/configure-access-multiple-clusters/ - GKE auth plugin: https://cloud.google.com/kubernetes-engine/docs/how-to/cluster-access-for-kubectl - EKS auth: https://docs.aws.amazon.com/eks/latest/userguide/create-kubeconfig.html

⏶ Definition

kubectl is the official Kubernetes CLI that talks to the API server over HTTPS. **kubeconfig** is a YAML file that tells kubectl *which cluster* to talk to, *how to authenticate*, and *which namespace* to default to.

⏶ About

kubectl in one paragraph

kubectl is a Go binary that reads your kubeconfig (default `$HOME/.kube/config`), builds a REST request, adds your credentials, sends it to the API server, and prints the response. Every command follows the pattern:

```
kubectl <verb> <resource> [name] [flags]
verbs   : get, describe, apply, create, delete, edit, rollout, scale, logs, exec, port-forward, cp, top, patch, replace, label, annotate, run, expose, drain, cord
```

Imperative vs Declarative

Style	Example	Use when
Imperative command	`kubectl run web --image=nginx`	quick one-offs, learning
Imperative object config	`kubectl create -f deploy.yaml`	scripts
Declarative	`kubectl apply -f deploy.yaml`	production, GitOps

kubeconfig sections

```
apiVersion: v1
kind: Config
clusters:                       # WHERE
- name: prod
  cluster:
    server: https://1.2.3.4:6443
    certificate-authority: /path/to/ca.crt
users:                          # WHO you are
- name: alice
  user:
    client-certificate: /path/to/alice.crt
    client-key:        /path/to/alice.key
contexts:                       # cluster + user + namespace triplet
- name: prod-alice
  context:
    cluster: prod
    user: alice
    namespace: default
current-context: prod-alice
```

Contexts are the powerful primitive — one command switches cluster, user, and namespace simultaneously.

Merging multiple kubeconfigs

```
export KUBECONFIG="$HOME/.kube/config:$HOME/.kube/gke.yaml:$HOME/.kube/eks.yaml"
kubectl config view --merge --flatten > ~/.kube/merged
mv ~/.kube/merged ~/.kube/config
```

Auth mechanisms

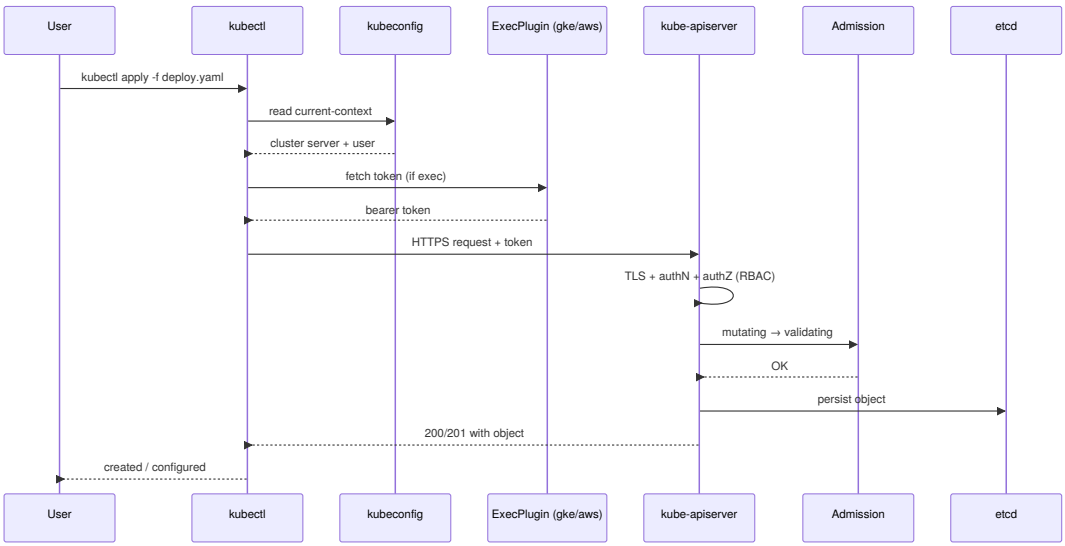
- **client certs** (kubeadm default)
- **bearer tokens / ServiceAccount tokens**
- **OIDC** (Dex, Google, Okta)
- **exec plugins** — kubectl calls an external binary to get creds (used by `gke-gcloud-auth-plugin` and `aws-iam-authenticator` / `aws eks get-token`)

🔷 How It Integrates With Other Kubernetes Components

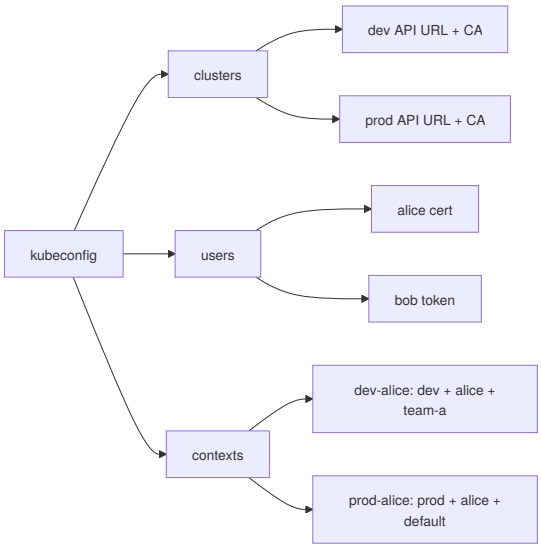
- kubectl never talks to etcd directly — always via kube-apiserver.
- **authN** happens after TLS handshake — cert CN, OIDC id_token, or bearer token.
- **authZ** goes through RBAC (Roles, ClusterRoles).
- **Admission** runs after authZ (mutating, then validating).
- On managed clouds, auth plugins fetch short-lived tokens (Google IAM, AWS STS).

🏗 Diagrams

kubectl Request Flow



kubeconfig Model



💻 Real-World Examples

Example 1 — Everyday inspection cheat sheet

```
kubectl get pods -A -o wide
kubectl get svc,ingress -A
kubectl get events --sort-by=.lastTimestamp | tail -20
kubectl top pods --containers -A
kubectl describe pod web-0
```

Example 2 — Multi-cluster context juggling

```
kubectl config get-contexts
kubectl config use-context prod-alice
kubectl config set-context --current --namespace=team-a
# Or with kubectx/kubens
kubectx prod-alice
kubens team-a
```

Example 3 — Copy a file into a running pod, view remotely

```
kubectl cp ./patch.sh web-0:/tmp/patch.sh
kubectl exec -it web-0 -- bash /tmp/patch.sh
kubectl port-forward svc/web 8080:80   # local:pod
```

Example 4 — Common apply patterns

```
kubectl apply -f manifests/                    # a directory
kubectl apply -k overlays/prod                 # kustomize
kubectl apply --server-side -f deploy.yaml     # SSA (safer for shared objects)
kubectl apply --dry-run=client -f deploy.yaml -o yaml   # preview
kubectl diff -f deploy.yaml                    # show live vs proposed diff
```

Example 5 — Debug a broken pod without editing it

```
kubectl debug -it web-0 --image=busybox --target=web
# creates an ephemeral debug container in the same PID/net namespaces
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get`	`-o wide`	Extra columns	`kubectl get pods -o wide`
`kubectl get`	`-o yaml` / `-o json`	Machine-readable	`kubectl get svc -o yaml`
`kubectl get`	`-w`	Watch changes	`kubectl get pods -w`
`kubectl get`	`-l app=web`	Label selector	`kubectl get pods -l app=web`
`kubectl get`	`--field-selector`	Field selector	`kubectl get pods --field-selector status.phase=Running`
`kubectl describe`	—	Verbose incl. events	`kubectl describe deploy web`
`kubectl apply`	`-f` / `-k`	Declarative apply	`kubectl apply -f d.yaml`
`kubectl apply`	`--server-side`	Server-Side Apply	`kubectl apply --server-side -f d.yaml`
`kubectl apply`	`--dry-run=client`	Preview	`kubectl apply --dry-run=client -f d.yaml -o yaml`
`kubectl create`	`--dry-run=client -o yaml`	Generate YAML skeleton	`kubectl create deployment web --image=nginx --dry-run=client -o yaml`
`kubectl delete`	`-f` / `--all`	Remove	`kubectl delete -f d.yaml`
`kubectl delete`	`--grace-period=0 --force`	Force kill	`kubectl delete pod stuck --grace-period=0 --force`
`kubectl edit`	—	Live edit an object	`kubectl edit deploy web`
`kubectl scale`	`--replicas=`	Change replicas	`kubectl scale deploy web --replicas=5`
`kubectl rollout`	`status` / `undo` / `history`	Deployment control	`kubectl rollout undo deploy/web`
`kubectl logs`	`-f -c c1 --previous`	Follow, container, previous	`kubectl logs -f -c app web-0 --previous`
`kubectl exec`	`-it -- <cmd>`	Interactive shell	`kubectl exec -it web-0 -- sh`
`kubectl port-forward`	`svc/…`	Local tunnel	`kubectl port-forward svc/web 8080:80`
`kubectl cp`	—	Copy files	`kubectl cp file web-0:/tmp/file`
`kubectl top`	`pod/node`	Live CPU/mem	`kubectl top pods -A`
`kubectl config`	`use-context`	Switch context	`kubectl config use-context prod`
`kubectl config`	`set-context --current --namespace=`	Change default NS	`kubectl config set-context --current --namespace=dev`
`kubectl config`	`view` / `--minify`	Show config	`kubectl config view --minify`
`kubectl auth`	`can-i` / `whoami`	RBAC test / identity	`kubectl auth can-i create pods -n dev`
`kubectl explain`	`--recursive`	Field reference	`kubectl explain pod.spec.containers --recursive`
`kubectl api-resources`	`--namespaced=true`	List kinds	`kubectl api-resources --namespaced=true`
`kubectl debug`	`--image` `--target`	Ephemeral container	`kubectl debug -it web-0 --image=busybox --target=web`
`kubectl wait`	`--for=condition=Ready`	Wait on state	`kubectl wait --for=condition=Ready pod/web-0 --timeout=60s`

🚨 Scenarios

Scenario 1 — "kubectl works on my laptop, fails on colleague's"

- **Root cause**: kubeconfig path or context differs. `$KUBECONFIG` env var unset on the second machine.
- **Fix**: Share the merged kubeconfig or export via `export KUBECONFIG=~/.kube/config-shared` .

Scenario 2 — "Wrong cluster deploy"

- **Root cause**: `kubectl apply` executed in the wrong context.
- **Fix**: Adopt `kube-ps1` in the shell prompt or use `kubectx` . For pipelines set `KUBECONFIG` per job explicitly, never rely on a shared file.

Scenario 3 — "gke-gcloud-auth-plugin: command not found"

- **Root cause**: since K8s 1.26, GKE requires a separate auth plugin.
- **Fix**:

```
gcloud components install gke-gcloud-auth-plugin
export USE_GKE_GCLOUD_AUTH_PLUGIN=True
gcloud container clusters get-credentials <name> --region <r>
```

Scenario 4 — "aws eks get-token expired"

- **Root cause**: kubeconfig uses `aws-iam-authenticator` which is deprecated; STS token expired.
- **Fix**: regenerate with `aws eks update-kubeconfig` (uses `aws eks get-token`).

Scenario 5 — "error: forbidden: user cannot list pods"

- **Root cause**: RBAC missing.

- **Fix**: `kubectl auth can-i list pods --as=<user>`. Bind a Role.

☁ GCP GKE Perspective

- Auth plugin: **gke-gcloud-auth-plugin** (installed via `gcloud components install`).
- `gcloud container clusters get-credentials <name> --region <r>` merges an entry into `~/.kube/config`.
- Under the hood the context uses exec plugin `gke-gcloud-auth-plugin` → returns a short-lived Google OAuth token.
- To grant cluster access to another user: give them GCP IAM role `roles/container.developer` **and** RBAC role inside the cluster.

```
gcloud container clusters get-credentials prod --region asia-south1
kubectl config current-context      # gke_project_asia-south1_prod
```

▩ AWS EKS Perspective

- Auth: `aws eks get-token` (from AWS CLI v2) via kubectl exec plugin.
- `aws eks update-kubeconfig --name <cluster> --region <r>` writes a context.
- Since 2024: **EKS Access Entries** replaces the aws-auth ConfigMap:

```
aws eks create-access-entry --cluster-name prod --principal-arn arn:aws:iam::<a>:user/alice --region ap-south-1
aws eks associate-access-policy \
  --cluster-name prod --principal-arn arn:aws:iam::<a>:user/alice \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSAdminPolicy \
  --access-scope type=cluster --region ap-south-1
```

- IAM Identity Center or SSO users work via `aws configure sso` and refresh automatically.

Hands-On Lab Exercises

Lab 5.1 — Master the imperative to declarative flow (Beginner, 30 min)

Objective: Generate YAML from imperative commands, edit, apply, diff.

Prerequisites: any cluster.

Steps

```
# 1. Generate skeleton
kubectl create deployment web --image=nginx:1.27 --dry-run=client -o yaml > deploy.yaml

# 2. Edit → set replicas: 3

# 3. Apply
kubectl apply -f deploy.yaml

# 4. See what would change
sed -i 's/nginx:1.27/nginx:1.28/' deploy.yaml
kubectl diff -f deploy.yaml
kubectl apply -f deploy.yaml
kubectl rollout status deploy/web
```

Verify: `kubectl get deploy web -o wide` shows image `nginx:1.28`.

Cleanup: `kubectl delete -f deploy.yaml`.

Bonus: Repeat with `--server-side` and observe managedFields.

Lab 5.2 — Multi-cluster kubeconfig (Intermediate, 45 min)

Objective: Switch between Minikube, Kind, GKE, EKS with one command.

Prerequisites: Minikube + Kind + GKE + EKS clusters (or a subset).

Steps

```
# Aggregate configs
cp ~/.kube/config ~/.kube/config.bak
export KUBECONFIG=~/.kube/config:~/.kube/gke:~/.kube/eks
kubectl config view --merge --flatten > ~/.kube/all
mv ~/.kube/all ~/.kube/config
unset KUBECONFIG

# Rename ugly contexts
kubectl config rename-context gke_myproj_asia-south1_prod gke-prod
kubectl config rename-context arn:aws:eks:ap-south-1:1234:cluster/prod eks-prod
kubectl config get-contexts

# Fast switching
kubectx minikube && kubectl get nodes
kubectx kind-multi && kubectl get nodes
kubectx gke-prod && kubectl get nodes
kubectx eks-prod && kubectl get nodes
```

Verify: `kubectl config get-contexts` shows 4 named contexts.

Cleanup: none, or `mv ~/.kube/config.bak ~/.kube/config`.

Bonus: Add a `kube-ps1` prompt so the shell always shows current context.

Lab 5.3 — Debug a pod with kubectl debug + auth can-i (Advanced, 60 min)

Objective: Diagnose a running pod and confirm RBAC.

Prerequisites: any cluster; deploy `nginx` first.

Steps 1. `kubectl run web --image=nginx:1.27` 2. `kubectl debug -it web --image=busybox --target=web -- sh` — you're in the same PID/net namespace. 3. Inside: `ps -ef` (see nginx PIDs), `wget -qO- localhost` — should return HTML. 4. Test RBAC as a different user:

```
kubectl auth can-i list pods                # yes
kubectl auth can-i create pods --as=svc:default    # depends on RBAC
kubectl auth whoami
```

5. Simulate a permission gap: create a Role with only `get pods` and bind to a ServiceAccount, then try `create pods` as that SA.

Cleanup: `kubectl delete pod web` , delete Role/Binding.

Bonus: Try `kubectl debug node/<node>` — spins up a privileged debug pod on that node.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: kubectl is the command-line client for the Kubernetes API. It reads a kubeconfig file that describes clusters, users, and contexts, then sends authenticated REST calls to the API server for every verb — get, apply, describe, exec, logs, port-forward, and so on.
Why: The kubeconfig model is what makes multi-cluster work sane. A single context is a triplet — cluster + user + namespace — so switching from dev to prod is one command. Combined with plugins like kubectx and kubens, it prevents the classic 'oops, wrong cluster' incident.
How: I keep separate config files per environment merged with `KUBECONFIG=…` , rename ugly generated contexts, and always show current context in my shell prompt. For managed clouds, I let `gcloud container clusters get-credentials` or `aws eks update-kubeconfig` write the entries, which set up exec-plugin authentication — GKE uses `gke-gcloud-auth-plugin` , EKS uses `aws eks get-token` .
Example: On a recent incident I diagnosed a crash-looping API by chaining `kubectl describe` , `kubectl logs --previous` , and `kubectl debug` with an ephemeral busybox container — within five minutes I confirmed a bad env var was set by a stale ConfigMap and rolled back with `kubectl rollout undo` ."

? Possible Interview Questions

Basic

Q1. What is kubectl and where is kubeconfig by default?
kubectl is the K8s CLI. Default kubeconfig is `$HOME/.kube/config` ; override with `--kubeconfig` or `$KUBECONFIG` .

Q2. `kubectl get pods` shows nothing — why?
Wrong namespace (`-n <ns>` or `-A`), wrong context, or no pods exist.

Q3. Difference between `apply` and `create` ?
`create` fails if object exists; `apply` patches by merging (or SSA). `apply` is idempotent and safe to re-run.

Intermediate

Q4. What is a kubectl context?
A named triple: cluster + user + namespace. `kubectl config use-context <name>` selects one.

Q5. Explain `--server-side` apply.
SSA sends the intent to the apiserver, which computes the merge patch tracking `managedFields` . Multiple actors can own different fields safely.

Q6. How do you inspect the exact YAML the apiserver stored?
`kubectl get <res> <name> -o yaml` . Add `--show-managed-fields=false` for readability. `kubectl get -o json | jq` for scripting.

Advanced

Q7. What does the exec plugin flow look like for GKE?
kubectl invokes `gke-gcloud-auth-plugin` , which uses gcloud's stored credentials to mint a Google OAuth access token, returns it as an ExecCredential JSON, which kubectl attaches as Bearer.

Q8. When would you use `kubectl replace --force` ?
When the target object cannot be patched cleanly (immutable fields changed, e.g., Service selector). It deletes and recreates — dangerous, use sparingly.

Q9. Difference between `kubectl edit` and `kubectl patch` ?
`edit` opens the object in `$EDITOR` (round-trip through YAML). `patch` applies a JSON/Strategic-merge/JSON-Patch operation programmatically — better for scripts.

Scenario-based

Q10. A junior deployed to prod by accident. How would you prevent it going forward?
Separate kubeconfig files per env, restricted RBAC per user, `kubectl` shell prompt showing context, and require CI/CD (ArgoCD) to be the only deployer for prod.

GKE-specific

Q11. Why does `kubectl` sometimes fail with `error: exec plugin: gke-gcloud-auth-plugin not found` after upgrading?
Since K8s 1.26 the plugin is separate. Install via `gcloud components install gke-gcloud-auth-plugin` .

EKS-specific

Q12. In EKS, how do you find your identity mapping?
Older: `kubectl -n kube-system get cm aws-auth -o yaml` . Newer: `aws eks list-access-entries --cluster-name <n>` and `list-associated-access-policies` .

Hands-on / Lab-based

Q13. Walk me through a debug flow you've used.
`kubectl describe pod` for events → `kubectl logs -f --previous` for crashed container → `kubectl exec` for live inspection → `kubectl debug -it <pod> --image=busybox --target=<c>` for ephemeral tooling → `kubectl port-forward` to hit the app locally.

Q14. Show me how to preview a change without applying.

`kubectl diff -f deploy.yaml` or `kubectl apply --dry-run=server -f deploy.yaml -o yaml` — server-side dry run runs admission and validation without persisting.

Q15. How would you list every namespaced resource in a namespace?

```
kubectl api-resources --verbs=list --namespaced -o name | \
  xargs -n1 kubectl get -n <ns> --ignore-not-found
```

MODULE

Module 2 — Workloads

🗐 Topic 6 — Pods

Official Documentation Reference - Pods: https://kubernetes.io/docs/concepts/workloads/pods/ - Pod Lifecycle: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/ - Init Containers: https://kubernetes.io/docs/concepts/workloads/pods/init-containers/ - Sidecar Containers: https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/ - Static Pods: https://kubernetes.io/docs/tasks/configure-pod-container/static-pod/ - Ephemeral Containers: https://kubernetes.io/docs/concepts/workloads/pods/ephemeral-containers/ - GKE Pods: https://cloud.google.com/kubernetes-engine/docs/concepts/pod - EKS Pods (VPC CNI): https://docs.aws.amazon.com/eks/latest/userguide/pod-networking.html

🔹 Definition

A **Pod** is the smallest deployable unit in Kubernetes — a group of one or more containers that share network namespace, IPC, PID (optional), storage volumes, and lifecycle. Every pod gets one IP address.

🔹 About

Why pods, not just containers?

Kubernetes chose Pods (a Borg-inspired concept) so tightly coupled processes could co-locate — for example a main app + log tailer that share `/var/log`. Containers in a pod always land on the same node and start/stop together.

Pod phases (`.status.phase`)

Phase	Meaning
`Pending`	Accepted by API, waiting to schedule or pull images
`Running`	Bound to a node, at least one container is running or starting
`Succeeded`	All containers exited 0 and won't restart
`Failed`	All containers exited, at least one non-zero
`Unknown`	Node lost contact (rare in modern K8s)

Pod conditions

`PodScheduled`, `Initialized`, `ContainersReady`, `Ready` — all must be True before the pod serves traffic.

Container states

`Waiting`, `Running`, `Terminated` — each with `reason` (e.g., `CrashLoopBackOff`, `ImagePullBackOff`, `OOMKilled`).

Init containers

Run **sequentially** before app containers. Must finish successfully. Use for schema migrations, waiting for a dependency, seeding config.

Sidecar containers (native since 1.29)

Long-running helpers declared with `restartPolicy: Always` in the initContainers list. Start before app containers, run in parallel, terminate after them. Perfect for log shippers, service-mesh proxies, secret refresh agents.

Static pods

Managed by kubelet directly from a manifest file (`/etc/kubernetes/manifests/`); NOT via apiserver. Control-plane components on kubeadm are static pods. A "mirror pod" appears in the apiserver so you can see it with `kubectl get pods`.

Ephemeral containers

Debug-only containers added into a running pod with `kubectl debug`. Cannot have ports/probes/resources. Great when the app image has no shell.

Restart policy

`Always` (default for Deployment pods), `OnFailure` (typical for Jobs), `Never` (one-shot). Applies to every container in the pod.

Multi-container pod patterns

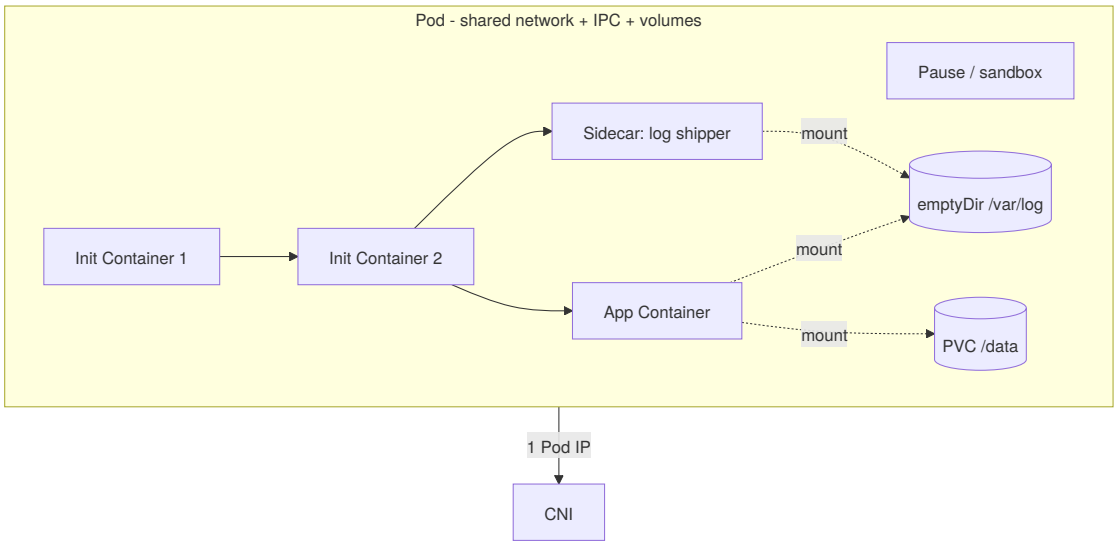
- **Sidecar** — proxy/log/secret helper.
- **Ambassador** — abstracts an external service behind localhost.
- **Adapter** — normalizes output format (e.g., app → statsd → Prometheus).

🔹 How It Integrates With Other Kubernetes Components

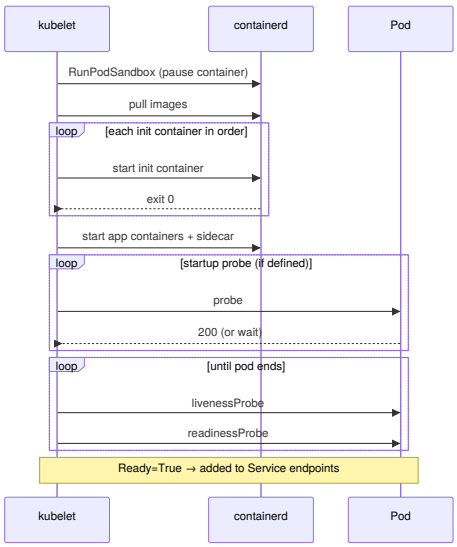
Component	Interaction
Scheduler	Binds Pod to Node (writes `spec.nodeName`)
kubelet	Runs containers via CRI; runs probes; reports status
CNI	Assigns Pod IP; wires netns
CSI	Mounts volumes into containers
kube-proxy	Programs Service → Pod endpoints
Deployment / ReplicaSet / StatefulSet / DaemonSet / Job	Create Pods from templates
ServiceAccount	Auto-mounted token for API auth
Admission (PSA, Kyverno, OPA)	Enforce security context, images, labels

🏗 Diagrams

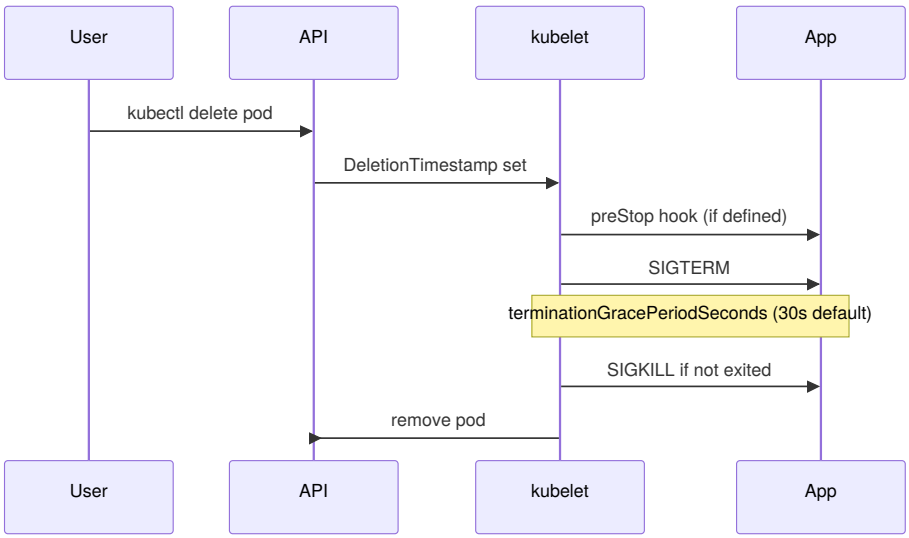
Pod Anatomy



Lifecycle & Probes Sequence



Termination (graceful shutdown)



💻 Real-World Examples

Example 1 — Multi-container pod with init + sidecar

```
apiVersion: v1
kind: Pod
metadata:
  name: web
  labels: { app: web }
spec:
  initContainers:
  - name: wait-for-db
    image: busybox:1.36
    command: ['sh', '-c', 'until nc -z db 5432; do sleep 2; done']
  - name: log-shipper       # native sidecar (K8s 1.29+)
    image: fluent/fluent-bit:3
    restartPolicy: Always
    volumeMounts:
    - { name: applogs, mountPath: /var/log/app }
  containers:
  - name: app
    image: nginx:1.27
    ports: [{ containerPort: 80 }]
    livenessProbe:
      httpGet: { path: /, port: 80 }
      initialDelaySeconds: 5
      periodSeconds: 10
    readinessProbe:
      httpGet: { path: /, port: 80 }
      periodSeconds: 5
    resources:
      requests: { cpu: 100m, memory: 128Mi }
      limits:   { cpu: 500m, memory: 256Mi }
    volumeMounts:
    - { name: applogs, mountPath: /var/log/nginx }
  volumes:
  - name: applogs
    emptyDir: {}
  terminationGracePeriodSeconds: 30
```

Example 2 — Pod with lifecycle hooks + security context

```
apiVersion: v1
kind: Pod
metadata: { name: hardened }
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile: { type: RuntimeDefault }
  containers:
  - name: app
    image: gcr.io/distroless/static:nonroot
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
    lifecycle:
      postStart:
        exec: { command: ["/bin/warmup"] }
      preStop:
        exec: { command: ["/bin/drain", "--wait=10s"] }
```

Example 3 — Static pod (kubeadm nodes)

Drop this file on any worker at `/etc/kubernetes/manifests/echo.yaml`:

```
apiVersion: v1
kind: Pod
metadata: { name: echo }
spec:
  containers:
  - { name: c, image: hashicorp/http-echo, args: ["-text=hi"] }
```

kubelet starts it automatically; `kubectl get pod echo-<node>` shows the mirror.

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl run`	`--image`	Ad-hoc pod	`kubectl run tmp --image=busybox --rm -it -- sh`
`kubectl run`	`--restart=Never`	Bare pod (not a Deployment)	`kubectl run p --image=nginx --restart=Never`
`kubectl get pods`	`-o wide`	See node + IP	`kubectl get pods -o wide`
`kubectl describe pod`	—	Events + probes + volumes	`kubectl describe pod web`
`kubectl logs`	`-c <container>`	Pick a container	`kubectl logs web -c log-shipper`
`kubectl logs`	`--previous`	Prior container instance	`kubectl logs web --previous`
`kubectl exec`	`-it -- sh`	Shell into a container	`kubectl exec -it web -c app -- sh`
`kubectl debug`	`--image --target`	Ephemeral debug container	`kubectl debug -it web --image=busybox --target=app`
`kubectl port-forward`	`pod/<n> local:remote`	Local tunnel	`kubectl port-forward pod/web 8080:80`
`kubectl delete pod`	`--grace-period=0 --force`	Force kill	`kubectl delete pod web --grace-period=0 --force`
`kubectl label pod`	`key=val`	Add label	`kubectl label pod web env=prod`
`kubectl annotate pod`	`key=val`	Add annotation	`kubectl annotate pod web owner=team-a`
`kubectl top pod`	`--containers`	Live CPU/mem per container	`kubectl top pod web --containers`
`kubectl explain pod.spec`	`--recursive`	Field docs	`kubectl explain pod.spec.containers --recursive`

🚨 Scenarios

Scenario 1 — Pod is `Pending`

- **Common causes**: no node fits resource requests; taints without tolerations; unschedulable node selector/affinity; PVC not bound.
- **Diagnose**: `kubectl describe pod` (events at the bottom).
- **Fix**: reduce requests, add tolerations, wait for PVC, add nodes.

Scenario 2 — Pod in `CrashLoopBackOff`

- **Cause**: container exits repeatedly (bad command, missing env, misconfig).
- **Diagnose**: `kubectl logs <pod> --previous`, `kubectl describe pod`.
- **Fix**: correct env/config; add readiness before liveness; check startupProbe if slow-boot.

Scenario 3 — Pod is `Running` but not `Ready`

- **Cause**: readinessProbe failing; sidecar not up; ephemeral storage full.
- **Diagnose**: `kubectl describe pod` → conditions block; `kubectl logs -c <sidecar>`.
- **Fix**: relax probe, fix container, mount larger emptyDir.

Scenario 4 — `OOMKilled`

- **Cause**: memory usage exceeded `limits.memory`; kernel OOM-killer terminated container.
- **Diagnose**: `kubectl describe pod | grep -A2 "Last State"` shows `OOMKilled`.
- **Fix**: bump limit, fix leak, add pod-level cgroup for a group of containers.

Scenario 5 — Static pod won't update

- **Cause**: file wasn't overwritten atomically; kubelet cache mismatch.
- **Fix**: `mv new.yaml /etc/kubernetes/manifests/kube-apiserver.yaml`; kubelet restarts the pod within seconds.

☁ GCP GKE Perspective

- **Autopilot pod defaults**: enforced securityContext (non-root, no privileged), resource requests required, no hostPath, no host network. Google keeps a fleet of preconfigured pod sizes; requests are rounded up.
- **Autopilot pricing**: per pod CPU-second, memory-second, and ephemeral-storage-second.
- **GKE Standard**: normal pod behavior; `kubectl get pods -o wide` shows node names.
- **GKE Sandbox** (`--sandbox=type=gvisor`) runs pods under gVisor for stronger isolation.
- Handy:

```
gcloud container node-pools describe default-pool --cluster prod --region asia-south1
```

▩ AWS EKS Perspective

- **VPC CNI** = every Pod gets a real VPC IP from a secondary ENI on the node. IP exhaustion is real: use `WARM_ENI_TARGET`, `MINIMUM_IP_TARGET`, or **prefix delegation** (`ENABLE_PREFIX_DELEGATION=true`) to pack more IPs per ENI.
- **Fargate pods**: run in Firecracker microVMs; no DaemonSets/privileged/hostPath. Have implicit resource limits (0.25–4 vCPU, 0.5–30 GB).
- **Pod Identity** (2023+) or **IRSA** attach IAM roles per pod ServiceAccount.
- Handy:

```
kubectl get pod web -o jsonpath='{.spec.nodeName}'
aws ec2 describe-network-interfaces --filters "Name=tag:node.k8s.amazonaws.com/instance_id,Values=<i>"
```

🧪 Hands-On Lab Exercises

Lab 6.1 — Init container + sidecar + probes (Beginner, 30 min)

Objective: Watch the exact order of init → sidecar → app.

Prerequisites: Minikube (K8s ≥ 1.29 for native sidecar).

Setup + Steps

```
# pod-order.yaml
apiVersion: v1
kind: Pod
metadata: { name: order-demo, labels: { app: order } }
spec:
  initContainers:
  - name: init-a
    image: busybox:1.36
    command: ['sh','-c','echo init-a; sleep 3']
  - name: sidecar-log
    image: busybox:1.36
    restartPolicy: Always
    command: ['sh','-c','while true; do echo sidecar $(date); sleep 5; done']
  containers:
  - name: app
    image: nginx:1.27
    readinessProbe:
      httpGet: { path: /, port: 80 }
      periodSeconds: 3
```

```
kubectl apply -f pod-order.yaml
kubectl get pod order-demo -w      # watch init → running
kubectl logs order-demo -c init-a
kubectl logs order-demo -c sidecar-log
kubectl logs order-demo -c app
kubectl describe pod order-demo | grep -A5 Conditions
```

Verify: `Initialized=True`, `Ready=True`, sidecar keeps logging every 5 s.

Cleanup: `kubectl delete pod order-demo`.

Bonus: Make init-a `exit 1` and observe pod stuck in `Init:CrashLoopBackOff`.

Lab 6.2 — Force `OOMKilled` and diagnose (Intermediate, 45 min)

Objective: Trigger a memory kill and read the signals.

Steps

```
apiVersion: v1
kind: Pod
metadata: { name: hungry }
spec:
  containers:
  - name: mem
    image: polinux/stress
    resources:
      requests: { memory: "50Mi", cpu: "100m" }
      limits:   { memory: "100Mi", cpu: "200m" }
    command: ["stress"]
    args: ["--vm", "1", "--vm-bytes", "250M", "--vm-hang", "1"]
```

```
kubectl apply -f hungry.yaml
kubectl get pod hungry -w
kubectl describe pod hungry | grep -A3 "Last State"     # OOMKilled
kubectl logs hungry --previous
```

Bonus: Raise limit to 300Mi; verify pod stays Running.

Lab 6.3 — Debug a distroless pod with ephemeral container (Advanced, 45 min)

Objective: Diagnose a pod whose image has no shell.

Steps

```
apiVersion: v1
kind: Pod
metadata: { name: distro }
spec:
  containers:
  - name: app
    image: gcr.io/distroless/static:nonroot
    command: ["/nonexistent"]  # will fail
```

```
kubectl apply -f distro.yaml
kubectl get pod distro               # CrashLoopBackOff
kubectl debug -it distro --image=busybox --target=app -- sh
# inside busybox:
ls /proc/1/root/              # inspect distroless image files
ps -ef                        # see app process (crashed) via shared PID (add shareProcessNamespace)
exit
kubectl describe pod distro | tail -15
```

Cleanup: `kubectl delete pod distro`.

Bonus: Add `shareProcessNamespace: true` to the pod spec and redo `ps -ef` inside the ephemeral container.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Pod is the smallest deployable unit in Kubernetes. It's a group of one or more containers that share a network namespace, IPC, and optionally the process namespace — they always land on the same node, get one Pod IP, and share their lifecycle.
Why: The pod abstraction lets us co-locate tightly coupled processes — a main app with a log shipper sidecar or a service-mesh proxy — without giving up per-container images or resource limits. Init containers guarantee ordered setup before the app starts.
How: In production I design pods with an init container that waits for dependencies, a native sidecar (K8s 1.29+) for log forwarding or Envoy, an app container with tight resource requests and limits, and explicit liveness/readiness/startup probes. I set a securityContext for non-root, drop capabilities, and use a preStop hook so the app can drain connections cleanly before SIGTERM.
Example: On a payment service we cut cold-start errors to zero by adding a startup probe on the app plus an init container that ran a migration; readiness kept the pod out of the Service endpoints until it was truly warm."

? Possible Interview Questions

Basic

Q1. What is a Pod?
Smallest deployable unit — group of containers sharing network/IPC/storage and lifecycle.

Q2. Can a Pod have more than one container?
Yes — one main container plus sidecars/init containers.

Q3. Do containers in the same Pod share an IP?
Yes — one Pod IP; containers talk over `localhost` on different ports.

Intermediate

Q4. Difference between init container and sidecar.
Init runs to completion before app containers. Sidecar (native since 1.29) runs alongside app for its whole lifetime; declared as an initContainer with `restartPolicy: Always`.

Q5. What are pod phases?
Pending, Running, Succeeded, Failed, Unknown.

Q6. What's a static pod?
Pod created directly by kubelet from a manifest directory, bypassing the API server. Control-plane components on kubeadm are static pods.

Advanced

Q7. Explain PID namespace sharing.
`shareProcessNamespace: true` merges PID namespaces so containers see each other's processes — useful for debug sidecars or signal delivery.

Q8. How do liveness, readiness, and startup probes differ?
Liveness restarts the container if it hangs; readiness gates it out of Service endpoints if not ready; startup delays the other two while a slow-boot app comes up.

Q9. What happens during graceful termination?
DeletionTimestamp set → pod removed from Service endpoints → preStop hook runs → SIGTERM → grace period (default 30 s) → SIGKILL. Meanwhile kubelet updates status.

Scenario-based

Q10. A pod is stuck in `ContainerCreating` for 10 minutes — where do you look?
`kubectl describe pod` (events) → check image pull errors, missing Secret/ConfigMap, PVC binding, CNI failures (`FailedCreatePodSandBox`).

GKE-specific

Q11. Why does Autopilot reject my pod with `hostPath` mount?
Autopilot enforces PSS restricted-ish policies; hostPath, hostNetwork, and privileged are blocked. Use a PVC or CSI-backed volume instead.

EKS-specific

Q12. Your EKS node ran out of pod IPs — options?
Enable `ENABLE_PREFIX_DELEGATION=true` on VPC CNI, or move to larger instance types (more ENIs), or use secondary CIDRs.

Hands-on / Lab-based

Q13. Show me how you'd diagnose a distroless pod with no shell.
`kubectl debug -it <pod> --image=busybox --target=<container>` — ephemeral container joins net/PID namespaces without touching the pod spec.

Q14. What order did you observe in your lab: init, sidecar, app?
Init containers ran serially first; native sidecar started, then app container. Sidecar continued after app was terminated on delete.

Q15. When would you avoid multi-container pods?
When components have independent scaling or lifecycle needs — those belong in separate Deployments and Services.

🗐 Topic 7 — ReplicaSets

Official Documentation Reference - ReplicaSet: https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/ - ReplicationController (legacy): https://kubernetes.io/docs/concepts/workloads/controllers/replicationcontroller/ - Owners and dependents: https://kubernetes.io/docs/concepts/overview/working-with-objects/owners-dependents/ - GKE workloads: https://cloud.google.com/kubernetes-engine/docs/concepts/workload - EKS workloads: https://docs.aws.amazon.com/eks/latest/userguide/workloads.html

---

🔷 Definition

A **ReplicaSet** maintains a stable set of replica Pods running at any given time — guaranteeing a specified number of Pods matching a label selector are always alive.

🔷 About

Why ReplicaSets exist

Before Deployments, ReplicaSets provided horizontal replication + healing. Today they still power that muscle, but you rarely create them by hand — a **Deployment owns a ReplicaSet** and creates a new one each time you change the pod template.

Key spec fields

Field	Meaning
`spec.replicas`	Desired pod count
`spec.selector`	Label selector matching the pods it owns
`spec.template`	Pod template

Selector semantics

- `matchLabels` — key/value equality.
- `matchExpressions` — set-based (`In` , `NotIn` , `Exists` , `DoesNotExist`).
- Selector and template labels **must match** or the API rejects it.

OwnerReferences and garbage collection

Every Pod created by a RS carries `ownerReferences` pointing back to the RS. Deleting the RS cascades to Pods unless you pass `--cascade=orphan` .

ReplicaSet vs ReplicationController

- RS supports set-based selectors; RC only equality.
- RC is legacy — do not use for new work.

When to touch a RS directly

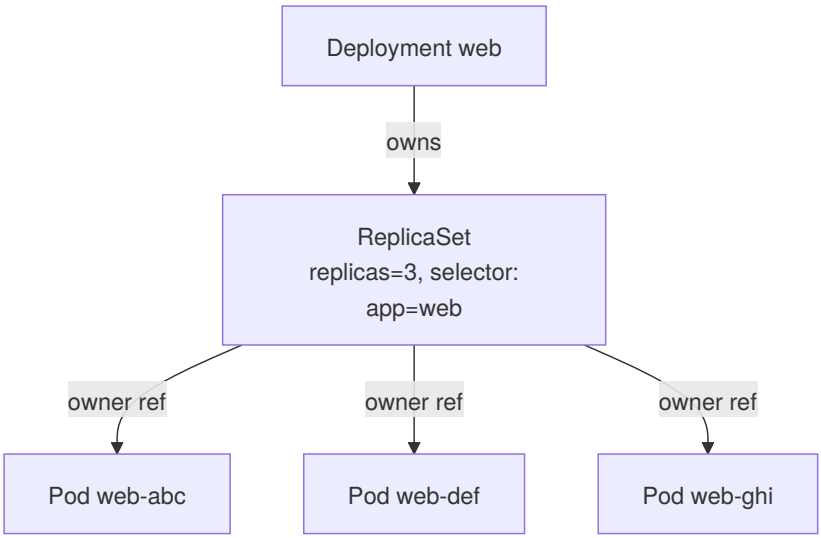
- Debugging a rollout (see which RS a Deployment created).
- Scaling manually with `kubectl scale rs/<name>` (rarely).

🔷 How It Integrates With Other Kubernetes Components

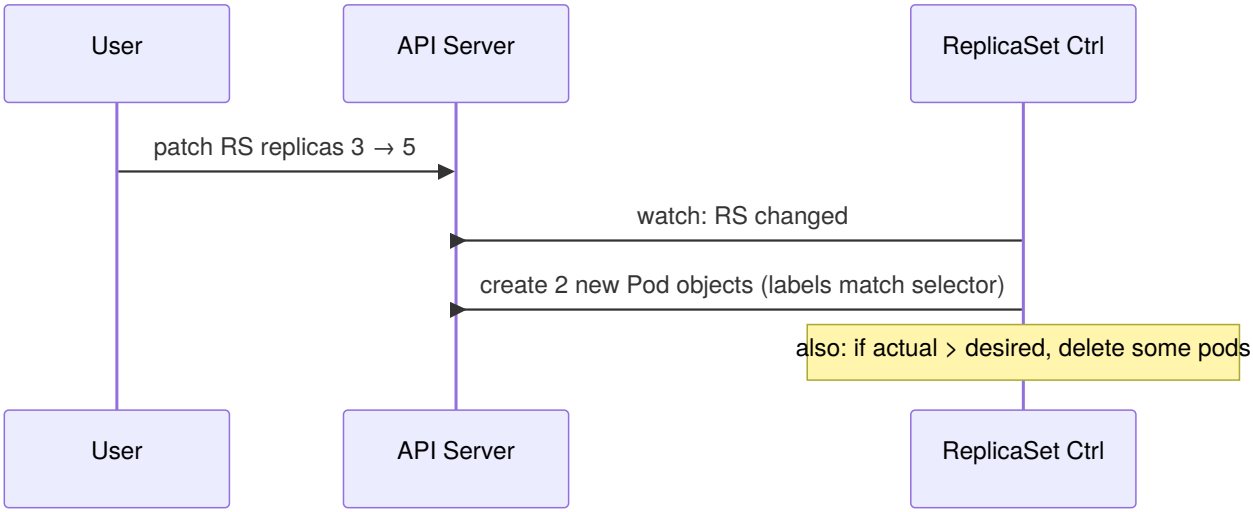
Component	Interaction
Deployment	Creates & manages RS (one per pod-template hash)
Pod	Owned by RS via ownerReferences
Scheduler	Places RS-owned pods on nodes
HPA	Scales `replicas` of the RS's owner Deployment
Kube-controller-manager	Hosts the `replicaset` controller

🏗 Diagrams

RS is a wrapper over Pods



Reconciliation



💻 Real-World Examples

Example 1 — Bare ReplicaSet

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web, tier: front }
  template:
    metadata:
      labels: { app: web, tier: front }
    spec:
      containers:
      - { name: nginx, image: nginx:1.27, ports: [{ containerPort: 80 }] }
```

```
kubectl apply -f rs.yaml
kubectl get rs
kubectl get pods -l app=web
kubectl delete pod $(kubectl get pod -l app=web -o jsonpath='{.items[0].metadata.name}')
kubectl get pods -l app=web        # replacement appears
```

Example 2 — Set-based selector

```
apiVersion: apps/v1
kind: ReplicaSet
metadata: { name: multi-env }
spec:
  replicas: 4
  selector:
    matchExpressions:
    - { key: env, operator: In, values: [dev, staging] }
    - { key: app, operator: Exists }
  template:
    metadata: { labels: { app: web, env: dev } }
    spec:
      containers: [{ name: nginx, image: nginx:1.27 }]
```

Example 3 — Scale a Deployment's underlying RS

```
kubectl get deploy web -o jsonpath='{.metadata.uid}'
kubectl get rs -l app=web -o wide         # spot the RS whose CURRENT ≠ DESIRED
kubectl scale rs/<rs-name> --replicas=6   # temporary; Deployment will reconcile next apply
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get rs`	`-o wide`	See RS + selector	`kubectl get rs -o wide`
`kubectl describe rs`	—	Events + pod status	`kubectl describe rs web-abc`
`kubectl scale rs`	`--replicas=`	Change replica count	`kubectl scale rs/web-abc --replicas=5`
`kubectl delete rs`	`--cascade=orphan`	Delete RS, keep pods	`kubectl delete rs web-abc --cascade=orphan`
`kubectl get pods`	`-l <selector>`	Query by selector	`kubectl get pods -l app=web`
`kubectl edit rs`	—	Manual edit	`kubectl edit rs web-abc`
`kubectl rollout status`	`rs/<name>`	Not supported (use deploy)	—

🚨 Scenarios

Scenario 1 — Pods keep respawning after `kubectl delete pod`

- **Cause**: RS controller ensures desired count.
- **Fix**: Delete the RS or scale it to 0: `kubectl scale rs/web --replicas=0`.

Scenario 2 — Selector mismatch across ReplicaSets

- **Cause**: Two RS have overlapping selectors → they "fight" for the same pods.
- **Fix**: Add a unique label (e.g., `pod-template-hash` that Deployments add automatically) and align selectors.

Scenario 3 — Deployment stuck with old RS pods

- **Cause**: New RS pods failing readiness; Deployment keeps old pods to satisfy availability.
- **Diagnose**: `kubectl get rs -l app=web` (READY column), `kubectl describe rs <new-hash>`.
- **Fix**: Fix probes/image; `kubectl rollout undo deploy/web` if needed.

☁ GCP GKE Perspective

- RS behavior is identical. On **Autopilot** each pod is billed per resource-second, so an idle-but-alive RS still costs money — scale to 0 when not needed.
- `gcloud container clusters describe` won't show RS specifics; use `kubectl get rs`.

▩ AWS EKS Perspective

- Same on EKS. On **Fargate**, each RS pod is a microVM; scaling replicas up = more microVMs allocated.
- IP planning: 10 replicas × VPC CNI without prefix delegation = 10 VPC IPs consumed.

🧪 Hands-On Lab Exercises

Lab 7.1 — See a RS heal itself (Beginner, 20 min)

Steps

```
cat <<'EOF' | kubectl apply -f -
apiVersion: apps/v1
kind: ReplicaSet
metadata: { name: rs-demo }
spec:
  replicas: 3
  selector: { matchLabels: { app: demo } }
  template:
    metadata: { labels: { app: demo } }
    spec: { containers: [{ name: c, image: nginx:1.27 }] }
EOF
kubectl get rs,pod -l app=demo
kubectl delete pod -l app=demo --wait=false
kubectl get pods -l app=demo -w    # new pod appears within seconds
```

Cleanup: `kubectl delete rs rs-demo`.

Bonus: `kubectl scale rs/rs-demo --replicas=0` then back to 3.

Lab 7.2 — Orphan pods with `--cascade=orphan` (Intermediate, 30 min)

Steps

```
kubectl apply -f rs.yaml
kubectl delete rs rs-demo --cascade=orphan
kubectl get pods -l app=demo          # pods survive
kubectl label pods -l app=demo orphaned=true
```

Bonus: Create a new RS with the same selector — it "adopts" the orphan pods.

Lab 7.3 — Two RS fight for the same label (Advanced, 30 min)

Steps 1. Apply RS A with `matchLabels: {app: fight}` replicas=3 and template labels `{app: fight, color: red}`. 2. Apply RS B with same selector but template labels `{app: fight, color: blue}`. 3. `kubectl get pods -l app=fight -w` — pods oscillate between red and blue as each RS creates and the other doesn't recognize. 4. Fix by adding `color` to each selector.

Cleanup: delete both RS.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A ReplicaSet is a controller that keeps a fixed number of pods matching a label selector alive. If pods are deleted or nodes die, it creates replacements; if too many exist, it deletes extras.
Why: It's the primitive that gives Kubernetes its self-healing property for stateless workloads.
How: In practice I never write a ReplicaSet by hand — I write a Deployment. The Deployment creates one ReplicaSet per pod template revision, which makes rolling updates and rollbacks trivial. If I need to inspect a rollout I look at the underlying ReplicaSets with `kubectl get rs`, watch which one is scaling up vs down.
Example: On a rollout gone bad, we saw both the old and new RS at 2/2 for hours. `kubectl describe rs <new-hash>` showed readiness failures. We fixed the probe, and the new RS scaled up while the old scaled down — the rollout completed without touching the Deployment definition."

? Possible Interview Questions

Basic

Q1. What does a ReplicaSet do? Ensures a specified number of replica pods are running.
Q2. Difference between RS and RC? RS supports set-based selectors; RC only equality. RC is legacy.
Q3. Do you create ReplicaSets directly? Rarely — Deployments manage them for you.

Intermediate

Q4. What links a Pod to its ReplicaSet? `metadata.ownerReferences` pointing to the RS.
Q5. What is `pod-template-hash`? A label Deployment adds to every RS/Pod based on the template's hash, ensuring unique selectors per revision.
Q6. What happens when you scale a RS to 0? All owned pods are deleted; RS remains.

Advanced

Q7. How does the RS controller decide which pods to delete when scaling down?
It ranks candidates: pods not on a real node > pending > unready > lowest-ready-time > lowest phase.
Q8. Can a RS adopt pre-existing pods? Yes — if labels match selector and pods have no controller ref.
Q9. What is cascade delete? Deleting an owner (RS) triggers deletion of dependents (Pods) by garbage collector.

Scenario-based

Q10. Two RS with overlapping selectors — what happens? They "fight" — each creates pods to satisfy its count but sees the other's pods as excess to delete, causing thrashing. Fix selectors.

GKE-specific

Q11. On Autopilot, does a paused RS still cost money? A scaled-to-0 RS costs nothing (no pods). A scaled-to-N RS is billed per pod-second regardless of activity.

EKS-specific

Q12. Your RS wants 20 replicas but only 6 pods run on EKS — likely reason? VPC CNI IP exhaustion on the node subnets. Confirm with `kubectl describe pod` events like `failed to assign an IP address`.

Hands-on / Lab-based

Q13. Show a RS healing after pod deletion. Delete a pod owned by RS; observe replacement appearing in `kubectl get pods -w`.
Q14. How do you inspect the RSes a Deployment created? `kubectl get rs -l app=<name>`; each has `.metadata.ownerReferences` pointing to the Deployment.
Q15. When would you orphan RS pods? Rare — during a delicate migration to a new controller, or when snapshotting a state for forensic analysis.

🗐 Topic 8 — Deployments (Rolling Updates, Rollbacks, Strategies)

Official Documentation Reference - Deployment: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ - Update deployment without downtime: https://kubernetes.io/docs/tasks/run-application/update-deployment-rolling/ - Canary tutorial: https://kubernetes.io/docs/tutorials/stateless-application/canary-deployment/ - GKE Deployment: https://cloud.google.com/kubernetes-engine/docs/how-to/updating-apps - EKS rollouts: https://docs.aws.amazon.com/eks/latest/userguide/workloads.html

🔹 Definition

A **Deployment** is a higher-level controller that manages ReplicaSets to run stateless application pods and to perform declarative, no-downtime updates and rollbacks.

🔹 About

Anatomy

- Deployment → creates a **new ReplicaSet** each time the pod template changes (identified by `pod-template-hash`).
- Old ReplicaSets are kept for **rollback history** (default 10, controlled by `revisionHistoryLimit`).

Update strategies

`spec.strategy.type`	Behavior	Downtime	Use case
`RollingUpdate` (default)	Gradually replace pods respecting `maxSurge` + `maxUnavailable`	None	Web/API services
`Recreate`	Kill all old, then create new	Yes	Apps that cannot run 2 versions concurrently (single-writer DB migrations, incompatible schemas)

RollingUpdate knobs

- `maxSurge` — extra pods allowed above `replicas` during update (default 25%).
- `maxUnavailable` — pods allowed to be down during update (default 25%).
- `minReadySeconds` — how long a new pod must be Ready before considered available (add to catch flaky pods).
- `progressDeadlineSeconds` — fail the rollout if no progress in this window (default 600 s).

Advanced patterns (via manifest + Service tricks)

- **Blue-green** — 2 Deployments (`blue`, `green`); switch Service selector from `color: blue` to `color: green`.
- **Canary** — one Deployment `stable`, one Deployment `canary` with fewer replicas; both selected by the same Service via a shared `app` label.
- **Progressive delivery** (Argo Rollouts, Flagger) — CRDs that add proper canary/blue-green with traffic-shift metrics.

Pause/resume

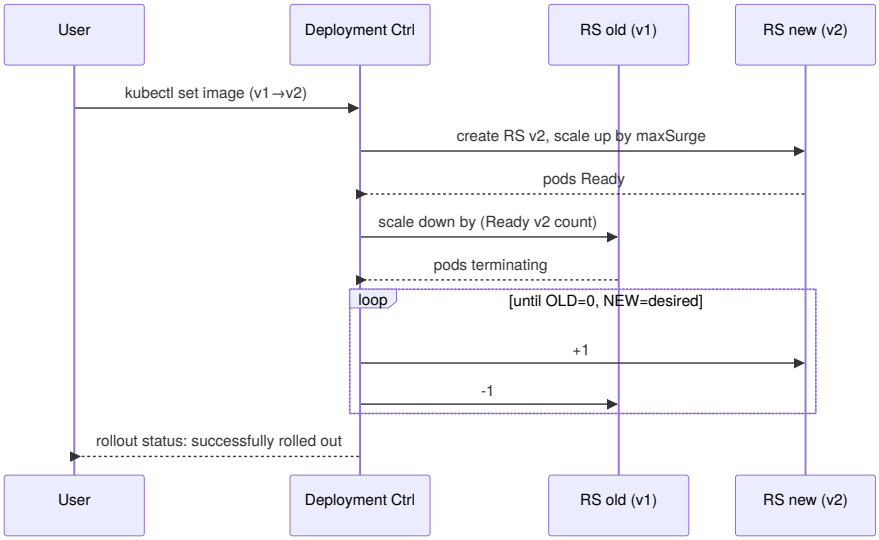
`kubectl rollout pause deploy/web` freezes updates; useful to bundle multiple edits before triggering a rollout.

🔹 How It Integrates With Other Kubernetes Components

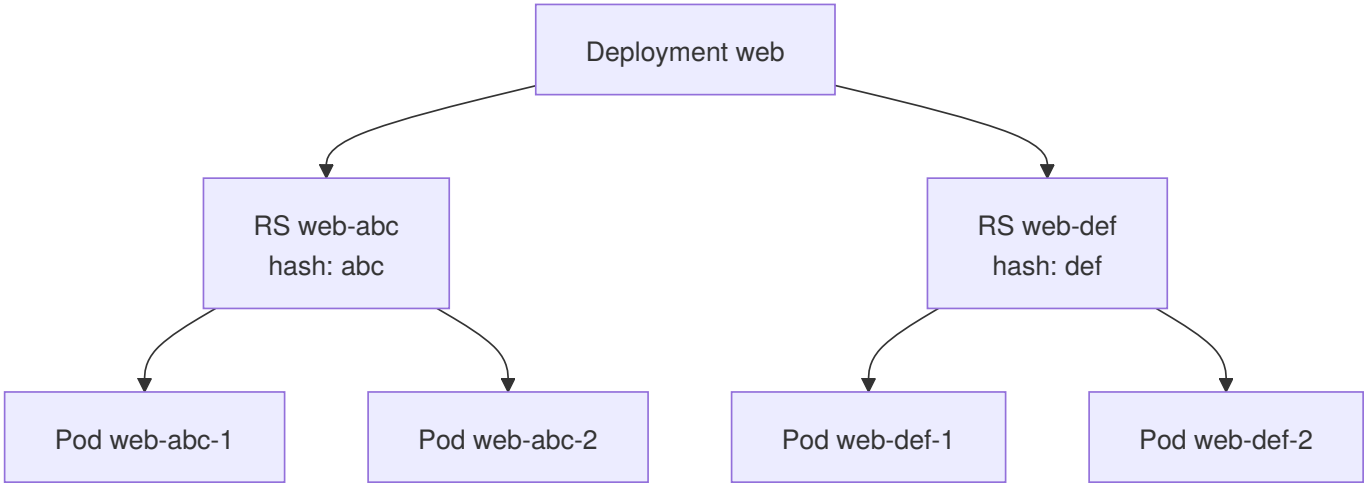
Component	Interaction
ReplicaSet	Deployment creates/scales/deletes RS per revision
Service	Points at Deployment's pods via selector (unchanged during rollout)
HPA	Scales `.spec.replicas` on the Deployment
Ingress	Routes external traffic to the Service
PodDisruptionBudget	Guards evictions during rollouts + node drains
Admission (Kyverno/OPA)	Can enforce probes, resources, image registries

🏗 Diagrams

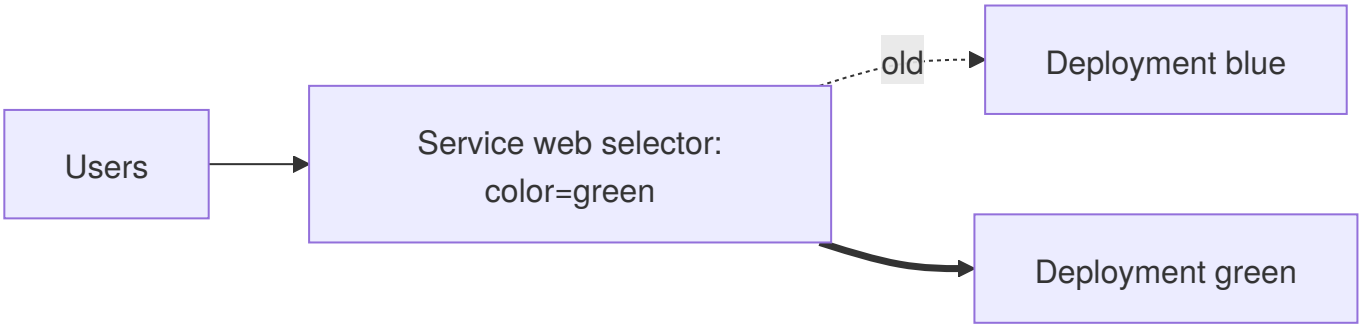
Rolling update timeline



Deployment ↔ RS ↔ Pod ownership



Blue-green switch



💻 Real-World Examples

Example 1 — Production-ready Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels: { app: web }
spec:
  replicas: 5
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 300
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 0        # zero-downtime rollouts
  minReadySeconds: 10
  selector:
    matchLabels: { app: web }
  template:
    metadata: { labels: { app: web } }
    spec:
      terminationGracePeriodSeconds: 45
      containers:
      - name: app
        image: myrepo/web:1.4.2
        ports: [{ containerPort: 8080 }]
        resources:
          requests: { cpu: 100m, memory: 128Mi }
          limits:   { cpu: 500m, memory: 512Mi }
        startupProbe:
          httpGet: { path: /healthz, port: 8080 }
          failureThreshold: 30
          periodSeconds: 2
        readinessProbe:
          httpGet: { path: /ready, port: 8080 }
          periodSeconds: 5
        livenessProbe:
          httpGet: { path: /healthz, port: 8080 }
          periodSeconds: 10
        lifecycle:
          preStop:
            exec: { command: ["/bin/drain","--wait=20s"] }
```

Example 2 — Rolling update, then rollback

```yaml
kubectl set image deploy/web app=myrepo/web:1.4.3
kubectl rollout status deploy/web
# oh no
kubectl rollout undo deploy/web                     # to previous
kubectl rollout undo deploy/web --to-revision=3     # to specific
kubectl rollout history deploy/web
```

Example 3 — Canary using two Deployments + one Service

```yaml
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web }        # matches both stable and canary
  ports: [{ port: 80, targetPort: 8080 }]
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: web-stable }
spec:
  replicas: 9
  selector: { matchLabels: { app: web, track: stable } }
  template: { metadata: { labels: { app: web, track: stable } },
              spec: { containers: [{ name: c, image: myrepo/web:1.4.2 }] } }
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: web-canary }
spec:
  replicas: 1
  selector: { matchLabels: { app: web, track: canary } }
  template: { metadata: { labels: { app: web, track: canary } },
              spec: { containers: [{ name: c, image: myrepo/web:1.4.3 }] } }
```

Ratio: 9:1 traffic split at Service level.

Example 4 — Recreate strategy

```yaml
spec:
  strategy: { type: Recreate }
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create deployment`	`--image`	Create quickly	`kubectl create deploy web --image=nginx`
`kubectl set image`	`deploy/<n> <c>=<img>`	Trigger a rollout	`kubectl set image deploy/web app=nginx:1.28`
`kubectl scale`	`--replicas=`	Manual scale	`kubectl scale deploy/web --replicas=5`
`kubectl rollout status`	`deploy/<n>`	Watch progress	`kubectl rollout status deploy/web`
`kubectl rollout history`	`deploy/<n>`	List revisions	`kubectl rollout history deploy/web`
`kubectl rollout history`	`--revision=3`	Show one revision	`kubectl rollout history deploy/web --revision=3`
`kubectl rollout undo`	`--to-revision=N`	Rollback	`kubectl rollout undo deploy/web --to-revision=2`
`kubectl rollout pause`	`deploy/<n>`	Hold updates	`kubectl rollout pause deploy/web`
`kubectl rollout resume`	`deploy/<n>`	Continue	`kubectl rollout resume deploy/web`
`kubectl rollout restart`	`deploy/<n>`	Recycle pods (no image change)	`kubectl rollout restart deploy/web`
`kubectl set env`	`deploy/<n> K=V`	Update env → rollout	`kubectl set env deploy/web LOG=info`
`kubectl edit deploy`	—	Live edit YAML	`kubectl edit deploy web`
`kubectl patch deploy`	`--type=…`	Scripted change	`kubectl patch deploy web -p '{"spec":{"replicas":3}}'`
`kubectl expose deploy`	`--port --type`	Create Service	`kubectl expose deploy web --port=80 --type=ClusterIP`

🚨 Scenarios

Scenario 1 — Rollout stuck at "1 old / 1 new"

- **Cause**: New pods failing readiness (bad image, config, DB down).
- **Diagnose**: `kubectl rollout status deploy/web`; `kubectl describe rs -l app=web` for new-hash RS events; `kubectl logs -l track=canary --previous`.
- **Fix**: `kubectl rollout undo deploy/web` or fix underlying cause.

Scenario 2 — Rollback to earlier version

- **Cause**: v1.4.3 causes 500s.
- **Fix**:

```
kubectl rollout undo deploy/web            # to last
kubectl rollout undo deploy/web --to-revision=3
```

Scenario 3 — Downtime during update

- **Cause**: `maxUnavailable=25%` with only 4 replicas → 1 pod down at a time; PDB not set.
- **Fix**: Set `maxUnavailable: 0`, `maxSurge: 1`, add PDB `minAvailable: 4`.

Scenario 4 — Rollout finishes but stale traffic

- **Cause**: readinessProbe succeeded before app fully warm.
- **Fix**: Add startup probe or `minReadySeconds: 30`.

Scenario 5 — Stuck at "waiting for deployment ... to be observed by the controller"

- **Cause**: `.spec.template.metadata.labels` doesn't match `.spec.selector.matchLabels`, or selector is being changed (immutable after creation).
- **Fix**: Recreate the Deployment with correct selector.

☁ GCP GKE Perspective

- On **Autopilot** the same rollout mechanics apply, but scheduling constraints may slow scale-ups if pod resources force a new node provision. Set `minReadySeconds` slightly higher.
- **Backup for GKE** can snapshot Deployment + PVC state before a risky rollout.
- **Cloud Deploy** (`gcloud deploy releases create`) is Google's managed CD that drives Deployments across GKE targets.

▩ AWS EKS Perspective

- Same Deployment mechanics.
- **AWS CodeDeploy for Kubernetes** and **App Mesh** can front canary/blue-green with weighted traffic.
- **Argo Rollouts** and **Flagger** are the go-to CRDs that shift traffic with Istio/App Mesh/ALB weighted target groups.

🧪 Hands-On Lab Exercises

Lab 8.1 — First rolling update + rollback (Beginner, 30 min)

Steps

```
kubectl create deploy web --image=nginx:1.26 --replicas=4
kubectl set image deploy/web nginx=nginx:1.27
kubectl rollout status deploy/web
kubectl rollout history deploy/web
kubectl set image deploy/web nginx=nginx:not-real
kubectl rollout status deploy/web       # will fail after progressDeadlineSeconds
kubectl rollout undo deploy/web
```

Bonus: change `maxUnavailable: 0` and time the rollout vs default 25%.

Lab 8.2 — Blue-Green via Service selector (Intermediate, 45 min)

Steps 1. Deploy blue: `kubectl create deploy blue --image=nginx:1.26 && kubectl label deploy blue color=blue --overwrite`. 2. Deploy green: `kubectl create deploy green --image=nginx:1.27 && kubectl label deploy green color=green --overwrite`. 3. Add labels to pods via template: `kubectl set env` won't do it — edit `spec.template.metadata.labels` to add `color: blue` / `green`. 4. Create Service pointing to `color: blue`. 5. Verify traffic hits blue. 6. Switch: `kubectl patch svc web -p '{"spec":{"selector":{"color":"green"}}}'`. 7. Verify traffic hits green.

Bonus: Automate with `kubectl` and `hey` load generator to measure switch latency.

Lab 8.3 — Canary with metrics (Advanced, 60 min)

Steps 1. Deploy `web-stable` (9 replicas) and `web-canary` (1 replica) as in Example 3. 2. Install `hey` and run: `hey -z 60s -c 20 http://<svc-ip>/`. 3. Inspect logs — ~10% requests go to canary. 4. Simulate canary failure: set `web-canary` image to `nginx:not-real`. Its readiness fails → Service excludes it. Traffic returns to stable. 5. Fix canary → gradually scale `web-canary=3, web-stable=7` for 30% canary. 6. Full cutover: scale stable=0, canary=10.

Cleanup: `kubectl delete deploy web-stable web-canary && kubectl delete svc web`.

Bonus: Replace with **Argo Rollouts** — install controller, replace both Deployments with one Rollout CR with `steps: [{setWeight: 10}, {pause: {}}, {setWeight: 50}, {pause: {}}, {setWeight: 100}]`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Deployment is the standard way to run stateless workloads in Kubernetes. Internally it manages ReplicaSets — one per pod-template revision — which manage the actual pods.
Why: Because Deployments give us declarative rolling updates and rollbacks in one line. `kubectl set image` triggers a new RS to scale up while the old scales down, and if anything goes wrong `kubectl rollout undo` gets us back to the previous revision within seconds.
How: I tune three knobs in production: `maxSurge` and `maxUnavailable` (usually 25% surge, 0% unavailable for zero-downtime), `minReadySeconds` to catch flaky pods, and a `PodDisruptionBudget` to protect availability during node drains. I combine that with startup, readiness, and liveness probes plus a preStop hook so we drain connections cleanly.
Example: Last month a bad config went to prod. Because we had `revisionHistoryLimit: 10` and `minReadySeconds: 20`, only two pods flipped before we detected the error rate spike, and `kubectl rollout undo` recovered us in five seconds — end users never noticed."

? Possible Interview Questions

Basic

Q1. What is a Deployment? A controller that manages ReplicaSets to run stateless pods with declarative updates.
Q2. Which command triggers a rolling update? `kubectl set image`, `kubectl edit`, or `kubectl apply` with a changed template.
Q3. What are `maxSurge` and `maxUnavailable`? Extra pods allowed during update / pods allowed to be down.

Intermediate

Q4. Deployment strategies? `RollingUpdate` (default), `Recreate`.
Q5. How does rollback work? Deployment keeps `revisionHistoryLimit` old ReplicaSets; `kubectl rollout undo` scales the previous RS back up.
Q6. Difference between Deployment and StatefulSet? Deployment: identical, interchangeable pods, no stable identity/storage. StatefulSet: ordered, stable identity, per-pod storage.

Advanced

Q7. Why is the selector immutable after Deployment creation? Mutating selector could orphan or capture pods across revisions unpredictably.
Q8. Can you do blue-green with a plain Deployment? Yes, using two Deployments and switching Service selector; but Argo Rollouts / Flagger handle it more safely.
Q9. What does `kubectl rollout restart deploy/web` actually do? Adds/updates an annotation on the pod template, triggering a new RS with identical image but new pods — used to reload ConfigMaps/Secrets.

Scenario-based

Q10. Rollout stuck; new RS has 0 Ready pods. Steps? `describe` new RS + pods → check probes → check image pull → check ConfigMap/Secret refs → undo or fix.

GKE-specific

Q11. On Autopilot, why might a rollout take longer than on Standard? Autopilot may provision extra nodes to satisfy `maxSurge` pods; startup includes node creation time.

EKS-specific

Q12. On EKS with ALB Ingress, how do you do canary with real weight? AWS Load Balancer Controller supports weighted target groups annotated on Ingress; combine with Argo Rollouts.

Hands-on / Lab-based

Q13. Walk me through a canary rollout you did. Two Deployments behind one Service, initial 90/10 split, monitor error rate via Prometheus, scale canary up progressively, cut over. Or migrate to Argo Rollouts + Analysis Templates.
Q14. How do you force pods to reload a changed ConfigMap? `kubectl rollout restart deploy/web` or use `stakater/Reloader` operator to auto-restart on ConfigMap change.
Q15. How would you verify a rollout succeeded? `kubectl rollout status deploy/web` returns 0; Ready=desired; Available=desired; error rate/latency SLOs still healthy.

🗄 Topic 9 — StatefulSets

Official Documentation Reference - StatefulSets: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/ - StatefulSet basics tutorial: https://kubernetes.io/docs/tutorials/stateful-application/basic-stateful-set/ - Run replicated stateful app: https://kubernetes.io/docs/tasks/run-application/run-replicated-stateful-application/ - Scale a StatefulSet: https://kubernetes.io/docs/tasks/run-application/scale-stateful-set/ - Delete: https://kubernetes.io/docs/tasks/run-application/delete-stateful-set/ - GKE StatefulSets: https://cloud.google.com/kubernetes-engine/docs/how-to/stateful-apps - EKS storage: https://docs.aws.amazon.com/eks/latest/userguide/storage.html

🔹 Definition

A **StatefulSet** manages the deployment and scaling of a set of Pods that require **stable identity**, **ordered creation/deletion**, and **stable persistent storage** — unlike Deployments where pods are interchangeable.

🔹 About

What makes StatefulSets different from Deployments

Property	Deployment	StatefulSet
Pod name	Random hash suffix	`<name>-0, <name>-1, ...` deterministic
Storage	Shared (or none)	One PVC per pod (via `volumeClaimTemplates`)
Creation order	Parallel	Ordered `-0` → `-N`
Deletion order	Parallel	Reverse: `-N` → `-0`
Networking	Random pod DNS via Service	Stable DNS via headless Service: `pod-0.svc.ns.svc.cluster.local`
Update strategy	RollingUpdate / Recreate	RollingUpdate (default) / OnDelete
Use case	Web, API, stateless workers	Databases, message queues, distributed systems

Headless Service

`spec.clusterIP: None`. Instead of a virtual IP, DNS returns A records for each pod. StatefulSets rely on this for stable pod DNS names.

volumeClaimTemplates

Each replica gets its own PersistentVolumeClaim generated from the template — `data-<sts>-0`, `data-<sts>-1`, etc. Deleting the StatefulSet does **not** delete PVCs by default; use `persistentVolumeClaimRetentionPolicy` (K8s 1.27+).

Update strategies

- `RollingUpdate` (default) — updates in reverse ordinal order; each pod must become Ready before next.
- `OnDelete` — you delete pods manually; StatefulSet recreates them with the new template.
- `partition` — freezes ordinal < partition (used for staged canaries: only pods with ordinal ≥ partition are updated).

Pod management policy

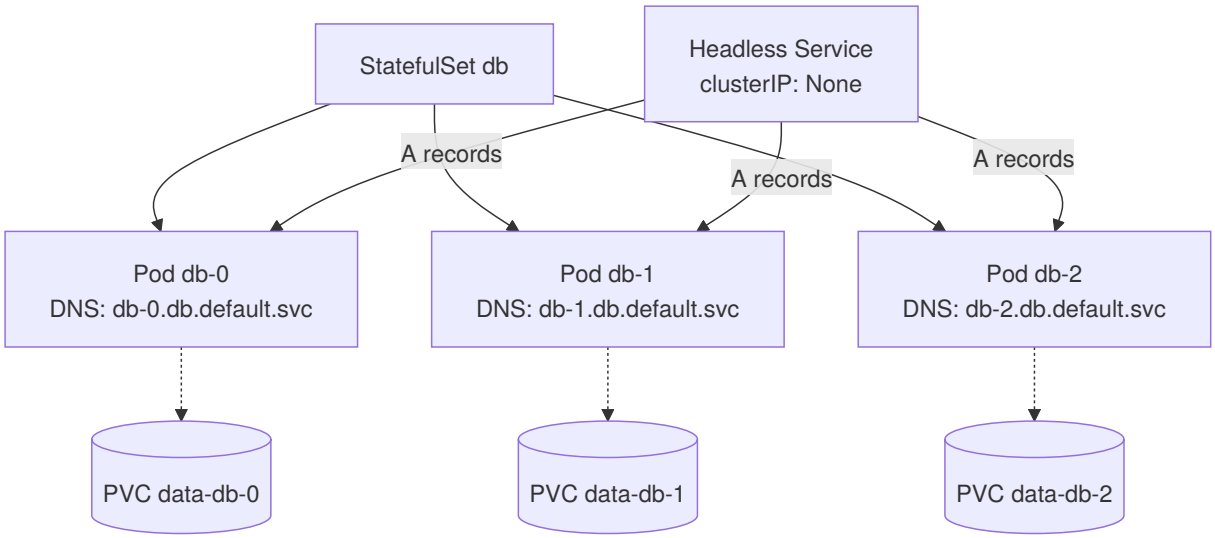
- `OrderedReady` (default) — sequential create/scale.
- `Parallel` — create/scale in parallel; still stable identity.

🔹 How It Integrates With Other Kubernetes Components

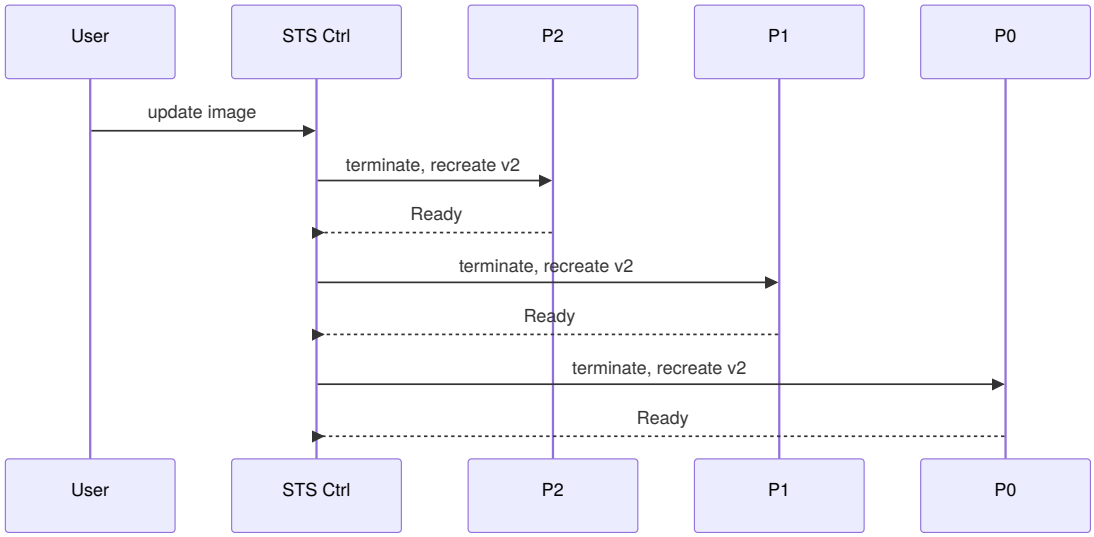
Component	Interaction
Headless Service	Provides stable per-pod DNS
PVC + StorageClass	Dynamic per-pod storage
CSI driver	Provisions actual PD/EBS/EFS/Filestore volume
Scheduler	Places pod on a node with the right zone (topology-aware for zonal disks)
PodDisruptionBudget	Protect during drains
Deployment	Not related — StatefulSet is a peer controller

🏗 Diagrams

Stable identity



Ordered rollout



💻 Real-World Examples

Example 1 — PostgreSQL primary + replicas

```yaml
apiVersion: v1
kind: Service
metadata: { name: pg }
spec:
  clusterIP: None
  selector: { app: pg }
  ports: [{ port: 5432 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: pg }
spec:
  serviceName: pg
  replicas: 3
  selector: { matchLabels: { app: pg } }
  template:
    metadata: { labels: { app: pg } }
    spec:
      terminationGracePeriodSeconds: 60
      containers:
      - name: pg
        image: postgres:16
        ports: [{ containerPort: 5432, name: pg }]
        env:
        - { name: POSTGRES_PASSWORD, valueFrom: { secretKeyRef: { name: pg-secret, key: password } } }
        - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
        volumeMounts:
        - { name: data, mountPath: /var/lib/postgresql/data }
        readinessProbe:
          exec: { command: ["pg_isready","-U","postgres"] }
          periodSeconds: 5
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: ["ReadWriteOnce"]
      resources: { requests: { storage: 10Gi } }
      storageClassName: standard-rwo
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain
    whenScaled:  Retain
```

Example 2 — Parallel pod management for a faster cluster of caches

```yaml
spec:
  podManagementPolicy: Parallel
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: { partition: 2 }   # only pods ordinal >= 2 are updated
```

Example 3 — Reach a specific pod by DNS

```bash
kubectl exec -it pg-1 -- ping pg-0.pg.default.svc.cluster.local
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get sts`	`-o wide`	List StatefulSets	`kubectl get sts`
`kubectl scale sts`	`--replicas=N`	Scale (orderly)	`kubectl scale sts pg --replicas=5`
`kubectl rollout status sts`	—	Watch rollout	`kubectl rollout status sts/pg`
`kubectl rollout undo sts`	`--to-revision=N`	Rollback	`kubectl rollout undo sts/pg`
`kubectl rollout restart sts`	—	Recycle pods	`kubectl rollout restart sts/pg`
`kubectl delete sts`	`--cascade=orphan`	Keep pods	`kubectl delete sts pg --cascade=orphan`
`kubectl get pvc`	`-l app=pg`	List per-pod PVCs	`kubectl get pvc -l app=pg`
`kubectl patch sts`	`--type='json' -p=...`	Change partition	`kubectl patch sts pg --type=json -p='[{"op":"replace","path":"/spec/updateStrategy/rollingUpdate/partition","value":2}]'`

🚨 Scenarios

Scenario 1 — Pod `pg-1` stuck Pending while `pg-2` was requested

- **Cause**: `podManagementPolicy: OrderedReady` blocks creation of higher ordinal until previous is Ready.
- **Fix**: Fix `pg-1`, or switch to `Parallel` if ordering isn't required.

Scenario 2 — Deleted the StatefulSet and lost data

- **Cause**: `persistentVolumeClaimRetentionPolicy.whenDeleted: Delete` set.

- **Fix**: Always `Retain` for stateful data; take snapshots regularly.

Scenario 3 — StatefulSet upgrade only partially rolled out

- **Cause**: `updateStrategy.rollingUpdate.partition: N` freezes ordinals below N.
- **Fix**: Lower `partition` to 0 gradually (staged canary).

Scenario 4 — Pod `db-2` scheduled to wrong zone; can't attach EBS

- **Cause**: EBS volume is zonal; scheduler placed pod in different AZ than volume.
- **Fix**: `volumeBindingMode: WaitForFirstConsumer` on StorageClass so the volume is created in the pod's zone.

Scenario 5 — Headless Service DNS returns nothing

- **Cause**: `serviceName` in STS doesn't match the Service name, or Service selector doesn't match pod labels.
- **Fix**: align both.

☁ GCP GKE Perspective

- **Regional Persistent Disks (regional-pd)** — synchronous replication across two zones; good for StatefulSet HA. `storageClassName: regional-pd` in some setups; typically use `pd-ssd` with topology constraints.
- **Filestore CSI** — ReadWriteMany, useful when app requires shared filesystem (rare for STS but useful for stateful workloads).
- **Backup for GKE** can snapshot PVs + StatefulSet manifests.
- **Autopilot** — StatefulSets supported; ensure StorageClass is `standard-rwo` (default managed) or `premium-rwo`.

▩ AWS EKS Perspective

- **EBS gp3** — most common per-pod volume; zonal.
- **EFS** — ReadWriteMany, cross-AZ; slower for typical DBs but works for shared workloads.
- **EBS CSI driver** must be installed as an add-on: `aws eks create-addon --cluster-name prod --addon-name aws-ebs-csi-driver`.
- Multi-AZ StatefulSet: use `topologySpreadConstraints` + `WaitForFirstConsumer` StorageClass to keep pods and volumes co-located.
- **Fargate** cannot use EBS; only EFS works — so most databases don't run on Fargate.

🧪 Hands-On Lab Exercises

Lab 9.1 — StatefulSet basics (Beginner, 30 min)

Steps

```
# sts.yaml
apiVersion: v1
kind: Service
metadata: { name: nginx-sts }
spec:
  clusterIP: None
  selector: { app: web }
  ports: [{ port: 80 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: web }
spec:
  serviceName: nginx-sts
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        volumeMounts: [{ name: data, mountPath: /usr/share/nginx/html }]
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: ["ReadWriteOnce"]
      resources: { requests: { storage: 1Gi } }
```

```
kubectl apply -f sts.yaml
kubectl get pods -l app=web -w    # web-0 → Ready → web-1 → Ready → web-2
kubectl get pvc                   # data-web-0/1/2
kubectl exec -it web-0 -- sh -c 'echo hello-0 > /usr/share/nginx/html/index.html'
kubectl delete pod web-0
kubectl exec -it web-0 -- cat /usr/share/nginx/html/index.html   # still hello-0
```

Cleanup: `kubectl delete sts web && kubectl delete pvc -l app=web && kubectl delete svc nginx-sts`.

Lab 9.2 — Partition-based canary (Intermediate, 45 min)

Steps 1. Deploy STS with 5 replicas, `partition: 0`, image `v1`. 2. Set `partition: 4` and image `v2`. Only `web-4` updates. 3. Verify: `kubectl describe pod web-4 | grep Image`. 4. Lower `partition: 2` → web-4, web-3, web-2 update. 5. Lower `partition: 0` → all updated.

Bonus: Use `updateStrategy.rollingUpdate.maxUnavailable: 2` (K8s 1.27+) to parallelize rollout while preserving ordinals.

Lab 9.3 — Multi-AZ Postgres StatefulSet on EKS with EBS (Advanced, 90 min)

Steps 1. Ensure EBS CSI add-on installed. 2. Create StorageClass:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: ebs-wait }
provisioner: ebs.csi.aws.com
parameters: { type: gp3 }
volumeBindingMode: WaitForFirstConsumer
```

3. Apply Postgres STS from Example 1 with `storageClassName: ebs-wait`. 4. Add `topologySpreadConstraints` to spread pods across 3 AZs. 5. `kubectl get pods -o custom-columns=NAME:.metadata.name,NODE:.spec.nodeName,ZONE:.metadata.annotations."topology\.kubernetes\.io/zone"`. 6. Delete pod `pg-1`; observe reschedule to same AZ (because PVC pins zone). 7. Trigger backup with Velero: `velero backup create pg-snap --include-namespaces=default --snapshot-volumes`.

Cleanup: `eksctl delete cluster`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A StatefulSet is Kubernetes' primitive for stateful workloads. Unlike a Deployment, its pods have stable ordinal names, stable DNS via a headless Service, and their own PersistentVolumeClaims created from a volumeClaimTemplate.
Why: Databases and distributed systems need identity. A PostgreSQL replica pod that becomes `pg-1` today must come back as `pg-1` after a restart, on the same data. Deployments can't guarantee that.
How: I pair a StatefulSet with a headless Service, per-pod PVCs on a StorageClass with `WaitForFirstConsumer`, and topology spread constraints so each pod lands in a different zone but with its zonal volume. Rollouts happen in reverse ordinal order — I sometimes stage them with `partition`. I set `persistentVolumeClaimRetentionPolicy: Retain` so accidental deletes don't drop data, and I combine with PDBs and regular Velero or Backup-for-GKE snapshots.
Example: We ran a 3-node Kafka on EKS as a StatefulSet with EBS gp3, spread across three AZs. When we upgraded Kafka to a new minor version, we set `partition: 2` first, watched the canary pod for an hour, then lowered `partition` to zero — no message loss, no downtime."

? Possible Interview Questions

Basic

Q1. Deployment vs StatefulSet? Deployment: interchangeable pods. StatefulSet: stable identity, stable storage, ordered lifecycle.
Q2. Do StatefulSet pods keep their name after restart? Yes: `web-0` will always be `web-0`.
Q3. What is a headless Service? A Service with `clusterIP: None`; DNS returns per-pod A records instead of a virtual IP.

Intermediate

Q4. What is `volumeClaimTemplates`? A template that generates a PVC per pod (e.g., `data-web-0`).
Q5. What is `podManagementPolicy: Parallel`? Skips ordered create/scale — still gives stable identity.
Q6. How do you scale down a StatefulSet safely? Ensure app supports it (drain leader, rebalance), then `kubectl scale sts <n> --replicas=X`. Pods terminate in reverse ordinal order.

Advanced

Q7. What is `partition` in updateStrategy? Ordinals `< partition` are frozen; only `>= partition` update. Used for staged canaries.
Q8. What happens to PVCs on delete? Retained by default; use `persistentVolumeClaimRetentionPolicy` (K8s 1.27+) to auto-delete on scale-down or STS delete.
Q9. How do you handle zonal storage in multi-AZ? `WaitForFirstConsumer` StorageClass so PV is created in the same zone the pod is scheduled to.

Scenario-based

Q10. STS pod stuck attaching volume after node crash — likely cause? Volume still attached to old node; needs force detach or wait for kubelet cleanup. Check `volumeattachments` and node status.

GKE-specific

Q11. What's the fastest way to get multi-zone durability for a StatefulSet on GKE? Use regional PDs (synchronous cross-zone replication) or app-level replication (e.g., Postgres streaming) across zonal PDs.

EKS-specific

Q12. Can you run a StatefulSet on Fargate? With limits: EBS not supported; only EFS (RWX) — most databases require RWO block storage, so managed node groups are the norm.

Hands-on / Lab-based

Q13. Show me how you'd do a canary rollout of a StatefulSet. Set `partition: N-1`; only the last pod updates. Validate. Lower `partition` in steps until 0.
Q14. What happens if I delete pod `db-0`? Same PVC re-attaches to the new `db-0` pod; data preserved; DNS name preserved.
Q15. How would you back up a database StatefulSet? Two layers: app-level (pg_dump, WAL to S3/GCS) + volume snapshots (CSI VolumeSnapshot / Velero / Backup for GKE) + off-cluster storage.

📄 Topic 10 — DaemonSets

Official Documentation Reference - DaemonSet: https://kubernetes.io/docs/concepts/workloads/controllers/daemonset/ - Basic DaemonSet task: https://kubernetes.io/docs/tasks/manage-daemon/create-daemon-set/ - Rolling update: https://kubernetes.io/docs/tasks/manage-daemon/update-daemon-set/ - Rollback: https://kubernetes.io/docs/tasks/manage-daemon/rollback-daemon-set/ - Running pods on some nodes: https://kubernetes.io/docs/tasks/manage-

daemon/pods-some-nodes/ - GKE DaemonSets: https://cloud.google.com/kubernetes-engine/docs/how-to/deploying-workloads-overview - EKS DaemonSets & VPC CNI: https://docs.aws.amazon.com/eks/latest/userguide/managing-vpc-cni.html

🔹 Definition

A **DaemonSet** ensures that a copy of a Pod runs on **every (matching) node** in the cluster. As nodes join, DaemonSet pods appear; as nodes leave, they're garbage-collected.

🔹 About

Typical uses

- **Node-level agents**: log shippers (Fluent Bit, Fluentd, Filebeat), metrics agents (node-exporter, Datadog, New Relic).
- **CNI plugins**: Calico, Cilium, kindnet, VPC CNI — deployed as DaemonSets.
- **CSI node components**: driver-registrar sidecars, node-plugin daemons.
- **Storage / networking daemons**: kube-proxy (on non-eBPF clusters), MetalLB speakers, ingress-controller in `hostNetwork` mode.
- **Security agents**: Falco, Sysdig, Aqua.

Scheduling model

- The DaemonSet controller uses the default scheduler (since K8s 1.17). Each pod carries a `nodeAffinity` targeting a single node.
- Respects **nodeSelector**, **affinity**, **taints/tolerations**.
- Special toleration `op: Exists` on `node.kubernetes.io/*` lets DS survive NotReady/unschedulable — hence log agents keep running on failing nodes.

Update strategies

- `RollingUpdate` (default) — updates one node at a time.
- `OnDelete` — you delete pods manually; DS recreates with new template.

Not on managed nodes?

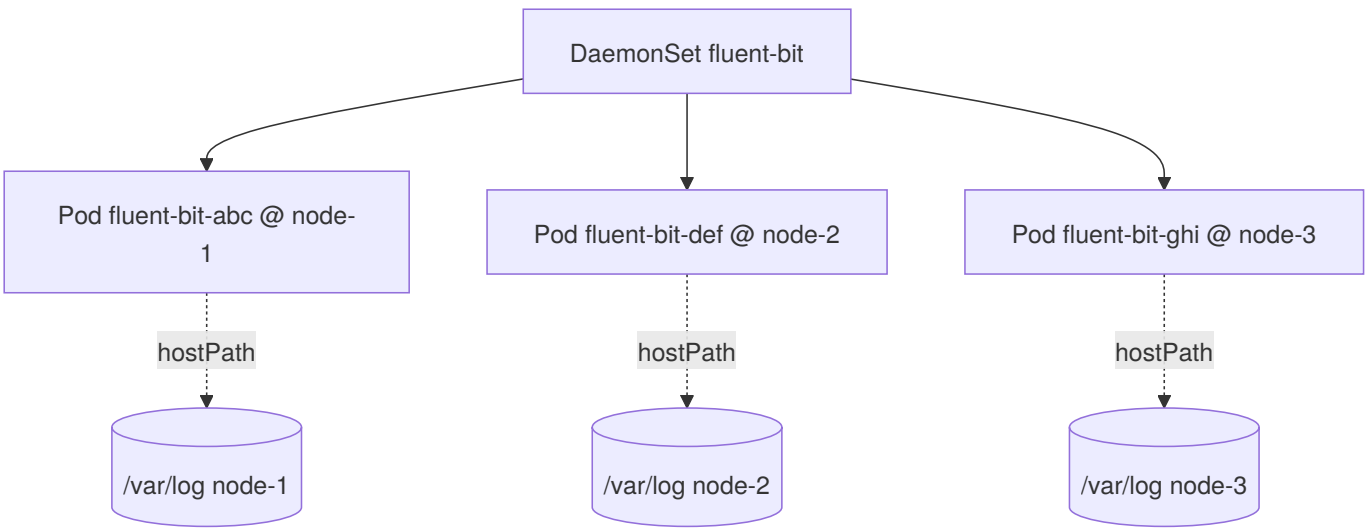
- **GKE Autopilot** blocks DaemonSets **unless** they belong to Google's allowlist (e.g., Datadog, GKE Metrics Agent).
- **EKS Fargate** does not support DaemonSets — Fargate nodes are per-pod microVMs; instead run the agent as a sidecar.

🔹 How It Integrates With Other Kubernetes Components

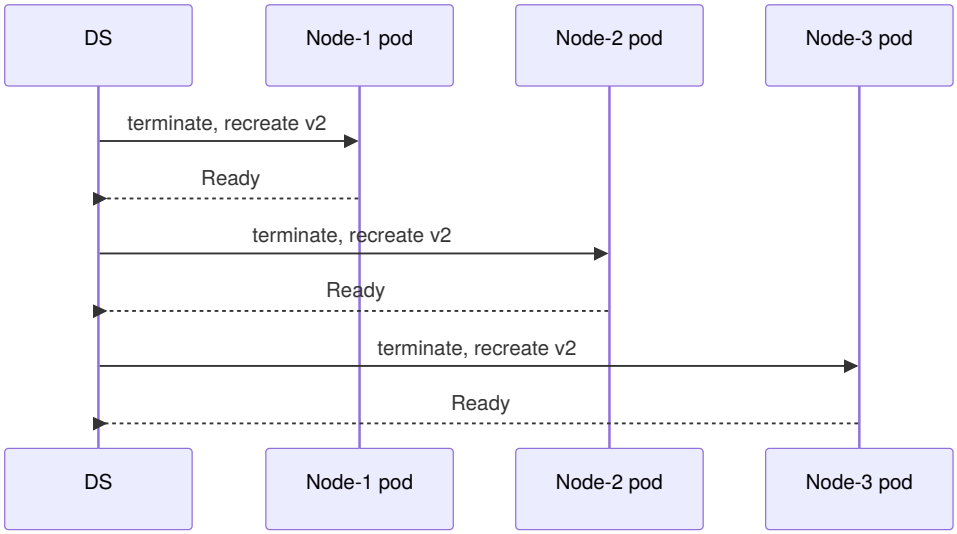
Component	Interaction
Node	DS runs one pod per matching node
Scheduler	Assigns DS pod to node via nodeAffinity
Taints/Tolerations	Control which nodes get DS pods
CNI	Often *itself* a DaemonSet (Calico, Cilium)
CSI	Node component runs as DS
Logging/monitoring stack	The heart of "one agent per node" pattern

🏗 Diagrams

One pod per node



Rolling update



💻 Real-World Examples

Example 1 — Fluent Bit log shipper

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector: { matchLabels: { app: fluent-bit } }
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: { maxUnavailable: 1 }
  template:
    metadata: { labels: { app: fluent-bit } }
    spec:
      serviceAccountName: fluent-bit
      tolerations:
      - operator: Exists                # run on every node, including tainted
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:3.1
        resources:
          requests: { cpu: 50m, memory: 64Mi }
          limits:   { cpu: 200m, memory: 200Mi }
        volumeMounts:
        - { name: varlog, mountPath: /var/log, readOnly: true }
        - { name: varlibdocker, mountPath: /var/lib/docker/containers, readOnly: true }
        - { name: config, mountPath: /fluent-bit/etc }
      volumes:
      - { name: varlog, hostPath: { path: /var/log } }
      - { name: varlibdocker, hostPath: { path: /var/lib/docker/containers } }
      - { name: config, configMap: { name: fluent-bit-config } }
```

Example 2 — Run only on GPU nodes

```
spec:
  template:
    spec:
      nodeSelector: { accelerator: nvidia }
      tolerations:
      - { key: nvidia.com/gpu, operator: Exists, effect: NoSchedule }
      containers:
      - { name: gpu-agent, image: nvidia/gpu-operator:1.0 }
```

Example 3 — Prometheus node-exporter

```
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: node-exporter, namespace: monitoring }
spec:
  selector: { matchLabels: { app: node-exporter } }
  template:
    metadata: { labels: { app: node-exporter } }
    spec:
      hostNetwork: true
      hostPID: true
      containers:
      - name: ne
        image: prom/node-exporter:v1.8.2
        args: ["--path.rootfs=/host", "--web.listen-address=0.0.0.0:9100"]
        ports: [{ containerPort: 9100, hostPort: 9100 }]
        volumeMounts: [{ name: root, mountPath: /host, readOnly: true }]
      volumes: [{ name: root, hostPath: { path: / } }]
      tolerations: [{ operator: Exists }]
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get ds`	`-A`	List DaemonSets everywhere	`kubectl get ds -A`
`kubectl describe ds`	`<name>`	See per-node status	`kubectl describe ds fluent-bit -n logging`
`kubectl rollout status ds`	—	Watch rolling update	`kubectl rollout status ds/fluent-bit -n logging`
`kubectl rollout undo ds`	`--to-revision=N`	Rollback	`kubectl rollout undo ds/fluent-bit -n logging`
`kubectl rollout restart ds`	—	Recycle pods	`kubectl rollout restart ds/fluent-bit -n logging`
`kubectl set image ds`	`<c>=<img>`	Trigger update	`kubectl set image ds/fluent-bit fluent-bit=fluent/fluent-bit:3.2 -n logging`
`kubectl label node`	`key=val`	Change which nodes match nodeSelector	`kubectl label node gpu-1 accelerator=nvidia`
`kubectl taint node`	`key=val:eff`	Add taint (DS needs matching toleration)	`kubectl taint node infra-1 dedicated=infra:NoSchedule`

🚨 Scenarios

Scenario 1 — DaemonSet pods missing on a new node

- **Cause**: node has taint the DS pod doesn't tolerate; or nodeSelector doesn't match.
- **Diagnose**: `kubectl describe node <n>` (taints/labels), `kubectl describe ds <n>` (desired vs current).
- **Fix**: add toleration or label the node.

Scenario 2 — Update rolled out but old pods remain on a NotReady node

- **Cause**: DS rolling update skips unavailable nodes.
- **Fix**: fix the node; DS will reconcile when it becomes Ready.

Scenario 3 — Log shipper causes noisy neighbor (100% CPU)

- **Cause**: no resource limits; parsing spike.
- **Fix**: set `resources.limits` + priorityClass so it can't starve app pods.

Scenario 4 — Agent needs privileged mode → Autopilot rejects

- **Cause**: Autopilot forbids privileged.
- **Fix**: use the vendor's Autopilot-compatible variant, or move to Standard.

Scenario 5 — DaemonSet not covered by Cluster Autoscaler

- **Cause**: Cluster Autoscaler ignores DS pods when scaling nodes.
- **Fix**: this is desirable — DS pods "come for free" on new nodes; watch node ceilings for total agent overhead.

☁ GCP GKE Perspective

- **Autopilot**: DaemonSets are restricted to a Google allowlist (Datadog, New Relic, Dynatrace, Splunk, etc.). Your own DS is generally rejected — build agents as sidecars instead.
- **Standard**: full DaemonSet freedom. Common defaults installed: `fluentbit-gke`, `gke-metrics-agent`, `gke-metadata-server`, `netd`, `gke-node-problem-detector`.
- Check with: `kubectl get ds -n kube-system`.

▩ AWS EKS Perspective

- **Managed NG**: full DS support. AWS ships several system DaemonSets: `aws-node` (VPC CNI), `kube-proxy`, `coredns` (Deployment). `aws-node` scaling and IP counts affect pod density.
- **Fargate**: DaemonSets NOT supported. Run monitoring/log agents as sidecars in every pod, or use CloudWatch Container Insights via built-in log driver.
- Common third-party DaemonSets: `cluster-autoscaler` (not DS), `aws-load-balancer-controller` (Deployment), `metrics-server` (Deployment), `node-exporter` (DS), Datadog Agent (DS).

🧪 Hands-On Lab Exercises

Lab 10.1 — Simple DaemonSet on Kind (Beginner, 20 min)

Steps

```
# ds.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: ping }
spec:
  selector: { matchLabels: { app: ping } }
  template:
    metadata: { labels: { app: ping } }
    spec:
      tolerations: [{ operator: Exists }]
      containers:
      - name: c
        image: busybox:1.36
        command: ["sh","-c","while true; do echo alive; sleep 30; done"]
```

```
kind create cluster --config kind-3.yaml
kubectl apply -f ds.yaml
kubectl get ds ping        # DESIRED=3, CURRENT=3, READY=3
kubectl get pods -o wide -l app=ping
```

Cleanup: `kubectl delete ds ping && kind delete cluster`.

Lab 10.2 — Node selector + taints (Intermediate, 40 min)

Steps 1. Label one worker: `kubectl label node <n> role=edge`. 2. Taint another: `kubectl taint node <n> dedicated=infra:NoSchedule`. 3. Update DS template to `nodeSelector: {role: edge}` and add tolerations only for `role=edge` nodes. 4. Verify: DS pod runs only on the `role=edge` node. 5. Add another label to the tainted node with `role=edge`; verify DS still won't schedule there without toleration for `dedicated=infra`. 6. Add matching toleration; verify pod appears.

Bonus: Use `preferredDuringSchedulingIgnoredDuringExecution` to prefer but not require.

Lab 10.3 — Fluent Bit log shipping (Advanced, 60 min)

Objective: real end-to-end log flow.

Prerequisites: any cluster with 2+ workers; ELK, Loki, or `stdout` sink.

Steps 1. Deploy Fluent Bit DS from Example 1. 2. Apply a ConfigMap `fluent-bit-config` with an input reading `/var/log/containers/*.log` and output `stdout`. 3. `kubectl -n logging logs -l app=fluent-bit --tail=20` — see aggregated logs. 4. Add Loki output; verify logs appear in Grafana. 5. Trigger rollout: change parser; `kubectl rollout status ds/fluent-bit -n logging`.

Cleanup: `kubectl delete ns logging`.

Bonus: Set `updateStrategy.rollingUpdate.maxUnavailable: 30%` and observe faster rollout across many nodes.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A DaemonSet is a controller that runs exactly one pod on every node — or every matching node — of a cluster. It's how we ship node-level agents like log shippers, metrics collectors, CNI and CSI drivers.
Why: These agents need to be on the node itself, usually with hostPath volumes to read `/var/log` or hostNetwork to expose metrics. A Deployment can't guarantee 'one on each node' — a DaemonSet does.
How: I define the DaemonSet with wide tolerations so it runs even on tainted nodes, tight resource requests/limits so it can't starve app pods, and a nodeSelector when the agent should only run on a subset (like GPU nodes). Rolling updates go one node at a time by default.
Managed nuance: On GKE Autopilot only allowlisted DaemonSets are permitted, and on EKS Fargate DaemonSets aren't supported at all — so for those environments I turn the agent into a sidecar or use vendor-provided cloud-native integrations.
Example: We standardized on Fluent Bit DaemonSet across 200 nodes on EKS with resource limits 200m/200Mi — total overhead was 40 vCPU and 40 GB for the whole log pipeline, and rollouts of parser changes finished in under two minutes."

? Possible Interview Questions

Basic

Q1. What is a DaemonSet? Controller that runs one pod per node.
Q2. Give 3 use cases. Log shipper, metrics agent, CNI plugin (or storage node plugin).
Q3. Does a DS need a Service? Usually no — the agent talks to the node's kernel or filesystem; if it exposes an endpoint, use hostPort or a headless Service.

Intermediate

Q4. What happens when a new node joins? DS controller creates a pod on it automatically.
Q5. How does DS handle tainted nodes? Only runs there if it has a matching toleration; commonly uses `operator: Exists` to run everywhere.
Q6. DS update strategies? `RollingUpdate` (default) and `OnDelete`.

Advanced

Q7. Does the scheduler place DS pods? Yes (since 1.17). The DS controller writes a nodeAffinity pointing to one specific node; the scheduler binds.
Q8. Can DS scale? Not by replica count — by the number of matching nodes. Change nodeSelector or add/remove nodes.
Q9. How does DS interact with Cluster Autoscaler? CA ignores DS pods when calculating scale-up need; DS pods appear automatically on new nodes.

Scenario-based

Q10. Node has 100 pods; log shipper drops messages. Steps? Increase agent's `resources.limits`, tune buffer/flush parameters, sample high-noise namespaces, run per-namespace collectors instead of one giant DS.

GKE-specific

Q11. Why does Autopilot reject my new DS? Autopilot restricts DaemonSets to a Google allowlist. Use an approved vendor variant or move to Standard.

EKS-specific

Q12. Fargate + logging — how? DaemonSets aren't allowed. Enable Fluent Bit sidecar per pod or use CloudWatch Container Insights (Fargate emits logs directly to CloudWatch).

Hands-on / Lab-based

Q13. Show me a DS that runs only on GPU nodes. `nodeSelector: {accelerator: nvidia}` + toleration for `nvidia.com/gpu: NoSchedule`.
Q14. How do you make a DS survive a node under memory pressure? Set `priorityClassName: system-node-critical` and modest resource requests; kubelet won't evict critical pods first.
Q15. How do you roll back a DS? `kubectl rollout undo ds/<name>` — works exactly like Deployment rollback; templates are versioned as ControllerRevisions.

🗐 Topic 11 — Jobs and CronJobs

Official Documentation Reference - Job: https://kubernetes.io/docs/concepts/workloads/controllers/job/ - CronJob: https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/ - Automated tasks with CronJob: https://kubernetes.io/docs/tasks/job/automated-tasks-with-cron-jobs/ - TTL cleanup for finished jobs: https://kubernetes.io/docs/concepts/workloads/controllers/ttlafterfinished/ - Pod failure policy: https://kubernetes.io/docs/tasks/job/pod-failure-policy/ - GKE batch: https://cloud.google.com/kubernetes-engine/docs/concepts/batch - EKS batch (Karpenter/Batch on EKS): https://docs.aws.amazon.com/eks/latest/userguide/batch.html

🔹 Definition

- **Job** — a workload that runs one or more pods to **completion** (success). Perfect for batch tasks, migrations, one-off scripts.
- **CronJob** — a Job scheduled by cron expression. Perfect for periodic tasks: backups, report generation, retention cleanup.

🔹 About

Job spec fields

Field	Meaning
`completions`	Total successful pod completions required
`parallelism`	How many pods run concurrently
`backoffLimit`	Retries before marking job Failed (default 6)
`activeDeadlineSeconds`	Hard wall-clock timeout
`ttlSecondsAfterFinished`	Auto-delete finished Job (and its pods) after N seconds
`completionMode`	`NonIndexed` (default) or `Indexed` (workers get `JOB_COMPLETION_INDEX`)
`podFailurePolicy`	Rules to fast-fail or ignore specific exit codes (K8s 1.26+ stable)
`suspend`	true/false — pause a Job

Job patterns

- **Non-parallel** — one pod, one completion.
- **Parallel with fixed completions** — worker pool doing N units.
- **Parallel with work queue** — one pod pulls from a queue until empty; `completions: 1` with `parallelism: N`.
- **Indexed** — each pod knows its index (e.g., `JOB_COMPLETION_INDEX=3`) for static work assignment.

CronJob spec

Field	Meaning
`schedule`	Cron expression (`"*/5 * * * *"`) — **UTC by default**; K8s 1.27+ supports `timeZone: "Asia/Kolkata"`
`concurrencyPolicy`	`Allow` (default), `Forbid` , `Replace`
`successfulJobsHistoryLimit` / `failedJobsHistoryLimit`	Retain last N Jobs
`startingDeadlineSeconds`	Skip missed schedules older than this
`suspend`	Pause future runs

CronJob quirks

- **Missed schedules**: if the controller was down and > 100 schedules were missed, CronJob stops trying (protective).
- **Idempotency**: schedule can double-fire (rare); jobs must be idempotent.
- **Time zones**: use `timeZone` field (K8s 1.27+) rather than relying on cluster TZ.

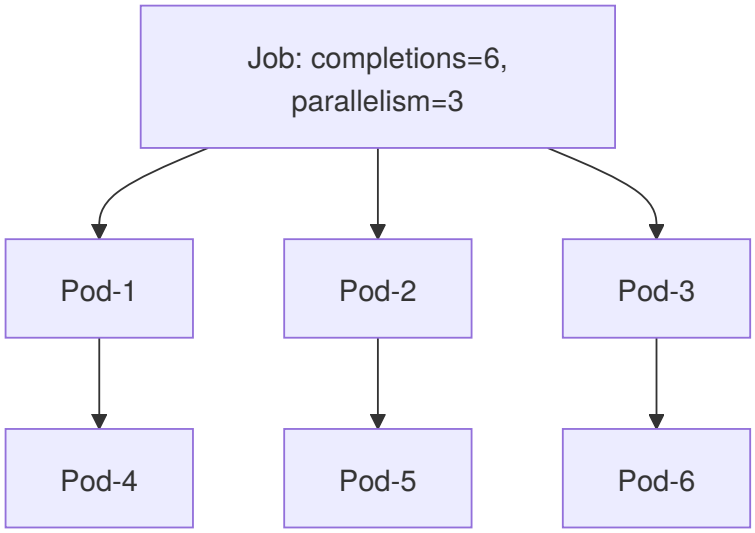
🔷 How It Integrates With Other Kubernetes Components

Component	Interaction
Pod	Job creates Pods with `restartPolicy: OnFailure` or `Never`
ReplicaSet/Deployment	Unrelated (different controller)
Node autoscaler	Scales up to run parallel jobs; scales down after `ttlSecondsAfterFinished`
Event	Job posts events on success/failure
Admission (LimitRange, ResourceQuota)	Enforced per pod including Job pods

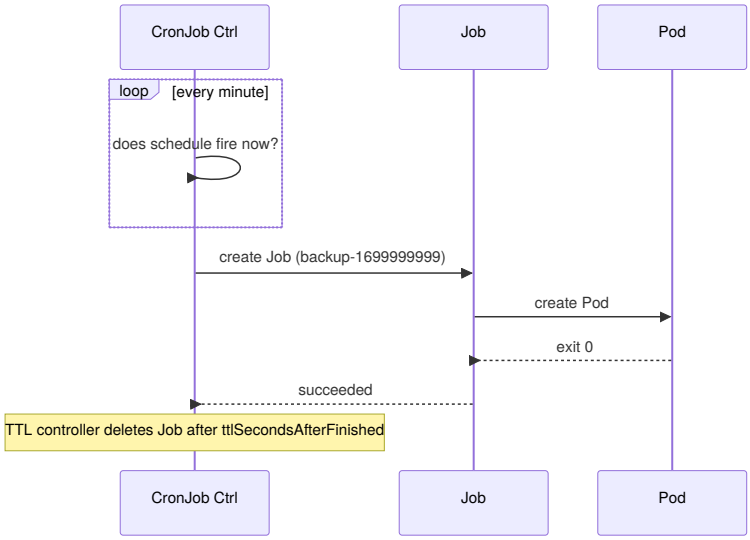
---

🏗 Diagrams

Job with parallelism



CronJob lifecycle



💻 Real-World Examples

Example 1 — One-shot migration Job

```
apiVersion: batch/v1
kind: Job
metadata: { name: db-migrate }
spec:
  backoffLimit: 2
  ttlSecondsAfterFinished: 3600
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: mig
        image: myrepo/migrate:1.4.2
        command: ["migrate", "-database", "$(DB_URL)", "up"]
        envFrom: [{ secretRef: { name: db-url } }]
```

Example 2 — Indexed parallel Job (batch ML inference)

```
apiVersion: batch/v1
kind: Job
metadata: { name: infer }
spec:
  completions: 20
  parallelism: 5
  completionMode: Indexed
  ttlSecondsAfterFinished: 600
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: worker
        image: myrepo/inference:2.1
        env:
        - name: SHARD
          valueFrom: { fieldRef: { fieldPath: metadata.annotations['batch.kubernetes.io/job-completion-index'] } }
        command: ["python","run.py","--shard","$(SHARD)","--of","20"]
```

Example 3 — CronJob with timezone

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: nightly-backup }
spec:
  schedule: "0 2 * * *"            # 2 AM
  timeZone: "Asia/Kolkata"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 1
      ttlSecondsAfterFinished: 86400
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: dump
            image: myrepo/pg-backup:1.0
            envFrom: [{ secretRef: { name: pg-secret } }]
            command: ["/usr/local/bin/backup.sh"]
```

Example 4 — Fast-fail on config error (podFailurePolicy)

```
spec:
  backoffLimit: 6
  podFailurePolicy:
    rules:
    - action: FailJob               # bail immediately
      onExitCodes: { operator: In, values: [42] }
    - action: Ignore                # infrastructure fault
      onPodConditions:
      - { type: DisruptionTarget }
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create job`	`--image`	Ad-hoc Job	`kubectl create job hello --image=busybox -- echo hi`
`kubectl create job`	`--from=cronjob/<n>`	Run a CronJob now	`kubectl create job manual --from=cronjob/nightly-backup`
`kubectl get jobs`	`-A`	List Jobs	`kubectl get jobs -A`
`kubectl describe job`	—	See conditions	`kubectl describe job db-migrate`
`kubectl logs job/<n>`	`--tail=100`	Aggregate pod logs of Job	`kubectl logs job/db-migrate`
`kubectl delete job`	`--cascade=orphan`	Keep pods	`kubectl delete job db-migrate --cascade=orphan`
`kubectl get cronjob`	—	Active/last schedule	`kubectl get cronjob`
`kubectl patch cj`	`-p '{"spec":{"suspend":true}}'`	Pause	`kubectl patch cj nightly-backup -p '{"spec":{"suspend":true}}'`
`kubectl create job`	`--dry-run=client -o yaml`	Skeleton	`kubectl create job x --image=busybox --dry-run=client -o yaml`

🚨 Scenarios

Scenario 1 — CronJob missed schedules

- **Cause**: controller manager down > 100 schedules; or `startingDeadlineSeconds` too small.
- **Fix**: raise `startingDeadlineSeconds`; investigate controller-manager availability; adopt CronJob v1 (batch/v1).

Scenario 2 — Migration Job re-runs on retry, breaks schema

- **Cause**: not idempotent; `backoffLimit` allows retries.
- **Fix**: Use idempotent migrations (e.g., `golang-migrate` versioned), `backoffLimit: 0` for destructive tasks, capture "already applied" errors with `podFailurePolicy: Ignore`.

Scenario 3 — Finished Jobs pile up

- **Cause**: no `ttlSecondsAfterFinished`; `successfulJobsHistoryLimit` too high.
- **Fix**: set both.

Scenario 4 — Parallel Job wastes work when one pod slow

- **Cause**: `completions=100, parallelism=10` with static work assignment.
- **Fix**: Use work-queue pattern (Redis/SQS/PubSub) and set `completions: 1, parallelism: 10`, workers exit when queue is empty.

Scenario 5 — CronJob runs twice at DST rollover

- **Cause**: naive UTC schedule + local intent.
- **Fix**: Use `timeZone` field; keep schedule at UTC hour or use `0 */6 * * *` style intervals.

☁ GCP GKE Perspective

- **Autopilot** supports Jobs and CronJobs directly; pods bill per-second while running.
- **GKE Batch (Batch API)** — Google's managed batch runtime that consumes GKE for underlying compute; useful for HPC.
- **Cloud Scheduler → Cloud Run / GKE** — Alternative for scheduled tasks outside cluster.
- Node auto-provisioning scales up for big batch jobs; add `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` on critical Job pods.

▩ AWS EKS Perspective

- **AWS Batch on EKS** — managed queue + priority + array jobs; runs pods on your EKS cluster.
- **Karpenter** shines here — provisions right-sized Spot nodes just to run a batch of Jobs, terminates when done.
- **Fargate** — Jobs work, but each pod = a microVM start-up latency (~30–60 s). For high-throughput short jobs, prefer managed NG.
- **EventBridge Scheduler** — cluster-external alternative to CronJob.

🧪 Hands-On Lab Exercises

Lab 11.1 — Your first Job and CronJob (Beginner, 30 min)

Steps

```
# Job
kubectl create job hello --image=busybox -- /bin/sh -c 'echo hello; sleep 3'
kubectl get pods -l job-name=hello
kubectl logs job/hello

# CronJob every minute
cat <<'EOF' | kubectl apply -f -
apiVersion: batch/v1
kind: CronJob
metadata: { name: heartbeat }
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      ttlSecondsAfterFinished: 60
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - { name: c, image: busybox, command: ["sh","-c","date; echo alive"] }
EOF
kubectl get cronjob heartbeat -w
```

Cleanup: `kubectl delete job hello && kubectl delete cronjob heartbeat`.

Lab 11.2 — Parallel indexed Job (Intermediate, 45 min)

Steps 1. Apply Example 2 with a script that writes `/shared/shard-$SHARD.txt` (add PVC or use hostPath emptyDir). 2. `kubectl get pods -l job-name=infer -o wide` — expect 5 at a time. 3. `kubectl logs -l job-name=infer --prefix=true` to see per-pod output. 4. Delete pod-0; verify replacement gets same index.

Bonus: Kill a node with 3 infer pods; observe them re-created on other nodes.

Lab 11.3 — Fault-tolerant work queue (Advanced, 90 min)

Objective: Consume 1,000 messages from Redis with 20 workers, retry-safe.

Steps 1. Deploy Redis: `helm install redis bitnami/redis`. 2. Preload 1,000 keys `task:1..1000`. 3. Deploy Job:

```
apiVersion: batch/v1
kind: Job
metadata: { name: worker }
spec:
  completions: 1
  parallelism: 20
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: w
        image: myrepo/queue-worker:1
        env: [{ name: REDIS, value: redis-master.default.svc:6379 }]
```

4. Watch: `kubectl get pods -l job-name=worker -w`. 5. Kill some pods mid-flight; verify workers idempotently re-pull messages until queue empty. 6. Check `kubectl get job worker -o yaml` — succeeded.

Cleanup: `kubectl delete job worker && helm uninstall redis`.

Bonus: Add `podFailurePolicy` to fast-fail on exit code 3 (unrecoverable).

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Job runs one or more pods to completion. A CronJob is a Job that fires on a schedule. Together they cover every batch pattern — migrations, backups, report generation, ML inference sweeps.
Why: Deployments are wrong for finite work — they'd restart pods forever. Jobs track successful completions, allow parallelism, and clean up when done using `ttlSecondsAfterFinished`.
How: For migrations I use a single-completion Job with `backoffLimit: 0`. For heavy batch work I use indexed Jobs or a work-queue pattern with `parallelism` up to my worker capacity. For nightlies I use CronJob with `timeZone`, `concurrencyPolicy: Forbid`, and small `successfulJobsHistoryLimit` to keep the namespace clean. On EKS I let Karpenter provision Spot nodes for the batch, and on GKE Autopilot the pods just run and I pay per second.
Example: We had a nightly Postgres backup CronJob that once double-ran during a DST transition and doubled our S3 bill. Setting the `timeZone` field and `concurrencyPolicy: Forbid` fixed it permanently."

? Possible Interview Questions

Basic

Q1. Job vs Deployment? Job = run to completion; Deployment = long-running services.
Q2. What's `completions` vs `parallelism`? Completions = total successes required. Parallelism = concurrent pods.
Q3. CronJob schedule format? Standard 5-field cron; UTC by default.

Intermediate

Q4. `restartPolicy` for Job pods? `OnFailure` or `Never`. Never `Always`.
Q5. What does `ttlSecondsAfterFinished` do? Auto-deletes Job (and pods) after N seconds.
Q6. Difference `concurrencyPolicy` values? `Allow` (parallel runs), `Forbid` (skip if previous still running), `Replace` (kill previous).

Advanced

Q7. What is `podFailurePolicy`? Rules to control retry vs immediate failure based on exit codes or Pod conditions (stable K8s 1.26+).

Q8. Indexed Job — what is `JOB_COMPLETION_INDEX`? An env var giving each pod a unique 0..N-1 index for static work partitioning.
Q9. Why does a CronJob stop firing after being paused a day? > 100 missed schedules trigger safety — recreate or bump `startingDeadlineSeconds`.

Scenario-based

Q10. Migration job stuck retrying — how to debug? `kubectl describe job` for backoff events, `kubectl logs --previous` on failed pod, review `podFailurePolicy`, ensure migration is idempotent.

GKE-specific

Q11. On Autopilot, my parallel Job scales slowly — why? New pods trigger node provisioning; add `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` and consider pre-warming node pools (Standard only).

EKS-specific

Q12. How would you cost-optimize a huge batch job on EKS? Karpenter provisioner with Spot capacity type, `podFailurePolicy` for Spot interruption exit codes, and `ttlSecondsAfterFinished` for cleanup.

Hands-on / Lab-based

Q13. How did you verify a CronJob ran on time? `kubectl get cronjob` shows `LAST SCHEDULE`; drill into `kubectl get jobs -l app=heartbeat` for actual pods; audit via events.
Q14. How would you rerun a CronJob manually? `kubectl create job manual --from=cronjob/<n>`.
Q15. How do you cancel a running Job? `kubectl patch job <n> -p '{"spec":{"suspend":true}}'` (K8s 1.24+ stable) or `kubectl delete job <n>`.

🗐 Topic 12 — ReplicationControllers (Legacy)

Official Documentation Reference - ReplicationController: https://kubernetes.io/docs/concepts/workloads/controllers/replicationcontroller/ - Deprecation policy: https://kubernetes.io/docs/reference/using-api/deprecation-policy/

🔹 Definition

A **ReplicationController (RC)** is the original Kubernetes controller that ensures a specified number of pod replicas are running at any given time. It has been superseded by **ReplicaSet** (and, at a higher level, **Deployment**) — new work should use those.

🔹 About

Why it exists in the docs

RC shipped with Kubernetes v1.0 in 2015. It solved the same problem RS solves today: "keep N pods alive". It's still in the API (`v1` group) for backward compatibility.

What changed with ReplicaSet

Feature	RC	RS
Selector	Equality only (`app=web`)	Equality + set-based (`env In (dev,stg)`)
API group	core `v1`	`apps/v1`
Used by Deployment	No	Yes
Recommended for new work?	No	Only via Deployment
Rolling updates	Manual via `kubectl rolling-update` (deprecated CLI)	Handled by Deployment

Should you ever create one?

Almost never. Recognize it in old Helm charts, legacy manifests, or CKA exam material — then migrate.

Migration path

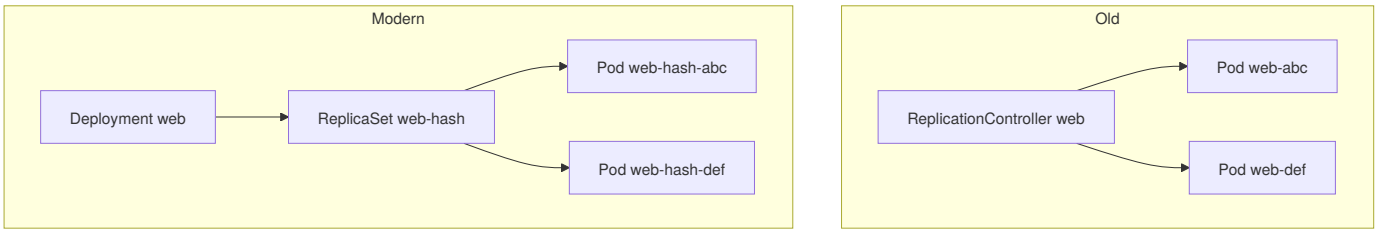
- `RC → RS` — straightforward: same fields, add `apps/v1` group, change `kind`.
- `RC → Deployment` — the modern answer; gain rolling updates and rollback for free.

🔹 How It Integrates With Other Kubernetes Components

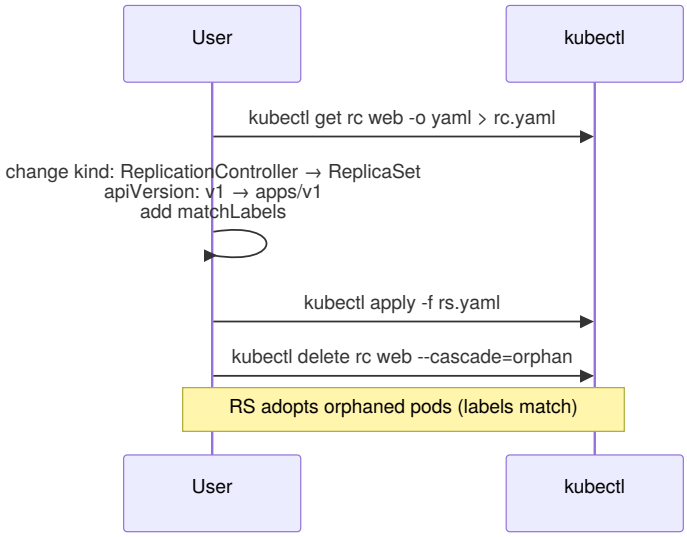
Component	Interaction
Pod	RC creates/deletes pods matching its selector
Service	Same as RS/Deployment — Service selects the pods by label
kube-controller-manager	Hosts the `replicationcontroller` controller
Deployment	**Does not manage RC** — Deployment always uses ReplicaSet

🏗 Diagrams

Legacy hierarchy



Migration flow



💻 Real-World Examples

Example 1 — Bare ReplicationController (recognize, don't build new)

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: web
spec:
  replicas: 3
  selector:
    app: web                  # equality only
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
      - { name: nginx, image: nginx:1.27, ports: [{ containerPort: 80 }] }
```

Example 2 — Convert RC to Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
    matchLabels: { app: web }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
      - { name: nginx, image: nginx:1.27, ports: [{ containerPort: 80 }] }
```

Apply the Deployment, delete the RC with `--cascade=orphan`, let Deployment/RS adopt the pods.

Example 3 — Scaling still works

```
kubectl scale rc/web --replicas=5
kubectl get rc web
kubectl delete rc web
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get rc`	`-A`	List RCs everywhere	`kubectl get rc -A`
`kubectl describe rc`	—	See pods and events	`kubectl describe rc web`
`kubectl scale rc`	`--replicas=`	Change count	`kubectl scale rc/web --replicas=5`
`kubectl delete rc`	`--cascade=orphan`	Preserve pods	`kubectl delete rc web --cascade=orphan`
`kubectl expose rc`	`--port --type`	Create Service	`kubectl expose rc web --port=80`
`kubectl rolling-update`	(deprecated CLI, removed)	Old rolling update	—

🚨 Scenarios

Scenario 1 — "We inherited a chart that uses RC"

- **Action**: rewrite as Deployment. Keep same labels so Services keep working. Apply the Deployment first, then delete the RC with `--cascade=orphan`.

Scenario 2 — RC exists but pods have extra labels

- **Cause**: RC only matches on equality; extras don't matter for matching but complicate selector overlap.
- **Fix**: still safe; just be careful with Services that select by broader labels.

Scenario 3 — CKA exam asks to fix a scaling issue on an RC

- **Action**: `kubectl scale rc/<n> --replicas=N` or edit the manifest.

☁ GCP GKE Perspective

- GKE supports RC (upstream compatibility). No GKE-specific features rely on RC. Autopilot allows RC pods provided they meet Autopilot restrictions.
- No `gcloud` shortcuts for RC — same `kubectl` verbs as anywhere.

🟩 AWS EKS Perspective

- Same story. EKS supports RC; no reason to prefer it. Managed add-ons never use RC.

🧪 Hands-On Lab Exercises

Lab 12.1 — Create + scale + delete an RC (Beginner, 20 min)

Steps

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: ReplicationController
metadata: { name: rc-demo }
spec:
  replicas: 2
  selector: { app: demo }
  template:
    metadata: { labels: { app: demo } }
    spec: { containers: [{ name: c, image: nginx:1.27 }] }
EOF
kubectl scale rc/rc-demo --replicas=4
kubectl get pods -l app=demo -o wide
kubectl delete pod -l app=demo --wait=false
kubectl get pods -l app=demo -w      # RC replaces them
kubectl delete rc rc-demo
```

Bonus: Try to add a `matchExpressions` selector — the API will reject it (RC doesn't support set-based).

Lab 12.2 — Migrate RC to Deployment without downtime (Intermediate, 45 min)

Steps 1. Deploy RC with 3 replicas + a Service selecting `app: demo`. 2. Run a `hey` load: `hey -z 60s http://<svc>/`. 3. Create Deployment with `matchLabels: {app: demo}` and same template, replicas 3. 4. Deployment will spawn new pods; the Service now has 6 endpoints. 5. `kubectl delete rc rc-demo --cascade=orphan` → RC's pods are orphaned; Deployment adopts them via label match. 6. Ensure `hey` shows zero failures throughout.

Bonus: Repeat but use RS (`apps/v1 ReplicaSet`) as the target instead of Deployment; then wrap that RS with a Deployment.

Lab 12.3 — RC on Autopilot compatibility check (Advanced, 30 min)

Steps 1. On GKE Autopilot: `kubectl apply -f rc.yaml`. 2. Confirm pods run. 3. Try to set `privileged: true` on the container — expect admission rejection (Autopilot policy). 4. Compare with same manifest on Standard/Minikube.

Cleanup: `kubectl delete rc rc-demo`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: The ReplicationController is Kubernetes' original replica controller from v1.0. Given a template and a count, it keeps that many pods alive.
Why-not: We rarely use RC today — ReplicaSet added set-based selectors and lives under the `apps/v1` group, and Deployments layered rolling updates and rollbacks on top of ReplicaSets. So the modern reflex is 'Deployment for stateless, StatefulSet for stateful, DaemonSet for node-level, Job/CronJob for finite work'.
How to handle in the wild: When I inherit an RC — usually in an old Helm chart — I migrate: write an equivalent Deployment, apply it, then delete the RC with `--cascade=orphan` so the Deployment's ReplicaSet adopts the running pods; no downtime.
Interview relevance: CKA and legacy code still test RC, so I know its selector limitation (equality only) and how to convert it.
Example: On a 2018-era code base we had 40 RCs. In a single sprint we replaced each with a Deployment using a scripted `sed` / `yq` transform and rolled through canary namespaces first."

? Possible Interview Questions

Basic

Q1. What does an RC do? Ensures N pod replicas exist for a given label selector.
Q2. Is RC still supported? Yes for backward compatibility, but not recommended.
Q3. Replacement for RC? ReplicaSet (managed by Deployment).

Intermediate

Q4. RC vs RS? RS adds set-based selectors and lives under `apps/v1`; RC is core `v1`, equality only.
Q5. Why did K8s deprecate `kubectl rolling-update`? Deployments do rolling updates declaratively; the imperative CLI became redundant and error-prone.
Q6. Can Deployment manage RC? No — Deployment always creates ReplicaSets.

Advanced

Q7. Explain how you'd migrate 100 RCs to Deployments safely.
Script the manifest transformation (change kind, apiVersion, selector to matchLabels, keep template labels). Apply Deployment. Verify Service endpoints. `kubectl delete rc --cascade=orphan`. Deployment/RS adopts orphan pods; no restarts.

Q8. What is the API group and version of RC? `v1` (core).
Q9. Any performance difference RC vs RS? Not meaningful; both are controllers watching pods. RS is the future.

Scenario-based

Q10. Two RCs select the same label — what happens? Same "fighting" problem as RS: each tries to reach its `replicas` count using overlapping pods. Fix selectors.

GKE-specific

Q11. Does Autopilot care if you use RC? No — it applies the same admission rules; but the recommendation is Deployment.

EKS-specific

Q12. Does IRSA work with RC pods? Yes — IRSA acts on the ServiceAccount, independent of the workload controller kind.

Hands-on / Lab-based

Q13. In your lab, how did you migrate an RC to a Deployment without downtime?
Applied Deployment first (adds new pods), verified traffic served correctly, then deleted the RC with `--cascade=orphan` so the Deployment's ReplicaSet adopted the surviving pods via label match.
Q14. Show me how to prove pod adoption worked. `kubectl get pod <name> -o jsonpath='{.metadata.ownerReferences[*].kind}'` — should print `ReplicaSet` after adoption.
Q15. Any pitfalls in RC-to-Deployment migration? Label mismatches, Service selector too broad (may pick up new/old pods indiscriminately), and forgetting `--cascade=orphan` (which would delete pods first, then Deployment recreates them = brief downtime).

MODULE

Module 3 — Configuration

🗐 Topic 13 — Labels, Selectors, Annotations

Official Documentation Reference - Labels & Selectors: https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/ - Annotations: https://kubernetes.io/docs/concepts/overview/working-with-objects/annotations/ - Recommended labels: https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/ - Field selectors: https://kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/ - Well-known labels/annotations/taints: https://kubernetes.io/docs/reference/labels-annotations-taints/

🔹 Definition

- **Label** — a small `key: value` pair on any Kubernetes object used to organize and **select** groups of objects.
- **Selector** — a query over labels (equality or set-based) used by controllers, Services, kubectl, and network policies.
- **Annotation** — a non-identifying `key: value` used to attach arbitrary metadata (build info, ownership, tooling hints); **not** used for selection.

🔹 About

Label rules

- Key: optional prefix `example.com/` + name (`app.kubernetes.io/name`).
- Prefix must be a DNS subdomain, ≤ 253 chars.
- Name/value: alphanumerics + `-_.` , ≤ 63 chars.
- Values may be empty.

Recommended labels (`app.kubernetes.io/…`)

Label	Purpose
`app.kubernetes.io/name`	App name (`mysql`)
`app.kubernetes.io/instance`	Unique instance (`mysql-abc`)
`app.kubernetes.io/version`	Version (`8.0.36`)
`app.kubernetes.io/component`	Role (`database`)
`app.kubernetes.io/part-of`	Higher-level app (`wordpress`)
`app.kubernetes.io/managed-by`	Deployer (`helm` , `argocd` , `kustomize`)

Selectors

Two flavors: - **Equality-based**: `app=web` , `tier!=cache` . - **Set-based**: `env in (dev, stg)` , `role notin (db)` , `key` , `!key` .

Consumers of selectors: - Services (`spec.selector`) - Deployments/RS/STS/DS (`spec.selector.matchLabels` / `matchExpressions`) - NetworkPolicies (`spec.podSelector`) - Node affinity / anti-affinity (`labelSelector`) - kubectl (`-l app=web,tier=front`)

Field selectors

Distinct from label selectors — filter by object field:

```
kubectl get pods --field-selector status.phase=Running,spec.nodeName=node-1
kubectl get events --field-selector type=Warning
```

Annotations vs labels

Property	Label	Annotation
Purpose	Identify & select	Arbitrary metadata
Used by K8s selectors	Yes	No
Size	Small	Larger (up to 256KB total)
Values	Constrained	Any string, arbitrary content
Examples	`app=web` , `env=prod`	`git-sha` , `owner=team-a` , `deprecated=true` , `nginx.ingress.kubernetes.io/rewrite-target`

Well-known annotations

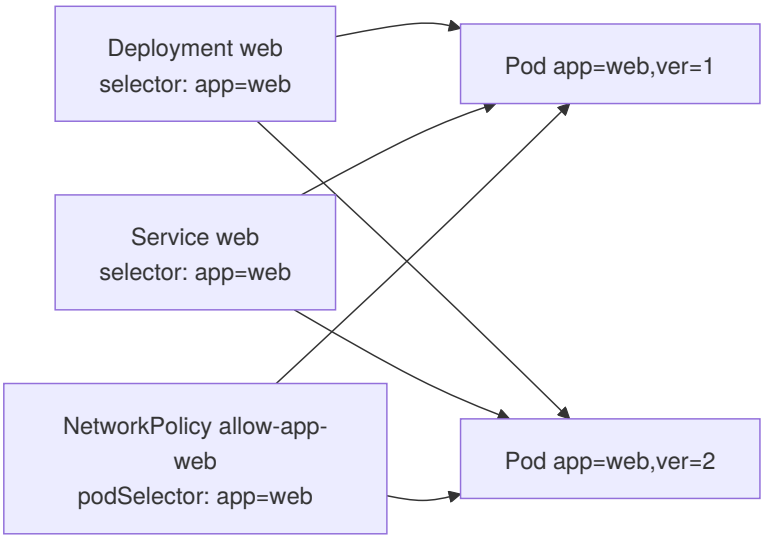
- `kubectl.kubernetes.io/last-applied-configuration`
- `deployment.kubernetes.io/revision`
- `pod-security.kubernetes.io/enforce`
- Cloud provider hints: `service.beta.kubernetes.io/aws-load-balancer-type` , `cloud.google.com/neg` .

🔹 How It Integrates With Other Kubernetes Components

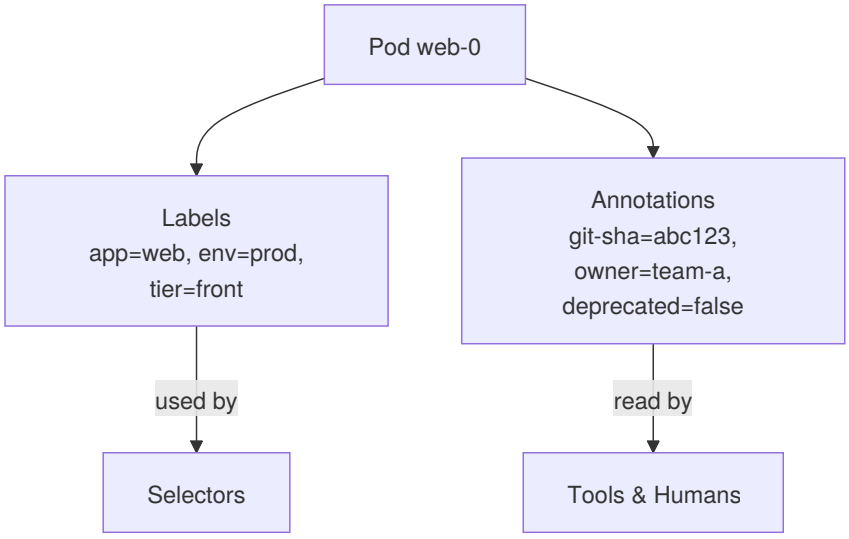
Component	Usage
Services	`selector` maps traffic to pods
Controllers	`spec.selector` determines which pods to manage
NetworkPolicies	`podSelector` + `namespaceSelector` gate traffic
Scheduling	`nodeAffinity` / `topologyKey` use labels
Cost / audit	Labels power cluster cost reporting (Kubecost, GKE cost breakdown)
Ingress controllers	Annotations enable extra features
Cloud LB provisioning	Annotations tune LB type/SG/SSL

🏗 Diagrams

Selector graph



Labels vs Annotations



💻 Real-World Examples

Example 1 — Full label set on a Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app.kubernetes.io/name: web
    app.kubernetes.io/instance: web-prod
    app.kubernetes.io/version: 1.4.2
    app.kubernetes.io/component: frontend
    app.kubernetes.io/part-of: shop
    app.kubernetes.io/managed-by: argocd
    env: prod
    team: platform
  annotations:
    git.example.com/commit: abc123
    owner: platform-team@company.io
    argocd.argoproj.io/tracking-id: shop:apps/Deployment:prod/web
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: web
      app.kubernetes.io/instance: web-prod
  template:
    metadata:
      labels:
        app.kubernetes.io/name: web
        app.kubernetes.io/instance: web-prod
        env: prod
    spec:
      containers: [{ name: c, image: myrepo/web:1.4.2 }]
```

Example 2 — Set-based selector for canary

```yaml
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: null                        # NOT allowed: use ports + selector below
  # Real trick: two Deployments (stable, canary) both labeled app=web
  # Service selector app=web catches both.
  ports: [{ port: 80 }]
```

Example 3 — kubectl label filtering

```bash
kubectl get pods -l 'app=web,env in (prod,stg)'
kubectl get pods -l 'env,tier!=cache'
kubectl label pod web-0 canary=true --overwrite
kubectl label pod web-0 canary-              # removes
kubectl annotate deploy web owner=team-x --overwrite
```

Example 4 — Ingress with NGINX annotations

```yaml
metadata:
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/limit-rps: "50"
    cert-manager.io/cluster-issuer: letsencrypt-prod
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl label`	`<obj> key=val`	Add/set label	`kubectl label pod web-0 env=prod`
`kubectl label`	`--overwrite`	Replace existing	`kubectl label pod web-0 env=prod --overwrite`
`kubectl label`	`key-`	Remove	`kubectl label pod web-0 env-`
`kubectl get`	`-l` / `--selector`	Filter by labels	`kubectl get pods -l app=web,tier=front`
`kubectl get`	`--show-labels`	Show label columns	`kubectl get pods --show-labels`
`kubectl get`	`-L key1,key2`	Show specific labels as columns	`kubectl get pods -L env,tier`
`kubectl get`	`--field-selector`	Filter by fields	`kubectl get pods --field-selector status.phase=Running`
`kubectl annotate`	`<obj> key=val`	Add/set annotation	`kubectl annotate pod web-0 owner=x`
`kubectl annotate`	`key-`	Remove	`kubectl annotate pod web-0 owner-`
`kubectl get`	`-o jsonpath='{.items[*].metadata.labels}'`	Extract labels	`kubectl get pods -o jsonpath='{.items[*].metadata.labels}'`

🚨 Scenarios

Scenario 1 — Service returns no endpoints

- **Cause**: Service selector doesn't match any pod labels.

- **Diagnose**: `kubectl get endpoints <svc>` ; `kubectl describe svc <svc>` .
- **Fix**: Align selector or pod labels: `kubectl label pod <p> app=web` .

Scenario 2 — Deployment "adopts" unrelated pods

- **Cause**: too-broad selector matches pods created by other controllers/humans.
- **Fix**: use unique labels (e.g., add `app.kubernetes.io/instance`) and set the selector accordingly.

Scenario 3 — Cost report groups all workloads together

- **Cause**: missing `team` / `env` / `cost-center` labels.
- **Fix**: enforce required labels via Kyverno/OPA admission.

Scenario 4 — Annotation exceeds size (256KB total)

- **Cause**: pushing entire config blobs into annotations.
- **Fix**: Move to a ConfigMap and reference by name.

Scenario 5 — Ingress feature not activating

- **Cause**: Annotation on wrong object (Ingress vs Service) or misspelled prefix.
- **Fix**: check controller docs; validate with `kubectl describe ing` .

☁ GCP GKE Perspective

- **GKE cost allocation** relies on labels; enable label sync in `gcloud container clusters update` .
- **Node labels**: `topology.kubernetes.io/zone` , `cloud.google.com/gke-nodepool` , `cloud.google.com/gke-preemptible` .
- Annotations for LB: `cloud.google.com/neg: '{"ingress": true}'` , `networking.gke.io/load-balancer-type` , `cloud.google.com/backend-config` .
- Workload Identity annotation on ServiceAccount: `iam.gke.io/gcp-service-account=<sa@project.iam.gserviceaccount.com>` .

▩ AWS EKS Perspective

- **VPC CNI** node labels: `topology.kubernetes.io/zone` , `karpenter.sh/provisioner-name` .
- **Service annotations** for AWS LB Controller:
- `service.beta.kubernetes.io/aws-load-balancer-type: nlb`
- `service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing`
- `service.beta.kubernetes.io/aws-load-balancer-ssl-cert: arn:aws:acm:…`
- **IRSA** annotation on ServiceAccount: `eks.amazonaws.com/role-arn=arn:aws:iam::…:role/…` .
- **AWS cost allocation tags** for CUR require namespace/pod labels + explicit CUR tag config.

Hands-On Lab Exercises

Lab 13.1 — Label & select (Beginner, 20 min)

Steps

```
kubectl run p1 --image=nginx --labels="app=web,env=dev"
kubectl run p2 --image=nginx --labels="app=web,env=prod"
kubectl run p3 --image=nginx --labels="app=api,env=prod"

kubectl get pods -l app=web
kubectl get pods -l 'env in (dev,prod), app=web'
kubectl get pods --show-labels
kubectl label pod p1 canary=true
kubectl get pods -L env,canary
```

Cleanup: `kubectl delete pod p1 p2 p3` .

Lab 13.2 — Enforce required labels with Kyverno (Intermediate, 60 min)

Steps 1. Install Kyverno: `kubectl apply -f https://raw.githubusercontent.com/kyverno/kyverno/main/config/install.yaml` . 2. Apply policy:

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: require-labels }
spec:
  validationFailureAction: Enforce
  rules:
  - name: check-labels
    match: { any: [{ resources: { kinds: ["Deployment","StatefulSet","DaemonSet"] } }] }
    validate:
      message: "Labels app.kubernetes.io/name, env, team required."
      pattern:
        metadata:
          labels:
            app.kubernetes.io/name: "?*"
            env: "?*"
            team: "?*"
```

3. Try to create a Deployment missing `team` — rejected. 4. Add labels; accepted.

Cleanup: `kubectl delete clusterpolicy require-labels` .

Bonus: Convert policy to `Audit` mode; view violations via `kubectl get policyreport -A` .

Lab 13.3 — Label-driven canary + cost allocation (Advanced, 60 min)

Steps 1. Deploy Deployments `web-stable` and `web-canary`, both with `app=web`. 2. Service selects `app=web` → 90/10 traffic split. 3. Install Kubecost (`helm install kubecost kubecost/cost-analyzer`). 4. Confirm cost dashboard breaks down by `env`, `team` labels. 5. Add cost dashboards per label; validate cost allocation.

Cleanup: `helm uninstall kubecost && kubectl delete deploy web-stable web-canary`.

Bonus: On GKE enable **Cost Allocation** in cluster settings and compare with Kubecost.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Labels are the key/value tags Kubernetes uses to organize and select objects. Selectors query those labels — equality-based or set-based — and are how Services find pods, controllers find their replicas, and NetworkPolicies scope traffic. Annotations are structurally similar but are not used for selection — they carry metadata for humans and tooling.
Why: Labels are the graph that ties every object together. Without a consistent labeling scheme, cost reports, RBAC scoping, alert routing, and multi-tenant policies all fall apart.
How: I standardize on the `app.kubernetes.io/*` recommended labels plus `env`, `team`, `cost-center`. I enforce them at admission with Kyverno so nothing lands without them. Annotations carry git SHAs, owner emails, and controller-specific hints — like NGINX Ingress rules or Workload Identity/IRSA bindings.
Example: In one platform we added a Kyverno policy requiring `team` and `cost-center` on every workload; within a month our monthly cloud bill had a clean per-team breakdown, and platform charges dropped 22% after the offending team optimized its overprovisioned deployments."

? Possible Interview Questions

Basic

Q1. Label vs annotation? Label = selectable metadata; annotation = arbitrary metadata not used for selection.
Q2. Give 3 recommended labels. `app.kubernetes.io/name`, `app.kubernetes.io/instance`, `app.kubernetes.io/version`.
Q3. Can labels be empty? Values can be empty; keys cannot.

Intermediate

Q4. Equality vs set-based selectors — examples.
Equality: `app=web`. Set-based: `env in (dev,stg)`.
Q5. Which controllers accept set-based selectors? RS, Deployment, DS, STS, Job — via `matchExpressions`. Service and RC support only equality-based.
Q6. What is a field selector? Filter by an object's field (e.g., `status.phase=Running`), not by labels.

Advanced

Q7. Why does the API forbid mutating a Deployment's `spec.selector` after creation? It could orphan/adopt pods across versions, breaking rollout and ownership guarantees.
Q8. How do you enforce label conventions across a fleet? Admission controllers — Kyverno/OPA Gatekeeper policies validating and mutating manifests.
Q9. Are annotations size-limited? All annotations on one object combined must be < 256KB.

Scenario-based

Q10. A Service has no endpoints but pods are Running — steps? Compare `kubectl get svc <n> -o yaml` selector vs pod labels; ensure ports/targetPort match container port; check readinessProbe.

GKE-specific

Q11. How does GKE's Workload Identity know which GCP SA to use? The annotation `iam.gke.io/gcp-service-account` on the Kubernetes ServiceAccount points to a GCP SA that trusts the K8s SA.

EKS-specific

Q12. How do you tell AWS LB Controller to create an NLB instead of ALB? Set `service.beta.kubernetes.io/aws-load-balancer-type: nlb` annotation on the Service.

Hands-on / Lab-based

Q13. Show me a set-based selector.

```
selector:
  matchExpressions:
  - { key: env, operator: In, values: [dev, stg] }
  - { key: tier, operator: NotIn, values: [cache] }
```

Q14. How would you find every deprecated Deployment? `kubectl get deploy -A -o json | jq '.items[] | select(.metadata.annotations["deprecated"]=="true")'`.
Q15. How do you rename a label across all pods? `kubectl label pods -A --all newkey=$OLDVAL` scripted via jsonpath, or use `kubectl label --overwrite` after Kyverno policy migration.

📄 Topic 14 — ConfigMaps

Official Documentation Reference - ConfigMap: https://kubernetes.io/docs/concepts/configuration/configmap/ - Configure a Pod to use a ConfigMap: https://kubernetes.io/docs/tasks/configure-pod-container/configure-pod-configmap/ - Updating configuration via a ConfigMap (tutorial): https://kubernetes.io/docs/tutorials/configuration/updating-configuration-via-a-configmap/ - GKE ConfigMap best practices: https://cloud.google.com/kubernetes-

engine/docs/concepts/configmap - EKS externalized config: https://docs.aws.amazon.com/eks/latest/best-practices/

🔷 Definition

A **ConfigMap** is a Kubernetes API object that stores **non-sensitive** configuration data as key/value pairs. Pods consume ConfigMaps as environment variables, command-line arguments, or as files mounted into volumes.

🔷 About

Why ConfigMaps

The Twelve-Factor App principle "config in the environment" — separate config from images so the same image ships to dev, staging, and prod with different behavior. ConfigMaps hold DB URLs, feature flags, log levels, arbitrary config files.

Ways a pod consumes a ConfigMap

Consumption	Behavior
`env` (single key → env var)	Value baked in at container start
`envFrom` (whole ConfigMap)	Every key becomes an env var
Volume (`volumes.configMap`)	Each key becomes a file in the mount path
Volume subPath	Mount a single key as one file next to existing files
`spec.containers.command/args` `$(VAR)` interpolation	Reference env vars from ConfigMap

Immutability

`immutable: true` (K8s 1.21+) → prevents any updates. Improves apiserver cache efficiency and prevents drift. Use for versioned ConfigMaps (`cm-v1` , `cm-v2`).

Size limit

Total size of a single ConfigMap ≤ **1 MiB**. Larger config → split into multiple, or use a volume/secret store.

Updates and reload behavior

- **Env vars**: NOT updated after container start; requires pod restart.
- **Volume mounts**: kubelet updates files in place, usually within ~1 minute (via projected volume periodic sync). App must watch the file to hot-reload.
- **subPath mounts**: NOT updated (mount is bind-mount to a single file).

Update propagation tools

- `stakater/Reloader` — watches ConfigMap/Secret changes and rolling-restarts owning Deployments.
- Kustomize `configMapGenerator` — appends a hash suffix to the name; any change → new ConfigMap → rolling update.
- Argo CD auto-syncs.

Binary data

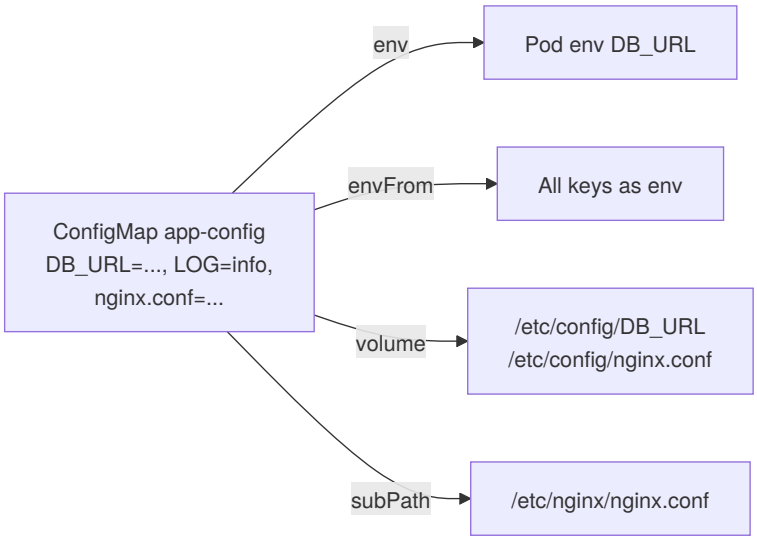
Use `.binaryData` (base64) for non-UTF-8 content.

🔷 How It Integrates With Other Kubernetes Components

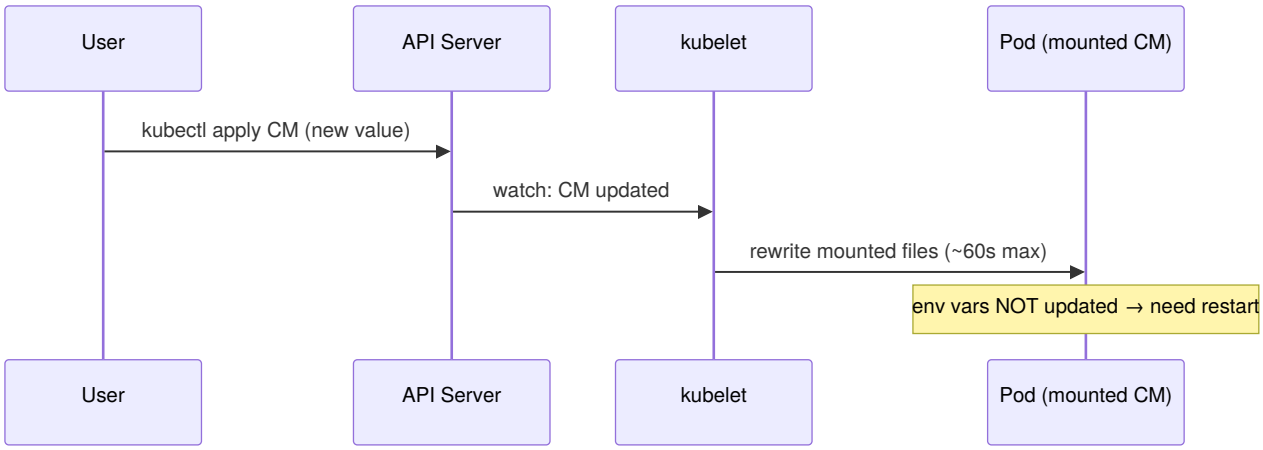
Component	Interaction
Pod	Consumes via env/volume/subPath
Deployment/STS/DS	Reference ConfigMap in template
RBAC	Namespace-scoped access controlled by Roles
Kustomize/Helm	Generate ConfigMaps with hash suffixes
Admission (Kyverno)	Enforce presence of required keys
External Secrets Operator	Can sync from external stores into ConfigMaps too

🏗 Diagrams

ConfigMap consumption modes



Update propagation



💻 Real-World Examples

Example 1 — ConfigMap from literals + files

```
kubectl create cm app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=FEATURE_X=true \
  --from-file=nginx.conf=./nginx.conf
```

Example 2 — Consume as env + as volume

```
apiVersion: v1
kind: Pod
metadata: { name: web }
spec:
  containers:
  - name: app
    image: nginx:1.27
    env:
    - name: LOG_LEVEL
      valueFrom: { configMapKeyRef: { name: app-config, key: LOG_LEVEL } }
    envFrom:
    - configMapRef: { name: app-config }        # every key
    volumeMounts:
    - { name: nginx-conf, mountPath: /etc/nginx/nginx.conf, subPath: nginx.conf }
  volumes:
  - name: nginx-conf
    configMap:
      name: app-config
      items:
      - { key: nginx.conf, path: nginx.conf }
```

Example 3 — Immutable, versioned

```
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config-v2 }
immutable: true
data:
  LOG_LEVEL: info
  FEATURE_X: "true"
```

Deployment references `app-config-v2`. Next change → `app-config-v3` + new Deployment revision.

Example 4 — Kustomize with automatic hash

```
# kustomization.yaml
configMapGenerator:
- name: app-config
  literals:
  - LOG_LEVEL=info
  - FEATURE_X=true
generatorOptions:
  disableNameSuffixHash: false      # hash appended
```

`kubectl apply -k .` — changed literal produces `app-config-<newhash>`, triggering rolling update on any Deployment that references the generated name.

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create cm`	`--from-literal=`	Inline value	`kubectl create cm c --from-literal=K=V`
`kubectl create cm`	`--from-file=`	File contents as value	`kubectl create cm c --from-file=./nginx.conf`
`kubectl create cm`	`--from-env-file=`	dotenv format	`kubectl create cm c --from-env-file=.env`
`kubectl get cm`	`-o yaml`	Show all data	`kubectl get cm app-config -o yaml`
`kubectl describe cm`	—	Human-readable	`kubectl describe cm app-config`
`kubectl edit cm`	—	Live edit	`kubectl edit cm app-config`
`kubectl delete cm`	`<name>`	Remove	`kubectl delete cm app-config`
`kubectl create cm`	`--dry-run=client -o yaml`	Skeleton	`kubectl create cm c --from-literal=A=B --dry-run=client -o yaml`
`kubectl rollout restart deploy`	—	Force pods to reload env-based CM changes	`kubectl rollout restart deploy web`

🚨 Scenarios

Scenario 1 — App doesn't see new config

- **Cause**: consumed as env; container was never restarted after CM edit.
- **Fix**: `kubectl rollout restart deploy/<n>` or move to volume mount + app watch.

Scenario 2 — subPath ignores updates

- **Cause**: subPath is a bind mount to a specific file; kubelet doesn't refresh.
- **Fix**: mount whole directory (drop subPath) or use symlink pattern + Reloader.

Scenario 3 — ConfigMap exceeded 1 MiB

- **Cause**: dumping large JSON/YAML blob.
- **Fix**: split into multiple CMs, or use a `Secret` with binary, or a mounted volume from object store via init container.

Scenario 4 — Rolling restart takes too long during config change

- **Cause**: many pods, tight PDB, `maxUnavailable: 0`.
- **Fix**: raise `maxSurge`, allow small `maxUnavailable`, use canary namespace first.

Scenario 5 — Two teams overwrite each other's CM

- **Cause**: shared CM name, no ownership annotations.
- **Fix**: namespace separation, RBAC per team, `app.kubernetes.io/managed-by` annotations.

☁ GCP GKE Perspective

- **ConfigMaps are cluster-local**. To share cluster-external config, use **GCP Secret Manager** (via CSI driver `secrets-store-csi-driver-provider-gcp`) or **Config Sync** (Anthos Config Management) to pull YAMLs from a Git repo.
- **Anthos Config Management** enforces ConfigMap standards across a fleet of clusters.
- **Autopilot**: normal ConfigMap semantics; volume mount refresh delay similar to Standard.

▩ AWS EKS Perspective

- **AWS AppConfig** or **AWS Secrets Manager** for cluster-external config.
- **CSI Secrets Store driver** with the AWS provider can mount SSM parameters/Secrets Manager entries as files or sync them into ConfigMaps.
- **EKS Blueprints** ships opinionated Reloader/Kustomize/ArgoCD wiring so config changes trigger rolling updates automatically.

🧪 Hands-On Lab Exercises

Lab 14.1 — Env vs volume consumption (Beginner, 30 min)

Steps

```
kubectl create cm greeting --from-literal=GREET=hello --from-literal=NAME=world
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata: { name: cm-demo }
spec:
  containers:
  - name: c
    image: busybox
    command: ["sh","-c","echo $GREET $NAME; sleep 3600"]
    envFrom: [{ configMapRef: { name: greeting } }]
    volumeMounts: [{ name: v, mountPath: /etc/greet }]
  volumes:
  - name: v
    configMap: { name: greeting }
EOF
kubectl logs cm-demo
kubectl exec cm-demo -- ls /etc/greet
kubectl exec cm-demo -- cat /etc/greet/GREET
# update
kubectl create cm greeting --from-literal=GREET=hi --from-literal=NAME=world --dry-run=client -o yaml | kubectl apply -f -
sleep 90
kubectl exec cm-demo -- cat /etc/greet/GREET     # updated
kubectl logs cm-demo                            # env still says hello
```

Cleanup: `kubectl delete pod cm-demo && kubectl delete cm greeting`.

Lab 14.2 — Kustomize hash-based rollout (Intermediate, 45 min)

Steps 1. Create `kustomization.yaml` with `configMapGenerator` and a Deployment referencing `app-config`. 2. `kubectl apply -k .` — CM created as `app-config-<hash>`; Deployment env references that name (Kustomize substitutes). 3. Change a literal; re-apply. New CM name → new Deployment revision → rolling update. 4. `kubectl rollout history deploy/web` shows revisions.

Bonus: Add `annotations: {reloader.stakater.com/auto: "true"}` and repeat without Kustomize hash to see Reloader restart pods.

Lab 14.3 — CSI Secrets Store for GSM (GKE) or SSM/SM (EKS) (Advanced, 90 min)

Steps (EKS example) 1. Install driver: `helm install csi-secrets-store secrets-store-csi-driver/secrets-store-csi-driver --set syncSecret.enabled=true`. 2. Install AWS provider: `kubectl apply -f https://raw.githubusercontent.com/aws/secrets-store-csi-driver-provider-aws/main/deployment/aws-provider-installer.yaml`. 3. Create IAM role with `SecretsManager:GetSecretValue` and IRSA-bind to SA `csi-app`. 4. Apply:

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: { name: aws-secret }
spec:
  provider: aws
  parameters: { objects: |
    - objectName: "prod/api-config"
      objectType: "secretsmanager"
      jmesPath:
      - path: LOG_LEVEL
        objectAlias: LOG_LEVEL }
  secretObjects:
  - secretName: sync-cm
    type: Opaque
    data: [{ objectName: LOG_LEVEL, key: LOG_LEVEL }]
```

5. Pod mounts the secret as file; `sync-cm` propagates a K8s Secret too (which you could also convert to CM via ExternalSecrets Operator).

Cleanup: `helm uninstall csi-secrets-store && kubectl delete secretproviderclass aws-secret`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A ConfigMap externalizes non-sensitive configuration from a container image — DB URLs, log levels, entire config files. Pods consume it as environment variables, envFrom groups, or as files mounted from a volume.
Why: This lets one image ship to every environment. It also decouples ops changes from image builds — flipping a feature flag doesn't require a new release pipeline.
How: For most apps I use `envFrom` for simple keys and volume mounts for structured files like `nginx.conf`. I always know the update semantics: env vars need a pod restart, volume mounts refresh in about a minute. To make config changes trigger a controlled rollout I use Kustomize's `configMapGenerator` with a name hash, so any diff produces a new ConfigMap name and rolls the Deployment automatically. As a backup I run Stakater Reloader.
Example: We once shipped a bug because a feature flag was baked as an env var and the pod never restarted. Since then we moved feature flags to a mounted JSON file the app watches; changes propagate in < 60 s without rolling anything."

? Possible Interview Questions

Basic

Q1. What is a ConfigMap? Namespaced object storing non-sensitive key/value config.

Q2. Ways a pod uses a ConfigMap? env, envFrom, volume mount, subPath, command args interpolation.
Q3. Is a ConfigMap encrypted at rest? Only if you configured apiserver encryption-at-rest; by default it's plaintext in etcd.

Intermediate

Q4. If you update a CM, do env-consumed pods see the change? No — they need to restart.
Q5. What's the size limit of a ConfigMap? 1 MiB per object.
Q6. What does `immutable: true` do? Locks the ConfigMap; improves scale, prevents drift; commonly paired with versioned names.

Advanced

Q7. How does Kustomize handle CM updates? Appends a hash suffix; any change makes a new CM name → triggers Deployment rollout automatically.
Q8. What's `stakater/Reloader` ? A controller that watches ConfigMap/Secret changes and adds an annotation to owner Deployments, triggering a rolling restart.
Q9. subPath vs full mount? subPath binds one file (no live refresh); full mount is a directory that refreshes.

Scenario-based

Q10. Pods missing env var — steps? `kubectl describe pod` (env), `kubectl get cm <n> -o yaml` , verify key name matches, verify SA has read RBAC, check for typos.

GKE-specific

Q11. How do you sync ConfigMaps across many GKE clusters? Anthos Config Management pulling YAMLs from Git; or ArgoCD applied per cluster.

EKS-specific

Q12. Common alternative to ConfigMap on EKS for dynamic config? AWS AppConfig with cross-app rollout policies; or SSM Parameter Store fetched by init container.

Hands-on / Lab-based

Q13. Show me how you trigger a rollout when a CM changes. Use Kustomize `configMapGenerator` (hash) or add Reloader annotation `reloader.stakater.com/auto: "true"` .
Q14. How would you enforce that all Deployments have `LOG_LEVEL` set? Kyverno policy validating `envFrom` or `env` includes `LOG_LEVEL` .
Q15. How do you inspect the actual bytes in a CM? `kubectl get cm <n> -o yaml | yq '.data'` ; for binary use `.binaryData` (base64-decode).

🗐 Topic 15 — Secrets (Including External Secret Managers)

Official Documentation Reference - Secrets concept: https://kubernetes.io/docs/concepts/configuration/secret/ - Manage Secrets with kubectl: https://kubernetes.io/docs/tasks/configmap-secret/managing-secret-using-kubectl/ - Distribute credentials securely: https://kubernetes.io/docs/tasks/inject-data-application/distribute-credentials-secure/ - Encrypting data at rest: https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/ - Good practices for Secrets: https://kubernetes.io/docs/concepts/security/secrets-good-practices/ - GKE Secret Manager add-on: https://cloud.google.com/kubernetes-engine/docs/how-to/secret-manager - Workload Identity Federation for GKE: https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity - EKS IRSA: https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html - EKS Pod Identity: https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html - External Secrets Operator: https://external-secrets.io

🔹 Definition

A **Secret** is a namespaced Kubernetes object that stores small amounts of **sensitive** data — passwords, tokens, TLS certs, SSH keys — separately from the container image. Data is base64-encoded in the object (not encrypted by itself).

🔹 About

Key point up front

Base64 is NOT encryption. Anyone with `get secrets` on the namespace can decode. Secure Secrets with: 1. **Encryption at rest** on etcd (KMS provider). 2. **RBAC** so only intended workloads/humans can read them. 3. **External secret stores** for the source of truth. 4. **Audit logging** on Secret access.

Secret types

Type	Purpose
`Opaque` (default)	Arbitrary keys
`kubernetes.io/service-account-token`	Legacy SA token (auto-created for SAs pre-1.24)
`kubernetes.io/dockerconfigjson`	Registry pull creds
`kubernetes.io/tls`	TLS cert + key
`kubernetes.io/basic-auth`	Username + password
`kubernetes.io/ssh-auth`	SSH private key
`bootstrap.kubernetes.io/token`	Cluster bootstrap tokens

Consumption

Same shapes as ConfigMaps: `env`, `envFrom`, volume, subPath. Also imagePullSecrets (list in pod spec / SA).

tmpfs mount

Secret volumes mount as `tmpfs` — never touch disk on the node. This is a hardening default.

Encryption at rest

Configure `--encryption-provider-config` on the apiserver with `aescbc`, `aesgcm`, or (best) an external **KMS provider** (`kms v2`). GKE and EKS enable this by default with Google Cloud KMS / AWS KMS.

External Secrets — the modern pattern

Instead of storing secrets in etcd, store them in a real secret manager (**GCP Secret Manager**, **AWS Secrets Manager**, **AWS SSM Parameter Store**, **HashiCorp Vault**, **Azure Key Vault**, **1Password Connect**, ...) and pull them into pods:

Approaches: - **CSI Secrets Store driver** — mounts secrets as files at pod start. - **External Secrets Operator (ESO)** — a CRD `ExternalSecret` continuously syncs a value into a native K8s `Secret`. - **Sealed Secrets (Bitnami)** — encrypt secrets with cluster public key; commit encrypted `SealedSecret` to Git; controller decrypts into `Secret`.

Cloud identity for accessing external stores

- **GKE**: Workload Identity — bind a Kubernetes SA to a GCP SA (annotation `iam.gke.io/gcp-service-account`).
- **EKS**: **IRSA** (OIDC-based per SA) or **Pod Identity** (2023+, simpler API + no OIDC bootstrap).

Best practices

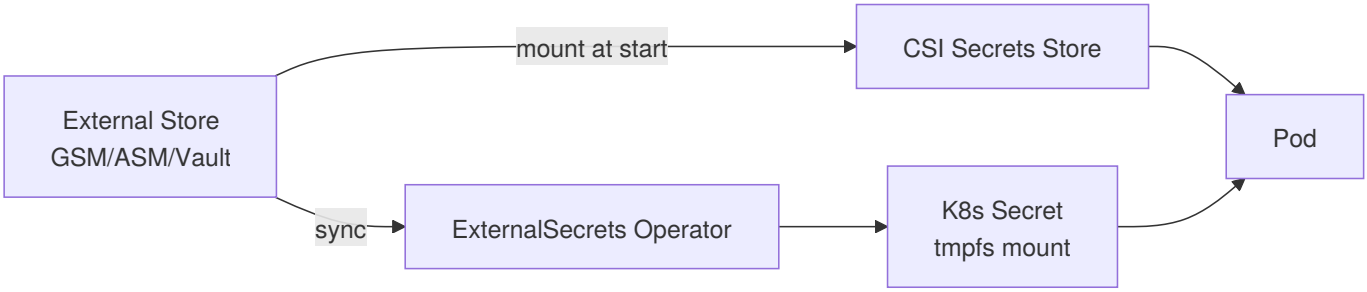
- Never `kubectl create secret` with a real value on a shared terminal (bash history!). Use `--from-file` referring to a file you `chmod 600`, or ExternalSecrets/CSI.
- Restrict RBAC: separate `list` from `get` from `describe`.
- Rotate: pods holding secrets in env don't auto-refresh; volume mounts refresh (~1 min).
- Enable `--audit-policy` to log Secret reads.
- Use **immutable Secrets** (K8s 1.21+) to prevent tampering.

How It Integrates With Other Kubernetes Components

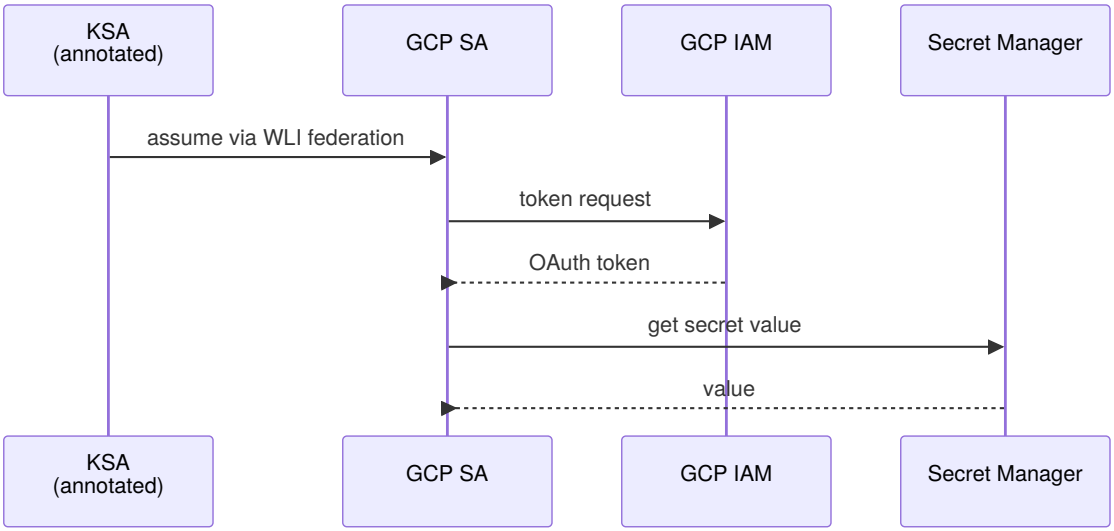
Component	Interaction
ServiceAccount	Auto-mounted projected token (bound audience, expiring)
kubelet	Mounts secrets as tmpfs
API server	Encrypts/decrypts via KMS
Admission (Kyverno)	Enforce no-secret-in-env / IRSA presence
ESO / CSI driver	Pull secrets from cloud managers
Ingress + cert-manager	TLS Secrets

Diagrams

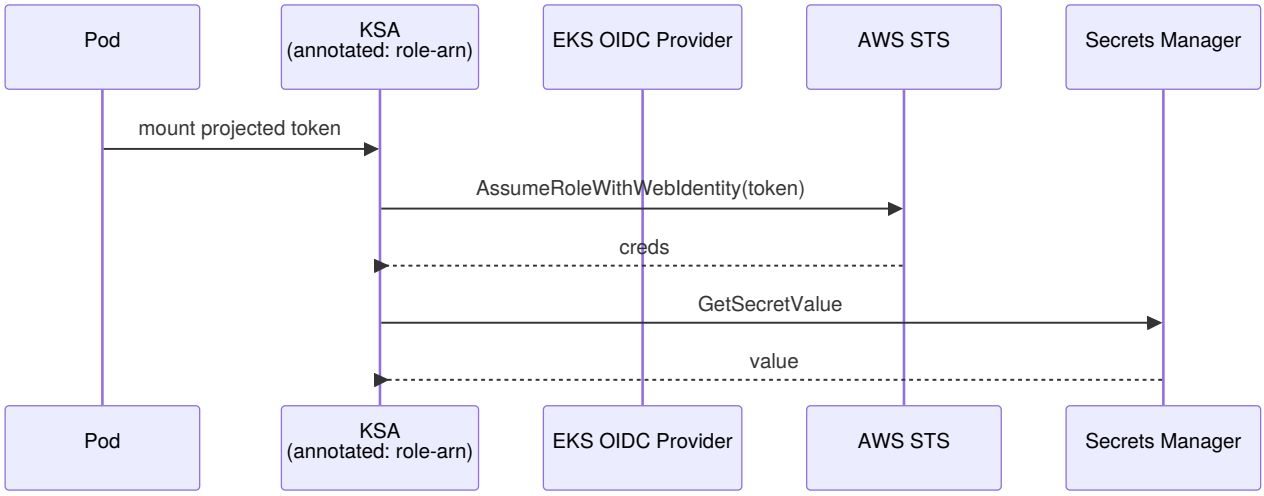
Secret data path



GKE Workload Identity



EKS IRSA



💻 Real-World Examples

Example 1 — Create a Secret and consume as env + volume

```
kubectl create secret generic db-cred \
  --from-literal=USER=app \
  --from-literal=PASSWORD='S3cure!'
```

```
apiVersion: v1
kind: Pod
metadata: { name: web }
spec:
  containers:
  - name: c
    image: alpine
    command: ["sh","-c","printenv | grep DB_; sleep 3600"]
    envFrom: [{ secretRef: { name: db-cred } }]
    volumeMounts: [{ name: creds, mountPath: /var/creds, readOnly: true }]
  volumes:
  - name: creds
    secret: { secretName: db-cred, defaultMode: 0400 }
```

Example 2 — TLS + image pull secrets

```
kubectl create secret tls web-tls --cert=tls.crt --key=tls.key
kubectl create secret docker-registry regcred \
  --docker-server=ghcr.io --docker-username=alice --docker-password=$TOKEN
```

```
spec:
  imagePullSecrets: [{ name: regcred }]
```

Example 3 — ExternalSecret (ESO) pulling from AWS Secrets Manager

```
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata: { name: aws-sm }
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-south-1
      auth: { jwt: { serviceAccountRef: { name: eso-sa } } }   # uses IRSA
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: db-cred }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: aws-sm, kind: SecretStore }
  target: { name: db-cred, creationPolicy: Owner }
  data:
  - { secretKey: PASSWORD, remoteRef: { key: prod/db/password } }
  - { secretKey: USER,     remoteRef: { key: prod/db/user } }
```

Example 4 — Workload Identity (GKE) accessing GCP Secret Manager

```
# GCP side
gcloud iam service-accounts create app-gsa --project $PROJECT
gcloud projects add-iam-policy-binding $PROJECT \
  --member=serviceAccount:app-gsa@$PROJECT.iam.gserviceaccount.com \
  --role=roles/secretmanager.secretAccessor
gcloud iam service-accounts add-iam-policy-binding \
  app-gsa@$PROJECT.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser \
  --member="serviceAccount:$PROJECT.svc.id.goog[prod/app-ksa]"
```

```
# K8s side
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-ksa
  namespace: prod
  annotations:
    iam.gke.io/gcp-service-account: app-gsa@PROJECT.iam.gserviceaccount.com
```

App code uses google-cloud-secret-manager library; no keys shipped.

Example 5 — Sealed Secret in Git

```
kubectl create secret generic db-cred --from-literal=P=x --dry-run=client -o yaml | \
  kubeseal --controller-name=sealed-secrets --format=yaml > sealed.yaml
git add sealed.yaml
```

Controller in cluster decrypts to `Secret`.

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create secret generic`	`--from-literal=`	Inline	`kubectl create secret generic s --from-literal=K=V`
`kubectl create secret generic`	`--from-file=`	File → key	`kubectl create secret generic s --from-file=key=./key.pem`
`kubectl create secret tls`	`--cert --key`	TLS	`kubectl create secret tls t --cert=c --key=k`
`kubectl create secret docker-registry`	`--docker-*`	Registry creds	`kubectl create secret docker-registry r --docker-server=... --docker-username=... --docker-password=...`
`kubectl get secret`	`-o jsonpath='{.data.PASSWORD}'`	Read (base64)	`... | base64 -d`
`kubectl describe secret`	—	Redacts data (only shows keys)	`kubectl describe secret db-cred`
`kubectl delete secret`	`<name>`	Remove	`kubectl delete secret db-cred`
`kubectl patch secret`	`-p`	Update field	`kubectl patch secret db-cred -p '{"stringData":{"P":"new"}}'`
`kubeseal`	`--format=yaml`	Encrypt for GitOps	see Example 5
`eksctl create iamserviceaccount`	`--role-name`	Bootstrap IRSA	`eksctl create iamserviceaccount --cluster=prod --namespace=default --name=eso-sa --attach-policy-arn=arn:aws:iam::aws:policy/SecretsManagerReadWrite --approve`
`gcloud iam`	`add-iam-policy-binding`	Bind KSA → GSA	see Example 4

🚨 Scenarios

Scenario 1 — Secret checked into Git in plaintext

- **Fix**: rotate the credential immediately; move to SealedSecrets or ExternalSecrets; add pre-commit hook `git-secrets`.

Scenario 2 — Pod restarted losing rotated password

- **Cause**: env consumption of secret; password rotated in secret manager but pod holds old value.
- **Fix**: use volume mount + app reload, or ESO with `refreshInterval` + Reloader.

Scenario 3 — Secret decoded by unintended user

- **Cause**: overly permissive RBAC (`list secrets` cluster-wide).
- **Fix**: least-privileged Role scoped to specific secret names; enable audit for `secrets`.

Scenario 4 — imagePullBackOff for private registry

- **Cause**: `imagePullSecrets` missing or wrong namespace.
- **Fix**: create secret in same namespace as pod, add to SA `imagePullSecrets`.

Scenario 5 — CSI Secrets Store won't mount

- **Cause**: SecretProviderClass mismatched namespace; missing driver DaemonSet; SA lacks IRSA/WLI.
- **Fix**: `kubectl describe pod` events; check `kubectl -n kube-system get ds secrets-store-csi-driver`.

☁ GCP GKE Perspective

- **Application-layer secret encryption** enabled with `--database-encryption-key` on cluster creation (uses Cloud KMS).
- **Secret Manager add-on** (`--enable-secret-manager`) provides native volume mounting of Secret Manager secrets without CSI-driver install.
- **Workload Identity Federation** is the recommended way for pods to authenticate to GCP APIs — no static keys.
- **Backup for GKE** captures Secrets as part of application backups (encrypted with your KMS key).

▩ AWS EKS Perspective

- **KMS envelope encryption** for K8s Secrets: `aws eks create-cluster --encryption-config resources=[secrets],provider={keyArn=…}`.
- **IRSA** for existing clusters (OIDC bootstrap + role trust). **Pod Identity** (2023+) is simpler — no OIDC required, uses eks-pod-identity-agent DaemonSet.
- **AWS Secrets Manager & SSM Parameter Store** are the two managed backends; both work through CSI Secrets Store or ESO.
- **AWS ACM** for TLS certificates in front of ALB — Ingress annotation `service.beta.kubernetes.io/aws-load-balancer-ssl-cert` avoids storing certs in K8s Secret.

🧪 Hands-On Lab Exercises

Lab 15.1 — Create & consume a Secret (Beginner, 30 min)

Steps

```
kubectl create secret generic api-key --from-literal=API_KEY=abc123
kubectl run demo --image=alpine --restart=Never -- \
  sh -c 'echo $API_KEY; sleep 3600' -- --overrides='
{"spec":{"containers":[{"name":"demo","image":"alpine",
  "envFrom":[{"secretRef":{"name":"api-key"}}]}]}}'
# or use a Pod manifest (cleaner)
kubectl logs demo
kubectl get secret api-key -o jsonpath='{.data.API_KEY}' | base64 -d
```

Cleanup: `kubectl delete pod demo && kubectl delete secret api-key`.

Bonus: Mount the secret as a volume and inspect `ls -la /var/secret` — permissions should be 0400 tmpfs.

Lab 15.2 — External Secrets Operator with AWS Secrets Manager (Intermediate, 90 min)

Prerequisites: EKS with OIDC (`eksctl utils associate-iam-oidc-provider --cluster=prod --approve`).

Steps 1. Install ESO: `helm repo add external-secrets https://charts.external-secrets.io && helm install external-secrets external-secrets/external-secrets -n external-secrets --create-namespace`. 2. Bootstrap IRSA: `eksctl create iamserviceaccount --cluster=prod --name=eso-sa --namespace=default --attach-policy-arn=arn:aws:iam::aws:policy/SecretsManagerReadWrite --approve`. 3. Push a value: `aws secretsmanager create-secret --name prod/db/password --secret-string 'S3cure!'`. 4. Apply SecretStore + ExternalSecret from Example 3. 5. Confirm K8s Secret `db-cred` synced: `kubectl get secret db-cred -o yaml`. 6. Rotate the AWS secret; wait `refreshInterval`; confirm K8s Secret updated.

Cleanup: `helm uninstall external-secrets -n external-secrets && aws secretsmanager delete-secret --secret-id prod/db/password --force-delete-without-recovery`.

Lab 15.3 — Workload Identity → GCP Secret Manager (Advanced, 90 min)

Prerequisites: GKE with Workload Identity enabled.

Steps 1. GCP: `gcloud secrets create db-password --replication-policy=automatic && echo -n 'S3cure!' | gcloud secrets versions add db-password --data-file=-`. 2. Create GSA + bind to KSA (Example 4). 3. Create KSA with annotation. 4. Deploy a pod using the Python google-cloud-secret-manager library; no static keys. 5. Verify pod reads the secret from Secret Manager. 6. Rotate: `echo 'NewS3cret' | gcloud secrets versions add db-password --data-file=-` ; pod re-reads

on next call.

Bonus: Replace app code with the GKE **Secret Manager add-on** volume mount (`--enable-secret-manager`); simpler and no code changes.

Interview Deliverable Answer (60–90 s)

"**What**: A Kubernetes Secret is a namespaced object for sensitive data — passwords, TLS certs, tokens. It looks like a ConfigMap, but base64-encoded and mounted as tmpfs. Base64 is not encryption; real security comes from encryption at rest, tight RBAC, and pulling from an external secret store.
Why: Committing secrets to Git or baking them into images is one of the fastest ways to a breach. Externalizing them means the source of truth is a hardened service — GCP Secret Manager, AWS Secrets Manager, or Vault — and rotation is one API call.
How: In production I don't put real secrets in etcd directly. I use ExternalSecrets Operator with cloud identity — Workload Identity on GKE, IRSA or Pod Identity on EKS — so pods pull from Secret Manager without any static credentials. I mount them as volumes so rotation propagates without pod restarts, or pair with Reloader when I need env vars. For GitOps I use SealedSecrets when there is no cloud store available.
Example: On a project we moved from storing DB passwords as inline K8s Secrets to ExternalSecrets from GCP Secret Manager. Rotation went from 'file a ticket, coordinate a rollout' to 'one gcloud command, done in under a minute across all environments'."

? Possible Interview Questions

Basic

Q1. Is a Secret encrypted? Base64-encoded, not encrypted, unless encryption-at-rest is configured.
Q2. Secret vs ConfigMap? Same shape; Secret intended for sensitive data, mounted as tmpfs, subject to stricter defaults.
Q3. What is `imagePullSecrets` ? Field on Pod/SA pointing to a `dockerconfigjson` Secret for private registries.

Intermediate

Q4. What is encryption at rest and how do you enable it? apiserver `EncryptionConfiguration` with a provider (aescbc/kms). GKE/EKS enable via cluster flags with cloud KMS.
Q5. Why prefer volume mount over env for secret? Volume updates propagate; env doesn't. Also, env vars can leak via `kubectl describe pod` , crash dumps, and process listings.
Q6. Difference between Sealed Secrets and External Secrets Operator? Sealed Secrets: encrypt at commit, decrypt in-cluster (secret still lives in etcd). ESO: source of truth stays outside; a K8s Secret is a projection.

Advanced

Q7. Explain IRSA end-to-end. Cluster has OIDC provider → IAM role trusts OIDC issuer with condition on SA → SA annotated with role ARN → projected token mounted into pod → AWS SDK exchanges token for STS creds via `AssumeRoleWithWebIdentity` .
Q8. Difference between IRSA and EKS Pod Identity? Pod Identity (2023+) uses a DaemonSet agent + associations API instead of OIDC — no OIDC provider setup; simpler for many clusters, but IRSA still works cross-account.
Q9. What is Workload Identity Federation on GKE? KSA presents an OIDC token to Google STS; STS returns credentials for the bound GSA. No keys downloaded.

Scenario-based

Q10. A pod needs a rotated secret every hour without restarting — design? ExternalSecret with `refreshInterval: 1h` , mount as volume, app watches the file, or use CSI Secrets Store with rotationPollInterval.

GKE-specific

Q11. What's the fastest way to give a pod access to a GCP Secret Manager secret? Enable Workload Identity, annotate KSA with GSA that has `secretmanager.secretAccessor` . Or enable the Secret Manager add-on and mount directly.

EKS-specific

Q12. Least-privilege pattern for an app that reads only one secret? Create IAM role scoped to `secretsmanager:GetSecretValue` on a specific ARN; IRSA-bind to the app's SA.

Hands-on / Lab-based

Q13. Show me creating a TLS secret and mounting it. `kubectl create secret tls web-tls --cert=t.crt --key=t.key` ; reference it under `ingress.spec.tls.secretName` .
Q14. How would you audit who read a Secret? Enable API server audit logging with rules on `secrets` verbs; ship to Cloud Logging / CloudWatch.
Q15. How do you rotate a K8s Secret without downtime? External store rotates → ESO/CSI syncs → volume file updates → app reloads (SIGHUP or file-watcher); Reloader rolls pods if using env consumption.

Topic 16 — Resource Requests and Limits

Official Documentation Reference - Resource management: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/ - Assign memory: https://kubernetes.io/docs/tasks/configure-pod-container/assign-memory-resource/ - Assign CPU: https://kubernetes.io/docs/tasks/configure-pod-container/assign-cpu-resource/ - QoS classes: https://kubernetes.io/docs/tasks/configure-pod-container/quality-service-pod/ - LimitRange: https://kubernetes.io/docs/concepts/policy/limit-range/ - ResourceQuota: https://kubernetes.io/docs/concepts/policy/resource-quotas/ - GKE resource management: https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-resource-requests - EKS Fargate resources: https://docs.aws.amazon.com/eks/latest/userguide/fargate-pod-configuration.html

---

🔷 Definition

Requests declare the minimum CPU/memory a container needs (used for scheduling). **Limits** declare the maximum a container may use (enforced at runtime). Together they drive scheduling, QoS class, and resource isolation.

🔷 About

Units

Resource	Unit	Example
CPU	millicores	`100m` = 0.1 vCPU; `1` = 1 vCPU
Memory	bytes	`128Mi` (mebibytes), `1Gi` (gibibytes)
Ephemeral storage	bytes	`500Mi`

Requests vs Limits

Property	Request	Limit
Purpose	Scheduling guarantee	Runtime ceiling
Scheduler uses	Yes — pod fits on a node only if `allocatable - sum(requests)` ≥ this pod's request	No
Enforced by kernel	CPU: no hard cap (CFS shares)	CPU: CFS quota throttling; Memory: OOM-killed if exceeded
Default if omitted	0 (BestEffort)	Unlimited

QoS (Quality of Service) classes

Class	Condition	Eviction priority
Guaranteed	All containers have `requests == limits` for CPU **and** memory	Last to evict
Burstable	At least one container has a request set, but not all match limits	Middle
BestEffort	No requests or limits at all	First to evict

LimitRange (per namespace defaults/constraints)

Forces containers to get default requests/limits if none specified, and caps maximums.

ResourceQuota (per namespace totals)

Caps aggregate resources consumed by all pods/PVCs in a namespace: total CPU requests, total memory limits, total PVC count, etc.

Overcommit and the bin-packing trap

- Requests can be < limits → node can be "overcommitted" (sum of limits > capacity).
- Under pressure, the kernel OOM-kills containers that exceeded limits, and kubelet evicts pods above requests that don't fit in memory.
- Production rule of thumb: set requests to P50 usage, limits to P99 + headroom.

CPU throttling

When a container hits its CPU limit, the kernel's CFS (Completely Fair Scheduler) throttles it — the container pauses mid-cycle. Visible as high `container_cpu_cfs_throttled_seconds_total` in Prometheus.

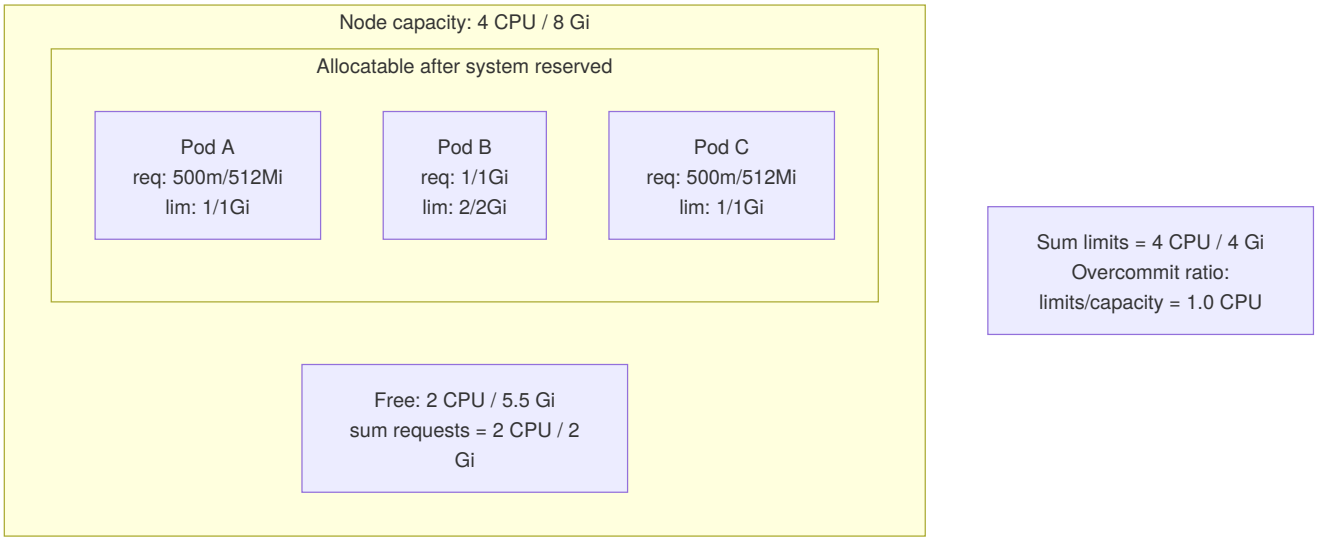
🔷 How It Integrates With Other Kubernetes Components

Component	Interaction
Scheduler	Uses requests for fit/score (NodeResourcesFit, NodeResourcesBalancedAllocation)
kubelet	Enforces limits via cgroups (v1 or v2)
HPA	Scales based on utilization = actual / request
VPA	Recommends request/limit values
LimitRange	Admission defaulting/capping
ResourceQuota	Admission rejecting if quota exceeded
PriorityClass	Higher-priority pods can preempt lower ones when resources are scarce

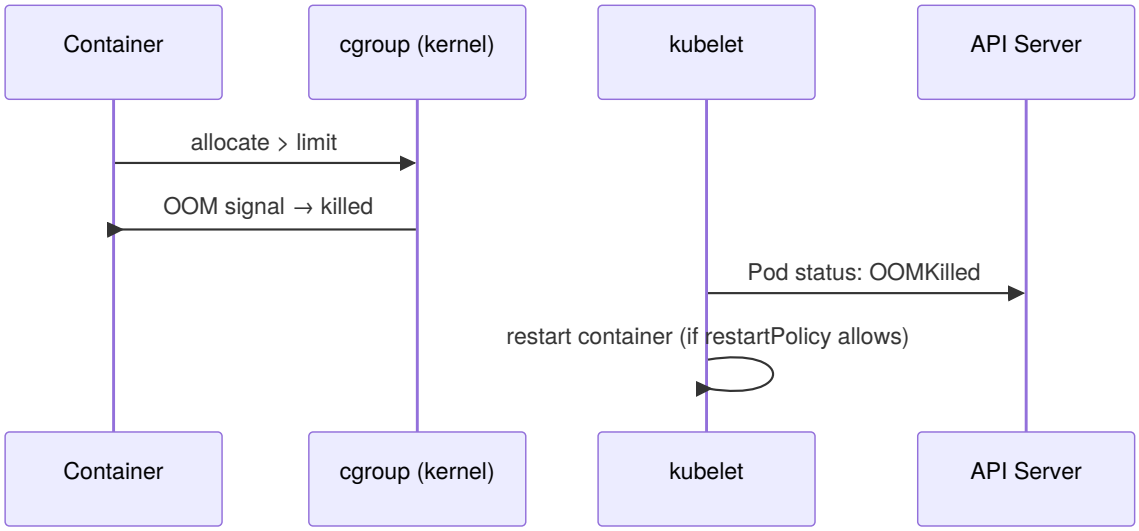
---

🏗 Diagrams

How requests vs limits map to the node



OOM sequence



💻 Real-World Examples

Example 1 — Deployment with proper resources

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: api }
spec:
  replicas: 3
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api } }
    spec:
      containers:
      - name: api
        image: myrepo/api:2.0
        resources:
          requests: { cpu: 250m, memory: 256Mi }
          limits:   { cpu: 1,    memory: 512Mi }
```

Example 2 — LimitRange defaults

```
apiVersion: v1
kind: LimitRange
metadata: { name: defaults, namespace: dev }
spec:
  limits:
  - type: Container
    default:        { cpu: 500m, memory: 256Mi }      # default limit
    defaultRequest: { cpu: 100m, memory: 128Mi }      # default request
    max:            { cpu: 2,    memory: 2Gi }
    min:            { cpu: 50m,  memory: 64Mi }
```

Example 3 — ResourceQuota

```
apiVersion: v1
kind: ResourceQuota
metadata: { name: team-quota, namespace: dev }
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    pods: "40"
    persistentvolumeclaims: "10"
```

```
kubectl describe quota team-quota -n dev
```

Example 4 — Guaranteed QoS

```
resources:
  requests: { cpu: 500m, memory: 512Mi }
  limits:   { cpu: 500m, memory: 512Mi }    # requests == limits
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl top pod`	`--containers`	Live CPU/mem	`kubectl top pod -n prod --containers`
`kubectl top node`	—	Node CPU/mem	`kubectl top node`
`kubectl describe node`	—	Allocatable, requests/limits totals	`kubectl describe node worker-1`
`kubectl describe pod`	—	See QoS, resource fields	`kubectl describe pod api-0`
`kubectl set resources`	`deploy/<n> -c <c> --requests=... --limits=...`	Patch resources	`kubectl set resources deploy/api -c api --requests=cpu=200m,memory=256Mi --limits=cpu=1,memory=512Mi`
`kubectl describe quota`	`-n <ns>`	Check quota usage	`kubectl describe quota team-quota -n dev`
`kubectl describe limitrange`	`-n <ns>`	Check defaults	`kubectl describe limitrange defaults -n dev`
`kubectl get pod -o jsonpath`	`'{.spec.containers[*].resources}'`	Extract resources	`kubectl get pod api-0 -o jsonpath='{.spec.containers[*].resources}'`

🚨 Scenarios

Scenario 1 — Pod stuck Pending: "Insufficient cpu"

- **Cause**: sum of existing requests + this pod's request exceeds node allocatable.
- **Fix**: add nodes, reduce requests, or delete idle workloads.

Scenario 2 — Container OOMKilled repeatedly

- **Cause**: memory limit too low or memory leak.
- **Diagnose**: `kubectl describe pod` → `Last State: Terminated, Reason: OOMKilled`. `kubectl top pod --containers`.
- **Fix**: raise limit (after confirming leak isn't the issue), or fix app leak.

Scenario 3 — API latency spikes at high load

- **Cause**: CPU throttling — `container_cpu_cfs_throttled_seconds_total` is high.
- **Fix**: raise CPU limit, or remove CPU limit entirely (set request only) and rely on CFS shares. This is a valid production pattern if nodes aren't contended.

Scenario 4 — ResourceQuota blocks new deploys

- **Cause**: quota exhausted.
- **Fix**: `kubectl describe quota` → identify used vs hard. Right-size existing workloads or increase quota.

Scenario 5 — VPA recommends lower requests but dev won't accept

- **Cause**: cultural inertia — "better safe than sorry".
- **Fix**: show Prometheus data for P99 usage. Start VPA in `Off` mode (recommend only), apply gradually.

☁ GCP GKE Perspective

- **Autopilot** enforces requests on every pod. If you omit them, Autopilot applies defaults (0.5 CPU, 512 Mi). Billing is per pod-second of requested resources.
- **Autopilot** has minimum pod sizes: 0.25 vCPU, 512 Mi memory.
- **Standard** has no enforcement by default; use LimitRange.
- **GKE Managed Prometheus** scrapes `container_cpu_cfs_throttled_*` automatically.

- **GKE VPA** recommendation is built-in: enable in `gcloud container clusters update`.

▩ AWS EKS Perspective

- **Fargate** rounds up pod resource requests to nearest Fargate configuration (e.g., 0.25/0.5/1/2/4 vCPU with memory buckets). Always set requests explicitly.
- **Managed NG**: no enforcement by default; use LimitRange + ResourceQuota.
- **CloudWatch Container Insights** exposes `pod_cpu_utilization`, `pod_memory_utilization` for right-sizing.
- **Karpenter** uses pod resource requests to select the cheapest instance type that fits.

🧪 Hands-On Lab Exercises

Lab 16.1 — Trigger OOMKill and throttling (Beginner, 30 min)

Steps

```
# oom.yaml
apiVersion: v1
kind: Pod
metadata: { name: oom }
spec:
  containers:
  - name: stress
    image: polinux/stress
    resources:
      requests: { memory: 50Mi, cpu: 100m }
      limits:   { memory: 100Mi, cpu: 200m }
    command: ["stress","--vm","1","--vm-bytes","250M","--vm-hang","1"]
```

```
kubectl apply -f oom.yaml
kubectl get pod oom -w               # OOMKilled → CrashLoopBackOff
kubectl describe pod oom | grep -A3 "Last State"
```

Cleanup: `kubectl delete pod oom`.

Bonus: Set `--vm-bytes 80M` → pod runs fine; confirm with `kubectl top pod oom`.

Lab 16.2 — LimitRange + ResourceQuota enforcement (Intermediate, 45 min)

Steps 1. Create namespace `lab16`: `kubectl create ns lab16`. 2. Apply LimitRange (Example 2) and ResourceQuota (Example 3) to `lab16`. 3. Deploy a pod without resources → LimitRange injects defaults. 4. Deploy pods until quota exhausted → admission rejects. 5. `kubectl describe quota -n lab16` → see used vs hard.

Cleanup: `kubectl delete ns lab16`.

Lab 16.3 — VPA recommendation (Advanced, 60 min)

Steps 1. Install VPA: `kubectl apply -f https://github.com/kubernetes/autoscaler/releases/latest/download/vpa-v1-crd-gen.yaml && kubectl apply -f https://github.com/kubernetes/autoscaler/releases/latest/download/vpa-rbac.yaml`. 2. Deploy a web app with initial `requests: {cpu: 1, memory: 1Gi}`. 3. Create VPA in `Off` mode:

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: web-vpa }
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: web }
  updatePolicy: { updateMode: "Off" }
```

4. Generate load with `hey -z 120s http://<svc>/`. 5. `kubectl describe vpa web-vpa` → read recommendations. 6. Apply recommended values.

Cleanup: delete VPA + Deployment.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Requests declare the floor a pod needs for scheduling — the scheduler ensures those resources are available on the node. Limits set the ceiling — the kernel throttles CPU and OOM-kills memory when a container exceeds them. Together they determine the pod's QoS class: Guaranteed when requests equal limits, Burstable when only requests are set, BestEffort with neither.
Why: Without requests, the scheduler packs pods blindly and nodes thrash under contention. Without limits, a single runaway container can starve the node. The QoS class drives eviction priority: BestEffort dies first, Guaranteed last.
How: I set requests to P50 observed usage and limits to P99 plus a 20% buffer, and validate with Prometheus metrics and VPA in Off mode before shrinking. I enforce LimitRange per namespace for defaults and ResourceQuota for team-level budgets. On GKE Autopilot requests are mandatory, and on EKS Fargate they're rounded to the nearest billing bucket — so I always specify them explicitly.
Example: We reduced our cluster from 80 to 50 nodes after a right-sizing sprint — VPA showed most services were requesting 4× their P99. Savings: ~$12K/month."

? Possible Interview Questions

Basic

Q1. Request vs limit? Request = scheduling guarantee; limit = runtime cap.
Q2. What happens when memory exceeds the limit? OOMKilled by the kernel.
Q3. CPU unit `250m` means? 0.25 vCPU (one quarter of a core).

Intermediate

Q4. Name the three QoS classes. Guaranteed, Burstable, BestEffort.
Q5. What is CPU throttling? CFS quota restricts a container's CPU cycles per period; visible as high `container_cpu_cfs_throttled_seconds_total`.
Q6. What does a LimitRange do? Sets per-container default requests/limits, min/max constraints, and per-PVC min/max in a namespace.

Advanced

Q7. Should you always set CPU limits? Debated. Setting limits prevents runaway but causes throttling even when the node has free CPU. Some teams set requests only (rely on CFS shares). Google's guidance for Autopilot: limits required.
Q8. How does the scheduler use requests? Two phases — filtering nodes where `allocatable - existingRequests >= podRequests`, then scoring nodes by resource balance.
Q9. Explain eviction order under memory pressure. BestEffort first, then Burstable (by how far they exceed request), then Guaranteed (only if system reserved is at risk).

Scenario-based

Q10. Cluster is 90% allocated but `kubectl top` shows 30% actual usage — what's wrong? Over-requested. Requests are high relative to actual. Right-size with VPA; reclaim with Cluster Autoscaler scale-down.

GKE-specific

Q11. What are Autopilot's minimum pod resources? 0.25 vCPU, 512 Mi memory (as of 2024+). Smaller requests are rounded up.

EKS-specific

Q12. Why does Fargate sometimes bill more than expected? It rounds to the nearest Fargate pod size (0.25/0.5/1/2/4 CPU). A pod requesting 0.3 vCPU is billed at 0.5 vCPU.

Hands-on / Lab-based

Q13. How would you observe CPU throttling? Query Prometheus: `rate(container_cpu_cfs_throttled_seconds_total[5m])`.
Q14. Show me how VPA works. Deploy app, create VPA in Off mode, generate load, `kubectl describe vpa` → read `Lower Bound / Target / Upper Bound` recommendations.
Q15. How would you prevent a team from deploying without resource requests? ResourceQuota with `requests.cpu` set → admission rejects pods without requests. Or Kyverno/OPA policy explicitly validating the field.

🗐 Topic 17 — Namespaces

Official Documentation Reference - Namespaces: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ - Share a cluster with namespaces: https://kubernetes.io/docs/tasks/administer-cluster/namespaces/ - Namespaces walkthrough: https://kubernetes.io/docs/tutorials/cluster-management/namespaces-walkthrough/ - GKE team-based multi-tenancy: https://cloud.google.com/kubernetes-engine/docs/concepts/multitenancy-overview - EKS multi-tenancy: https://docs.aws.amazon.com/eks/latest/best-practices/multi-tenancy.html

🔹 Definition

A **Namespace** is a virtual cluster inside a physical Kubernetes cluster. It provides a scope for names, RBAC, resource quotas, network policies, and limit ranges — enabling multi-tenancy on a single cluster.

🔹 About

Default namespaces

Namespace	Purpose
`default`	Where objects land when no namespace is given
`kube-system`	Control-plane + cluster-critical add-ons
`kube-public`	Publicly readable (cluster-info ConfigMap)
`kube-node-lease`	Node heartbeat Lease objects

Namespaced vs cluster-scoped resources

Namespaced	Cluster-scoped
Pod, Service, Deployment, ConfigMap, Secret, PVC, Role, RoleBinding, Ingress, NetworkPolicy	Node, PV, ClusterRole, ClusterRoleBinding, Namespace, StorageClass, IngressClass, PriorityClass, CRD

Check with: `kubectl api-resources --namespaced=true/false`.

What namespaces give you

- **Name isolation** — two Deployments named `web` can coexist in different namespaces.
- **RBAC scoping** — Roles and RoleBindings are namespace-scoped.

- **Resource quotas** — cap CPU, memory, pod count per namespace.
- **LimitRange** — per-container default resources.
- **Network policies** — namespace-level traffic rules.
- **Service DNS** — `svc.ns.svc.cluster.local`.

What namespaces do NOT do

- They are **not** a security boundary — pods in different namespaces can still reach each other by default (until NetworkPolicy is enforced).
- They do **not** isolate workloads at the kernel level (that's what node isolation, sandboxing, or separate clusters do).

Naming conventions

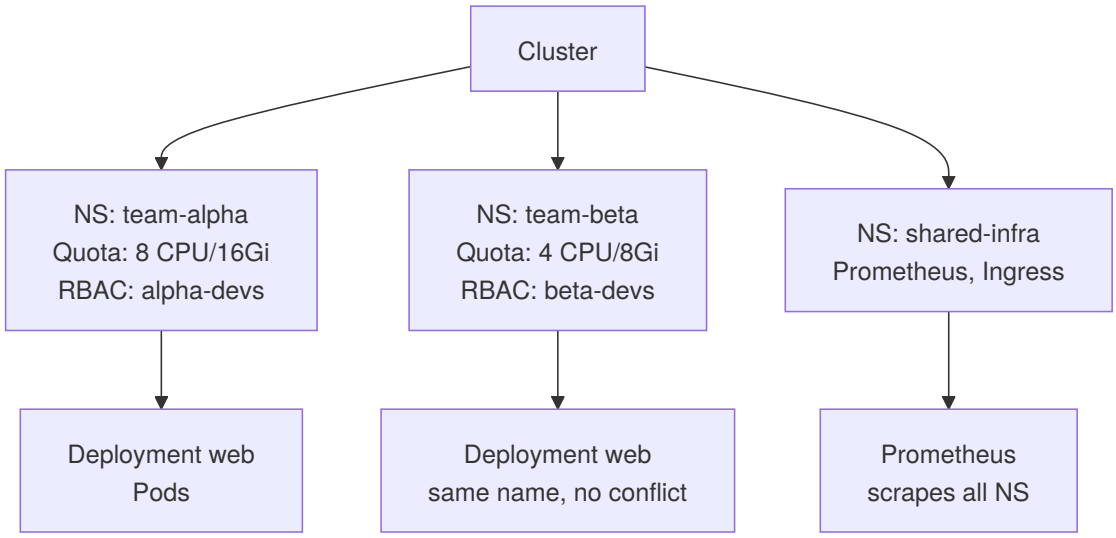
Common patterns: `team-<name>`, `env-<name>` (prod/stg/dev), `app-<name>`.

🔷 How It Integrates With Other Kubernetes Components

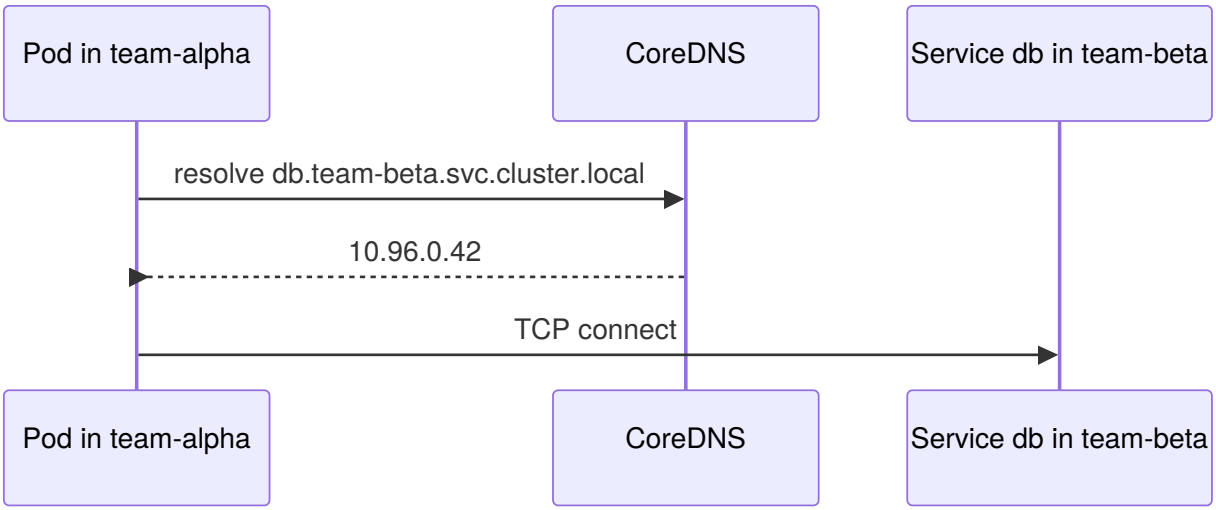
Component	Interaction
RBAC (Role/RoleBinding)	Scoped per namespace
ResourceQuota	Enforces per-NS limits
LimitRange	Defaults per NS
NetworkPolicy	`namespaceSelector` gates cross-NS traffic
Service DNS	`<svc>.<ns>.svc.cluster.local`
Kyverno/OPA	Policies can target specific namespaces
Admission (PSA)	`pod-security.kubernetes.io/*` labels set per NS

🏗 Diagrams

Multi-tenant namespace layout



Cross-namespace DNS



💻 Real-World Examples

Example 1 — Create and use

```
kubectl create namespace staging
kubectl run web --image=nginx --namespace=staging
kubectl get pods -n staging
kubectl config set-context --current --namespace=staging
```

Example 2 — Full tenant namespace setup

```
# 1. Namespace
apiVersion: v1
kind: Namespace
metadata:
  name: team-alpha
  labels:
    pod-security.kubernetes.io/enforce: restricted
    team: alpha
---
# 2. ResourceQuota
apiVersion: v1
kind: ResourceQuota
metadata: { name: quota, namespace: team-alpha }
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    pods: "50"
---
# 3. LimitRange
apiVersion: v1
kind: LimitRange
metadata: { name: defaults, namespace: team-alpha }
spec:
  limits:
  - type: Container
    default:        { cpu: 500m, memory: 256Mi }
    defaultRequest: { cpu: 100m, memory: 128Mi }
---
# 4. RBAC
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: alpha-devs, namespace: team-alpha }
subjects:
- { kind: Group, name: alpha-devs, apiGroup: rbac.authorization.k8s.io }
roleRef: { kind: ClusterRole, name: edit, apiGroup: rbac.authorization.k8s.io }
---
# 5. NetworkPolicy — deny all ingress by default
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all, namespace: team-alpha }
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

Example 3 — kubens for fast switching

```
kubens team-alpha     # all kubectl commands now target team-alpha
kubens -              # switch back
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create ns`	—	Create	`kubectl create ns staging`
`kubectl get ns`	—	List	`kubectl get ns`
`kubectl delete ns`	—	Delete (cascades all objects inside!)	`kubectl delete ns staging`
`kubectl get all`	`-n <ns>`	List core objects	`kubectl get all -n kube-system`
`kubectl get pods`	`-A`	All namespaces	`kubectl get pods -A`
`kubectl config set-context`	`--current --namespace=<ns>`	Change default NS	`kubectl config set-context --current --namespace=prod`
`kubens`	`<ns>`	Fast switch	`kubens prod`
`kubectl api-resources`	`--namespaced=true`	List NS-scoped kinds	`kubectl api-resources --namespaced=true`
`kubectl describe quota`	`-n <ns>`	Check usage	`kubectl describe quota -n team-alpha`
`kubectl label ns`	`key=val`	Add labels (PSA etc)	`kubectl label ns team-alpha pod-security.kubernetes.io/enforce=restricted`

🚨 Scenarios

Scenario 1 — Accidentally deleted a namespace

- **Impact**: all objects inside are deleted (pods, services, secrets, PVCs).
- **Fix**: restore from backup (Velero / Backup for GKE). Prevention: restrict `delete namespaces` RBAC; add finalizers or Kyverno policies.

Scenario 2 — Namespace stuck in `Terminating`

- **Cause**: a finalizer on an object inside is never cleared (often CRDs or stuck PVCs).
- **Fix**: identify stuck finalizers: `kubectl get all,pvc,crd -n <ns>`. Patch the object to remove the finalizer. As a last resort: `kubectl get ns <ns> -o json | jq '.spec.finalizers=[]' | kubectl replace --raw "/api/v1/namespaces/<ns>/finalize" -f -`.

Scenario 3 — Service in NS-A can't reach DB in NS-B

- **Cause**: NetworkPolicy in NS-B blocks cross-NS ingress, or DNS name is wrong.
- **Fix**: Allow from NS-A via `namespaceSelector` in NetworkPolicy, and use FQDN: `db.ns-b.svc.cluster.local`.

Scenario 4 — Team exhausted their quota

- **Cause**: deploying without monitoring quota.
- **Fix**: `kubectl describe quota -n team-x`. Right-size workloads or increase quota after review.

☁ GCP GKE Perspective

- **GKE Enterprise** provides **Policy Controller** (OPA Gatekeeper) to enforce namespace labels, quota presence, and naming.
- **Config Sync** can reconcile namespace YAML from Git across fleet clusters.
- **Autopilot** charges per pod-second; namespaces don't affect billing — just organizational.
- **Hierarchical Namespaces Controller (HNC)** — propagate Roles, NetworkPolicies from parent NS to children (mature on GKE).

▩ AWS EKS Perspective

- **EKS Blueprints** ships opinionated "team" namespaces with quota, LimitRange, RBAC, and ArgoCD AppProjects.
- **Fargate profiles** bind to one or more namespaces/labels — a Fargate profile `selector: [{namespace: team-alpha}]` runs only pods in `team-alpha`.
- **AWS RAM** + multiple EKS clusters is the alternative when namespace isolation is insufficient for compliance.

🧪 Hands-On Lab Exercises

Lab 17.1 — Namespace + quota + LimitRange (Beginner, 30 min)

Steps 1. `kubectl create ns lab17`. 2. Apply ResourceQuota (hard.pods: 5) and LimitRange from Example 2. 3. Deploy 6 replicas → 5th succeeds, 6th rejected by quota. 4. `kubectl describe quota -n lab17`.

Cleanup: `kubectl delete ns lab17`.

Lab 17.2 — Cross-namespace networking + NetworkPolicy (Intermediate, 45 min)

Steps 1. Create `ns-a`, `ns-b`. 2. Deploy `web` in `ns-a` and `db` in `ns-b` (nginx for demo). 3. From `ns-a` pod: `curl db.ns-b.svc.cluster.local` → works. 4. Apply `deny-all` ingress NetworkPolicy in `ns-b`. 5. Retry curl → timeout. 6. Add allow rule with `namespaceSelector: {matchLabels: {team: alpha}}` and label `ns-a` → works again.

Cleanup: `kubectl delete ns ns-a ns-b`.

Lab 17.3 — Stuck Terminating namespace rescue (Advanced, 30 min)

Steps 1. `kubectl create ns stuck`. 2. Apply a custom CRD + CR with a finalizer in `stuck`. 3. `kubectl delete ns stuck` → hangs in Terminating. 4. Identify stuck

object: `kubectl get all -n stuck`. 5. Patch finalizer: `kubectl patch <obj> -n stuck -p '{"metadata":{"finalizers":[]}}' --type=merge`. 6. Namespace terminates.

Bonus: Investigate the `replace --raw finalize` escape hatch.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Namespace is a virtual cluster partition inside one physical Kubernetes cluster. It scopes object names, RBAC, resource quotas, limit ranges, network policies, and Pod Security Standards.
Why: On a shared cluster, namespaces are how we enforce soft multi-tenancy — each team gets its own namespace with a budget (ResourceQuota), guardrails (LimitRange, NetworkPolicy), and least-privilege access (RoleBinding). Service DNS is namespace-aware, so teams can both name their services 'web' without conflict.
How: I provision each team's namespace as a bundle: Namespace + ResourceQuota + LimitRange + RoleBinding + default-deny NetworkPolicy + PSA label. I enforce this with Config Sync on GKE or ArgoCD on EKS so every namespace is Git-managed. Cross-namespace traffic is explicit — allowed only by NetworkPolicy rules with namespaceSelector.
Example: When our platform grew from 2 to 8 teams, we replaced separate clusters with one large GKE Standard cluster, one namespace per team, each with its own quota. Savings were ~40% — instead of 8 idle control planes, we had one, with full RBAC isolation."

? Possible Interview Questions

Basic

Q1. What is a namespace? Virtual cluster partition scoping names, RBAC, quotas, and policies.
Q2. Name the four default namespaces. `default`, `kube-system`, `kube-public`, `kube-node-lease`.
Q3. Are nodes namespaced? No — they are cluster-scoped.

Intermediate

Q4. How does namespace isolation differ from network isolation? Namespaces only scope API objects. Pods in different namespaces can still network freely unless NetworkPolicy enforces isolation.
Q5. How do you set the default namespace for kubectl? `kubectl config set-context --current --namespace=<n>` or `kubens <n>`.
Q6. What is PSA per namespace? Labels like `pod-security.kubernetes.io/enforce: restricted` activate Pod Security Admission at the namespace level.

Advanced

Q7. How do you prevent accidental namespace deletion? Restrict `delete` verb on `namespaces` via RBAC; add a Kyverno policy blocking deletion if pods exist; or use a finalizer.
Q8. What is HNC? Hierarchical Namespace Controller — lets a parent namespace propagate Roles, NetworkPolicies, Secrets down to child namespaces.
Q9. Namespace stuck Terminating — what's happening? An object inside has a finalizer that was never removed. Find it, clear the finalizer, and the namespace controller completes deletion.

Scenario-based

Q10. Two teams share a cluster but one team's batch jobs starve the other — fix? ResourceQuota per namespace caps total requests; PriorityClass to preempt low-priority batch; separate node pools with taints per team.

GKE-specific

Q11. How do Fargate profiles relate to namespaces on EKS? A Fargate profile selector binds to a namespace + optional labels; pods matching run on Fargate, others on managed NG.

EKS-specific

Q12. How does GKE Config Sync help namespace management? It reconciles namespace manifests from a Git repo across a fleet of GKE clusters — any drift is corrected automatically.

Hands-on / Lab-based

Q13. Show me cross-namespace DNS. From a pod in `ns-a`: `nslookup db.ns-b.svc.cluster.local` resolves to the ClusterIP.
Q14. How would you carve up a cluster for 5 teams? 5 namespaces, each with quota + LimitRange + RoleBinding + NetworkPolicy + PSA label, provisioned from Git.
Q15. How do you list all resources in a namespace? `kubectl api-resources --verbs=list --namespaced -o name | xargs -n1 kubectl get -n <ns> --ignore-not-found`.

📄 Topic 18 — Environment Variables and Downward API

Official Documentation Reference - Define env: https://kubernetes.io/docs/tasks/inject-data-application/define-environment-variable-container/ - Dependent env vars: https://kubernetes.io/docs/tasks/inject-data-application/define-interdependent-environment-variables/ - Downward API (env): https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/ - Downward API (volume): https://kubernetes.io/docs/tasks/inject-data-application/downward-api-volume-expose-pod-information/ - Downward API concept: https://kubernetes.io/docs/concepts/workloads/pods/downward-api/ - GKE workload config: https://cloud.google.com/kubernetes-engine/docs/how-to/configuring-workloads - EKS env vars: https://docs.aws.amazon.com/eks/latest/userguide/workloads.html

---

🔹 Definition

Environment variables in Kubernetes inject configuration into containers at startup. The **Downward API** is a mechanism that exposes pod and container metadata — name, namespace, labels, annotations, resource requests/limits, node name — as environment variables or files, without calling the API server.

🔹 About

Ways to set env vars

Source	Field	Example
Static literal	`env[].value`	`DB_HOST: postgres`
ConfigMap key	`env[].valueFrom.configMapKeyRef`	one key from a CM
Secret key	`env[].valueFrom.secretKeyRef`	one key from a Secret
All ConfigMap keys	`envFrom[].configMapRef`	bulk import
All Secret keys	`envFrom[].secretRef`	bulk import
Downward API (pod field)	`env[].valueFrom.fieldRef`	`metadata.name`, `metadata.namespace`, `spec.nodeName`
Downward API (resource)	`env[].valueFrom.resourceFieldRef`	`requests.cpu`, `limits.memory`
Dependent interpolation	`$(VAR)` in `value`	`URL: http://$(DB_HOST):5432`

Available downward API fields

Via `fieldRef.fieldPath` - `metadata.name` — pod name - `metadata.namespace` — pod namespace - `metadata.uid` — pod UID - `metadata.labels['key']` — specific label - `metadata.annotations['key']` — specific annotation - `spec.nodeName` — node name - `spec.serviceAccountName` — SA name - `status.podIP` — pod IP - `status.hostIP` — node IP

Via `resourceFieldRef` - `requests.cpu`, `limits.cpu`, `requests.memory`, `limits.memory`, `requests.ephemeral-storage`, `limits.ephemeral-storage`

Volume-based Downward API

Mount all labels/annotations/fields as files under a directory. Files update live (unlike env vars).

Limitations

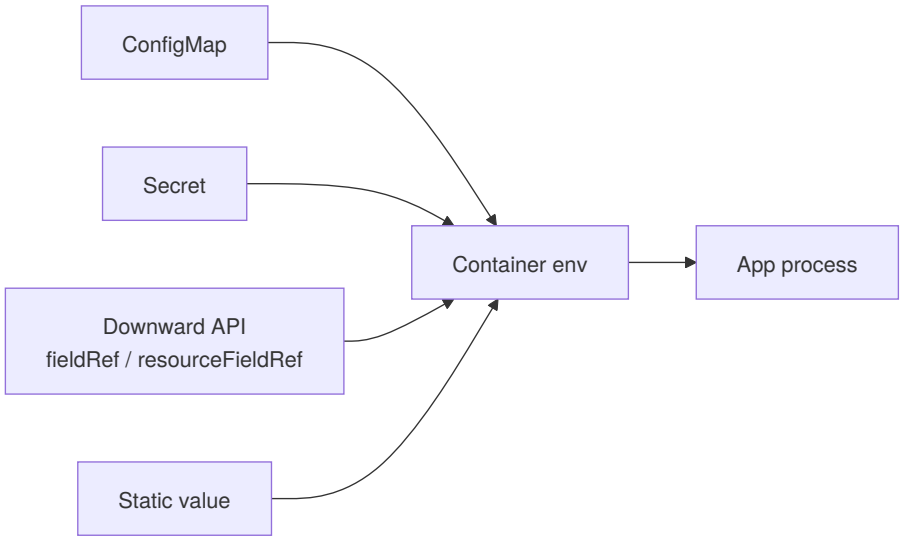
- Env vars are set at container start and never refreshed.
- Env vars from ConfigMaps/Secrets are snapshots. For live updates, use volume mounts.
- Env var name collisions: last source wins (order matters).
- `envFrom` skips keys that aren't valid env-var names (e.g., containing dots).

🔹 How It Integrates With Other Kubernetes Components

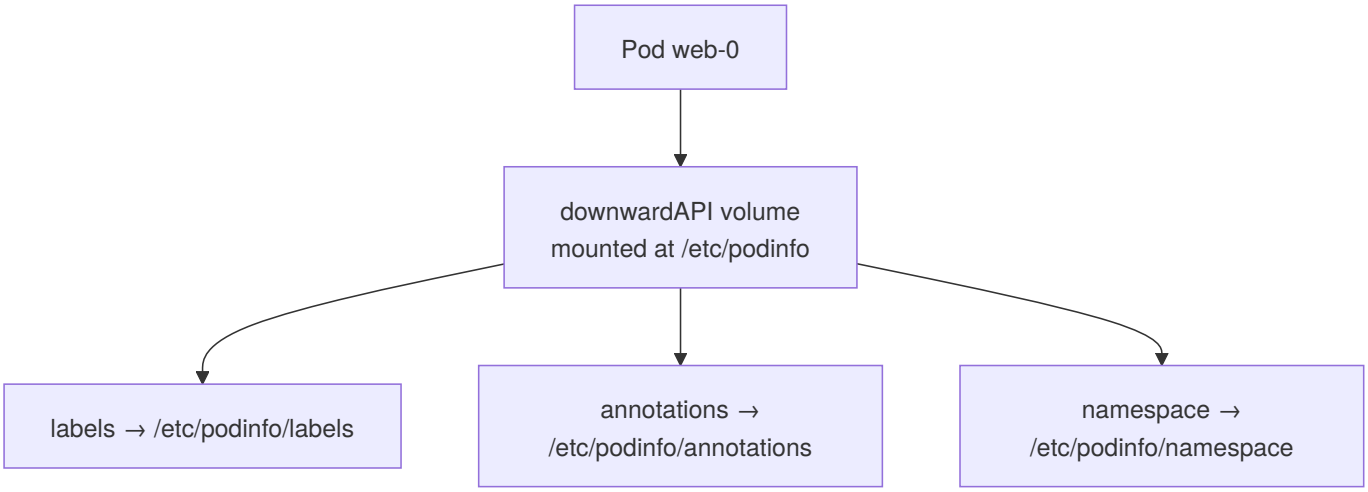
Component	Interaction
ConfigMap / Secret	Most common env sources
Downward API	Injects pod/container metadata
kubelet	Resolves `fieldRef` at pod start
Init containers / sidecars	Same env mechanisms
HPA custom metrics	App exporting its own pod-name label for Prometheus
Logging	`HOSTNAME` / `POD_NAME` env vars enrich log lines

🏗 Diagrams

Env var resolution flow



Downward API volume



💻 Real-World Examples

Example 1 — All env var sources in one pod

```
apiVersion: v1
kind: Pod
metadata:
  name: env-demo
  labels: { app: demo, version: "2.0" }
  annotations: { owner: team-alpha }
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","env | sort; sleep 3600"]
    env:
    # static
    - { name: APP_ENV, value: production }
    # from ConfigMap key
    - name: LOG_LEVEL
      valueFrom: { configMapKeyRef: { name: app-config, key: LOG_LEVEL } }
    # from Secret key
    - name: DB_PASSWORD
      valueFrom: { secretKeyRef: { name: db-cred, key: PASSWORD } }
    # Downward API — pod fields
    - name: POD_NAME
      valueFrom: { fieldRef: { fieldPath: metadata.name } }
    - name: POD_NAMESPACE
      valueFrom: { fieldRef: { fieldPath: metadata.namespace } }
    - name: NODE_NAME
      valueFrom: { fieldRef: { fieldPath: spec.nodeName } }
    - name: POD_IP
      valueFrom: { fieldRef: { fieldPath: status.podIP } }
    # Downward API — resources
    - name: CPU_LIMIT
      valueFrom: { resourceFieldRef: { containerName: app, resource: limits.cpu } }
    - name: MEM_REQUEST
      valueFrom: { resourceFieldRef: { containerName: app, resource: requests.memory, divisor: "1Mi" } }
    # dependent
    - { name: DB_HOST, value: postgres.default.svc }
    - name: DB_URL
      value: "postgres://app:$(DB_PASSWORD)@$(DB_HOST):5432/prod"
    resources:
      requests: { cpu: 250m, memory: 128Mi }
      limits:   { cpu: 500m, memory: 256Mi }
```

Example 2 — Downward API volume

```
apiVersion: v1
kind: Pod
metadata:
  name: dw-vol
  labels: { app: web, version: "3" }
  annotations: { owner: team-alpha }
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","cat /etc/podinfo/labels; sleep 3600"]
    volumeMounts:
    - { name: podinfo, mountPath: /etc/podinfo }
  volumes:
  - name: podinfo
    downwardAPI:
      items:
      - { path: labels,      fieldRef: { fieldPath: metadata.labels } }
      - { path: annotations,  fieldRef: { fieldPath: metadata.annotations } }
      - { path: namespace,    fieldRef: { fieldPath: metadata.namespace } }
      - { path: cpu-request,  resourceFieldRef: { containerName: app, resource: requests.cpu, divisor: "1m" } }
```

Example 3 — envFrom bulk import

```
envFrom:
- configMapRef: { name: app-config }
- secretRef:    { name: db-cred }
- prefix: CFG_                    # add prefix to all imported keys
  configMapRef: { name: app-config }
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl set env`	`deploy/<n> K=V`	Set static env → triggers rollout	`kubectl set env deploy/web LOG=debug`
`kubectl set env`	`--from=configmap/<n>`	Bulk set from CM	`kubectl set env deploy/web --from=configmap/app-config`
`kubectl set env`	`--list`	Show env vars of a pod template	`kubectl set env deploy/web --list`
`kubectl exec`	`-- env`	Print runtime env	`kubectl exec web-0 -- env | sort`
`kubectl exec`	`-- printenv VAR`	Read one var	`kubectl exec web-0 -- printenv POD_NAME`
`kubectl describe pod`	—	See env section	`kubectl describe pod web-0`

🚨 Scenarios

Scenario 1 — Env var from ConfigMap is stale

- **Cause**: env vars are snapshots at container start.

- **Fix**: `kubectl rollout restart deploy/<n>` or switch to volume mount + file watcher.

Scenario 2 — `envFrom` silently skips keys

- **Cause**: key contains invalid env var chars (e.g., `my.key` with a dot).
- **Fix**: rename the key or use volume mount.

Scenario 3 — Dependent env var is empty

- **Cause**: `$(DB_PASSWORD)` references a var that appears later in the list (ordering matters).
- **Fix**: place the source var before the dependent var in the `env` array.

Scenario 4 — App doesn't see `POD_IP` at startup

- **Cause**: (rare) IP not assigned yet. This is resolved for almost all modern runtimes.
- **Fix**: if needed, use Downward API volume (file-based) which can update.

☁ GCP GKE Perspective

- **Autopilot** injects `requests.cpu` / `limits.memory` automatically if not set. The Downward API `resourceFieldRef` reflects the actual (possibly adjusted) values.
- **Workload Identity env vars**: the `GKE_METADATA_SERVER` and projected SA token are managed by GKE — no manual env needed.
- **Config Connector**: CRDs that provision GCP resources; their status can feed back as ConfigMap env vars.

▩ AWS EKS Perspective

- **IRSA** auto-injects `AWS_WEB_IDENTITY_TOKEN_FILE` and `AWS_ROLE_ARN` into pods with annotated SAs — no manual env.
- **Pod Identity** similarly injects `AWS_CONTAINER_CREDENTIALS_FULL_URI`.
- **Fargate**: Downward API works normally, including `spec.nodeName` (which shows the fargate-node name).

🧪 Hands-On Lab Exercises

Lab 18.1 — Explore all env var sources (Beginner, 30 min)

Steps 1. Create ConfigMap `app-config` and Secret `db-cred`. 2. Apply Example 1 pod. 3. `kubectl logs env-demo | sort` → see every resolved env var. 4. `kubectl exec env-demo -- printenv DB_URL` → verify interpolated URL.

Cleanup: `kubectl delete pod env-demo && kubectl delete cm app-config && kubectl delete secret db-cred`.

Bonus: Change `LOG_LEVEL` in the ConfigMap; verify `env-demo` still shows old value (snapshot).

Lab 18.2 — Downward API volume with live labels (Intermediate, 45 min)

Steps 1. Apply Example 2 pod. 2. `kubectl exec dw-vol -- cat /etc/podinfo/labels` → `app=web, version=3`. 3. `kubectl label pod dw-vol canary=true`. 4. Wait ~60 s, then `kubectl exec dw-vol -- cat /etc/podinfo/labels` → `canary=true` appears. 5. `kubectl exec dw-vol -- cat /etc/podinfo/cpu-request` → see millicores.

Cleanup: `kubectl delete pod dw-vol`.

Bonus: Compare this to env-based Downward API — label the pod, observe that `POD_LABEL_CANARY` env var doesn't exist (env vars are fixed at start).

Lab 18.3 — Self-registering app using Downward API (Advanced, 60 min)

Objective: App reads its own pod name, namespace, IP, and resource limits to register with a service registry or tag its Prometheus metrics.

Steps 1. Write a tiny Python HTTP server that prints all `POD_*` env vars as JSON. 2. Dockerfile + deploy. 3. Set Downward API env vars: `POD_NAME`, `POD_NAMESPACE`, `POD_IP`, `CPU_LIMIT`, `MEM_LIMIT`. 4. Scale to 5 replicas; curl each pod's `http://<pod-ip>:8080/info`. 5. Verify each reports unique name/IP but same resource limits.

Bonus: Feed these into a Prometheus counter label via `OTEL_RESOURCE_ATTRIBUTES=k8s.pod.name=$(POD_NAME)`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Environment variables are the main way to inject configuration into a Kubernetes container at startup. Sources include static values, ConfigMaps, Secrets, and the Downward API — which exposes pod metadata like pod name, namespace, node name, labels, annotations, and resource requests/limits without calling the API server.
Why: The twelve-factor pattern says config belongs in the environment. Kubernetes takes it further with `envFrom` for bulk imports, `$(VAR)` interpolation for building URLs, and the Downward API so the app knows its own identity for logging, metrics, and service registration — all declaratively, no code changes.
How: For simple key-values I use `envFrom` pointing to a ConfigMap. For sensitive data, `secretKeyRef`. For self-awareness — pod name, IP, resource limits — I use Downward API `fieldRef` and `resourceFieldRef`. When I need live-updating metadata (like labels changing after start), I use a Downward API volume mount, not env.
Example: On our tracing setup, every pod exports `OTEL_RESOURCE_ATTRIBUTES` built from `POD_NAME`, `POD_NAMESPACE`, and `NODE_NAME` via Downward API, so traces carry Kubernetes context without the app importing any K8s library."

? Possible Interview Questions

Basic

Q1. How do you set an env var from a ConfigMap? `env[].valueFrom.configMapKeyRef: {name: cm, key: K}`.
Q2. What is the Downward API? Mechanism to expose pod/container metadata as env vars or files, without API calls.
Q3. Do env vars update when a ConfigMap changes? No — they're set at container start. Need pod restart.

Intermediate

Q4. Name 5 fields available via `fieldRef`. `metadata.name`, `metadata.namespace`, `status.podIP`, `spec.nodeName`, `metadata.uid`.
Q5. Difference between env-based and volume-based Downward API? Env: snapshot at start. Volume: files that update live.
Q6. What is `$(VAR)` interpolation? Kubernetes expands `$(VAR)` in `value` fields using previously defined env vars. Order in the `env` array matters.

Advanced

Q7. Why does `envFrom` skip some keys? Keys that aren't valid POSIX env var names (contain dots, start with digits) are silently skipped. Events note the skip.
Q8. How do you expose resource limits to an app? `resourceFieldRef: {resource: limits.memory, divisor: "1Mi"}` → gives the value in MiB as an integer string.
Q9. Can you read arbitrary API objects via Downward API? No — only the pod's own metadata and container resource fields. For anything else, call the API server or use a sidecar.

Scenario-based

Q10. Your Java app ignores `-Xmx` because it reads heap from env, but the CM env var wasn't refreshed after a change. Fix? Rollout restart to pick up the new CM value; or mount the CM as a file and configure the JVM to read it at startup.

GKE-specific

Q11. Does Autopilot adjust resourceFieldRef values if it rounds up pod resources? Yes — the Downward API reflects the actual (possibly adjusted) requests/limits.

EKS-specific

Q12. IRSA injects env vars automatically — which ones? `AWS_WEB_IDENTITY_TOKEN_FILE` and `AWS_ROLE_ARN`.

Hands-on / Lab-based

Q13. Show me how to verify Downward API is working. `kubectl exec <pod> -- printenv POD_NAME` or `cat /etc/podinfo/labels`.
Q14. If you label a pod after it starts, does the env-based Downward API reflect it? No — env is fixed. Volume-based will update within ~1 minute.
Q15. How would you build a connection string from multiple sources? Use dependent env vars: `DB_URL: postgres://$(DB_USER):$(DB_PASS)@$(DB_HOST):5432/$(DB_NAME)`.

MODULE

Module 4 — Networking

🗐 Topic 19 — Kubernetes Networking Model

Official Documentation Reference - Cluster networking: https://kubernetes.io/docs/concepts/cluster-administration/networking/ - Services, load balancing, networking: https://kubernetes.io/docs/concepts/services-networking/ - DNS for Services and Pods: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/ - Network plugins: https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/ - GKE networking overview: https://cloud.google.com/kubernetes-engine/docs/concepts/network-overview - EKS networking: https://docs.aws.amazon.com/eks/latest/userguide/pod-networking.html

🔹 Definition

The Kubernetes networking model requires that every **Pod gets its own IP address**, all pods can communicate with all other pods without NAT, all nodes can communicate with all pods without NAT, and the IP a pod sees for itself is the same IP others see. This is called the **flat network** model.

🔹 About

The four networking problems K8s solves

Problem	Solution
Container-to-container (same pod)	`localhost` (shared network namespace)
Pod-to-pod (same/different node)	CNI plugin provides flat L3 network
Pod-to-Service	kube-proxy / eBPF — ClusterIP virtual IP
External-to-Service	NodePort / LoadBalancer / Ingress

The CNI contract

Kubernetes delegates the IP assignment and packet forwarding to a **CNI (Container Network Interface)** plugin. K8s itself does not implement any networking — it defines the rules, the plugin implements them.

How pod-to-pod works across nodes

Implementation	How
Overlay (VXLAN/GENEVE)	Encapsulate pod packets in outer UDP; works anywhere but adds latency/overhead. Calico tunnel, Flannel VXLAN.
BGP routing	CNI announces pod CIDR routes via BGP; no encapsulation. Calico native mode.
Cloud-native routing	Provider programs VPC routes / ENIs. GKE Dataplane V2 (eBPF), VPC CNI on EKS.

Key CIDRs

CIDR	Controlled by	Holds
Pod CIDR	`--pod-network-cidr` / CNI	Pod IPs
Service CIDR	`--service-cluster-ip-range`	ClusterIP range
Node CIDR	Cloud VPC / bare-metal	Node IPs

kube-proxy modes

Mode	Mechanism	Use case
iptables	iptables DNAT rules per Service	Default on most clusters
IPVS	kernel IPVS tables (hashing)	High service count (>1000 services)
nftables	nftables (K8s 1.29+ stable)	Modern kernel replacement for iptables
eBPF (cilium, GKE DP-v2)	Replaces kube-proxy entirely	Best performance, visibility

DNS (CoreDNS)

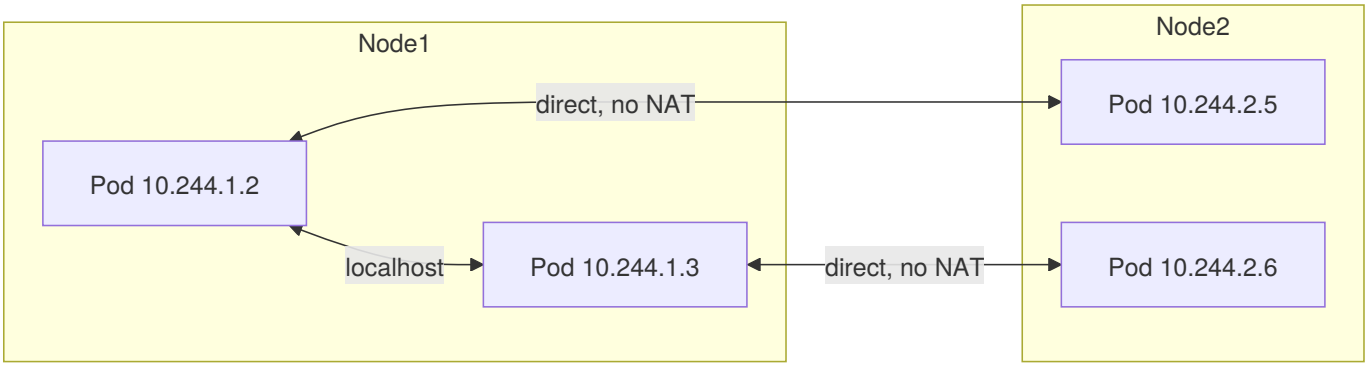
- Every Service gets `<svc>.<ns>.svc.cluster.local`.
- Pods get `<pod-ip-dashed>.<ns>.pod.cluster.local`.
- Headless Services return per-pod A records.
- CoreDNS config in ConfigMap `coredns` in `kube-system`.

🔹 How It Integrates With Other Kubernetes Components

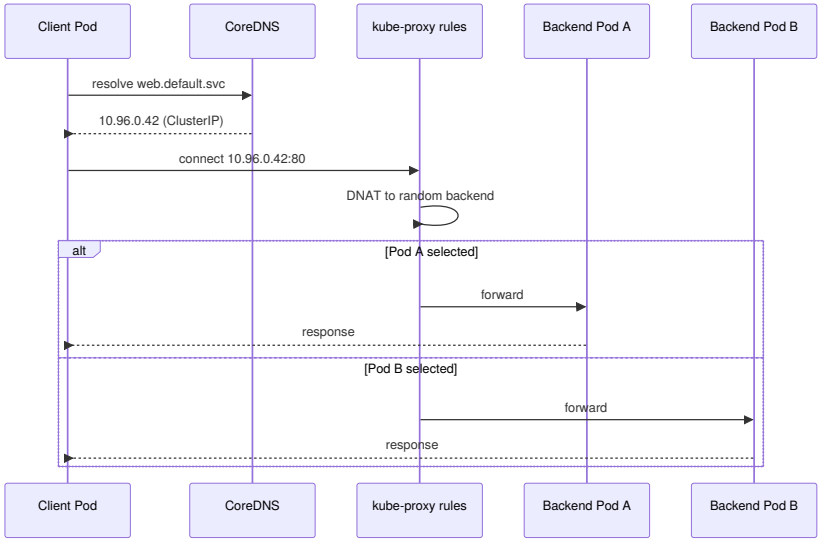
Component	Interaction
CNI plugin	Assigns pod IP, programs routes
kube-proxy	Programs Service → pod rules
CoreDNS	Resolves svc names → ClusterIP
Ingress controller	L7 routing, TLS termination
NetworkPolicy	CNI enforces (Calico, Cilium, DP-v2) — kube-proxy does not
Service mesh (Istio/Linkerd)	Sidecar proxies add mTLS, traffic shaping

🏗 Diagrams

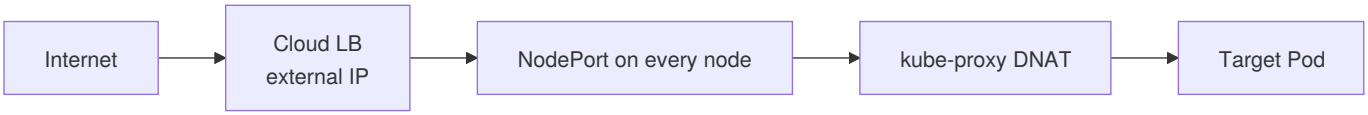
Flat network model



Service traffic path



External traffic via LoadBalancer



💻 Real-World Examples

Example 1 — Verify pod-to-pod connectivity

```
kubectl run a --image=busybox --rm -it -- sh -c 'wget -qO- http://10.244.2.5'
```

Example 2 — Inspect CoreDNS

```
kubectl get cm coredns -n kube-system -o yaml
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl run dns-test --image=busybox --rm -it -- nslookup web.default.svc.cluster.local
```

Example 3 — Check kube-proxy mode

```
kubectl get cm kube-proxy -n kube-system -o yaml | grep mode
# or check kube-proxy logs
kubectl logs -n kube-system -l k8s-app=kube-proxy --tail=5
```

Example 4 — Inspect iptables Service rules (on a node)

```
sudo iptables-save | grep <ClusterIP>
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pods`	`-o wide`	Show pod IPs	`kubectl get pods -o wide`
`kubectl get svc`	`-o wide`	Show ClusterIP, ports	`kubectl get svc -o wide`
`kubectl run`	`--rm -it -- nslookup`	DNS test	`kubectl run dns --image=busybox --rm -it -- nslookup web`
`kubectl exec`	`-- wget -qO-`	HTTP test	`kubectl exec c -- wget -qO- http://web:80`
`kubectl get endpoints`	—	Show backend pod IPs	`kubectl get endpoints web`
`kubectl get endpointslices`	—	EndpointSlice (scalable)	`kubectl get endpointslice -l kubernetes.io/service-name=web`
`kubectl describe svc`	—	See endpoints + events	`kubectl describe svc web`
`kubectl get cm coredns`	`-n kube-system -o yaml`	DNS config	see Example 2
`kubectl logs`	`-n kube-system -l k8s-app=kube-dns`	CoreDNS logs	

🚨 Scenarios

Scenario 1 — Pod can't resolve DNS

- **Cause**: CoreDNS crashed, or `resolv.conf` misconfigured, or CNI failure.
- **Diagnose**: `kubectl get pods -n kube-system -l k8s-app=kube-dns` ; `kubectl exec <pod> -- cat /etc/resolv.conf` ; `kubectl logs -n kube-system <coredns-pod>` .
- **Fix**: restart CoreDNS, check CNI, verify `ndots:5` isn't causing excessive lookups.

Scenario 2 — Pod-to-pod works inside a node but not across nodes

- **Cause**: CNI overlay not functioning; cloud routes missing; firewall blocking VXLAN UDP 4789.
- **Fix**: verify `kubectl get nodes -o wide` (internal IPs reachable), check security groups / VPC routes, check CNI DaemonSet pods.

Scenario 3 — Service has ClusterIP but no endpoints

- **Cause**: selector mismatch or pods not Ready.
- **Fix**: `kubectl get endpoints <svc>` → empty. Align labels or fix readinessProbe.

Scenario 4 — High DNS latency (`ndots` issue)

- **Cause**: default `ndots:5` causes 4 failed queries before absolute resolution for external domains.
- **Fix**: set pod DNS config: `dnsConfig: {options: [{name: ndots, value: "2"}]}` or use FQDNs in config.

Scenario 5 — Intermittent timeouts connecting to Service

- **Cause**: conntrack table full; or kube-proxy iptables rule count too high.
- **Fix**: increase `net.nf_conntrack_max` , or switch to IPVS/eBPF mode.

☁ GCP GKE Perspective

- **GKE Dataplane V2** (default on new clusters) = eBPF-based networking powered by Cilium. Replaces kube-proxy. Provides built-in NetworkPolicy enforcement.
- **VPC-native clusters** (alias IP) = pods get IPs from a secondary range on the VPC subnet. No overlay, native routing.
- **Cloud DNS for GKE** (`--cluster-dns=clouddns`) replaces CoreDNS with a managed service.
- **Private clusters** — nodes have no external IPs; API server is private. Control with `--enable-private-nodes --master-ipv4-cidr=...` .

▩ AWS EKS Perspective

- **VPC CNI** (default) — every pod gets a real ENI secondary IP from the VPC subnet. No overlay; native VPC routing.
- **IP planning** is critical: pods consume VPC IPs. Mitigate with **prefix delegation** (`ENABLE_PREFIX_DELEGATION=true`) or secondary CIDRs.
- **kube-proxy** deployed as a DaemonSet add-on; upgrade with `aws eks update-addon` .
- **CoreDNS** deployed as an EKS add-on.
- **Security groups for pods** — assign SGs directly to pods (VPC CNI feature `ENABLE_POD_ENI`).

🧪 Hands-On Lab Exercises

Lab 19.1 — Verify the flat network (Beginner, 30 min)

Steps

```
kind create cluster --config kind-3.yaml
kubectl run a --image=nginx --labels=app=a
kubectl run b --image=busybox --rm -it -- sh
# inside b:
wget -qO- http://$(kubectl get pod a -o jsonpath='{.status.podIP}')
nslookup kubernetes.default.svc.cluster.local
exit
```

Verify: response from pod `a` by IP and by DNS.

Cleanup: `kubectl delete pod a && kind delete cluster` .

Lab 19.2 — DNS debugging deep-dive (Intermediate, 45 min)

Steps 1. Deploy `dnsutils` pod:

```
kubectl run dns --image=registry.k8s.io/e2e-test-images/agnhost:2.45 -- serve-hostname
kubectl expose pod dns --port=80 --target-port=9376 --name=dns-svc
```

2. From another pod:

```
kubectl run tmp --image=busybox --rm -it -- sh
nslookup dns-svc
nslookup dns-svc.default.svc.cluster.local
nslookup google.com                          # external
cat /etc/resolv.conf                         # see ndots:5, search domains
exit
```

3. Check CoreDNS: `kubectl logs -n kube-system -l k8s-app=kube-dns` .

Bonus: Change `ndots` on a pod and measure lookup time for external domains.

Lab 19.3 — Inspect kube-proxy iptables vs IPVS (Advanced, 60 min)

Prerequisites: kubeadm or Kind cluster.

Steps 1. Check mode: `kubectl get cm kube-proxy -n kube-system -o jsonpath='{.data.config\.conf}' | grep mode` . 2. Create a Service with 3 backend pods. 3. On a node: `sudo iptables-save | grep <ClusterIP>` — see DNAT rules. 4. Switch to IPVS:

```
kubectl edit cm kube-proxy -n kube-system    # change mode: "ipvs"
kubectl rollout restart ds kube-proxy -n kube-system
```

5. On a node: `sudo ipvsadm -Ln` → see virtual service + real servers. 6. Compare rule count for 100 Services (iptables: O(n²) rules; IPVS: O(n) entries).

Cleanup: delete cluster.

Bonus: Install Cilium and observe eBPF replacing both kube-proxy and iptables.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: The Kubernetes networking model guarantees every pod gets its own routable IP and can communicate with every other pod without NAT — a flat network. This is implemented by a CNI plugin, and service discovery is layered on top with kube-proxy and CoreDNS.
Why: A flat network eliminates the port-mapping complexity of Docker bridge mode. Any pod can talk to any pod by IP, and DNS resolves Service names to stable virtual IPs that kube-proxy maps to backend pods via iptables, IPVS, or eBPF.
How: On GKE I use VPC-native clusters with Dataplane V2 — pods get alias IPs from the VPC, no overlay, and eBPF replaces kube-proxy for both performance and built-in NetworkPolicy. On EKS I use VPC CNI where every pod gets a real ENI IP. IP planning is essential — I enable prefix delegation or secondary CIDRs to avoid exhaustion. For DNS I keep CoreDNS with sensible `ndots` settings and upstream caching.
Example: On a 200-service EKS cluster we migrated from iptables to IPVS mode and cut Service-related p99 latency by 15% — the O(n²) iptables rules were adding measurable overhead."

? Possible Interview Questions

Basic

Q1. State the K8s networking rules. Every pod gets an IP; pods talk to pods without NAT; nodes talk to pods without NAT; pod sees its own IP.
Q2. What is CNI? Container Network Interface — the standard K8s uses to delegate IP management and routing to plugins.
Q3. What does kube-proxy do? Programs iptables/IPVS/nftables/eBPF rules that map Service ClusterIP → backend pod IPs.

Intermediate

Q4. Overlay vs native routing? Overlay encapsulates (VXLAN); native programs host or VPC routes. Native is faster but environment-dependent.

Q5. Name 3 popular CNI plugins. Calico, Cilium, Flannel (or cloud-native: VPC CNI, GKE DP-v2).
Q6. What is `ndots:5` ? The default search config that appends `.svc.cluster.local` , etc. to short names before resolving absolutely — can cause 4 extra DNS lookups.

Advanced

Q7. How does GKE Dataplane V2 replace kube-proxy? It uses eBPF programs at the kernel socket layer to intercept Service traffic, bypassing iptables entirely — lower latency, less CPU.
Q8. How does VPC CNI assign IPs on EKS? Each node gets secondary IPs via ENI. Prefix delegation assigns /28 blocks per ENI slot, increasing density.
Q9. iptables vs IPVS — when to switch? IPVS at scale (>1000 Services); iptables O(n) per packet, IPVS O(1) hash lookup. nftables is the next-gen iptables replacement.

Scenario-based

Q10. Pods across nodes can't reach each other — steps? Check CNI DaemonSet running, node firewall / SG rules (allow pod CIDR + VXLAN UDP 4789), VPC routes, `kubectl get nodes -o wide` .

GKE-specific

Q11. What's a VPC-native cluster? Pods use secondary ranges (alias IPs) on the VPC; routes are native GCP; no overlay, seamless peering.

EKS-specific

Q12. You ran out of IPs for pods on EKS — options? Enable prefix delegation, add secondary CIDRs, use smaller pod CIDRs, or move to Calico/Cilium overlay (trade VPC routing for overlayed IPs).

Hands-on / Lab-based

Q13. How do you test DNS from inside a pod? `kubectl run tmp --image=busybox --rm -it -- nslookup <svc>.<ns>.svc.cluster.local` .
Q14. How do you inspect iptables rules for a Service? SSH to node: `sudo iptables-save | grep <ClusterIP>` .
Q15. Walk me through what happens when Pod A curls Service B. A resolves B's name via CoreDNS → gets ClusterIP → kube-proxy's iptables/IPVS DNAT → packet routes to a backend Pod → response follows reverse path.

🗐 Topic 20 — Services (ClusterIP, NodePort, LoadBalancer, ExternalName, Headless)

Official Documentation Reference - Service: https://kubernetes.io/docs/concepts/services-networking/service/ - EndpointSlices: https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/ - Virtual IPs and Service Proxies: https://kubernetes.io/docs/reference/networking/virtual-ips/ - Connecting apps with Services (tutorial): https://kubernetes.io/docs/tutorials/services/connect-applications-service/ - External LB: https://kubernetes.io/docs/tasks/access-application-cluster/create-external-load-balancer/ - GKE Services: https://cloud.google.com/kubernetes-engine/docs/concepts/service - EKS load balancing: https://docs.aws.amazon.com/eks/latest/userguide/network-load-balancing.html

🔹 Definition

A **Service** is an abstract way to expose a set of Pods as a network service. It provides a **stable IP** (ClusterIP), **DNS name**, and **load balancing** — decoupling clients from the dynamic lifecycle of pods.

🔹 About

Service types

Type	Scope	How
ClusterIP (default)	Internal	Virtual IP reachable only inside cluster
NodePort	External via node IP	Allocates port 30000–32767 on every node; builds on ClusterIP
LoadBalancer	External via cloud LB	Provisions cloud LB (GCLB, NLB, ALB); builds on NodePort
ExternalName	DNS alias	Returns a CNAME record; no proxying; points to an external DNS
Headless (`clusterIP: None`)	Direct pod DNS	DNS returns A records for each pod; used by StatefulSets

Selector → EndpointSlice

When a Service has `spec.selector` , the EndpointSlice controller creates EndpointSlice objects listing the IPs of matching, Ready pods. kube-proxy watches EndpointSlices to program rules.

Service without selector

Manually created Endpoints/EndpointSlices — useful for pointing to an external DB or non-K8s host.

Session affinity

`spec.sessionAffinity: ClientIP` routes all requests from the same source IP to the same pod (default: None = round-robin).

Traffic policies

- `spec.internalTrafficPolicy: Local` — keeps traffic on the same node.
- `spec.externalTrafficPolicy: Local` — preserves client source IP but only routes to local pods (avoids extra hop).

Port mapping

```
ports:
- name: http
  port: 80         # Service port (what clients use)
  targetPort: 8080  # Container port (where pod listens)
  protocol: TCP
```

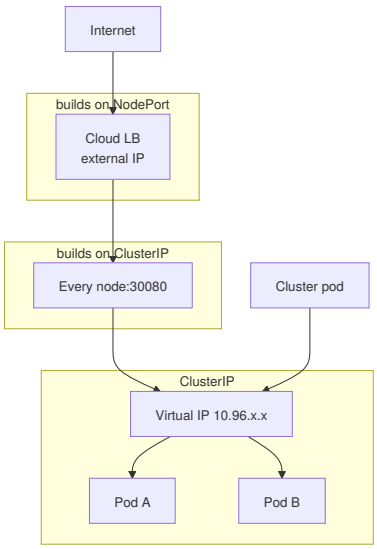
`targetPort` can be a named port referencing the pod's `containerPort.name`.

How It Integrates With Other Kubernetes Components

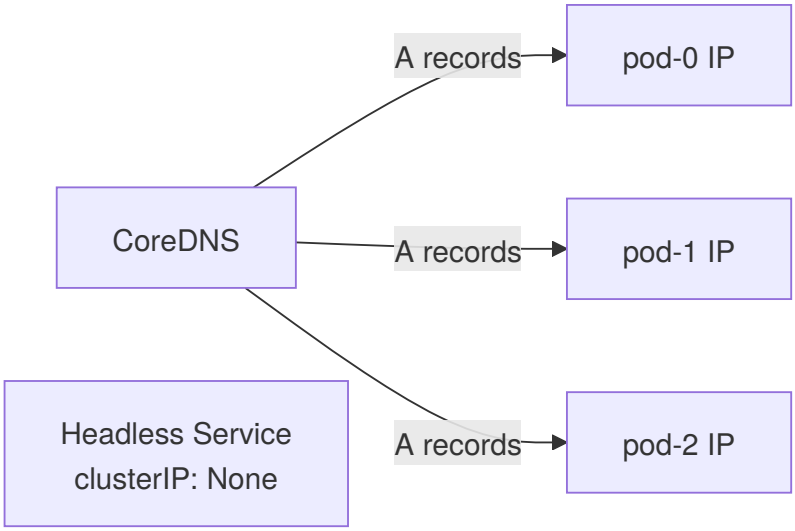
Component	Interaction
Pods	Matched by selector labels → become endpoints
kube-proxy	Programs iptables/IPVS for ClusterIP/NodePort
CoreDNS	Resolves `<svc>.<ns>.svc.cluster.local` → ClusterIP
Ingress / Gateway API	Routes external L7 traffic to Services
cloud-controller-manager	Provisions external LB for LoadBalancer type
NetworkPolicy	Controls ingress/egress to Services' pod targets
HPA	Doesn't scale Services; scales Deployments behind them
StatefulSet	Needs headless Service for stable DNS

Diagrams

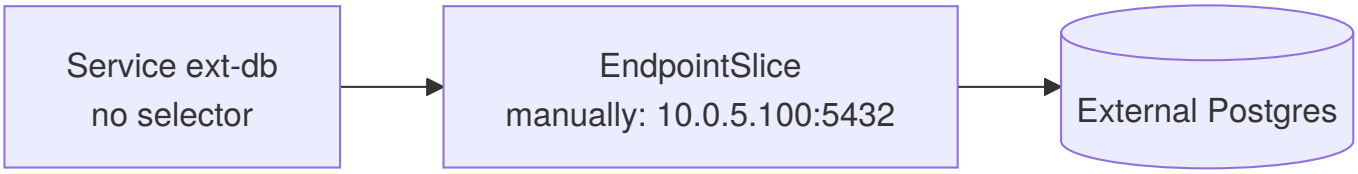
Service types stacked



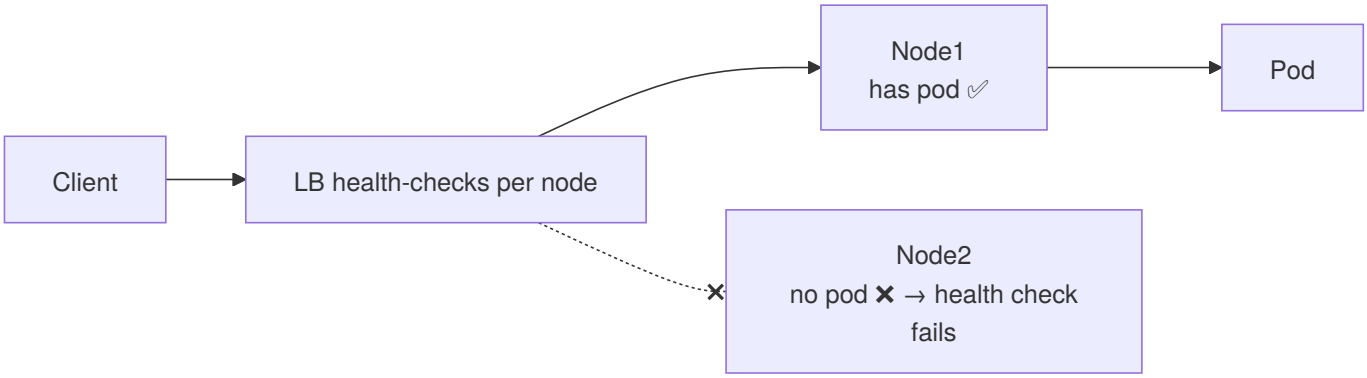
Headless Service



Service without selector



externalTrafficPolicy: Local



💻 Real-World Examples

Example 1 — ClusterIP

```
apiVersion: v1
kind: Service
metadata: { name: backend }
spec:
  selector: { app: api }
  ports:
  - { name: http, port: 80, targetPort: 8080 }
  type: ClusterIP
```

Example 2 — NodePort

```
apiVersion: v1
kind: Service
metadata: { name: web-np }
spec:
  selector: { app: web }
  ports:
  - { port: 80, targetPort: 80, nodePort: 30080 }
  type: NodePort
```

Example 3 — LoadBalancer with annotations

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-lb
  annotations:
    # AWS NLB
    service.beta.kubernetes.io/aws-load-balancer-type: nlb
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
    # OR GKE Internal LB
    # networking.gke.io/load-balancer-type: Internal
spec:
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
  type: LoadBalancer
  externalTrafficPolicy: Local
```

Example 4 — ExternalName

```yaml
apiVersion: v1
kind: Service
metadata: { name: legacy-db }
spec:
  type: ExternalName
  externalName: db.legacy.example.com
```

Pods resolve `legacy-db` → CNAME → `db.legacy.example.com`.

Example 5 — Headless for StatefulSet

```yaml
apiVersion: v1
kind: Service
metadata: { name: pg }
spec:
  clusterIP: None
  selector: { app: pg }
  ports: [{ port: 5432, name: pg }]
```

Example 6 — Service without selector (external DB)

```yaml
apiVersion: v1
kind: Service
metadata: { name: ext-db }
spec:
  ports: [{ port: 5432, targetPort: 5432 }]
---
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: ext-db-1
  labels: { kubernetes.io/service-name: ext-db }
addressType: IPv4
ports: [{ port: 5432, protocol: TCP }]
endpoints:
- { addresses: ["10.0.5.100"] }
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl expose deploy`	`--port --target-port --type`	Quick Service creation	`kubectl expose deploy web --port=80 --target-port=8080 --type=ClusterIP`
`kubectl get svc`	`-o wide`	Show selector, ports	`kubectl get svc -o wide`
`kubectl describe svc`	—	Endpoints, events, LB status	`kubectl describe svc web-lb`
`kubectl get endpoints`	`<svc>`	Show backend IPs	`kubectl get endpoints backend`
`kubectl get endpointslice`	`-l kubernetes.io/service-name=<svc>`	Scalable endpoints	`kubectl get endpointslice -l kubernetes.io/service-name=web`
`kubectl port-forward svc`	`<svc> local:remote`	Local tunnel	`kubectl port-forward svc/backend 8080:80`
`kubectl delete svc`	—	Remove (cloud LB deleted too)	`kubectl delete svc web-lb`
`kubectl patch svc`	`-p '{"spec":{"type":"LoadBalancer"}}'`	Change type	`kubectl patch svc web -p '{"spec":{"type":"LoadBalancer"}}'`
`kubectl create svc clusterip`	`--tcp=80:8080`	Skeleton	`kubectl create svc clusterip web --tcp=80:8080 --dry-run=client -o yaml`

🚨 Scenarios

Scenario 1 — Service has no endpoints

- **Cause**: selector doesn't match pods; or pods aren't Ready.

- **Fix**: `kubectl get endpoints <svc>` → empty. Compare `svc.spec.selector` with pod labels. Fix readinessProbe.

Scenario 2 — Client sees wrong source IP behind LoadBalancer

- **Cause**: `externalTrafficPolicy: Cluster` (default) SNAT-s the client IP.
- **Fix**: Set `externalTrafficPolicy: Local` — preserves source IP; pods only receive traffic on nodes where they run.

Scenario 3 — NodePort service not reachable externally

- **Cause**: firewall / security group blocking the high port.
- **Fix**: open port range 30000–32767 in SG / firewall rules.

Scenario 4 — LoadBalancer never gets an external IP

- **Cause**: cloud-controller-manager not running; quota exhausted; wrong annotations.
- **Diagnose**: `kubectl describe svc <n>` → events.
- **Fix**: check cloud quota; verify CCM DaemonSet; on bare metal use MetalLB.

Scenario 5 — DNS returns stale endpoint after scale-down

- **Cause**: EndpointSlice updated, but CoreDNS cache TTL or pod-level `ndots` caching.
- **Fix**: reduce CoreDNS TTL (`template` section), or `dnsConfig: {options: [{name: single-request-reopen}]}` .

☁ GCP GKE Perspective

- **GKE Network Endpoint Groups (NEGs)** enable container-native load balancing — the cloud LB routes directly to pod IPs, bypassing kube-proxy + NodePort. Enabled via annotation `cloud.google.com/neg: '{"ingress": true}'` .
- **Internal TCP/UDP LB**: annotation `networking.gke.io/load-balancer-type: Internal` .
- **GKE Gateway API** is the successor to Ingress, providing multi-cluster, traffic splitting, header-based routing.
- **Autopilot** defaults: NEG-based Services; no NodePort access unless explicitly configured.

▩ AWS EKS Perspective

- **AWS Load Balancer Controller** (install separately) provisions NLB for `type: LoadBalancer` and ALB for Ingress.
- **NLB**: annotation `service.beta.kubernetes.io/aws-load-balancer-type: nlb` .
- **IP target mode** = controller routes directly to pod IPs (like GKE NEGs): annotation `service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip` .
- **Fargate**: only `type: LoadBalancer` with IP mode works (no NodePort access).
- **Security groups for pods** let you SG-gate traffic at the pod level on VPC CNI.

Hands-On Lab Exercises

Lab 20.1 — All Service types (Beginner, 30 min)

Steps

```
kubectl create deploy web --image=nginx:1.27 --replicas=3
# ClusterIP
kubectl expose deploy web --port=80 --target-port=80 --name=web-cip
# NodePort
kubectl expose deploy web --port=80 --target-port=80 --type=NodePort --name=web-np
# Access
kubectl run tmp --image=busybox --rm -it -- wget -qO- http://web-cip
minikube service web-np --url | xargs curl -s | head
# Headless
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata: { name: web-hl }
spec:
  clusterIP: None
  selector: { app: web }
  ports: [{ port: 80 }]
EOF
kubectl run tmp --image=busybox --rm -it -- nslookup web-hl
```

Verify: ClusterIP returns one IP; headless returns multiple A records.

Cleanup: `kubectl delete deploy web && kubectl delete svc web-cip web-np web-hl` .

Lab 20.2 — externalTrafficPolicy: Local (Intermediate, 45 min)

Steps 1. Deploy nginx with `hostPort: 80` (or use NodePort). 2. Create LoadBalancer Service with `externalTrafficPolicy: Cluster` . 3. From outside: `curl -s <external-ip>` → check access logs for source IP (it's SNAT'd). 4. Switch to `externalTrafficPolicy: Local` . 5. `curl` again → source IP matches your real IP. 6. Scale down to 1 pod; hit the node that has no pod → timeout (LB health-check removes it).

Bonus: on GKE, enable container-native LB (NEG) and compare — source IP always preserved, no NodePort hop.

Lab 20.3 — Service without selector → external database (Advanced, 45 min)

Steps 1. Run a Postgres container outside K8s (`docker run -d --name pg -p 5432:5432 postgres:16`). 2. Get its IP (host IP or bridge IP). 3. Create a selectorless Service + EndpointSlice (Example 6) pointing to that IP. 4. Deploy an app pod that connects to `ext-db:5432` using the K8s Service name. 5. Verify connection works: `kubectl exec app -- pg_isready -h ext-db` .

Cleanup: stop external Postgres; delete Service + EndpointSlice.

Bonus: Replace with `ExternalName` pointing to a DNS name (e.g., `db.example.com`) and verify CNAME resolution.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Kubernetes Service is a stable network abstraction over a dynamic set of pods. It gives you a fixed ClusterIP and DNS name, while kube-proxy load-balances traffic to whichever pods match the selector and pass readinessProbe.
Why: Pods are ephemeral — their IPs change on restarts. Services decouple producers from consumers. ClusterIP for internal, NodePort for dev access, LoadBalancer for production external traffic, ExternalName for DNS aliasing to off-cluster resources, and Headless for StatefulSet direct-pod DNS.
How: I always use ClusterIP for inter-service traffic and expose externally with either a cloud LoadBalancer (NLB on EKS, NEG-based GCLB on GKE) or an Ingress/Gateway for L7. I set `externalTrafficPolicy: Local` when I need client IP preservation. For StatefulSets I pair a headless Service so each pod gets `pod-N.svc.ns.svc.cluster.local`.
Example: On EKS we switched from iptables-mode NodePort to NLB with IP target mode — the LB routes directly to pod IPs. p99 latency dropped 8% and we no longer needed to manage NodePort security groups."

? Possible Interview Questions

Basic

Q1. What is a Kubernetes Service? Stable IP + DNS + load balancing over a set of pods.
Q2. Name the Service types. ClusterIP, NodePort, LoadBalancer, ExternalName, Headless.
Q3. Default Service type? ClusterIP.

Intermediate

Q4. NodePort builds on ClusterIP — what does that mean? A NodePort Service also gets a ClusterIP; the NodePort routes to it.
Q5. What is an EndpointSlice? An API object listing the IPs of Ready pods for a Service; replaces the older Endpoints object for scale.
Q6. `externalTrafficPolicy: Local` — trade-offs? Preserves source IP; but traffic only goes to nodes hosting a pod, so uneven distribution if pods aren't spread.

Advanced

Q7. What is a selectorless Service? A Service with no selector; you manually create EndpointSlice entries pointing to any IP — commonly used for external DBs.
Q8. How does headless DNS work? `clusterIP: None` → CoreDNS returns A records for each Ready pod IP, not a virtual IP.
Q9. How does container-native load balancing (NEG/IP-target) skip kube-proxy? The cloud LB sends traffic directly to pod IPs instead of through NodePort → iptables. Reduces hops, preserves source IP.

Scenario-based

Q10. Service works but traffic is uneven — 70% goes to one pod. Why? Connection-based balancing (not request-based); long-lived HTTP/2 or gRPC connections. Fix: use L7 LB (Ingress/ALB) for request-based; or Service mesh.

GKE-specific

Q11. What are NEGs on GKE? Network Endpoint Groups — the GCLB routes directly to pod endpoints registered in a NEG instead of through NodePort.

EKS-specific

Q12. How do you get an NLB instead of a CLB on EKS? Install the AWS Load Balancer Controller and annotate the Service with `service.beta.kubernetes.io/aws-load-balancer-type: nlb`.

Hands-on / Lab-based

Q13. Show me how to test a Service from inside the cluster. `kubectl run tmp --image=busybox --rm -it -- wget -qO- http://svc-name.namespace.svc.cluster.local`.
Q14. How do you check if a Service has endpoints? `kubectl get endpoints <svc>` or `kubectl get endpointslice -l kubernetes.io/service-name=<svc>`.
Q15. How would you expose a legacy VM to pods? Selectorless Service with an EndpointSlice pointing to the VM's IP, or ExternalName if a DNS name exists.

🗐 Topic 21 — Ingress and Ingress Controllers

Official Documentation Reference - Ingress: https://kubernetes.io/docs/concepts/services-networking/ingress/ - Ingress Controllers: https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/ - Gateway API: https://kubernetes.io/docs/concepts/services-networking/gateway/ - GKE Ingress: https://cloud.google.com/kubernetes-engine/docs/concepts/ingress - GKE Gateway: https://cloud.google.com/kubernetes-engine/docs/concepts/gateway-api - EKS ALB Ingress: https://docs.aws.amazon.com/eks/latest/userguide/alb-ingress.html - AWS LB Controller: https://kubernetes-sigs.github.io/aws-load-balancer-controller/

⏚ Definition

An **Ingress** is a Kubernetes API object that manages external HTTP/HTTPS access to Services within a cluster — providing host-based and path-based routing, TLS termination, and name-based virtual hosting. An **Ingress Controller** is the component that watches Ingress objects and configures a reverse proxy (NGINX, Envoy, HAProxy, cloud LB) accordingly.

🔷 About

The problem Ingress solves

Without Ingress, every public Service needs a cloud LoadBalancer (~$18/month each). Ingress consolidates traffic through one LB with L7 routing rules — one IP, many hosts/paths.

Key Ingress fields

Field	Purpose
`spec.ingressClassName`	Which controller handles this Ingress
`spec.rules[].host`	Hostname matching (`app.example.com`)
`spec.rules[].http.paths`	URL path routing
`spec.rules[].http.paths[].pathType`	`Prefix` , `Exact` , `ImplementationSpecific`
`spec.rules[].http.paths[].backend.service`	Target Service + port
`spec.tls[].hosts` + `secretName`	TLS termination
`spec.defaultBackend`	Catch-all 404 handler

Popular Ingress Controllers

Controller	Proxy	Best for
ingress-nginx	NGINX	General purpose, on-prem & cloud
GCE Ingress (gce)	Google Cloud LB	GKE native; provisions GCLB
AWS LB Controller	ALB	EKS native; provisions ALB
Traefik	Traefik	Auto-discovery, Let's Encrypt
HAProxy	HAProxy	High-performance TCP/HTTP
Contour	Envoy	HTTPProxy CRD, advanced routing
Istio Gateway	Envoy	Part of Istio mesh

Gateway API (successor to Ingress)

Gateway API is the next-generation L4/L7 routing standard — more expressive, role-oriented (infra admin → Gateway, app dev → HTTPRoute), and extensible. GKE and the AWS LB Controller both support it.

TLS with cert-manager

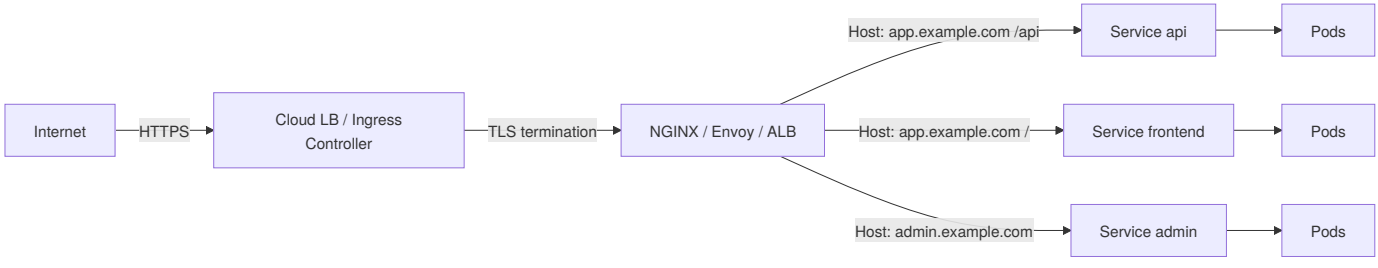
cert-manager watches Ingress annotations like `cert-manager.io/cluster-issuer: letsencrypt-prod` , creates a Certificate CR, completes ACME challenge, stores cert in a K8s Secret, and Ingress controller serves it.

🔷 How It Integrates With Other Kubernetes Components

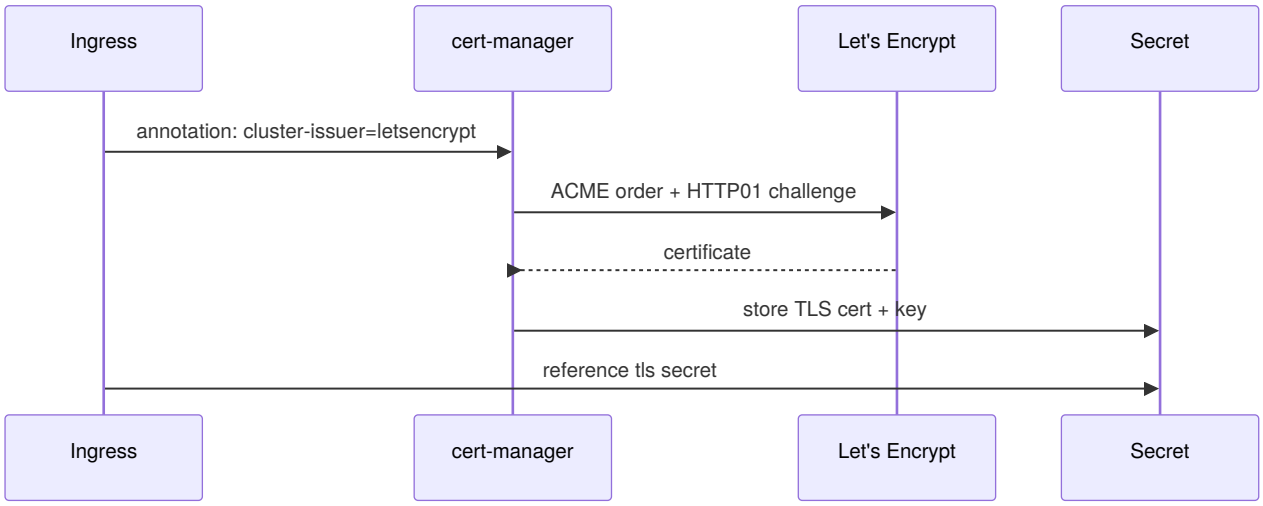
Component	Interaction
Service (ClusterIP)	Backend target of Ingress rules
Secret (TLS)	Stores certs referenced by `spec.tls`
cert-manager	Automates TLS cert issuance/renewal
cloud-controller-manager	GCE Ingress / ALB provisioning
NetworkPolicy	Controls which pods can receive Ingress traffic
DNS (external)	A/CNAME records pointing to Ingress LB IP

🏗 Diagrams

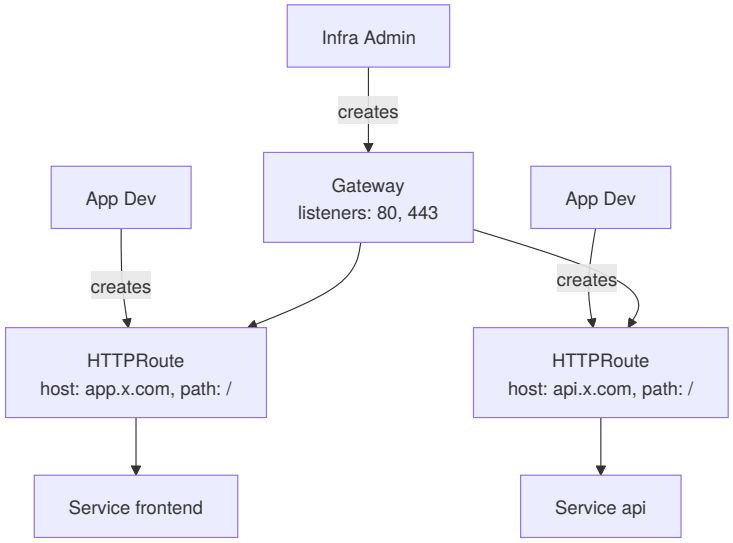
Ingress traffic flow



cert-manager flow



Gateway API model



💻 Real-World Examples

Example 1 — NGINX Ingress with host + path routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts: [app.example.com]
    secretName: app-tls
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api(/|$)(.*)
        pathType: ImplementationSpecific
        backend: { service: { name: api, port: { number: 80 } } }
      - path: /
        pathType: Prefix
        backend: { service: { name: frontend, port: { number: 80 } } }
```

Example 2 — GKE GCE Ingress with managed cert

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: gke-app
  annotations:
    kubernetes.io/ingress.class: gce
    networking.gke.io/managed-certificates: app-cert
    kubernetes.io/ingress.global-static-ip-name: app-ip
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /*
        pathType: ImplementationSpecific
        backend: { service: { name: web, port: { number: 80 } } }
---
apiVersion: networking.gke.io/v1
kind: ManagedCertificate
metadata: { name: app-cert }
spec:
  domains: [app.example.com]
```

Example 3 — AWS ALB Ingress

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: alb-app
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:ap-south-1:123:certificate/abc
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: "443"
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: { service: { name: web, port: { number: 80 } } }
```

Example 4 — Install ingress-nginx on Minikube/Kind

```bash
# Minikube
minikube addons enable ingress

# Kind
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/kind/deploy.yaml

# Helm (any cluster)
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm install ingress-nginx ingress-nginx/ingress-nginx -n ingress-nginx --create-namespace
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get ingress`	`-A`	List Ingress objects	`kubectl get ingress -A`
`kubectl describe ingress`	`<name>`	Rules + endpoints + events	`kubectl describe ingress app`
`kubectl get ingressclass`	—	Available controllers	`kubectl get ingressclass`
`kubectl create ingress`	`--rule`	Quick Ingress	`kubectl create ingress demo --rule="app.com/*=web:80" --class=nginx`
`kubectl logs -n ingress-nginx`	`-l app.kubernetes.io/name=ingress-nginx`	Controller logs	
`kubectl get svc -n ingress-nginx`	—	Find LB external IP	`kubectl get svc -n ingress-nginx`
`kubectl cert-manager status`	—	cert-manager cert status	`kubectl get certificate`

🚨 Scenarios

Scenario 1 — Ingress created but no external IP

- **Cause**: no Ingress controller installed; or wrong `ingressClassName`.
- **Fix**: install a controller; set `ingressClassName` or annotation.

Scenario 2 — 404 on all paths

- **Cause**: Service backend has no endpoints; or path regex wrong.

- **Fix**: `kubectl describe ingress` → check backends; verify Service has Ready pods.

Scenario 3 — TLS certificate not served

- **Cause**: Secret doesn't exist; or cert-manager challenge failing (DNS, firewall).
- **Fix**: `kubectl get certificate`, `kubectl describe certificate`, `kubectl describe order`, `kubectl describe challenge`.

Scenario 4 — GKE Ingress takes 5–10 minutes

- **Cause**: GCE Ingress provisions a GCLB + health checks + NEGs → slower than NGINX.
- **Fix**: expected behavior. Use NGINX Ingress for faster iteration in dev; GCE Ingress for production.

Scenario 5 — ALB Ingress returns 502

- **Cause**: health check path returns non-200; target group not registered; security group blocks pod.
- **Fix**: set `alb.ingress.kubernetes.io/healthcheck-path: /healthz`; check SG allows ALB → pod CIDR.

☁ GCP GKE Perspective

- **GCE Ingress** (default on GKE) provisions a Google Cloud HTTP(S) LB. Supports ManagedCertificates, Cloud Armor WAF, Cloud CDN.
- **GKE Gateway Controller** implements Gateway API natively — supports multi-cluster Gateway (Fleet Ingress), traffic splitting, header-based routing.
- **Autopilot** uses GCE Ingress or Gateway by default. NGINX Ingress works but needs a `LoadBalancer` Service in front.
- Key annotations: `networking.gke.io/managed-certificates`, `cloud.google.com/backend-config`, `kubernetes.io/ingress.global-static-ip-name`.

▩ AWS EKS Perspective

- **AWS Load Balancer Controller** provisions ALB (Ingress) or NLB (Service). Install separately:

```
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=prod --set serviceAccount.create=false --set serviceAccount.name=aws-lb-sa
```

- ALB supports target-type `ip` (direct to pod) or `instance` (via NodePort).
- **Fargate** requires `target-type: ip` (no NodePort).
- TLS: reference ACM cert via annotation `alb.ingress.kubernetes.io/certificate-arn`.
- **Weighted target groups** for canary: `alb.ingress.kubernetes.io/actions.forward` annotation.

🧪 Hands-On Lab Exercises

Lab 21.1 — NGINX Ingress on Minikube (Beginner, 30 min)

Steps

```
minikube addons enable ingress
kubectl create deploy web --image=nginx --replicas=2
kubectl expose deploy web --port=80
kubectl create deploy api --image=hashicorp/http-echo -- -text="api"
kubectl expose deploy api --port=80 --target-port=5678
cat <<EOF | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: demo }
spec:
  ingressClassName: nginx
  rules:
  - host: demo.local
    http:
      paths:
      - { path: /api, pathType: Prefix, backend: { service: { name: api, port: { number: 80 } } } }
      - { path: /,    pathType: Prefix, backend: { service: { name: web, port: { number: 80 } } } }
EOF
echo "$(minikube ip) demo.local" | sudo tee -a /etc/hosts
curl http://demo.local/
curl http://demo.local/api
```

Cleanup: `kubectl delete ingress demo && kubectl delete deploy,svc web api`.

Lab 21.2 — TLS with cert-manager + Let's Encrypt staging (Intermediate, 60 min)

Prerequisites: cluster with public IP (GKE/EKS), real domain pointing to the LB IP.

Steps 1. Install cert-manager: `kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.16.0/cert-manager.yaml`. 2. Create ClusterIssuer:

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: letsencrypt-staging }
spec:
  acme:
    server: https://acme-staging-v02.api.letsencrypt.org/directory
    email: you@example.com
    privateKeySecretRef: { name: letsencrypt-staging }
    solvers: [{ http01: { ingress: { class: nginx } } }]
```

3. Create Ingress with annotation `cert-manager.io/cluster-issuer: letsencrypt-staging` and `tls` block. 4. `kubectl get certificate` → should become

`Ready=True`. 5. `curl -k https://app.yourdomain.com`.

Cleanup: delete Ingress, ClusterIssuer, `kubectl delete -f cert-manager.yaml`.

Lab 21.3 — GKE GCE Ingress + ALB Ingress side-by-side (Advanced, 90 min)

Steps 1. On GKE: apply Example 2 with a ManagedCertificate and static IP. 2. On EKS: install AWS LB Controller, apply Example 3 with ACM cert ARN. 3. Compare: creation time, LB type, health check behavior, WAF/CDN options. 4. Test with `hey -z 60s https://app.example.com`.

Cleanup: delete Ingress on both → LBs auto-deleted.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: An Ingress is a Kubernetes object that defines L7 routing rules — host, path, TLS — for external HTTP/S traffic. An Ingress Controller (NGINX, GCE Ingress, ALB) watches those rules and programs a reverse proxy or cloud load balancer.
Why: Without Ingress, each public service needs its own cloud LB. Ingress consolidates all traffic through one IP with path/host-based routing — saving cost and simplifying DNS and TLS management.
How: On GKE I use GCE Ingress with ManagedCertificates for production and NGINX Ingress for dev — or better, the new Gateway API for multi-cluster routing. On EKS I install the AWS Load Balancer Controller to provision ALBs with ACM certs via annotation. For automatic TLS I pair any controller with cert-manager and a ClusterIssuer pointing to Let's Encrypt.
Example: We consolidated 14 LoadBalancer Services into one ALB Ingress with path routing — saved $250/month in LB costs and simplified our DNS to a single CNAME."

? Possible Interview Questions

Basic

Q1. What is an Ingress? An API object defining L7 routing rules for external HTTP(S) traffic to internal Services.
Q2. Why do you need an Ingress Controller? The Ingress object is just a spec; a controller (NGINX, ALB, GCE) implements it.
Q3. What is `ingressClassName`? Selects which controller should handle the Ingress.

Intermediate

Q4. `pathType` values? `Exact` (match full path), `Prefix` (match path prefix), `ImplementationSpecific` (controller-defined, often regex).
Q5. How does TLS work with Ingress? Ingress references a TLS Secret; the controller terminates TLS before forwarding to backend.
Q6. What is Gateway API? The next-gen routing standard superseding Ingress — more expressive, role-based (Gateway for infra, HTTPRoute for app devs).

Advanced

Q7. GCE Ingress vs NGINX Ingress on GKE? GCE provisions a Google Cloud HTTP(S) LB with global anycast, CDN, Cloud Armor — but slower to set up (~5 min). NGINX is faster, local, more customizable via annotations, but runs in-cluster.
Q8. How does the ALB Ingress Controller work? It watches Ingress objects, creates/updates an ALB via AWS APIs, configures listeners, target groups (IP or instance mode), and health checks.
Q9. How does cert-manager automate TLS? It watches Ingress annotations or Certificate CRs, performs ACME HTTP01/DNS01 challenges with Let's Encrypt, stores certs in Secrets, and renews before expiry.

Scenario-based

Q10. Ingress returns 503 — steps? Check controller pods running; `describe ingress` for backend status; verify Service endpoints; check health check configuration on cloud LB.

GKE-specific

Q11. What is a ManagedCertificate on GKE? Google-managed TLS cert that auto-provisions and auto-renews for domains configured in the CR; referenced via annotation on GCE Ingress.

EKS-specific

Q12. `target-type: ip` vs `instance` on ALB Ingress? `ip` routes to pod IPs directly (required for Fargate, lower latency). `instance` routes via NodePort.

Hands-on / Lab-based

Q13. How would you troubleshoot a cert-manager certificate stuck in `False` ready? `kubectl describe certificate` → `kubectl describe order` → `kubectl describe challenge` → check DNS/firewall for ACME challenge.
Q14. Show me path-based routing with NGINX Ingress. Two Services, one Ingress with `paths: [{path: /api, backend: api}, {path: /, backend: frontend}]`.
Q15. How would you do canary with Ingress? NGINX: `nginx.ingress.kubernetes.io/canary: "true"` + `canary-weight: "10"` annotation on a second Ingress. ALB: weighted target group annotations.

🗐 Topic 22 — Network Policies

Official Documentation Reference - Network Policies: https://kubernetes.io/docs/concepts/services-networking/network-policies/ - Declare Network Policy: https://kubernetes.io/docs/tasks/administer-cluster/declare-network-policy/ - GKE Network Policy: https://cloud.google.com/kubernetes-engine/docs/how-to/network-policy - EKS Network Policy (Calico/VPC CNI): https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html

---

Definition

A **NetworkPolicy** is a namespaced Kubernetes object that controls L3/L4 traffic flow (TCP/UDP/SCTP) to and from pods using label selectors, namespace selectors, and IP blocks. Without any policy, all traffic is allowed (default-allow).

About

The zero-trust starting point

By default, every pod can talk to every other pod. NetworkPolicy lets you move to **default-deny** and then explicitly whitelist needed traffic.

Policy model

- A policy selects pods via `spec.podSelector`.
- It declares `policyTypes: [Ingress]`, `[Egress]`, or `[Ingress, Egress]`.
- An **empty** `spec.podSelector: {}` targets ALL pods in the namespace.
- An **empty** `ingress: []` means deny all ingress. An **omitted** `ingress` field means no change (still allowed unless another policy denies).
- Policies are **additive** — multiple policies union their allows; there is no explicit "deny" rule.

Rule structure

Each ingress/egress rule has `from` / `to` (peers) and `ports`: - `podSelector` — match pods by labels (same namespace by default). - `namespaceSelector` — match pods in namespaces matching labels. - `ipBlock` — CIDR range + optional `except`. - `ports` — port + protocol.

Critical subtlety: AND vs OR

```
ingress:
- from:
  - podSelector: { matchLabels: { app: api } }    # source 1
  - namespaceSelector: { matchLabels: { env: prod } }  # source 2
  ports: [{ port: 80 }]
```

The two `from` entries are **OR** — either match suffices. But:

```
- from:
  - podSelector: { matchLabels: { app: api } }
    namespaceSelector: { matchLabels: { env: prod } }
```

Here podSelector AND namespaceSelector are in the same item → **AND** — must match both.

CNI requirement

NetworkPolicy is just an API object. Enforcement depends on the CNI plugin. **kube-proxy does NOT enforce NetworkPolicy.** The CNI must support it: - Calico ✅ - Cilium ✅ - GKE Dataplane V2 ✅ - EKS VPC CNI (native policy, K8s 1.25+) ✅ - Flannel ❌ (no policy enforcement)

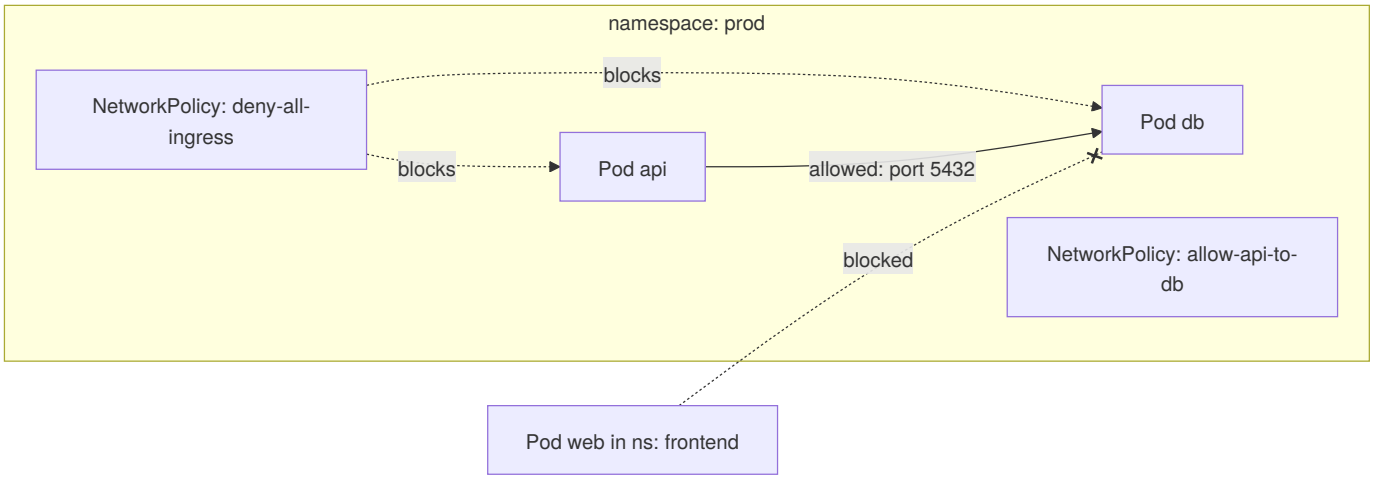
How It Integrates With Other Kubernetes Components

Component	Interaction
CNI plugin	Enforces the actual iptables/eBPF rules
Pods	Selected by `podSelector`
Namespaces	Selected by `namespaceSelector` (need labels on NS)
Services	Policies target pods, not Services; traffic to Service ClusterIP resolves to pod IPs before policy check
Ingress controller	Must be allowed in NetworkPolicy; controller pods live in their own NS
DNS (CoreDNS)	Must explicitly allow egress to `kube-system` for DNS resolution

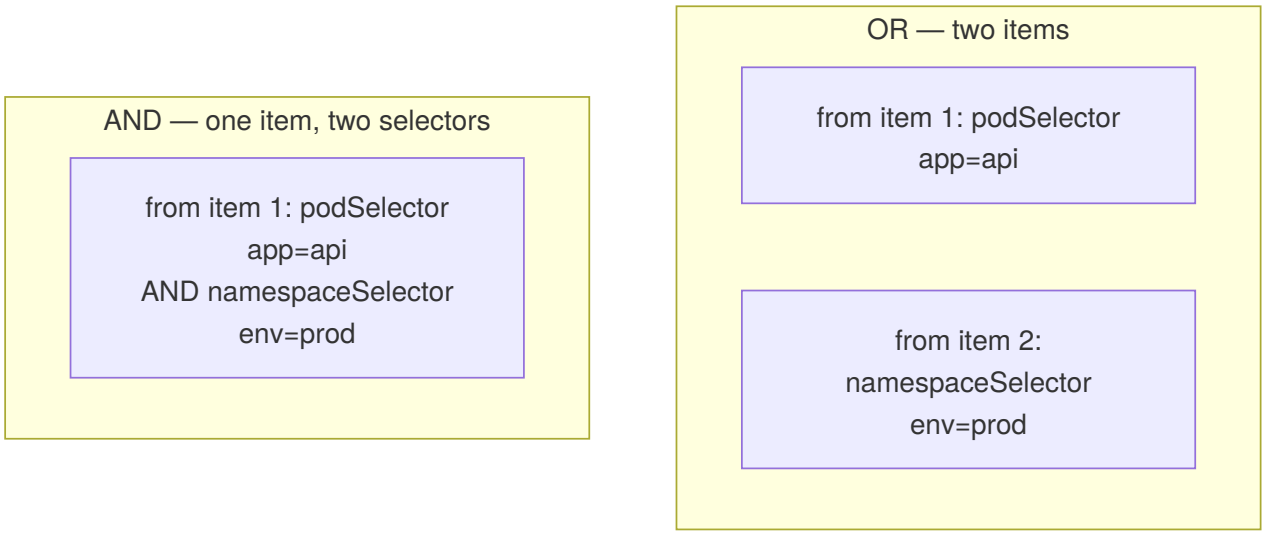
---

Diagrams

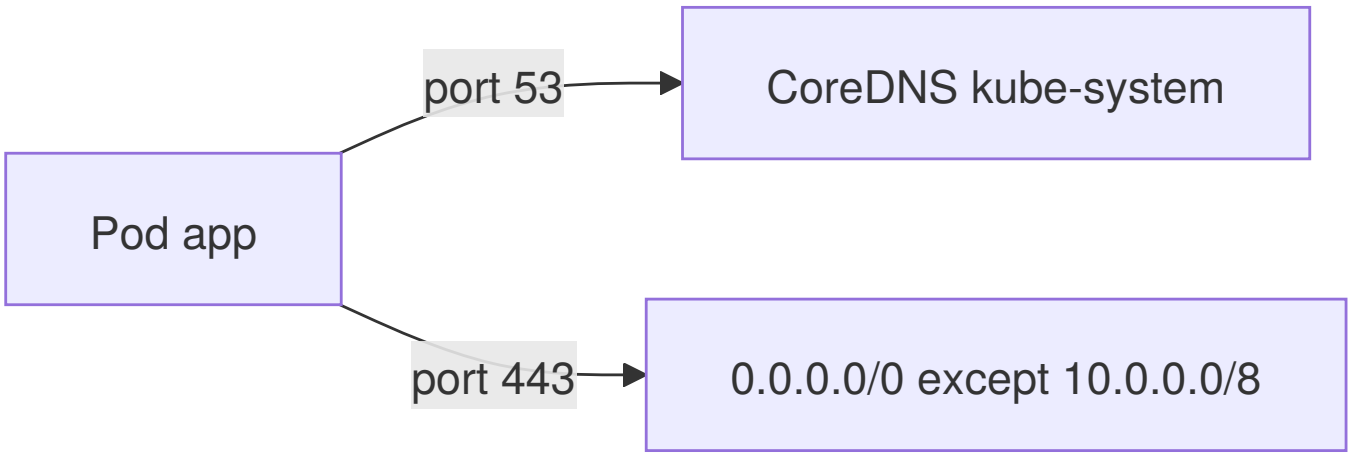
Default deny + explicit allow



AND vs OR in `from`



Egress DNS + external



💻 Real-World Examples

Example 1 — Default deny all ingress

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all-ingress, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Ingress]
```

Example 2 — Default deny all egress

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all-egress, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Egress]
```

Example 3 — Allow api → db on port 5432

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-api-db, namespace: prod }
spec:
  podSelector: { matchLabels: { app: db } }
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: { matchLabels: { app: api } }
    ports: [{ protocol: TCP, port: 5432 }]
```

Example 4 — Allow DNS egress for all pods

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-dns, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to: []                          # any destination
    ports:
    - { protocol: UDP, port: 53 }
    - { protocol: TCP, port: 53 }
```

Example 5 — Cross-namespace: frontend NS → api in prod NS

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-frontend, namespace: prod }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress]
  ingress:
  - from:
    - namespaceSelector: { matchLabels: { name: frontend } }
      podSelector: { matchLabels: { app: web } }
    ports: [{ port: 8080 }]
```

Example 6 — Allow egress to external CIDR only

```
spec:
  podSelector: { matchLabels: { app: worker } }
  policyTypes: [Egress]
  egress:
  - to:
    - ipBlock: { cidr: 0.0.0.0/0, except: [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16] }
    ports: [{ port: 443 }]
  - ports: [{ protocol: UDP, port: 53 }]     # DNS
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get netpol`	`-n <ns>`	List policies	`kubectl get netpol -n prod`
`kubectl describe netpol`	`<name>`	Show rules	`kubectl describe netpol allow-api-db -n prod`
`kubectl apply -f`	—	Apply policy	`kubectl apply -f deny-all.yaml`
`kubectl delete netpol`	`<name>`	Remove (reverts to default-allow)	`kubectl delete netpol deny-all-ingress -n prod`
`kubectl label ns`	`key=val`	Label NS for namespaceSelector	`kubectl label ns frontend name=frontend`
`kubectl exec`	`-- wget -qO- --timeout=3`	Test connectivity	`kubectl exec api-0 -n prod -- wget -qO- --timeout=3 http://db:5432`

🚨 Scenarios

Scenario 1 — Applied deny-all, now DNS is broken

- **Cause**: deny-all egress blocks port 53 to CoreDNS.
- **Fix**: add an egress rule allowing UDP/TCP 53 (Example 4).

Scenario 2 — Policy in place but traffic still flows

- **Cause**: CNI doesn't enforce NetworkPolicy (e.g., Flannel, or GKE Standard without DP-v2 / Calico enabled).
- **Fix**: enable a policy-capable CNI or switch to Dataplane V2 / Calico.

Scenario 3 — Cross-namespace traffic blocked unexpectedly

- **Cause**: namespace doesn't have the label `namespaceSelector` expects.
- **Fix**: `kubectl label ns <ns> name=<ns>` (K8s 1.21+ auto-labels `kubernetes.io/metadata.name`).

Scenario 4 — Ingress controller can't reach backend pods

- **Cause**: deny-all ingress blocks traffic from `ingress-nginx` namespace.
- **Fix**: add ingress rule with `namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: ingress-nginx}}`.

Scenario 5 — Network policy makes debugging impossible

- **Cause**: `kubectl exec` still works (it goes through apiserver, not the network), but `curl` from debug pods fails.
- **Fix**: temporarily relax policy for debug, or deploy debug pod in the same namespace with allowed labels.

☁ GCP GKE Perspective

- **Dataplane V2** (default on new clusters) enforces NetworkPolicy natively via eBPF. No additional setup.
- **GKE Standard without DP-v2**: enable Calico with `--enable-network-policy` at cluster creation.
- **Autopilot**: DP-v2 is always enabled; NetworkPolicy works out of the box.
- **GKE Enterprise**: **Anthos Network Policy** + **Policy Controller** for fleet-wide enforcement.

▩ AWS EKS Perspective

- **VPC CNI native Network Policy** (K8s 1.25+): enable with add-on parameter `enableNetworkPolicy: true`. Uses eBPF.
- Alternatively install **Calico** as a DaemonSet alongside VPC CNI (Calico handles policy, VPC CNI handles IP assignment).
- **Fargate**: limited NetworkPolicy support (VPC CNI native policy only, K8s 1.25+).
- **Security Groups for Pods** — a VPC-level alternative (or complement) to K8s NetworkPolicy; assigns actual AWS SGs to pods.

🧪 Hands-On Lab Exercises

Lab 22.1 — Default deny + whitelist (Beginner, 30 min)

Steps

```
kubectl create ns lab22
kubectl run web --image=nginx -n lab22 --labels=app=web
kubectl run client --image=busybox -n lab22 --labels=app=client -- sleep 3600
# Before policy
kubectl exec -n lab22 client -- wget -qO- --timeout=3 http://$(kubectl get pod web -n lab22 -o jsonpath='{.status.podIP}')
# → works

# Apply deny-all
kubectl apply -n lab22 -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all }
spec:
  podSelector: {}
  policyTypes: [Ingress]
EOF
kubectl exec -n lab22 client -- wget -qO- --timeout=3 http://$(kubectl get pod web -n lab22 -o jsonpath='{.status.podIP}')
# → timeout

# Whitelist client → web
kubectl apply -n lab22 -f - <<EOF
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-client }
spec:
  podSelector: { matchLabels: { app: web } }
  policyTypes: [Ingress]
  ingress:
  - from: [{ podSelector: { matchLabels: { app: client } } }]
    ports: [{ port: 80 }]
EOF
kubectl exec -n lab22 client -- wget -qO- --timeout=3 http://$(kubectl get pod web -n lab22 -o jsonpath='{.status.podIP}')
# → works
```

Cleanup: `kubectl delete ns lab22`.

Lab 22.2 — Cross-namespace + egress DNS (Intermediate, 45 min)

Steps 1. Create `ns-a` (label: `name=ns-a`) and `ns-b`. 2. Deploy `api` in `ns-b` with deny-all ingress. 3. Allow only pods from `ns-a` with label `role=gateway` to reach `api` on port 8080. 4. Test from `ns-a` pod with `role=gateway` → works. 5. Test from `ns-a` pod without label → blocked. 6. Add egress deny-all to `ns-b` → DNS breaks. Fix with egress DNS rule.

Cleanup: `kubectl delete ns ns-a ns-b`.

Lab 22.3 — Full zero-trust namespace on GKE/EKS (Advanced, 90 min)

Steps 1. On GKE (DP-v2) or EKS (VPC CNI native policy enabled). 2. Create namespace `secure` with: - deny-all ingress - deny-all egress - allow DNS egress - allow api → db ingress on 5432 - allow frontend NS → api on 8080 - allow egress to external HTTPS only (Example 6) 3. Deploy web, api, db pods with correct labels. 4. Verify each path: `kubectl exec` + `wget --timeout=3` . 5. Attempt unauthorized paths → all timeout.

Cleanup: delete namespace.

Bonus: Install Cilium Hubble (`cilium hubble ui`) and visually observe allowed vs denied flows.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A NetworkPolicy is a Kubernetes resource that controls L3/L4 traffic flow using label selectors. By default, all traffic is allowed. Applying a policy creates an implicit deny for the traffic type it covers — you then whitelist specific flows.
Why: Defense in depth. Even if an attacker compromises a frontend pod, a NetworkPolicy prevents it from reaching the database directly. It's the Kubernetes equivalent of a firewall.
How: I start every namespace with a deny-all ingress and deny-all egress policy, then add explicit allows: DNS egress to kube-system, app-to-app on specific ports, Ingress controller to frontend, and external HTTPS for API calls. I label namespaces for cross-namespace selectors and verify with `kubectl exec` connectivity tests. On GKE Dataplane V2 or EKS VPC CNI with native policy, enforcement is automatic — no extra CNI install.
Example: After a pen-test finding that any pod could reach our Postgres database, we rolled out default-deny plus whitelist across all production namespaces in a single sprint. The pen-tester re-tested and confirmed lateral movement was blocked."

? Possible Interview Questions

Basic

Q1. What does a NetworkPolicy do? Controls ingress/egress traffic to/from pods at L3/L4.
Q2. Default behavior without any policy? All traffic allowed.
Q3. Does kube-proxy enforce NetworkPolicy? No — the CNI plugin does.

Intermediate

Q4. How do you implement default-deny? Empty podSelector + policyTypes [Ingress] + no ingress rules.
Q5. AND vs OR in `from` rules? Separate list items = OR. podSelector + namespaceSelector in the same item = AND.
Q6. Which CNIs enforce NetworkPolicy? Calico, Cilium, GKE DP-v2, EKS VPC CNI (native). Flannel does NOT.

Advanced

Q7. How do you allow DNS after deny-all egress? Explicit egress rule for UDP+TCP port 53 to any destination (or to kube-system CoreDNS pods via namespaceSelector).
Q8. Can NetworkPolicy filter by Service name? No — it works at pod IP level. Service resolution happens before policy check.
Q9. Are policies additive or subtractive? Additive — you can only add allows, never explicit denies. Multiple policies union their allows.

Scenario-based

Q10. After applying deny-all, DNS broke — what happened? DNS queries (UDP 53) to CoreDNS blocked by egress deny. Fix: add egress DNS allow rule.

GKE-specific

Q11. Does Autopilot enforce NetworkPolicy? Yes — DP-v2 is always enabled on Autopilot.

EKS-specific

Q12. How do you enable native NetworkPolicy on EKS? Update VPC CNI add-on with `enableNetworkPolicy: true` (K8s 1.25+).

Hands-on / Lab-based

Q13. How do you test that a NetworkPolicy works? `kubectl exec <client> -- wget -qO- --timeout=3 http://<target-ip>:<port>` — should timeout if blocked, return content if allowed.
Q14. Show me a cross-namespace policy. `namespaceSelector: {matchLabels: {name: frontend}}` + `podSelector: {matchLabels: {role: gateway}}` in the ingress rule.
Q15. How would you visualize network flows? Cilium Hubble UI, Calico Flow Logs, or GKE Flow Logs in Cloud Logging.

📓 Topic 23 — CNI Plugins (Calico, Cilium, VPC CNI, GKE Dataplane V2)

Official Documentation Reference - CNI / Network Plugins: https://kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/ - Cluster Networking: https://kubernetes.io/docs/concepts/cluster-administration/networking/ - Calico: https://docs.tigera.io/calico/latest/about/ - Cilium: https://docs.cilium.io/en/stable/overview/intro/ - GKE DP-v2: https://cloud.google.com/kubernetes-engine/docs/concepts/dataplane-v2 - EKS VPC CNI: https://docs.aws.amazon.com/eks/latest/userguide/managing-vpc-cni.html - Flannel: https://github.com/flannel-io/flannel

---

🔷 Definition

A **CNI (Container Network Interface) plugin** is the component that assigns IP addresses to pods, wires their network namespaces, programs routes between nodes, and (optionally) enforces NetworkPolicy. Kubernetes delegates all networking to the CNI — it has no built-in network implementation.

🔹 About

Comparison table

Plugin	Data plane	Overlay	Network Policy	eBPF	Best for
Calico	iptables or eBPF	VXLAN/IPIP/none (BGP native)	✅ (rich)	Optional	On-prem, multi-cloud, policy-heavy
Cilium	eBPF	VXLAN/GENEVE/native	✅ (L3–L7)	Required	Observability, L7 policy, performance
Flannel	iptables	VXLAN/host-gw	❌	No	Simplest setup, learning
GKE Dataplane V2	eBPF (Cilium)	Native VPC routing	✅	Built-in	GKE (default)
VPC CNI (AWS)	iptables/eBPF	No overlay (native ENI IPs)	✅ (native, K8s 1.25+)	Optional	EKS default
Azure CNI	iptables	VNet / overlay	✅ (Calico or Cilium)	Optional	AKS
Weave Net	user-space / kernel	VXLAN	✅ (basic)	No	Legacy
kindnet	iptables	ptp	❌	No	Kind clusters (dev)

Overlay vs native routing

- **Overlay (VXLAN/GENEVE/IPIP)**: encapsulates pod packets in outer UDP. Works anywhere (even across cloud VPCs, on-prem). Adds ~50 bytes overhead + slight latency.
- **Native/BGP routing**: no encapsulation. Calico peers with ToR switches via BGP; cloud CNIs use VPC route tables or ENIs. Best performance but requires infrastructure support.

IP address management (IPAM)

Each CNI manages its own IPAM: - Calico: `IPPool` CRD defines pod CIDRs. - Cilium: `CiliumNode` stores pod CIDRs per node. - VPC CNI: secondary ENI IPs from VPC subnets; `WARM_ENI_TARGET`, `ENABLE_PREFIX_DELEGATION`. - GKE DP-v2: alias IP ranges from VPC.

eBPF advantage

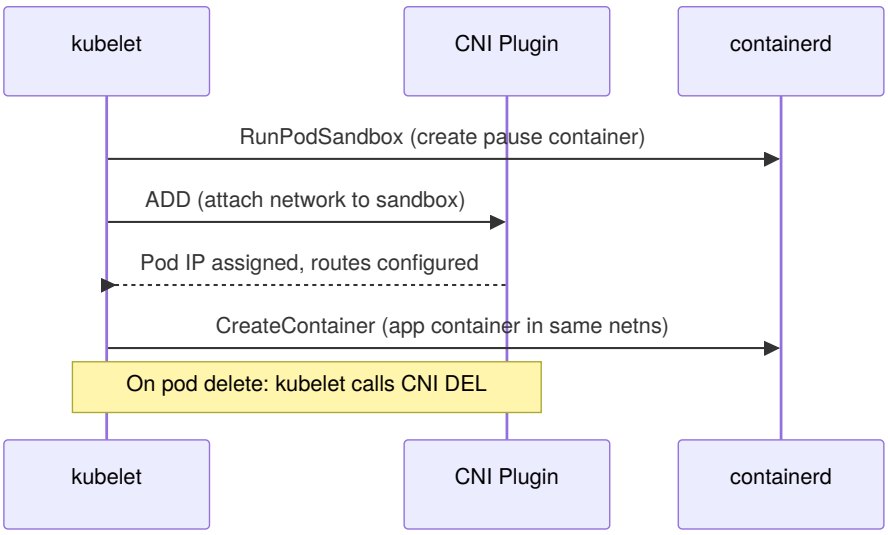
eBPF programs run inside the Linux kernel at socket, TC, and XDP hooks. They replace iptables for service routing (kube-proxy replacement), provide L7 visibility (Hubble on Cilium), and enforce NetworkPolicy with lower overhead and better debugging.

🔹 How It Integrates With Other Kubernetes Components

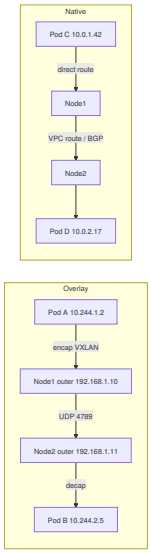
Component	Interaction
kubelet	Calls CNI binary at pod creation/deletion
kube-proxy	Works alongside CNI (or replaced by eBPF)
NetworkPolicy	CNI enforces (Calico/Cilium/DP-v2/VPC CNI)
CoreDNS	Depends on CNI for pod-to-pod connectivity
Service mesh	Runs on top of CNI networking layer
Scheduler	Node available only after CNI assigns IP

🏗 Diagrams

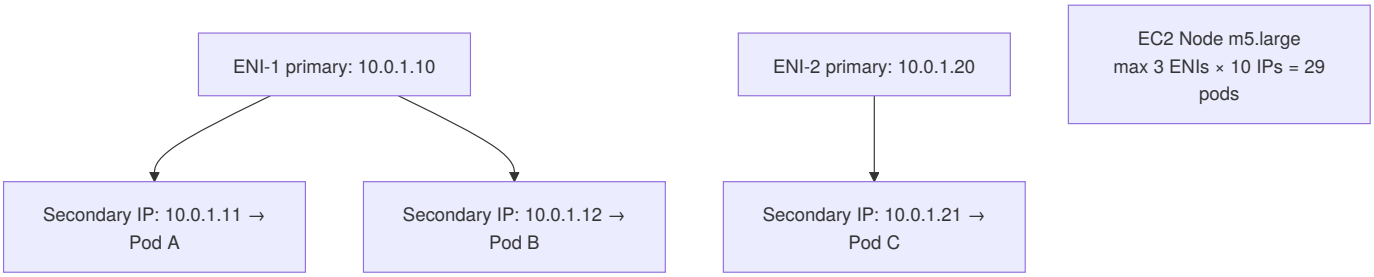
CNI in the pod lifecycle



Overlay vs native



VPC CNI IP allocation



💻 Real-World Examples

Example 1 — Install Calico on kubeadm

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/calico.yaml
kubectl get pods -n calico-system
kubectl get ippool -o yaml            # see CIDR, VXLAN mode
```

Example 2 — Install Cilium via Helm

```
helm repo add cilium https://helm.cilium.io/
helm install cilium cilium/cilium --version 1.16.0 \
  --namespace kube-system \
  --set kubeProxyReplacement=true \
  --set hubble.relay.enabled=true \
  --set hubble.ui.enabled=true
kubectl -n kube-system get pods -l k8s-app=cilium
cilium status
```

Example 3 — Check GKE Dataplane V2

```
gcloud container clusters describe prod --region asia-south1 \
  --format='value(networkPolicy.provider)'    # PROVIDER_UNSPECIFIED means DP-v2
kubectl get pods -n kube-system -l k8s-app=cilium
```

Example 4 — EKS VPC CNI tuning

```
# Enable prefix delegation for more pod IPs
kubectl set env daemonset aws-node -n kube-system \
  ENABLE_PREFIX_DELEGATION=true \
  WARM_PREFIX_TARGET=1

# Check max pods per node
kubectl get node <n> -o jsonpath='{.status.capacity.pods}'
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pods -n kube-system`	`-l k8s-app=calico-node`	Calico agent pods	
`kubectl get pods -n kube-system`	`-l k8s-app=cilium`	Cilium agent pods	
`calicoctl get ippool`	—	Calico IP pools	`calicoctl get ippool -o yaml`
`cilium status`	—	Cilium health	`cilium status`
`cilium connectivity test`	—	E2E connectivity	`cilium connectivity test`
`hubble observe`	`--follow`	Live flow logs	`hubble observe -n prod --follow`
`kubectl get ds aws-node`	`-n kube-system`	VPC CNI DaemonSet	
`kubectl describe node`	—	Pod CIDR, addresses	`kubectl describe node worker-1`
`kubectl logs -n kube-system`	`<cni-pod>`	CNI agent logs	

🚨 Scenarios

Scenario 1 — Nodes stay NotReady after kubeadm init

- **Cause**: no CNI installed.
- **Fix**: `kubectl apply -f calico.yaml` or install Cilium.

Scenario 2 — Pod-to-pod across nodes fails, same-node works

- **Cause**: overlay tunnel blocked (UDP 4789 / IP-in-IP protocol 4); or missing VPC routes.
- **Fix**: open firewall ports; on cloud check VPC route table for pod CIDRs.

Scenario 3 — VPC CNI IP exhaustion on EKS

- **Cause**: many pods per node without prefix delegation.
- **Fix**: enable `ENABLE_PREFIX_DELEGATION=true`; add secondary CIDRs to VPC; use `MINIMUM_IP_TARGET`.

Scenario 4 — Cilium Hubble shows drops on legitimate traffic

- **Cause**: L7 policy too restrictive or missing `toFQDNs` rule.
- **Fix**: `hubble observe --verdict DROPPED` → identify source/destination, refine CiliumNetworkPolicy.

Scenario 5 — Calico performance degraded at scale

- **Cause**: iptables mode with thousands of policies.
- **Fix**: switch to Calico eBPF data plane (`calicoctl` patch felixConfiguration).

☁ GCP GKE Perspective

- **Dataplane V2** (default since ~2023 on new clusters) = Cilium eBPF data plane managed by Google. Replaces kube-proxy. NetworkPolicy enforced natively. No user install needed.
- **VPC-native clusters** use alias IP ranges for pod CIDRs — no overlay.
- To get Cilium-level features (L7 policy, Hubble): install open-source Cilium alongside DP-v2 is not recommended — use **GKE Enterprise** features or standalone Cilium on Standard.
- On **Autopilot**: DP-v2 always active; no CNI choice.

▩ AWS EKS Perspective

- **VPC CNI** (default) — every pod gets a real VPC IP. No overlay.
- **Native Network Policy** (K8s 1.25+ add-on parameter `enableNetworkPolicy: true`) uses eBPF.
- **Calico** can run alongside VPC CNI for richer policy (Calico handles policy, VPC CNI handles IPAM).
- **Cilium** can replace VPC CNI entirely (with `--set eni.enabled=true` for ENI integration) — gives Hubble, L7 policy, kube-proxy replacement.
- Key tuning: `WARM_ENI_TARGET`, `ENABLE_PREFIX_DELEGATION`, `MINIMUM_IP_TARGET`, `MAX_ENI`.

🧪 Hands-On Lab Exercises

Lab 23.1 — Compare Flannel vs Calico on Kind (Beginner, 45 min)

Steps 1. Create Kind cluster with default CNI (kindnet). Run `kubectl get pods -A`. Test pod-to-pod. 2. Delete cluster. Create new Kind cluster with `disableDefaultCNI: true`:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
networking: { disableDefaultCNI: true }
nodes: [{ role: control-plane }, { role: worker }]
```

3. Nodes are NotReady. Install Calico: `kubectl apply -f calico.yaml`. Wait for Ready. 4. Apply a NetworkPolicy → enforced (unlike kindnet).

Cleanup: `kind delete cluster`.

Lab 23.2 — Cilium + Hubble observability (Intermediate, 60 min)

Steps 1. Kind cluster with `disableDefaultCNI: true`. 2. `helm install cilium ... --set kubeProxyReplacement=true --set hubble.relay.enabled=true --set hubble.ui.enabled=true`. 3. `cilium status` → OK. 4. `cilium connectivity test` → all pass. 5. Deploy web + client. Apply deny-all + whitelist NetworkPolicy. 6. `hubble observe -n default --follow` → see ALLOWED and DROPPED flows. 7. Port-forward Hubble UI: `cilium hubble ui`.

Cleanup: `kind delete cluster`.

Lab 23.3 — VPC CNI prefix delegation on EKS (Advanced, 60 min)

Steps 1. Create EKS cluster with `t3.medium` nodes (3 ENIs, 15 IPs default max). 2. Deploy 20 replicas → some Pending (IP exhaustion). 3. Enable prefix delegation:

```
kubectl set env ds aws-node -n kube-system ENABLE_PREFIX_DELEGATION=true WARM_PREFIX_TARGET=1
kubectl rollout restart ds aws-node -n kube-system
```

4. `kubectl get node <n> -o jsonpath='{.status.capacity.pods}'` → now higher (110+). 5. Scale to 50 replicas → all Running.

Cleanup: `eksctl delete cluster`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A CNI plugin is the component that assigns pod IPs, programs routes between nodes, and optionally enforces NetworkPolicy. Kubernetes has no built-in networking — the CNI contract delegates all of it.
Why: Different environments need different trade-offs. On-prem Calico with BGP gives native routing without overlay overhead. Cilium's eBPF data plane replaces kube-proxy and adds L7 visibility. Cloud-native CNIs like VPC CNI on EKS give every pod a real VPC IP for native security-group integration.
How: On GKE I rely on Dataplane V2 — it's Cilium-based, managed by Google, replaces kube-proxy, and enforces NetworkPolicy out of the box. On EKS I use VPC CNI with prefix delegation enabled so we get 100+ pods per node without IP exhaustion; and I enable native Network Policy. For on-prem kubeadm clusters I install Calico with VXLAN overlay and tight IPPool CIDRs.
Example: On a 50-node EKS cluster we hit pod IP limits at 15 pods per node. Enabling prefix delegation increased capacity to 110 pods per node — we avoided adding 30 extra nodes, saving roughly $8K/month."

? Possible Interview Questions

Basic

Q1. What is CNI? Container Network Interface — the standard Kubernetes uses for pod networking.
Q2. Name 3 CNI plugins. Calico, Cilium, Flannel (or VPC CNI, DP-v2).
Q3. Does Flannel enforce NetworkPolicy? No.

Intermediate

Q4. Overlay vs native routing? Overlay encapsulates (VXLAN); native uses host/VPC routes. Native is faster but needs infra support.
Q5. What does VPC CNI do differently? Assigns real VPC IPs to pods via secondary ENI addresses — no overlay, native VPC routing and SG integration.
Q6. What is GKE Dataplane V2? Google-managed Cilium eBPF data plane that replaces kube-proxy and enforces NetworkPolicy natively.

Advanced

Q7. Explain eBPF advantage for networking. eBPF programs run in-kernel at socket/TC/XDP hooks — replacing iptables with O(1) lookups, providing L7 visibility (Hubble), and eliminating kube-proxy overhead.
Q8. How does Calico BGP mode work? Calico's BIRD daemon peers with infrastructure routers, advertising pod CIDR routes. No encapsulation — packets are routed natively.
Q9. What is prefix delegation on VPC CNI? Instead of one IP per ENI slot, VPC CNI allocates a /28 prefix (16 IPs) per slot — dramatically increasing pod density.

Scenario-based

Q10. Nodes NotReady after kubeadm init — first check? Is a CNI installed? `kubectl get pods -n kube-system` → look for calico/cilium/flannel pods.

GKE-specific

Q11. Can you install your own Cilium on Autopilot? No — Autopilot uses DP-v2 exclusively. On Standard you can install open-source Cilium instead.

EKS-specific

Q12. How do you check max pods per node on EKS? `kubectl get node <n> -o jsonpath='{.status.capacity.pods}'` — determined by ENI count × IPs per ENI (or prefix delegation formula).

Hands-on / Lab-based

Q13. How did you enable Hubble? `helm install cilium ... --set hubble.relay.enabled=true --set hubble.ui.enabled=true`; then `cilium hubble ui` or `hubble observe`.
Q14. Walk me through troubleshooting cross-node connectivity. Check CNI DaemonSet running; check node firewall (VXLAN UDP 4789 or IPIP protocol 4); verify pod CIDR routes; `tcpdump` on node interfaces; check MTU.
Q15. How would you migrate from Flannel to Calico? Install Calico with `--set networking: none` (policy only) alongside Flannel first; then drain nodes, reconfigure to full Calico, uncordon.

📄 Topic 24 — DNS in Kubernetes (CoreDNS)

Official Documentation Reference - DNS for Services and Pods: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/ - Customizing DNS: https://kubernetes.io/docs/tasks/administer-cluster/dns-custom-nameservers/ - Debugging DNS resolution: https://kubernetes.io/docs/tasks/administer-cluster/dns-debugging-resolution/ - Using CoreDNS: https://kubernetes.io/docs/tasks/administer-cluster/coredns/ - NodeLocal DNSCache: https://kubernetes.io/docs/tasks/administer-cluster/nodelocaldns/ - GKE Cloud DNS: https://cloud.google.com/kubernetes-engine/docs/how-to/cloud-dns - EKS CoreDNS add-on: https://docs.aws.amazon.com/eks/latest/userguide/managing-coredns.html

Definition

CoreDNS is the default cluster DNS server in Kubernetes (since 1.13). It resolves Service names, headless pod names, and external hostnames for every pod in the cluster. Every pod's `/etc/resolv.conf` points to CoreDNS.

About

DNS records Kubernetes creates

Object	DNS record
ClusterIP Service	`<svc>.<ns>.svc.cluster.local` → ClusterIP
Headless Service	`<svc>.<ns>.svc.cluster.local` → set of pod A records
StatefulSet pod	`<pod>.<svc>.<ns>.svc.cluster.local` → pod IP
Pod (opt-in)	`<pod-ip-dashed>.<ns>.pod.cluster.local` → pod IP
ExternalName Service	CNAME → `spec.externalName`

Pod resolv.conf

```
nameserver 10.96.0.10          # CoreDNS ClusterIP
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

The `ndots:5` trap

If a name has fewer than 5 dots, the resolver appends each search domain before trying the absolute name. Querying `google.com` (1 dot) causes 4 failed queries before the absolute one succeeds. This wastes DNS capacity and adds latency.

Fix: set `dnsConfig.options: [{name: ndots, value: "2"}]` on pods that make many external calls, or always use FQDNs.

CoreDNS Corefile

The Corefile (ConfigMap `coredns` in `kube-system`) controls plugins: - `kubernetes` — handles cluster DNS - `forward` — upstream forwarder (default `/etc/resolv.conf` of the node) - `cache` — TTL caching (default 30 s) - `ready` — readiness endpoint - `health` — liveness endpoint - `log` — query logging (for debug) - `errors` — error logging

Stub domains / conditional forwarding

Forward `*.corp.example.com` to a corporate DNS:

```
corp.example.com:53 {
  forward . 10.0.0.2
}
```

Add as an entry in the Corefile (edit the ConfigMap).

NodeLocal DNSCache

A DaemonSet (`node-local-dns`) that runs a local caching DNS on each node at a link-local IP (`169.254.20.10`). Pods resolve via this cache → reduces cross-cluster DNS traffic, eliminates conntrack issues for UDP, and survives CoreDNS restarts.

DNS policies for pods

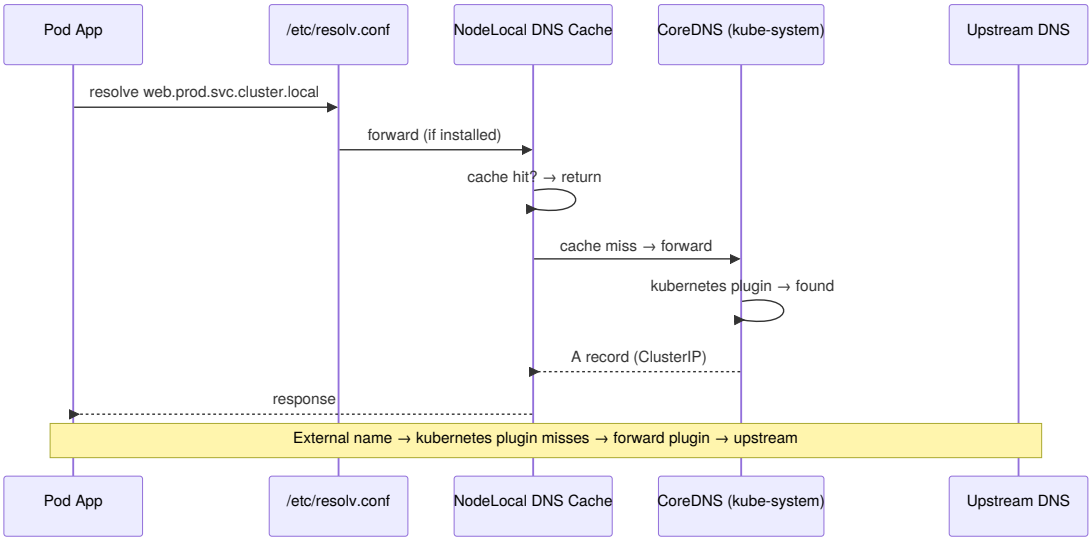
`dnsPolicy`	Behavior
`ClusterFirst` (default)	Use CoreDNS; fall back to node DNS for external
`Default`	Inherit node's `/etc/resolv.conf`
`None`	Entirely custom via `dnsConfig`
`ClusterFirstWithHostNet`	ClusterFirst even when `hostNetwork: true`

How It Integrates With Other Kubernetes Components

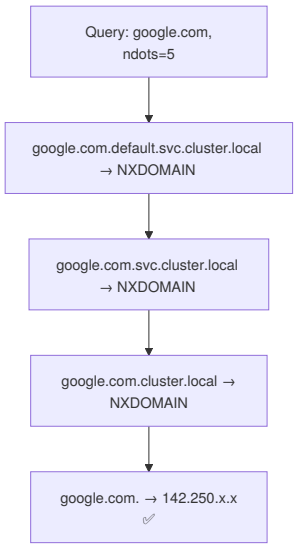
Component	Interaction
Service	CoreDNS watches Service/EndpointSlice objects to build DNS records
StatefulSet	Headless Service → per-pod DNS via CoreDNS
kubelet	Writes pod's `/etc/resolv.conf` with CoreDNS ClusterIP
NetworkPolicy	Must allow egress to CoreDNS (port 53 UDP/TCP)
Ingress / Gateway	External DNS (not CoreDNS) maps `app.example.com` → Ingress LB IP
NodeLocal DNSCache	DaemonSet that caches in front of CoreDNS

🏗 Diagrams

DNS resolution flow



ndots search expansion



💻 Real-World Examples

Example 1 — Debug DNS from a pod

```
kubectl run dns-debug --image=registry.k8s.io/e2e-test-images/agnhost:2.45 --rm -it -- \
  /agnhost dns-test --upstream=10.96.0.10 --query=web.default.svc.cluster.local --count=1
# Or simpler
kubectl run tmp --image=busybox --rm -it -- nslookup web.default.svc.cluster.local
```

Example 2 — Custom Corefile with stub domain

```
kubectl edit cm coredns -n kube-system
```

Add after the main `.:53` block:

```
corp.example.com:53 {
    errors
    cache 30
    forward . 10.0.0.2
}
```

Then: `kubectl rollout restart deploy coredns -n kube-system`.

Example 3 — Pod with custom DNS config

```
spec:
  dnsPolicy: None
  dnsConfig:
    nameservers: ["10.96.0.10"]
    searches: ["prod.svc.cluster.local","svc.cluster.local"]
    options: [{ name: ndots, value: "2" }]
```

Example 4 — NodeLocal DNSCache install

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/kubernetes/master/cluster/addons/dns/nodelocaldns/nodelocaldns.yaml
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get svc -n kube-system`	`kube-dns`	CoreDNS Service ClusterIP	`kubectl get svc kube-dns -n kube-system`
`kubectl get pods -n kube-system`	`-l k8s-app=kube-dns`	CoreDNS pods	
`kubectl logs -n kube-system`	`<coredns-pod>`	DNS logs	
`kubectl get cm coredns`	`-n kube-system -o yaml`	Corefile	
`kubectl exec`	`-- nslookup <svc>`	DNS lookup	`kubectl exec tmp -- nslookup web`
`kubectl exec`	`-- cat /etc/resolv.conf`	Check resolver config	
`kubectl rollout restart`	`deploy coredns -n kube-system`	Reload after Corefile edit	
`kubectl top pod`	`-n kube-system -l k8s-app=kube-dns`	CoreDNS resource usage	

🚨 Scenarios

Scenario 1 — All DNS resolution fails

- **Cause**: CoreDNS pods crashed or unschedulable.
- **Fix**: `kubectl get pods -n kube-system -l k8s-app=kube-dns` → check status. Restart: `kubectl rollout restart deploy coredns -n kube-system`.

Scenario 2 — External DNS slow but internal fast

- **Cause**: `ndots:5` causing 4 wasted queries.
- **Fix**: set `ndots:2` on affected pods, or use FQDNs with trailing dot (`google.com.`).

Scenario 3 — DNS 5-second delay (conntrack race)

- **Cause**: UDP conntrack race condition in kernel < 5.0.
- **Fix**: install NodeLocal DNSCache (uses TCP or link-local); or upgrade kernel.

Scenario 4 — Custom stub domain not resolving

- **Cause**: Corefile syntax error; or CoreDNS not restarted.
- **Fix**: validate Corefile; `kubectl rollout restart deploy coredns -n kube-system`.

Scenario 5 — CoreDNS OOMKilled at scale (thousands of Services)

- **Cause**: too many watches, cache pressure.
- **Fix**: increase CoreDNS memory limits; scale CoreDNS replicas; use dns-autoscaler; enable NodeLocal DNSCache.

☁ GCP GKE Perspective

- **Cloud DNS for GKE** (`--cluster-dns=clouddns`): replaces CoreDNS with a fully managed Google Cloud DNS service. Eliminates CoreDNS scaling concerns. Supports cross-cluster DNS with GKE Multi-cluster Services.
- **NodeLocal DNSCache**: enabled with `gcloud container clusters update --update-addons=NodeLocalDNS=ENABLED`.
- **Autopilot**: Cloud DNS is default; CoreDNS not user-visible.

▩ AWS EKS Perspective

- **CoreDNS** is an EKS managed add-on. Upgrade with `aws eks update-addon --addon-name coredns`.
- **Cluster proportional autoscaler** (`dns-autoscaler`) scales CoreDNS replicas based on node/pod count.
- **NodeLocal DNSCache**: not managed by EKS; install manually or via Helm.
- **Route 53 Resolver** can be forwarded to for private hosted zones via Corefile stub domains.
- **ExternalDNS controller** automatically creates Route 53 records for Ingress/Service objects.

🧪 Hands-On Lab Exercises

Lab 24.1 — DNS resolution basics (Beginner, 30 min)

Steps

```
kubectl create deploy web --image=nginx
kubectl expose deploy web --port=80
kubectl run tmp --image=busybox --rm -it -- sh
# inside:
nslookup web
nslookup web.default.svc.cluster.local
nslookup google.com
cat /etc/resolv.conf
exit
```

Verify: `web` resolves to ClusterIP; `google.com` to public IP; resolv.conf shows `ndots:5`.

Lab 24.2 — Fix ndots latency (Intermediate, 45 min)

Steps 1. Deploy a pod that curls `api.github.com` in a loop; time each request. 2. Observe 4 NXDOMAIN + 1 success per lookup: `kubectl logs -n kube-system <coredns> | grep github` (enable `log` plugin first). 3. Patch pod `dnsConfig: {options: [{name: ndots, value: "2"}]}`. 4. Time again — faster (1 query instead of 5).

Bonus: compare with FQDN `api.github.com.` (trailing dot).

Lab 24.3 — NodeLocal DNSCache + stub domain (Advanced, 60 min)

Steps 1. Install NodeLocal DNSCache. 2. Verify pod resolv.conf points to `169.254.20.10`. 3. `kubectl exec tmp -- nslookup web` → served by local cache. 4. Add a stub domain for `corp.example.com` pointing to a local DNS server (deploy CoreDNS as a "corp" server). 5. Verify `kubectl exec tmp -- nslookup intranet.corp.example.com` resolves via stub.

Cleanup: remove NodeLocal DNSCache DaemonSet; revert Corefile.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: CoreDNS is the default cluster DNS in Kubernetes. It watches Services and EndpointSlices and creates DNS records — `svc.ns.svc.cluster.local` for ClusterIP Services, per-pod records for headless Services. Every pod's `/etc/resolv.conf` points to CoreDNS.
Why: DNS is how pods discover Services without hard-coding IPs. It's also the path for external resolution — pods reach the internet via CoreDNS forwarding to upstream.
How: I tune two things in production. First, `ndots` — the default of 5 causes wasted lookups for external names; I set it to 2 on pods that do heavy external calls. Second, I install NodeLocal DNSCache to eliminate the UDP conntrack race and reduce cross-node DNS traffic. On GKE I use Cloud DNS for zero-ops scaling; on EKS I scale CoreDNS with the proportional autoscaler.
Example: We saw intermittent 5-second DNS delays on a 100-node EKS cluster. Root cause was the UDP conntrack race. Installing NodeLocal DNSCache eliminated it completely — p99 DNS latency dropped from 5 s to 2 ms."

❓ Possible Interview Questions

Basic

Q1. What is CoreDNS? Kubernetes' default cluster DNS server; resolves Service names to IPs.
Q2. What DNS record does a ClusterIP Service get? `<svc>.<ns>.svc.cluster.local` → ClusterIP.
Q3. Where is the CoreDNS config? ConfigMap `coredns` in `kube-system`.

Intermediate

Q4. What is `ndots:5` and why is it a problem? Names with < 5 dots try all search domains before absolute — 4 wasted queries for external names.
Q5. What is a headless Service's DNS record? Returns A records for each pod IP, not a ClusterIP.
Q6. How do you forward queries for `*.corp.com` to a corporate DNS? Add a stub-domain block in the Corefile.

Advanced

Q7. What is NodeLocal DNSCache? A per-node caching DNS DaemonSet at a link-local IP. Reduces CoreDNS load, eliminates conntrack race for UDP.
Q8. What is the 5-second DNS delay on Linux < 5.0? A conntrack table race where simultaneous A and AAAA queries over the same UDP socket cause a 5 s timeout on one.
Q9. How does GKE Cloud DNS differ from CoreDNS? Managed by Google, auto-scales, supports multi-cluster DNS (MCS), no in-cluster pods to manage.

Scenario-based

Q10. DNS works for internal Services but not for `google.com` — steps? Check CoreDNS `forward` plugin upstream, node DNS config (`/etc/resolv.conf` on host), firewall allowing UDP 53 outbound.

GKE-specific

Q11. How do you enable Cloud DNS on GKE? `gcloud container clusters update <n> --cluster-dns=clouddns --cluster-dns-scope=cluster`.

EKS-specific

Q12. How do you auto-scale CoreDNS on EKS? Install `cluster-proportional-autoscaler` targeting the `coredns` Deployment.

Hands-on / Lab-based

Q13. How do you debug DNS from a pod? `kubectl exec -- nslookup <name>`, `kubectl exec -- cat /etc/resolv.conf`, `kubectl logs <coredns-pod>`.
Q14. How would you measure the ndots impact? Enable CoreDNS `log` plugin; query an external domain; count NXDOMAIN entries.
Q15. How do you test a stub-domain change? Edit Corefile ConfigMap, restart CoreDNS, `nslookup <stub-domain-name>` from a pod.

🗐 Topic 25 — Service Mesh Overview (Istio, Linkerd)

Official Documentation Reference - Istio: https://istio.io/latest/docs/overview/what-is-istio/ - Linkerd: https://linkerd.io/2/overview/ - Kubernetes Service concept: https://kubernetes.io/docs/concepts/services-networking/service/ - GKE Anthos Service Mesh / Istio: https://cloud.google.com/service-mesh/docs/overview - EKS App Mesh: https://docs.aws.amazon.com/app-mesh/latest/userguide/what-is-app-mesh.html - EKS + Istio: https://docs.aws.amazon.com/eks/latest/best-practices/service-mesh.html

🔹 Definition

A **service mesh** is a dedicated infrastructure layer that adds transparent networking capabilities to microservices — **mutual TLS (mTLS)**, **traffic management** (canary, retries, timeouts, circuit breaking), **observability** (distributed tracing, metrics, access logs), and **policy enforcement** — without changing application code. It does this by injecting a **sidecar proxy** (Envoy or linkerd2-proxy) into every pod.

🔹 About

What problem does it solve?

Without a mesh, every microservice team must implement retries, timeouts, TLS, tracing headers, and circuit breaking in code (per language). A mesh extracts those cross-cutting concerns into a proxy layer — consistent across Go, Java, Python, Node, etc.

Comparison

Feature	Istio	Linkerd	AWS App Mesh
Proxy	Envoy	linkerd2-proxy (Rust)	Envoy
mTLS	✅ auto	✅ auto	✅
Traffic mgmt	Rich (VirtualService, DestinationRule)	Basic (HTTPRoute, TrafficSplit)	VirtualNode/VirtualRouter
Observability	Prometheus, Jaeger, Kiali	Built-in dashboard, Prometheus	CloudWatch, X-Ray
Complexity	High (many CRDs, Envoy config)	Low (opinionated, lightweight)	Medium (AWS-managed CP)
Resource overhead	~100 Mi / sidecar	~10–20 Mi / sidecar	~100 Mi / sidecar
CNCF status	Graduated	Graduated	AWS proprietary
Best for	Enterprise, complex traffic	Simplicity, low overhead	AWS-native shops

Sidecar injection

Both Istio and Linkerd use a **mutating admission webhook** that auto-injects a proxy sidecar into every pod in labeled namespaces.

Key Istio CRDs

CRD	Purpose
`VirtualService`	L7 routing (canary weight, header match, retries)
`DestinationRule`	Load balancing policy, connection pool, outlier detection
`Gateway`	Ingress point for mesh traffic
`PeerAuthentication`	mTLS mode (STRICT / PERMISSIVE)
`AuthorizationPolicy`	L7 access control (source, method, path)

Key Linkerd CRDs

CRD	Purpose
`HTTPRoute` (Gateway API)	L7 routing
`TrafficSplit` (SMI)	Canary weight shift
`Server` / `ServerAuthorization`	Fine-grained L4/L7 policy

Ambient mesh (Istio 1.22+)

Sidecar-less mode: replaces per-pod Envoy with a **ztunnel** DaemonSet (L4 mTLS) + optional **waypoint proxies** (L7). Reduces resource overhead dramatically.

When NOT to use a mesh

- < 5 services (overhead > benefit).
- Latency-critical path where even 1 ms sidecar hop matters.
- Team lacks ops capacity for mesh upgrades.
- Simple mTLS-only need → consider SPIFFE/SPIRE or native Cilium mTLS.

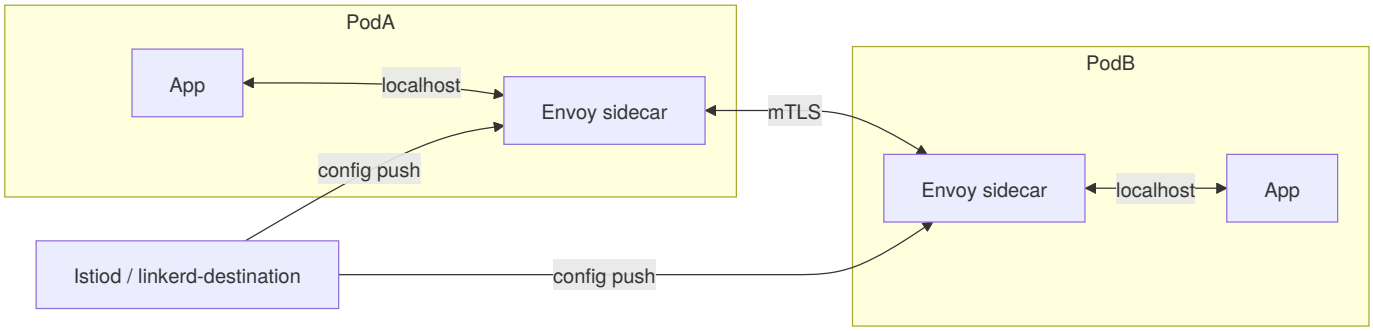
🔷 How It Integrates With Other Kubernetes Components

Component	Interaction
Pods	Sidecar proxy injected via admission webhook
Services	Mesh intercepts Service traffic via iptables/eBPF redirect
Ingress / Gateway	Mesh Gateway replaces or supplements K8s Ingress
NetworkPolicy	Mesh operates at L7 above L3/L4 NetworkPolicy
cert-manager	Istio uses its own CA (or integrates with cert-manager / Vault for custom CA)
Prometheus	Mesh proxies expose rich metrics (`istio_requests_total` , `response_latency`)
Tracing (Jaeger/Zipkin/Tempo)	Mesh propagates trace headers automatically

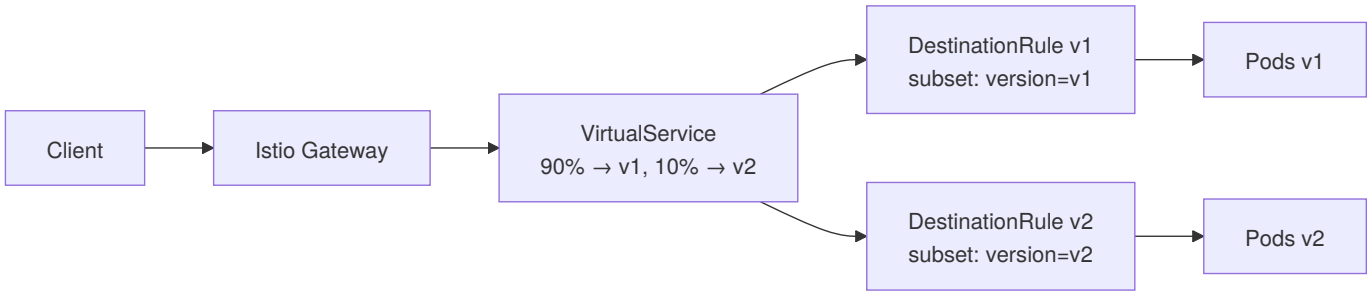
---

🏗 Diagrams

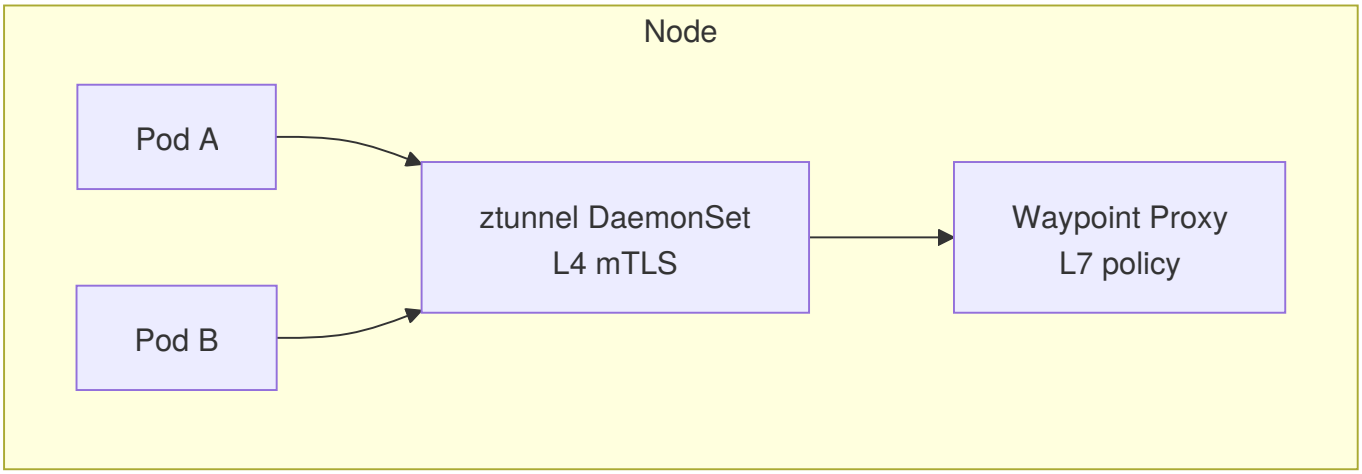
Sidecar model



Istio traffic management



Ambient mesh (sidecar-less)



💻 Real-World Examples

Example 1 — Install Istio + inject sidecar

```
istioctl install --set profile=demo -y
kubectl label namespace default istio-injection=enabled
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.22/samples/bookinfo/platform/kube/bookinfo.yaml
kubectl get pods       # each pod has 2/2 containers (app + envoy)
```

Example 2 — Canary with VirtualService

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: { name: reviews }
spec:
  hosts: [reviews]
  http:
  - route:
    - destination: { host: reviews, subset: v1 }
      weight: 90
    - destination: { host: reviews, subset: v2 }
      weight: 10
---
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata: { name: reviews }
spec:
  host: reviews
  subsets:
  - { name: v1, labels: { version: v1 } }
  - { name: v2, labels: { version: v2 } }
```

Example 3 — Install Linkerd

```
linkerd install --crds | kubectl apply -f -
linkerd install | kubectl apply -f -
linkerd check
kubectl annotate namespace default linkerd.io/inject=enabled
kubectl rollout restart deploy/web
linkerd viz dashboard
```

Example 4 — Strict mTLS (Istio)

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata: { name: strict, namespace: prod }
spec:
  mtls: { mode: STRICT }
```

Example 5 — AuthorizationPolicy (Istio)

```
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata: { name: allow-api, namespace: prod }
spec:
  selector: { matchLabels: { app: db } }
  action: ALLOW
  rules:
  - from: [{ source: { principals: ["cluster.local/ns/prod/sa/api"] } }]
    to: [{ operation: { ports: ["5432"] } }]
```

💻 Related Commands

Command	Flag	Purpose	Example
`istioctl install`	`--set profile=`	Install Istio	`istioctl install --set profile=default`
`istioctl analyze`	`-n <ns>`	Validate mesh config	`istioctl analyze -n prod`
`istioctl proxy-status`	—	Check sidecar sync	`istioctl proxy-status`
`istioctl proxy-config`	`routes <pod>`	Inspect Envoy routes	`istioctl proxy-config routes web-0`
`istioctl dashboard kiali`	—	Open Kiali UI	`istioctl dashboard kiali`
`linkerd install`	—	Install Linkerd	`linkerd install | kubectl apply -f -`
`linkerd check`	—	Health check	`linkerd check`
`linkerd viz dashboard`	—	Open dashboard	`linkerd viz dashboard`
`linkerd viz stat`	`deploy -n <ns>`	Live metrics	`linkerd viz stat deploy -n prod`
`kubectl get vs,dr`	—	Istio routing objects	`kubectl get virtualservice,destinationrule`

🚨 Scenarios

Scenario 1 — Sidecar not injected

- **Cause**: namespace not labeled `istio-injection=enabled` (or `linkerd.io/inject=enabled`).
- **Fix**: label the namespace; restart pods.

Scenario 2 — mTLS breaks non-meshed service

- **Cause**: strict mTLS enforced; unmeshed pod can't present cert.
- **Fix**: use `PeerAuthentication: PERMISSIVE` during migration; mesh all services before going STRICT.

Scenario 3 — Canary weight not taking effect

- **Cause**: VirtualService host doesn't match Service name; or DestinationRule subsets don't match pod labels.
- **Fix**: `istioctl analyze` → shows mismatches.

Scenario 4 — Increased latency after mesh install

- **Cause**: sidecar adds ~1–3 ms per hop; also resource contention if sidecar limits too low.
- **Fix**: set sidecar resource requests/limits; use Linkerd for lower overhead; or consider Istio Ambient for critical paths.

Scenario 5 — Mesh upgrade breaks traffic

- **Cause**: control-plane/data-plane version mismatch after partial upgrade.
- **Fix**: follow canary upgrade path: upgrade control plane → upgrade sidecars namespace by namespace → verify with `istioctl proxy-status`.

☁ GCP GKE Perspective

- **Anthos Service Mesh (ASM)** — Google's managed Istio distribution. `asmcli install` or Fleet API. Includes managed control plane, telemetry → Cloud Monitoring, and multi-cluster mesh via Fleet.
- **GKE Gateway API** can coexist with or replace Istio Gateway.
- **Autopilot** supports ASM (managed Istiod).
- **Cloud Service Mesh** (2024+) rebranding — includes multi-cluster, multi-cloud traffic management.

▩ AWS EKS Perspective

- **AWS App Mesh** — managed Envoy-based mesh. VirtualNode, VirtualService, VirtualRouter CRDs. Integrates with X-Ray for tracing, CloudWatch for metrics.
- **Open-source Istio on EKS** — community-supported; use `istioctl` + Helm. Many teams prefer this over App Mesh for portability.
- **Linkerd on EKS** — lightweight, CNCF graduated, works well with VPC CNI.
- EKS Blueprints addons include Istio and App Mesh patterns.

🧪 Hands-On Lab Exercises

Lab 25.1 — Istio Bookinfo with canary (Beginner, 60 min)

Steps 1. `istioctl install --set profile=demo -y`. 2. `kubectl label ns default istio-injection=enabled`. 3. Deploy Bookinfo sample. 4. Apply Example 2 VirtualService + DestinationRule. 5. `for i in $(seq 1 100); do curl -s http://$GATEWAY_URL/productpage | grep reviews; done` → 90% v1, 10% v2. 6. Shift to 50/50; then 100% v2.

Cleanup: `istioctl uninstall --purge -y && kubectl delete ns istio-system`.

Lab 25.2 — Linkerd with live dashboard (Intermediate, 45 min)

Steps 1. Install Linkerd + viz extension. 2. `linkerd check` → all green. 3. Inject namespace: `kubectl annotate ns default linkerd.io/inject=enabled && kubectl rollout restart deploy`. 4. `linkerd viz dashboard` → see live p50/p99 latency, success rate, RPS per deployment. 5. Simulate failure → observe success rate drop in dashboard.

Cleanup: `linkerd viz uninstall | kubectl delete -f - && linkerd uninstall | kubectl delete -f -`.

Lab 25.3 — mTLS verification + AuthorizationPolicy (Advanced, 90 min)

Steps 1. Install Istio (default profile). 2. Set PeerAuthentication to STRICT in `prod` namespace. 3. Deploy `api` and `db`. Verify traffic works (both have sidecars). 4. Deploy `attacker` pod **without** sidecar (`sidecar.istio.io/inject: "false"`). 5. `kubectl exec attacker -- curl db:5432` → **connection refused** (no mTLS cert). 6. Apply AuthorizationPolicy (Example 5) → only `api` SA can reach `db:5432`. 7. Test from another meshed pod with different SA → **denied (403)**.

Cleanup: `istioctl uninstall --purge -y`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A service mesh is an infrastructure layer that transparently handles mTLS, traffic management, observability, and policy enforcement between microservices — typically by injecting a sidecar proxy like Envoy (Istio) or linkerd2-proxy (Linkerd) into every pod.
Why: It decouples cross-cutting networking concerns from application code. Every team doesn't need to implement retries, circuit breaking, TLS, and tracing in their language — the proxy does it uniformly. It also enables zero-trust networking via mutual TLS without code changes.
How: For most teams I recommend starting with Linkerd if they want simplicity, or Istio if they need advanced traffic management like weighted canaries, header-based routing, and fine-grained AuthorizationPolicy. On GKE I use Anthos Service Mesh (managed Istio). On EKS, either open-source Istio or Linkerd — App Mesh is an option but less portable. I roll out in stages: install control plane → inject one namespace → verify metrics/latency → expand.
Example: We adopted Istio to implement canary releases for our payment service. Before the mesh, canaries required custom NGINX config. With Istio, a VirtualService weight shift from 5% to 100% replaced a 2-hour manual process with a 30-second `kubectl apply`."

? Possible Interview Questions

Basic

Q1. What is a service mesh? Infrastructure layer adding mTLS, traffic control, observability, and policy via sidecar proxies — without app code changes.
Q2. Name two popular meshes. Istio (Envoy), Linkerd (linkerd2-proxy).
Q3. How does the sidecar get injected? Mutating admission webhook adds a proxy container to the pod spec.

Intermediate

Q4. Istio vs Linkerd? Istio: powerful, complex, Envoy, rich CRDs. Linkerd: lightweight (~10 Mi sidecar), simple, CNCF graduated.
Q5. What is mTLS and why does a mesh enable it? Mutual TLS = both client and server authenticate via certificates. The mesh's control plane issues certs automatically; proxies encrypt pod-to-pod traffic transparently.
Q6. What is a VirtualService in Istio? L7 routing rule — canary weights, header matches, retries, fault injection.

Advanced

Q7. Explain Istio Ambient mode. Replaces per-pod sidecars with a ztunnel DaemonSet (L4 mTLS) + optional waypoint proxies (L7). Reduces memory overhead by ~90%.
Q8. How does AuthorizationPolicy differ from NetworkPolicy? NetworkPolicy is L3/L4 (IP/port). AuthorizationPolicy is L7 (HTTP method, path, source identity/SA).
Q9. When would you NOT use a mesh? < 5 services, latency-critical with no budget for sidecar hop, team can't maintain mesh upgrades, or Cilium mTLS covers the need.

Scenario-based

Q10. After enabling strict mTLS, external traffic fails — why? External callers don't have mesh certs. Fix: use an Istio Gateway (terminates external TLS, presents mesh cert internally) or set PeerAuthentication to PERMISSIVE on the edge service.

GKE-specific

Q11. What is Anthos Service Mesh? Google-managed Istio. Managed control plane, Cloud Monitoring integration, multi-cluster mesh via Fleet.

EKS-specific

Q12. Istio or App Mesh on EKS? Istio for portability and community; App Mesh for tight AWS integration (X-Ray, CloudWatch). Many teams prefer Istio.

Hands-on / Lab-based

Q13. How would you verify mTLS is working? `istioctl proxy-config secret <pod>` shows active certs; or `kubectl exec -c istio-proxy <pod> -- curl localhost:15000/certs`; or check Kiali for padlock icons.
Q14. Show me a canary rollout with Istio. VirtualService with `route: [{destination: {subset: v1}, weight: 90}, {destination: {subset: v2}, weight: 10}]`.
Q15. How do you monitor mesh health? `istioctl proxy-status` for sync status; Prometheus scraping `istio_requests_total`; Kiali/Linkerd viz for topology and success rates.

MODULE

Module 5 — Storage

🗐 Topic 26 — Volumes (emptyDir, hostPath, etc.)

Official Documentation Reference - Volumes: https://kubernetes.io/docs/concepts/storage/volumes/ - Ephemeral volumes: https://kubernetes.io/docs/concepts/storage/ephemeral-volumes/ - Projected volumes: https://kubernetes.io/docs/concepts/storage/projected-volumes/ - Configure volume storage: https://kubernetes.io/docs/tasks/configure-pod-container/configure-volume-storage/ - GKE volumes: https://cloud.google.com/kubernetes-engine/docs/concepts/volumes-overview - EKS storage: https://docs.aws.amazon.com/eks/latest/userguide/storage.html

🔹 Definition

A **Volume** in Kubernetes is a directory accessible to containers in a pod. Unlike a container's writable layer (ephemeral, dies with the container), a volume has a lifetime tied to the **pod** (or longer, for persistent volumes). Volumes solve data sharing between containers, crash-safe scratch space, and persistence.

🔹 About

Volume types (most common)

Type	Lifetime	Shared?	Use case
`emptyDir`	Pod	Between containers in same pod	Scratch space, cache, sidecar data exchange
`emptyDir` (medium: Memory)	Pod	Yes	tmpfs RAM disk — fast, counts against memory limit
`hostPath`	Node	No (node-local)	Access node files — logs, Docker socket, device files
`configMap`	Pod	Yes	Mount config as files
`secret`	Pod	Yes (tmpfs)	Mount secrets as files
`downwardAPI`	Pod	Yes	Expose pod metadata as files
`projected`	Pod	Yes	Combine configMap + secret + downwardAPI + SA token
`nfs`	External server	Yes (RWX)	Shared filesystem across pods/nodes
`persistentVolumeClaim`	PVC lifetime	Depends on access mode	Durable block/file storage
`csi`	CSI driver	Depends	Any CSI-backed storage

emptyDir deep dive

- Created when pod is scheduled to a node; deleted when pod is removed.
- `sizeLimit` (K8s 1.24+ stable) caps usage; exceeds → pod eviction.
- `medium: Memory` creates a tmpfs — faster but uses pod's memory budget.

hostPath warnings

- **Never use in production Deployments** — pod can land on any node; data is not portable.
- Security risk: mounting `/` or `/var/run/docker.sock` gives container root-level node access.
- **GKE Autopilot blocks hostPath** by default.
- **EKS Fargate blocks hostPath**.
- Valid uses: DaemonSets reading node logs (`/var/log`), single-node dev clusters.

projected volumes

Merge multiple sources into one mount:

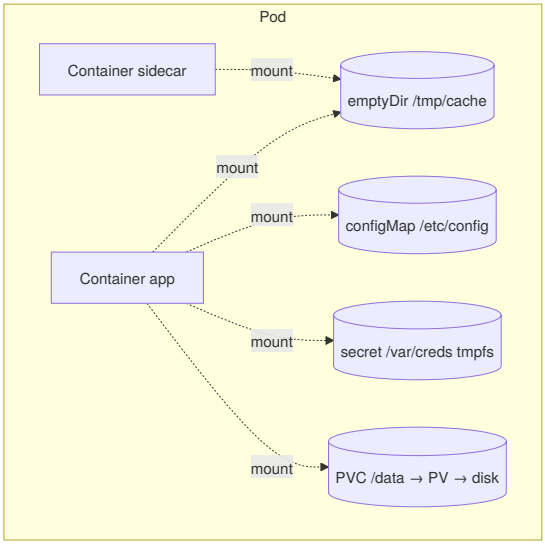
```
volumes:
- name: all
  projected:
    sources:
    - configMap: { name: app-config }
    - secret: { name: app-secret }
    - downwardAPI: { items: [{ path: labels, fieldRef: { fieldPath: metadata.labels } }] }
    - serviceAccountToken: { path: token, expirationSeconds: 3600, audience: vault }
```

🔹 How It Integrates With Other Kubernetes Components

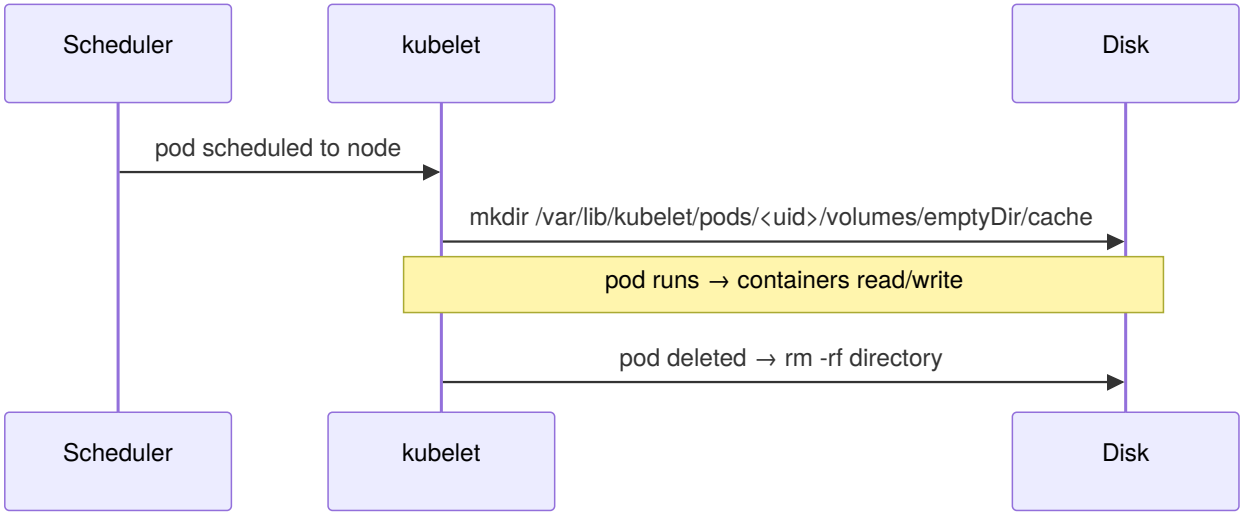
Component	Interaction
Pod	`spec.volumes` + container `volumeMounts`
ConfigMap / Secret	Special volume types
PVC	`persistentVolumeClaim` volume type links to durable storage
CSI driver	`csi` volume type for external providers
kubelet	Manages mount lifecycle per pod
Scheduler	Topology-aware scheduling for zonal volumes
SecurityContext	`readOnlyRootFilesystem`, `fsGroup`, `supplementalGroups` affect volume permissions

🏗 Diagrams

Volume types in a pod



emptyDir lifecycle



💻 Real-World Examples

Example 1 — emptyDir for sidecar data sharing

```
apiVersion: v1
kind: Pod
metadata: { name: share }
spec:
  containers:
  - name: writer
    image: busybox
    command: ["sh","-c","while true; do date >> /data/log.txt; sleep 5; done"]
    volumeMounts: [{ name: shared, mountPath: /data }]
  - name: reader
    image: busybox
    command: ["sh","-c","tail -f /data/log.txt"]
    volumeMounts: [{ name: shared, mountPath: /data }]
  volumes:
  - name: shared
    emptyDir: {}
```

Example 2 — emptyDir with sizeLimit + memory medium

```
volumes:
- name: cache
  emptyDir:
    medium: Memory
    sizeLimit: 256Mi
```

Example 3 — hostPath for node-exporter DaemonSet

```
volumes:
- name: rootfs
  hostPath: { path: /, type: Directory }
- name: proc
  hostPath: { path: /proc, type: "" }
- name: sys
  hostPath: { path: /sys, type: "" }
```

Example 4 — Projected volume (ConfigMap + Secret + SA token)

```
volumes:
- name: all-in-one
  projected:
    sources:
    - configMap: { name: app-config, items: [{ key: config.yaml, path: config.yaml }] }
    - secret:    { name: db-cred,    items: [{ key: PASSWORD, path: db-password }] }
    - serviceAccountToken: { path: token, expirationSeconds: 3600, audience: api }
```

Example 5 — NFS volume

```
volumes:
- name: nfs-data
  nfs:
    server: 10.0.0.50
    path: /exports/data
    readOnly: false
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl describe pod`	—	See volume mounts + events	`kubectl describe pod share`
`kubectl exec`	`-- ls /data`	List mounted files	`kubectl exec share -c reader -- ls /data`
`kubectl exec`	`-- df -h /data`	Check disk usage	`kubectl exec share -- df -h /data`
`kubectl get pod`	`-o jsonpath='{.spec.volumes}'`	Extract volumes	
`kubectl explain pod.spec.volumes`	`--recursive`	Field reference	

🚨 Scenarios

Scenario 1 — emptyDir fills up → pod evicted

- **Cause**: no `sizeLimit`; app writes unbounded logs/cache.
- **Fix**: set `sizeLimit`; rotate/truncate within the app; monitor with `container_fs_usage_bytes`.

Scenario 2 — hostPath breaks on multi-node

- **Cause**: pod rescheduled to different node; data gone.
- **Fix**: replace hostPath with PVC + StorageClass for durable storage.

Scenario 3 — Secret volume writable by root only

- **Cause**: default mode is 0644; app runs as non-root UID.
- **Fix**: `defaultMode: 0400` or set `fsGroup` in pod securityContext to match app GID.

Scenario 4 — Config file not updating (subPath)

- **Cause**: `subPath` mounts bypass the symlink-based update mechanism.
- **Fix**: mount full directory (drop subPath) or use Reloader.

☁ GCP GKE Perspective

- **Autopilot**: `hostPath` blocked. `emptyDir` capped at pod ephemeral storage request.
- **GKE CSI ephemeral volumes**: use `csi` volume type with `pd.csi.storage.gke.io` for ephemeral local SSD.
- **Filestore CSI**: for NFS-like ReadWriteMany.

▩ AWS EKS Perspective

- **Fargate**: `hostPath` blocked. `emptyDir` limited to 20 GiB (pod storage). EFS CSI for shared.
- **Instance store (NVMe)**: access via `hostPath` on `i3` / `d3` instances for DaemonSet caches.
- **EFS CSI** replaces raw NFS mounts with K8s-native PVC flow.

🧪 Hands-On Lab Exercises

Lab 26.1 — emptyDir data sharing (Beginner, 20 min)

Steps: Apply Example 1. `kubectl logs share -c reader -f` — see timestamps from writer.
Cleanup: `kubectl delete pod share` .
Bonus: Delete the pod; recreate → data is gone (ephemeral).

Lab 26.2 — emptyDir sizeLimit eviction (Intermediate, 30 min)

Steps:

```
volumes: [{ name: tmp, emptyDir: { sizeLimit: 10Mi } }]
```

Container writes: `dd if=/dev/zero of=/tmp/data/big bs=1M count=20` . Pod gets evicted.
Cleanup: delete pod.

Lab 26.3 — Projected volume combining 3 sources (Advanced, 45 min)

Steps: Apply Example 4. Exec into pod → `ls /mnt/all-in-one` → see `config.yaml` , `db-password` , `token` . Decode SA token: `cat /mnt/all-in-one/token | jwt decode -` .
Cleanup: delete pod.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Kubernetes Volume is a directory available to containers in a pod. Unlike the container's writable layer, a volume's lifetime is at least the pod's lifetime — and for PVCs, it outlives the pod entirely. Types include emptyDir for scratch space, hostPath for node access, configMap/secret for config injection, projected for combining multiple sources, and PVC for durable storage.
Why: Containers are ephemeral — their filesystem dies on restart. Volumes solve three problems: sharing data between containers in a pod, surviving container restarts, and persisting data beyond the pod's life.
How: For inter-container sharing I use emptyDir with a sizeLimit. For config and secrets I mount configMap/secret volumes (or projected to combine them with an SA token). For persistent data I use PVC backed by a CSI StorageClass. I never use hostPath in production Deployments — only in DaemonSets accessing node-level resources.
Example: Our log sidecar pattern uses an emptyDir at `/var/log/app` — the app writes structured JSON logs, and Fluent Bit reads them from the same volume without any filesystem coupling."

? Possible Interview Questions

Basic

Q1. What is a Volume in K8s? A directory accessible to containers, with lifetime ≥ pod lifetime.
Q2. Difference between emptyDir and hostPath? emptyDir is pod-scoped scratch; hostPath maps a node directory into the pod.
Q3. When is emptyDir deleted? When the pod is removed from the node.

Intermediate

Q4. What is `medium: Memory` ? Mounts a tmpfs (RAM-backed) emptyDir; faster but counts against memory limit.
Q5. Why is hostPath dangerous? Data is node-local (not portable), and mounting sensitive paths gives host access.
Q6. What is a projected volume? Merges configMap + secret + downwardAPI + SA token into one mount.

Advanced

Q7. How does `sizeLimit` on emptyDir work? kubelet monitors usage; if exceeded, pod is evicted.
Q8. Why doesn't subPath update on ConfigMap change? subPath uses a bind mount to a specific file, not the symlink tree kubelet refreshes.
Q9. What `fsGroup` does for volumes? Sets the group ownership of volume files so non-root containers can read/write.

Scenario-based

Q10. Pod evicted for ephemeral storage — where do you look? `kubectl describe pod` → `Evicted` reason; `kubectl describe node` → ephemeral storage pressure; check emptyDir sizeLimit.

GKE-specific

Q11. Why does Autopilot reject hostPath? Security — hostPath can expose node filesystem. Use PVC or emptyDir.

EKS-specific

Q12. What's the emptyDir size limit on Fargate? ~20 GiB pod ephemeral storage.

Hands-on

Q13. How do you share data between two containers? emptyDir mounted in both at different or same paths.
Q14. Show me a projected volume. See Example 4 — configMap + secret + SA token in one mount.
Q15. When would you use NFS volume type vs PVC? Legacy in-line NFS for quick mounts; PVC + NFS CSI / Filestore CSI / EFS CSI for managed lifecycle.

🗄 Topic 27 — Persistent Volumes (PV)

Official Documentation Reference - Persistent Volumes: https://kubernetes.io/docs/concepts/storage/persistent-volumes/ - GKE persistent disks: https://cloud.google.com/kubernetes-engine/docs/concepts/persistent-volumes - EKS EBS: https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html

🔹 Definition

A **PersistentVolume (PV)** is a cluster-scoped piece of storage provisioned by an admin or dynamically by a StorageClass. It abstracts the underlying storage — a cloud disk, NFS share, or local SSD — from the pod consuming it.

🔹 About

PV is the "supply" side of storage

Concept	Analogy
PV	A pre-provisioned disk sitting in a rack
PVC	A purchase order requesting "I need 50 Gi RWO SSD"
StorageClass	The catalog of available disk types

Key PV fields

Field	Purpose
`capacity.storage`	Size (e.g., `50Gi`)
`accessModes`	`ReadWriteOnce (RWO)`, `ReadOnlyMany (ROX)`, `ReadWriteMany (RWX)`, `ReadWriteOncePod (RWOP)`
`persistentVolumeReclaimPolicy`	`Retain`, `Delete`, `Recycle` (deprecated)
`storageClassName`	Link to a StorageClass (empty string = no class)
`volumeMode`	`Filesystem` (default) or `Block`
`nodeAffinity`	Pin PV to specific node/zone (for local/zonal disks)
`csi` / `nfs` / `hostPath`	Backend-specific config

Access modes explained

Mode	Short	Meaning
ReadWriteOnce	RWO	One node can mount read-write
ReadOnlyMany	ROX	Many nodes mount read-only
ReadWriteMany	RWX	Many nodes mount read-write (NFS/EFS/Filestore)
ReadWriteOncePod	RWOP	Only one pod can mount read-write (K8s 1.29+ stable)

Reclaim policies

Policy	What happens when PVC is deleted
`Retain`	PV kept with data; admin manually reclaims
`Delete`	PV and underlying storage deleted (cloud disk removed)
`Recycle`	Deprecated — `rm -rf` on volume

PV phases

`Available` → `Bound` (to a PVC) → `Released` (PVC deleted, data retained) → `Failed`.

Static vs Dynamic provisioning

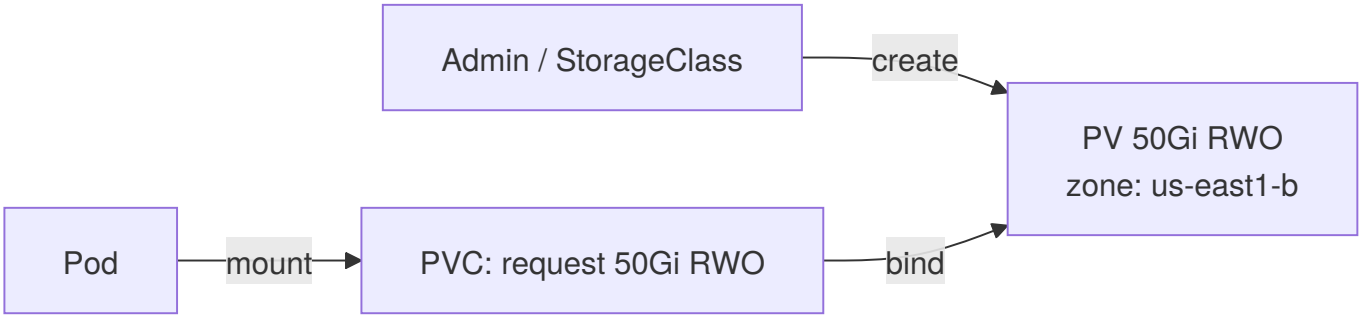
- **Static**: admin pre-creates PV objects; PVC binds to a matching one.
- **Dynamic**: PVC references a StorageClass; the provisioner automatically creates a PV + disk.

🔌 How It Integrates With Other Kubernetes Components

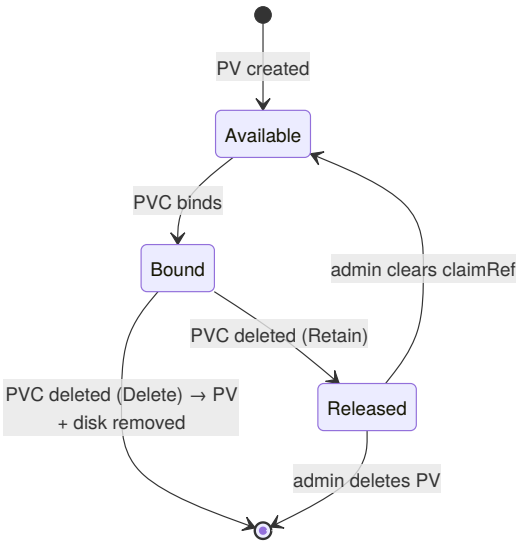
Component	Interaction
PVC	Claims a PV based on size, access mode, class
StorageClass	Template for dynamic PV creation
CSI driver	Provisions/attaches/mounts the actual disk
Scheduler	Topology-aware: places pod near the PV's zone
StatefulSet	`volumeClaimTemplates` → one PVC/PV per pod
Node	PV attachment is per-node (RWO) or cross-node (RWX)

🏗 Diagrams

PV ↔ PVC binding



PV lifecycle



💻 Real-World Examples

Example 1 — Static PV (NFS)

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: nfs-data }
spec:
  capacity: { storage: 100Gi }
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""              # no class = static
  nfs:
    server: 10.0.0.50
    path: /exports/data
```

Example 2 — Static PV (GCE PD)

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: gce-disk }
spec:
  capacity: { storage: 50Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: standard-rwo
  csi:
    driver: pd.csi.storage.gke.io
    volumeHandle: projects/myproj/zones/us-east1-b/disks/my-disk
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: topology.gke.io/zone, operator: In, values: [us-east1-b] }
```

Example 3 — Static PV (EBS)

```
apiVersion: v1
kind: PersistentVolume
metadata: { name: ebs-disk }
spec:
  capacity: { storage: 50Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-0abcdef1234567890
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: topology.ebs.csi.aws.com/zone, operator: In, values: [ap-south-1a] }
```

Example 4 — Change reclaim policy on existing PV

```
kubectl patch pv gce-disk -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pv`	`-o wide`	List PVs + status + claim	`kubectl get pv -o wide`
`kubectl describe pv`	`<name>`	See binding, events	`kubectl describe pv gce-disk`
`kubectl patch pv`	`-p`	Change reclaim policy	see Example 4
`kubectl delete pv`	`<name>`	Remove PV (data untouched if Retain)	`kubectl delete pv nfs-data`
`kubectl get pv -o jsonpath`	`'{.items[*].spec.claimRef}'`	Find which PVC bound	

🚨 Scenarios

Scenario 1 — PV stuck in `Released`, not `Available`

- **Cause**: after PVC deletion with Retain policy, PV keeps `claimRef`.
- **Fix**: `kubectl patch pv <name> -p '{"spec":{"claimRef":null}}'` — PV becomes Available.

Scenario 2 — PVC Pending — no matching PV

- **Cause**: PVC requests 100Gi but only 50Gi PV exists; or access mode mismatch; or class mismatch.
- **Fix**: create matching PV or use dynamic provisioning.

Scenario 3 — Data loss after PVC delete

- **Cause**: reclaim policy was `Delete` (default for many dynamic StorageClasses).
- **Fix**: always `Retain` for production data. Take snapshots before deletes.

☁ GCP GKE Perspective

- Static PVs referencing GCE PDs use `pd.csi.storage.gke.io` driver.
- **Regional PD**: replicated across two zones; excellent for HA StatefulSets.
- Default reclaim for `standard-rwo` SC is `Delete` — change to `Retain` for prod.

▩ AWS EKS Perspective

- Static PVs for EBS use `ebs.csi.aws.com` driver + `volumeHandle`.
- EBS is zonal; PV + pod must be in same AZ.
- **EFS** PVs are `ReadWriteMany` + cross-AZ.
- Ensure EBS CSI driver add-on is installed.

🧪 Hands-On Lab Exercises

Lab 27.1 — Static PV + PVC binding (Beginner, 30 min)

Steps

```
# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata: { name: lab-pv }
spec:
  capacity: { storage: 1Gi }
  accessModes: [ReadWriteOnce]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath: { path: /mnt/data }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: lab-pvc }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 1Gi } }
  storageClassName: manual
```

```
kubectl apply -f pv.yaml
kubectl get pv,pvc     # Bound
```

Cleanup: `kubectl delete pvc lab-pvc && kubectl delete pv lab-pv`.

Lab 27.2 — Reclaim policy behavior (Intermediate, 30 min)

1. Create PV with `Delete` policy + PVC.
2. Delete PVC → PV disappears (and disk if cloud).
3. Recreate with `Retain` → delete PVC → PV remains in `Released`.
4. Clear claimRef → PV becomes `Available`.

Lab 27.3 — Static GCE PD PV on GKE (Advanced, 45 min)

1. `gcloud compute disks create lab-disk --size=10Gi --zone=asia-south1-a`.
2. Create PV referencing `pd.csi.storage.gke.io` + volumeHandle.
3. Create PVC + pod. Write data. Delete pod. Recreate → data persists.
4. Delete PVC → PV Released (Retain). Patch claimRef → Available.

Cleanup: `gcloud compute disks delete lab-disk`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A PersistentVolume is a cluster-scoped storage resource — the 'supply' side. It represents a physical disk, NFS share, or cloud volume. A PVC is the 'demand' side — a pod's request for storage. The PV controller binds them by matching size, access mode, and StorageClass.
Why: Pods are ephemeral; PVs decouple storage lifecycle from pod lifecycle. Data survives pod restarts, rescheduling, and even cluster upgrades.
How: In production I almost always use dynamic provisioning via StorageClass — the CSI driver creates the PV and disk automatically when a PVC is applied. I set reclaimPolicy to Retain for databases so a PVC delete doesn't wipe data, and I take VolumeSnapshots before any destructive operation.
Example: We had an incident where a junior engineer deleted a namespace. The StatefulSet PVCs were deleted and, because the StorageClass was `Delete`, the EBS volumes vanished. After that we patched every StorageClass to `Retain` and added a Kyverno policy blocking PVC deletion without a `approved-deletion=true` label."

? Possible Interview Questions

Basic

Q1. PV vs PVC? PV = storage resource (supply). PVC = storage request (demand).
Q2. Is PV namespaced? No — cluster-scoped.
Q3. What are access modes? RWO, ROX, RWX, RWOP.

Intermediate

Q4. Reclaim policies? Retain (keep data), Delete (remove disk), Recycle (deprecated).
Q5. Static vs dynamic provisioning? Static: admin creates PV. Dynamic: PVC triggers StorageClass provisioner to create PV+disk.

Q6. What is `volumeMode: Block`? Exposes the volume as a raw block device instead of a filesystem.

Advanced

Q7. How do you make a Released PV Available again? Patch `spec.claimRef` to null.
Q8. What is `ReadWriteOncePod` (RWOP)? Only one pod — not just one node — can mount RW. Prevents multi-attach bugs (K8s 1.29+ stable).
Q9. How does topology affect PV binding? `WaitForFirstConsumer` delays binding until a pod is scheduled, ensuring PV is created in the right zone.

Scenario-based

Q10. PVC stuck Pending — steps? Check PV exists with matching size/access/class. Check StorageClass provisioner. Check events: `kubectl describe pvc`.

GKE-specific

Q11. What is a regional PD? A persistent disk replicated across two zones — survives a single-zone outage.

EKS-specific

Q12. EBS volume stuck attaching after node crash — fix? Force-detach via `aws ec2 detach-volume --force` or wait for node cleanup timeout.

Hands-on

Q13. How do you verify PV data survives pod deletion? Write file → delete pod → recreate pod with same PVC → read file.
Q14. Show me how to change reclaim policy. `kubectl patch pv <n> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'`.
Q15. When would you use static PVs? Pre-existing disks, NFS exports, or environments without CSI dynamic provisioning.

🗐 Topic 28 — Persistent Volume Claims (PVC)

Official Documentation Reference - PVC: https://kubernetes.io/docs/concepts/storage/persistent-volumes/#persistentvolumeclaims - Configure PV storage tutorial: https://kubernetes.io/docs/tutorials/configuration/configure-persistent-volume-storage/ - GKE PVC: https://cloud.google.com/kubernetes-engine/docs/concepts/persistent-volumes - EKS EBS PVC: https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html

🔹 Definition

A **PersistentVolumeClaim (PVC)** is a namespaced request for storage. It specifies how much, what access mode, and which StorageClass — then the PV controller binds it to a matching PersistentVolume (static or dynamically provisioned).

🔹 About

PVC spec fields

Field	Purpose
`accessModes`	RWO / ROX / RWX / RWOP
`resources.requests.storage`	Minimum size (e.g., `50Gi`)
`storageClassName`	Which StorageClass to use (empty = default)
`volumeMode`	`Filesystem` or `Block`
`selector`	Label selector for static PV matching
`dataSource` / `dataSourceRef`	Clone from VolumeSnapshot or PVC

PVC phases

`Pending` → `Bound` → (deleted → PV goes to Released/Deleted based on reclaim policy).

PVC in a Pod

```
spec:
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: my-pvc
  containers:
  - volumeMounts:
    - { name: data, mountPath: /data }
```

PVC expansion (online resize)

If StorageClass has `allowVolumeExpansion: true`:

```
kubectl patch pvc my-pvc -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'
```

CSI driver expands the underlying volume; filesystem expansion happens automatically (ext4/xfs).

PVC cloning & snapshots

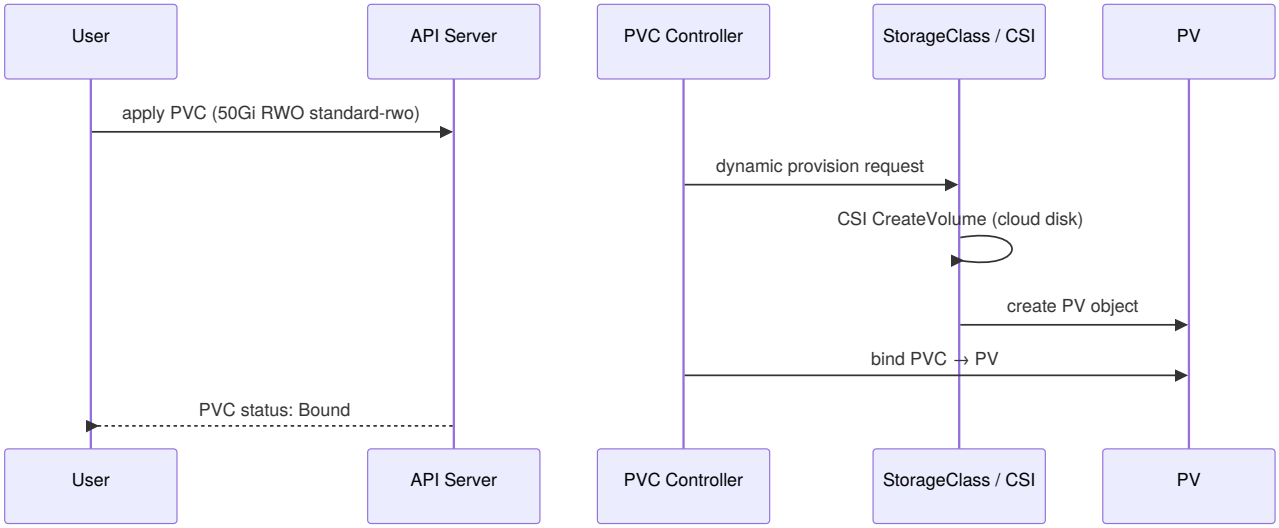
- **Clone**: `dataSource: { kind: PersistentVolumeClaim, name: source-pvc }` — creates a new PV from an existing PVC.
- **Snapshot restore**: `dataSource: { kind: VolumeSnapshot, name: snap-1 }` — provisions PV from a snapshot.

🔗 How It Integrates With Other Kubernetes Components

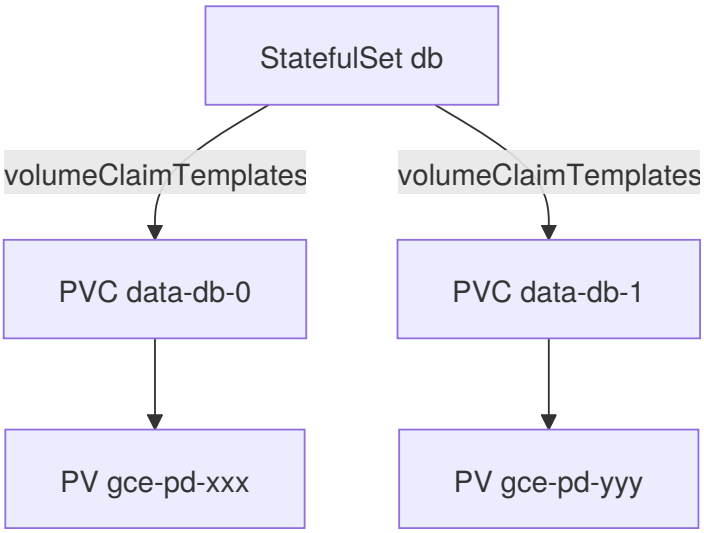
Component	Interaction
PV	PVC binds to a matching PV
StorageClass	Triggers dynamic provisioning
Pod	References PVC by `claimName`
StatefulSet	`volumeClaimTemplates` auto-creates PVCs
VolumeSnapshot	PVC can be source or restored from snapshot
ResourceQuota	Caps total PVC count / storage per namespace

🏗 Diagrams

PVC binding flow



PVC in StatefulSet



💻 Real-World Examples

Example 1 — Basic PVC with dynamic provisioning

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: app-data }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 20Gi } }
  storageClassName: standard-rwo
```

Example 2 — Pod using PVC

```
apiVersion: v1
kind: Pod
metadata: { name: writer }
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","echo persist > /data/test.txt; sleep 3600"]
    volumeMounts: [{ name: data, mountPath: /data }]
  volumes:
  - name: data
    persistentVolumeClaim: { claimName: app-data }
```

Example 3 — Expand PVC online

```
kubectl patch pvc app-data -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'
kubectl get pvc app-data    # CAPACITY updates after resize
```

Example 4 — Clone PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: app-data-clone }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 20Gi } }
  storageClassName: standard-rwo
  dataSource:
    kind: PersistentVolumeClaim
    name: app-data
```

Example 5 — Restore from VolumeSnapshot

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: app-data-restored }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 20Gi } }
  storageClassName: standard-rwo
  dataSource:
    kind: VolumeSnapshot
    name: snap-app-data-2024
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pvc`	`-o wide`	List PVCs + bound PV	`kubectl get pvc -o wide`
`kubectl describe pvc`	`<name>`	Events, capacity	`kubectl describe pvc app-data`
`kubectl delete pvc`	`<name>`	Delete claim (triggers reclaim)	`kubectl delete pvc app-data`
`kubectl patch pvc`	`-p`	Expand	see Example 3
`kubectl get volumesnapshot`	—	List snapshots	`kubectl get volumesnapshot`

🚨 Scenarios

Scenario 1 — PVC stuck Pending

- **Cause**: no matching PV (static); StorageClass provisioner failing (dynamic); `WaitForFirstConsumer` waiting for a pod.
- **Fix**: `kubectl describe pvc` → events. Check SC provisioner, CSI driver DaemonSet, quota.

Scenario 2 — Pod stuck ContainerCreating — "unable to attach volume"

- **Cause**: EBS/PD still attached to old node after crash. Multi-attach not allowed for RWO.
- **Fix**: wait for kubelet cleanup or force-detach cloud volume; delete stale VolumeAttachment object.

Scenario 3 — PVC expansion fails

- **Cause**: StorageClass doesn't have `allowVolumeExpansion: true`.
- **Fix**: patch SC, or create new SC with expansion and migrate.

☁ GCP GKE Perspective

- Default StorageClass: `standard-rwo` (PD balanced), `premium-rwo` (PD SSD).
- `allowVolumeExpansion: true` enabled on defaults.
- **Autopilot**: PVC works; ephemeral storage has separate billing.

▩ AWS EKS Perspective

- Ensure EBS CSI add-on installed.
- Default SC: `gp2` (legacy) or `gp3` (recommended).
- EFS CSI: `ReadWriteMany` PVC across AZs.
- **Fargate**: only EFS PVC supported (no EBS).

🧪 Hands-On Lab Exercises

Lab 28.1 — PVC lifecycle (Beginner, 30 min)

Apply Example 1+2. Write data, delete pod, recreate, verify data persists.

Lab 28.2 — Online PVC expansion (Intermediate, 30 min)

Create 5Gi PVC, fill 4Gi, expand to 20Gi online, verify with `df -h`.

Lab 28.3 — Clone + snapshot restore (Advanced, 60 min)

1. Create PVC, write data.
2. Create VolumeSnapshot.
3. Create clone PVC from snapshot (Example 5).
4. Mount clone in new pod; verify data matches.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A PVC is a namespaced request for storage — size, access mode, and StorageClass. The PV controller binds it to a matching PersistentVolume, either pre-created or dynamically provisioned.
Why: PVCs decouple pod specs from infrastructure details. A developer asks for '50Gi RWO SSD' without knowing whether it's GCE PD, EBS, or NFS — the StorageClass handles the rest.
How: I set every production PVC via a StorageClass with `reclaimPolicy: Retain` and `allowVolumeExpansion: true`. For StatefulSets I use `volumeClaimTemplates` so each pod gets its own PVC. I take VolumeSnapshots before destructive operations and automate backup with Velero or Backup for GKE.
Example: When our Postgres database needed more space, we expanded the PVC from 50Gi to 200Gi with a single `kubectl patch` — zero downtime, the filesystem resized automatically."

? Possible Interview Questions

Q1. What is a PVC? Namespaced storage request that binds to a PV.
Q2. How does PVC bind to PV? By matching size, accessModes, storageClassName, and optionally label selector.
Q3. Can you expand a PVC? Yes, if SC has `allowVolumeExpansion: true`.
Q4. What happens to the PV when PVC is deleted? Depends on reclaimPolicy: Delete removes it, Retain keeps it.
Q5. PVC Pending — troubleshooting? `kubectl describe pvc` → events → check SC provisioner, CSI driver, node zones, quota.
Q6. How does StatefulSet create PVCs? `volumeClaimTemplates` in the STS spec auto-generates one PVC per pod.
Q7. What is a VolumeSnapshot? A point-in-time copy of a PVC's data; can be used to restore a new PVC.
Q8. PVC clone vs snapshot? Clone duplicates a live PVC. Snapshot is a frozen copy you restore from.
Q9. RWO vs RWOP? RWO: one node. RWOP: one pod (stricter).
Q10. Why is `WaitForFirstConsumer` important? Delays PV creation until pod is scheduled, ensuring volume and pod land in the same zone.

🗄 Topic 29 — Storage Classes and Dynamic Provisioning

Official Documentation Reference - StorageClass: https://kubernetes.io/docs/concepts/storage/storage-classes/ - Dynamic provisioning: https://kubernetes.io/docs/concepts/storage/dynamic-provisioning/ - Change default SC: https://kubernetes.io/docs/tasks/administer-cluster/change-default-storage-class/ - GKE SC: https://cloud.google.com/kubernetes-engine/docs/concepts/persistent-volumes#storageclasses - EKS SC: https://docs.aws.amazon.com/eks/latest/userguide/storage-classes.html

---

🔹 Definition

A **StorageClass** is a cluster-scoped object that defines a "class" of storage — provisioner, parameters, reclaim policy, and volume binding mode. When a PVC references a StorageClass, the provisioner dynamically creates a PV and the underlying disk.

🔹 About

Key fields

Field	Purpose
`provisioner`	Which CSI driver creates volumes (e.g., `pd.csi.storage.gke.io`, `ebs.csi.aws.com`)
`parameters`	Driver-specific (disk type, IOPS, throughput, encryption key)
`reclaimPolicy`	`Delete` (default) or `Retain`
`volumeBindingMode`	`Immediate` or `WaitForFirstConsumer`
`allowVolumeExpansion`	`true` / `false`
`mountOptions`	Filesystem mount flags

volumeBindingMode

Mode	When PV/disk is created	Use case
`Immediate`	As soon as PVC is created	Single-zone clusters, NFS
`WaitForFirstConsumer`	When a pod using the PVC is scheduled	Multi-zone clusters (ensures disk + pod co-locate)

Default StorageClass

Annotated with `storageclass.kubernetes.io/is-default-class: "true"`. PVCs with no `storageClassName` use it. Only one default at a time is recommended.

Common GKE StorageClasses

Name	Provisioner	Disk type
`standard-rwo` (default)	`pd.csi.storage.gke.io`	PD balanced
`premium-rwo`	`pd.csi.storage.gke.io`	PD SSD
`standard-rwx`	`filestore.csi.storage.gke.io`	Filestore basic

Common EKS StorageClasses

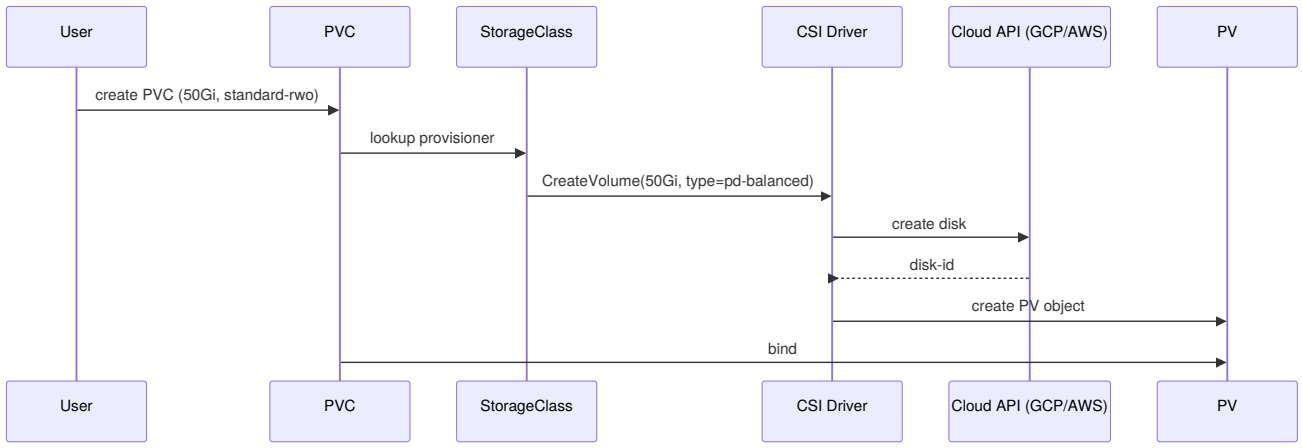
Name	Provisioner	Disk type
`gp2` (legacy default)	`kubernetes.io/aws-ebs` (in-tree)	gp2
`gp3` (recommended)	`ebs.csi.aws.com`	gp3 (create manually)
`efs-sc`	`efs.csi.aws.com`	EFS (RWX)

🔷 How It Integrates With Other Kubernetes Components

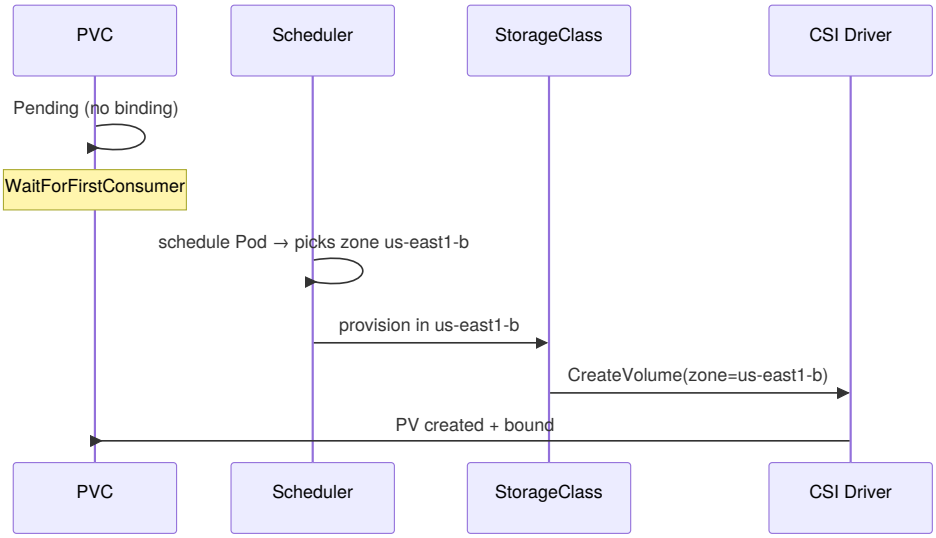
Component	Interaction
PVC	References `storageClassName` → triggers provisioning
PV	Created dynamically by SC provisioner
CSI driver	The actual backend that creates/deletes/expands volumes
Scheduler	`WaitForFirstConsumer` defers binding until scheduling
VolumeSnapshot	SC parameters may affect snapshot support

🏗 Diagrams

Dynamic provisioning flow



WaitForFirstConsumer



💻 Real-World Examples

Example 1 — Custom gp3 StorageClass on EKS

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  iops: "3000"
  throughput: "125"
  encrypted: "true"
  kmsKeyId: arn:aws:kms:ap-south-1:123:key/abc
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Example 2 — GKE PD SSD with regional replication

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: regional-ssd }
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
  replication-type: regional-pd
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

Example 3 — EFS StorageClass (ReadWriteMany)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: efs }
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  fileSystemId: fs-0abc123
  directoryPerms: "700"
```

Example 4 — Change default StorageClass

```
# Remove default from old
kubectl patch sc gp2 -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"false"}}}'
# Set new default
kubectl patch sc gp3 -p '{"metadata":{"annotations":{"storageclass.kubernetes.io/is-default-class":"true"}}}'
kubectl get sc    # (default) marker moves
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get sc`	—	List StorageClasses	`kubectl get sc`
`kubectl describe sc`	`<name>`	Provisioner + params	`kubectl describe sc gp3`
`kubectl patch sc`	`-p`	Change default / reclaim	see Example 4
`kubectl delete sc`	`<name>`	Remove class	`kubectl delete sc old-class`

🚨 Scenarios

Scenario 1 — PVC uses wrong SC → expensive disk

- **Cause**: no SC specified; default SC is `premium-rwo` (SSD).
- **Fix**: set `storageClassName` explicitly on PVC; review default SC.

Scenario 2 — Volume provisioned in wrong zone

- **Cause**: `volumeBindingMode: Immediate`.
- **Fix**: switch to `WaitForFirstConsumer`.

Scenario 3 — Two defaults — unpredictable binding

- **Cause**: both `gp2` and `gp3` annotated as default.
- **Fix**: only one default SC at a time.

☁ GCP GKE Perspective

- Autopilot default: `standard-rwo` (PD balanced). Premium: `premium-rwo`.
- Regional PD: `replication-type: regional-pd` for cross-zone HA.
- Filestore: `filestore.csi.storage.gke.io` for RWX.

▩ AWS EKS Perspective

- EBS CSI: create `gp3` SC manually (gp2 is legacy in-tree default).
- EFS CSI: `efs-ap` provisioning mode creates access points per PVC.
- Fargate: only EFS SC works.

🧪 Hands-On Lab Exercises

Lab 29.1 — Create custom SC + PVC (Beginner, 30 min)

Create hostPath SC on Minikube. Apply PVC. Verify binding + data persistence.

Lab 29.2 — WaitForFirstConsumer on Kind (Intermediate, 45 min)

Create SC with `WaitForFirstConsumer`. Apply PVC → stays Pending. Apply pod → PVC binds. Verify correct node via `kubectl describe pv`.

Lab 29.3 — Encrypted gp3 on EKS (Advanced, 60 min)

1. Create KMS key. 2. Create gp3 SC (Example 1). 3. PVC + pod. 4. `aws ec2 describe-volumes` → verify encryption + type.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A StorageClass is the template for dynamic volume provisioning. It defines which CSI provisioner to use, what disk type and parameters, the reclaim policy, and when to bind — Immediate or WaitForFirstConsumer.
Why: Without StorageClass, admins must pre-create every PV. Dynamic provisioning scales — a developer writes a PVC, the SC does the rest. `WaitForFirstConsumer` ensures the disk lands in the same zone as the pod, critical for multi-AZ clusters.
How: I create tailored StorageClasses: gp3 encrypted for EKS databases, pd-ssd regional for GKE HA, efs-ap for shared workloads. I always set `reclaimPolicy: Retain` and `allowVolumeExpansion: true` for production, and mark exactly one SC as the default.
Example: After switching the EKS default from gp2 to gp3, our database IOPS tripled (3000 base) while storage cost dropped 20% — gp3 is cheaper per GB than gp2."

? Possible Interview Questions

Q1. What does a StorageClass do? Defines how dynamic PVs are provisioned — provisioner, parameters, reclaim, binding mode.
Q2. Immediate vs WaitForFirstConsumer? Immediate creates disk on PVC create. WaitForFirstConsumer waits for pod scheduling to pick the zone.
Q3. How do you set the default SC? Annotation `storageclass.kubernetes.io/is-default-class: "true"`.
Q4. Can you change a SC's reclaim policy after PVs exist? SC change only affects new PVs. Existing PVs must be patched individually.
Q5. How does EFS SC provision per-PVC storage? `provisioningMode: efs-ap` creates an EFS access point per PVC — separate directory + auth.
Q6. What is `allowVolumeExpansion`? Enables online PVC resize via `kubectl patch pvc`.
Q7. What is a regional PD on GKE? PD replicated synchronously across two zones — survives single-zone failure.
Q8. How many defaults should there be? Exactly one (or zero). Multiple defaults → undefined behavior.

Q9. When would you use static PV over SC? Pre-existing disks, external NFS, or environments without CSI.
Q10. How do you encrypt EBS volumes via SC? `parameters: { encrypted: "true", kmsKeyId: arn:aws:kms:... }`.

🗐 Topic 30 — CSI Drivers (GCE PD, EBS, EFS, Filestore)

Official Documentation Reference - CSI overview: https://kubernetes.io/docs/concepts/storage/volumes/#csi - CSI driver list: https://kubernetes-csi.github.io/docs/drivers.html - Volume snapshots: https://kubernetes.io/docs/concepts/storage/volume-snapshots/ - GCE PD CSI: https://cloud.google.com/kubernetes-engine/docs/how-to/persistent-volumes/gce-pd-csi-driver - EBS CSI: https://docs.aws.amazon.com/eks/latest/userguide/ebs-csi.html - EFS CSI: https://docs.aws.amazon.com/eks/latest/userguide/efs-csi.html - Filestore CSI: https://cloud.google.com/kubernetes-engine/docs/how-to/persistent-volumes/filestore-csi-driver

🔹 Definition

CSI (Container Storage Interface) is the standard that decouples Kubernetes from storage backends. A **CSI driver** is a plugin that implements volume operations — create, delete, attach, detach, mount, unmount, snapshot, expand — for a specific storage system.

🔹 About

CSI architecture

Component	Where it runs	Role
Controller plugin	Deployment (1–3 replicas)	CreateVolume, DeleteVolume, Snapshot
Node plugin	DaemonSet (every node)	NodeStageVolume, NodePublishVolume (mount)
Sidecar containers	With controller	external-provisioner, external-attacher, external-snapshotter, external-resizer

Key CSI drivers for GKE + EKS

Driver	Provisioner string	Storage	Access	Platform
GCE PD	`pd.csi.storage.gke.io`	Block (PD balanced/SSD/extreme)	RWO, ROX	GKE
Filestore	`filestore.csi.storage.gke.io`	File (NFS)	RWX	GKE
EBS	`ebs.csi.aws.com`	Block (gp2/gp3/io1/io2)	RWO	EKS
EFS	`efs.csi.aws.com`	File (NFS)	RWX	EKS
FSx for Lustre	`fsx.csi.aws.com`	File (Lustre)	RWX	EKS

VolumeSnapshots

CSI drivers that support snapshots implement `CreateSnapshot` / `DeleteSnapshot`. K8s objects: - **VolumeSnapshotClass** — defines which CSI driver handles snapshots. - **VolumeSnapshot** — user request: "snapshot PVC X now". - **VolumeSnapshotContent** — the actual snapshot resource (cloud snapshot ID).

Migration from in-tree to CSI

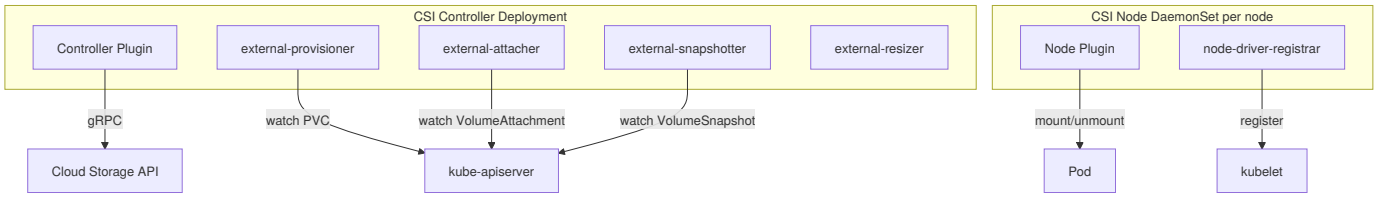
Kubernetes deprecated in-tree volume plugins (e.g., `kubernetes.io/aws-ebs`, `kubernetes.io/gce-pd`). CSI migration translates in-tree calls to CSI calls transparently. Fully migrated by K8s 1.29+.

🔹 How It Integrates With Other Kubernetes Components

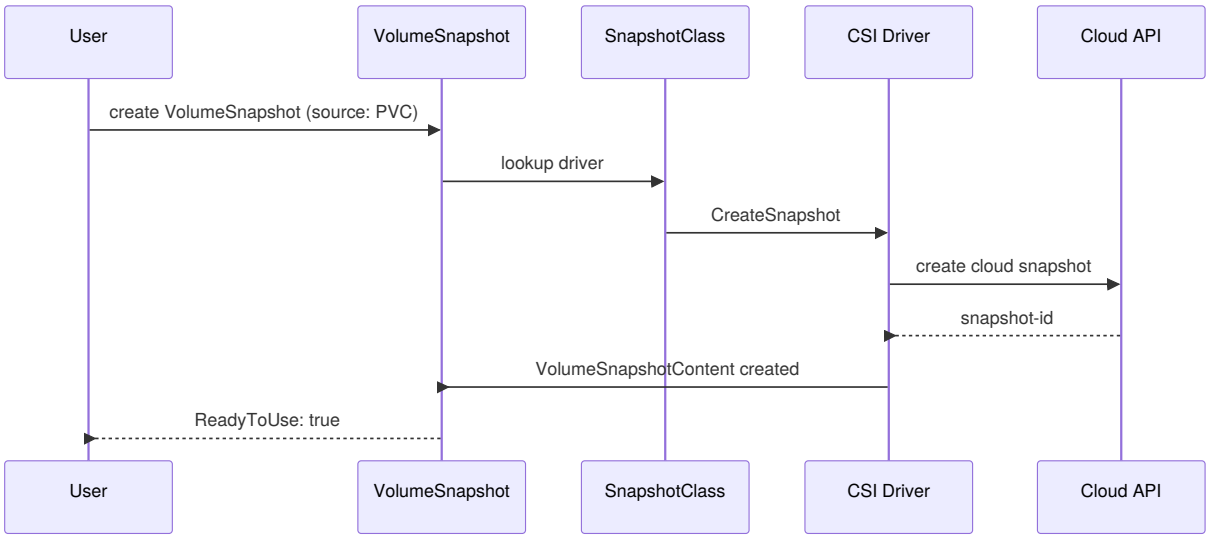
Component	Interaction
StorageClass	`provisioner` field points to CSI driver
PVC	Triggers `CreateVolume` via external-provisioner sidecar
kubelet	Calls `NodeStageVolume` + `NodePublishVolume` to mount
Scheduler	Topology keys from CSI topology → zone-aware scheduling
VolumeSnapshot controller	Coordinates snapshot lifecycle

🏗 Diagrams

CSI driver architecture



Snapshot flow



💻 Real-World Examples

Example 1 — Install EBS CSI on EKS

```
eksctl create iamserviceaccount \
  --cluster=prod --name=ebs-csi-controller-sa --namespace=kube-system \
  --attach-policy-arn=arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve
aws eks create-addon --cluster-name=prod --addon-name=aws-ebs-csi-driver \
  --service-account-role-arn=arn:aws:iam::<acct>:role/AmazonEKS_EBS_CSI_DriverRole
```

Example 2 — VolumeSnapshot on GKE

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata: { name: pd-snap }
driver: pd.csi.storage.gke.io
deletionPolicy: Retain
---
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata: { name: db-snap }
spec:
  volumeSnapshotClassName: pd-snap
  source: { persistentVolumeClaimName: data-db-0 }
```

Example 3 — Restore PVC from snapshot

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: db-restored }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 50Gi } }
  storageClassName: standard-rwo
  dataSource:
    kind: VolumeSnapshot
    name: db-snap
    apiGroup: snapshot.storage.k8s.io
```

Example 4 — EFS PVC (ReadWriteMany)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: shared-data }
spec:
  accessModes: [ReadWriteMany]
  resources: { requests: { storage: 5Gi } }
  storageClassName: efs
```

Example 5 — GKE Filestore PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: shared-files }
spec:
  accessModes: [ReadWriteMany]
  resources: { requests: { storage: 1Ti } }   # Filestore min 1Ti basic
  storageClassName: standard-rwx
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get csidrivers`	—	Installed drivers	`kubectl get csidrivers`
`kubectl get csinodes`	—	Per-node CSI info	`kubectl get csinodes`
`kubectl get volumesnapshot`	—	Snapshots	`kubectl get volumesnapshot`
`kubectl get volumesnapshotcontent`	—	Cloud snapshot details	`kubectl get vsc`
`kubectl get volumesnapshotclass`	—	Snapshot classes	`kubectl get vsclass`
`kubectl describe volumesnapshot`	`<name>`	Ready status	`kubectl describe vs db-snap`
`aws eks describe-addon`	`--addon-name aws-ebs-csi-driver`	EBS CSI status	
`gcloud container clusters describe`	`--format='value(addonsConfig)'`	GKE driver status	

🚨 Scenarios

Scenario 1 — PVC Pending: "no CSI driver"

- **Cause**: CSI driver not installed or crashed.
- **Fix**: `kubectl get csidrivers` + check DaemonSet/Deployment in `kube-system`.

Scenario 2 — Snapshot stuck ReadyToUse: false

- **Cause**: snapshot-controller not installed; or cloud API quota hit.
- **Fix**: install snapshot CRDs + controller; check cloud quota.

Scenario 3 — Volume attach timeout

- **Cause**: previous node still holding attachment (RWO). Max-attach-limit reached.
- **Fix**: force detach old node; or wait. Check `kubectl get volumeattachment`.

Scenario 4 — Filestore PVC needs 1 TiB minimum

- **Cause**: Filestore basic HDD minimum is 1 TiB.
- **Fix**: use Filestore Enterprise (256 GiB min) or Cloud Volumes Service.

☁ GCP GKE Perspective

- **PD CSI**: enabled by default on GKE Standard and Autopilot.
- **Filestore CSI**: enable via `--addons=GcpFilestoreCsiDriver`.
- **Snapshot support**: install VolumeSnapshot CRDs + controller (GKE does this automatically on 1.24+).
- **Backup for GKE**: layer on top of snapshots for full application backup.

▩ AWS EKS Perspective

- **EBS CSI**: install as EKS add-on + IRSA. Required for gp3/dynamic provisioning.
- **EFS CSI**: separate add-on; needs security group allowing NFS (2049).
- **Snapshot controller**: not installed by default on EKS; install separately.
- **FSx CSI**: for Lustre HPC workloads.

🧪 Hands-On Lab Exercises

Lab 30.1 — Install EBS CSI + gp3 SC (Beginner, 45 min)

Install driver, create gp3 SC, PVC, pod. Write data. `aws ec2 describe-volumes` confirms gp3.

Lab 30.2 — VolumeSnapshot + restore on GKE (Intermediate, 60 min)

Create PVC, write data, create VolumeSnapshot (Example 2), restore to new PVC (Example 3), verify data.

Lab 30.3 — EFS RWX across pods on EKS (Advanced, 60 min)

1. Create EFS file system + SG. 2. Install EFS CSI. 3. Create SC + PVC (RWX). 4. Deploy 3 pods mounting same PVC. 5. Write in one, read from another.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: CSI is the standard interface between Kubernetes and storage providers. A CSI driver has a controller component for provisioning/snapshot and a node component for mounting. On GKE the PD and Filestore CSI drivers are built-in; on EKS you install EBS and EFS CSI as add-ons.
Why: CSI replaced the monolithic in-tree volume plugins, letting storage vendors ship drivers independently of K8s releases and enabling features like volume snapshots, online expansion, and topology-aware provisioning.
How: On EKS I install EBS CSI with IRSA for least-privilege, create a gp3 StorageClass with encryption, and take VolumeSnapshots for point-in-time recovery. On GKE PD CSI is already enabled; I add Filestore CSI for RWX workloads and use Backup for GKE for application-level backups.
Example: We needed a shared filesystem for a 10-pod ML training job. On EKS, I provisioned an EFS PVC with ReadWriteMany — all pods mounted the same dataset. On GKE the equivalent was a Filestore PVC. Zero code changes across clouds because the CSI abstraction handled the backend."

? Possible Interview Questions

Q1. What is CSI? Container Storage Interface — standard for K8s storage plugins.
Q2. Controller vs node plugin? Controller: provision/delete/snapshot (Deployment). Node: mount/unmount (DaemonSet).
Q3. How do you install EBS CSI on EKS? `aws eks create-addon --addon-name aws-ebs-csi-driver` + IRSA role.
Q4. What is a VolumeSnapshot? Point-in-time copy of PVC data via CSI snapshot.
Q5. EFS vs EBS? EBS: block, RWO, zonal. EFS: file (NFS), RWX, multi-AZ.
Q6. Filestore vs PD on GKE? PD: block, RWO. Filestore: NFS, RWX, min 1 TiB basic.
Q7. How does CSI migration work? In-tree plugin calls are transparently translated to CSI calls. Fully migrated by K8s 1.29+.
Q8. What sidecar containers run with the CSI controller? external-provisioner, external-attacher, external-snapshotter, external-resizer.
Q9. How do you encrypt EBS at rest? SC parameter `encrypted: "true"` + optional `kmsKeyId`.
Q10. Can Fargate use EBS? No — only EFS (no block device attachment on Firecracker microVMs).

🗄 Topic 31 — StatefulSet Storage Patterns

Official Documentation Reference - StatefulSet: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/ - volumeClaimTemplates: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/#stable-storage - persistentVolumeClaimRetentionPolicy: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/#persistentvolumeclaim-retention - Run replicated stateful app: https://kubernetes.io/docs/tasks/run-application/run-replicated-stateful-application/ - GKE stateful apps: https://cloud.google.com/kubernetes-engine/docs/how-to/stateful-apps - EKS storage best practices: https://docs.aws.amazon.com/eks/latest/best-practices/storage.html

🔹 Definition

StatefulSet storage patterns describe how to pair StatefulSets with per-pod PersistentVolumeClaims, zonal/regional disks, backup strategies, and multi-AZ layout — the building blocks for running databases, message queues, and distributed systems on Kubernetes.

🔹 About

Core pattern: volumeClaimTemplates

Every StatefulSet pod gets its own PVC, named `<template-name>-<sts-name>-<ordinal>` (e.g., `data-pg-0`). The PVC persists even if the pod is deleted or the STS is scaled down — data is always recoverable.

Storage topology

Scenario	Recommendation
Single-zone dev	`Immediate` binding, standard PD/gp3
Multi-zone prod	`WaitForFirstConsumer` + `topologySpreadConstraints`
HA database (GKE)	Regional PD (cross-zone replication)
HA database (EKS)	Application-level replication (Patroni, Stolon) + zonal EBS
Shared dataset (ML)	EFS / Filestore `ReadWriteMany` PVC (not per-pod)

PVC retention (K8s 1.27+ stable)

```
spec:
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain     # keep PVCs if STS deleted
    whenScaled:  Delete     # delete PVCs on scale-down
```

Default behavior: PVCs are **never deleted** — you must clean up manually.

Backup strategies

Layer	Tool	What it captures
Application	pg_dump, mongodump, Redis BGSAVE	Consistent logical backup
Volume snapshot	VolumeSnapshot CRD + CSI	Block-level point-in-time
Cluster backup	Velero / Backup for GKE	Manifests + volume data

Zonal pinning problem

EBS and standard PD are zonal. Once `data-pg-0` is created in `us-east1-b`, pod `pg-0` must always schedule there. If that zone goes down, `pg-0` is stuck Pending until the zone recovers. Mitigations: - **App-level replication** (replicas in different zones). - **Regional PD** on GKE (synchronous cross-zone). - **EFS** on EKS for RWX (but unsuitable for databases due to latency). - **Velero cross-region restore** as disaster recovery.

Common database patterns on K8s

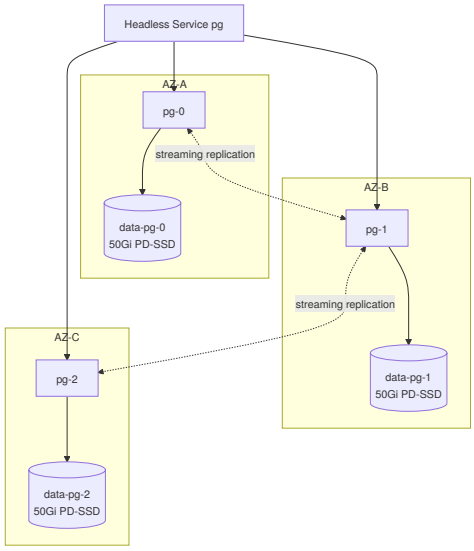
Database	Operator	Storage
PostgreSQL	CloudNativePG, Zalando, CrunchyData	RWO SSD per pod
MySQL	MySQL Operator (Oracle), Vitess	RWO per pod
MongoDB	MongoDB Community Operator	RWO per pod
Kafka	Strimzi	RWO per broker
Elasticsearch	ECK (Elastic)	RWO per data node
Redis	Redis Operator (Spotahome)	RWO or emptyDir (ephemeral cache)

🔷 How It Integrates With Other Kubernetes Components

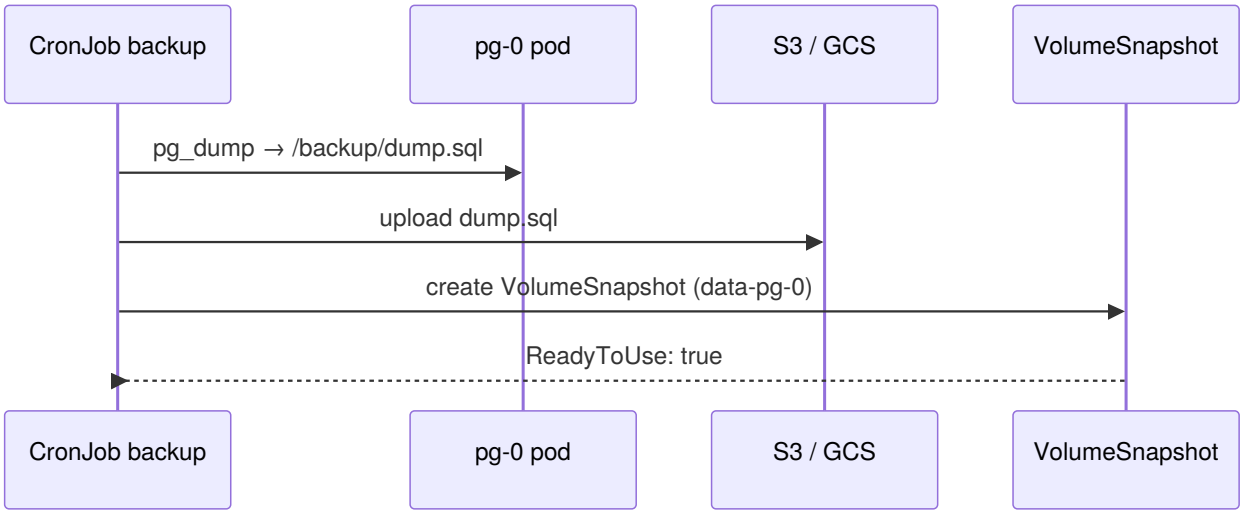
Component	Interaction
StatefulSet	`volumeClaimTemplates` creates PVCs
StorageClass	Determines disk type, zone, encryption
Scheduler	`WaitForFirstConsumer` + `topologySpreadConstraints`
Headless Service	Stable DNS per pod for replication topology
PodDisruptionBudget	Protect during drains + upgrades
VolumeSnapshot	Point-in-time backup per PVC
Operators	Manage replication, failover, backup around the STS

🏗 Diagrams

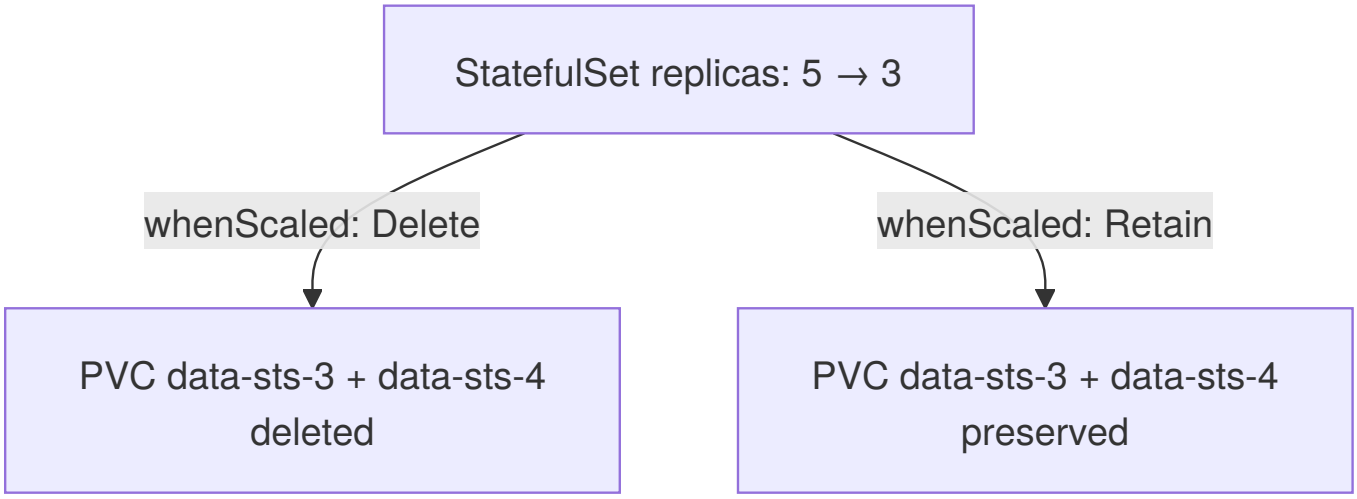
Multi-AZ StatefulSet layout



Backup flow



PVC retention on scale-down



💻 Real-World Examples

Example 1 — Production Postgres STS with backup CronJob

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: pg }
spec:
  serviceName: pg
  replicas: 3
  selector: { matchLabels: { app: pg } }
  template:
    metadata: { labels: { app: pg } }
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector: { matchLabels: { app: pg } }
      containers:
      - name: pg
        image: postgres:16
        ports: [{ containerPort: 5432 }]
        env: [{ name: PGDATA, value: /var/lib/postgresql/data/pgdata }]
        envFrom: [{ secretRef: { name: pg-cred } }]
        volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
        readinessProbe:
          exec: { command: ["pg_isready","-U","postgres"] }
          periodSeconds: 5
        resources:
          requests: { cpu: 500m, memory: 1Gi }
          limits:   { cpu: 2,    memory: 4Gi }
  volumeClaimTemplates:
  - metadata: { name: data }
    spec:
      accessModes: [ReadWriteOnce]
      resources: { requests: { storage: 50Gi } }
      storageClassName: gp3     # or standard-rwo on GKE
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain
    whenScaled:  Retain
---
apiVersion: batch/v1
kind: CronJob
metadata: { name: pg-backup }
spec:
  schedule: "0 2 * * *"
  timeZone: "Asia/Kolkata"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: dump
            image: postgres:16
            command: ["sh","-c","pg_dump -h pg-0.pg -U postgres mydb | gzip > /backup/$(date +%Y%m%d).sql.gz && aws s3 cp /backup/*.gz s3://backups/pg/"]
            envFrom: [{ secretRef: { name: pg-cred } }]
```

Example 2 — VolumeSnapshot before upgrade

```
kubectl apply -f - <<EOF
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata: { name: pre-upgrade-pg-0 }
spec:
  volumeSnapshotClassName: pd-snap
  source: { persistentVolumeClaimName: data-pg-0 }
EOF
kubectl wait --for=jsonpath='{.status.readyToUse}'=true vs/pre-upgrade-pg-0
# now safe to upgrade
kubectl set image sts/pg pg=postgres:17
```

Example 3 — Restore a single pod's data from snapshot

```
# Delete the corrupted PVC
kubectl delete pvc data-pg-1
# Recreate from snapshot
kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data-pg-1 }
spec:
  accessModes: [ReadWriteOnce]
  resources: { requests: { storage: 50Gi } }
  storageClassName: gp3
  dataSource: { kind: VolumeSnapshot, name: snap-pg-1-20240101, apiGroup: snapshot.storage.k8s.io }
EOF
# STS recreates pg-1 pod; it mounts the restored PVC
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pvc -l app=pg`	—	List STS PVCs	`kubectl get pvc -l app=pg`
`kubectl get volumesnapshot`	—	List snapshots	`kubectl get vs`
`kubectl describe sts`	—	volumeClaimTemplates	`kubectl describe sts pg`
`kubectl scale sts`	`--replicas=`	Scale (PVCs retained by default)	`kubectl scale sts pg --replicas=5`
`kubectl delete pvc`	`data-pg-3 data-pg-4`	Clean up scaled-down PVCs	
`velero backup create`	`--include-namespaces=prod`	Full backup	`velero backup create pg-full --include-namespaces=prod --snapshot-volumes`

🚨 Scenarios

Scenario 1 — Zone failure; primary pod stuck Pending

- **Cause**: zonal disk can't move to another AZ.
- **Fix**: promote a replica in another zone (app-level failover). Long-term: regional PD (GKE) or cross-AZ app replication.

Scenario 2 — Scale-down left orphan PVCs consuming cost

- **Cause**: default retention = Retain; PVCs kept after scale-down.
- **Fix**: set `whenScaled: Delete` or manually `kubectl delete pvc data-pg-3`.

Scenario 3 — Snapshot inconsistent for active DB

- **Cause**: block-level snapshot during write → data on disk may be inconsistent.
- **Fix**: use `CHECKPOINT` / `pg_start_backup()` before snapshot; or rely on WAL replay on restore. Application-level dump is always consistent.

Scenario 4 — PVC disk full → DB read-only

- **Cause**: PVC capacity exhausted.
- **Fix**: `kubectl patch pvc data-pg-0 -p '{"spec":{"resources":{"requests":{"storage":"100Gi"}}}}'` (online expansion); then `ALTER SYSTEM SET default_transaction_read_only = off;` inside Postgres.

Scenario 5 — Velero restore creates duplicate PVCs

- **Cause**: restoring to same namespace without cleanup.
- **Fix**: delete old resources first; or restore to a new namespace and switch DNS.

☁ GCP GKE Perspective

- **Regional PD** for HA: `replication-type: regional-pd` in SC. Synchronous cross-zone replication.
- **Backup for GKE**: scheduled, application-consistent backups of STS + PVCs + manifests. Restore to same or different cluster.
- **Autopilot**: STS works; charges per pod + PVC storage.

▩ AWS EKS Perspective

- EBS is zonal → app-level replication (Patroni, Strimzi MirrorMaker) is the HA pattern.
- **EFS** for RWX shared datasets (not for DB primary).
- **Velero + EBS snapshots** for DR; cross-region with `velero backup-location create --bucket=...`.
- **Karpenter** can provision specific instance types (storage-optimized) for data-heavy STS pods.

🧪 Hands-On Lab Exercises

Lab 31.1 — STS PVC persistence through delete/recreate (Beginner, 30 min)

1. Deploy 3-replica STS with `volumeClaimTemplates` (1Gi each).
2. Write `hostname > /data/id.txt` in each pod.
3. `kubectl delete sts pg --cascade=orphan` → pods remain.
4. Recreate STS → pods reattach PVCs → `cat /data/id.txt` shows same hostname.

Lab 31.2 — Snapshot + restore single pod (Intermediate, 60 min)

1. Write data to `pg-1`.
2. Create VolumeSnapshot of `data-pg-1`.
3. Corrupt data in `pg-1`: `kubectl exec pg-1 -- rm /data/pgdata -rf`.
4. Delete PVC `data-pg-1`, create from snapshot (Example 3).
5. Verify `pg-1` comes back with original data.

Lab 31.3 — Multi-AZ Postgres with Patroni on EKS (Advanced, 120 min)

1. Install Zalando Postgres Operator: `kubectl apply -k github.com/zalando/postgres-operator/manifests`.
2. Create `postgresql` CR with 3 replicas, `topologySpreadConstraints`, gp3 SC.
3. Verify pods spread across 3 AZs: `kubectl get pods -o wide`.
4. Simulate AZ failure: cordon all nodes in one AZ.
5. Observe Patroni failover: `kubectl exec pg-0 -- patronictl list` → new leader in healthy AZ.
6. Uncordon → follower catches up via streaming replication.

Cleanup: delete postgresql CR + operator; `eksctl delete cluster`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: StatefulSet storage patterns describe how to give each database or stateful pod its own persistent disk using `volumeClaimTemplates`, handle zonal constraints, and implement backup/recovery.
Why: A database pod needs stable identity AND stable storage. The volumeClaimTemplate guarantees that `data-pg-0` is always the same PVC, on the same disk, with the same data — even if the pod restarts or reschedules to the same zone.
How: I use a StorageClass with `WaitForFirstConsumer` so the disk is created in the zone the scheduler picks. I spread pods across zones with `topologySpreadConstraints`. For HA I rely on app-level replication — Patroni for Postgres, Strimzi for Kafka — rather than hoping for cross-zone disk magic. I back up two ways: a nightly CronJob running `pg_dump` to object storage, and hourly VolumeSnapshots for point-in-time block-level recovery. The `persistentVolumeClaimRetentionPolicy` is set to Retain on delete and Retain on scale, so accidental deletions never wipe data.
Example: On EKS we ran a 5-node Kafka cluster with Strimzi. Each broker had a 500Gi gp3 PVC. When we needed to expand to 1TB, we patched each PVC — online expansion took 30 seconds per broker with zero downtime."

? Possible Interview Questions

Q1. What is `volumeClaimTemplates`? STS field that auto-creates one PVC per pod from a template.
Q2. What happens to PVCs when STS is deleted? They are retained by default (not deleted).
Q3. How do you handle zonal storage in multi-AZ? `WaitForFirstConsumer` SC + `topologySpreadConstraints` on STS.
Q4. Can you expand an STS PVC? Yes — `kubectl patch pvc` if SC has `allowVolumeExpansion: true`.
Q5. How do you back up a StatefulSet DB? App-level dump (consistent) + VolumeSnapshot (fast, block-level) + Velero / Backup for GKE (full cluster).
Q6. What is `persistentVolumeClaimRetentionPolicy`? Controls auto-deletion of PVCs on STS delete (`whenDeleted`) or scale-down (`whenScaled`).
Q7. Regional PD on GKE — what is it? PD replicated across two zones; pod can reschedule to the other zone and reattach.
Q8. Why can't EBS do cross-AZ? EBS is zonal block storage. Cross-AZ requires app-level replication.
Q9. How do you restore a single pod's PVC from snapshot? Delete PVC → recreate with `dataSource: VolumeSnapshot` → STS recreates pod → mounts restored PVC.
Q10. Name a Postgres operator for K8s. CloudNativePG, Zalando Postgres Operator, CrunchyData PGO.
Q11. How does Patroni handle failover? Leader election via DCS (K8s Endpoints/ConfigMap/etcd); promotes a replica if leader unreachable.
Q12. What's the risk of a block-level snapshot on an active DB? May capture inconsistent on-disk state. Mitigate with `CHECKPOINT` / freeze before snapshot, or rely on WAL replay.

MODULE

Module 6 — Scheduling

🗐 Topic 32 — Scheduler Deep Dive

Official Documentation Reference - Kubernetes Scheduler: https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/ - Scheduling framework: https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/ - Scheduler performance tuning: https://kubernetes.io/docs/concepts/scheduling-eviction/scheduler-perf-tuning/ - Assigning Pods to Nodes: https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/ - GKE scheduling: https://cloud.google.com/kubernetes-engine/docs/concepts/scheduling - EKS scheduling: https://docs.aws.amazon.com/eks/latest/best-practices/scheduling.html

🔹 Definition

The **kube-scheduler** is the control-plane component that watches for newly created Pods with no assigned node, selects the best node for each Pod, and writes the binding. It uses a two-phase process: **filtering** (which nodes can run the pod?) then **scoring** (which node is best?).

🔹 About

Scheduling cycle

1. **Queue** — new unscheduled pod enters the scheduling queue (priority-sorted).
2. **Filter (Predicates)** — eliminate nodes that can't run the pod:
3. `PodFitsResources` — enough CPU/memory?
4. `PodFitsHostPorts` — host ports available?
5. `NodeAffinity` / `NodeSelector` — labels match?
6. `TaintToleration` — pod tolerates node taints?
7. `PodTopologySpread` — maxSkew respected?
8. `VolumeBinding` — volume zone compatible?
9. `NodeUnschedulable` — node cordoned?
10. **Score (Priorities)** — rank surviving nodes 0–100:
11. `NodeResourcesFit` (LeastAllocated, MostPacked, RequestedToCapacityRatio)
12. `InterPodAffinity` — prefers nodes satisfying pod affinity
13. `ImageLocality` — prefers nodes that already have the image
14. `TaintToleration` — prefers nodes with fewer taints
15. `NodeResourcesBalancedAllocation` — balances CPU/mem ratio
16. `PodTopologySpread` — spreads across domains
17. **Select** — highest-scoring node wins (ties broken randomly).
18. **Bind** — scheduler writes `spec.nodeName` on the pod.

Scheduling framework (extension points)

The scheduler is built on a plugin framework with extension points: `QueueSort → PreFilter → Filter → PostFilter → PreScore → Score → Reserve → Permit → PreBind → Bind → PostBind`

percentageOfNodesToScore

On large clusters (5000+ nodes), the scheduler doesn't score all feasible nodes — it scores a percentage (default scales down with cluster size). Tune with `percentageOfNodesToScore` in scheduler config.

Scheduling profiles

Multiple profiles can be configured on a single scheduler, each with different plugins. Pods reference a profile via `spec.schedulerName`.

Unschedulable pods

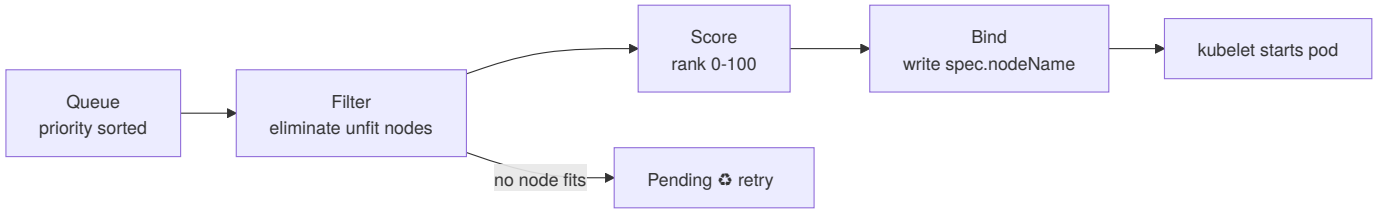
If no node passes filtering, the pod stays `Pending` with events like `0/5 nodes are available: 2 Insufficient cpu, 3 node(s) had taint ...`. The scheduler retries when conditions change (new node, freed resources).

🔹 How It Integrates With Other Kubernetes Components

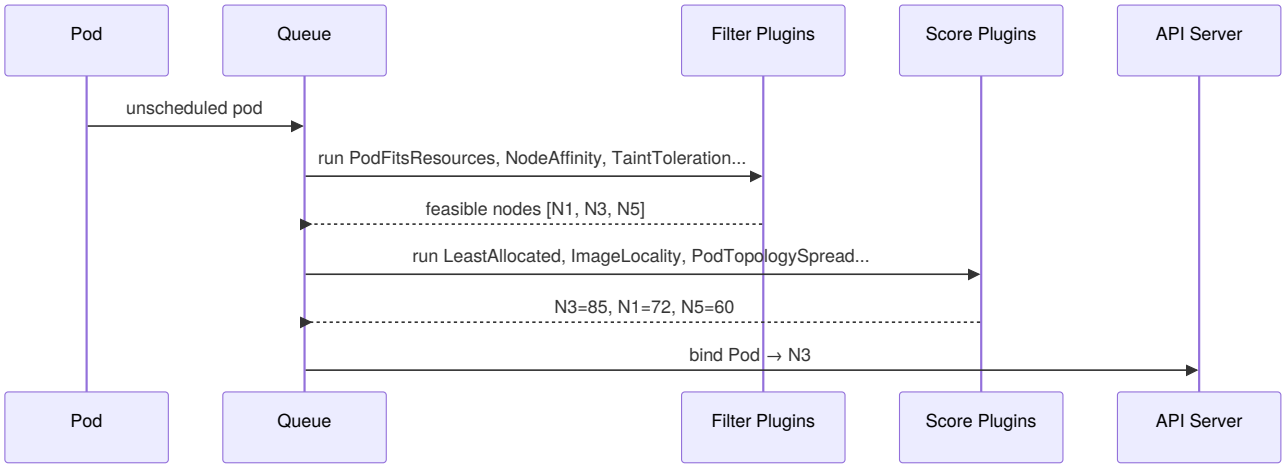
Component	Interaction
API server	Scheduler watches unscheduled pods; writes `spec.nodeName`
kubelet	Picks up bound pods and runs them
Nodes	Supply capacity and labels for filter/score
PriorityClass	Determines queue ordering and preemption
Cluster Autoscaler / Karpenter	Trigger node creation when scheduler can't place pods
CSI (WaitForFirstConsumer)	Scheduler coordinates volume binding
PDB	Preemption respects PDB

🏗 Diagrams

Scheduling cycle



Filter + Score in detail



Extension points



💻 Real-World Examples

Example 1 — See why a pod is Pending

```
kubectl describe pod stuck-pod | tail -20
# Events:
#   Warning  FailedScheduling  0/5 nodes are available:
#     2 Insufficient cpu, 3 node(s) had taint {dedicated: gpu} that the pod didn't tolerate.
```

Example 2 — Force a pod to a specific node (bypass scheduler)

```
spec:
  nodeName: worker-3          # skips scheduler entirely — rarely used
```

Example 3 — Scheduler config (KubeSchedulerConfiguration)

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
  plugins:
    score:
      enabled:
      - { name: NodeResourcesFit, weight: 2 }
      - { name: ImageLocality, weight: 1 }
  pluginConfig:
  - name: NodeResourcesFit
    args:
      scoringStrategy:
        type: LeastAllocated
        resources:
        - { name: cpu, weight: 1 }
        - { name: memory, weight: 1 }
percentageOfNodesToScore: 50
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl describe pod`	—	See scheduling events	`kubectl describe pod stuck`
`kubectl get pods`	`--field-selector spec.nodeName=""`	Find unscheduled pods	
`kubectl get nodes`	`-o custom-columns=NAME:.metadata.name,CPU:.status.allocatable.cpu,MEM:.status.allocatable.memory`	Node capacity	
`kubectl describe node`	`<n>`	Allocated resources section	`kubectl describe node worker-1`
`kubectl cordon`	`<n>`	Mark unschedulable	`kubectl cordon worker-1`
`kubectl uncordon`	`<n>`	Mark schedulable	`kubectl uncordon worker-1`
`kubectl get events`	`--field-selector reason=FailedScheduling`	Scheduling failures	

🚨 Scenarios

Scenario 1 — Pod Pending: Insufficient CPU

- **Cause**: sum of existing requests + new pod exceeds node allocatable.
- **Fix**: scale cluster (add nodes), right-size existing pods, or set higher PriorityClass to preempt.

Scenario 2 — Pod scheduled to wrong zone (volume mismatch)

- **Cause**: `Immediate` binding created disk in zone A; scheduler picked zone B.
- **Fix**: use `WaitForFirstConsumer` StorageClass.

Scenario 3 — Scheduler latency spikes on 2000-node cluster

- **Cause**: scoring all nodes per pod.
- **Fix**: tune `percentageOfNodesToScore` down; or use scheduling profiles with fewer score plugins.

Scenario 4 — Pod keeps bouncing between Pending and Running

- **Cause**: preemption cycle — lower-priority pod evicted, higher placed, then lower can't find a home.
- **Fix**: review PriorityClasses; add PDB; use Cluster Autoscaler for capacity.

☁ GCP GKE Perspective

- **Autopilot** handles scheduling transparently; pods may trigger node provisioning (Node Auto-Provisioning).
- **Standard**: default scheduler; customize with node pools + taints + affinities.
- GKE does NOT expose scheduler config customization — use a custom scheduler Deployment if needed.

▩ AWS EKS Perspective

- EKS runs the managed scheduler — no access to `KubeSchedulerConfiguration`.
- **Karpenter** bypasses the scheduler's node selection by provisioning right-sized nodes for pending pods.
- **Fargate**: the AWS Fargate scheduler (not kube-scheduler) places pods into microVMs.

🧪 Hands-On Lab Exercises

Lab 32.1 — Understand Pending (Beginner, 30 min)

Create a pod requesting 100 CPUs. Observe Pending. `kubectl describe pod` → events. Reduce to 100m → schedules.

Lab 32.2 — Scoring: LeastAllocated vs MostPacked (Intermediate, 60 min)

1. Kind 3-worker cluster. Deploy 3 pods with 200m CPU each.
2. Deploy 3 more → observe spread (default: LeastAllocated scores empty nodes higher).
3. Change scheduler config to `MostPacked` (requires deploying a second scheduler on Kind).
4. Re-test — pods now pack onto fewer nodes.

Lab 32.3 — Custom scheduler profile (Advanced, 90 min)

1. Deploy a second scheduler as a Deployment with a custom `KubeSchedulerConfiguration`.
2. Create pods with `schedulerName: my-scheduler`.
3. Verify different placement from default scheduler via `kubectl get pods -o wide`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: The kube-scheduler watches for unscheduled pods, runs them through a filter phase that eliminates unfit nodes — checking resources, taints, affinity, topology — and a score phase that ranks the survivors by criteria like least-allocated resources and image locality. It then binds the pod to the highest-scoring node.
Why: Smart scheduling is what turns a cluster into more than a bag of VMs. It ensures pods land where they can run, resources are balanced, topology constraints are met, and higher-priority pods can preempt lower ones.
How: I influence scheduling through nodeSelector, node/pod affinity, taints+tolerations, topologySpreadConstraints, and PriorityClasses — rather than modifying the scheduler itself, which is managed on GKE/EKS. On EKS, Karpenter complements the scheduler by provisioning exactly the right node type for pending pods.
Example: We had pods that sat Pending for minutes because the scheduler scored all 500 nodes. After tuning `percentageOfNodesToScore` to 30% (via a custom scheduler on our kubeadm cluster), scheduling latency dropped from 8 seconds to under 1 second."

? Possible Interview Questions

Basic

Q1. What does the scheduler do? Assigns unscheduled pods to nodes via filter + score.
Q2. What happens if no node passes filtering? Pod stays Pending.
Q3. Where is the binding written? `spec.nodeName` on the Pod object.

Intermediate

Q4. Name 3 filter plugins. PodFitsResources, TaintToleration, NodeAffinity.
Q5. Name 3 scoring plugins. NodeResourcesFit (LeastAllocated), ImageLocality, PodTopologySpread.
Q6. What is `percentageOfNodesToScore`? Controls how many feasible nodes get scored — tuning for large clusters.

Advanced

Q7. Explain the scheduling framework extension points. QueueSort → PreFilter → Filter → PostFilter (preemption) → PreScore → Score → Reserve → Permit → PreBind → Bind → PostBind.
Q8. What is PostFilter? Runs when no node passes Filter; implements preemption logic.
Q9. How do scheduling profiles work? One scheduler binary can host multiple profiles with different plugin configs; pods select via `spec.schedulerName`.

Scenario-based

Q10. Pod Pending: "0/10 nodes are available: 10 node(s) didn't match Pod's node affinity/selector" — fix? Node labels don't match pod's nodeAffinity/nodeSelector. Label nodes or relax affinity.

GKE-specific

Q11. Can you customize the scheduler on GKE? Not the managed one. Deploy a custom scheduler as a Deployment alongside.

EKS-specific

Q12. How does Karpenter interact with the scheduler? Karpenter watches Pending pods, provisions right-sized nodes, and the scheduler then binds pods to those nodes.

Hands-on

Q13. How do you debug scheduling? `kubectl describe pod` → events; `kubectl get nodes -o custom-columns` for capacity; `kubectl get events --field-selector reason=FailedScheduling`.
Q14. Show me how to force a pod to a node. `spec.nodeName: worker-3` (bypasses scheduler — not recommended for production).
Q15. What is the difference between `nodeName` and `nodeSelector`? `nodeName` bypasses the scheduler. `nodeSelector` is evaluated by the scheduler's filter phase.

📓 Topic 33 — Node Selectors, Node Affinity, Anti-Affinity

Official Documentation Reference - Assign pods to nodes: https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/ - Node affinity: https://kubernetes.io/docs/tasks/configure-pod-container/assign-pods-nodes-using-node-affinity/ - GKE node affinity: https://cloud.google.com/kubernetes-engine/docs/concepts/node-affinity - EKS scheduling: https://docs.aws.amazon.com/eks/latest/best-practices/scheduling.html

---

🔷 Definition

- **nodeSelector** — simplest way to constrain a pod to nodes matching specific labels (equality match).
- **Node Affinity** — expressive rules (required or preferred, set-based operators) that attract pods to specific nodes.
- **Pod Affinity / Anti-Affinity** — attract or repel pods relative to other pods' locations.

🔷 About

nodeSelector (simplest)

```
spec:
  nodeSelector:
    disktype: ssd
```

Pod only runs on nodes labeled `disktype=ssd`. Hard requirement — no matching node = Pending.

Node Affinity

Two flavors: | Type | Behavior | Use case | |---|---|---| | `requiredDuringSchedulingIgnoredDuringExecution` | Hard — must match | GPU workloads, zonal pinning | | `preferredDuringSchedulingIgnoredDuringExecution` | Soft — prefer, not require | Spread across cheaper nodes |

`IgnoredDuringExecution` = if node label changes after scheduling, pod is NOT evicted (today's only mode).

Operators: `In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt`, `Lt`.

Pod Affinity / Anti-Affinity

Attract or repel pods relative to other pods on the same topology domain (node, zone, region).

Type	Behavior	Example
Pod affinity (required)	Must co-locate with matching pods	Cache pod near app pod
Pod affinity (preferred)	Prefer co-location	
Pod anti-affinity (required)	Must NOT be on same topology as matching pods	Spread replicas across nodes
Pod anti-affinity (preferred)	Prefer spreading	

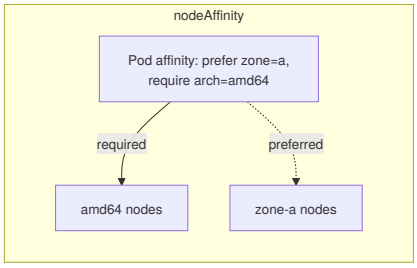
`topologyKey` defines the domain: `kubernetes.io/hostname` (per-node), `topology.kubernetes.io/zone` (per-zone).

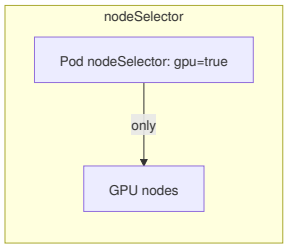
Performance warning

Pod anti-affinity with `requiredDuringScheduling` and `topologyKey: kubernetes.io/hostname` scans all pods on every node — expensive at scale. Prefer `topologySpreadConstraints` (Topic 35) for even spread.

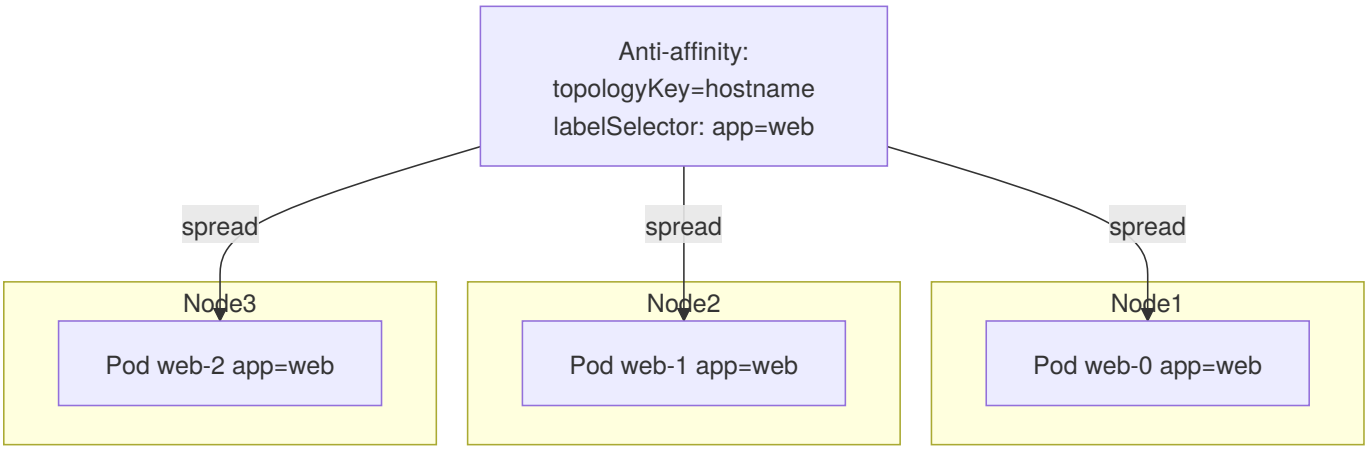
🏗 Diagrams

nodeSelector vs node affinity





Pod anti-affinity across nodes



💻 Real-World Examples

Example 1 — nodeSelector

```
spec:
  nodeSelector:
    accelerator: nvidia-tesla-t4
```

Example 2 — Node affinity (required + preferred)

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - { key: kubernetes.io/arch, operator: In, values: [amd64] }
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 80
        preference:
          matchExpressions:
          - { key: topology.kubernetes.io/zone, operator: In, values: [ap-south-1a] }
```

Example 3 — Pod anti-affinity (spread across nodes)

```
spec:
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels: { app: web }
        topologyKey: kubernetes.io/hostname
```

Example 4 — Pod affinity (co-locate cache with app)

```
spec:
  affinity:
    podAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels: { app: api }
        topologyKey: kubernetes.io/hostname
```

Example 5 — Preferred anti-affinity across zones

```
spec:
  affinity:
    podAntiAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector: { matchLabels: { app: web } }
          topologyKey: topology.kubernetes.io/zone
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl label node`	`key=val`	Add node label	`kubectl label node w1 disktype=ssd`
`kubectl label node`	`key-`	Remove label	`kubectl label node w1 disktype-`
`kubectl get nodes`	`--show-labels`	Show all labels	`kubectl get nodes --show-labels`
`kubectl get nodes`	`-L key1,key2`	Show specific labels	`kubectl get nodes -L zone,disktype`
`kubectl get pods`	`-o wide`	See which node each pod landed on	`kubectl get pods -o wide`
`kubectl describe pod`	—	See affinity + scheduling events	`kubectl describe pod web-0`

🚨 Scenarios

Scenario 1 — Pod Pending: node affinity has no matches

- **Cause**: `required` affinity uses label that no node carries.
- **Fix**: label nodes, or switch to `preferred`.

Scenario 2 — Anti-affinity blocks scaling beyond node count

- **Cause**: `required` anti-affinity on hostname = 1 pod/node. 3 nodes → max 3 pods.
- **Fix**: switch to `preferred`, or use `topologySpreadConstraints` with `whenUnsatisfiable: ScheduleAnyway`.

Scenario 3 — Pod affinity creates hotspot

- **Cause**: all pods co-located on one node; node fails → all pods down.
- **Fix**: combine affinity with anti-affinity or topology spread to balance.

☁ GCP GKE Perspective

- Common node labels: `cloud.google.com/gke-nodepool`, `topology.kubernetes.io/zone`, `cloud.google.com/gke-accelerator`.
- **Autopilot**: node labels set by Google; use `cloud.google.com/compute-class` for workload separation.

▩ AWS EKS Perspective

- Common labels: `topology.kubernetes.io/zone`, `node.kubernetes.io/instance-type`, `karpenter.sh/capacity-type` (spot/on-demand).
- **Karpenter**: reads pod affinity/nodeSelector requirements and provisions matching instance types.

🧪 Hands-On Lab Exercises

Lab 33.1 — nodeSelector (Beginner, 20 min)

Label one node `tier=frontend`. Deploy pod with `nodeSelector: {tier: frontend}`. Verify pod lands there.

Lab 33.2 — Pod anti-affinity spread (Intermediate, 45 min)

Deploy 3-replica Deployment with `requiredDuringScheduling` anti-affinity on hostname. Verify one pod per node. Try scaling to 4 → Pending.

Lab 33.3 — Mixed affinity: co-locate cache + spread app (Advanced, 60 min)

1. Deploy `app` with anti-affinity across zones.
2. Deploy `cache` with pod affinity to `app` on hostname.
3. Verify cache lands on same node as app, but app pods are in different zones.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: nodeSelector is the simplest way to pin a pod to labeled nodes. Node affinity gives required or preferred rules with set-based operators. Pod affinity and anti-affinity attract or repel pods relative to other pods' locations on a topology domain — node, zone, or region.
Why: These controls let you match workloads to hardware (GPU nodes, SSD nodes), separate tenants, co-locate latency-sensitive pairs, and spread replicas for high availability.
How: I use nodeSelector for simple cases (GPU workloads). For multi-zone HA I combine preferred node affinity with required pod anti-affinity on the zone topology key. For co-located caches I use pod affinity on hostname. I'm moving toward topologySpreadConstraints for most spread use cases — they're more predictable and performant than pod anti-affinity.
Example: Our ML inference pods needed T4 GPUs. We labeled the GPU pool nodes `accelerator=nvidia-tesla-t4` and set `nodeSelector` — pods never land on CPU-only nodes, and Karpenter provisions GPU instances only when inference pods are pending."

? Possible Interview Questions

Q1. nodeSelector vs node affinity? nodeSelector: equality only, hard. Affinity: set-based operators, required or preferred.
Q2. What does `requiredDuringSchedulingIgnoredDuringExecution` mean? Must match at scheduling time; pod not evicted if label changes later.
Q3. Pod affinity `topologyKey` — what is it? The node label defining the topology domain (e.g., hostname, zone).
Q4. When would you use pod affinity? Co-locate tightly coupled services (app + cache) on the same node/zone for low latency.
Q5. Why is pod anti-affinity expensive at scale? Scheduler must check all existing pods on every candidate node — O(pods × nodes).
Q6. `preferred` with weight — how does it work? Each matching preferred term adds its weight to the node's score; highest total wins.
Q7. Can you combine node affinity and pod anti-affinity? Yes — both are in `spec.affinity`; evaluated independently.
Q8. What replaces pod anti-affinity for even spread? `topologySpreadConstraints` (Topic 35).
Q9. What `Gt` / `Lt` operators do in node affinity? Compare numeric label values (e.g., `nvidia.com/gpu.count Gt 2`).
Q10. How does Karpenter use pod affinity? Reads pod requirements (affinity, nodeSelector, resources) and provisions matching instance types/zones.

🗐 Topic 34 — Taints and Tolerations

Official Documentation Reference - Taints and Tolerations: https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/ - Well-known taints: https://kubernetes.io/docs/reference/labels-annotations-taints/ - GKE taints: https://cloud.google.com/kubernetes-engine/docs/how-to/node-taints - EKS taints: https://docs.aws.amazon.com/eks/latest/userguide/node-taints-managed-node-groups.html

---

⏚ Definition

A **taint** is applied to a node to repel pods that don't explicitly tolerate it. A **toleration** is applied to a pod to allow scheduling on tainted nodes. Together they ensure only specific workloads run on designated nodes.

⏚ About

Taint = node-level repellent

```
key=value:effect
```

Effects

Effect	Behavior
`NoSchedule`	New pods without toleration won't be scheduled; existing stay
`PreferNoSchedule`	Scheduler avoids, but may still place if no other option
`NoExecute`	New pods rejected AND existing pods without toleration are evicted

Toleration = pod-level permission

```
tolerations:
- key: dedicated
  operator: Equal
  value: gpu
  effect: NoSchedule
```

Operators: - `Equal` — key + value + effect must all match. - `Exists` — key + effect match (value ignored). If key is empty + `Exists` → matches all taints.

`tolerationSeconds` (NoExecute only)

How long to keep the pod before eviction after taint is added. Omit = tolerate forever.

Built-in taints (auto-added by K8s)

Taint	When
`node.kubernetes.io/not-ready`	Node condition Ready=False
`node.kubernetes.io/unreachable`	Node condition Ready=Unknown
`node.kubernetes.io/disk-pressure`	Disk pressure
`node.kubernetes.io/memory-pressure`	Memory pressure
`node.kubernetes.io/pid-pressure`	PID pressure
`node.kubernetes.io/unschedulable`	`kubectl cordon`
`node.kubernetes.io/network-unavailable`	Network not ready

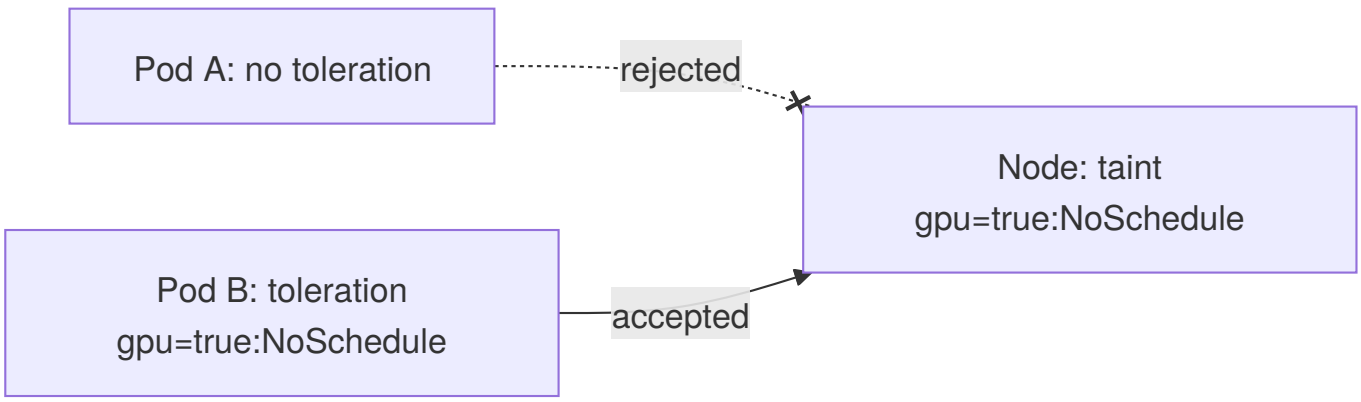
Default tolerations on every pod (added by DefaultTolerationSeconds admission plugin): - `not-ready` : tolerationSeconds=300 - `unreachable` : tolerationSeconds=300

Common patterns

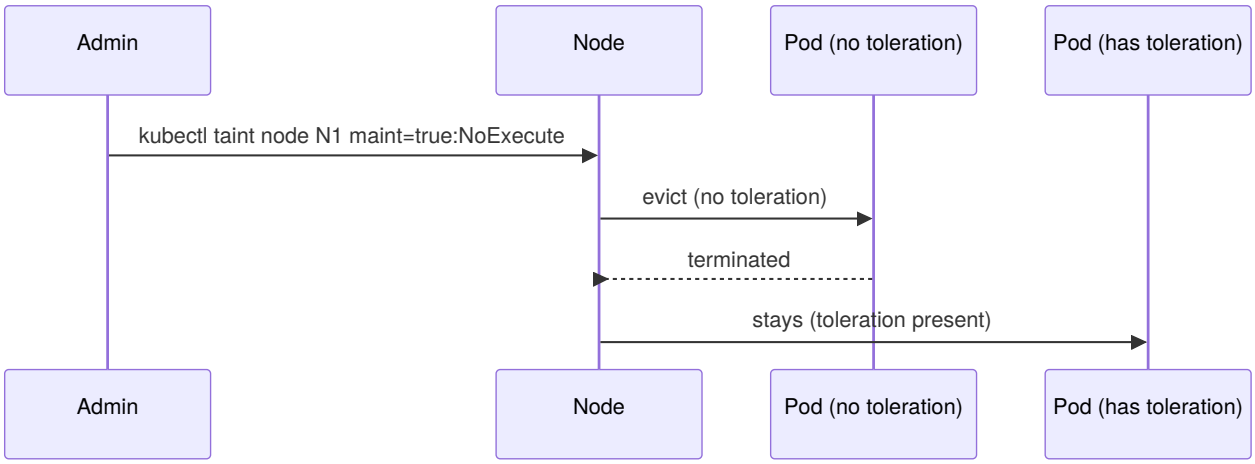
Pattern	How
Dedicated node pool	Taint nodes `dedicated=team-x:NoSchedule` ; only team-x pods have toleration
GPU nodes	Taint `nvidia.com/gpu=present:NoSchedule` ; GPU pods tolerate
Infra nodes (monitoring/logging)	Taint `role=infra:NoSchedule`
Spot/preemptible nodes	Taint `cloud.google.com/gke-preemptible=true:NoSchedule`

🏗 Diagrams

Taint + toleration gate



NoExecute eviction



💻 Real-World Examples

Example 1 — Taint a node

```
kubectl taint node worker-1 dedicated=ml:NoSchedule
kubectl describe node worker-1 | grep Taint
```

Example 2 — Remove a taint

```
kubectl taint node worker-1 dedicated=ml:NoSchedule-    # trailing minus
```

Example 3 — Pod with toleration

```
spec:
  tolerations:
  - { key: dedicated, operator: Equal, value: ml, effect: NoSchedule }
  nodeSelector: { dedicated: ml }      # pair with selector to ATTRACT
  containers: [{ name: train, image: myrepo/ml:1.0 }]
```

Example 4 — Tolerate everything (DaemonSet pattern)

```
tolerations:
- operator: Exists
```

Example 5 — NoExecute with grace period

```
tolerations:
- key: node.kubernetes.io/not-ready
  operator: Exists
  effect: NoExecute
  tolerationSeconds: 60          # evict after 60s
```

Example 6 — GKE node pool with taint

```
gcloud container node-pools create gpu-pool \
  --cluster=prod --zone=asia-south1-a \
  --machine-type=n1-standard-8 --accelerator=type=nvidia-tesla-t4,count=1 \
  --node-taints=nvidia.com/gpu=present:NoSchedule \
  --num-nodes=2
```

Example 7 — EKS managed NG with taint

```
eksctl create nodegroup --cluster=prod --name=gpu \
  --node-type=g4dn.xlarge --nodes=2 \
  --node-taints="nvidia.com/gpu=present:NoSchedule"
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl taint node`	`key=val:effect`	Add taint	`kubectl taint node w1 dedicated=ml:NoSchedule`
`kubectl taint node`	`key=val:effect-`	Remove taint	`kubectl taint node w1 dedicated=ml:NoSchedule-`
`kubectl taint node`	`key:effect-`	Remove by key+effect	`kubectl taint node w1 dedicated:NoSchedule-`
`kubectl describe node`	—	See taints section	`kubectl describe node w1`
`kubectl get nodes`	`-o jsonpath='{.items[*].spec.taints}'`	List all taints	

🚨 Scenarios

Scenario 1 — All pods evicted from node after taint

- **Cause**: `NoExecute` taint applied; existing pods lacked toleration.
- **Fix**: intended behavior for maintenance drain. For gradual: use `NoSchedule` first, then drain.

Scenario 2 — DaemonSet pod not running on tainted node

- **Cause**: DS pod template missing toleration.
- **Fix**: add `tolerations: [{operator: Exists}]` (wide) or specific key match.

Scenario 3 — Dedicated node pool empty even though workloads exist

- **Cause**: taint applied, but nodeSelector/affinity missing — pods tolerate but don't attract.
- **Fix**: always pair toleration with nodeSelector or node affinity.

Scenario 4 — Pod evicted too fast on node flap

- **Cause**: default `tolerationSeconds: 300` for `not-ready` too short for network glitch.
- **Fix**: increase `tolerationSeconds` on critical pods or set PDB.

☁ GCP GKE Perspective

- GKE auto-taints preemptible/Spot nodes: `cloud.google.com/gke-preemptible=true:NoSchedule`.
- GPU node pools auto-tainted: `nvidia.com/gpu=present:NoSchedule`.
- **Autopilot**: Google manages taints; you can't taint Autopilot nodes directly.

▩ AWS EKS Perspective

- Managed NG supports taints via `eksctl` or `aws eks create-nodegroup --taints`.
- Karpenter: taints defined in `NodePool` spec; Karpenter provisions tainted nodes only for pods that tolerate them.
- Spot nodes: Karpenter adds `karpenter.sh/capacity-type=spot` but doesn't auto-taint by default — configure via NodePool.

🧪 Hands-On Lab Exercises

Lab 34.1 — Taint + tolerate (Beginner, 20 min)

1. Taint a node. 2. Deploy pod without toleration → Pending. 3. Add toleration → schedules.

Lab 34.2 — NoExecute eviction (Intermediate, 30 min)

1. Deploy pod on node. 2. `kubectl taint node <n> maint=true:NoExecute`. 3. Pod evicted. 4. Deploy pod with toleration + `tolerationSeconds: 30`. 5. After 30s → evicted.

Lab 34.3 — Dedicated GPU pool on GKE/EKS (Advanced, 60 min)

1. Create tainted GPU node pool (Example 6 or 7).
2. Deploy ML pod with toleration + nodeSelector.
3. Deploy regular pod → stays on default pool.
4. Verify with `kubectl get pods -o wide`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Taints are node-level labels that repel pods; tolerations are pod-level declarations that let pods tolerate specific taints. Effects control severity: NoSchedule prevents new scheduling, PreferNoSchedule is soft, NoExecute also evicts existing pods.
Why: Taints enforce node specialization. GPU nodes, Spot nodes, infra-only nodes — taints ensure only the right workloads land there. Kubernetes also auto-taints nodes under pressure (disk, memory, not-ready) to protect the cluster.
How: I pair every taint with a toleration AND a nodeSelector or affinity — toleration only opens the gate, it doesn't attract. For DaemonSets I use `operator: Exists` to tolerate everything. For maintenance I add `NoExecute` with `tolerationSeconds` so pods drain gracefully.
Example: We created a dedicated GPU node pool on EKS tainted `nvidia.com/gpu=present:NoSchedule`. ML pods carry the toleration and nodeSelector; nothing else runs there. This cut our GPU costs 35% because non-GPU pods no longer wasted GPU-instance resources."

? Possible Interview Questions

Q1. Taint vs nodeSelector? Taint repels (on node). nodeSelector attracts (on pod). Use both together.
Q2. Three taint effects? NoSchedule, PreferNoSchedule, NoExecute.
Q3. NoExecute vs NoSchedule? NoExecute also evicts running pods. NoSchedule only prevents new scheduling.
Q4. What does `operator: Exists` do? Matches any taint with that key (value ignored). Empty key + Exists = match all taints.
Q5. What is `tolerationSeconds`? How long a pod stays on a NoExecute-tainted node before eviction.
Q6. Name 3 built-in taints. `not-ready`, `unreachable`, `disk-pressure`.
Q7. Why do DaemonSets need `operator: Exists`? To run on every node including tainted ones (infra, GPU, Spot).
Q8. How do you drain a node for maintenance? `kubectl drain <n> --ignore-daemonsets --delete-emptydir-data` — adds `NoSchedule` + evicts pods.
Q9. How does Karpenter handle taints? NodePool spec includes taints; Karpenter provisions tainted nodes only when pending pods tolerate them.
Q10. Can you taint the control plane? Yes — kubeadm auto-taints CP: `node-role.kubernetes.io/control-plane:NoSchedule`. Remove to schedule user pods on CP (not recommended in prod).

📄 Topic 35 — Topology Spread Constraints

Official Documentation Reference - Topology Spread Constraints: https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/ - GKE topology: https://cloud.google.com/kubernetes-engine/docs/concepts/topology-spread-constraints - EKS best practices: https://docs.aws.amazon.com/eks/latest/best-practices/scheduling.html

🔷 Definition

TopologySpreadConstraints control how pods are distributed across failure domains — nodes, zones, or regions — ensuring even spread for high availability. They're the recommended replacement for pod anti-affinity for most spread use cases.

🔷 About

Key fields

Field	Purpose
`maxSkew`	Maximum allowed difference in pod count between any two topology domains
`topologyKey`	Node label defining the domain (`kubernetes.io/hostname`, `topology.kubernetes.io/zone`)
`whenUnsatisfiable`	`DoNotSchedule` (hard) or `ScheduleAnyway` (soft + minimize skew)
`labelSelector`	Which pods count toward the constraint
`minDomains`	Minimum number of domains required (K8s 1.30+ stable)
`matchLabelKeys`	Use pod label values at scheduling time (K8s 1.27+ beta)
`nodeAffinityPolicy` / `nodeTaintsPolicy`	Whether filtered-out nodes are included in skew calculation

Why better than pod anti-affinity

- Anti-affinity is binary (co-locate or not); spread constraints are quantitative (`maxSkew`).
- Anti-affinity is O(pods × nodes) in computation; TSC is more efficient.
- TSC works across any topology key, with soft or hard enforcement.

Multiple constraints

You can stack multiple TSC entries — all must be satisfied.

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: { matchLabels: { app: web } }
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway
  labelSelector: { matchLabels: { app: web } }
```

This says: "hard spread across zones (max skew 1), soft spread across nodes."

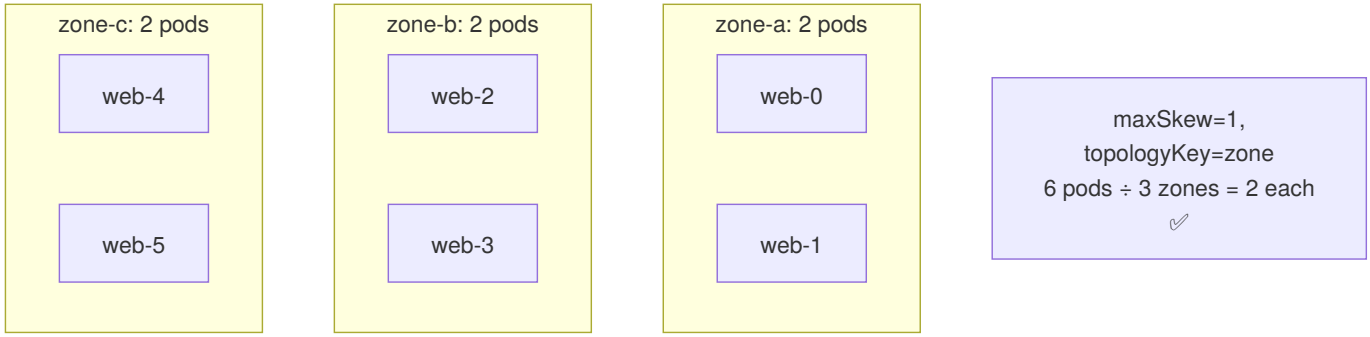
Cluster-level defaults

Since K8s 1.24, cluster-wide default constraints can be set in the scheduler config:

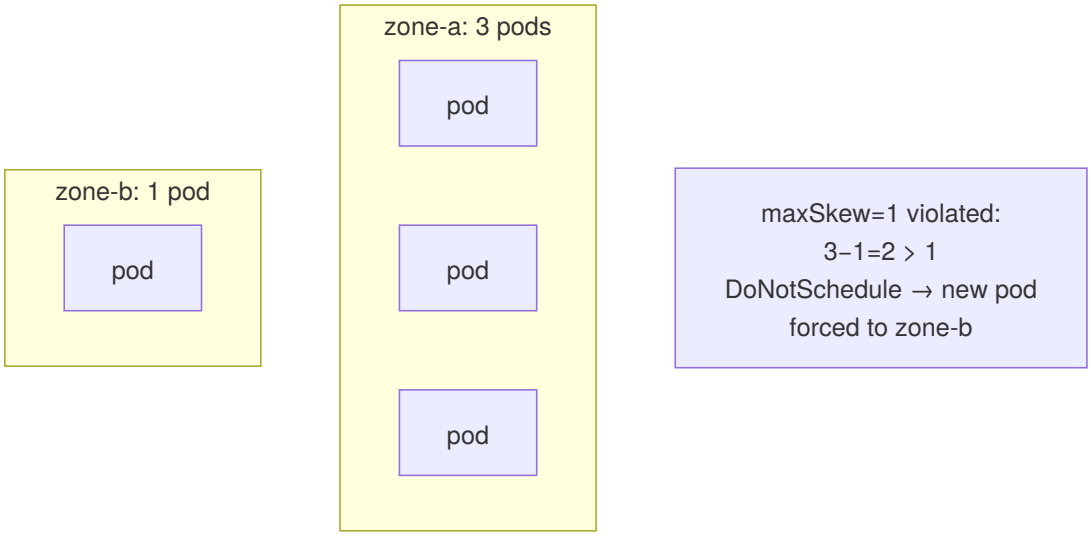
```
profiles:
- pluginConfig:
  - name: PodTopologySpread
    args:
      defaultConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
```

🏗 Diagrams

Even spread across zones



Skew violation



💻 Real-World Examples

Example 1 — Spread Deployment across zones

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 6
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector: { matchLabels: { app: web } }
      containers: [{ name: c, image: nginx:1.27 }]
```

Example 2 — Spread across nodes (soft) + zones (hard)

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: { matchLabels: { app: web } }
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway
  labelSelector: { matchLabels: { app: web } }
```

Example 3 — minDomains

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  minDomains: 3
  labelSelector: { matchLabels: { app: web } }
```

Scheduler won't place until at least 3 zones have nodes.

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pods`	`-o wide`	See pod-to-node mapping	`kubectl get pods -o wide -l app=web`
`kubectl get nodes`	`-L topology.kubernetes.io/zone`	Show zone labels	
`kubectl describe pod`	—	See TSC events	`kubectl describe pod web-0`
`kubectl get events`	`--field-selector reason=FailedScheduling`	TSC failures	

🚨 Scenarios

Scenario 1 — Skew violated; new pod Pending

- **Cause**: `DoNotSchedule` and the only zone with capacity would increase skew beyond maxSkew.
- **Fix**: scale the cluster in the under-populated zone; or use `ScheduleAnyway`.

Scenario 2 — After node failure, pods bunch up in surviving zones

- **Cause**: rescheduled pods all go to one zone; existing pods already there → skew worsens.
- **Fix**: `maxSkew: 2` for more flexibility during failures; or add capacity in all zones.

Scenario 3 — TSC + PDB conflict

- **Cause**: PDB prevents eviction needed to rebalance; TSC prevents scheduling to maintain balance.
- **Fix**: loosen PDB or set TSC to `ScheduleAnyway`.

☁ GCP GKE Perspective

- **Autopilot** may auto-spread pods with built-in defaults. Node labels `topology.kubernetes.io/zone` set automatically.
- **Regional clusters** have nodes across 3 zones → TSC `maxSkew: 1` across zones is ideal.

▩ AWS EKS Perspective

- EKS nodes get `topology.kubernetes.io/zone` from EC2 metadata.
- **Karpenter** reads TSC and provisions nodes in under-represented zones to satisfy the constraint.
- Fargate: TSC works, but you can't control which AZ a Fargate node lands in — AWS decides.

🧪 Hands-On Lab Exercises

Lab 35.1 — Zone spread on Kind (Beginner, 30 min)

1. Create Kind cluster with 3 workers. Label each worker with a fake zone: `zone-a`, `zone-b`, `zone-c`.
2. Deploy 6-replica Deployment with TSC `maxSkew: 1, topologyKey: zone, DoNotSchedule`.
3. `kubectl get pods -o wide` → 2 per zone.

Lab 35.2 — ScheduleAnyway fallback (Intermediate, 30 min)

1. Same setup. Scale to 9 replicas but only 2 nodes in `zone-c`.
2. With `DoNotSchedule`: 3rd pod in zone-c Pending.
3. Switch to `ScheduleAnyway`: pod schedules, skew > 1 but cluster runs.

Lab 35.3 — TSC + Karpenter on EKS (Advanced, 60 min)

1. Deploy Karpenter with NodePool across 3 AZs.
2. Deploy Deployment with TSC across zones.
3. Scale up → Karpenter provisions nodes in under-represented AZs.
4. Drain one AZ's nodes → TSC forces rescheduling to remaining AZs.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: TopologySpreadConstraints control how pods distribute across failure domains — zones, nodes, or custom topology keys — using a `maxSkew` parameter that caps the imbalance.

Why: High availability requires spreading replicas across zones. Pod anti-affinity can do this but is binary and expensive. TSC is quantitative — `maxSkew: 1` means the difference between any two zones is at most 1 pod — and more efficient.
How: I stack two constraints: a hard spread across zones (`maxSkew: 1, DoNotSchedule`) for HA, and a soft spread across nodes (`ScheduleAnyway`) for balance within a zone. On EKS, Karpenter reads TSC and provisions nodes in under-represented zones. I pair TSC with PDB so drains and zone failures don't collapse availability.
Example: We ran 12 replicas of an API behind a LoadBalancer. Without TSC, 8 landed in zone-a. When zone-a had a 10-minute outage, we served at 33% capacity. After adding TSC `maxSkew: 1` , pods spread 4-4-4 across zones — the same outage now leaves us at 66% capacity."

? Possible Interview Questions

Q1. What is a TopologySpreadConstraint? Controls pod distribution across topology domains with a max skew.
Q2. `maxSkew: 1` **means what?** The difference in pod count between any two matching topology domains is at most 1.
Q3. TSC vs pod anti-affinity? TSC is quantitative (skew); anti-affinity is binary (co-locate or not). TSC is more performant.
Q4. `DoNotSchedule` **vs** `ScheduleAnyway` **?** Hard vs soft. ScheduleAnyway still tries to minimize skew but won't block the pod.
Q5. What is `minDomains` **?** Minimum number of topology domains required before scheduling begins.
Q6. Can you stack multiple TSCs? Yes — all must be satisfied for the pod to schedule.
Q7. What `topologyKey` **values are common?** `kubernetes.io/hostname` (node), `topology.kubernetes.io/zone` (zone).
Q8. How does Karpenter interact with TSC? It reads pending pod TSC requirements and provisions nodes in under-represented zones/domains.
Q9. What happens to skew during a zone failure? Surviving zones absorb rescheduled pods; skew may temporarily exceed maxSkew if `ScheduleAnyway` .
Q10. Where do you set cluster-wide default TSC? In `KubeSchedulerConfiguration` under `PodTopologySpread.defaultConstraints` .

🗐 Topic 36 — Pod Priority and Preemption

Official Documentation Reference - Pod Priority and Preemption: https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/ - PriorityClass API: https://kubernetes.io/docs/reference/kubernetes-api/scheduling/priority-class-v1/ - GKE priority: https://cloud.google.com/kubernetes-engine/docs/concepts/priority-classes - EKS best practices: https://docs.aws.amazon.com/eks/latest/best-practices/scheduling.html

---

⏥ Definition

Pod Priority assigns an integer priority to pods via a `PriorityClass` . **Preemption** is the scheduler's ability to evict lower-priority pods from a node to make room for a higher-priority pending pod.

⏥ About

PriorityClass object (cluster-scoped)

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: high }
value: 1000000
globalDefault: false
preemptionPolicy: PreemptLowerPriority    # or Never
description: "High priority workloads"
```

Built-in system classes

Class	Value	Purpose
`system-cluster-critical`	2000000000	Cluster-critical add-ons (CoreDNS, kube-proxy)
`system-node-critical`	2000001000	Node-critical daemons (kubelet, CNI)

User-defined priority values must be < 1,000,000,000 (1 billion).

How preemption works

1. Pod `high-prio` can't schedule (no resources).
2. Scheduler runs PostFilter → finds a node where evicting `low-prio` pods frees enough capacity.
3. Respects PDB — won't preempt if it violates a PodDisruptionBudget.
4. Evicts selected pods (graceful termination).
5. Schedules `high-prio` on the freed node.

`preemptionPolicy: Never`

Pod still has high priority in the queue (scheduled before lower-priority pending pods) but does NOT evict running pods. Useful for batch jobs that should go first when capacity frees up but shouldn't kill prod services.

`globalDefault: true`

One PriorityClass can be the default for all pods that don't specify one. At most one can be default.

Priority + Cluster Autoscaler

- CA adds nodes when pods are Pending.
- Priority doesn't change CA's scale-up behavior directly — but higher-priority pods trigger scale-up before lower ones are considered.
- **Overprovisioning pattern**: deploy low-priority "placeholder" pods that waste resources. When a real pod arrives, placeholders are preempted → real pod

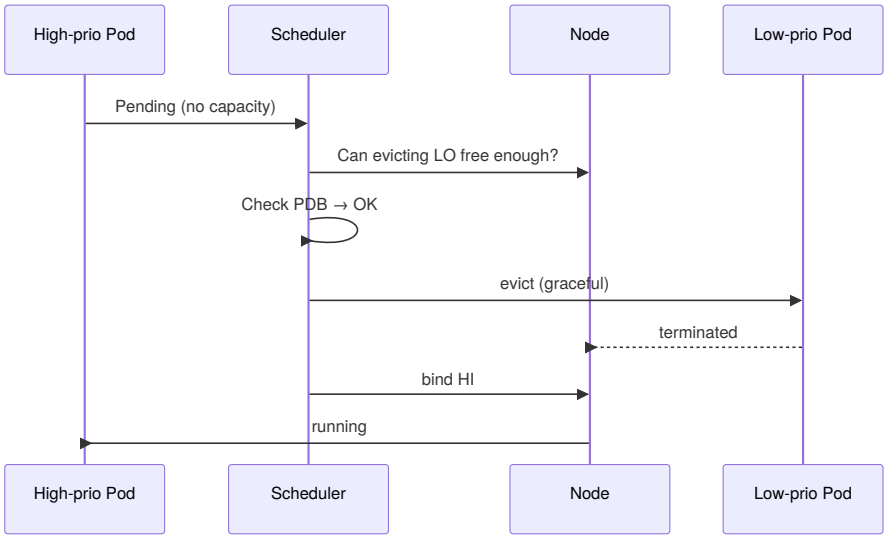
runs immediately while CA adds capacity in the background.

🔌 How It Integrates With Other Kubernetes Components

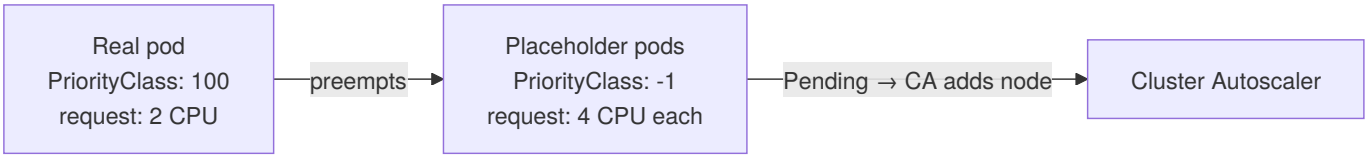
Component	Interaction
Scheduler	Queue ordering + preemption
PDB	Preemption respects PDB; won't violate minAvailable
Cluster Autoscaler	Triggers scale-up for Pending pods by priority order
Karpenter	Reads priority; provisions nodes for highest-priority pending first
Eviction	kubelet eviction order considers priority

🏗 Diagrams

Preemption flow



Overprovisioning pattern



💻 Real-World Examples

Example 1 — PriorityClasses

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: critical }
value: 900000
preemptionPolicy: PreemptLowerPriority
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: batch }
value: 100
preemptionPolicy: Never
globalDefault: false
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: placeholder }
value: -1
preemptionPolicy: Never
```

Example 2 — Use in a Deployment

```
spec:
  template:
    spec:
      priorityClassName: critical
      containers: [{ name: api, image: myrepo/api:2.0 }]
```

Example 3 — Overprovisioning with placeholders

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: overprovisioner }
spec:
  replicas: 5
  selector: { matchLabels: { app: placeholder } }
  template:
    metadata: { labels: { app: placeholder } }
    spec:
      priorityClassName: placeholder
      containers:
      - name: pause
        image: registry.k8s.io/pause:3.9
        resources: { requests: { cpu: 2, memory: 4Gi } }
```

When a real pod with priority > -1 arrives, placeholders are preempted instantly → real pod runs while CA provisions replacement node.

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get priorityclass`	—	List classes	`kubectl get pc`
`kubectl describe priorityclass`	`<name>`	Details	`kubectl describe pc critical`
`kubectl get pods`	`-o custom-columns=NAME:.metadata.name,PRIO:.spec.priority`	Show priority values	
`kubectl get events`	`--field-selector reason=Preempted`	See preemption events	

🚨 Scenarios

Scenario 1 — Batch jobs preempt production

- **Cause**: batch given higher priority than production.
- **Fix**: review PriorityClass values. Production > batch always. Set batch `preemptionPolicy: Never`.

Scenario 2 — Preemption cascade

- **Cause**: high-priority pod preempts mid-priority; mid-priority preempts low — chain reaction.
- **Fix**: limit the number of priority tiers. Use PDB on mid-priority. Add Cluster Autoscaler capacity.

Scenario 3 — Overprovisioner wastes money

- **Cause**: placeholders running 24/7 on expensive nodes.
- **Fix**: right-size placeholder count and requests. Use Spot/preemptible nodes for placeholders.

☁ GCP GKE Perspective

- GKE system pods use `system-cluster-critical` / `system-node-critical`.
- **Autopilot** supports user-defined PriorityClasses.
- **Node Auto-Provisioning** (NAP) considers priority when deciding which node pool to create.

▩ AWS EKS Perspective

- EKS supports PriorityClasses natively.
- **Karpenter** provisions nodes based on highest-priority pending pods first.
- **Spot interruption**: Karpenter handles graceful migration; combine with PDB for zero-downtime.

🧪 Hands-On Lab Exercises

Lab 36.1 — See preemption in action (Beginner, 30 min)

1. Create `low` PC (value 10) and `high` PC (value 1000).
2. Fill a 2-node Kind cluster with `low` pods (exhaust all CPU).
3. Deploy a `high` pod → scheduler evicts a `low` pod → `high` runs.
4. `kubectl get events --field-selector reason=Preempted`.

Lab 36.2 — preemptionPolicy: Never (Intermediate, 30 min)

1. Same setup but `high` PC has `preemptionPolicy: Never`.

2. `high` pod stays Pending — doesn't evict anyone.
3. Delete a `low` pod manually → `high` pod schedules (it was queued first).

Lab 36.3 — Overprovisioning pattern (Advanced, 60 min)

1. Deploy placeholder pods (Example 3) on a 3-node EKS/GKE cluster.
2. Deploy a real app pod with priority > -1 → placeholder preempted → real pod runs in < 5s.
3. CA/Karpenter provisions new node for the now-Pending placeholder.
4. Measure time-to-running for the real pod with and without overprovisioning.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Pod Priority assigns an integer value via a PriorityClass. Preemption is the scheduler's ability to evict lower-priority pods to make room for higher-priority ones — respecting PDBs.
Why: In a shared cluster, production API pods must never wait behind batch jobs. Priority ensures the scheduler queues and places critical pods first, and preemption frees capacity when the cluster is full.
How: I define three tiers: `critical` (900K), `default` (0, globalDefault), `batch` (-100, preemptionPolicy: Never). Batch pods can't preempt production but get leftover capacity. For instant scaling I use the overprovisioning pattern — low-priority pause pods hold capacity that real workloads preempt immediately while the Cluster Autoscaler adds nodes in the background.
Example: On Black Friday, our API pods had priority 900K and batch-ML had priority 100 with `preemptionPolicy: Never`. When traffic spiked, the scheduler preempted non-critical dashboard pods (priority 500) — the API scaled from 10 to 50 replicas in under 30 seconds. Batch pods ran later when nodes scaled up."

? Possible Interview Questions

Q1. What is a PriorityClass? Cluster-scoped object mapping a name to an integer priority value.
Q2. How does preemption work? Scheduler evicts lower-priority pods to free resources for a higher-priority Pending pod, respecting PDB.
Q3. What is `preemptionPolicy: Never`? Pod gets queue priority (scheduled before lower) but does NOT evict running pods.
Q4. Built-in critical classes? `system-cluster-critical` (2B), `system-node-critical` (2B+1K).
Q5. What is the overprovisioning pattern? Low-priority placeholder pods hold capacity. Real pods preempt them instantly; CA replaces the placeholder's capacity in the background.
Q6. Does preemption respect PDB? Yes — won't violate minAvailable / maxUnavailable.
Q7. Can preemption cascade? Yes — careful with many priority tiers. Keep tiers minimal.
Q8. `globalDefault: true`? That PriorityClass is used when a pod doesn't specify one.
Q9. Priority affects kubelet eviction? Yes — under memory pressure, kubelet evicts BestEffort first, then by priority within QoS.
Q10. How does Karpenter use priority? Provisions nodes for highest-priority pending pods first; may deprioritize lower-priority pods for Spot interruption handling.

📄 Topic 37 — Custom Schedulers

Official Documentation Reference - Configure multiple schedulers: https://kubernetes.io/docs/tasks/extend-kubernetes/configure-multiple-schedulers/ - Scheduling framework: https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/ - Scheduler profiles: https://kubernetes.io/docs/reference/scheduling/config/ - GKE scheduling: https://cloud.google.com/kubernetes-engine/docs/concepts/scheduling - EKS scheduling: https://docs.aws.amazon.com/eks/latest/best-practices/scheduling.html

---

🔷 Definition

A **custom scheduler** is a separate scheduler binary (or profile within the default scheduler) that implements scheduling logic different from the built-in defaults. Pods select their scheduler via `spec.schedulerName`.

🔷 About

Why use a custom scheduler?

- **Specialized workloads**: gang scheduling (all-or-nothing for distributed ML), co-scheduling, GPU bin-packing.
- **Custom scoring**: prefer nodes with specific hardware, data locality, network topology.
- **Multi-tenant fairness**: weighted fair queuing across teams.
- **Research/experimentation**: prototype new scheduling algorithms.

Two approaches

Approach	How
Scheduling profiles	Configure multiple profiles in the same `kube-scheduler` binary with different plugin sets. No extra Deployment.
Separate scheduler Deployment	Deploy a second `kube-scheduler` binary (or a Go binary implementing the scheduler framework) as a Deployment in `kube-system`.

Writing a custom scheduler

1. **Simplest**: fork `kube-scheduler`, change plugin weights or add a custom Score plugin.
2. **Scheduler framework**: write a Go plugin implementing one or more extension points (Filter, Score, Permit, etc.).
3. **Webhook-based extender** (legacy): configure the default scheduler to call an HTTP endpoint for filter/score. Slower, deprecated in favor of framework plugins.

Notable open-source custom schedulers

Scheduler	Purpose
Volcano	Batch / HPC / gang scheduling (CNCF Sandbox)
Koordinator	Colocation / QoS-aware scheduling (Alibaba)
Yunikorn	Multi-tenant resource queuing (Apache, used by Databricks on K8s)
Scheduler Plugins (sigs)	Community plugins: Coscheduling, Trimaran (load-aware), CapacityScheduling

Deploying a second scheduler

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: my-scheduler, namespace: kube-system }
spec:
  replicas: 1
  selector: { matchLabels: { component: my-scheduler } }
  template:
    metadata: { labels: { component: my-scheduler } }
    spec:
      serviceAccountName: my-scheduler
      containers:
      - name: scheduler
        image: registry.k8s.io/kube-scheduler:v1.31.0
        command:
        - kube-scheduler
        - --config=/etc/kubernetes/scheduler-config.yaml
        - --leader-elect=false
        volumeMounts:
        - { name: config, mountPath: /etc/kubernetes/scheduler-config.yaml, subPath: scheduler-config.yaml }
      volumes:
      - name: config
        configMap: { name: my-scheduler-config }
```

Pod selecting the custom scheduler

```
spec:
  schedulerName: my-scheduler
  containers: [{ name: app, image: myrepo/app:1 }]
```

If `schedulerName` doesn't match any running scheduler, the pod stays Pending indefinitely.

RBAC for a custom scheduler

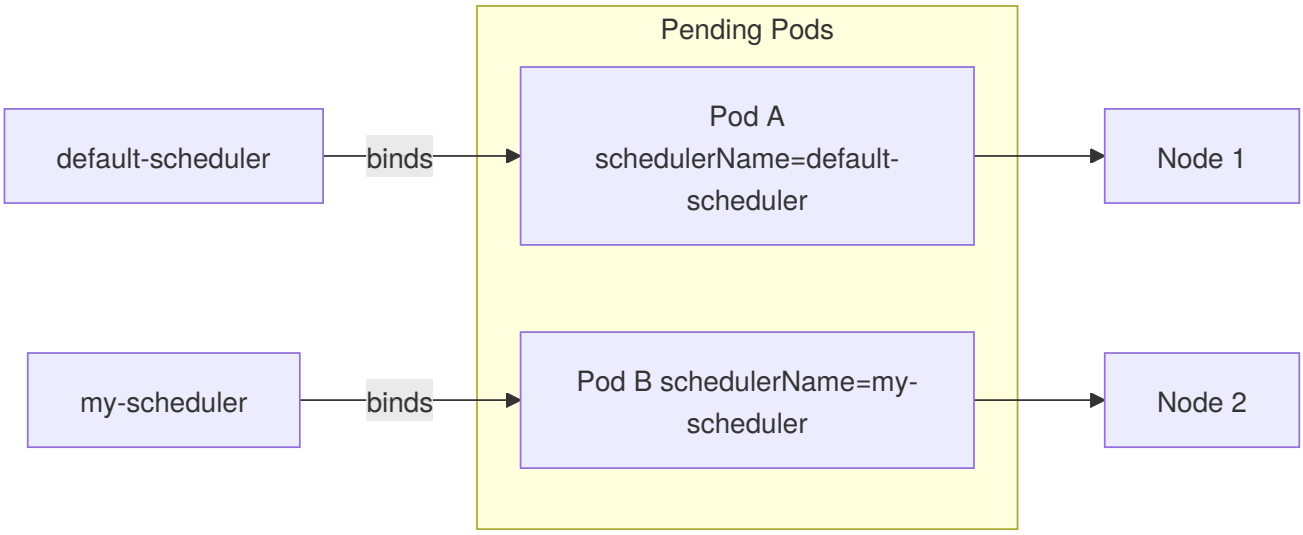
The scheduler SA needs permissions to watch/patch Pods, Nodes, PVs, PVCs, Events, Leases, and more. Use the `system:kube-scheduler` ClusterRole as a starting point.

🔷 How It Integrates With Other Kubernetes Components

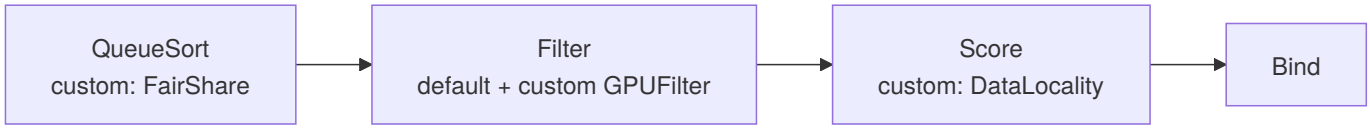
Component	Interaction
API server	Custom scheduler watches + binds pods like the default
Default scheduler	Ignores pods with a different `schedulerName`
Cluster Autoscaler	Only aware of the default scheduler's Pending pods by default; configure CA to consider other schedulers
Karpenter	Works with any scheduler — reacts to Pending pods regardless of `schedulerName`
PDB	Respected by any scheduler's preemption logic

🏗 Diagrams

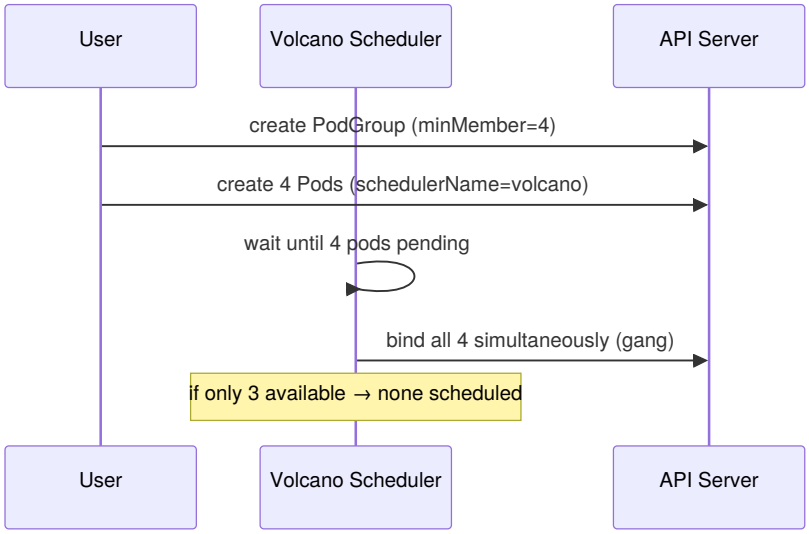
Two schedulers side by side



Scheduling framework plugin example



Volcano gang scheduling



💻 Real-World Examples

Example 1 — Scheduling profile with different scoring

```
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
- schedulerName: default-scheduler
  plugins:
    score:
      enabled: [{ name: NodeResourcesFit, weight: 1 }]
  pluginConfig:
  - name: NodeResourcesFit
    args: { scoringStrategy: { type: LeastAllocated } }
- schedulerName: pack-scheduler
  plugins:
    score:
      enabled: [{ name: NodeResourcesFit, weight: 1 }]
  pluginConfig:
  - name: NodeResourcesFit
    args: { scoringStrategy: { type: MostAllocated } }
```

One binary, two profiles: `default-scheduler` spreads; `pack-scheduler` packs.

Example 2 — Install Volcano for gang scheduling

```
kubectl apply -f https://raw.githubusercontent.com/volcano-sh/volcano/master/installer/volcano-development.yaml
kubectl get pods -n volcano-system
```

```
apiVersion: batch.volcano.sh/v1alpha1
kind: Job
metadata: { name: gang-job }
spec:
  schedulerName: volcano
  minAvailable: 4
  tasks:
  - replicas: 4
    name: worker
    template:
      spec:
        containers: [{ name: w, image: myrepo/worker:1 }]
```

Example 3 — Pod with custom scheduler

```
apiVersion: v1
kind: Pod
metadata: { name: packed }
spec:
  schedulerName: pack-scheduler
  containers: [{ name: c, image: nginx:1.27 }]
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get pods -A`	`-o custom-columns=NAME:.metadata.name,SCHEDULER:.spec.schedulerName`	See which scheduler per pod	
`kubectl logs -n kube-system`	`<my-scheduler-pod>`	Custom scheduler logs	
`kubectl get leases -n kube-system`	—	Check leader election	
`kubectl describe pod`	—	See scheduling events from custom scheduler	

🚨 Scenarios

Scenario 1 — Pod Pending: no scheduler matches `schedulerName`

- **Cause**: custom scheduler not deployed; or name typo.
- **Fix**: deploy the scheduler Deployment; verify it's running; check `schedulerName` spelling.

Scenario 2 — Two schedulers race for the same pod

- **Cause**: both default and custom scheduler configured to handle same `schedulerName`.
- **Fix**: each scheduler must have a unique `schedulerName`. Disable leader-elect on custom scheduler if only one replica.

Scenario 3 — Gang scheduling: not enough nodes for full gang

- **Cause**: Volcano waits for `minAvailable` pods to be schedulable simultaneously.
- **Fix**: add nodes or reduce gang size. Karpenter can help by provisioning right-sized nodes.

Scenario 4 — Cluster Autoscaler ignores custom-scheduler pods

- **Cause**: CA only watches pods scheduled by `default-scheduler` by default.
- **Fix**: configure CA `--expandable-pod-groups` or use Karpenter (scheduler-agnostic).

☁ GCP GKE Perspective

- GKE does NOT expose the managed scheduler's config. To use custom scheduling, deploy a second scheduler as a Deployment.
- **Autopilot**: custom schedulers are NOT supported (Google controls scheduling).
- **Standard**: full support for additional schedulers.
- **Batch on GKE**: Google's batch API handles gang-like semantics without a custom scheduler.

▩ AWS EKS Perspective

- EKS managed scheduler cannot be customized. Deploy a custom scheduler Deployment.
- **Volcano on EKS** is a common choice for ML/HPC batch workloads.
- **Karpenter** is scheduler-agnostic — provisions nodes for any Pending pod regardless of `schedulerName`.
- **AWS Batch on EKS**: managed queue + priority without a custom scheduler.

🧪 Hands-On Lab Exercises

Lab 37.1 — Two scheduling profiles (Beginner, 45 min)

1. On a kubeadm or Kind cluster, modify scheduler config to add a `pack-scheduler` profile (Example 1).
2. Deploy pods with `schedulerName: pack-scheduler` → observe packing.
3. Deploy pods with `schedulerName: default-scheduler` → observe spreading.
4. Compare with `kubectl get pods -o wide`.

Lab 37.2 — Deploy a separate custom scheduler (Intermediate, 60 min)

1. Create ServiceAccount + ClusterRoleBinding (clone `system:kube-scheduler`).
2. Deploy second `kube-scheduler` as a Deployment with custom config (Example above).
3. Create pods with `schedulerName: my-scheduler`.
4. `kubectl describe pod` → events from `my-scheduler`.

Lab 37.3 — Volcano gang scheduling (Advanced, 90 min)

1. Install Volcano (Example 2).
2. Create a Volcano Job with `minAvailable: 4` and 4 worker tasks.
3. Only 3 nodes available → all 4 pods stay Pending (gang semantics).
4. Add a 4th node → all 4 schedule simultaneously.
5. Kill one worker → Volcano reschedules the full gang.

Cleanup: `kubectl delete -f volcano.yaml`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A custom scheduler is an alternative to the default kube-scheduler — either a separate profile within the same binary or a standalone Deployment. Pods opt in via `spec.schedulerName`.
Why: The default scheduler is great for general workloads but doesn't cover specialized needs like gang scheduling (all-or-nothing for distributed training), data-locality-aware scoring, or multi-tenant fair queuing.
How: For simple customization I use scheduling profiles — one default-scheduler profile with LeastAllocated and one pack-scheduler profile with MostAllocated, both in the same binary. For advanced use cases like ML batch I deploy Volcano alongside the default scheduler. On managed clusters (GKE Standard, EKS) I deploy the custom scheduler as a Deployment in kube-system with its own RBAC.
Example: Our ML team needed 8-GPU gang scheduling — all 8 pods had to start simultaneously or not at all. The default scheduler placed them one by one, causing partial starts and wasted GPU time. Volcano's PodGroup with `minAvailable: 8` solved it — either all 8 start or none do, and training jobs became 40% more efficient."

? Possible Interview Questions

Q1. How does a pod select its scheduler? `spec.schedulerName` field.
Q2. What happens if `schedulerName` doesn't match any running scheduler? Pod stays Pending indefinitely.
Q3. Scheduling profiles vs separate scheduler? Profiles run in one binary with different configs. Separate scheduler is a full Deployment.
Q4. What is gang scheduling? All-or-nothing: all pods in a group must be schedulable simultaneously or none are placed.
Q5. Name a gang scheduler. Volcano (CNCF Sandbox).
Q6. Can you run a custom scheduler on GKE Autopilot? No — Autopilot controls scheduling.
Q7. What RBAC does a custom scheduler need? Watch/patch Pods, Nodes, PVs, PVCs, Events, Leases — similar to `system:kube-scheduler`.
Q8. What is a scheduler extender? Legacy webhook approach: default scheduler calls HTTP endpoint for filter/score. Deprecated in favor of framework plugins.
Q9. Does Cluster Autoscaler work with custom schedulers? By default CA only watches `default-scheduler` pods. Karpenter is scheduler-agnostic.
Q10. When would you NOT use a custom scheduler? When the default scheduler + affinity/taints/TSC covers the need. Custom schedulers add operational complexity.

MODULE

Module 7 — Security

🗐 Topic 38 — Authentication and Authorization

Official Documentation Reference - Controlling access: https://kubernetes.io/docs/concepts/security/controlling-access/ - Authenticating: https://kubernetes.io/docs/reference/access-authn-authz/authentication/ - Authorization: https://kubernetes.io/docs/reference/access-authn-authz/authorization/ - GKE access control: https://cloud.google.com/kubernetes-engine/docs/concepts/access-control - EKS access: https://docs.aws.amazon.com/eks/latest/userguide/cluster-auth.html

🔹 Definition

Authentication (AuthN) answers "Who are you?" — verifying the identity of a request. **Authorization (AuthZ)** answers "Are you allowed to do this?" — deciding if the authenticated identity can perform the requested action on the requested resource.

🔹 About

Request flow through the API server

```
Client → TLS → Authentication → Authorization → Admission → Validation → etcd
```

Authentication methods (AuthN)

Method	How	Used by
Client certificates (X.509)	CN=user, O=group in cert signed by cluster CA	kubeadm default
Bearer tokens	Static token file / ServiceAccount token	SAs, legacy users
OIDC	External IdP (Google, Okta, Dex) issues JWT	Enterprise SSO
Exec plugins	kubectl calls external binary for creds	GKE (`gke-gcloud-auth-plugin`), EKS (`aws eks get-token`)
Webhook token	API server calls external webhook for validation	Custom identity providers
Bootstrap tokens	Short-lived tokens for cluster bootstrap	`kubeadm join`
Anonymous	Unauthenticated — maps to `system:anonymous`	Usually disabled or tightly scoped

Key fact: Kubernetes has **no user database**. Users are external (certs, IdP). Only ServiceAccounts are native K8s objects.

Authorization modes (AuthZ)

Mode	How	When
RBAC	Role + RoleBinding (namespace) / ClusterRole + ClusterRoleBinding (cluster)	Default on all modern clusters
ABAC	JSON policy file on API server	Legacy, not recommended
Node	Authorizes kubelet requests	Always enabled alongside RBAC
Webhook	External HTTP service decides allow/deny	Custom policies, OPA
AlwaysAllow / AlwaysDeny	Testing only	Never in prod

Multiple modes run in chain: first "allow" or "deny" wins (except deny-only modes that just skip).

Audit logging

API server records every request to an audit log (or webhook sink). Configure with `--audit-policy-file` and `--audit-log-path`.

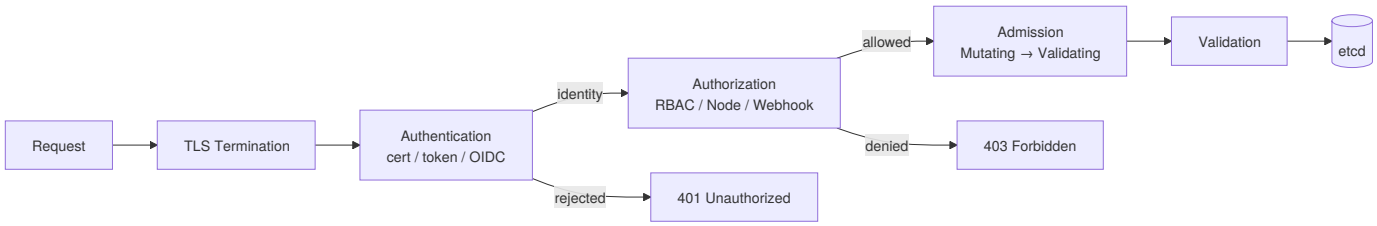
Audit levels: `None`, `Metadata`, `Request`, `RequestResponse`.

🔹 How It Integrates With Other Kubernetes Components

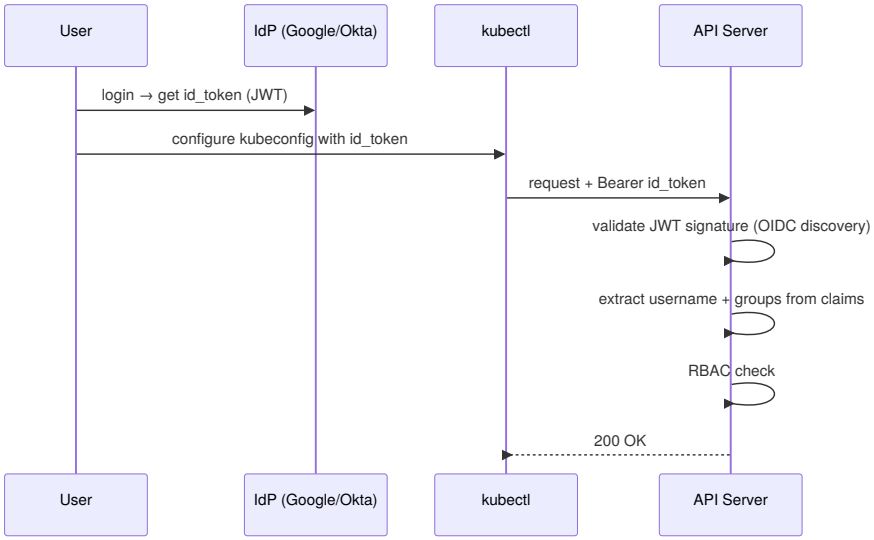
Component	Interaction
kubectl	Sends creds from kubeconfig (cert, token, exec plugin)
ServiceAccount	Auto-mounted projected token used for in-cluster auth
Admission controllers	Run after AuthZ — can mutate or reject
kubelet	Authenticates via Node auth mode
Cloud IAM (GCP/AWS)	Maps cloud identity to K8s identity via OIDC/exec plugin
RBAC objects	Stored in etcd; evaluated by API server

🏗 Diagrams

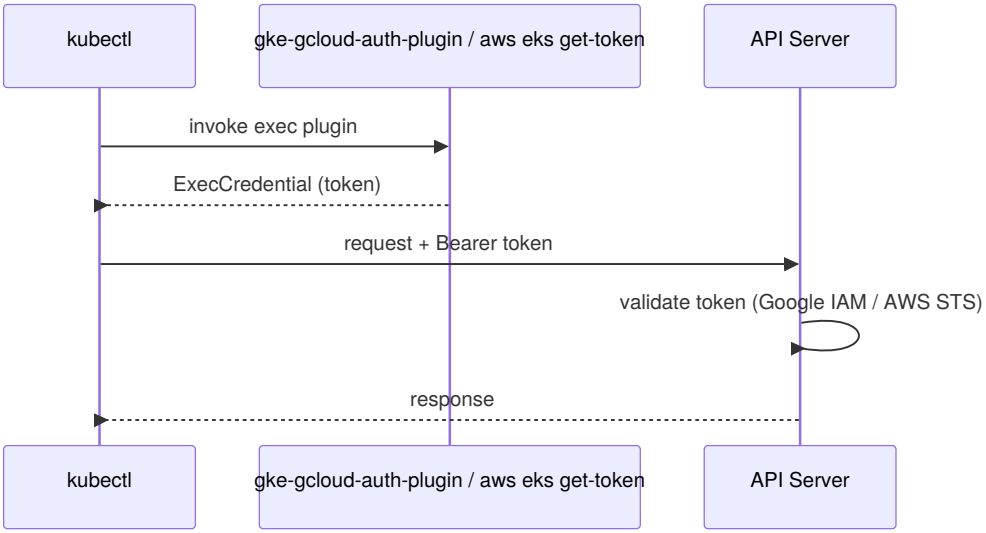
API request lifecycle



OIDC flow



GKE / EKS exec plugin flow



💻 Real-World Examples

Example 1 — Check current identity

```
kubectl auth whoami                    # K8s 1.27+ stable
kubectl config view --minify -o jsonpath='{.users[0].user}'
```

Example 2 — Test authorization

```
kubectl auth can-i create deployments               # yes/no
kubectl auth can-i delete pods -n prod              # namespace-scoped
kubectl auth can-i '*' '*'                          # cluster-admin check
kubectl auth can-i list pods --as=system:serviceaccount:default:app-sa   # impersonate
```

Example 3 — OIDC setup (Dex example)

```
# apiserver flags
--oidc-issuer-url=https://dex.example.com
--oidc-client-id=kubernetes
--oidc-username-claim=email
--oidc-groups-claim=groups
```

Example 4 — Audit policy

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  resources: [{ group: "", resources: ["secrets"] }]
  verbs: ["get","list","watch"]
- level: RequestResponse
  resources: [{ group: "", resources: ["pods"] }]
  verbs: ["create","delete"]
- level: None
  resources: [{ group: "", resources: ["events"] }]
```

Example 5 — EKS Access Entries (2024+)

```
aws eks create-access-entry --cluster-name prod \
  --principal-arn arn:aws:iam::123:user/alice
aws eks associate-access-policy \
  --cluster-name prod \
  --principal-arn arn:aws:iam::123:user/alice \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSViewPolicy \
  --access-scope type=namespace,namespaces=dev
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl auth whoami`	—	Show authenticated identity	`kubectl auth whoami`
`kubectl auth can-i`	`<verb> <resource>`	Check permission	`kubectl auth can-i create pods`
`kubectl auth can-i`	`--as=<user>`	Impersonate	`kubectl auth can-i get secrets --as=alice`
`kubectl auth can-i`	`--list`	List all permissions	`kubectl auth can-i --list -n prod`
`kubectl config view`	`--minify`	Current context details	`kubectl config view --minify`
`kubectl get clusterrolebinding`	—	List bindings	`kubectl get clusterrolebinding`
`kubectl create clusterrolebinding`	`--clusterrole= --user=`	Bind role	`kubectl create clusterrolebinding alice-admin --clusterrole=admin --user=alice`

🚨 Scenarios

Scenario 1 — 403 Forbidden on a command that worked yesterday

- **Cause**: RBAC binding removed; or token expired; or IAM role changed.
- **Fix**: `kubectl auth can-i <verb> <resource> --as=<identity>`. Check RoleBinding. On GKE/EKS verify cloud IAM role.

Scenario 2 — New team member can't access cluster

- **Cause**: no kubeconfig; or no RBAC binding; or OIDC not configured.
- **Fix**: generate kubeconfig (`gcloud get-credentials` / `aws eks update-kubeconfig`); create RoleBinding.

Scenario 3 — ServiceAccount token leaked in logs

- **Cause**: app printed auth header.
- **Fix**: rotate SA token (`kubectl delete secret` ; or use bound projected tokens that expire). Restrict SA RBAC to minimum.

Scenario 4 — Audit log shows unknown identity accessing secrets

- **Cause**: potential credential compromise.
- **Fix**: identify the identity, revoke certs/tokens, rotate secrets, tighten RBAC.

☁ GCP GKE Perspective

- **AuthN**: `gke-gcloud-auth-plugin` → Google OAuth token. Identity = Google Cloud IAM user/SA.
- **AuthZ**: GKE layers **IAM** (at the GKE API level: `container.clusters.get`) on top of **RBAC** (inside the cluster: `get pods`). Both must allow.
- **Workload Identity**: binds KSA → GSA for pod-level GCP access without keys.
- **GKE RBAC audit**: `gcloud logging read 'resource.type="k8s_cluster"'` .

▩ AWS EKS Perspective

- **AuthN**: `aws eks get-token` → STS token. Identity = IAM user/role/SSO.
- **AuthZ**: Legacy `aws-auth` ConfigMap maps IAM → K8s users/groups. New: **EKS Access Entries** (API-based, no ConfigMap).
- **IRSA / Pod Identity**: pod-level IAM role binding for SA.
- **Control-plane logging**: enable `api`, `audit`, `authenticator` → CloudWatch Logs.

🧪 Hands-On Lab Exercises

Lab 38.1 — AuthN + AuthZ basics (Beginner, 30 min)

```
kubectl auth whoami
kubectl auth can-i create pods
kubectl auth can-i create pods --as=system:serviceaccount:default:default
kubectl auth can-i --list -n default
```

Lab 38.2 — Create a user with limited access (Intermediate, 60 min)

1. Generate client cert: `openssl genrsa -out alice.key 2048 && openssl req -new -key alice.key -out alice.csr -subj "/CN=alice/O=dev-team"`.
2. Create CertificateSigningRequest in K8s.
3. Approve: `kubectl certificate approve alice`.
4. Create kubeconfig entry for alice.
5. Create Role + RoleBinding in `dev` namespace with only `get,list pods`.
6. Test: `kubectl --context=alice get pods -n dev` → works. `kubectl --context=alice delete pod -n dev` → forbidden.

Lab 38.3 — EKS Access Entries (Advanced, 45 min)

1. Create IAM user `bob`.
2. `aws eks create-access-entry` with ViewPolicy scoped to `dev` namespace.
3. `aws eks update-kubeconfig --name prod` as bob.
4. Verify bob can list pods in `dev` but not `prod`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Authentication identifies who you are — via client certs, OIDC tokens, ServiceAccount tokens, or cloud exec plugins. Authorization decides if you can do what you're asking — primarily RBAC in production.
Why: Zero-trust requires both. Authentication without authorization is an open door. Authorization without authentication is meaningless — you don't know who to authorize.
How: On GKE I rely on Google IAM for cluster-level access and RBAC for namespace-level. On EKS I use Access Entries (replacing the aws-auth ConfigMap) to map IAM identities to K8s groups, then bind groups to Roles. For pod-level access to cloud APIs, I use Workload Identity on GKE and IRSA/Pod Identity on EKS — no static keys. I always enable audit logging so every secret access and privilege escalation attempt is traceable.
Example: After a security audit flagged 12 users with cluster-admin, we migrated to group-based RBAC with namespaced Roles. Each team got a RoleBinding to `edit` in their namespace only. Cluster-admin was restricted to 2 break-glass identities with MFA."

❓ Possible Interview Questions

Q1. AuthN vs AuthZ? AuthN = who are you. AuthZ = are you allowed.
Q2. Does K8s have a user database? No — users are external (certs, IdP). Only ServiceAccounts are native.
Q3. What is RBAC? Role-Based Access Control — Roles define permissions, RoleBindings assign them.
Q4. Name 3 auth methods. Client certs, OIDC, ServiceAccount tokens.
Q5. How does GKE authenticate kubectl? `gke-gcloud-auth-plugin` fetches a Google OAuth token.
Q6. What replaced `aws-auth` ConfigMap on EKS? Access Entries (API-based, no ConfigMap editing).
Q7. What is `kubectl auth can-i --as=`? Impersonation — test permissions for another identity.
Q8. What are audit levels? None, Metadata, Request, RequestResponse.
Q9. What does Node authorization mode do? Restricts kubelet to only access resources bound to its node.
Q10. How do you revoke a compromised SA token? Delete the secret (legacy) or rely on bound projected tokens that expire. Recreate the SA.

📘 Topic 39 — RBAC (Roles, ClusterRoles, RoleBindings, ClusterRoleBindings)

Official Documentation Reference - RBAC: https://kubernetes.io/docs/reference/access-authn-authz/rbac/ - RBAC good practices: https://kubernetes.io/docs/concepts/security/rbac-good-practices/ - GKE RBAC: https://cloud.google.com/kubernetes-engine/docs/how-to/role-based-access-control - EKS RBAC: https://docs.aws.amazon.com/eks/latest/userguide/security_iam_id-based-policy-examples.html

---

🔹 Definition

RBAC (Role-Based Access Control) is the authorization mode that grants permissions to identities (users, groups, ServiceAccounts) by binding them to Roles or ClusterRoles that define allowed API verbs on specific resources.

🔹 About

Four objects

Object	Scope	Purpose
Role	Namespace	Defines permissions in one namespace
ClusterRole	Cluster	Defines permissions cluster-wide OR reusable across namespaces
RoleBinding	Namespace	Binds a Role/ClusterRole to subjects in one namespace
ClusterRoleBinding	Cluster	Binds a ClusterRole to subjects cluster-wide

Permission model: allow-only

RBAC is additive — you can only grant, never deny. Multiple bindings union their permissions.

Verbs

`get`, `list`, `watch`, `create`, `update`, `patch`, `delete`, `deletecollection`; wildcard `*` = all.

Resource scoping

```
rules:
- apiGroups: [""]                  # core group
  resources: ["pods", "pods/log"]   # sub-resource
  verbs: ["get", "list"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["*"]
  resourceNames: ["web"]            # optional — specific object
```

Built-in ClusterRoles

ClusterRole	Permissions
`cluster-admin`	Everything (superuser)
`admin`	Full namespace access (no quota/namespace changes)
`edit`	Read/write pods, deployments, configmaps, secrets — no RBAC
`view`	Read-only (no secrets)

Aggregated ClusterRoles

```
aggregationRule:
  clusterRoleSelectors:
  - matchLabels: { rbac.example.com/aggregate-to-monitoring: "true" }
```

Any ClusterRole with that label auto-merges into this one. Used by built-in `admin`, `edit`, `view` (CRD authors add rules via label).

Subjects

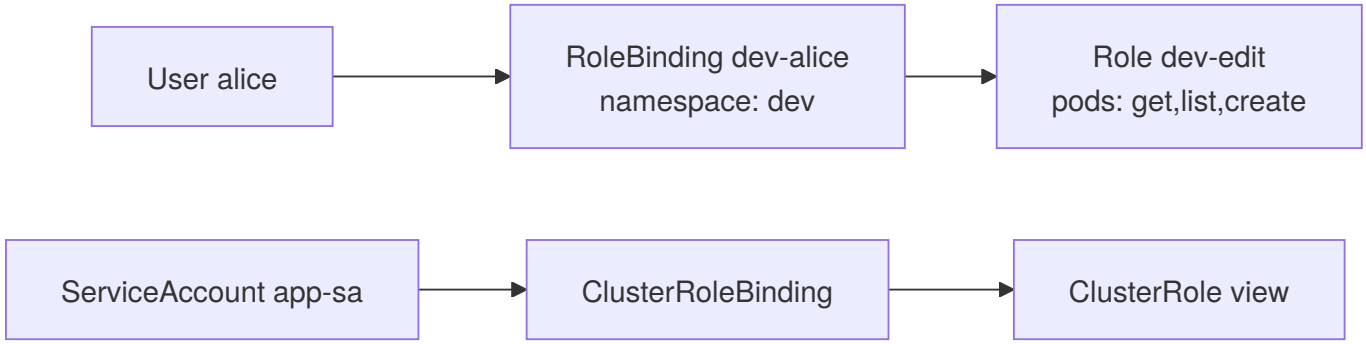
```
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: dev-team
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: app-sa
  namespace: prod
```

🔷 How It Integrates With Other Kubernetes Components

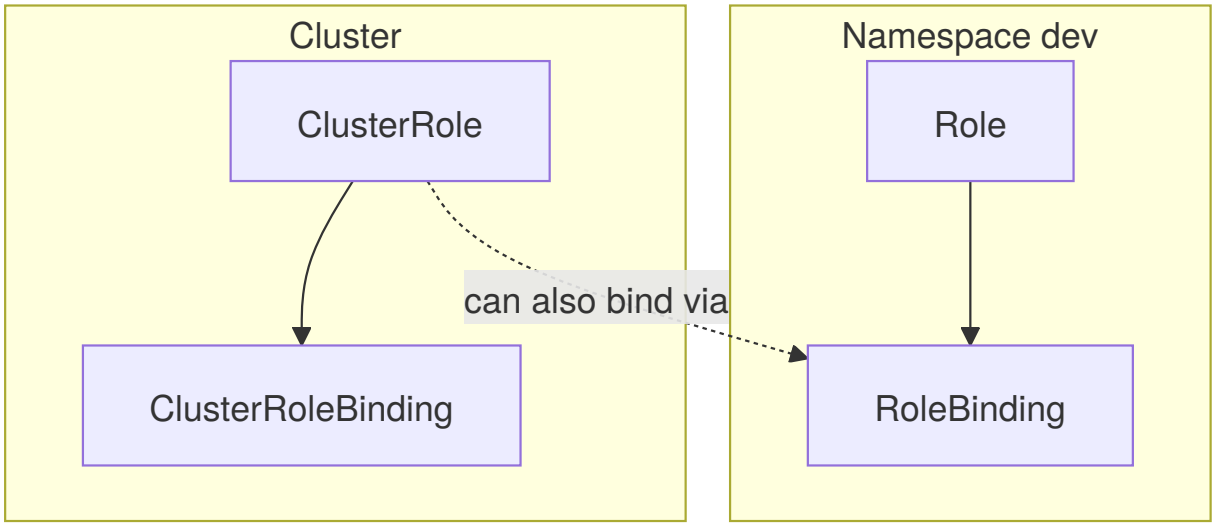
Component	Interaction
API server	Evaluates RBAC rules on every request
ServiceAccount	Bound via RoleBinding/ClusterRoleBinding
Admission (OPA/Kyverno)	Runs after RBAC allows — can still deny
kubectl	`kubectl auth can-i` tests RBAC
Helm	Needs RBAC to install CRDs, Deployments
Operators	Need ClusterRole for CRD management

🏗 Diagrams

RBAC binding model



Namespace vs cluster scope



💻 Real-World Examples

Example 1 — Role + RoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: alice-pod-reader, namespace: dev }
subjects:
- { kind: User, name: alice, apiGroup: rbac.authorization.k8s.io }
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

Example 2 — ClusterRole bound to namespace

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: dev-edit, namespace: dev }
subjects:
- { kind: Group, name: dev-team, apiGroup: rbac.authorization.k8s.io }
roleRef: { kind: ClusterRole, name: edit, apiGroup: rbac.authorization.k8s.io }
```

Reuses built-in `edit` ClusterRole but scoped to `dev` namespace only.

Example 3 — ServiceAccount ClusterRoleBinding

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata: { name: prometheus-reader }
subjects:
- { kind: ServiceAccount, name: prometheus, namespace: monitoring }
roleRef: { kind: ClusterRole, name: view, apiGroup: rbac.authorization.k8s.io }
```

Example 4 — Imperative RBAC commands

```
kubectl create role pod-reader --verb=get,list --resource=pods -n dev
kubectl create rolebinding alice-reader --role=pod-reader --user=alice -n dev
kubectl create clusterrolebinding admin-alice --clusterrole=cluster-admin --user=alice
kubectl auth can-i list pods -n dev --as=alice     # yes
kubectl auth can-i delete pods -n dev --as=alice   # no
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get roles`	`-n <ns>`	List Roles	`kubectl get roles -n dev`
`kubectl get clusterroles`	—	List ClusterRoles	`kubectl get clusterroles`
`kubectl get rolebindings`	`-n <ns>`	List bindings	`kubectl get rolebindings -n dev`
`kubectl describe role`	`<name>`	See rules	`kubectl describe role pod-reader -n dev`
`kubectl create role`	`--verb= --resource=`	Create	`kubectl create role r --verb=get --resource=pods -n dev`
`kubectl create rolebinding`	`--role= --user=`	Bind	`kubectl create rolebinding rb --role=r --user=alice -n dev`
`kubectl create clusterrolebinding`	`--clusterrole= --group=`	Cluster bind	`kubectl create clusterrolebinding cb --clusterrole=view --group=viewers`
`kubectl auth can-i`	`--as= --list`	Test	`kubectl auth can-i --list --as=alice -n dev`
`kubectl auth reconcile`	`-f rbac.yaml`	Apply without conflict	`kubectl auth reconcile -f rbac.yaml`

🚨 Scenarios

Scenario 1 — Developer can read but not create deployments

- **Cause**: bound to `view` not `edit`.
- **Fix**: bind to `edit` or create custom Role with `deployments: create,update,patch`.

Scenario 2 — Helm install fails with RBAC errors

- **Cause**: SA used by Helm/Tiller/CI lacks permissions for CRDs or cluster-scoped resources.
- **Fix**: create ClusterRoleBinding for the CI SA with appropriate ClusterRole.

Scenario 3 — Privilege escalation: user creates RoleBinding granting themselves cluster-admin

- **Cause**: user has `create rolebindings` permission without `escalate` verb or `bind` restriction.
- **Fix**: K8s prevents escalation by default — you can only bind a role if you already have all its permissions. But review carefully.

Scenario 4 — Too many ClusterRoleBindings → hard to audit

- **Fix**: adopt group-based bindings (bind groups, not individual users). Use `kubectl auth can-i --list --as=` for audit. Tools: `rbac-lookup`, `rakkess`.

☁ GCP GKE Perspective

- GKE layers IAM (cluster-level: `roles/container.developer`) + RBAC (namespace-level). Both must allow.
- `gcloud projects add-iam-policy-binding` gives cluster access; RoleBinding gives namespace access.
- **Autopilot** pre-creates restrictive RBAC; some operations blocked.

▩ AWS EKS Perspective

- Legacy: `aws-auth` ConfigMap maps IAM users/roles → K8s groups → RBAC.
- **Access Entries** (2024+): `aws eks create-access-entry` + `associate-access-policy` replaces ConfigMap.
- Built-in EKS access policies: `AmazonEKSClusterAdminPolicy`, `AmazonEKSAdminPolicy`, `AmazonEKSEditPolicy`, `AmazonEKSViewPolicy`.

🧪 Hands-On Lab Exercises

Lab 39.1 — Create Role + RoleBinding (Beginner, 30 min)

Create a `pod-reader` Role in `dev` namespace. Bind to user alice. Test with `kubectl auth can-i`.

Lab 39.2 — Least-privilege SA for an app (Intermediate, 45 min)

1. Create SA `app-sa` in `prod`.
2. Create Role allowing only `get,list configmaps` and `get secrets` (specific name).
3. Bind. Deploy pod with SA. Exec and test with `curl` to API server.

Lab 39.3 — Audit RBAC with rbac-lookup (Advanced, 45 min)

1. Install `rbac-lookup`: `brew install rbac-lookup`.
2. `rbac-lookup alice` → see all bindings.
3. `rbac-lookup --kind=serviceaccount` → audit SA permissions.
4. Identify overly permissive bindings; remediate.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: RBAC is Kubernetes' primary authorization mode. It uses four objects: Roles and ClusterRoles define permissions — API groups, resources, verbs — and RoleBindings and ClusterRoleBindings assign those permissions to users, groups, or ServiceAccounts.
Why: Least-privilege is a security fundamental. RBAC lets me give a developer `edit` in their namespace without touching production, give a monitoring SA read-only cluster-wide, and give CI/CD exactly `create deployments` in staging.
How: I standardize on group-based bindings — map OIDC/IAM groups to K8s groups, then bind groups to the built-in `view`, `edit`, or `admin` ClusterRoles at the namespace level. I never give `cluster-admin` to individuals — only to break-glass accounts with MFA. For ServiceAccounts I create custom Roles with the minimum verbs on the minimum resources.
Example: After a security review found 15 users with `cluster-admin`, we migrated to 4 groups: `platform-admins` (admin in all NS), `dev-team-X` (edit in their NS), `readonly` (view), and `ci-deploy` (custom Role: create/update deployments + secrets). Audit with `rbac-lookup` confirmed no privilege creep."

? Possible Interview Questions

Q1. Four RBAC objects? Role, ClusterRole, RoleBinding, ClusterRoleBinding.
Q2. Role vs ClusterRole? Role = namespaced. ClusterRole = cluster-wide or reusable.
Q3. Can a RoleBinding reference a ClusterRole? Yes — scopes it to the binding's namespace.
Q4. RBAC is allow-only — meaning? No deny rules. Multiple bindings union permissions.
Q5. How do you test RBAC? `kubectl auth can-i <verb> <resource> --as=<identity>`.
Q6. What is aggregation? ClusterRoles with matching labels auto-merge into an aggregated ClusterRole.
Q7. What are sub-resources? `pods/log`, `pods/exec`, `deployments/scale` — scoped verbs.
Q8. How do you prevent privilege escalation? K8s prevents binding a Role you don't already have permissions for. Also restrict `escalate` and `bind` verbs.
Q9. GKE: IAM vs RBAC? IAM gates cluster-level access. RBAC gates resource-level access inside the cluster. Both must allow.
Q10. EKS: aws-auth vs Access Entries? aws-auth is legacy ConfigMap. Access Entries is the new API-based mapping of IAM → K8s RBAC.

🗐 Topic 40 — Service Accounts

Official Documentation Reference - Service Accounts: https://kubernetes.io/docs/concepts/security/service-accounts/ - Configure SA for pods: https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/ - Token request API: https://kubernetes.io/docs/reference/kubernetes-api/authentication-resources/token-request-v1/ - GKE Workload Identity: https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity - EKS IRSA: https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html

---

🔹 Definition

A **ServiceAccount (SA)** is a namespaced Kubernetes identity for pods and system components. Every pod runs as a ServiceAccount, which determines its API permissions (via RBAC) and, on managed clouds, its cloud IAM identity (via Workload Identity / IRSA).

🔹 About

Default SA

Every namespace has a `default` SA. Pods that don't specify `serviceAccountName` run as `default`.

Projected token (modern, K8s 1.22+)

- Bound to a specific audience and expiration (default 1 hour, auto-refreshed by kubelet).
- Mounted via projected volume at `/var/run/secrets/kubernetes.io/serviceaccount/token`.
- Replaces the legacy non-expiring Secret-based token.

`automountServiceAccountToken`

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: no-api }
automountServiceAccountToken: false
```

Pods using this SA won't get a token mounted — useful when the pod never talks to the API server.

imagePullSecrets on SA

```
imagePullSecrets:
- name: regcred
```

All pods using this SA automatically pull from private registries.

Token request API

```
kubectl create token app-sa --audience=vault --expiration=3600s
```

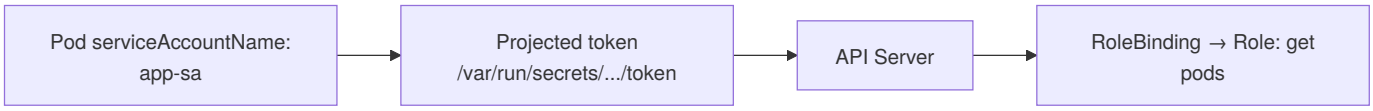
Returns a short-lived JWT for external use (e.g., authenticating to Vault).

🔷 How It Integrates

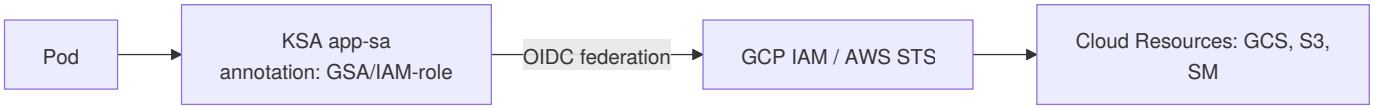
Component	Interaction
RBAC	RoleBinding binds SA to a Role
Pod	`spec.serviceAccountName` assigns SA
kubelet	Mounts projected SA token
Workload Identity (GKE)	Annotation maps KSA → GCP SA
IRSA / Pod Identity (EKS)	Annotation maps KSA → IAM Role
Admission	PSA can restrict SA capabilities

🏗 Diagrams

SA → RBAC → API



Workload Identity / IRSA



💻 Real-World Examples

Example 1 — Dedicated SA with RBAC

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: app-sa, namespace: prod }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: app-rb, namespace: prod }
subjects: [{ kind: ServiceAccount, name: app-sa, namespace: prod }]
roleRef: { kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io }
```

Example 2 — Disable automount

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: no-api }
automountServiceAccountToken: false
```

Example 3 — GKE Workload Identity SA

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: gcs-reader
  namespace: prod
  annotations:
    iam.gke.io/gcp-service-account: gcs-reader@myproj.iam.gserviceaccount.com
```

Example 4 — EKS IRSA SA

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: s3-reader
  namespace: prod
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123:role/s3-reader
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl create sa`	`<name>`	Create SA	`kubectl create sa app-sa -n prod`
`kubectl get sa`	`-n <ns>`	List SAs	`kubectl get sa -n prod`
`kubectl create token`	`<sa> --audience= --expiration=`	Generate token	`kubectl create token app-sa --audience=vault`
`kubectl set sa`	`deploy/<n> <sa>`	Assign SA to Deployment	`kubectl set sa deploy/web app-sa`
`kubectl describe sa`	`<name>`	Secrets, imagePullSecrets	`kubectl describe sa app-sa`

🚨 Scenarios

Scenario 1 — Pod can't call API: "forbidden"

- **Cause**: default SA has no RBAC binding.
- **Fix**: create dedicated SA + RoleBinding with minimum permissions.

Scenario 2 — Token expired but pod still running

- **Cause**: projected token auto-rotates; app caches old token.
- **Fix**: app should re-read token file on each request (K8s client libraries do this automatically).

Scenario 3 — ImagePullBackOff on private registry

- **Fix**: add `imagePullSecrets` to SA or pod spec.

☁ GCP GKE Perspective

- **Workload Identity Federation**: KSA annotated → projected OIDC token → Google STS → GCP SA credentials. No keys.
- GKE recommends disabling automount for pods that don't need API access.

🟩 AWS EKS Perspective

- **IRSA**: OIDC provider trusts EKS cluster; IAM role trusts KSA via condition. Auto-injects `AWS_WEB_IDENTITY_TOKEN_FILE`.
- **Pod Identity** (2023+): simpler — `aws eks create-pod-identity-association`. DaemonSet agent provides creds.

🧪 Labs

Lab 40.1 — Dedicated SA + RBAC (Beginner, 30 min)

Create SA, Role (get pods), RoleBinding, deploy pod with SA, exec into pod and test API access.

Lab 40.2 — Workload Identity on GKE (Intermediate, 60 min)

Bind KSA to GSA. Deploy pod. Verify it can list GCS buckets without keys.

Lab 40.3 — IRSA on EKS (Advanced, 60 min)

`eksctl create iamserviceaccount`. Deploy pod. Verify it reads from S3.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A ServiceAccount is a K8s-native identity for pods. Every pod runs as one. Modern K8s mounts a short-lived, audience-bound projected token that auto-rotates — replacing the old non-expiring Secret token.
Why: Pods need identity for API access (RBAC), image pulling (imagePullSecrets), and cloud access (Workload Identity / IRSA). Least-privilege means every app gets its own SA with minimum permissions.
How: I create a dedicated SA per microservice, bind it to a Role with only the verbs and resources it needs, and disable automount on SAs that never call the API. On GKE I annotate the SA for Workload Identity so the pod authenticates to GCP APIs without static keys. On EKS I use IRSA or Pod Identity.
Example: Our payment service needed access to AWS Secrets Manager. I created an IRSA-bound SA with a policy allowing only `GetSecretValue` on one ARN. Zero static credentials, automatic token rotation, principle of least privilege."

? Possible Interview Questions

Q1. What is a ServiceAccount? Namespaced identity for pods. **Q2. What token do pods get?** Projected bound token (K8s 1.22+) — short-lived, audience-scoped, auto-refreshed. **Q3. How do you disable token mounting?** `automountServiceAccountToken: false` on SA or pod. **Q4. Workload Identity — what is it?** KSA→GSA mapping via OIDC federation for keyless GCP access. **Q5. IRSA vs Pod Identity?** IRSA: OIDC-based per SA. Pod Identity: simpler API + DaemonSet agent. **Q6. How do you create a short-lived token for Vault?** `kubectl create token <sa> --audience=vault --expiration=600s`. **Q7. What happens if SA doesn't exist?** Pod fails to create (admission rejects). **Q8. imagePullSecrets on SA — what does it do?** Auto-adds registry credentials to every pod using that SA. **Q9. Best practice for default SA?** Don't use it for workloads. Create dedicated SAs. Disable automount on `default`. **Q10. How do you verify pod**

identity? `kubectl exec <pod> -- cat /var/run/secrets/kubernetes.io/serviceaccount/token | jwt decode -`.

🗐 Topic 41 — Pod Security Standards / Pod Security Admission

Official Documentation Reference - Pod Security Standards: https://kubernetes.io/docs/concepts/security/pod-security-standards/ - Pod Security Admission: https://kubernetes.io/docs/concepts/security/pod-security-admission/ - Enforce PSS: https://kubernetes.io/docs/tasks/configure-pod-container/enforce-standards-admission-controller/ - GKE PSS: https://cloud.google.com/kubernetes-engine/docs/how-to/podsecurityadmission - EKS PSS: https://docs.aws.amazon.com/eks/latest/best-practices/pod-security.html

🔹 Definition

Pod Security Standards (PSS) define three security profiles — Privileged, Baseline, Restricted — for pod configurations. **Pod Security Admission (PSA)** is the built-in admission controller that enforces PSS at the namespace level via labels.

🔹 About

Three profiles

Profile	What it allows	Target
Privileged	Everything — no restrictions	System/infra workloads
Baseline	Blocks known privilege escalations (hostNetwork, hostPID, privileged containers)	Default for most workloads
Restricted	Strictest — non-root, drop ALL capabilities, readOnlyRootFilesystem, seccomp RuntimeDefault	Hardened production

PSA modes (per namespace)

Mode	Behavior
`enforce`	Reject non-compliant pods
`audit`	Allow but log violations to audit log
`warn`	Allow but return warnings to kubectl client

Namespace labels

```
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: v1.31
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

PSP → PSA migration

PodSecurityPolicy was removed in K8s 1.25. PSA is the built-in replacement. For richer policy (mutating, image allowlists), use Kyverno or OPA/Gatekeeper alongside PSA.

Restricted profile requirements

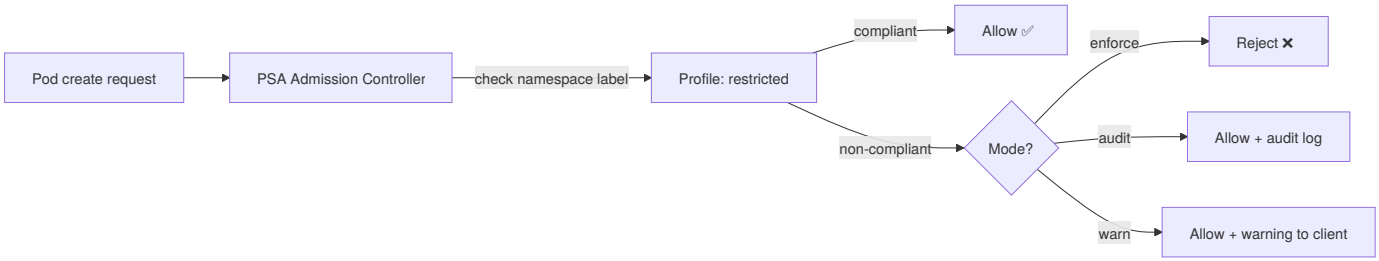
- `runAsNonRoot: true`
- No `privileged: true`
- No `hostNetwork`, `hostPID`, `hostIPC`
- Drop `ALL` capabilities; may add only `NET_BIND_SERVICE`
- `seccompProfile.type: RuntimeDefault` or `Localhost`
- `allowPrivilegeEscalation: false`
- No `hostPath` volumes
- `runAsUser` must not be 0

🔹 How It Integrates

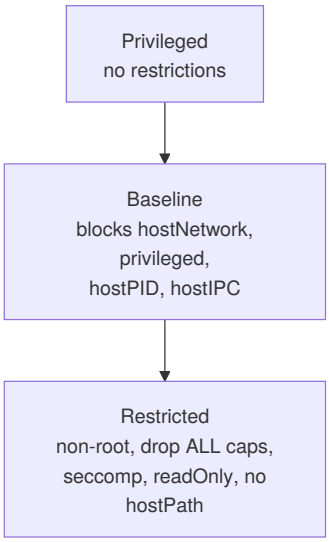
Component	Interaction
Namespace labels	Configure PSA mode + profile
Admission controller	Built-in, always available (K8s 1.25+)
Kyverno / OPA	Complementary policy engines for richer rules
SecurityContext	Pod/container fields that PSA validates
DaemonSets	May need Privileged or Baseline for node access

🏗 Diagrams

PSA enforcement flow



Profile comparison



💻 Real-World Examples

Example 1 — Label a namespace for restricted

```
kubectl label namespace prod \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted
```

Example 2 — Compliant pod

```
apiVersion: v1
kind: Pod
metadata: { name: safe, namespace: prod }
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 1000
    fsGroup: 2000
    seccompProfile: { type: RuntimeDefault }
  containers:
  - name: app
    image: myrepo/app:1
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
```

Example 3 — Non-compliant pod → rejected

```
kubectl run priv --image=nginx --overrides='{"spec":{"containers":[{"name":"priv","image":"nginx","securityContext":{"privileged":true}}]}}' -n prod
# Error: pods "priv" is forbidden: violates PodSecurity "restricted:v1.31"
```

Example 4 — Dry-run namespace migration

```
kubectl label --dry-run=server --overwrite ns dev \
  pod-security.kubernetes.io/enforce=baseline
# Shows warnings for existing non-compliant pods
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl label ns`	`pod-security.kubernetes.io/enforce=<profile>`	Set PSA	see Example 1
`kubectl label ns`	`--dry-run=server`	Preview violations	see Example 4
`kubectl get ns`	`--show-labels`	See PSA labels	`kubectl get ns --show-labels`
`kubectl describe ns`	—	Labels section	`kubectl describe ns prod`

🚨 Scenarios

Scenario 1 — System DaemonSet rejected in restricted namespace

- **Fix**: `kube-system` stays Privileged. Create separate namespace for infra with Baseline.

Scenario 2 — Migrating from PSP to PSA

- **Steps**: 1. Enable PSA in `warn` + `audit` mode. 2. Fix violations. 3. Switch to `enforce`. 4. Remove PSP.

Scenario 3 — Third-party Helm chart fails in restricted NS

- **Cause**: chart doesn't set securityContext.
- **Fix**: override values (`securityContext.runAsNonRoot=true`), or use Kyverno to mutate defaults.

☁ GCP GKE Perspective

- **Autopilot** enforces a Baseline-equivalent by default; some Restricted fields enforced.
- **Standard**: PSA available; label namespaces manually.
- **Policy Controller** (GKE Enterprise): OPA-based, richer than PSA alone.

🟧 AWS EKS Perspective

- PSA available by default (K8s 1.25+). EKS doesn't label namespaces by default — you must apply labels.
- **EKS Best Practices Guide** recommends Baseline as minimum, Restricted for sensitive workloads.
- Complement with **Kyverno** or **OPA/Gatekeeper** for image allowlists and custom policies.

🧪 Labs

Lab 41.1 — Enforce Restricted (Beginner, 30 min)

Label namespace, try compliant + non-compliant pods. Verify rejection.

Lab 41.2 — Audit + warn before enforce (Intermediate, 45 min)

1. Label `audit` + `warn` on a namespace with existing workloads.
2. `kubectl get events --field-selector reason=FailedCreate` + check audit logs.
3. Fix violations. 4. Switch to `enforce`.

Lab 41.3 — PSA + Kyverno combined (Advanced, 60 min)

1. PSA `enforce: baseline`. 2. Kyverno policy: mutate pods to add `seccompProfile: RuntimeDefault` if missing. 3. Deploy without seccomp → Kyverno mutates → PSA allows.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Pod Security Standards define three profiles — Privileged, Baseline, Restricted — that codify which pod security settings are required. Pod Security Admission is the built-in controller that enforces these profiles per namespace via labels, in enforce, audit, or warn mode.
Why: PSP was complex and removed in 1.25. PSA is simpler — just label a namespace. It prevents the top privilege-escalation vectors: privileged containers, host namespaces, running as root, missing seccomp.
How: I set every production namespace to `enforce: restricted`, `audit: restricted`, `warn: restricted`. Infra namespaces like `kube-system` stay Privileged. During migration I start with `warn` + `audit` to identify violations, fix pods, then flip to `enforce`. For anything PSA can't cover — like image registry allowlists or label requirements — I layer Kyverno on top.
Example: We migrated 40 namespaces from PSP to PSA. First week in `warn` mode surfaced 23 non-compliant Deployments. After fixing securityContext on each, we switched to `enforce: restricted` with zero pod rejections."

? Interview Questions

Q1. Three PSS profiles? Privileged, Baseline, Restricted. **Q2. Three PSA modes?** enforce, audit, warn. **Q3. How do you apply PSA?** Labels on the namespace. **Q4. What replaced PSP?** PSA (built-in) + optional Kyverno/OPA for richer policy. **Q5. What does Restricted require?** runAsNonRoot, drop ALL caps, seccomp RuntimeDefault, no hostPath, no privileged, allowPrivilegeEscalation false. **Q6. Can PSA mutate pods?** No — it only validates. Use Kyverno for mutation. **Q7. How**

to test before enforcing? `--dry-run=server` on label, or start with `warn` + `audit`. **Q8. GKE Autopilot PSA?** Enforced by default (Baseline-equivalent). **Q9. How to exempt system DaemonSets?** Keep `kube-system` as Privileged; or use PSA exemptions in the admission config. **Q10. PSA version pinning?** `enforce-version: v1.31` locks validation rules to a specific K8s version.

🗐 Topic 42 — Security Contexts

Official Documentation Reference - Security context: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/ - Linux kernel security: https://kubernetes.io/docs/concepts/security/linux-kernel-security-constraints/ - Seccomp: https://kubernetes.io/docs/reference/node/seccomp/

🔹 Definition

A **Security Context** defines privilege and access control settings for a pod or container — user/group IDs, Linux capabilities, filesystem permissions, seccomp profiles, AppArmor profiles, and SELinux labels. It's the low-level knob that PSA validates at a higher level.

🔹 About

Pod-level vs container-level

Field	Pod-level (`spec.securityContext`)	Container-level (`spec.containers[].securityContext`)
`runAsUser`	✅	✅ (overrides pod)
`runAsNonRoot`	✅	✅
`runAsGroup`	✅	✅
`fsGroup`	✅ (sets GID on volumes)	—
`supplementalGroups`	✅	—
`seccompProfile`	✅	✅
`capabilities`	—	✅
`allowPrivilegeEscalation`	—	✅
`readOnlyRootFilesystem`	—	✅
`privileged`	—	✅

Key fields explained

Field	What it does
`runAsUser: 1000`	Container runs as UID 1000
`runAsNonRoot: true`	Reject if image has USER root
`runAsGroup: 3000`	Primary GID
`fsGroup: 2000`	All volume files owned by GID 2000
`allowPrivilegeEscalation: false`	Prevent `setuid` / `setgid` binaries
`readOnlyRootFilesystem: true`	Container can't write to `/` ; use emptyDir for tmp
`privileged: true`	Full host access — almost never in prod
`capabilities.drop: ["ALL"]`	Drop all Linux capabilities
`capabilities.add: ["NET_BIND_SERVICE"]`	Allow binding to ports < 1024
`seccompProfile.type: RuntimeDefault`	Apply containerd's default seccomp profile
`seccompProfile.type: Localhost`	Custom seccomp profile on node

Hardened template

```
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 65534          # nobody
    runAsGroup: 65534
    fsGroup: 65534
    seccompProfile: { type: RuntimeDefault }
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities: { drop: ["ALL"] }
    volumeMounts:
    - { name: tmp, mountPath: /tmp }
  volumes:
  - { name: tmp, emptyDir: {} }
```

AppArmor

```
metadata:
  annotations:
    container.apparmor.security.beta.kubernetes.io/app: runtime/default
```

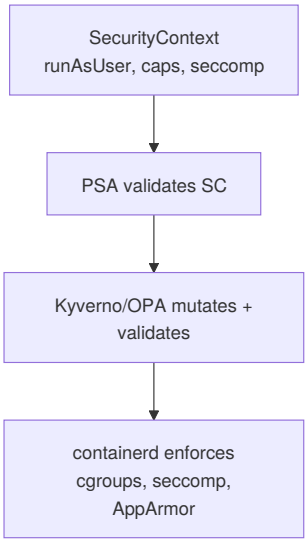
K8s 1.30+ supports `securityContext.appArmorProfile` field (GA).

How It Integrates

Component	Interaction
PSA	Validates securityContext fields against PSS profile
Container runtime	Applies cgroups, namespaces, seccomp, AppArmor
Volumes	`fsGroup` sets GID on mounted volume files
Admission (Kyverno)	Can mutate missing securityContext
Image	`USER` directive in Dockerfile sets default UID

Diagrams

Security layers



Real-World Examples

Example 1 — Verify UID inside container

```
kubectl apply -f hardened.yaml
kubectl exec safe -- id
# uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
```

Example 2 — readOnlyRootFilesystem + tmp emptyDir

```
securityContext:
  readOnlyRootFilesystem: true
volumeMounts: [{ name: tmp, mountPath: /tmp }]
volumes: [{ name: tmp, emptyDir: {} }]
```

Example 3 — Custom seccomp profile

```
securityContext:
  seccompProfile:
    type: Localhost
    localhostProfile: profiles/custom.json
```

Related Commands

Command	Flag	Purpose	Example
`kubectl exec`	`-- id`	Check UID/GID	`kubectl exec safe -- id`
`kubectl exec`	`-- cat /proc/1/status | grep Cap`	Check capabilities	
`kubectl describe pod`	—	See security context	`kubectl describe pod safe`
`kubectl get pod`	`-o jsonpath='{.spec.securityContext}'`	Extract	

🚨 Scenarios

Scenario 1 — App can't write to `/var/log`

- **Cause**: `readOnlyRootFilesystem: true`.
- **Fix**: mount emptyDir at `/var/log`.

Scenario 2 — Container needs port 80 but runs as non-root

- **Cause**: ports < 1024 need `CAP_NET_BIND_SERVICE`.
- **Fix**: add `capabilities: { add: ["NET_BIND_SERVICE"] }`. Or use port > 1024 and let the Service map.

Scenario 3 — `fsGroup` slow on large volumes

- **Cause**: `fsGroup` recursively `chown`s on mount.
- **Fix**: `fsGroupChangePolicy: OnRootMismatch` (K8s 1.26+ stable) — only changes if root GID doesn't match.

☁ GKE Perspective

- **Autopilot** enforces much of the Restricted profile. `privileged: true` blocked.
- **GKE Sandbox** (gVisor) adds kernel-level isolation on top of securityContext.

🟩 EKS Perspective

- No enforcement by default — you must set securityContext + PSA labels.
- **Fargate**: `privileged` blocked. `hostNetwork/PID/IPC` blocked.
- **Bottlerocket AMI**: read-only OS + SELinux enforced on nodes.

🧪 Labs

Lab 42.1 — Hardened pod (Beginner, 20 min)

Apply hardened template. `kubectl exec -- id`, `kubectl exec -- touch /test` (fails — readOnly).

Lab 42.2 — Drop ALL + add NET_BIND_SERVICE (Intermediate, 30 min)

Deploy nginx on port 80 as non-root with `drop: ALL, add: NET_BIND_SERVICE`. Verify it serves traffic.

Lab 42.3 — Custom seccomp profile (Advanced, 60 min)

1. Write a seccomp JSON blocking `chmod`. 2. Place on node. 3. Deploy pod with `Localhost` profile. 4. `kubectl exec -- chmod 777 /tmp` → blocked.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A Security Context sets pod and container-level privilege controls — UID/GID, capabilities, filesystem permissions, seccomp, AppArmor. Pod Security Admission validates these at a namespace level; the container runtime enforces them.
Why: Defense in depth. Even if an attacker gets code execution, they're running as non-root, can't escalate privileges, can't write to the filesystem, and are confined by a seccomp profile that blocks dangerous syscalls.
How: My template for every production container: `runAsNonRoot: true, runAsUser: 65534, allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: {drop: [ALL]}, seccompProfile: RuntimeDefault`. If the app needs `/tmp`, I mount an emptyDir. If it needs port 80, I add `NET_BIND_SERVICE`. I set `fsGroupChangePolicy: OnRootMismatch` to avoid slow volume mounts.
Example: A pen-test on our EKS cluster exploited a log4j RCE. Because the container ran as non-root with readOnlyRootFilesystem and dropped capabilities, the attacker couldn't install tools, escalate, or pivot — the blast radius was one read-only container."

? Interview Questions

Q1. Pod-level vs container-level securityContext? Pod sets defaults (UID, fsGroup, seccomp). Container overrides (caps, readOnly, privileged). **Q2. What does `runAsNonRoot: true` do?** Rejects the pod if the container image runs as UID 0. **Q3. `capabilities: drop: [ALL]` — why?** Drops all Linux capabilities. Minimum-privilege baseline. **Q4. `readOnlyRootFilesystem` — why?** Prevents writing to container filesystem — attacker can't install tools. Mount emptyDir for writable dirs. **Q5. What is fsGroup?** Sets GID ownership on all mounted volume files so non-root can read/write. **Q6. What's `allowPrivilegeEscalation: false`?** Prevents setuid/setgid binaries from escalating privileges. **Q7. seccomp RuntimeDefault vs Localhost?** RuntimeDefault = containerd's built-in profile. Localhost = custom JSON on the node. **Q8. When would you set `privileged: true`?** Almost never in prod. Only for node-level agents (CNI, CSI node plugin) that need full host access. **Q9. `fsGroupChangePolicy: OnRootMismatch`?** Only chowns if root directory GID doesn't match fsGroup — avoids slow recursive chown on large volumes. **Q10. AppArmor in K8s?** `securityContext.appArmorProfile` (GA 1.30+) or annotation. Confines file access and program execution.

🗐 Topic 43 — Secrets Management Deep Dive (Sealed Secrets, External Secrets, Vault, Workload Identity, IRSA)

Official Documentation Reference - Secrets good practices: https://kubernetes.io/docs/concepts/security/secrets-good-practices/ - Encryption at rest: https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/ - External Secrets Operator: https://external-secrets.io/latest/ - Sealed Secrets: https://github.com/bitnami-labs/sealed-secrets - HashiCorp Vault: https://developer.hashicorp.com/vault/docs/platform/k8s - CSI Secrets Store: https://secrets-store-csi-driver.sigs.k8s.io/ - GKE Workload Identity: https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity - GKE Secret Manager add-on: https://cloud.google.com/kubernetes-engine/docs/how-to/secret-manager - EKS IRSA: https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html - EKS Pod Identity: https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html

⏚ Definition

Secrets management in Kubernetes covers the full lifecycle of sensitive data — creation, storage, delivery to pods, rotation, and auditing — using a combination of K8s Secrets, encryption-at-rest, external secret stores, and cloud identity federation.

⏚ About

Maturity levels

Level	Pattern	Security	Ops burden
1	K8s Secret in YAML, committed to Git	✖ Plaintext in Git + etcd	Low
2	K8s Secret + encryption at rest (KMS)	⚠ Encrypted in etcd	Low
3	SealedSecrets in Git → controller decrypts	✅ Encrypted in Git + etcd	Medium
4	ExternalSecrets Operator → cloud SM	✅ Secret never in Git	Medium
5	CSI Secrets Store → mount at pod start	✅ Secret never in etcd	Medium
6	Vault with dynamic secrets + TTL	✅✅ Short-lived, audited	High

Tool comparison

Tool	Source of truth	How secret reaches pod	Rotation	GitOps-safe
Sealed Secrets	Git (encrypted)	K8s Secret (controller decrypts)	Manual re-seal	✅
External Secrets Operator	Cloud SM / Vault	K8s Secret (synced by ESO)	`refreshInterval`	✅
CSI Secrets Store	Cloud SM / Vault	Mounted file (no K8s Secret)	`rotationPollInterval`	✅
Vault Agent Injector	Vault	Sidecar writes to shared volume	Vault TTL	✅
Vault CSI Provider	Vault	CSI mount	Vault TTL	✅

Cloud identity (keyless access to secret stores)

Cloud	Mechanism	How
GKE	Workload Identity Federation	KSA annotation → OIDC → GCP SA → Secret Manager
EKS	IRSA	KSA annotation → OIDC → STS → IAM Role → Secrets Manager
EKS	Pod Identity (2023+)	Association API → DaemonSet agent → IAM Role

Rotation strategies

Strategy	When
Volume-based + file watcher	App reloads config file; works with ESO/CSI
Reloader controller	Triggers rollout restart on Secret change; for env consumption
Vault dynamic secrets	DB creds generated per-lease; auto-expire
Immutable Secret + kustomize hash	New Secret name → Deployment rolls

⏚ How It Integrates

Component	Interaction
K8s Secret	Target object ESO/SealedSecrets sync into
CSI driver	Mounts secret as tmpfs without K8s Secret
ServiceAccount	Identity for OIDC federation (WLI/IRSA)
RBAC	Controls who can `get secrets`
Audit log	Records Secret access
Kyverno / OPA	Can enforce no-inline-secrets, require external refs

🏗 Diagrams

ESO flow



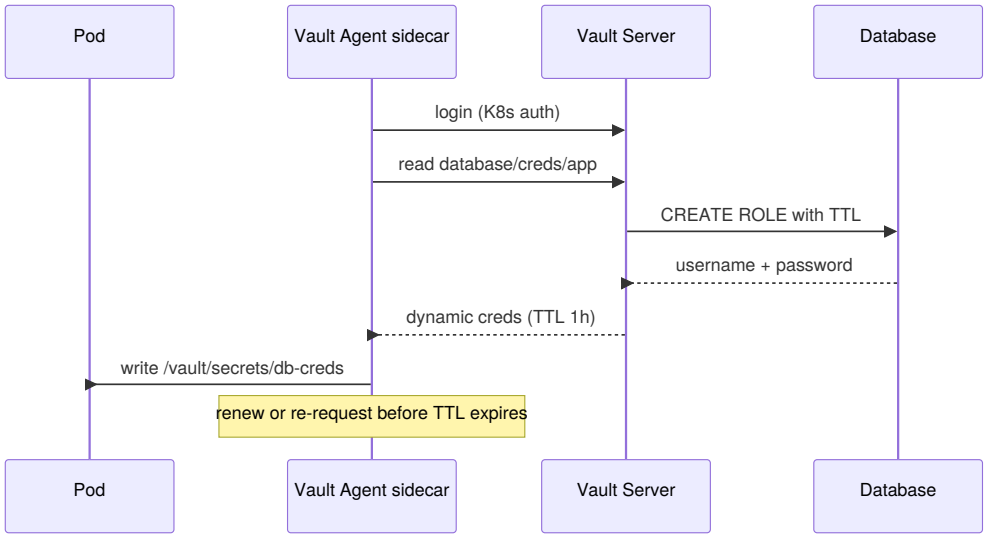
Sealed Secrets flow



CSI Secrets Store flow



Vault dynamic secrets



💻 Real-World Examples

Example 1 — Sealed Secrets end-to-end

```
# Install controller
helm repo add sealed-secrets https://bitnami-labs.github.io/sealed-secrets
helm install sealed-secrets sealed-secrets/sealed-secrets -n kube-system

# Seal a secret
kubectl create secret generic db-cred --from-literal=PASSWORD=S3cure! --dry-run=client -o yaml | \
  kubeseal --format yaml > sealed-db-cred.yaml

# Commit to Git
git add sealed-db-cred.yaml && git commit -m "sealed db cred"

# Apply — controller decrypts → K8s Secret created
kubectl apply -f sealed-db-cred.yaml
kubectl get secret db-cred -o yaml
```

Example 2 — ESO with GCP Secret Manager

```
apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata: { name: gcp-sm }
spec:
  provider:
    gcpsm:
      projectID: my-project
---
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: { name: db-cred, namespace: prod }
spec:
  refreshInterval: 1h
  secretStoreRef: { name: gcp-sm, kind: ClusterSecretStore }
  target: { name: db-cred }
  data:
  - { secretKey: PASSWORD, remoteRef: { key: prod-db-password } }
```

Example 3 — CSI Secrets Store with AWS provider

```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata: { name: aws-secrets }
spec:
  provider: aws
  parameters:
    objects: |
      - objectName: "prod/api-key"
        objectType: "secretsmanager"
---
spec:
  containers:
  - name: app
    volumeMounts: [{ name: secrets, mountPath: /mnt/secrets, readOnly: true }]
  volumes:
  - name: secrets
    csi:
      driver: secrets-store.csi.k8s.io
      readOnly: true
      volumeAttributes: { secretProviderClass: aws-secrets }
```

Example 4 — Vault Agent Injector

```
metadata:
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/agent-inject-secret-db: "database/creds/app"
    vault.hashicorp.com/role: "app"
spec:
  serviceAccountName: app-sa
```

💻 Related Commands

Command	Purpose	Example
`kubeseal --format yaml`	Encrypt Secret	`kubeseal < secret.yaml > sealed.yaml`
`kubectl get externalsecret`	Check ESO sync	`kubectl get es -n prod`
`kubectl describe externalsecret`	Sync status + errors	`kubectl describe es db-cred`
`kubectl get secretproviderclass`	CSI store config	`kubectl get spc`
`vault kv get secret/data/app`	Read Vault secret	

🚨 Scenarios

Scenario 1 — ESO shows `SecretSyncedError`

- **Cause**: SA lacks IAM permission on secret store; or secret name wrong.
- **Fix**: check ESO controller logs; verify IRSA/WLI; verify remote key name.

Scenario 2 — Sealed Secret decryption fails after cluster recreate

- **Cause**: sealing key (certificate) differs between old and new cluster.
- **Fix**: backup the sealed-secrets key pair. Restore before applying sealed secrets.

Scenario 3 — Vault lease expired; app fails

- **Cause**: Vault agent didn't renew; or max TTL exceeded.
- **Fix**: tune `default_lease_ttl` and `max_lease_ttl` in Vault; ensure agent sidecar is healthy.

☁ GKE Perspective

- **Secret Manager add-on** (`--enable-secret-manager`): mounts GSM secrets as volumes without CSI driver install.
- **Workload Identity**: recommended identity mechanism for ESO/CSI accessing GSM.
- **Binary Authorization**: can attest that only containers with verified secrets are deployed.

▩ EKS Perspective

- **IRSA** or **Pod Identity** for ESO/CSI authenticating to AWS SM/SSM.
- **AWS Secrets Manager**: versioning + rotation lambda.
- **SSM Parameter Store**: cheaper; good for non-rotating config.
- CSI + ESO both supported; ESO is more flexible (multiple backends, refresh sync).

🧪 Labs

Lab 43.1 — Sealed Secrets end-to-end (Beginner, 45 min)

Install controller, seal a secret, commit, apply, verify K8s Secret created.

Lab 43.2 — ESO with AWS Secrets Manager + IRSA (Intermediate, 90 min)

Install ESO, create IRSA SA, store secret in ASM, apply ExternalSecret, verify K8s Secret synced, mount in pod.

Lab 43.3 — Vault dynamic database credentials (Advanced, 120 min)

1. Deploy Vault via Helm. 2. Enable database secrets engine. 3. Configure Postgres dynamic role. 4. Deploy app with Vault Agent Injector. 5. Verify pod receives unique DB creds that auto-expire.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Secrets management in Kubernetes spans from basic K8s Secrets to enterprise-grade patterns: encryption at rest with KMS, Sealed Secrets for GitOps, External Secrets Operator for continuous sync from cloud managers, and CSI Secrets Store for mount-only secrets that never touch etcd.
Why: The default K8s Secret is base64, not encrypted. Real security requires the secret to never exist in Git, to be encrypted at rest in etcd, to rotate automatically, and to be audited. External stores like GCP Secret Manager, AWS Secrets Manager, or Vault provide all of that.
How: My standard pattern is ESO + Workload Identity (GKE) or IRSA (EKS). Secrets live in the cloud secret manager — versioned, rotation-capable, IAM-gated. ESO syncs them into K8s Secrets at a refresh interval. For ultra-sensitive workloads I use Vault with dynamic credentials — the database password is generated per-pod-lease and expires automatically. For GitOps I use Sealed Secrets so the encrypted blob can be committed safely.
Example: We migrated 200 K8s Secrets from inline YAML to External Secrets Operator backed by AWS Secrets Manager. Rotation went from a manual Jira ticket to an automated `aws secretsmanager rotate-secret` call. Our last audit showed zero stale credentials."

? Interview Questions

Q1. Why is base64 not secure? Anyone with `kubectl get secret` can decode. It's encoding, not encryption. **Q2. Sealed Secrets vs External Secrets?** SealedSecrets: source in Git (encrypted). ESO: source in external store (never in Git). **Q3. CSI Secrets Store vs ESO?** CSI mounts files directly (no K8s Secret object). ESO creates a K8s Secret (usable as env/volume). **Q4. What is Vault dynamic secrets?** Vault generates unique, time-limited credentials per consumer — e.g., a new DB user per pod lease. **Q5. How does Workload Identity avoid static keys?** KSA → OIDC token → Google STS → GCP SA credentials. No key file. **Q6. IRSA vs Pod Identity?** IRSA: OIDC + role trust policy per SA. Pod Identity: simpler API + DaemonSet agent, no OIDC setup. **Q7. How do you rotate a secret with ESO?** Rotate in cloud SM; ESO re-syncs at `refreshInterval` . Volume mount updates; env needs restart. **Q8. How do you backup Sealed Secrets keys?** `kubectl get secret -n kube-system -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml > backup.yaml` . **Q9. Encryption at rest on EKS?** `aws eks create-cluster --encryption-config resources=[secrets],provider={keyArn=...}` . **Q10. Best practice: env vs volume for secrets?** Volume — auto-updates, no leak via `kubectl describe` .

🗐 Topic 44 — Network Policies (Deep Dive)

Official Documentation Reference - Network Policies: https://kubernetes.io/docs/concepts/services-networking/network-policies/ - Cilium Network Policy: https://docs.cilium.io/en/stable/security/policy/ - Calico Network Policy: https://docs.tigera.io/calico/latest/network-policy/ - GKE DP-v2 policy: https://cloud.google.com/kubernetes-engine/docs/how-to/network-policy - EKS VPC CNI network policy: https://docs.aws.amazon.com/eks/latest/userguide/cni-network-policy.html

---

⟐ Definition

This deep dive covers **advanced Network Policy patterns**: multi-tier microsegmentation, egress control with FQDN rules, L7 policies (Cilium), audit/logging of

policy decisions, and production rollout strategies.

⎆ About — Advanced Patterns

Pattern 1 — Full zero-trust namespace

```
# 1. Default deny all ingress + egress
# 2. Allow DNS egress (UDP/TCP 53)
# 3. Allow app → db on specific port
# 4. Allow ingress-controller → frontend
# 5. Allow egress to external HTTPS (0.0.0.0/0 except RFC1918)
# 6. Allow monitoring scrape from Prometheus NS
```

Pattern 2 — FQDN-based egress (Cilium only)

Standard K8s NetworkPolicy only supports `ipBlock` for egress. **CiliumNetworkPolicy** adds `toFQDNs`:

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: { name: allow-github }
spec:
  endpointSelector: { matchLabels: { app: ci } }
  egress:
  - toFQDNs: [{ matchPattern: "*.github.com" }]
    toPorts: [{ ports: [{ port: "443", protocol: TCP }] }]
  - toEndpoints: [{ matchLabels: { k8s:io.kubernetes.pod.namespace: kube-system, k8s-app: kube-dns } }]
    toPorts: [{ ports: [{ port: "53", protocol: ANY }] }]
```

Pattern 3 — L7 HTTP policy (Cilium)

```
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: { name: l7-api }
spec:
  endpointSelector: { matchLabels: { app: api } }
  ingress:
  - fromEndpoints: [{ matchLabels: { app: frontend } }]
    toPorts:
    - ports: [{ port: "8080", protocol: TCP }]
      rules:
        http:
        - method: GET
          path: "/api/v1/.*"
        - method: POST
          path: "/api/v1/orders"
```

Pattern 4 — Calico GlobalNetworkPolicy

Cluster-scoped policy (not namespaced):

```
apiVersion: projectcalico.org/v3
kind: GlobalNetworkPolicy
metadata: { name: deny-egress-to-metadata }
spec:
  selector: all()
  types: [Egress]
  egress:
  - action: Deny
    destination:
      nets: ["169.254.169.254/32"]     # block IMDS
```

Pattern 5 — Policy audit mode (Calico)

```
spec:
  types: [Ingress]
  ingress:
  - action: Log          # Calico: Log + Allow for audit before enforce
```

Policy rollout strategy

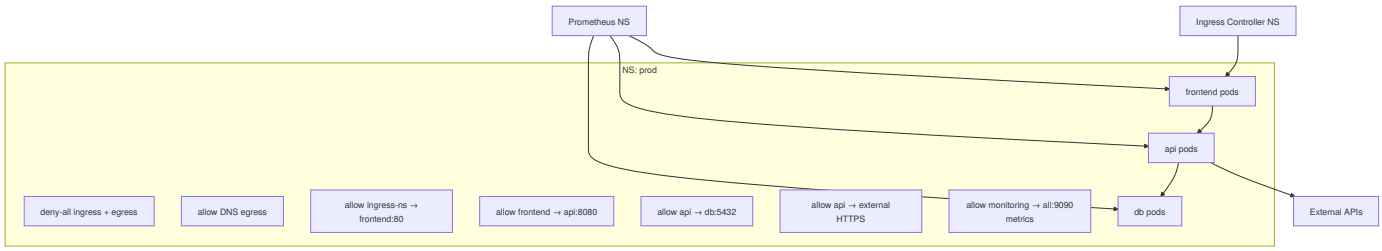
1. **Audit/log only** — deploy policies with `Log` action (Calico) or use `warn` PSA.
2. **Allow existing + deny new** — use `default-deny` + explicit allows matching current traffic.
3. **Gradual enforcement** — one namespace at a time, monitoring for dropped traffic (Hubble/Flow Logs).
4. **Full enforcement** — all namespaces, default-deny, explicit allows.

Observability tools for policy

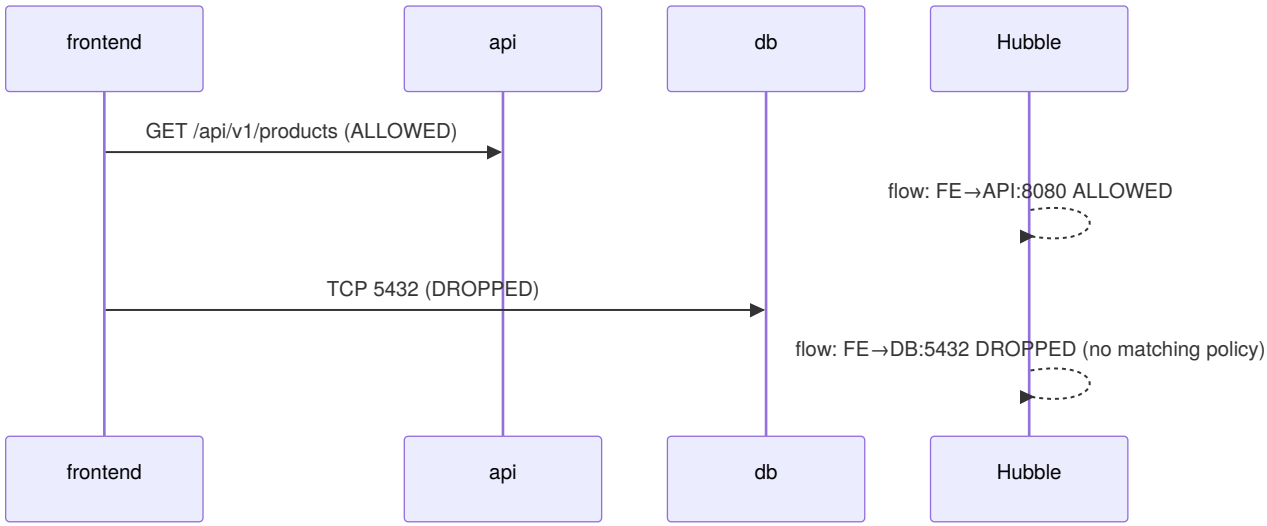
Tool	What it shows
Cilium Hubble	Real-time flow logs: allowed/dropped, source/dest, verdict
Calico Flow Logs	Structured logs of policy decisions
GKE Flow Logs	VPC-level flow logs + DP-v2 policy logs
Network Policy Editor (cilium.io)	Visual policy builder

🏗 Diagrams

Multi-tier microsegmentation



Hubble flow visibility



💻 Real-World Examples

Example 1 — Complete zero-trust namespace

```
# deny-all.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: deny-all, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
# allow-dns.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-dns, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
---
# allow-frontend-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-ingress-to-frontend, namespace: prod }
spec:
  podSelector: { matchLabels: { app: frontend } }
  policyTypes: [Ingress]
  ingress:
  - from: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: ingress-nginx } } }]
    ports: [{ port: 80 }]
---
# allow-frontend-to-api.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-frontend-to-api, namespace: prod }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress]
  ingress:
  - from: [{ podSelector: { matchLabels: { app: frontend } } }]
    ports: [{ port: 8080 }]
---
# allow-api-to-db.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-api-to-db, namespace: prod }
spec:
  podSelector: { matchLabels: { app: db } }
  policyTypes: [Ingress]
  ingress:
  - from: [{ podSelector: { matchLabels: { app: api } } }]
    ports: [{ port: 5432 }]
---
# allow-external-egress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-external-https, namespace: prod }
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Egress]
  egress:
  - to: [{ ipBlock: { cidr: 0.0.0.0/0, except: [10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16] } }]
    ports: [{ port: 443, protocol: TCP }]
  - ports: [{ protocol: UDP, port: 53 }]
---
# allow-prometheus-scrape.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-prometheus, namespace: prod }
spec:
  podSelector: {}
  policyTypes: [Ingress]
  ingress:
  - from: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: monitoring } } }]
    ports: [{ port: 9090 }]
```

💻 Related Commands

Command	Purpose	Example
`kubectl get netpol -n prod`	List policies	
`hubble observe -n prod --verdict DROPPED`	See dropped flows	
`hubble observe -n prod --to-label app=db`	Filter by destination	
`calicoctl get networkpolicy -n prod`	Calico-specific policies	
`cilium policy get -n prod`	Cilium-specific	
`kubectl exec <client> -- wget -qO- --timeout=3 http://db:5432`	Test connectivity	

🚨 Scenarios

Scenario 1 — Policy deployed but egress to payment gateway broken

- **Cause**: external IP changed; ipBlock didn't include new CIDR.
- **Fix**: use Cilium `toFQDNs` instead of ipBlock for dynamic external IPs.

Scenario 2 — Monitoring broken after deny-all

- **Cause**: Prometheus scrape ingress not allowed.
- **Fix**: add allow rule for monitoring namespace on metrics port (9090/9100).

Scenario 3 — Cilium L7 policy blocks valid POST

- **Cause**: HTTP path regex too restrictive.
- **Fix**: `hubble observe` → see exact path denied → fix regex.

☁ GKE Perspective

- **Dataplane V2** enforces K8s NetworkPolicy natively + provides GKE-specific flow logging via Cloud Logging.
- **GKE Enterprise**: Anthos Network Policy for fleet-wide policies.

▩ EKS Perspective

- **VPC CNI native policy** (K8s 1.25+, eBPF-based).
- **Calico** as overlay for richer policy (GlobalNetworkPolicy, DNS policy).
- **Security Groups for Pods**: VPC-level ACL complement to K8s NetworkPolicy.

🧪 Labs

Lab 44.1 — Full zero-trust namespace (Intermediate, 60 min)

Apply Example 1 (all 7 policies). Deploy frontend, api, db. Test every path with `wget --timeout=3`. Verify allowed paths work, unauthorized paths timeout.

Lab 44.2 — Cilium FQDN + L7 policy (Advanced, 90 min)

1. Install Cilium + Hubble. 2. Apply FQDN egress for `*.github.com:443`. 3. Apply L7 HTTP policy allowing only `GET /api/v1/*`. 4. `hubble observe --verdict DROPPED` → see blocked POST.

Lab 44.3 — Calico GlobalNetworkPolicy block IMDS (Advanced, 45 min)

1. Install Calico. 2. Apply GlobalNetworkPolicy blocking `169.254.169.254/32` egress. 3. Verify pod can't reach metadata endpoint. 4. Exemption: add `pass` rule for `kube-system` pods.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Advanced Network Policy goes beyond basic allow/deny — it includes multi-tier microsegmentation, FQDN-based egress (Cilium), L7 HTTP policy, cluster-wide GlobalNetworkPolicy (Calico), and flow-level observability via Hubble or Calico Flow Logs.
Why: A flat default-allow network means a compromised frontend pod can reach the database directly. Microsegmentation limits blast radius — frontend can only talk to the API on port 8080, API can only talk to DB on 5432, nothing else.
How: I deploy 7 policies per namespace: deny-all ingress+egress, allow DNS, allow ingress-controller → frontend, allow frontend → API, allow API → DB, allow API → external HTTPS (ipBlock or toFQDNs), allow Prometheus scrape. I roll out in audit mode first (Cilium Hubble or Calico Log action), fix any legitimate drops, then enforce. For external APIs with dynamic IPs I use Cilium's `toFQDNs` instead of fragile ipBlock.
Example: After a pen-test found lateral movement from a compromised pod to our database, we implemented the full zero-trust pattern across 12 production namespaces in two weeks. Re-test confirmed the attack path was eliminated."

? Interview Questions

Q1. Standard K8s NetworkPolicy limitations? L3/L4 only; no FQDN; no L7; no cluster-scoped; no deny rule; no audit mode. **Q2. How does Cilium extend NetworkPolicy?** `CiliumNetworkPolicy` adds toFQDNs, L7 HTTP rules, DNS-aware egress, cluster-wide `CiliumClusterwideNetworkPolicy`. **Q3. How does Calico extend?** `GlobalNetworkPolicy` (cluster-scoped), `action: Log` for audit, DNS policy, ICMP rules. **Q4. What is the IMDS attack vector?** Pods can reach `169.254.169.254` for instance metadata (IAM creds on EC2). Block with NetworkPolicy egress ipBlock deny. **Q5. How do you audit before enforcing?** Cilium Hubble `--verdict DROPPED` in audit mode; Calico `action: Log`; GKE DP-v2 flow logs. **Q6. FQDN egress — why not just ipBlock?** External service IPs change; FQDN resolves dynamically. **Q7. L7 policy use case?** Allow `GET /api/v1/public` but block `DELETE /api/v1/admin`. **Q8. How do you allow Prometheus scrape through deny-all?** Ingress rule: `from: namespaceSelector matching monitoring NS` on metrics port. **Q9. What is `except` in ipBlock?** Exclude CIDRs from the allowed range (e.g., allow all public IPs except RFC1918). **Q10. How do you test policies?** `kubectl exec <pod> -- wget -qO- --timeout=3 http://target:port` — timeout = blocked, content = allowed.

🗐 Topic 45 — Image Security and Admission Controllers (OPA/Gatekeeper, Kyverno)

Official Documentation Reference - Admission controllers: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/ - Dynamic admission: https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/ - ValidatingAdmissionPolicy (K8s native): https://kubernetes.io/docs/reference/access-authn-authz/validating-admission-policy/ - OPA Gatekeeper: https://open-policy-agent.github.io/gatekeeper/ - Kyverno: https://kyverno.io/docs/ - GKE Binary Authorization: https://cloud.google.com/binary-authorization/docs/overview - GKE Policy Controller: https://cloud.google.com/kubernetes-engine/docs/concepts/policy-controller - EKS image scanning: https://docs.aws.amazon.com/AmazonECR/latest/userguide/image-scanning.html - Sigstore / cosign: https://docs.sigstore.dev/

Definition

Image security ensures only trusted, scanned, signed container images run in the cluster. **Admission controllers** intercept API requests before persistence — **mutating** webhooks modify objects, **validating** webhooks accept/reject them. **OPA/Gatekeeper** and **Kyverno** are the two dominant policy engines that implement admission control as code.

About

Image security lifecycle

Stage	Tool	What it does
Build	Multi-stage Dockerfile, distroless base	Minimize attack surface
Scan	Trivy, Grype, Snyk, ECR scanning, Artifact Analysis (GCP)	Find CVEs
Sign	cosign (Sigstore), Notation	Cryptographic proof of origin
Admit	Kyverno / OPA / Binary Authorization / Connaisseur	Block unsigned/unscanned images
Runtime	Falco, Sysdig, KubeArmor	Detect anomalous behavior

Admission controller types

Type	Phase	Purpose
Mutating	Before validation	Add defaults (labels, sidecar injection, resource limits)
Validating	After mutation	Reject non-compliant resources

Order: Mutating → Validating → Persist. A single webhook can be both.

Built-in admission controllers (selected)

`NamespaceLifecycle`, `LimitRanger`, `ResourceQuota`, `PodSecurity`, `ServiceAccount`, `DefaultStorageClass`, `MutatingAdmissionWebhook`, `ValidatingAdmissionWebhook`.

ValidatingAdmissionPolicy (K8s native, 1.30+ stable)

Write validation rules in CEL (Common Expression Language) directly — no webhook server needed:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: { name: require-labels }
spec:
  matchConstraints:
    resourceRules: [{ apiGroups: ["apps"], apiVersions: ["v1"], resources: ["deployments"], operations: ["CREATE","UPDATE"] }]
  validations:
  - expression: "has(object.metadata.labels) && 'team' in object.metadata.labels"
    message: "Deployment must have 'team' label"
```

OPA / Gatekeeper

- OPA is the general-purpose policy engine (Rego language).
- Gatekeeper is the K8s-native wrapper: `ConstraintTemplate` (defines the policy in Rego) + `Constraint` (applies it to specific resources).
- Supports both audit (report violations on existing resources) and enforcement (reject new resources).

Kyverno

- K8s-native policy engine — policies are YAML, no Rego.
- Supports **validate**, **mutate**, **generate**, **verifyImages**.
- `ClusterPolicy` for cluster-wide; `Policy` for namespaced.
- `validationFailureAction: Enforce` or `Audit`.

Comparison

Feature	Kyverno	OPA/Gatekeeper	ValidatingAdmissionPolicy
Language	YAML (K8s-native)	Rego	CEL
Mutate	✅	✅ (with assign)	❌ (MutatingAdmissionPolicy in 1.32)
Generate	✅	❌	❌
Image verification	✅ (`verifyImages`)	via OCI plugin	❌
Audit existing	✅ (PolicyReport)	✅ (audit)	✅
Complexity	Low	High (Rego)	Low
External data	✅	✅	Limited

Image signature verification

```
# Kyverno
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: verify-images }
spec:
  validationFailureAction: Enforce
  rules:
  - name: verify-cosign
    match: { any: [{ resources: { kinds: ["Pod"] } }] }
    verifyImages:
    - imageReferences: ["myrepo/*"]
      attestors:
      - entries:
        - keys: { publicKeys: "-----BEGIN PUBLIC KEY-----\n..." }
```

GKE Binary Authorization

Google-managed image attestation service. Only images with a valid attestation (from Cloud Build, Artifact Analysis, or manual) are admitted.

```
gcloud container binauthz policy export > policy.yaml
# edit to require attestation
gcloud container binauthz policy import policy.yaml
```

ECR Image Scanning

```
aws ecr describe-image-scan-findings --repository-name myrepo --image-id imageTag=latest
```

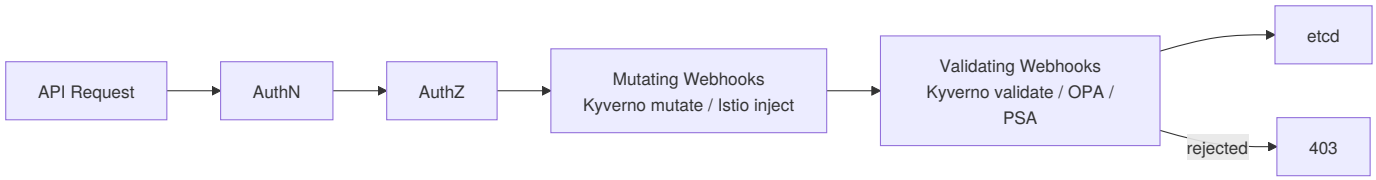
Combine with an admission webhook (Kyverno/OPA) that checks scan results before admitting.

🔷 How It Integrates

Component	Interaction
API server	Calls mutating then validating webhooks
Kyverno/OPA	Deployed as Deployments; registered as webhooks
Image registry	Source of images; scan results; signatures
CI/CD	Signs images with cosign; pushes attestations
PSA	Complementary — PSA validates securityContext; Kyverno/OPA validate everything else

🏗 Diagrams

Admission flow



Kyverno policy model

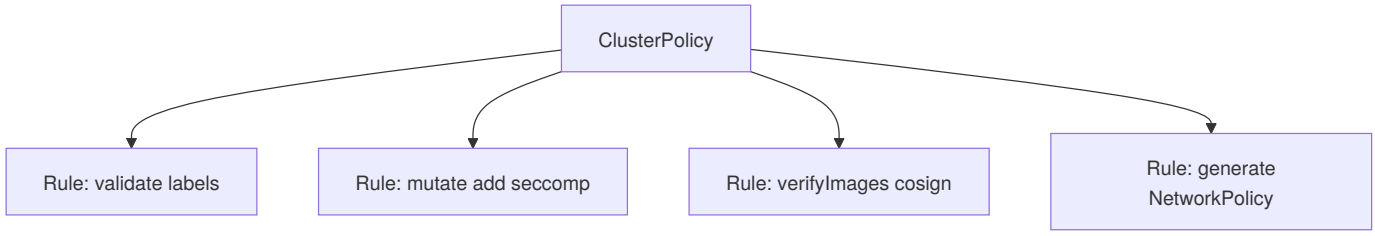
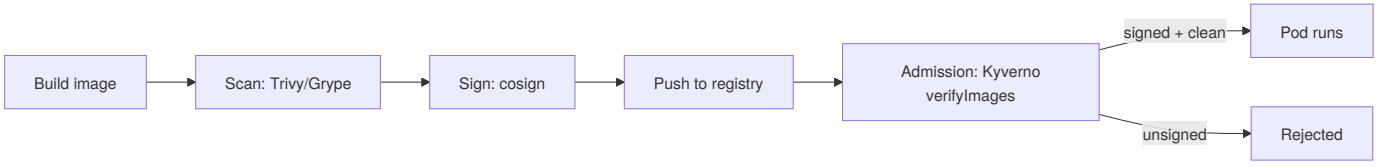


Image security pipeline



💻 Real-World Examples

Example 1 — Kyverno: require labels

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: require-labels }
spec:
  validationFailureAction: Enforce
  rules:
  - name: check
    match: { any: [{ resources: { kinds: ["Deployment","StatefulSet","DaemonSet"] } }] }
    validate:
      message: "Labels 'team' and 'env' are required."
      pattern:
        metadata:
          labels:
            team: "?*"
            env: "?*"
```

Example 2 — Kyverno: mutate add seccomp default

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: add-seccomp }
spec:
  rules:
  - name: add-seccomp
    match: { any: [{ resources: { kinds: ["Pod"] } }] }
    mutate:
      patchStrategicMerge:
        spec:
          securityContext:
            seccompProfile:
              type: RuntimeDefault
```

Example 3 — Kyverno: restrict image registries

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: restrict-registries }
spec:
  validationFailureAction: Enforce
  rules:
  - name: allowed-registries
    match: { any: [{ resources: { kinds: ["Pod"] } }] }
    validate:
      message: "Images must come from approved registries."
      pattern:
        spec:
          containers:
          - image: "gcr.io/myproj/* | asia-south1-docker.pkg.dev/myproj/* | *.dkr.ecr.ap-south-1.amazonaws.com/*"
```

Example 4 — OPA Gatekeeper: require resource limits

```
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata: { name: k8srequiredlimits }
spec:
  crd: { spec: { names: { kind: K8sRequiredLimits } } }
  targets:
  - target: admission.k8s.gatekeeper.sh
    rego: |
      package k8srequiredlimits
      violation[{"msg": msg}] {
        container := input.review.object.spec.containers[_]
        not container.resources.limits.cpu
        msg := sprintf("Container %v must have CPU limits", [container.name])
      }
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLimits
metadata: { name: require-limits }
spec:
  match: { kinds: [{ apiGroups: [""], kinds: ["Pod"] }] }
```

Example 5 — Sign + verify with cosign

```
# Sign
cosign sign --key cosign.key myrepo/app:1.0

# Verify (Kyverno does this automatically via verifyImages)
cosign verify --key cosign.pub myrepo/app:1.0
```

💻 Related Commands

Command	Purpose	Example
`kubectl get clusterpolicy`	List Kyverno policies	`kubectl get cpol`
`kubectl get policyreport -A`	Kyverno audit results	
`kubectl get constraint`	OPA Gatekeeper constraints	`kubectl get constraint`
`kubectl get constrainttemplate`	Gatekeeper templates	
`kubectl describe cpol require-labels`	See policy status	
`kubectl get validatingwebhookconfiguration`	Active webhooks	
`kubectl get mutatingwebhookconfiguration`	Active mutation hooks	
`gcloud container binauthz policy export`	GKE Binary Auth	
`cosign sign --key <k> <image>`	Sign image	

🚨 Scenarios

Scenario 1 — Kyverno webhook down → all pod creation fails

- **Cause**: validating webhook has `failurePolicy: Fail`.
- **Fix**: set `failurePolicy: Ignore` for non-critical policies; monitor webhook health; set `webhookTimeoutSeconds`.

Scenario 2 — Developer deploys unsigned image

- **Cause**: CI pipeline didn't run cosign step; Kyverno `verifyImages` rejects.
- **Fix**: fix CI pipeline to sign after push. For emergency: exempt namespace temporarily.

Scenario 3 — OPA Rego policy has a bug → legitimate pods rejected

- **Fix**: test policies with `gator test` or `conftest` before applying. Use `dryRun: true` on constraints.

Scenario 4 — Binary Authorization blocks emergency hotfix

- **Cause**: hotfix image not attested by Cloud Build.
- **Fix**: use break-glass annotation: `alpha.image-policy.k8s.io/break-glass: "true"` (GKE-specific).

☁ GCP GKE Perspective

- **Binary Authorization**: Google-managed attestation. Integrates with Cloud Build, Artifact Analysis.
- **Policy Controller** (GKE Enterprise): OPA-based with a library of pre-built policy bundles.
- **Artifact Analysis**: auto-scans images pushed to Artifact Registry.

▩ AWS EKS Perspective

- **ECR image scanning**: on-push or on-demand via Inspector.
- **Kyverno/OPA**: install manually; no EKS-managed equivalent.
- **AWS Signer + Notation**: alternative to cosign for image signing.
- **GuardDuty EKS Protection**: runtime anomaly detection (complements admission).

🧪 Labs

Lab 45.1 — Kyverno require labels + restrict registries (Beginner, 45 min)

1. Install Kyverno: `kubectl apply -f https://raw.githubusercontent.com/kyverno/kyverno/main/config/install.yaml`.
2. Apply Examples 1 + 3.
3. Try deploying without `team` label → rejected.
4. Try deploying from `docker.io` → rejected.
5. Deploy compliant pod → accepted.

Lab 45.2 — OPA Gatekeeper require limits (Intermediate, 60 min)

1. Install Gatekeeper: `kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/master/deploy/gatekeeper.yaml`.
2. Apply Example 4 ConstraintTemplate + Constraint.
3. Deploy pod without limits → rejected.
4. `kubectl get constraint require-limits -o yaml` → see violations.

Lab 45.3 — cosign sign + Kyverno verifyImages (Advanced, 90 min)

1. Generate cosign key pair: `cosign generate-key-pair`.
2. Build + push image. Sign: `cosign sign --key cosign.key <image>`.
3. Apply Kyverno verifyImages policy with public key.
4. Deploy signed image → allowed. Deploy unsigned → rejected.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Image security covers scanning, signing, and admitting container images. Admission controllers — mutating and validating webhooks — are the enforcement point. Kyverno and OPA/Gatekeeper are the two main policy engines; ValidatingAdmissionPolicy with CEL is the emerging K8s-native option.
Why: Running an unscanned or unsigned image in production is like deploying unreviewed code. Admission controllers are the last gate before etcd — they enforce that every pod meets the org's security standards: approved registries, signed images, required labels, resource limits, security context.
How: I use Kyverno for its simplicity — YAML policies for validation, mutation, image verification, and resource generation. My baseline policies: require `team` + `env` labels, restrict registries to our private Artifact Registry/ECR, verify cosign signatures, mutate missing seccomp profiles. On EKS I complement with Binary Authorization for build-pipeline attestation. On GKE I combine Kyverno with ECR image scanning.
Example: After a CVE in a base image caused a production incident, we added two controls: Trivy in CI that blocks builds with Critical CVEs, and a Kyverno `verifyImages` policy that rejects any image not signed by our CI pipeline. Since then, zero unsigned images have reached production."

? Interview Questions

Q1. Mutating vs validating admission? Mutating modifies the request (add labels, inject sidecar). Validating accepts/rejects. Mutating runs first. **Q2. Kyverno vs OPA?** Kyverno: YAML-native, validate+mutate+generate+verifyImages. OPA: Rego language, more powerful but steeper learning curve. **Q3. What is `failurePolicy: Fail` vs `Ignore`?** Fail = if webhook is down, reject all requests (safer but can block cluster). Ignore = allow if webhook unreachable (less safe). **Q4. What is Binary Authorization?** GKE service that only admits images with valid attestations from trusted authorities. **Q5. How do you sign images?** `cosign sign --key <key> <image>`. Stores signature as an OCI artifact alongside the image. **Q6. ValidatingAdmissionPolicy — what is it?** K8s-native admission rules in CEL. No webhook server needed. GA in 1.30. **Q7. How do you audit existing violations?** Kyverno: `kubectl get policyreport -A`. Gatekeeper: `kubectl get constraint -o yaml` (violations section). **Q8. What is `verifyImages` in Kyverno?** A rule type that checks cosign/Notation signatures and attestations on images before admitting pods. **Q9. How do you prevent webhook from blocking the cluster?** `failurePolicy: Ignore` for non-critical; `namespaceSelector` to exclude `kube-system`; health checks + PDB on webhook pods. **Q10. Name 3 built-in admission controllers.** NamespaceLifecycle, LimitRanger, PodSecurity (PSA).

MODULE

Module 8 — Observability

🗐 Topic 46 — Logging (kubectl logs, EFK, Cloud Logging, CloudWatch)

Official Documentation Reference - Logging architecture: https://kubernetes.io/docs/concepts/cluster-administration/logging/ - kubectl logs: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_logs/ - GKE Cloud Logging: https://cloud.google.com/kubernetes-engine/docs/how-to/logging - EKS CloudWatch: https://docs.aws.amazon.com/eks/latest/userguide/logging-monitoring.html

Definition

Logging in Kubernetes captures container stdout/stderr, node-level system logs, and cluster component logs. Kubernetes does not ship a built-in log aggregation system — it relies on the container runtime to write logs to files on the node, and on external stacks (EFK, Loki, Cloud Logging, CloudWatch) to collect, store, and query them.

About

Where logs live

Source	Location on node	Who writes
Container stdout/stderr	`/var/log/containers/*.log` (symlinks to `/var/log/pods/`)	containerd log driver
kubelet	`journalctl -u kubelet`	systemd
kube-apiserver, etcd, scheduler	`/var/log/` or `journalctl` (kubeadm: container logs)	static pods
Audit logs	`--audit-log-path` or audit webhook	API server

Log rotation

containerd rotates container logs when they exceed `--container-log-max-size` (default 10Mi) and keeps `--container-log-max-files` (default 5). On GKE/EKS the runtime manages this.

Aggregation patterns

Pattern	How
Node-level agent (DaemonSet)	Fluent Bit / Fluentd / Filebeat reads `/var/log/containers/` → ships to backend
Sidecar streaming	Sidecar reads app log file → streams to stdout (captured by node agent)
Sidecar shipping	Sidecar (Fluent Bit) reads log file → ships directly to backend
Direct push	App sends logs via API (e.g., OTLP) — bypasses files

Popular stacks

Stack	Components	Best for
EFK	Elasticsearch + Fluent Bit/Fluentd + Kibana	Self-hosted, full-text search
PLG (Grafana Loki)	Promtail/Fluent Bit + Loki + Grafana	Lightweight, label-indexed (like Prometheus for logs)
Cloud Logging (GCP)	Built-in on GKE; auto-collects	GKE zero-ops
CloudWatch (AWS)	Fluent Bit DaemonSet → CloudWatch Logs	EKS native
Datadog / Splunk / New Relic	Agent DaemonSet → SaaS	Enterprise observability

Structured logging best practices

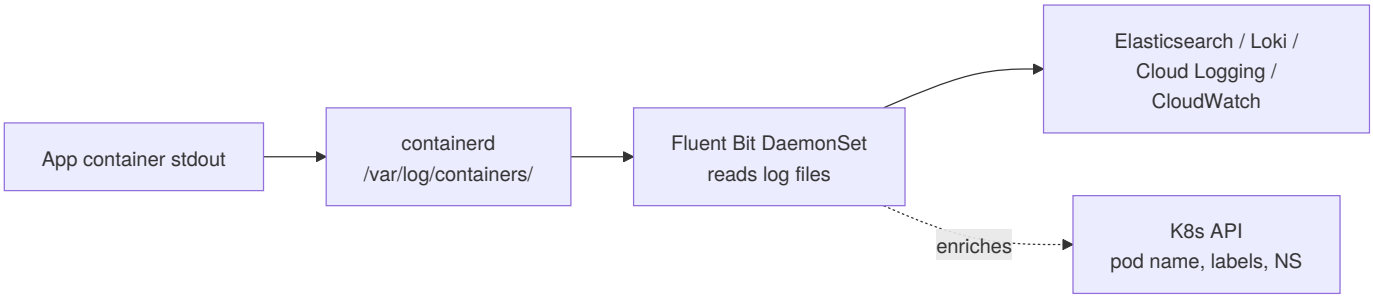
- Write JSON to stdout (not plain text).
- Include `timestamp` , `level` , `message` , `service` , `trace_id` .
- Avoid multi-line logs (or configure parser for stack traces).
- Use K8s metadata (pod name, namespace) injected by the log agent.

How It Integrates

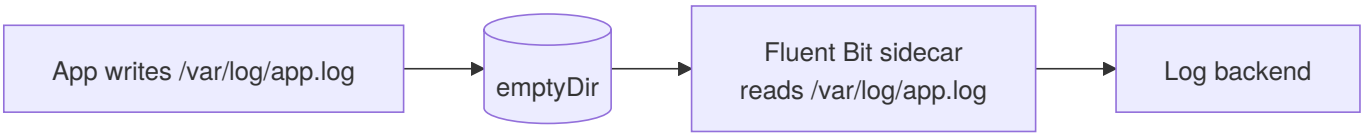
Component	Interaction
containerd	Writes container logs to `/var/log/containers/`
DaemonSet agent	Reads node logs, enriches with K8s metadata, ships
CoreDNS	Agent needs DNS to reach log backend
NetworkPolicy	Must allow agent egress to log backend
RBAC	Agent SA needs `get/list/watch pods/namespaces` for metadata enrichment
PriorityClass	Agent DaemonSet should be `system-node-critical` to survive eviction

🏗 Diagrams

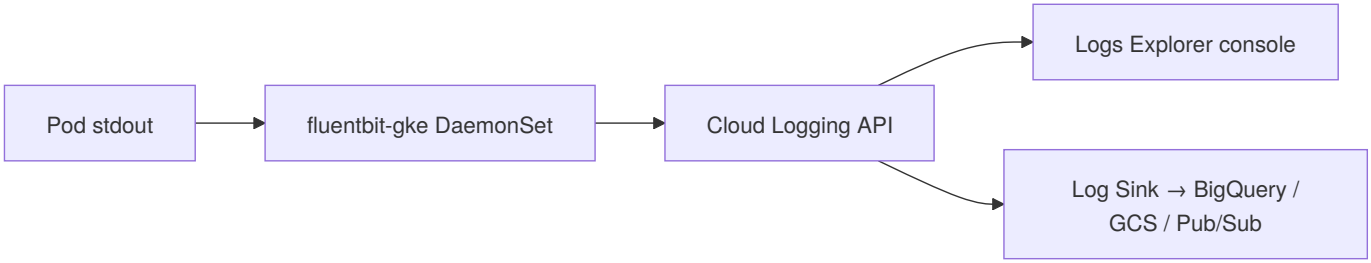
Node-level agent pattern



Sidecar pattern



GKE Cloud Logging



💻 Real-World Examples

Example 1 — kubectl log commands

```
kubectl logs web-0                          # current container
kubectl logs web-0 -c sidecar               # specific container
kubectl logs web-0 --previous               # prior crash
kubectl logs -f web-0                       # follow (stream)
kubectl logs -l app=web --all-containers    # all pods matching label
kubectl logs -l app=web --tail=100          # last 100 lines per pod
kubectl logs -l app=web --since=1h          # last hour
kubectl logs job/db-migrate                 # aggregate job pod logs
```

Example 2 — stern (multi-pod tailing)

```
stern web -n prod                           # tail all pods matching "web"
stern . -n prod --since 5m                  # all pods, last 5 min
stern web -n prod -o json                   # JSON output
```

Example 3 — Fluent Bit DaemonSet (basic)

```
apiVersion: apps/v1
kind: DaemonSet
metadata: { name: fluent-bit, namespace: logging }
spec:
  selector: { matchLabels: { app: fluent-bit } }
  template:
    metadata: { labels: { app: fluent-bit } }
    spec:
      serviceAccountName: fluent-bit
      tolerations: [{ operator: Exists }]
      priorityClassName: system-node-critical
      containers:
      - name: fb
        image: fluent/fluent-bit:3.1
        volumeMounts:
        - { name: varlog, mountPath: /var/log, readOnly: true }
        - { name: config, mountPath: /fluent-bit/etc }
      volumes:
      - { name: varlog, hostPath: { path: /var/log } }
      - { name: config, configMap: { name: fluent-bit-config } }
```

Example 4 — GKE: query logs in Cloud Logging

```
gcloud logging read 'resource.type="k8s_container" AND resource.labels.namespace_name="prod" AND resource.labels.container_name="api"' --limit=20 --format=json
```

Example 5 — EKS: CloudWatch with Fluent Bit

```
# Install via EKS add-on or Helm
helm repo add aws https://aws.github.io/eks-charts
helm install aws-for-fluent-bit aws/aws-for-fluent-bit \
  -n logging --create-namespace \
  --set cloudWatch.region=ap-south-1 \
  --set cloudWatch.logGroupName=/eks/prod
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl logs`	`<pod>`	View logs	`kubectl logs web-0`
`kubectl logs`	`-f`	Follow/stream	`kubectl logs -f web-0`
`kubectl logs`	`-c <container>`	Specific container	`kubectl logs web-0 -c sidecar`
`kubectl logs`	`--previous`	Prior crash	`kubectl logs web-0 --previous`
`kubectl logs`	`-l app=web`	Label selector	`kubectl logs -l app=web --all-containers`
`kubectl logs`	`--since=1h`	Time window	`kubectl logs web-0 --since=1h`
`kubectl logs`	`--tail=100`	Last N lines	`kubectl logs web-0 --tail=100`
`stern`	`<pattern>`	Multi-pod tail	`stern web -n prod`
`kubectl logs`	`job/<name>`	Job pod logs	`kubectl logs job/migrate`

🚨 Scenarios

Scenario 1 — `kubectl logs` shows nothing

- **Cause**: pod not writing to stdout/stderr (logging to a file instead).
- **Fix**: redirect app logs to stdout; or add sidecar to tail the log file to stdout.

Scenario 2 — Logs lost after pod restart

- **Cause**: `--previous` shows last crash only; older logs rotated off node.
- **Fix**: centralize logs to EFK/Loki/CloudWatch before rotation.

Scenario 3 — Fluent Bit OOMKilled

- **Cause**: parsing spike or large log burst.
- **Fix**: increase memory limits; tune `Mem_Buf_Limit`; rate-limit noisy namespaces.

Scenario 4 — Cloud Logging costs too high on GKE

- **Cause**: verbose debug logging from all pods.
- **Fix**: use log exclusion filters; set `--logging=NONE` on non-critical node pools; route to cheaper sinks (GCS).

Scenario 5 — CloudWatch log group missing on EKS

- **Cause**: Fluent Bit doesn't auto-create log groups by default.
- **Fix**: set `autoCreateGroup: true` in Fluent Bit CloudWatch config; or pre-create with `aws logs create-log-group`.

☁ GCP GKE Perspective

- **Auto-logging**: GKE installs `fluentbit-gke` DaemonSet by default. Logs appear in Cloud Logging automatically.
- **System vs workload logs**: configurable at cluster level (`--logging=SYSTEM,WORKLOAD`).
- **Log sinks**: route to BigQuery (analytics), GCS (archive), Pub/Sub (streaming).
- **Autopilot**: logging always on; no DaemonSet management.
- **Cloud Logging pricing**: first 50 GiB/month free; then ~$0.50/GiB.

▩ AWS EKS Perspective

- **Control-plane logs**: enable api/audit/authenticator/controllerManager/scheduler → CloudWatch `/aws/eks/<cluster>/cluster`.
- **Container logs**: install Fluent Bit DaemonSet → CloudWatch Logs.
- **Container Insights**: enhanced metrics + logs for EKS (enable via `aws eks create-addon --addon-name amazon-cloudwatch-observability`).
- **Fargate**: built-in Fluent Bit sidecar support; configure via `fluentbit.conf` in a ConfigMap.
- **CloudWatch Logs pricing**: ~$0.50/GiB ingestion; $0.03/GiB storage/month.

🧪 Hands-On Lab Exercises

Lab 46.1 — kubectl logs mastery (Beginner, 20 min)

```
kubectl create deploy web --image=nginx --replicas=3
kubectl logs -l app=web --all-containers --tail=5
kubectl logs -l app=web -f
# Force a crash
kubectl set image deploy/web nginx=nginx:not-real
kubectl logs -l app=web --previous
```

Lab 46.2 — Grafana Loki stack (Intermediate, 60 min)

1. `helm install loki grafana/loki-stack --set grafana.enabled=true -n logging --create-namespace`.
2. Port-forward Grafana: `kubectl port-forward svc/loki-grafana 3000:80 -n logging`.
3. Add Loki data source. Query: `{namespace="default", container="nginx"}`.
4. Deploy an app that writes JSON logs → query by JSON fields.

Lab 46.3 — Fluent Bit → CloudWatch on EKS (Advanced, 60 min)

1. Install Fluent Bit Helm chart (Example 5).
2. Verify: `aws logs describe-log-groups --log-group-name-prefix=/eks/prod`.
3. Query: `aws logs filter-log-events --log-group-name=/eks/prod --filter-pattern="ERROR"`.
4. Set up CloudWatch Insights query: `fields @timestamp, @message | filter @message like /ERROR/ | sort @timestamp desc`.

🎙 Interview Deliverable Answer (60–90 s)

"**What**: Kubernetes logging captures container stdout/stderr via the container runtime, stores them as files on the node, and relies on external stacks to aggregate, store, and query them. kubectl logs is the quick tool; production uses EFK, Grafana Loki, Cloud Logging, or CloudWatch.
Why: Containers are ephemeral — logs disappear on restart. Centralized logging is non-negotiable for debugging, auditing, and compliance. Structured JSON logs with trace IDs enable correlation across microservices.
How: On GKE, Cloud Logging works out of the box — I tune exclusion filters and sinks. On EKS I deploy Fluent Bit as a DaemonSet shipping to CloudWatch or Loki. I always set the agent's priorityClass to `system-node-critical` and tune `Mem_Buf_Limit` to survive log bursts. Apps write structured JSON to stdout with `level`, `service`, `trace_id`.
Example: We moved from EFK (self-managed Elasticsearch) to Grafana Loki on EKS. Log storage costs dropped 60% because Loki stores only labels + compressed chunks instead of full-text indexing. Query performance was fine for our use case."

? Interview Questions

Q1. Where do container logs live on a node? `/var/log/containers/` (symlinks to `/var/log/pods/`). **Q2.** `kubectl logs --previous` **— what does it show?** Logs from the prior container instance (last crash). **Q3. K8s ships a log stack?** No — you must deploy one (EFK, Loki, CloudWatch, etc.). **Q4. DaemonSet vs sidecar for logging?** DaemonSet: one agent per node, reads all containers. Sidecar: per-pod, reads specific files — more isolation. **Q5. Why structured JSON logs?** Enables field-based querying, alerting, and correlation without regex. **Q6. What is stern?** CLI that tails logs from multiple pods matching a pattern with color coding. **Q7. GKE default logging?** `fluentbit-gke` DaemonSet ships to Cloud Logging automatically. **Q8. EKS control-plane logs?** Disabled by default; enable per log type → CloudWatch. **Q9. How do you handle multi-line logs?** Configure parser (e.g., Fluent Bit `Multiline` parser for Java stack traces). **Q10. How do you reduce log costs?** Exclusion filters, sampling, shorter retention, cheaper sinks (GCS/S3), log levels (no DEBUG in prod).

📊 Topic 47 — Monitoring (Metrics Server, Prometheus, Grafana, GKE Managed Prometheus, CloudWatch Container Insights)

Official Documentation Reference - Metrics server: https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/ - Resource monitoring tools: https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-usage-monitoring/ - System metrics: https://kubernetes.io/docs/concepts/cluster-administration/system-metrics/ - GKE Managed Prometheus: https://cloud.google.com/stackdriver/docs/managed-prometheus - EKS Container Insights: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Container-Insights-EKS.html

---

🔷 Definition

Monitoring in Kubernetes collects, stores, and visualizes metrics — CPU, memory, network, custom application metrics — to enable autoscaling (HPA/VPA), alerting, capacity planning, and debugging.

🔷 About

Metrics pipeline layers

Layer	Component	Metrics
Resource metrics	Metrics Server	CPU/memory per pod/node (for HPA/VPA/ `kubectl top`)
Full metrics	Prometheus / cloud monitoring	Everything: container, node, app, kube-state, etcd
Custom metrics	Prometheus Adapter / Stackdriver Adapter	App-defined metrics for HPA
Dashboards	Grafana / Cloud Monitoring / CloudWatch	Visualization + alerting
Alerting	Alertmanager / PagerDuty / Cloud Alerting	Page on SLO breach

Metrics Server

- In-memory, lightweight aggregator of kubelet `/metrics/resource` endpoint.
- Required for `kubectl top` , HPA, VPA.
- NOT a monitoring solution — no storage, no history.

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl top pods -A
kubectl top nodes
```

Prometheus stack

Component	Purpose
Prometheus	Scrapes + stores time-series metrics (pull model)
kube-state-metrics	Exports K8s object state (deployment replicas, pod status) as metrics
node-exporter	Exports node hardware/OS metrics
Alertmanager	Routes alerts to Slack/PagerDuty/email
Grafana	Dashboards + visualization
Prometheus Adapter	Expose custom metrics for HPA via `custom.metrics.k8s.io` API

Install (kube-prometheus-stack Helm chart)

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prom prometheus-community/kube-prometheus-stack -n monitoring --create-namespace
```

Key PromQL queries

```
# CPU usage per pod
rate(container_cpu_usage_seconds_total{namespace="prod"}[5m])

# Memory usage
container_memory_working_set_bytes{namespace="prod"}

# CPU throttling
rate(container_cpu_cfs_throttled_seconds_total[5m])

# Pod restart count
kube_pod_container_status_restarts_total

# Node capacity vs allocatable
kube_node_status_capacity - kube_node_status_allocatable
```

ServiceMonitor / PodMonitor (Prometheus Operator)

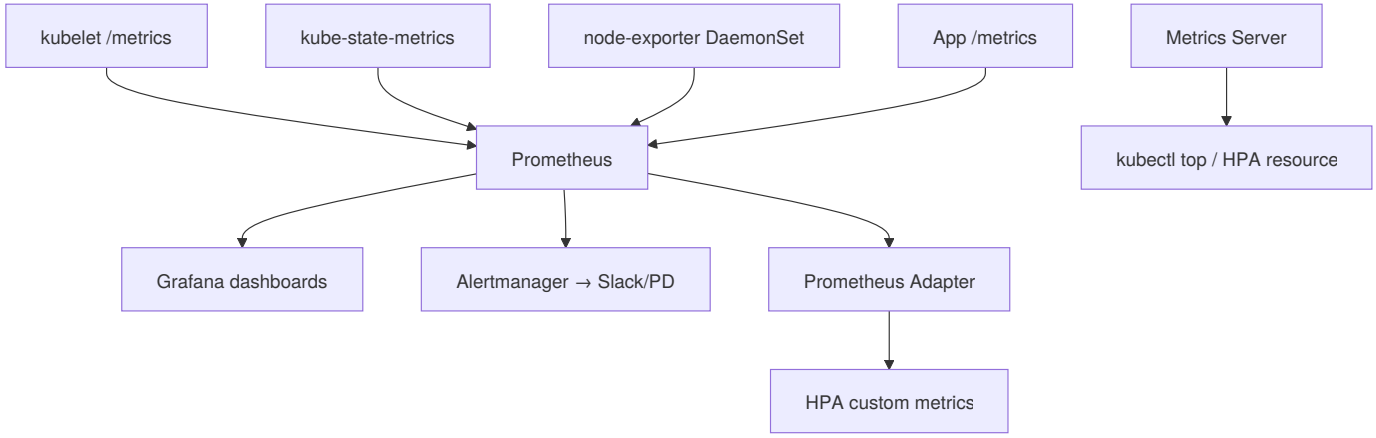
```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: api-monitor, namespace: monitoring }
spec:
  selector: { matchLabels: { app: api } }
  namespaceSelector: { matchNames: [prod] }
  endpoints:
  - { port: metrics, interval: 15s, path: /metrics }
```

How It Integrates

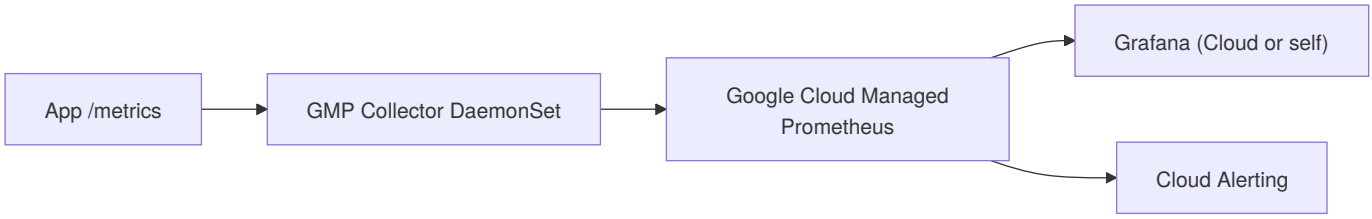
Component	Interaction
HPA	Reads from Metrics Server (resource) or Prometheus Adapter (custom)
VPA	Reads from Metrics Server
kubelet	Exposes `/metrics` , `/metrics/resource` , `/metrics/cadvisor`
Grafana	Queries Prometheus for dashboards
Alertmanager	Fires alerts based on Prometheus rules
Application	Exposes `/metrics` endpoint (Prometheus client library)

🏗 Diagrams

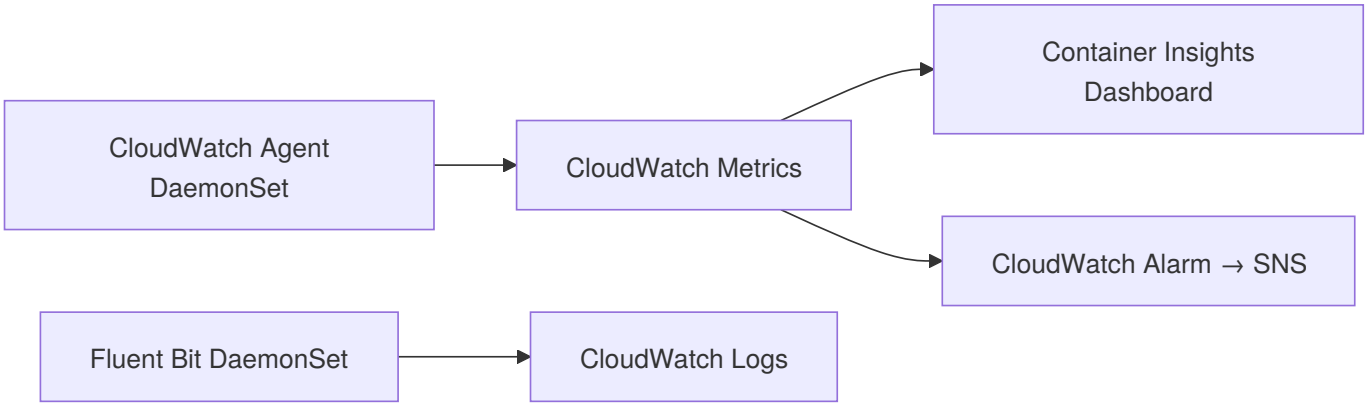
Monitoring architecture



GKE Managed Prometheus



EKS Container Insights



💻 Real-World Examples

Example 1 — Install Metrics Server on Minikube

```
minikube addons enable metrics-server
kubectl top nodes
kubectl top pods -A
```

Example 2 — kube-prometheus-stack

```
helm install kube-prom prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace \
  --set grafana.adminPassword=admin \
  --set prometheus.prometheusSpec.retention=7d
kubectl port-forward svc/kube-prom-grafana 3000:80 -n monitoring
```

Example 3 — GKE Managed Prometheus

```
gcloud container clusters update prod --region=asia-south1 --enable-managed-prometheus
# Prometheus targets auto-discovered via PodMonitoring CRD
kubectl apply -f - <<EOF
apiVersion: monitoring.googleapis.com/v1
kind: PodMonitoring
metadata: { name: api, namespace: prod }
spec:
  selector: { matchLabels: { app: api } }
  endpoints: [{ port: metrics, interval: 30s }]
EOF
```

Example 4 — EKS Container Insights

```
aws eks create-addon --cluster-name prod --addon-name amazon-cloudwatch-observability
```

Example 5 — App exposing /metrics (Go)

```
import "github.com/prometheus/client_golang/prometheus/promhttp"
http.Handle("/metrics", promhttp.Handler())
```

💻 Related Commands

Command	Purpose	Example
`kubectl top nodes`	Node CPU/mem	`kubectl top nodes`
`kubectl top pods`	Pod CPU/mem	`kubectl top pods -A --containers`
`kubectl get --raw /apis/metrics.k8s.io/v1beta1/pods`	Raw metrics API	
`kubectl port-forward svc/kube-prom-grafana 3000:80 -n monitoring`	Open Grafana	
`kubectl get servicemonitor -A`	List scrape targets	
`kubectl get prometheusrule -A`	List alert rules	
`promtool check rules rules.yaml`	Validate alert rules	

🚨 Scenarios

Scenario 1 — `kubectl top` returns "Metrics API not available"

- **Cause**: Metrics Server not installed.
- **Fix**: install Metrics Server.

Scenario 2 — HPA not scaling despite high CPU

- **Cause**: Metrics Server installed but pod has no `resources.requests` → utilization = actual/0 = undefined.
- **Fix**: set CPU requests on all HPA-target pods.

Scenario 3 — Prometheus OOMKilled

- **Cause**: too many series (cardinality explosion from high-cardinality labels).
- **Fix**: drop unused metrics via `metric_relabel_configs`; reduce scrape targets; increase memory; shard.

Scenario 4 — Alert fatigue from noisy alerts

- **Fix**: tune thresholds (burn-rate alerts for SLOs); add inhibition rules; group related alerts.

☁ GCP GKE Perspective

- **GKE Managed Prometheus (GMP)**: Google runs Prometheus for you. Use `PodMonitoring` CRD instead of `ServiceMonitor`. Query via Cloud Monitoring PromQL UI or Grafana with Prometheus data source pointing to GMP.
- **Cloud Monitoring**: auto-collects system metrics (CPU, mem, disk) from GKE nodes.
- **Autopilot**: GMP supported; system metrics auto-collected.

▩ AWS EKS Perspective

- **Container Insights**: CloudWatch add-on; collects node/pod/container metrics.
- **Amazon Managed Service for Prometheus (AMP)**: AWS-managed Prometheus. Write via remote-write; query via PromQL.
- **Amazon Managed Grafana (AMG)**: managed Grafana with AMP/CloudWatch data sources.
- **ADOT (AWS Distro for OpenTelemetry)**: alternative collector for metrics + traces → AMP/X-Ray/CloudWatch.

🧪 Labs

Lab 47.1 — Metrics Server + kubectl top (Beginner, 20 min)

Install Metrics Server. `kubectl top nodes`, `kubectl top pods`. Deploy stress pod, observe CPU spike.

Lab 47.2 — kube-prometheus-stack + Grafana (Intermediate, 60 min)

1. Helm install. 2. Port-forward Grafana. 3. Explore built-in dashboards (K8s cluster, node, pod). 4. Create ServiceMonitor for a custom app. 5. Write a PromQL query for p99 latency.

Lab 47.3 — GKE Managed Prometheus + custom metrics HPA (Advanced, 90 min)

1. Enable GMP on GKE. 2. Deploy app exposing `/metrics` with `http_requests_total`. 3. Apply PodMonitoring. 4. Verify metrics in Cloud Monitoring. 5. Configure HPA with custom metric via Stackdriver Adapter.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Kubernetes monitoring has two layers. Metrics Server provides lightweight CPU/memory for kubectl top and HPA. Prometheus provides full observability — scraping kubelet, kube-state-metrics, node-exporter, and application endpoints — with Grafana for dashboards and Alertmanager for alerts.
Why: Without monitoring you're flying blind. Autoscaling (HPA/VPA) depends on Metrics Server. Capacity planning, SLO tracking, and incident response depend on Prometheus or a cloud equivalent.
How: On GKE I use Managed Prometheus (GMP) with PodMonitoring CRDs and query via Cloud Monitoring or Grafana. On EKS I deploy kube-prometheus-stack or use Amazon Managed Service for Prometheus. I expose application metrics via `/metrics` endpoints with Prometheus client libraries and define ServiceMonitors to scrape them. Alerting rules follow the Google SRE burn-rate model — multi-window alerts that page only on meaningful SLO burn.
Example: We set up a Grafana dashboard showing p99 latency, error rate, and CPU throttling per service. When p99 crossed 500ms, Alertmanager paged the on-call. The dashboard traced the issue to CPU throttling — we raised limits and latency dropped within minutes."

? Interview Questions

Q1. Metrics Server vs Prometheus? Metrics Server: in-memory, resource metrics for top/HPA. Prometheus: persistent, full metrics, alerting, custom queries. **Q2. What does kube-state-metrics export?** K8s object state: deployment replicas, pod status, node conditions — not resource usage. **Q3. How does HPA get custom metrics?** Prometheus Adapter exposes custom metrics via `custom.metrics.k8s.io` API. **Q4. What is a ServiceMonitor?** Prometheus Operator CRD that tells Prometheus to scrape specific Services. **Q5. PromQL for CPU throttling?** `rate(container_cpu_cfs_throttled_seconds_total[5m])`. **Q6. GKE Managed Prometheus vs self-hosted?** GMP: Google runs Prometheus, no storage management. Use `PodMonitoring` CRD. Integrates with Cloud Alerting. **Q7. EKS Container Insights?** CloudWatch add-on collecting node/pod metrics + logs. **Q8. How do you reduce Prometheus cardinality?** Drop high-cardinality labels via `metric_relabel_configs`; aggregate in recording rules. **Q9. Alertmanager grouping/inhibition?** Group: batch related alerts into one notification. Inhibition: suppress lower-severity when higher fires. **Q10. How do you monitor the monitor?** Prometheus self-monitoring + `watchdog` alert (fires if Alertmanager is alive); dead-man's switch via PagerDuty.

📓 Topic 48 — Events and Debugging

Official Documentation Reference - Debug pods: https://kubernetes.io/docs/tasks/debug/debug-application/debug-pods/ - Debug running pods: https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/ - Get shell to running container: https://kubernetes.io/docs/tasks/debug/debug-application/get-shell-running-container/ - Determine reason for pod failure: https://kubernetes.io/docs/tasks/debug/debug-application/determine-reason-pod-failure/ - Events API: https://kubernetes.io/docs/reference/kubernetes-api/events/event-v1/

🔷 Definition

Events are short-lived Kubernetes objects that record state changes and errors — scheduling decisions, image pulls, probe failures, OOM kills, volume attach/detach. **Debugging** is the practice of using events, logs, describe, exec, debug, and port-forward to diagnose issues.

🔷 About

Events

- Stored in etcd with a default TTL of **1 hour**.
- Types: `Normal`, `Warning`.
- Key fields: `reason`, `message`, `involvedObject`, `count`, `firstTimestamp`, `lastTimestamp`.
- Not a monitoring system — use for immediate triage; persist via log sink for history.

Debugging toolkit

Tool	Use
`kubectl describe`	Object spec + events
`kubectl logs`	Container stdout/stderr (current + `--previous`)
`kubectl exec`	Shell into running container
`kubectl debug`	Ephemeral container into running pod or node
`kubectl port-forward`	Tunnel traffic to a pod/service
`kubectl cp`	Copy files to/from a pod
`kubectl events`	Better event viewer (K8s 1.26+)
`kubectl get --raw /debug/pprof/`	Profile API server
`crictl`	Debug at container runtime level on a node

Common error patterns

Error	Meaning	Debug path
`CrashLoopBackOff`	Container keeps exiting	`logs --previous` → exit code → fix app
`ImagePullBackOff`	Can't pull image	`describe` → image name, registry auth, network
`OOMKilled`	Memory exceeded limit	`describe` → Last State OOMKilled; raise limit or fix leak
`Pending`	Can't schedule	`describe` → events → resource/taint/affinity issue
`ContainerCreating`	Stuck mounting	`describe` → PVC/Secret/ConfigMap not found, CNI failure
`ErrImagePull`	Image doesn't exist	Check tag, registry, imagePullSecrets
`CreateContainerConfigError`	Bad env ref	Missing ConfigMap/Secret key
`FailedScheduling`	No fit node	Resource/taint/affinity mismatch

`kubectl debug` modes

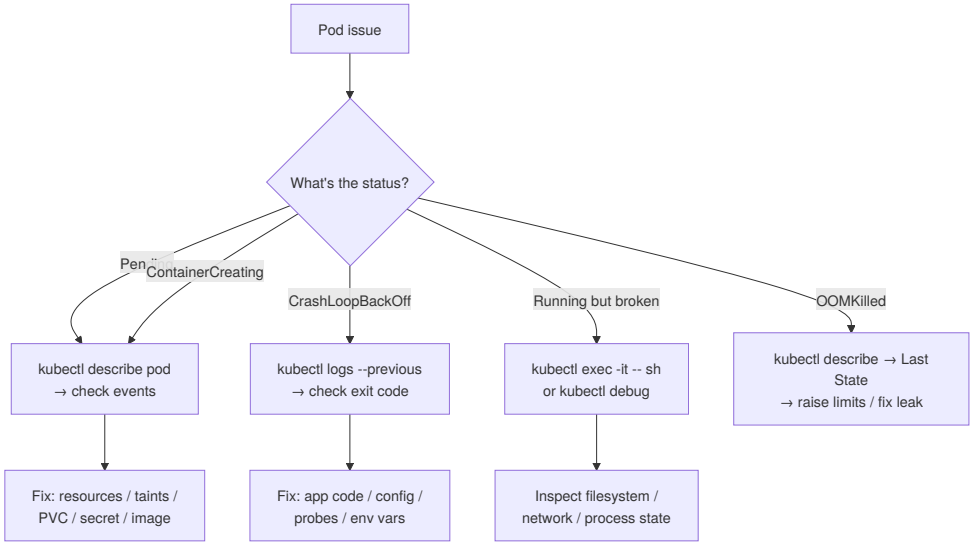
Mode	Command	Use
Ephemeral container	`kubectl debug -it <pod> --image=busybox --target=<container>`	Debug running pod without restart
Copy pod	`kubectl debug <pod> --copy-to=debug-pod --container=debug --image=busybox`	Debug with modified spec
Node debug	`kubectl debug node/<node> -it --image=busybox`	Privileged pod on node (chroot `/host`)

🔷 How It Integrates

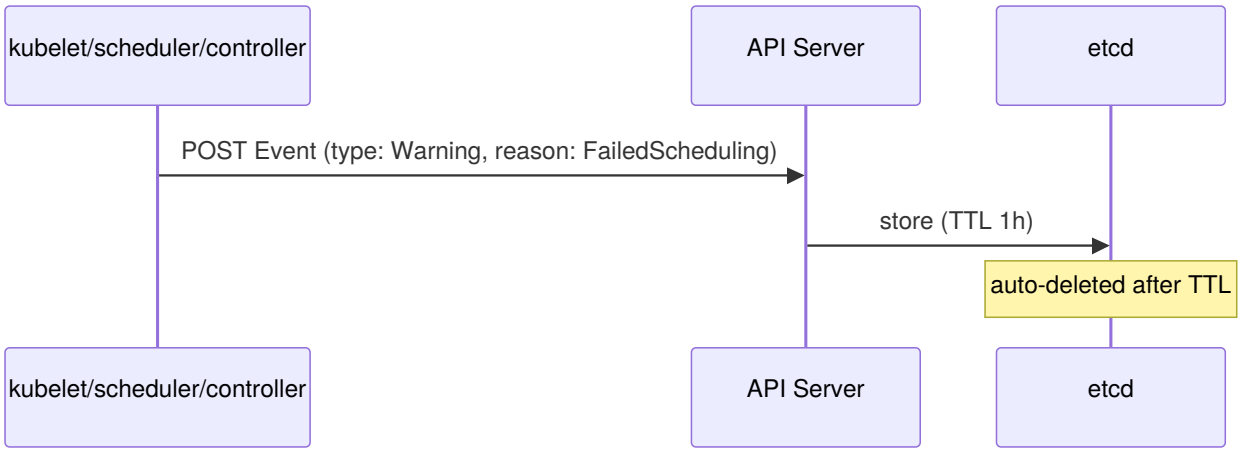
Component	Interaction
API server	Stores events; exposes debug endpoints
kubelet	Generates pod lifecycle events
Scheduler	Generates FailedScheduling events
kube-proxy	Generates Service sync events
Controllers	Generate events for owned objects

🏗 Diagrams

Debugging decision tree



Event lifecycle



💻 Real-World Examples

Example 1 — Full debug flow for CrashLoopBackOff

```
kubectl get pods -n prod          # STATUS: CrashLoopBackOff
kubectl describe pod web-0 -n prod | tail -30    # events + last state
kubectl logs web-0 -n prod --previous            # see crash output
kubectl logs web-0 -n prod -c init-migrate       # check init container
kubectl get events -n prod --sort-by=.lastTimestamp | head -20
kubectl debug -it web-0 -n prod --image=busybox --target=web  # inspect
```

Example 2 — Debug a node

```
kubectl debug node/worker-1 -it --image=ubuntu
# inside:
chroot /host
journalctl -u kubelet --since "10 min ago"
crictl ps -a
crictl logs <container-id>
df -h
free -m
exit
```

Example 3 — Events viewer (K8s 1.26+)

```
kubectl events -n prod --types=Warning
kubectl events -n prod --for=pod/web-0
kubectl events -A --types=Warning --sort-by=lastTimestamp
```

Example 4 — terminationMessagePath

```
spec:
  containers:
  - name: app
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: FallbackToLogsOnError
```

```
kubectl get pod web-0 -o jsonpath='{.status.containerStatuses[0].state.terminated.message}'
```

Example 5 — Port-forward for local debugging

```
kubectl port-forward pod/web-0 8080:80          # pod
kubectl port-forward svc/web 8080:80             # service (any backend)
kubectl port-forward deploy/web 8080:80          # deployment (any pod)
```

💻 Related Commands

Command	Purpose	Example
`kubectl describe pod`	Spec + events + status	`kubectl describe pod web-0`
`kubectl events`	Better event viewer	`kubectl events -n prod --types=Warning`
`kubectl get events`	Raw events list	`kubectl get events --sort-by=.lastTimestamp`
`kubectl logs`	Container logs	`kubectl logs web-0 --previous`
`kubectl exec -it`	Shell into container	`kubectl exec -it web-0 -- sh`
`kubectl debug -it`	Ephemeral debug container	`kubectl debug -it web-0 --image=busybox --target=web`
`kubectl debug node/`	Node debug pod	`kubectl debug node/worker-1 -it --image=ubuntu`
`kubectl port-forward`	Local tunnel	`kubectl port-forward svc/web 8080:80`
`kubectl cp`	Copy files	`kubectl cp web-0:/var/log/app.log ./app.log`
`kubectl get pod -o jsonpath`	Extract status	`kubectl get pod web-0 -o jsonpath='{.status.phase}'`

🚨 Scenarios

Scenario 1 — Pod in `CrashLoopBackOff` with exit code 137

- **Meaning**: 137 = 128 + 9 (SIGKILL) = OOMKilled.
- **Fix**: raise `resources.limits.memory`; check `kubectl describe pod` for OOMKilled in Last State.

Scenario 2 — Pod in `CrashLoopBackOff` with exit code 1

- **Meaning**: application error.
- **Fix**: `kubectl logs --previous` → read error message → fix config/code.

Scenario 3 — `CreateContainerConfigError`

- **Meaning**: ConfigMap or Secret referenced in env/volume doesn't exist.
- **Fix**: `kubectl describe pod` → events → create the missing CM/Secret.

Scenario 4 — Events show "Back-off restarting failed container" but logs are empty

- **Cause**: container exits before writing to stdout.
- **Fix**: use `terminationMessagePolicy: FallbackToLogsOnError`; or `kubectl debug` to inspect filesystem.

Scenario 5 — Need to debug a distroless image (no shell)

- **Fix**: `kubectl debug -it <pod> --image=busybox --target=<container>` — ephemeral container joins namespaces.

☁ GKE Perspective

- Events visible in Cloud Console → Workloads → Events tab.
- **Cloud Logging** persists events beyond 1 hour: `resource.type="k8s_cluster" AND protoPayload.methodName="io.k8s.core.v1.events.create"`.
- `kubectl debug node/` works on Standard. Not available on Autopilot (no node SSH).

▩ EKS Perspective

- Events visible via `kubectl events` or CloudWatch if control-plane logging enabled.
- `kubectl debug node/` works on managed NG nodes. Not on Fargate (no node access).
- **EKS Pod Identity troubleshooting**: `kubectl describe pod` → check IRSA/Pod Identity annotations + injected env vars.

🧪 Labs

Lab 48.1 — Debug CrashLoopBackOff (Beginner, 30 min)

Deploy a pod with a bad command. Use `describe`, `logs --previous`, `events` to diagnose. Fix and redeploy.

Lab 48.2 — Ephemeral debug container (Intermediate, 30 min)

Deploy a distroless pod. `kubectl debug -it <pod> --image=busybox --target=<c>`. Run `ps`, `wget`, `ls /proc/1/root/`.

Lab 48.3 — Node-level debug (Advanced, 45 min)

`kubectl debug node/<n> -it --image=ubuntu`. Inside: `chroot /host`, `journalctl -u kubelet`, `crictl ps -a`, `df -h`, `free -m`.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Events are short-lived K8s objects that record what happened — scheduling, pulling, probing, killing. Debugging uses events plus logs, describe, exec, debug, and port-forward to triage issues.
Why: When a pod misbehaves, events are the first clue — they tell you *what* happened. Logs tell you *why*. exec and debug let you inspect the live state.
How: My debug flow: 1) `kubectl get pods` → status. 2) `kubectl describe pod` → events section for scheduling/mount/pull errors. 3) `kubectl logs --previous` → crash output. 4) `kubectl debug -it` with a busybox image for distroless containers. 5) `kubectl port-forward` to hit the app locally. For node issues: `kubectl debug node/` → `chroot /host` → `journalctl -u kubelet`.
Example: A pod was CrashLooping with exit code 1. `logs --previous` showed a missing env var. `describe` showed the ConfigMap existed but the key was misspelled. Fixed the key → pod healthy in 10 seconds."

? Interview Questions

Q1. How long do events last? Default 1 hour (TTL in etcd). **Q2. Exit code 137?** 128+9 = SIGKILL = likely OOMKilled. **Q3. Exit code 1?** Application error. **Q4. `CrashLoopBackOff` vs `Error`?** CrashLoopBackOff = K8s is backing off on restarts after repeated failures. Error = container terminated with error. **Q5. How do you debug a distroless image?** `kubectl debug -it <pod> --image=busybox --target=<c>` — ephemeral container. **Q6. `kubectl debug node/` — what does it do?** Creates a privileged pod on the node with `chroot /host` access. **Q7. `terminationMessagePolicy: FallbackToLogsOnError`?** If termination message file is empty, use last chunk of container log as termination message. **Q8. Events say "Back-off pulling image" — steps?** Check image name, tag, registry auth (imagePullSecrets), network access to registry. **Q9. `CreateContainerConfigError`?** Missing ConfigMap/Secret referenced in env or volume. **Q10. How do you persist events beyond 1 hour?** Ship to Cloud Logging, CloudWatch, or Elasticsearch via event exporter.

📄 Topic 49 — Liveness, Readiness, Startup Probes

Official Documentation Reference - Probes: https://kubernetes.io/docs/concepts/workloads/pods/probes/ - Configure probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/ - GKE probes: https://cloud.google.com/kubernetes-engine/docs/concepts/probe - EKS best practices: https://docs.aws.amazon.com/eks/latest/best-practices/reliability.html

---

🔷 Definition

Probes are periodic health checks kubelet runs against containers. **Liveness** restarts a hung container. **Readiness** gates traffic to a ready container. **Startup** delays liveness/readiness until a slow-booting app is initialized.

🔷 About

Three probe types

Probe	Question	Action on failure	Added to Service endpoints?
Startup	"Has the app finished booting?"	Restart container	No (not yet ready)
Liveness	"Is the app alive?"	Restart container	N/A
Readiness	"Can the app serve traffic?"	Remove from Service endpoints	No

Probe mechanisms

Mechanism	How
`httpGet`	HTTP GET to `path:port`. Success = 200–399
`tcpSocket`	TCP connect to `port`. Success = connection established
`exec`	Run a command in container. Success = exit code 0
`grpc`	gRPC health check (K8s 1.27+ stable). Success = `SERVING`

Timing parameters

Parameter	Default	Meaning
`initialDelaySeconds`	0	Wait before first probe
`periodSeconds`	10	Interval between probes
`timeoutSeconds`	1	Probe must respond within
`successThreshold`	1	Consecutive successes for healthy
`failureThreshold`	3	Consecutive failures before action

Critical rules

1. **Always add readinessProbe** — without it, pods receive traffic before they're ready.
2. **Add startupProbe for slow-boot apps** (Java, ML model loading) — it prevents liveness from killing a healthy but slow app during init.
3. **Liveness should check simple things** — don't call external dependencies. A DB-down shouldn't restart your app.
4. **Readiness should check dependencies** — if DB is down, mark unready so traffic stops.
5. **Don't set `initialDelaySeconds` too high** — use startupProbe instead.

Probe flow

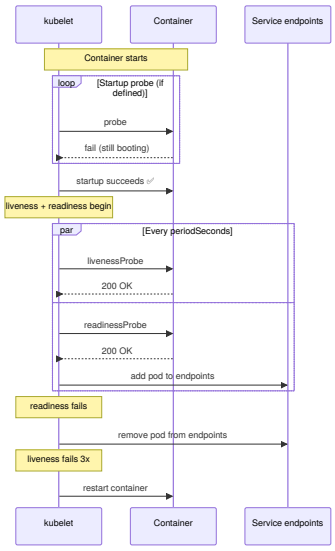
1. **Startup probe runs first** (if defined): liveness and readiness are disabled until startup succeeds.
2. Once startup succeeds (or if no startup probe), **liveness** and **readiness** run in parallel at `periodSeconds`.

🔷 How It Integrates

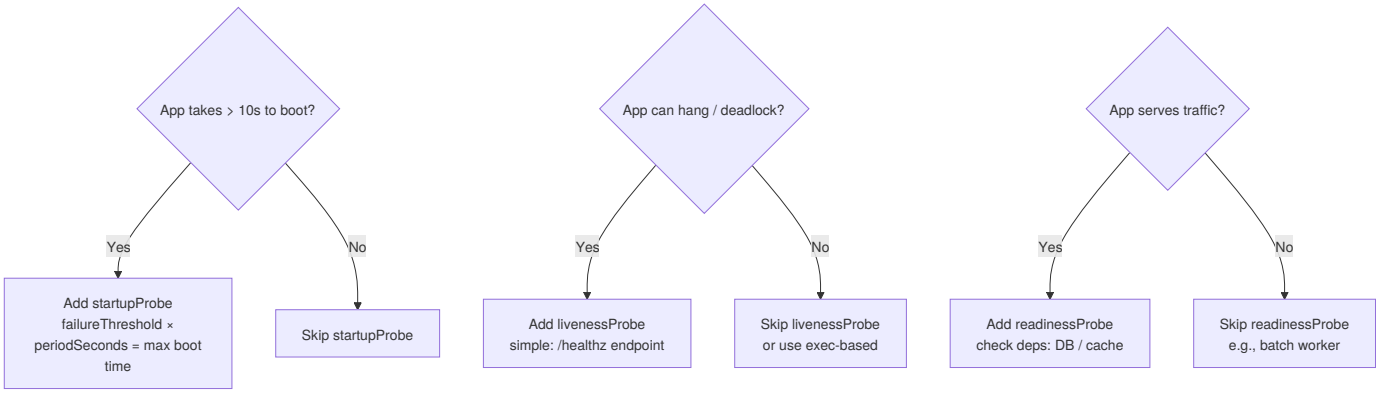
Component	Interaction
kubelet	Executes probes
Service / EndpointSlice	Readiness gates traffic
Deployment rollout	`minReadySeconds` + readiness determine rollout speed
HPA	Only Ready pods count toward utilization
PDB	Readiness affects disruption calculations

🏗 Diagrams

Probe lifecycle



Decision matrix



💻 Real-World Examples

Example 1 — All three probes (HTTP)

```
spec:
  containers:
  - name: api
    image: myrepo/api:2.0
    ports: [{ containerPort: 8080 }]
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      failureThreshold: 30        # 30 × 2s = 60s max boot
      periodSeconds: 2
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      periodSeconds: 10
      timeoutSeconds: 3
      failureThreshold: 3
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 2
```

Example 2 — gRPC probe

```
livenessProbe:
  grpc:
    port: 50051
    service: ""          # empty = overall health
  periodSeconds: 10
```

Example 3 — exec probe (database check)

```
readinessProbe:
  exec:
    command: ["pg_isready", "-U", "postgres"]
  periodSeconds: 5
```

Example 4 — TCP probe (Redis)

```
readinessProbe:
  tcpSocket: { port: 6379 }
  periodSeconds: 5
```

Example 5 — Separate liveness vs readiness endpoints

```
# /healthz — simple (no external deps)
@app.get("/healthz")
def healthz():
    return {"status": "alive"}

# /ready — checks dependencies
@app.get("/ready")
def ready():
    if not db.is_connected():
        raise HTTPException(503, "DB not ready")
    return {"status": "ready"}
```

💻 Related Commands

Command	Purpose	Example
`kubectl describe pod`	See probe config + failure events	`kubectl describe pod api-0`
`kubectl get pods`	READY column (0/1 = readiness failing)	`kubectl get pods`
`kubectl get endpoints`	See which pods are in endpoints	`kubectl get endpoints web`
`kubectl logs`	App logs during probe failure	`kubectl logs api-0`
`kubectl exec -- curl localhost:8080/healthz`	Manually test probe endpoint	

🚨 Scenarios

Scenario 1 — Liveness kills healthy slow-boot app

- **Cause**: no startupProbe; liveness `initialDelaySeconds` too short.
- **Fix**: add startupProbe with `failureThreshold × periodSeconds ≥ max boot time`.

Scenario 2 — Pod marked Ready before app is warm → 503 errors

- **Cause**: readinessProbe too lenient (succeeds before app loads cache).
- **Fix**: readiness checks a real functional endpoint; or add `minReadySeconds`.

Scenario 3 — Liveness probe depends on external DB → cascading restarts

- **Cause**: DB briefly down → liveness fails → all pods restart simultaneously.
- **Fix**: liveness should be self-contained (process alive, no deadlock). Readiness checks deps.

Scenario 4 — Pod oscillates between Ready and Not Ready

- **Cause**: readinessProbe flapping (intermittent dependency).
- **Fix**: increase `failureThreshold` and `successThreshold`; add jitter; fix dependency.

Scenario 5 — Deployment rollout stuck at 1 new pod

- **Cause**: new pod's readinessProbe never passes → Deployment won't continue.
- **Fix**: debug probe endpoint; check container logs; `kubectl port-forward` and test manually.

☁ GKE Perspective

- Probes work identically on Autopilot and Standard.
- **NEG health checks** (container-native LB): GKE uses readinessProbe config to create GCLB health checks automatically. Ensure probe path/port match.

▩ EKS Perspective

- Probes work identically on managed NG and Fargate.
- **ALB target group health checks**: AWS LB Controller derives health check from readinessProbe. Override with annotations if needed.

🧪 Labs

Lab 49.1 — Liveness restart demo (Beginner, 20 min)

```
apiVersion: v1
kind: Pod
metadata: { name: liveness-demo }
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh","-c","touch /tmp/healthy; sleep 30; rm /tmp/healthy; sleep 600"]
    livenessProbe:
      exec: { command: ["cat","/tmp/healthy"] }
      periodSeconds: 5
      failureThreshold: 3
```

After 30s file deleted → probe fails → container restarts. `kubectl describe pod liveness-demo` shows restart count.

Lab 49.2 — Readiness gates traffic (Intermediate, 30 min)

1. Deploy 3-replica Deployment with readinessProbe on `/ready`.
2. Create Service. `kubectl get endpoints` → 3 IPs.
3. `kubectl exec <pod> -- rm /tmp/ready` → `kubectl get endpoints` → 2 IPs.
4. Curl Service → only healthy pods respond.

Lab 49.3 — Startup probe for Java app (Advanced, 45 min)

1. Deploy a Java Spring Boot app (40s boot time).
2. Without startupProbe: liveness kills it during boot → CrashLoopBackOff.

3. Add startupProbe: `failureThreshold: 30, periodSeconds: 2` (60s window).
4. App boots successfully; liveness begins after startup succeeds.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Kubernetes has three probes. Liveness checks if the app is alive — failure triggers a restart. Readiness checks if the app can serve — failure removes it from Service endpoints. Startup delays the other two while a slow app boots.
Why: Without readiness, users hit pods that aren't ready yet — causing 503s during rollouts. Without liveness, a deadlocked process sits there forever without recycling. Without startup, liveness kills slow-booting apps like Java or ML services during their legitimate initialization.
How: My pattern is: readiness on every serving container (checks `/ready` including deps), liveness on `/healthz` (self-contained, no external deps), and startupProbe when boot exceeds 10 seconds. Liveness should never check the database — otherwise a brief DB outage cascades into pod restart storms. Separate `/healthz` (am I alive?) from `/ready` (can I serve?).
Example: Our Spring Boot API took 45 seconds to load a ML model. Without a startupProbe, liveness killed it 3 times before it could boot → CrashLoopBackOff. Adding `startupProbe: failureThreshold: 30, periodSeconds: 2` gave it a 60-second window. Problem solved, zero code changes."

? Interview Questions

Q1. Three probes? Liveness (restart), readiness (traffic gate), startup (boot gate). **Q2. What happens when readiness fails?** Pod removed from Service endpoints — no traffic. **Q3. What happens when liveness fails?** Container restarted. **Q4. Probe mechanisms?** httpGet, tcpSocket, exec, grpc. **Q5. Why not check DB in liveness?** DB outage → liveness fails → mass restart → worsens the outage. **Q6. What is startupProbe for?** Slow-booting apps. Delays liveness/readiness until app initializes. **Q7. How to calculate startup window?** `failureThreshold × periodSeconds ≥ max boot time`. **Q8.** `successThreshold` **for liveness?** Must be 1 (K8s enforces). For readiness, can be > 1 to avoid flapping. **Q9. How do probes affect rolling updates?** Deployment waits for new pod's readiness before continuing. Stuck readiness = stuck rollout. **Q10. gRPC probe — how?** `grpc: {port: 50051}` — K8s native since 1.27. App must implement gRPC health checking protocol.

📄 Topic 50 — Distributed Tracing Basics

Official Documentation Reference - Traces for K8s components: https://kubernetes.io/docs/concepts/cluster-administration/system-traces/ - OpenTelemetry: https://opentelemetry.io/docs/ - Jaeger: https://www.jaegertracing.io/docs/ - GKE Cloud Trace: https://cloud.google.com/trace/docs - EKS X-Ray: https://docs.aws.amazon.com/xray/latest/devguide/xray-services-eks.html - Grafana Tempo: https://grafana.com/docs/tempo/latest/

🔹 Definition

Distributed tracing follows a single request as it flows through multiple microservices, recording timing (spans) and metadata at each hop. A **trace** is a tree of **spans** connected by a shared trace ID — enabling you to see where latency lives and which service failed.

🔹 About

Key concepts

Concept	Meaning
Trace	End-to-end journey of one request
Span	One unit of work (one service call, one DB query)
Trace ID	Unique identifier linking all spans of a trace
Span ID	Unique identifier for one span
Parent Span ID	Links a span to its caller
Context propagation	Passing trace ID between services via HTTP headers (`traceparent`)
Baggage	Key-value pairs carried alongside trace context

Propagation headers

Standard	Header
W3C Trace Context (recommended)	`traceparent` , `tracestate`
B3 (Zipkin)	`X-B3-TraceId` , `X-B3-SpanId` , `X-B3-Sampled`
Jaeger	`uber-trace-id`

Tracing backends

Backend	Type	Best for
Jaeger	CNCF graduated	Self-hosted, rich UI
Zipkin	Open source	Lightweight
Grafana Tempo	Open source	Pairs with Loki + Grafana (no indexing, cheap)
Cloud Trace (GCP)	Managed	GKE native
X-Ray (AWS)	Managed	EKS/Lambda native
Datadog APM / New Relic	SaaS	Enterprise observability

OpenTelemetry (OTel)

The unified standard for instrumentation — **traces, metrics, and logs**. Components: - **OTel SDK** — instrument app code (auto or manual). - **OTel Collector** — receive, process, export telemetry data. - **OTLP protocol** — standard export format (gRPC/HTTP).

Instrumentation approaches

Approach	How	Effort
Auto-instrumentation	OTel agent/SDK hooks into frameworks (Spring, Express, Django)	Low
Manual instrumentation	Code creates spans with `tracer.start_span()`	Medium
Service mesh	Istio/Linkerd sidecars auto-generate spans for L7 calls	None (mesh handles)
eBPF-based	Cilium Tetragon / Pixie captures kernel-level traces	None (kernel hooks)

Sampling

At scale, tracing 100% of requests is expensive. Sampling strategies: - **Head-based**: decide at ingress (e.g., 10% of requests). - **Tail-based**: collect all spans, then decide post-hoc (keep errors, slow requests). - **Adaptive**: adjust rate based on traffic volume.

Three pillars integration

Pillar	How traces connect
Logs	Include `trace_id` in log fields → click from log to trace
Metrics	RED metrics (Rate, Error, Duration) derived from spans
Traces	The detailed path for one request

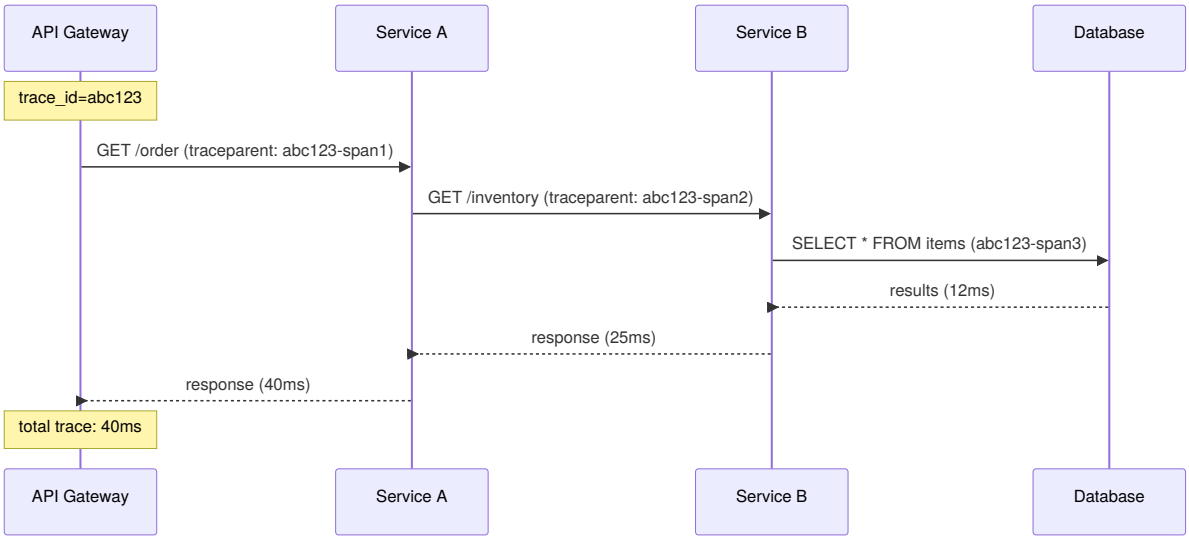
🔀 How It Integrates

Component	Interaction
Application	OTel SDK creates spans; propagates context
Service mesh	Sidecar auto-injects spans for inter-service calls
OTel Collector	Aggregates, samples, exports spans
Logging (Loki/CW)	`trace_id` field enables log-to-trace correlation
Prometheus	Span metrics (latency histograms) exported by OTel

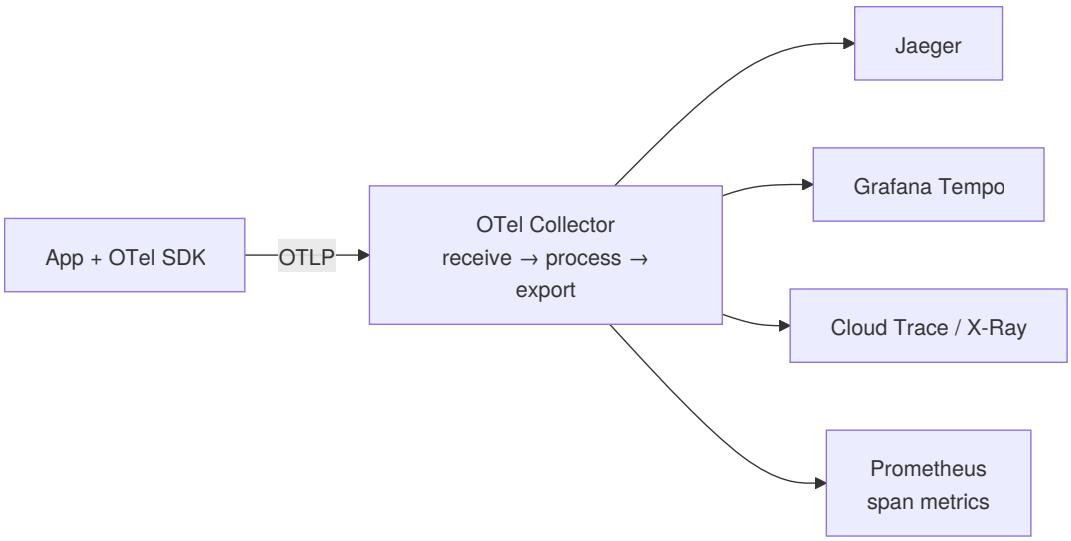
---

🏗 Diagrams

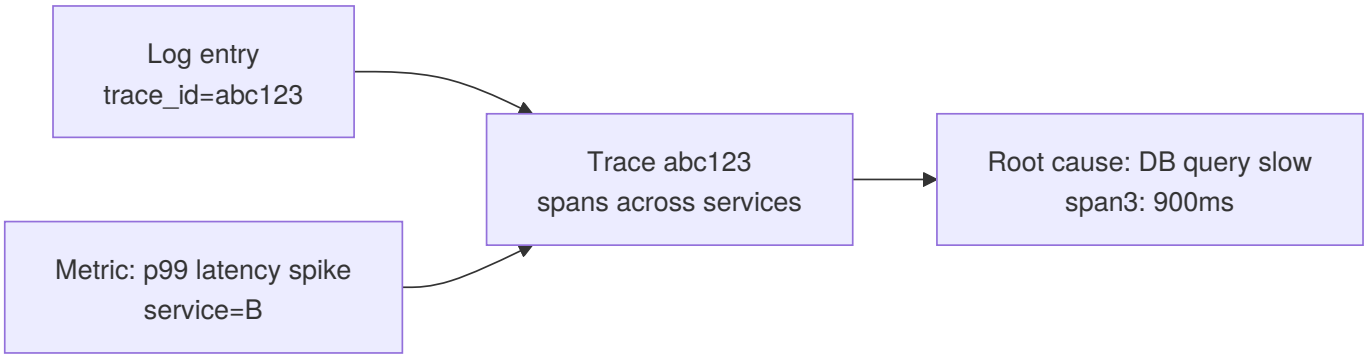
Trace through microservices



OTel Collector architecture



Three pillars correlation



💻 Real-World Examples

Example 1 — OTel Collector Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: otel-collector, namespace: observability }
spec:
  replicas: 2
  selector: { matchLabels: { app: otel-collector } }
  template:
    metadata: { labels: { app: otel-collector } }
    spec:
      containers:
      - name: collector
        image: otel/opentelemetry-collector-contrib:0.100.0
        args: ["--config=/etc/otel/config.yaml"]
        ports:
        - { containerPort: 4317, name: otlp-grpc }
        - { containerPort: 4318, name: otlp-http }
        volumeMounts: [{ name: config, mountPath: /etc/otel }]
      volumes:
      - name: config
        configMap: { name: otel-config }
---
apiVersion: v1
kind: ConfigMap
metadata: { name: otel-config, namespace: observability }
data:
  config.yaml: |
    receivers:
      otlp:
        protocols:
          grpc: { endpoint: 0.0.0.0:4317 }
          http: { endpoint: 0.0.0.0:4318 }
    processors:
      batch: {}
      tail_sampling:
        policies: [{ name: errors, type: status_code, status_code: { status_codes: [ERROR] } }]
    exporters:
      otlp/tempo:
        endpoint: tempo.observability.svc:4317
        tls: { insecure: true }
    service:
      pipelines:
        traces:
          receivers: [otlp]
          processors: [batch, tail_sampling]
          exporters: [otlp/tempo]
```

Example 2 — Python app with OTel auto-instrumentation

```
pip install opentelemetry-distro opentelemetry-exporter-otlp
opentelemetry-bootstrap -a install   # auto-detect frameworks
OTEL_SERVICE_NAME=order-svc \
OTEL_EXPORTER_OTLP_ENDPOINT=http://otel-collector.observability:4317 \
opentelemetry-instrument python app.py
```

Example 3 — Kubernetes env vars for OTel

```
env:
- { name: OTEL_SERVICE_NAME, value: order-svc }
- { name: OTEL_EXPORTER_OTLP_ENDPOINT, value: "http://otel-collector.observability:4317" }
- name: OTEL_RESOURCE_ATTRIBUTES
  value: "k8s.pod.name=$(POD_NAME),k8s.namespace.name=$(POD_NS)"
- name: POD_NAME
  valueFrom: { fieldRef: { fieldPath: metadata.name } }
- name: POD_NS
  valueFrom: { fieldRef: { fieldPath: metadata.namespace } }
```

Example 4 — Jaeger quick install

```
kubectl apply -f https://github.com/jaegertracing/jaeger-operator/releases/download/v1.57.0/jaeger-operator.yaml -n observability
kubectl apply -f - <<EOF
apiVersion: jaegertracing.io/v1
kind: Jaeger
metadata: { name: jaeger, namespace: observability }
spec:
  strategy: allInOne
EOF
kubectl port-forward svc/jaeger-query 16686:16686 -n observability
```

Example 5 — GKE Cloud Trace

```
# Auto-enabled on GKE. App uses OTel SDK with Google Cloud exporter:
pip install opentelemetry-exporter-gcp-trace
# Or configure OTel Collector with googlecloud exporter
```

Example 6 — EKS X-Ray with ADOT

```
# Install AWS Distro for OpenTelemetry add-on
aws eks create-addon --cluster-name prod --addon-name adot
# ADOT Collector exports to X-Ray
```

💻 Related Commands

Command	Purpose	Example
`kubectl port-forward svc/jaeger-query 16686:16686`	Open Jaeger UI	
`kubectl port-forward svc/tempo 3200:3200`	Tempo API	
`kubectl get pods -n observability`	Check collector/backend health	
`kubectl logs -n observability <collector-pod>`	Debug export errors	

🚨 Scenarios

Scenario 1 — Traces missing between two services

- **Cause**: context propagation broken — service B doesn't forward `traceparent` header.
- **Fix**: ensure all services instrument outgoing HTTP calls (OTel SDK auto-instruments most frameworks).

Scenario 2 — Trace storage costs exploding

- **Cause**: sampling at 100%.
- **Fix**: use tail-based sampling (keep errors + slow traces only); or head-based at 10%.

Scenario 3 — Can't correlate logs to traces

- **Cause**: app doesn't include `trace_id` in log output.
- **Fix**: configure OTel SDK to inject trace context into logging framework (MDC for Java, structlog for Python).

☁ GKE Perspective

- **Cloud Trace** auto-collects traces from GKE with OTel SDK or Google client libraries.
- **Managed Prometheus** + **Grafana** + **Tempo**: full open-source stack on GKE.
- **Autopilot**: OTel Collector runs as a Deployment; no DaemonSet needed for tracing.

▩ EKS Perspective

- **X-Ray**: native tracing service. ADOT Collector (EKS add-on) exports spans.
- **Amazon Managed Grafana + Tempo**: alternative open-source path.
- **Service map**: X-Ray auto-generates service dependency maps.

🧪 Labs

Lab 50.1 — Jaeger all-in-one (Beginner, 45 min)

1. Install Jaeger (Example 4). 2. Deploy a Python app with OTel auto-instrumentation (Example 2). 3. Generate traffic. 4. Open Jaeger UI → find traces → inspect spans.

Lab 50.2 — OTel Collector + Grafana Tempo (Intermediate, 60 min)

1. Install Tempo: `helm install tempo grafana/tempo -n observability` . 2. Deploy OTel Collector (Example 1). 3. Instrument 2 services. 4. Grafana → Explore → Tempo data source → search by trace ID.

Lab 50.3 — Log-to-trace correlation (Advanced, 90 min)

1. App writes structured JSON logs with `trace_id` field. 2. Logs → Loki. 3. Traces → Tempo. 4. Grafana: click a log line → "View trace" → opens Tempo trace. 5. Click a trace span → "View logs" → opens Loki query filtered by trace_id.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Distributed tracing follows a request through multiple microservices by propagating a trace ID. Each service creates a span — together they form a trace tree showing exactly where time is spent.
Why: Logs tell you what happened in one service. Metrics tell you something is slow. Traces tell you *where* in the call chain the slowness lives — was it the API, the auth service, or the database?
How: I standardize on OpenTelemetry for instrumentation — auto-instrumentation where possible, manual spans for critical paths. The OTel Collector receives OTLP, batches, samples (tail-based for errors and slow traces), and exports to Grafana Tempo or Cloud Trace/X-Ray. I inject `trace_id` into structured logs so Grafana can link logs ↔ traces ↔ metrics in one click.
Example: A payment endpoint had intermittent p99 spikes to 5 seconds. Metrics showed the spike but not the cause. A trace from Tempo revealed the bottleneck was a downstream fraud-check service that occasionally timed out. We added a circuit breaker + cached the result — p99 dropped to 200ms."

? Interview Questions

Q1. What is a trace? End-to-end journey of one request, composed of spans across services. **Q2. Trace ID vs Span ID?** Trace ID links all spans of one request. Span ID identifies one unit of work. **Q3. What is context propagation?** Passing trace ID between services — typically via `traceparent` header (W3C). **Q4. OpenTelemetry — what is it?** Unified standard for traces + metrics + logs. SDK + Collector + OTLP protocol. **Q5. Auto vs manual instrumentation?** Auto: agent hooks into frameworks, zero code. Manual: create spans in code for custom logic. **Q6. Head vs tail sampling?** Head: decide at ingress. Tail: collect all,

decide later (keep errors/slow). **Q7. How does a service mesh help tracing?** Sidecar proxy auto-generates spans for all inter-service calls. **Q8. Log-trace correlation?** Include `trace_id` in log JSON → Grafana links log line to trace. **Q9. Cloud Trace vs Jaeger?** Cloud Trace: GCP managed, auto-collects. Jaeger: self-hosted, CNCF graduated, more control. **Q10. Why not trace 100% of requests?** Storage/cost. Use tail-based sampling to keep errors + slow traces.

MODULE

Module 9 — Scaling & Reliability

🗐 Topic 51 — Horizontal Pod Autoscaler (HPA)

Official Documentation Reference - HPA concept: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/ - HPA walkthrough: https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/ - GKE HPA: https://cloud.google.com/kubernetes-engine/docs/concepts/horizontalpodautoscaler - EKS HPA: https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html

🔹 Definition

The **Horizontal Pod Autoscaler (HPA)** automatically scales the number of pod replicas in a Deployment, ReplicaSet, or StatefulSet based on observed CPU utilization, memory utilization, or custom/external metrics.

🔹 About

How it works

1. HPA controller (in kube-controller-manager) queries metrics every **15 seconds** (default `--horizontal-pod-autoscaler-sync-period`).
2. Calculates desired replicas: `desired = ceil(current × (currentMetric / targetMetric))`.
3. Scales up/down within `minReplicas` and `maxReplicas`.

Metric types

Type	Source	Example
Resource	Metrics Server	CPU utilization, memory utilization
Pods	Custom metrics API	`requests_per_second` per pod
Object	Custom metrics API	`queue_depth` on a Service
External	External metrics API	SQS queue length, Pub/Sub backlog

Scaling behavior (K8s 1.23+ stable)

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0        # scale up immediately
    policies:
    - type: Percent
      value: 100                         # double pods in one step
      periodSeconds: 60
  scaleDown:
    stabilizationWindowSeconds: 300      # wait 5 min before scaling down
    policies:
    - type: Percent
      value: 10                          # remove max 10% per minute
      periodSeconds: 60
```

Key fields

Field	Purpose
`minReplicas`	Floor (default 1)
`maxReplicas`	Ceiling
`metrics[].type`	Resource / Pods / Object / External
`metrics[].resource.target.type`	`Utilization` (%) or `AverageValue`
`behavior`	Fine-grained scale-up/down policies

Requirements

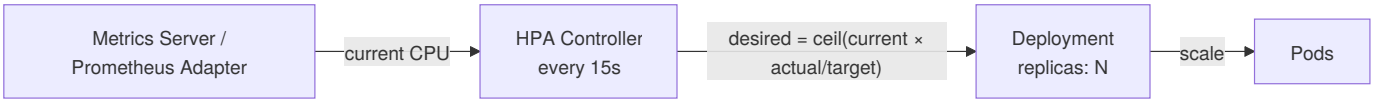
- **Metrics Server** must be installed (for resource metrics).
- **Prometheus Adapter** for custom metrics.
- **Stackdriver/CloudWatch Adapter** for external metrics.
- Pods **must have** `resources.requests` — HPA calculates utilization as `actual / request`.

HPA + VPA interaction

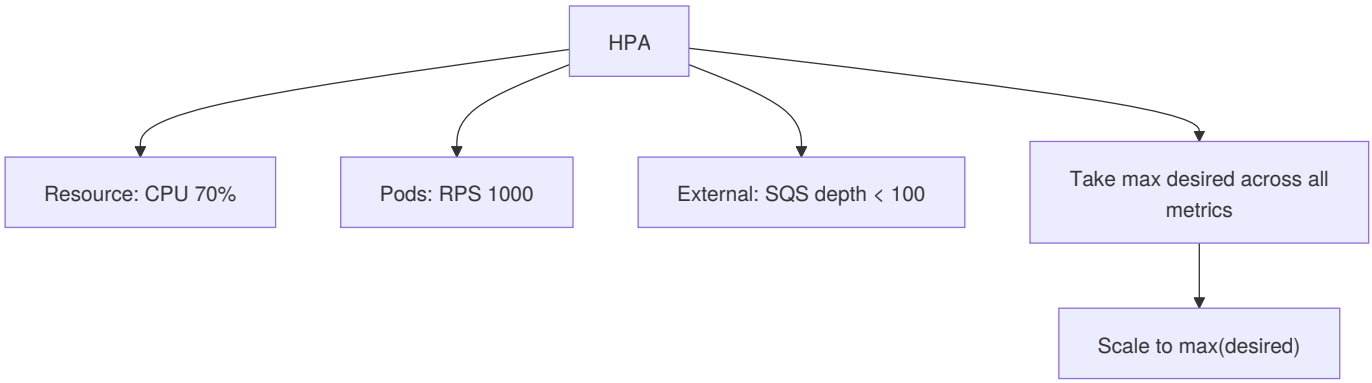
Running both on the same Deployment is tricky — HPA changes replicas, VPA changes requests. Best practice: use HPA for scaling, VPA in `Off` mode for recommendations only.

🏗 Diagrams

HPA control loop



Multi-metric HPA



💻 Real-World Examples

Example 1 — Simple CPU-based HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: web-hpa }
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }
```

Example 2 — Multi-metric (CPU + custom RPS)

```
metrics:
- type: Resource
  resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
- type: Pods
  pods:
    metric: { name: http_requests_per_second }
    target: { type: AverageValue, averageValue: "1000" }
```

Example 3 — External metric (SQS queue)

```
metrics:
- type: External
  external:
    metric:
      name: sqs_queue_length
      selector: { matchLabels: { queue: orders } }
    target: { type: AverageValue, averageValue: "50" }
```

Example 4 — Behavior tuning

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
    - { type: Pods, value: 4, periodSeconds: 60 }     # add max 4 pods/min
    - { type: Percent, value: 100, periodSeconds: 60 } # or double, whichever is larger
    selectPolicy: Max
  scaleDown:
    stabilizationWindowSeconds: 300
    policies:
    - { type: Percent, value: 10, periodSeconds: 60 }
```

Example 5 — Imperative HPA

```
kubectl autoscale deployment web --cpu-percent=70 --min=2 --max=20
kubectl get hpa
kubectl describe hpa web-hpa
```

💻 Related Commands

Command	Purpose	Example
`kubectl autoscale deploy`	Create HPA imperatively	`kubectl autoscale deploy web --cpu-percent=70 --min=2 --max=20`
`kubectl get hpa`	Status + current replicas	`kubectl get hpa`
`kubectl describe hpa`	Metrics + events + conditions	`kubectl describe hpa web-hpa`
`kubectl top pods`	Current CPU/mem	`kubectl top pods -l app=web`
`kubectl delete hpa`	Remove	`kubectl delete hpa web-hpa`

🚨 Scenarios

Scenario 1 — HPA shows `<unknown>` for CPU

- **Cause**: Metrics Server not installed or pods missing `resources.requests`.
- **Fix**: install Metrics Server; set CPU requests on all target pods.

Scenario 2 — HPA scales too aggressively → thrashing

- **Cause**: `stabilizationWindowSeconds: 0` for scale-down; traffic is bursty.
- **Fix**: set `scaleDown.stabilizationWindowSeconds: 300`.

Scenario 3 — HPA never scales down

- **Cause**: `stabilizationWindowSeconds` too long; or utilization stays above target.
- **Fix**: verify actual utilization with `kubectl top`; tune target percentage.

Scenario 4 — Custom metric HPA stuck at minReplicas

- **Cause**: Prometheus Adapter not configured for that metric; or metric name mismatch.
- **Fix**: `kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1` → verify metric exists.

☁ GKE Perspective

- HPA works on Autopilot and Standard.
- **GKE custom metrics**: use Stackdriver Adapter or GKE Managed Prometheus + Prometheus Adapter.
- **Autopilot**: min pod resources may affect utilization calculations.
- **Multidimensional Pod Autoscaler (MPA)**: GKE-specific CRD combining HPA + VPA.

▩ EKS Perspective

- Metrics Server installed via EKS add-on or Helm.
- **Custom metrics**: Prometheus Adapter or KEDA.
- **External metrics**: CloudWatch Adapter or KEDA with SQS/Kinesis triggers.
- **Karpenter**: complements HPA — HPA adds pods, Karpenter adds nodes.

🧪 Labs

Lab 51.1 — CPU-based HPA (Beginner, 30 min)

```
kubectl create deploy web --image=registry.k8s.io/hpa-example --requests='cpu=200m'
kubectl expose deploy web --port=80
kubectl autoscale deploy web --cpu-percent=50 --min=1 --max=10
# Load test
kubectl run load --image=busybox --rm -it -- sh -c "while true; do wget -qO- http://web; done"
# Watch
kubectl get hpa web -w
```

Lab 51.2 — Custom metric HPA with Prometheus (Intermediate, 90 min)

1. Deploy app exposing `http_requests_total`. 2. Install Prometheus + Prometheus Adapter. 3. Create HPA on `http_requests_per_second`. 4. Load test → observe scaling.

Lab 51.3 — External metric HPA with SQS on EKS (Advanced, 90 min)

1. Create SQS queue. 2. Install KEDA or CloudWatch Adapter. 3. Create HPA on SQS `ApproximateNumberOfMessages`. 4. Push 1000 messages → pods scale up. 5. Drain queue → pods scale down.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: HPA automatically scales pod replicas based on metrics — CPU, memory, custom (RPS), or external (queue depth). It queries metrics every 15 seconds and

calculates desired replicas using the formula `ceil(current × actual/target)`.
Why: Manual scaling doesn't respond to traffic. HPA matches capacity to demand — scaling up under load and down during quiet periods to save cost.
How: I always use `autoscaling/v2` for multi-metric support. CPU target at 70%, plus a custom metric like RPS for traffic-driven services, plus external metrics (SQS depth) for queue workers. I tune `behavior` — fast scale-up (0s stabilization), slow scale-down (300s window, 10%/min) to avoid thrashing. Pods must have `resources.requests` or HPA can't calculate utilization.
Example: On Black Friday our API HPA scaled from 5 to 45 replicas in under 3 minutes based on CPU. Cluster Autoscaler added 8 nodes to accommodate. After traffic subsided, the 300-second cooldown and 10%/min scale-down policy brought us back to 5 replicas over 20 minutes — no manual intervention."

? Interview Questions

Q1. HPA formula? `desired = ceil(current × actual/target)`. **Q2. What does HPA need to work?** Metrics Server (or custom API) + `resources.requests` on pods. **Q3.** `<unknown>` **in HPA — why?** Metrics Server missing or no requests set. **Q4. Metric types?** Resource, Pods, Object, External. **Q5. Multi-metric: which wins?** HPA takes the max desired across all metrics. **Q6. How to prevent thrashing?** `behavior.scaleDown.stabilizationWindowSeconds`. **Q7. HPA + VPA together?** Risky — HPA scales replicas, VPA changes requests (which changes utilization). Use VPA in Off mode. **Q8. Custom metrics source?** Prometheus Adapter or KEDA exposing via `custom.metrics.k8s.io`. **Q9. External metrics example?** SQS queue depth, Pub/Sub backlog, CloudWatch metric. **Q10. HPA on StatefulSet?** Supported but consider ordered scaling semantics and PVC lifecycle.

🗐 Topic 52 — Vertical Pod Autoscaler (VPA)

Official Documentation Reference - VPA: https://kubernetes.io/docs/concepts/workloads/autoscaling/vertical-pod-autoscale/ - VPA GitHub: https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler - GKE VPA: https://cloud.google.com/kubernetes-engine/docs/concepts/verticalpodautoscaler

---

⏥ Definition

The **Vertical Pod Autoscaler (VPA)** automatically adjusts CPU and memory **requests** (and optionally limits) for containers based on historical usage — right-sizing pods without manual guesswork.

⏥ About

Components

Component	Role
Recommender	Analyzes metrics history → generates recommendations
Updater	Evicts pods that need updated requests (Auto mode)
Admission Controller	Mutates new pods with recommended requests at creation

Update modes

Mode	Behavior	Use case
`Off`	Recommend only — no changes applied	Safest; review first
`Initial`	Apply recommendations only when pod starts (no evictions)	Non-disruptive
`Auto`	Apply on start + evict running pods to apply updates	Full automation
`Recreate`	Same as Auto (legacy name)	—

VPA output

```
kubectl describe vpa web-vpa
# Recommendation:
#   Container: web
#     Lower Bound:  cpu: 25m,  memory: 64Mi
#     Target:       cpu: 100m, memory: 128Mi
#     Upper Bound:  cpu: 500m, memory: 512Mi
#     Uncapped Target: cpu: 100m, memory: 128Mi
```

VPA + HPA

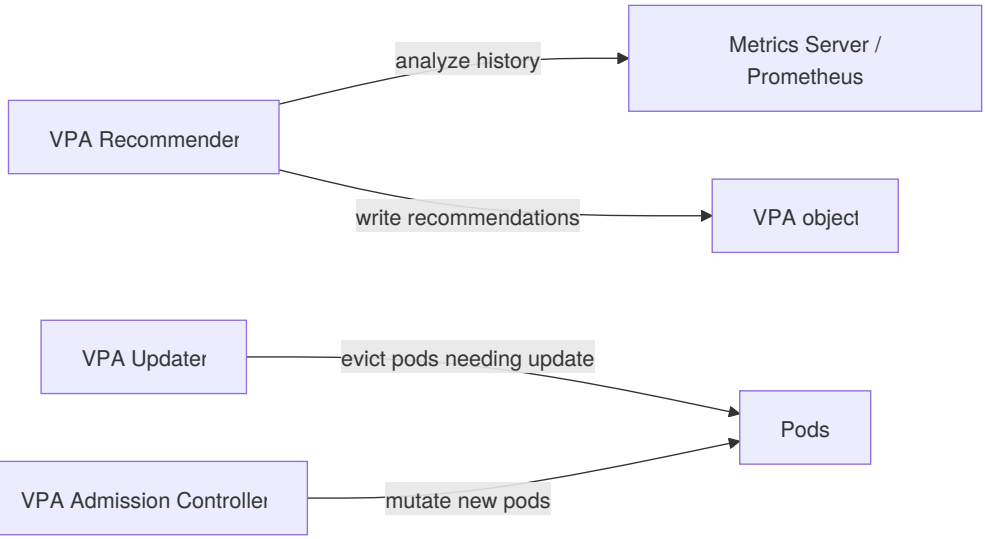
- **Don't run both on CPU for the same Deployment** — HPA scales replicas based on `actual/request`; VPA changes `request`, which changes HPA's denominator.
- **Safe combination**: HPA on custom metric (RPS), VPA on CPU/memory. Or VPA in `Off` mode + manual apply.
- **Multidimensional Pod Autoscaler (MPA)** on GKE handles both.

Resource policies

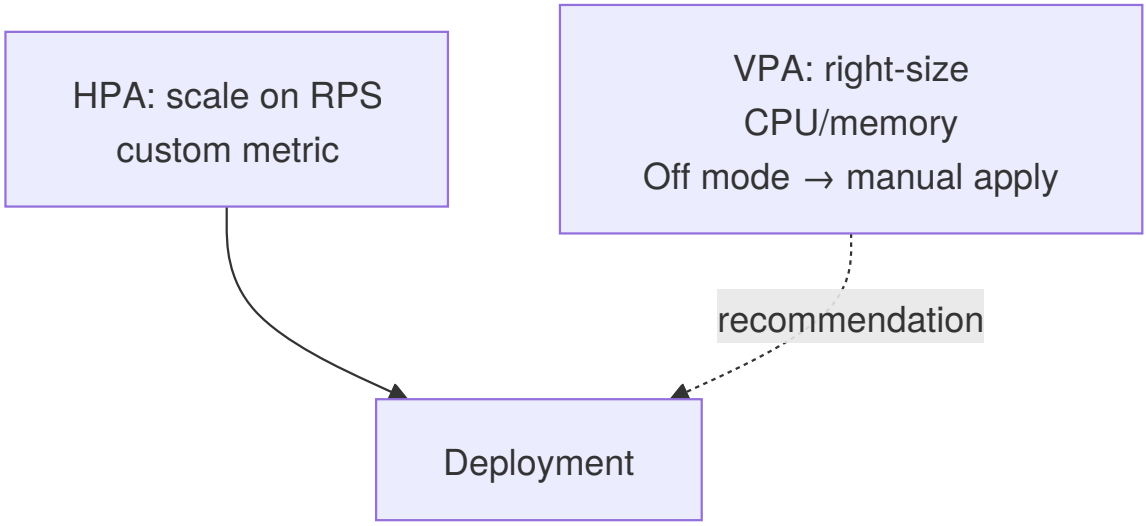
```
spec:
  resourcePolicy:
    containerPolicies:
    - containerName: web
      minAllowed: { cpu: 50m, memory: 64Mi }
      maxAllowed: { cpu: 2, memory: 4Gi }
      controlledResources: [cpu, memory]
      controlledValues: RequestsAndLimits    # or RequestsOnly
```

🏗 Diagrams

VPA architecture



VPA + HPA safe combo



💻 Real-World Examples

Example 1 — VPA in Off mode (recommend only)

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: { name: web-vpa }
spec:
  targetRef: { apiVersion: apps/v1, kind: Deployment, name: web }
  updatePolicy: { updateMode: "Off" }
  resourcePolicy:
    containerPolicies:
    - containerName: web
      minAllowed: { cpu: 50m, memory: 64Mi }
      maxAllowed: { cpu: 4, memory: 8Gi }
```

Example 2 — VPA in Auto mode

```
spec:
  updatePolicy: { updateMode: "Auto" }
```

Example 3 — Read recommendations

```
kubectl describe vpa web-vpa
kubectl get vpa web-vpa -o jsonpath='{.status.recommendation.containerRecommendations[0].target}'
```

💻 Related Commands

Command	Purpose	Example
`kubectl get vpa`	List VPAs	`kubectl get vpa`
`kubectl describe vpa`	Recommendations + status	`kubectl describe vpa web-vpa`
`kubectl get vpa -o yaml`	Full output	`kubectl get vpa web-vpa -o yaml`

🚨 Scenarios

Scenario 1 — VPA evicts all pods at once

- **Cause**: Auto mode + no PDB.
- **Fix**: add PDB `minAvailable: 50%`. VPA respects PDB.

Scenario 2 — VPA + HPA on CPU → infinite loop

- **Cause**: VPA raises request → HPA sees lower utilization → scales down → VPA adjusts → loop.
- **Fix**: use HPA on custom metric (not CPU); VPA on CPU/memory.

Scenario 3 — VPA recommends too low

- **Cause**: insufficient history or workload is spiky.
- **Fix**: wait for more data; set `minAllowed` as a floor.

☁ GKE Perspective

- VPA built-in on GKE: `gcloud container clusters update --enable-vertical-pod-autoscaling`.
- **Multidimensional Pod Autoscaler (MPA)**: GKE-specific — combines HPA + VPA safely.

🏁 EKS Perspective

- Install VPA manually: `kubectl apply -f https://github.com/kubernetes/autoscaler/releases/latest/download/vpa-v1-crd-gen.yaml`.
- Or use Helm chart.

🧪 Labs

Lab 52.1 — VPA Off mode (Beginner, 30 min)

Deploy app with high requests. Apply VPA Off mode. Generate load. `kubectl describe vpa` → read recommendations. Manually apply recommended values.

Lab 52.2 — VPA Auto mode with PDB (Intermediate, 45 min)

VPA Auto. Add PDB `minAvailable: 1`. Observe VPA evicting pods one at a time. Verify new pods have updated requests.

Lab 52.3 — Right-sizing sprint (Advanced, 60 min)

Apply VPA Off to all Deployments. Wait 24h. Script: extract recommendations → compare to current requests → generate patches → apply in staging → validate.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: VPA automatically right-sizes pod CPU and memory requests based on observed usage. It has three components: recommender (analyzes), updater (evicts for updates), admission controller (mutates new pods).
Why: Teams over-request 3–5× on average. VPA eliminates guesswork — it uses historical data to set requests at the right level, improving bin-packing and reducing cost.
How: I start with `Off` mode to gather recommendations without disruption. After reviewing Target vs current requests across all Deployments, I apply the recommendations manually or via a CI job. For non-critical services I use `Auto` mode with a PDB so evictions are safe. I never combine VPA on CPU with HPA on CPU — I use HPA on custom metrics instead.
Example: A right-sizing sprint using VPA Off across 60 Deployments showed 70% were over-requesting CPU by 4×. Applying VPA recommendations freed enough capacity to remove 15 nodes — saving $6K/month."

? Interview Questions

Q1. What does VPA do? Adjusts pod CPU/memory requests based on usage history. **Q2. Three VPA components?** Recommender, Updater, Admission Controller. **Q3. Update modes?** Off, Initial, Auto. **Q4. VPA + HPA safe?** Not on the same metric (CPU). Safe: HPA on RPS + VPA on CPU/memory. **Q5. Lower/Target/Upper Bound?** Lower = minimum safe. Target = recommended. Upper = peak observed. **Q6. Does VPA restart pods?** Auto mode evicts pods to apply new requests. **Q7. How does VPA respect PDB?** Won't evict if it would violate PDB. **Q8.** `controlledValues: RequestsOnly` **— what?** VPA adjusts requests but leaves limits unchanged. **Q9. GKE MPA?** Multidimensional Pod Autoscaler — combines HPA + VPA safely. **Q10. When to use VPA vs HPA?** VPA: right-size individual pods. HPA: scale out/in based on load. Both serve different purposes.

🗐 Topic 53 — Cluster Autoscaler

Official Documentation Reference - Node autoscaling concept: https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/ - Cluster Autoscaler GitHub: https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler - GKE CA: https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-autoscaler - EKS CA: https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html

---

🔹 Definition

The **Cluster Autoscaler (CA)** automatically adjusts the number of **nodes** in a cluster — adding nodes when pods are Pending (unschedulable) and removing underutilized nodes when workloads can be consolidated.

🔹 About

Scale-up trigger

Pod is Pending → scheduler can't find a fitting node → CA simulates if adding a node from a node group would help → provisions the node.

Scale-down trigger

Node utilization < threshold (default 50%) for `--scale-down-unneeded-time` (default 10 min) AND all pods can be moved to other nodes AND no PDB violation.

Expanders (how CA picks which node group to scale)

Expander	Strategy
`random`	Random node group
`most-pods`	Group that fits most pending pods
`least-waste`	Group that wastes least resources
`priority`	User-defined priority per group
`price`	Cheapest (cloud pricing)

Key flags

Flag	Default	Meaning
`--scale-down-utilization-threshold`	0.5	Node underutilized below this → candidate
`--scale-down-unneeded-time`	10m	How long a node must be underutilized
`--max-graceful-termination-sec`	600	Time to drain a node
`--skip-nodes-with-system-pods`	true	Don't remove nodes with kube-system pods
`--skip-nodes-with-local-storage`	true	Don't remove nodes with emptyDir/hostPath
`--expander`	random	Expansion strategy

Annotations

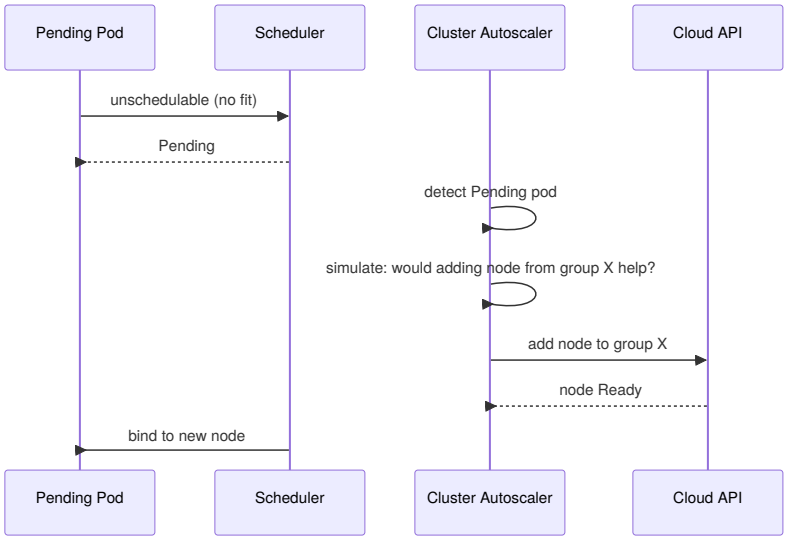
- `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` → CA won't remove this node.
- `cluster-autoscaler.kubernetes.io/scale-down-disabled: "true"` → node exempt.

CA does NOT

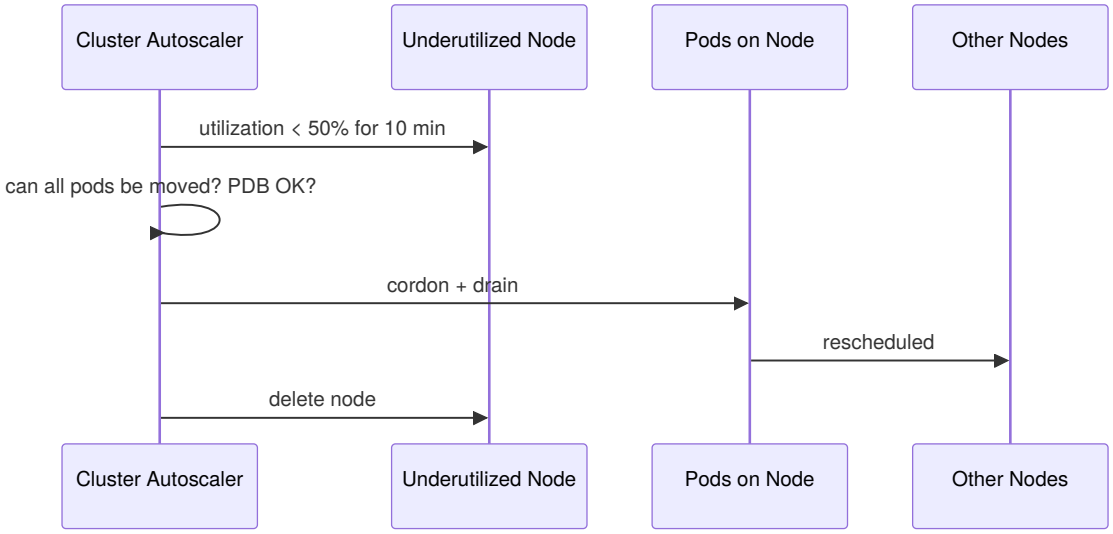
- Scale nodes for DaemonSet-only pods (they're expected on every node).
- Move pods between nodes for balancing (use Descheduler for that).
- Handle custom cloud instances (use Karpenter for that).

🏗 Diagrams

Scale-up flow



Scale-down flow



💻 Real-World Examples

Example 1 — GKE Cluster Autoscaler

```
gcloud container clusters update prod --region=asia-south1 \
  --enable-autoscaling --min-nodes=1 --max-nodes=10 \
  --node-pool=default-pool
```

Example 2 — EKS Cluster Autoscaler

```
helm repo add autoscaler https://kubernetes.github.io/autoscaler
helm install cluster-autoscaler autoscaler/cluster-autoscaler \
  -n kube-system \
  --set autoDiscovery.clusterName=prod \
  --set awsRegion=ap-south-1 \
  --set extraArgs.scale-down-utilization-threshold=0.5 \
  --set extraArgs.expander=least-waste
```

Example 3 — Prevent scale-down on a node

```
kubectl annotate node worker-3 cluster-autoscaler.kubernetes.io/scale-down-disabled="true"
```

💻 Related Commands

Command	Purpose	Example
`kubectl get nodes`	Current node count	`kubectl get nodes`
`kubectl describe configmap cluster-autoscaler-status -n kube-system`	CA status + decisions	
`kubectl logs -n kube-system -l app.kubernetes.io/name=cluster-autoscaler`	CA logs	
`gcloud container clusters describe --format='value(autoscaling)'`	GKE CA config	

🚨 Scenarios

Scenario 1 — Pods Pending but CA doesn't add nodes

- **Cause**: max nodes reached; or node group doesn't match pod constraints (affinity/taints).
- **Fix**: increase max-nodes; add a node group matching pod requirements.

Scenario 2 — CA removes a node during business hours

- **Cause**: utilization dropped below threshold.
- **Fix**: lower `scale-down-utilization-threshold`; or use PDB + `safe-to-evict: "false"` on critical pods.

Scenario 3 — Scale-up takes 5 minutes

- **Cause**: cloud VM provisioning + image pull time.
- **Fix**: use overprovisioning pattern (placeholder pods preempt → instant scaling, CA backfills). Or Karpenter (faster node selection).

☁ GKE Perspective

- **Autopilot**: CA is built-in and fully managed. You don't configure it.
- **Standard**: enable per node pool. Supports **Node Auto-Provisioning (NAP)** for auto-creating node pools.
- **Scale-down blocking**: DaemonSets, PDB, local storage.

▩ EKS Perspective

- CA runs as a Deployment with IRSA.
- Auto-discovery: tags on ASG `k8s.io/cluster-autoscaler/enabled=true` + `k8s.io/cluster-autoscaler/<cluster-name>=owned`.
- **Karpenter** is the recommended alternative — faster, more flexible.

🧪 Labs

Lab 53.1 — CA scale-up (Beginner, 30 min)

Set max-nodes=3, deploy pods requesting more CPU than 1 node has → CA adds nodes.

Lab 53.2 — CA scale-down (Intermediate, 30 min)

Scale deployment to 1 replica. Wait 10 min. Observe CA removes underutilized node.

Lab 53.3 — Overprovisioning pattern (Advanced, 60 min)

Deploy placeholder pods (priority=-1). Deploy real pod → preempts placeholder → instant start. CA backfills placeholder's node in background.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Cluster Autoscaler adds nodes when pods can't schedule (Pending) and removes nodes when they're underutilized (< 50% for 10 min, all pods movable, PDB respected).
Why: Without it, you either over-provision (waste money) or under-provision (Pending pods, degraded service).
How: On GKE I enable CA per node pool with min/max. On EKS I deploy CA via Helm with auto-discovery and `least-waste` expander. I tune scale-down to be conservative (10 min + PDB) and use the overprovisioning pattern for instant pod placement while CA provisions nodes in the background.
Example: During a traffic spike, HPA scaled our API from 5 to 40 pods. CA added 7 nodes in under 4 minutes. After traffic subsided, CA removed the extra nodes over 30 minutes — saving ~$50/day."

? Interview Questions

Q1. How does CA know to add nodes? Pods in Pending state that can't schedule. **Q2. How does CA know to remove nodes?** Utilization below threshold, all pods reschedulable, no PDB violation. **Q3. What are expanders?** Strategies for choosing which node group to scale: random, most-pods, least-waste, priority, price. **Q4. CA vs Karpenter?** CA works with pre-defined node groups (ASGs). Karpenter provisions any instance type per pod requirements — more flexible. **Q5. What does `safe-to-evict: false` do?** Prevents CA from removing the node. **Q6. Does CA balance nodes?** No — it only adds/removes. Use Descheduler for rebalancing. **Q7. CA on Autopilot?** Fully managed, no config needed. **Q8. Scale-up latency?** 2–5 min (VM provisioning + image pull). Mitigate with overprovisioning. **Q9. Does CA handle DaemonSets?** Ignores DS pods for scale-up decisions; skips nodes with system DS pods for scale-down. **Q10. What if max-nodes is reached?** Pods stay Pending. Alert on this.

🗐 Topic 54 — Karpenter (EKS) and Node Auto-Provisioning (GKE)

Official Documentation Reference - Karpenter: https://karpenter.sh/docs/ - GKE NAP: https://cloud.google.com/kubernetes-engine/docs/concepts/node-auto-provisioning - EKS Karpenter: https://docs.aws.amazon.com/eks/latest/best-practices/karpenter.html

🔹 Definition

Karpenter (EKS) and **Node Auto-Provisioning (NAP)** (GKE) are next-generation node autoscalers that provision right-sized, right-type nodes on demand — without pre-defined node groups. They select instance types, zones, and purchase options (Spot/on-demand) per pending pod's requirements.

🔹 About

Karpenter vs Cluster Autoscaler

Feature	Cluster Autoscaler	Karpenter
Node selection	Pre-defined ASG/node group	Any instance type per pod
Speed	2–5 min (scale ASG)	30–90 s (direct EC2 API)
Spot handling	Limited	Built-in interruption handling + consolidation
Consolidation	No (only scale down)	Replaces underutilized nodes with smaller ones
Multi-arch	Requires separate node groups	Selects arm64/amd64 based on pod
Cloud	Any (GKE, EKS, AKS)	Primarily AWS EKS (GKE has NAP)

Karpenter objects (v1 API, 2024+)

Object	Purpose
NodePool	Defines constraints: instance types, zones, capacity type (spot/on-demand), taints, limits
EC2NodeClass	AWS-specific: AMI, subnets, security groups, user data, block device mappings
NodeClaim	Internal: represents a single provisioned node

GKE NAP

- **What**: GKE automatically creates and deletes node pools based on pod requirements.
- **How**: enable with `--enable-autoprovisioning --min-cpu=1 --max-cpu=100 --min-memory=1 --max-memory=400`.
- **Supports**: GPU auto-provisioning, Spot VMs, custom machine families.
- **Limitation**: less flexible than Karpenter (can't mix arbitrary instance types per pod).

Karpenter consolidation

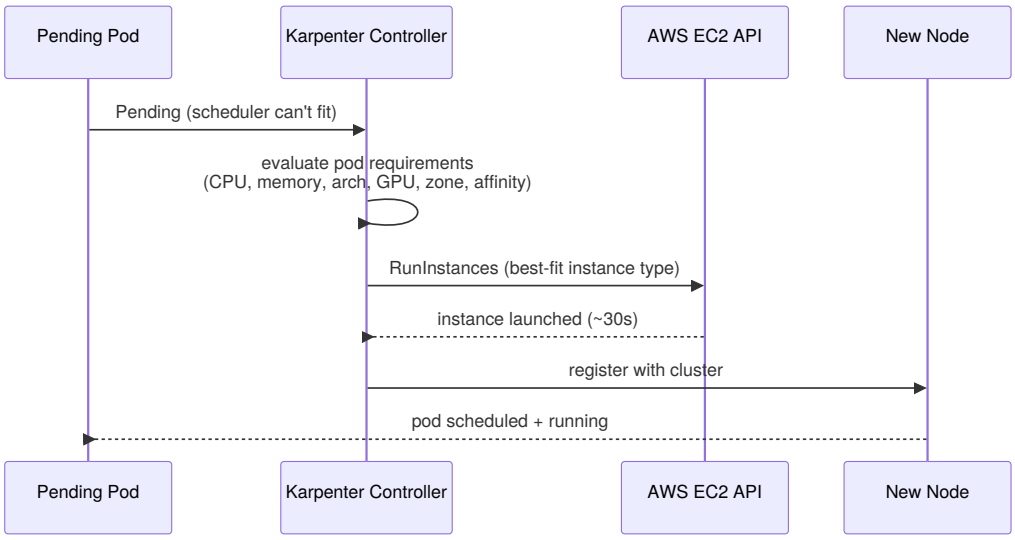
Karpenter continuously watches for underutilized nodes and replaces them with cheaper/smaller ones: - **Deletion**: removes empty nodes. - **Replacement**: replaces a node with a smaller instance if pods fit. - **Policy**: `consolidationPolicy: WhenEmptyOrUnderutilized` or `WhenEmpty`. - **Budget**: `disruption.budgets` control how many nodes can be disrupted at once.

Spot/preemptible handling

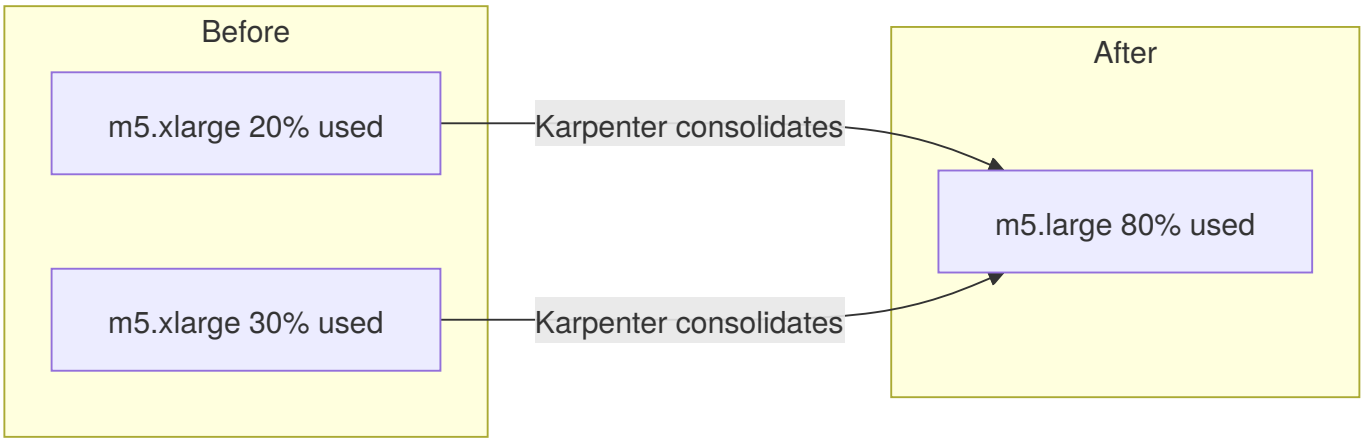
- **Karpenter**: detects Spot interruption (2 min warning), cordons node, drains pods.
- **NAP**: GKE handles preemptible VM eviction; pods rescheduled automatically.

🏗 Diagrams

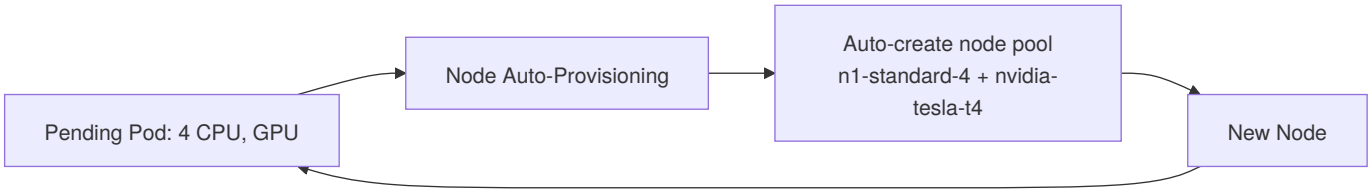
Karpenter provisioning flow



Karpenter consolidation



GKE NAP



💻 Real-World Examples

Example 1 — Karpenter NodePool + EC2NodeClass

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: { name: default }
spec:
  template:
    spec:
      requirements:
      - { key: karpenter.sh/capacity-type, operator: In, values: [spot, on-demand] }
      - { key: kubernetes.io/arch, operator: In, values: [amd64] }
      - { key: node.kubernetes.io/instance-type, operator: In, values: [m5.large, m5.xlarge, m6i.large, c5.large] }
      - { key: topology.kubernetes.io/zone, operator: In, values: [ap-south-1a, ap-south-1b] }
      nodeClassRef: { group: karpenter.k8s.aws, kind: EC2NodeClass, name: default }
  limits:
    cpu: 100
    memory: 400Gi
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 60s
    budgets:
    - nodes: "20%"
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata: { name: default }
spec:
  amiSelectorTerms: [{ alias: al2023@latest }]
  subnetSelectorTerms: [{ tags: { karpenter.sh/discovery: prod } }]
  securityGroupSelectorTerms: [{ tags: { karpenter.sh/discovery: prod } }]
  role: KarpenterNodeRole-prod
```

Example 2 — GKE NAP

```bash
gcloud container clusters update prod --region=asia-south1 \
  --enable-autoprovisioning \
  --min-cpu=2 --max-cpu=200 \
  --min-memory=4 --max-memory=800 \
  --autoprovisioning-scopes=https://www.googleapis.com/auth/cloud-platform
```

Example 3 — Karpenter install on EKS

```bash
helm install karpenter oci://public.ecr.aws/karpenter/karpenter \
  --version 1.0.0 \
  --namespace kube-system \
  --set clusterName=prod \
  --set clusterEndpoint=$(aws eks describe-cluster --name prod --query "cluster.endpoint" --output text)
```

💻 Related Commands

Command	Purpose	Example
`kubectl get nodepools`	List Karpenter NodePools	`kubectl get nodepools`
`kubectl get nodeclaims`	Active node claims	`kubectl get nodeclaims`
`kubectl describe nodepool default`	Pool config + status	
`kubectl get nodes -L karpenter.sh/capacity-type`	Spot vs on-demand	
`kubectl logs -n kube-system -l app.kubernetes.io/name=karpenter`	Controller logs	
`gcloud container operations list`	GKE NAP operations	

🚨 Scenarios

Scenario 1 — Karpenter picks too-expensive instance

- **Cause**: no instance type constraints in NodePool.
- **Fix**: add `node.kubernetes.io/instance-type In [m5.large, c5.large, ...]`.

Scenario 2 — Spot interruption during deploy

- **Cause**: Spot node reclaimed by AWS.
- **Fix**: Karpenter handles gracefully — cordons, drains, provisions replacement. Add PDB for safety.

Scenario 3 — NAP creates wrong machine type

- **Cause**: NAP picks based on pod requests; if requests are tiny, it picks small VMs.
- **Fix**: set `min-cpu` / `min-memory` on NAP; or add resource floors via LimitRange.

Scenario 4 — Karpenter consolidation disrupts long-running job

- **Fix**: annotate job pods `karpenter.sh/do-not-disrupt: "true"`. Or set `disruption.budgets` to limit concurrent disruptions.

☁ GKE Perspective

- **NAP** is the GKE equivalent of Karpenter. Managed by Google; fewer knobs.

- **Autopilot** = NAP on steroids — you don't even see nodes.
- NAP supports GPU node pools, Spot VMs, custom machine families.
- For richer node selection, some teams install Karpenter on GKE Standard (community-supported, less mature than on EKS).

▩ EKS Perspective

- **Karpenter is the recommended node autoscaler for EKS** (AWS official).
- Replaces Cluster Autoscaler for most use cases.
- Supports: Spot + on-demand mixing, arm64/amd64, GPU, EFA, capacity reservations.
- **EKS Auto Mode** (2024+): managed Karpenter — AWS runs the controller.

🧪 Labs

Lab 54.1 — Karpenter basics on EKS (Intermediate, 60 min)

1. Install Karpenter (Example 3). 2. Apply NodePool + EC2NodeClass (Example 1). 3. Deploy pods requesting 4 CPU → Karpenter provisions node in ~60s. 4. Scale down → Karpenter consolidates.

Lab 54.2 — Spot + on-demand mix (Advanced, 60 min)

1. NodePool with `capacity-type: In [spot, on-demand]`. 2. Deploy 10 replicas. 3. `kubectl get nodes -L karpenter.sh/capacity-type` → mix of spot and on-demand. 4. Simulate Spot interruption → Karpenter replaces.

Lab 54.3 — GKE NAP with GPU (Advanced, 60 min)

1. Enable NAP with GPU support. 2. Deploy ML pod requesting `nvidia.com/gpu: 1`. 3. NAP creates GPU node pool automatically. 4. Pod runs. 5. Delete pod → NAP scales down GPU pool.

🎙 Interview Deliverable Answer (60–90 s)

"**What**: Karpenter (EKS) and NAP (GKE) are next-gen node autoscalers that provision right-sized nodes per pod — no pre-defined node groups. Karpenter selects from a pool of instance types, zones, and Spot/on-demand based on each pending pod's requirements, and consolidates underutilized nodes by replacing them with smaller ones.
Why: Cluster Autoscaler is limited to scaling pre-defined ASGs — you must guess which instance types to pre-configure. Karpenter eliminates that guesswork, provisions in ~60 seconds (vs 3–5 min), and saves money through intelligent consolidation and Spot mixing.
How: I define a NodePool with broad instance type families (m5/m6i/c5), allow Spot + on-demand, set CPU/memory limits, and enable consolidation. Karpenter picks the cheapest instance that fits. I protect long-running jobs with `do-not-disrupt` annotations and set disruption budgets. On GKE I use NAP or Autopilot.
Example: Migrating from Cluster Autoscaler to Karpenter on a 200-pod EKS cluster reduced node count by 30% (better bin-packing), cut provisioning time from 4 minutes to 45 seconds, and saved $15K/month through Spot + consolidation."

? Interview Questions

Q1. Karpenter vs CA? Karpenter: any instance type, faster, consolidation. CA: pre-defined ASGs, slower, wider cloud support. **Q2. NodePool vs EC2NodeClass?** NodePool: scheduling constraints. EC2NodeClass: AWS infra (AMI, subnets, SGs). **Q3. What is consolidation?** Karpenter replaces underutilized nodes with smaller/cheaper ones. **Q4. How does Karpenter handle Spot?** Detects 2-min interruption warning, cordons, drains, provisions replacement. **Q5. What is GKE NAP?** Node Auto-Provisioning: auto-creates node pools based on pod requirements. **Q6. Karpenter speed?** ~30–90 seconds (direct EC2 API vs ASG scaling). **Q7.** `do-not-disrupt` **annotation?** Prevents Karpenter from consolidating/disrupting that node. **Q8. EKS Auto Mode?** AWS-managed Karpenter. **Q9. GKE Autopilot vs NAP?** Autopilot = nodes fully hidden. NAP = auto-creates node pools you can see. **Q10. How to limit Karpenter spending?** `limits: {cpu: 100, memory: 400Gi}` on NodePool caps total provisioned resources.

🗐 Topic 55 — Pod Disruption Budgets (PDB)

Official Documentation Reference - Disruptions: https://kubernetes.io/docs/concepts/workloads/pods/disruptions/ - PDB: https://kubernetes.io/docs/tasks/run-application/configure-pdb/ - GKE PDB: https://cloud.google.com/kubernetes-engine/docs/concepts/pod-disruption-budgets - EKS PDB: https://docs.aws.amazon.com/eks/latest/best-practices/reliability.html

---

⟠ Definition

A **PodDisruptionBudget (PDB)** limits the number of pods of a replicated application that can be **voluntarily disrupted** at the same time — ensuring availability during node drains, cluster upgrades, and autoscaler scale-downs.

⟠ About

Voluntary vs involuntary disruptions

Type	Examples	PDB applies?
Voluntary	`kubectl drain`, cluster upgrade, CA scale-down, Karpenter consolidation, VPA eviction	✅ Yes
Involuntary	Node crash, kernel panic, OOMKill, hardware failure	❌ No

PDB spec

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: web-pdb }
spec:
  selector: { matchLabels: { app: web } }
  minAvailable: 2        # OR maxUnavailable: 1 (use one, not both)
```

Field	Meaning
`minAvailable`	Minimum pods that must remain running (absolute or %)
`maxUnavailable`	Maximum pods that can be down (absolute or %)
`selector`	Which pods this PDB protects
`unhealthyPodEvictionPolicy`	`IfHealthy` (default) or `AlwaysAllow` (K8s 1.31+ stable)

Rules

- Only ONE of `minAvailable` or `maxUnavailable`.
- `minAvailable: 100%` = no voluntary disruption allowed (blocks drains/upgrades — dangerous!).
- `maxUnavailable: 0` = same as above — blocks everything.
- PDB applies to pods matching the selector, regardless of controller.
- Multiple PDBs can cover the same pods — all must allow.

`unhealthyPodEvictionPolicy: AlwaysAllow`

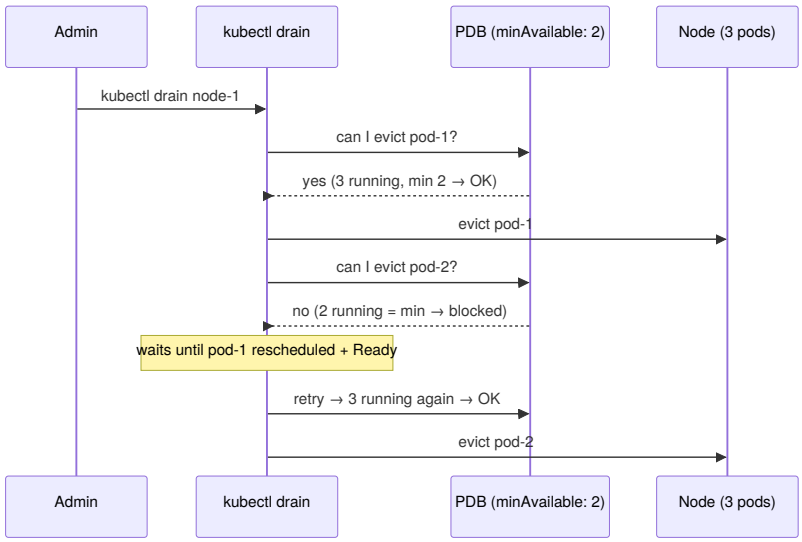
By default, PDB counts unhealthy pods as "available" to protect them from eviction during investigations. `AlwaysAllow` lets unhealthy pods be evicted even when PDB is at its limit — prevents stuck drains when pods are already failing.

PDB + kubectl drain

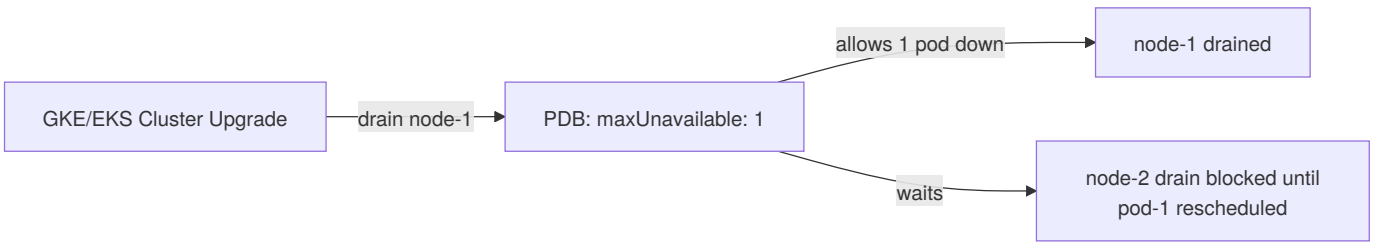
`kubectl drain` respects PDB. If evicting a pod would violate PDB, drain waits (up to `--timeout`). `--disable-eviction` bypasses PDB (dangerous).

🏗 Diagrams

PDB during node drain



PDB with cluster upgrade



💻 Real-World Examples

Example 1 — PDB with minAvailable

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: api-pdb }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: api } }
```

Example 2 — PDB with maxUnavailable percentage

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: web-pdb }
spec:
  maxUnavailable: "25%"
  selector: { matchLabels: { app: web } }
```

Example 3 — PDB with unhealthyPodEvictionPolicy

```
spec:
  maxUnavailable: 1
  unhealthyPodEvictionPolicy: AlwaysAllow
  selector: { matchLabels: { app: web } }
```

Example 4 — Imperative

```
kubectl create pdb web-pdb --selector=app=web --min-available=2
kubectl get pdb
kubectl describe pdb web-pdb
```

💻 Related Commands

Command	Purpose	Example
`kubectl get pdb`	List PDBs	`kubectl get pdb -A`
`kubectl describe pdb`	Status + allowed disruptions	`kubectl describe pdb api-pdb`
`kubectl create pdb`	Imperative creation	`kubectl create pdb p --selector=app=web --min-available=2`
`kubectl drain`	Drain node (respects PDB)	`kubectl drain node-1 --ignore-daemonsets --delete-emptydir-data`
`kubectl drain`	`--disable-eviction`	Bypass PDB (dangerous)

🚨 Scenarios

Scenario 1 — Drain blocked indefinitely

- **Cause**: PDB `minAvailable: 100%` or only 2 replicas with `minAvailable: 2`.
- **Fix**: lower `minAvailable` to `replicas - 1`; or increase replicas first.

Scenario 2 — Cluster upgrade stuck

- **Cause**: PDB too tight; upgraded node can't drain.
- **Fix**: same as above. GKE shows upgrade status; EKS shows node group update stuck.

Scenario 3 — PDB + single replica

- **Cause**: `minAvailable: 1` with 1 replica → drain can't evict the only pod.
- **Fix**: increase replicas to ≥ 2 before applying PDB.

Scenario 4 — Unhealthy pod blocking drain

- **Cause**: default `unhealthyPodEvictionPolicy: IfHealthy` counts unhealthy pods as needing protection.
- **Fix**: set `AlwaysAllow` so unhealthy pods can be evicted.

☁ GKE Perspective

- GKE respects PDB during node upgrades and node pool resize.
- **Autopilot** enforces PDB for node maintenance.
- PDB too strict → upgrade times out; GKE retries but eventually fails.

▩ EKS Perspective

- EKS managed node group upgrades respect PDB during rolling replacement.
- Karpenter respects PDB during consolidation and Spot replacement.

- `kubectl drain` (manual patching) respects PDB.

🧪 Labs

Lab 55.1 — PDB basics (Beginner, 20 min)

1. Deploy 3-replica Deployment + PDB `minAvailable: 2`.
2. `kubectl drain node-1 --ignore-daemonsets` → one pod evicted; second blocked.
3. `kubectl uncordon node-1`.

Lab 55.2 — PDB blocks upgrade (Intermediate, 30 min)

1. Set `minAvailable: 100%`. Try `kubectl drain` → blocked forever.
2. Fix to `maxUnavailable: 1`. Retry → succeeds.

Lab 55.3 — unhealthyPodEvictionPolicy (Advanced, 30 min)

1. Create PDB `maxUnavailable: 1, unhealthyPodEvictionPolicy: AlwaysAllow`.
2. Make one pod unhealthy (fail readiness).
3. Drain → unhealthy pod evicted despite being "needed".

🎯 Interview Deliverable Answer (60–90 s)

"**What**: A PodDisruptionBudget limits how many pods of a replicated workload can be voluntarily disrupted at once — protecting availability during drains, upgrades, and autoscaler consolidation.
Why: Without PDB, a `kubectl drain` or cluster upgrade can take down all replicas simultaneously. PDB guarantees N pods stay running throughout.
How: For every production Deployment with ≥ 2 replicas, I create a PDB with `maxUnavailable: 1` (or 25% for large fleets). I pair it with replicas ≥ 3 so drain is never blocked. I set `unhealthyPodEvictionPolicy: AlwaysAllow` to prevent stuck drains when pods are already failing. I never set `minAvailable: 100%` — that blocks all maintenance.
Example: During a GKE cluster upgrade, our PDB `maxUnavailable: 1` ensured only one API pod was down at a time across 12 replicas. The upgrade took 45 minutes instead of 5, but zero requests were dropped."

❓ Interview Questions

Q1. What is a PDB? Limits voluntary disruption of pods — ensures minimum availability. **Q2. Voluntary vs involuntary?** Voluntary: drain, upgrade, autoscaler. Involuntary: node crash, OOM. PDB only affects voluntary. **Q3. `minAvailable` vs `maxUnavailable`?** Two ways to express the same thing. Use one. **Q4. PDB `minAvailable: 100%` — problem?** Blocks all drains and upgrades — never use in production. **Q5. Does PDB protect against OOMKill?** No — that's involuntary. **Q6. PDB + single replica?** PDB can't protect — drain can't evict the only pod. Increase replicas first. **Q7. `unhealthyPodEvictionPolicy: AlwaysAllow`?** Allows eviction of unhealthy pods even when PDB is at limit — prevents stuck drains. **Q8. Does Karpenter respect PDB?** Yes — disruption budgets in NodePool + K8s PDB. **Q9. How does drain interact with PDB?** Drain sends eviction API calls; PDB admission rejects if it would violate the budget. **Q10. PDB with StatefulSet?** Works — but ordered scale-down semantics apply; PDB limits how many can be down.

📄 Topic 56 — Multi-Zone / Multi-Region HA

Official Documentation Reference - Running in multiple zones: https://kubernetes.io/docs/setup/best-practices/multiple-zones/ - Topology spread: https://kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/ - GKE regional clusters: https://cloud.google.com/kubernetes-engine/docs/concepts/regional-clusters - GKE multi-cluster: https://cloud.google.com/kubernetes-engine/docs/concepts/multi-cluster-services - EKS multi-AZ: https://docs.aws.amazon.com/eks/latest/best-practices/reliability.html

---

🔷 Definition

Multi-zone HA distributes control-plane and worker nodes across availability zones within one region. **Multi-region HA** replicates entire clusters across geographic regions for disaster recovery and global latency optimization.

🔷 About

Multi-zone (within a region)

Component	Multi-zone behavior
Control plane	GKE regional: 3 CP replicas across 3 zones. EKS: managed, multi-AZ by default.
Nodes	Node pools spread across zones (GKE regional auto, EKS ASG/Karpenter multi-AZ)
Pods	`topologySpreadConstraints` with `topology.kubernetes.io/zone`
Storage	Zonal PD/EBS = pinned to one zone. Regional PD (GKE) = cross-zone. EFS (EKS) = cross-AZ.
Services	kube-proxy/eBPF routes to pods in any zone
LoadBalancer	Cloud LB distributes across zones

Multi-region (across regions)

Pattern	How
Active-passive	Primary cluster serves traffic; standby cluster for DR failover
Active-active	Both clusters serve traffic; global LB routes by latency
Federation	KubeFed or Fleet manages resources across clusters
GitOps multi-cluster	ArgoCD ApplicationSet deploys to multiple clusters

Tools for multi-cluster

Tool	Cloud	Purpose
GKE Fleet / MCS	GCP	Multi-cluster Services, Fleet-level policies
EKS Anywhere + Connector	AWS	Hybrid multi-cluster management
ArgoCD ApplicationSet	Any	GitOps deploy to N clusters
Cilium ClusterMesh	Any	Cross-cluster pod-to-pod networking
Submariner	Any	Cross-cluster L3 connectivity
Liqo	Any	Virtual node — schedule pods across clusters

Zone failure design

1. **Pods**: `topologySpreadConstraints maxSkew: 1` across zones.
2. **Storage**: regional PD (GKE) or app-level replication (Patroni, Strimzi) on EKS.
3. **PDB**: `maxUnavailable: 1` — drain one zone at a time.
4. **LB**: cloud LB auto-routes away from unhealthy zone.
5. **DNS**: health-checked DNS (Route 53, Cloud DNS) for failover.

Region failure design

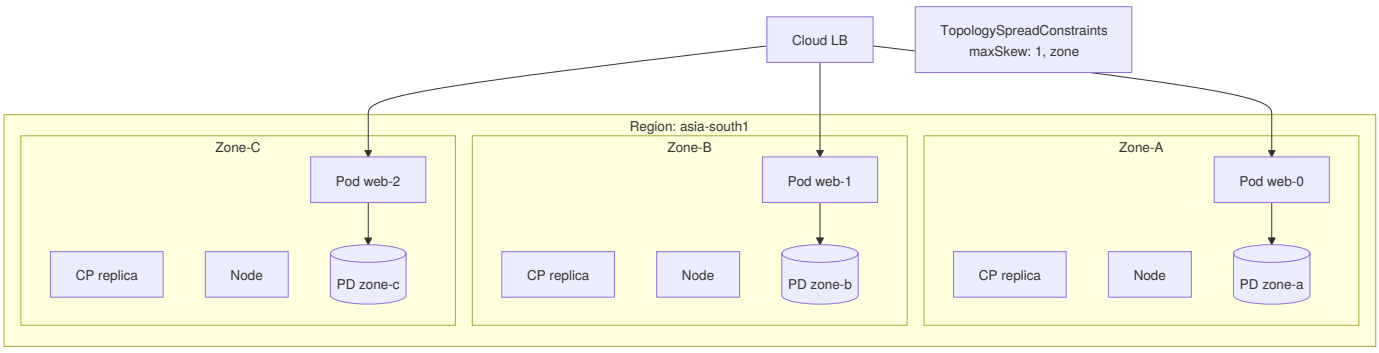
1. **Second cluster** in another region (standby or active).
2. **Data replication**: async DB replication, GCS/S3 cross-region, Velero backups.
3. **Global LB**: Google Cloud Global LB or AWS Global Accelerator.
4. **DNS failover**: health-checked routing policies.
5. **GitOps**: same manifests deployed to both clusters via ArgoCD.

RPO / RTO targets

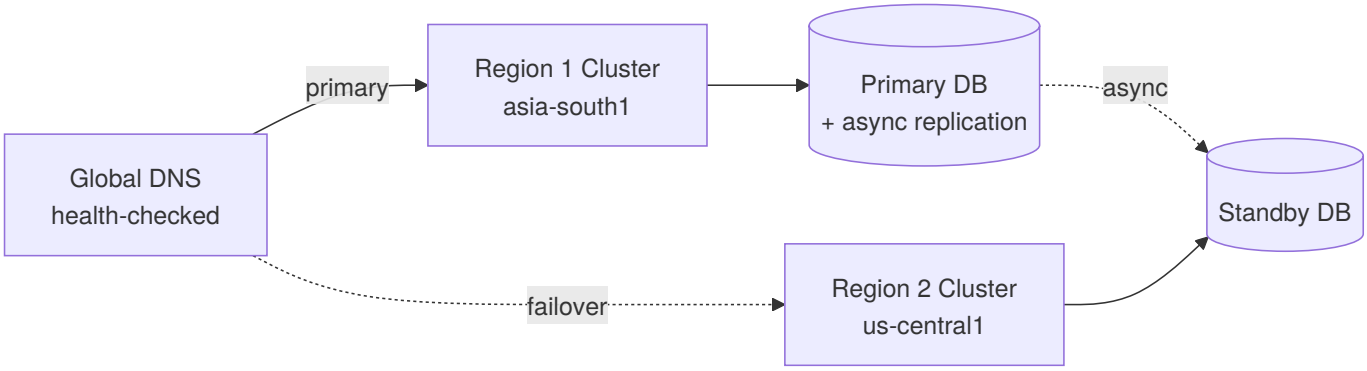
Strategy	RPO	RTO
Multi-zone (single region)	0 (synchronous PD)	Minutes
Active-passive (multi-region)	Minutes (async replication)	10–30 min (manual/auto failover)
Active-active (multi-region)	Near-zero	Near-zero

🏗 Diagrams

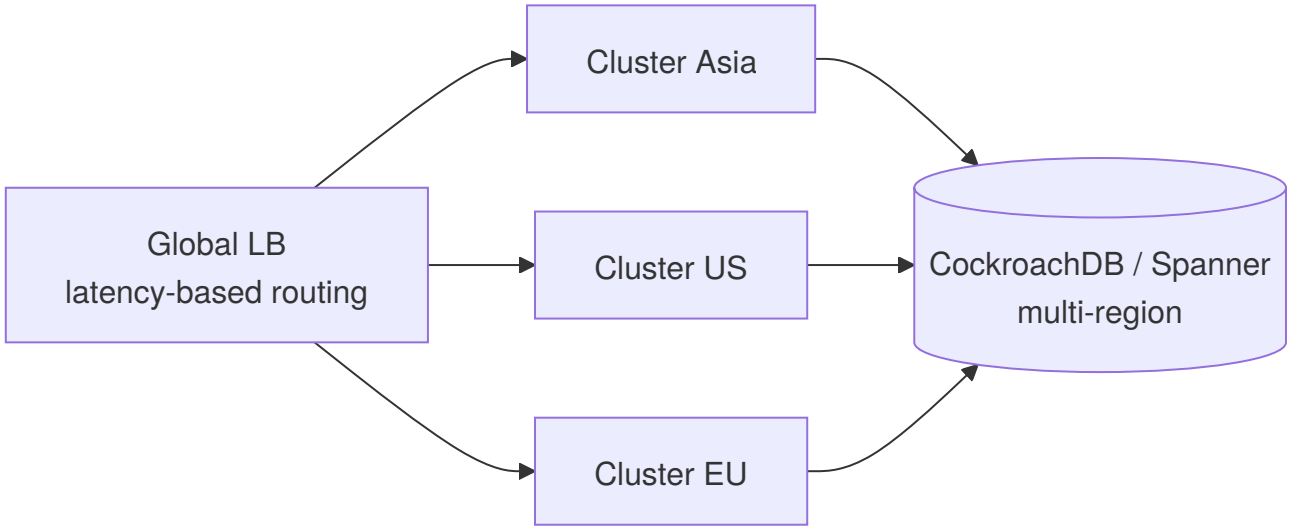
Multi-zone architecture



Multi-region active-passive



Multi-region active-active



💻 Real-World Examples

Example 1 — GKE regional cluster

```
gcloud container clusters create prod \
  --region=asia-south1 \
  --num-nodes=2          # 2 nodes per zone × 3 zones = 6 nodes
```

Example 2 — EKS multi-AZ managed NG

```
eksctl create cluster --name prod --region ap-south-1 --zones ap-south-1a,ap-south-1b,ap-south-1c
```

Example 3 — Deployment with zone spread

```
spec:
  replicas: 6
  template:
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector: { matchLabels: { app: web } }
```

Example 4 — GKE Multi-Cluster Services

```
gcloud container fleet memberships register cluster-asia --gke-cluster=asia-south1/prod
gcloud container fleet multi-cluster-services enable
# Export a Service across clusters
kubectl apply -f - <<EOF
apiVersion: net.gke.io/v1
kind: ServiceExport
metadata: { name: web, namespace: prod }
EOF
```

Example 5 — ArgoCD multi-cluster

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: web }
spec:
  generators:
  - clusters:
      selector: { matchLabels: { env: prod } }
  template:
    spec:
      source: { repoURL: https://github.com/org/k8s-manifests, path: prod }
      destination: { server: '{{server}}', namespace: prod }
```

Example 6 — Velero cross-region DR

```
# Backup in region 1
velero backup create full --include-namespaces=prod --snapshot-volumes

# Restore in region 2 cluster
velero restore create --from-backup full
```

💻 Related Commands

Command	Purpose	Example
`kubectl get nodes -L topology.kubernetes.io/zone`	See zone spread	
`kubectl get pods -o wide`	Pod-to-node-to-zone mapping	
`gcloud container clusters describe --format='value(locations)'`	GKE zones	
`aws eks describe-cluster --query 'cluster.resourcesVpcConfig.subnetIds'`	EKS subnets (AZs)	
`velero backup create`	Cross-region backup	

🚨 Scenarios

Scenario 1 — Zone failure: pods Pending (zonal PD)

- **Cause**: PD stuck in failed zone; pod can't reschedule without its volume.
- **Fix**: use regional PD (GKE) or app-level replication. For DR: restore from VolumeSnapshot in healthy zone.

Scenario 2 — All pods in one zone despite 3-zone cluster

- **Cause**: no TopologySpreadConstraints.
- **Fix**: add TSC `maxSkew: 1` across zones.

Scenario 3 — Region outage: no failover

- **Cause**: single-region cluster; no DR plan.
- **Fix**: deploy standby cluster in second region; set up DNS failover + data replication.

Scenario 4 — Cross-region latency for DB writes

- **Cause**: synchronous replication across regions = high latency.
- **Fix**: async replication (accept RPO > 0); or use globally distributed DB (CockroachDB, Spanner).

☁ GKE Perspective

- **Regional cluster**: 3 CP replicas across zones, auto-zone-spread for node pools.
- **Regional PD**: synchronous cross-zone replication — survives single-zone failure.
- **Multi-cluster Gateway**: global HTTP LB routing across GKE clusters.
- **Fleet**: centralized management of multiple GKE clusters.
- **Backup for GKE**: cross-region restore supported.

▩ EKS Perspective

- **Control plane**: always multi-AZ (AWS-managed).
- **Managed NG**: spread across specified AZs.
- **EBS**: zonal — app-level replication for HA databases.
- **EFS**: multi-AZ — good for shared data.
- **Global Accelerator**: anycast IP routing to nearest healthy EKS cluster.
- **Velero**: cross-region backup/restore with EBS snapshots.

🧪 Labs

Lab 56.1 — Multi-zone spread (Beginner, 30 min)

Deploy on Kind with fake zone labels. Apply TSC. Verify even spread with `kubectl get pods -o wide`.

Lab 56.2 — Simulate zone failure on GKE (Intermediate, 60 min)

1. Regional GKE cluster. Deploy 6 replicas with TSC.
2. Cordon all nodes in one zone: `kubectl cordon <nodes-in-zone-b>`.
3. Observe pods rescheduling to remaining zones. Service stays up.
4. Uncordon.

Lab 56.3 — Cross-region DR with Velero (Advanced, 120 min)

1. Cluster in region 1 with app + PVCs.
2. Install Velero with S3 backend.
3. `velero backup create full --snapshot-volumes`.
4. Create cluster in region 2.
5. `velero restore create --from-backup full`.
6. Verify app + data restored.

🎯 Interview Deliverable Answer (60–90 s)

"**What**: Multi-zone HA spreads pods and nodes across availability zones within a region. Multi-region HA replicates entire clusters across regions for disaster recovery.
Why: A single zone can fail — hardware, power, network. Multi-zone with TSC `maxSkew: 1` ensures losing one zone degrades capacity to 66%, not 0%. Multi-region protects against regional outages and can reduce global latency.
How: On GKE I use regional clusters (3-zone CP + nodes) with regional PD for stateful workloads. On EKS the control plane is multi-AZ by default; I spread nodes and use TSC. For multi-region I deploy a second cluster via ArgoCD, async-replicate the database, and use DNS health checks for failover. Velero provides application-level backup/restore across regions.
Example: When a GKE zone went down during peak hours, our API stayed at 66% capacity (4/6 replicas in 2 surviving zones). PDB prevented more than 1 pod being disrupted during rebalancing. We recovered to 100% in 3 minutes when the zone came back. Zero customer impact."

? Interview Questions

Q1. Multi-zone vs multi-region? Zone: within one region, low latency, no data replication needed (regional PD). Region: across geography, higher latency, needs data sync. **Q2. How do you spread pods across zones?** TSC: `topologyKey: topology.kubernetes.io/zone, maxSkew: 1`. **Q3. Zonal PD problem?** Pod stuck Pending if zone fails — PD can't move. Fix: regional PD or app replication. **Q4. Regional PD on GKE?** Synchronous replication across 2 zones. Survives single-zone failure. **Q5. EKS equivalent of regional PD?** No direct equivalent. Use app-level replication (Patroni) or EFS (for RWX). **Q6. Active-passive vs active-active?** Passive: standby cluster, failover DNS. Active: both serve, global LB. **Q7. How to do multi-region GitOps?** ArgoCD ApplicationSet with cluster generator — same manifests to N clusters. **Q8. What is GKE Fleet?** Centralized management of multiple GKE clusters — policies, config, services. **Q9. RPO/RTO for multi-zone?** RPO ≈ 0 (sync PD), RTO ≈ minutes (pod reschedule). **Q10. Cross-region DB replication — sync or async?** Async (RPO > 0) for performance. Sync (RPO = 0) only with geo-distributed DBs (Spanner, CockroachDB).

MODULE

Module 10 — Advanced Topics

Module 10 — Advanced Topics (Part 1)

Topics 57 – 61

🗄 Topic 57 — Custom Resource Definitions (CRDs)

Official Documentation References - https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/ - https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/ - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#customresourcedefinition-v1-apiextensions-k8s-io

🗄 Definition

A Custom Resource Definition (CRD) is a Kubernetes API object that lets you register a brand-new resource type with the Kubernetes API server so that `kubectl` and controllers can treat your object just like a built-in one.

📖 About

Kubernetes ships with resources like `Pod`, `Deployment`, and `Service`. A CRD is how you tell the cluster "please also accept a new kind of object called `Database`, or `Certificate`, or `KafkaTopic`."

Why CRDs exist - Kubernetes is designed to be extensible without recompiling the API server. - Teams need to model higher-level, domain-specific concepts (a full database, a certificate, a backup job) as first-class objects. - CRDs make those objects storable in `etcd`, watchable, describable, RBAC-controllable, and reconcilable — all for free.

What problem it solves - Before CRDs, you'd either fork Kubernetes (expensive) or run out-of-band tools. CRDs turn any concept into a declarative API object that the cluster natively understands. - CRDs pair with a **controller** (the "operator pattern") to give behavior. A CRD alone is just schema and storage.

Key sub-concepts - **Scope**: `Namespaced` or `Cluster`. - **Versions**: A CRD can serve multiple versions (`v1alpha1`, `v1beta1`, `v1`) with conversion webhooks. - **Schema validation**: OpenAPI v3 schema enforces structure at admission. - **Subresources**: `status` and `scale` subresources unlock rich behavior (status updates without touching spec, HPA integration). - **Categories & short names**: e.g. `kubectl get all` catch-alls and `kubectl get db` shortcuts. - **Printer columns**: extra columns shown in `kubectl get`.

🔗 How It Integrates With Other Kubernetes Components

Component	Integration
kube-apiserver	Registers new REST endpoints under `/apis/<group>/<version>/<plural>`
etcd	Stores custom resource instances just like built-ins
kubectl	Auto-discovers the new resource via OpenAPI and lets you `get/apply/delete`
RBAC	Roles can grant verbs on the new resource (`kafkatopics.messaging.example.com`)
Admission controllers	Validating/mutating webhooks fire on custom resources
Controller-manager / Operators	A custom controller watches the CR and reconciles it
Metrics server / HPA	If `scale` subresource is defined, HPA can autoscale the CR
Audit logs	CRs appear in audit logs like any other object

🏗 Architecture / Flow Diagram

Diagram 1 — CRD registration + custom resource lifecycle

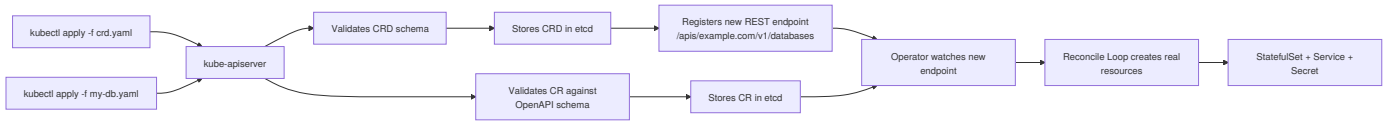


Diagram 2 — Request lifecycle for a custom resource

```
User -> kubectl -> API Server -> Admission (Mutating -> Validation) -> etcd
                                      |
                                      +--> OpenAPI schema check
                                      +--> Conversion webhook (if multi-version)
                                      +--> Custom validating webhook (optional)
etcd change -> API Server watch -> Operator controller -> creates Pods/Services
```

💻 Real-World Examples

Example 1 — Simple CRD for a `Website` resource

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: websites.web.example.com
spec:
  group: web.example.com
  scope: Namespaced
  names:
    plural: websites
    singular: website
    kind: Website
    shortNames: ["web"]
    categories: ["all"]
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["gitRepo", "replicas"]
              properties:
                gitRepo:
                  type: string
                replicas:
                  type: integer
                  minimum: 1
                  maximum: 20
            status:
              type: object
              properties:
                url:
                  type: string
      subresources:
        status: {}
      additionalPrinterColumns:
        - name: Replicas
          type: integer
          jsonPath: .spec.replicas
        - name: URL
          type: string
          jsonPath: .status.url
```

Corresponding custom resource:

```yaml
apiVersion: web.example.com/v1
kind: Website
metadata:
  name: my-blog
spec:
  gitRepo: https://github.com/me/blog.git
  replicas: 3
```

Example 2 — Multi-version CRD with conversion

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: backups.data.example.com
spec:
  group: data.example.com
  scope: Namespaced
  names:
    plural: backups
    singular: backup
    kind: Backup
  conversion:
    strategy: Webhook
    webhook:
      clientConfig:
        service:
          namespace: default
          name: backup-conversion-webhook
          path: /convert
        caBundle: <base64-CA>
      conversionReviewVersions: ["v1"]
  versions:
    - name: v1alpha1
      served: true
      storage: false
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                target: { type: string }
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              required: ["target", "schedule"]
              properties:
                target: { type: string }
                schedule: { type: string }
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get crd`	–	List all CRDs	`kubectl get crd`
`kubectl get crd`	`-o yaml`	Show full CRD spec	`kubectl get crd websites.web.example.com -o yaml`
`kubectl explain`	–	Show CR schema	`kubectl explain website.spec`
`kubectl api-resources`	`--api-group=`	List CRDs in a group	`kubectl api-resources --api-group=web.example.com`
`kubectl apply`	`-f`	Register a CRD	`kubectl apply -f crd.yaml`
`kubectl delete crd`	–	Remove CRD **and** all its instances	`kubectl delete crd websites.web.example.com`
`kubectl get`	`<plural>`	List custom objects	`kubectl get websites`
`kubectl describe`	–	Show details of a CR	`kubectl describe website my-blog`
`kubectl patch`	`--subresource=status`	Update status subresource	`kubectl patch website my-blog --subresource=status --type=merge -p '{"status":{"url":"..."}}'`
`kubectl convert`	–	Convert between CRD versions	`kubectl convert -f old.yaml --output-version=data.example.com/v1`

🛎 Scenarios

Scenario 1 — CRD deletion accidentally wipes production data - **Problem**: An engineer runs `kubectl delete crd databases.example.com`; all `Database` objects are removed and the operator deletes the underlying StatefulSets. - **Resolution**: 1. Before deleting a CRD, always run `kubectl get <plural> -A` to see instances. 2. Add `metadata.finalizers` and set `spec.preserveUnknownFields=false`. 3. Restrict CRD deletion via RBAC to a small `sre` group. 4. Recover from etcd snapshot or from operator-managed backup.

Scenario 2 — CR is stuck in `Terminating` - **Problem**: A finalizer on the CR never gets removed because the operator crashed. - **Resolution**: 1. `kubectl get website my-blog -o json | jq .metadata.finalizers` to inspect. 2. Restart the operator: `kubectl rollout restart deploy/website-operator`. 3. If broken permanently, patch to remove the finalizer: `kubectl patch website my-blog -p '{"metadata":{"finalizers":[]}}' --type=merge` 4. Investigate why the operator failed to remove it (bad code, missing RBAC).

Scenario 3 — CRD version bump breaks existing clients - **Problem**: Team promotes `v1beta1` to `v1` and drops `v1beta1`, breaking CI pipelines still applying old YAML. - **Resolution**: 1. Keep old version `served: true` for at least one release cycle. 2. Provide a conversion webhook so both versions read/write the same data. 3. Update CI YAML gradually; monitor apiserver `apiserver_requested_deprecated_apis` metric.

☁ GCP GKE Perspective

- GKE fully supports CRDs; nothing GKE-specific for the CRD API itself.
- **GKE-installed CRDs** you'll routinely see: `BackendConfig`, `FrontendConfig`, `ManagedCertificate`, `ServiceExport`, `ServiceImport`, `GCPBackupPlan` (Backup for GKE).
- **Config Connector** (KCC) is GCP's product that adds CRDs for GCP resources (BigQuery, Pub/Sub, CloudSQL) — manage GCP with `kubectl`.
- **Autopilot**: CRDs work identically, but you cannot install DaemonSet-based operators in some system namespaces; use `kube-system` alternatives Google provides.
- Command: `gcloud container clusters describe <cluster> --region=<region> --format="value(currentMasterVersion)"` — confirm apiVersion of installed CRDs is served on the cluster version.

▩ AWS EKS Perspective

- EKS supports CRDs identically to upstream.
- **EKS-installed CRDs** you'll see: `ENIConfig` (VPC CNI), `SecurityGroupPolicy`, `IngressClassParams` (AWS Load Balancer Controller), `TargetGroupBinding`.
- **ACK (AWS Controllers for Kubernetes)** installs CRDs for AWS services (S3, SNS, DynamoDB) letting you manage AWS from `kubectl`.
- **Fargate**: CRDs work fine, but operator Pods that must run as DaemonSets won't run on Fargate — put them on managed node groups.
- Command: `eksctl utils update-cluster-logging --enable-types=api,audit --cluster=<name>` — enable audit logs to trace CRD/CR changes.

🧪 Hands-On Lab Exercises

🧪 **Lab 57-A — Create Your First CRD (Beginner)** - **Objective**: Register a `Website` CRD and create instances. - **Prerequisites**: Local Minikube/Kind cluster, `kubectl` v1.28+. - **Environment Setup**:

```
minikube start --kubernetes-version=v1.31.0
kubectl cluster-info
```

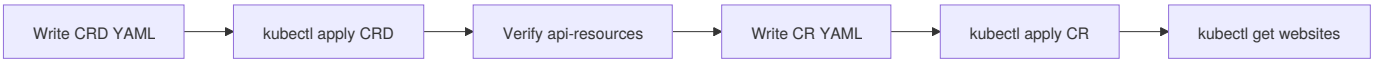
- **Step-by-Step**: 1. Save the CRD YAML from *Example 1* above as `website-crd.yaml`. 2. `kubectl apply -f website-crd.yaml` Expected: `customresourcedefinition.apiextensions.k8s.io/websites.web.example.com created` 3. `kubectl get crd | grep websites` 4. `kubectl explain website.spec` — confirms schema is loaded. 5. Create an instance file `my-blog.yaml` (from the example) and apply. 6. `kubectl get websites -o wide` — see the printer columns. - **Verification**:

```
kubectl get websites
kubectl describe website my-blog
kubectl get website my-blog -o yaml
```

- **Expected Output**: `my-blog` appears with 3 replicas. - **Cleanup**:

```
kubectl delete website my-blog
kubectl delete crd websites.web.example.com
```

- **Troubleshooting Tips**: - "no matches for kind Website" → CRD not applied, or apiVersion mismatch. - Validation errors → check `openAPIV3Schema` block. - **Bonus Challenge**: Add a status subresource update using `kubectl patch --subresource=status`. - **Lab Workflow Diagram**:



Lab 57-B — Multi-version CRD with Deprecation (Intermediate) - **Objective**: Serve `v1alpha1` and `v1` of a CRD; mark `v1alpha1` deprecated. - **Prerequisites**: Kind cluster. - **Steps**: 1. Create CRD with both versions, storage on `v1`. 2. Add `deprecated: true` and `deprecationWarning` on `v1alpha1`. 3. Apply a `v1alpha1` CR and observe the deprecation warning. 4. `kubectl get backup.v1alpha1.data.example.com -o yaml` — see conversion. - **Verification**: kubectl prints the deprecation warning on every apply. - **Cleanup**: `kubectl delete crd backups.data.example.com` - **Bonus**: Add a webhook conversion server (Go/Python).

Lab 57-C — Enforcing Validation with OpenAPI (Advanced) - **Objective**: Prevent invalid CRs via `x-kubernetes-validations` (CEL) rules. - **Prerequisites**: Cluster v1.29+ (CEL GA). - **Steps**: 1. Add to schema:

```
x-kubernetes-validations:
  - rule: "self.replicas <= 10 || self.tier == 'enterprise'"
    message: "More than 10 replicas requires enterprise tier"
```

2. Apply valid + invalid CRs; observe admission rejection. - **Cleanup**: Delete CRD. - **Bonus**: Add `messageExpression` for dynamic error messages.

Interview Deliverable Answer (60–90s)

> "A Custom Resource Definition, or CRD, is how you teach the Kubernetes API server about a brand-new resource type — for example, a `Database` or a `KafkaTopic`. You describe the resource with an OpenAPI schema; once the CRD is applied, kubectl, RBAC, admission controllers, and etcd all treat the new resource exactly like a built-in. The **why** is extensibility: Kubernetes was built to be extended without forking. The **how** is: apply the CRD, then apply instances of the new resource. Behavior comes from pairing the CRD with a controller — that's the Operator Pattern. On GKE we see this with `BackendConfig` and Config Connector, on EKS with `TargetGroupBinding` and ACK. In one of my projects, I authored a CRD for tenant provisioning; my controller reconciled it into a Namespace, a ResourceQuota, and a NetworkPolicy — giving platform users a self-service API in a single YAML."

? Interview Questions

Basic (0–1 yr) 1. **Q**: What is a CRD? **A**: A CRD registers a new resource type with the Kubernetes API server, giving it storage, discovery, and RBAC without changing the API server code.

1. **Q**: How is a CRD different from a Custom Resource (CR)? **A**: A CRD is the *schema* (like a class definition). A CR is an *instance* of that schema (like an object of that class).
2. **Q**: Do you need a controller for a CRD to work? **A**: No — a CRD alone gives you storage and API endpoints. A controller is needed to *act* on the CR (create Pods, etc.). Without one, the CR is just data in etcd.

Intermediate (2 yr) 4. **Q**: How do you validate the fields of a custom resource? **A**: Add `openAPIV3Schema` with types, `required`, `minimum`, `pattern`, and (v1.29+) `x-kubernetes-validations` CEL rules. Validation runs at admission time.

1. **Q**: What is the `status` subresource and why is it important? **A**: It splits `/status` from `/spec` so controllers can update status without racing with user edits to spec. It also enables generation tracking (`metadata.generation` vs `status.observedGeneration`).
2. **Q**: How does multi-version CRD conversion work? **A**: Set multiple `versions`. One is `storage: true`. A conversion webhook (or the simpler `None` strategy) converts requests between served versions and the stored version on read/write.

Advanced (3–4 yr) 7. **Q**: What happens when you delete a CRD? **A**: All CR instances are also deleted. If they have finalizers, delete blocks until finalizers are removed. RBAC and audit records remain until manually cleaned.

1. **Q**: How do finalizers interact with CRs? **A**: A finalizer string on `metadata.finalizers` blocks actual deletion until removed. Controllers use finalizers to run cleanup (e.g., release cloud resources) before Kubernetes garbage-collects the object.
2. **Q**: How do you scale a custom resource with HPA? **A**: Define a `scale` subresource on the CRD pointing to `spec.replicas` and `status.replicas`. Then HPA can target the CR via `scaleTargetRef`.

Scenario-based 10. **Q**: Your CR is stuck in `Terminating` for hours — how do you debug? **A**: Check finalizers, controller logs, and controller RBAC. Restart the controller. As last resort, patch to remove the finalizer.

GKE-specific 11. **Q**: What are `BackendConfig` and `FrontendConfig` on GKE? **A**: GKE-provided CRDs that let you attach GCP Load Balancer settings (CDN, IAP, SSL policy, timeouts) to a `Service` or `Ingress` declaratively.

EKS-specific 12. **Q**: What is `TargetGroupBinding` and when do you use it? **A**: A CRD from AWS Load Balancer Controller that binds an existing AWS Target Group to a Kubernetes Service — useful when the TG is managed by Terraform or shared across clusters.

Hands-on / Lab-based 13. **Q**: Walk me through creating a CRD and validating a bad input is rejected. **A**: (Reference Lab 57-A/C) Apply CRD → kubectl explain → apply valid CR → apply CR violating a CEL rule → observe admission error message.

Topic 58 — Operators and the Operator Pattern

Official Documentation References - https://kubernetes.io/docs/concepts/extend-kubernetes/operator/ - https://operatorframework.io/ - https://book.kubebuilder.io/

Definition

An **Operator** is a Kubernetes controller that packages the operational knowledge (deploy, upgrade, backup, failover) for a specific application, using CRDs to expose that knowledge as a Kubernetes-native API.

About

Kubernetes ships with built-in controllers (Deployment, StatefulSet) that reconcile generic workloads. But stateful apps (PostgreSQL, Kafka, Elasticsearch) have domain-specific operational steps that generic controllers can't express — like "when a primary fails, promote a replica and update DNS."

Why Operators exist - Encode day-2 operations (backup, restore, upgrade, scaling, failover) as code. - Provide a single YAML API for complex systems (`kind: PostgresCluster`). - Eliminate runbooks and human toil.

How they work — the reconciliation loop 1. Watch: subscribe to CR changes via the API server watch. 2. Diff: compare desired state (CR spec) with actual state (real resources). 3. Reconcile: create/update/delete resources to close the gap. 4. Update status: write observed state into `status`. 5. Requeue on error.

Operator maturity model (Capability Levels) | Level | Meaning | |-------|---------| | 1 | Basic install | | 2 | Seamless upgrades | | 3 | Full lifecycle (backup, restore) | | 4 | Deep insights (metrics, alerts) | | 5 | Auto-pilot (tuning, scaling, self-healing) |

Common frameworks - **Kubebuilder** — Go, most idiomatic, controller-runtime based. - **Operator SDK** — wraps Kubebuilder + Ansible + Helm operators. - **KUDO** — declarative operators. - **Metacontroller** — write in any language via webhooks.

🔗 How It Integrates With Other Components

Component	Integration
CRDs	Define the operator's API surface
API Server	Serves the CR endpoints; operator watches via list/watch
Controllers	Operator itself IS a controller
RBAC	Operator runs under a ServiceAccount with tight Role/ClusterRole
Webhooks	Optional mutating/validating webhooks for the CR
Metrics	Operators expose Prometheus metrics on `/metrics`
Leader Election	HA operators use lease-based election
OLM (OperatorLifecycleManager)	Optional package/install/upgrade layer (used on OpenShift)

🏗 Architecture Diagrams

Diagram 1 — Operator reconcile loop

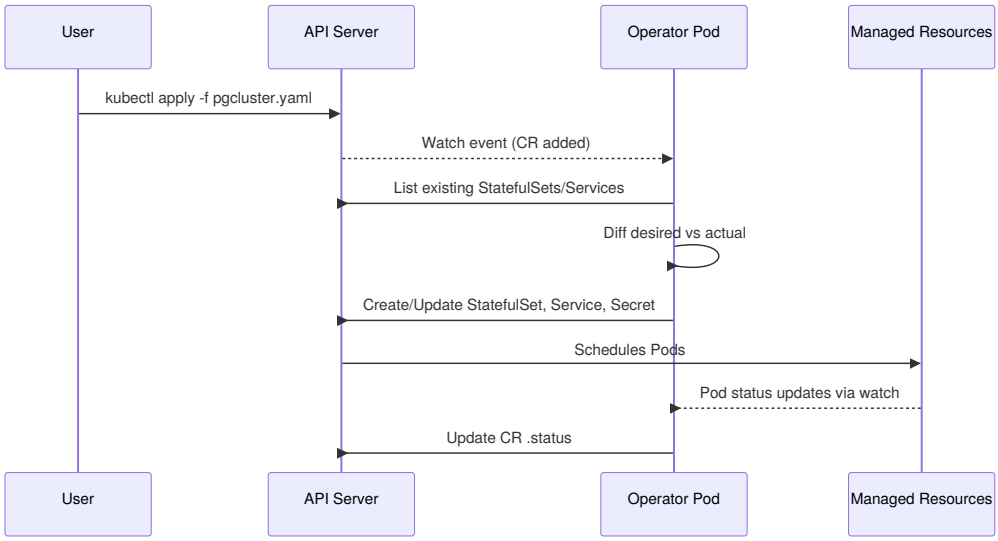


Diagram 2 — Operator components

```
+-----------------------------------------------+
| Operator Pod                                  |
|                                               |
|  [Manager] -> [Controller: PostgresCluster]   |
|      |            |                           |
|      |            +--> Reconciler.Reconcile()  |
|      |                    |                   |
|      |                    +-> client.Get(CR)     |
|      |                    +-> client.Create(SS)  |
|      |                    +-> client.Update(Svc) |
|      |                    +-> client.Status.Update|
|      |                                        |
|      +-> Webhook Server (optional)            |
|      +-> Metrics Server (/metrics)            |
|      +-> Leader Election Lease                |
+-----------------------------------------------+
```

💻 Real-World Examples

Example 1 — Postgres Operator custom resource (Zalando)

```yaml
apiVersion: acid.zalan.do/v1
kind: postgresql
metadata:
  name: acid-minimal-cluster
  namespace: default
spec:
  teamId: "acid"
  volume:
    size: 10Gi
  numberOfInstances: 3
  users:
    zalando:
      - superuser
      - createdb
  databases:
    foo: zalando
  postgresql:
    version: "16"
```

The operator creates: a StatefulSet, headless Service, PDB, Secret with credentials, and a ConfigMap.

Example 2 — Minimal reconciler pseudocode (Kubebuilder / Go)

```
func (r *WebsiteReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    var site webv1.Website
    if err := r.Get(ctx, req.NamespacedName, &site); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }

    // Desired Deployment
    dep := buildDeployment(&site)
    if err := ctrl.SetControllerReference(&site, dep, r.Scheme); err != nil {
        return ctrl.Result{}, err
    }

    // Create or update
    var existing appsv1.Deployment
    if err := r.Get(ctx, client.ObjectKeyFromObject(dep), &existing); err != nil {
        if apierrors.IsNotFound(err) {
            return ctrl.Result{}, r.Create(ctx, dep)
        }
        return ctrl.Result{}, err
    }
    existing.Spec = dep.Spec
    if err := r.Update(ctx, &existing); err != nil {
        return ctrl.Result{}, err
    }

    // Update status
    site.Status.URL = fmt.Sprintf("http://%s.svc", site.Name)
    return ctrl.Result{RequeueAfter: 30 * time.Second}, r.Status().Update(ctx, &site)
}
```

💻 Related Commands

Command	Flag	Purpose	Example
`operator-sdk init`	`--domain=`	Bootstrap operator project	`operator-sdk init --domain=example.com --repo=github.com/me/op`
`operator-sdk create api`	`--group --version --kind`	Scaffold a CRD + controller	`operator-sdk create api --group=web --version=v1 --kind=Website`
`kubebuilder init`	`--domain=`	Same, Kubebuilder	`kubebuilder init --domain=example.com`
`make manifests`	–	Regenerate CRD/RBAC YAML	`make manifests`
`make install`	–	Apply CRDs to cluster	`make install`
`make run`	–	Run controller locally against cluster	`make run`
`make docker-build docker-push`	`IMG=`	Build/push operator image	`make docker-build IMG=me/op:v1`
`make deploy`	`IMG=`	Deploy operator to cluster	`make deploy IMG=me/op:v1`
`kubectl logs`	`-f`	Tail operator logs	`kubectl logs -n op-system deploy/op-controller-manager -f`
`kubectl get pods -n <op-ns>`	–	Verify operator running	`kubectl get pods -n postgres-operator`

🚨 Scenarios

Scenario 1 — Operator hot loop / thrash - **Problem**: Operator writes status → status write triggers watch → reconcile → status write → infinite loop consuming CPU. - **Resolution**: 1. Compare `status.observedGeneration` with `metadata.generation`; only reconcile when spec changes. 2. Use `Result{RequeueAfter: X}` rather than immediate requeue. 3. Predicate-filter watch events to ignore status-only changes.

Scenario 2 — Two operator replicas fighting - **Problem**: Both replicas reconcile, causing duplicate resources and flapping status. - **Resolution**: 1. Enable **leader election** (`--leader-elect=true` , uses a Lease object). 2. Only the leader runs the reconciler; the other stands by. 3. Verify with `kubectl get lease -n <op-ns>` .

Scenario 3 — Operator upgrade breaks CRD schema - **Problem**: v2 operator requires new required field; existing CRs fail validation → outages. - **Resolution**: 1. Ship a conversion webhook or make new field optional. 2. Use `served` / `storage` versioning strategy. 3. Ship a migration Job that updates CRs before switching operator version.

☁ GCP GKE Perspective

- Use the **Google Cloud Marketplace for Kubernetes** to install curated Operators.

- **Config Connector** IS an operator that reconciles GCP resources.
- Best practice: run operators on a **dedicated node pool** with taints so tenant workloads don't preempt them.
- **Workload Identity** integration: assign the operator's ServiceAccount to a GCP IAM identity so the operator can call GCP APIs (e.g., create a Filestore).
- Command: `gcloud container fleet config-management enable` to run a fleet-wide operator (Config Sync).

AWS EKS Perspective

- **ACK (AWS Controllers for Kubernetes)** = per-service operators (S3, RDS, SNS).
- **AWS Load Balancer Controller** is an operator that watches Ingress + Service and provisions ALB/NLB.
- Best practice: give operator pod an **IRSA-bound ServiceAccount** to call AWS APIs.
- On **Fargate**, operators run fine but leader-election Lease must have low renewal deadline (Fargate has cold-starts).
- Command: `eksctl create iamserviceaccount --name=op-sa --namespace=op --attach-policy-arn=...` for IRSA.

Hands-On Lab Exercises

Lab 58-A — Install & Use the Zalando Postgres Operator (Beginner) - **Objective**: Deploy a Postgres cluster via an operator. - **Prerequisites**: Kind/Minikube, kubectl, Helm. - **Environment**:

```
kind create cluster --name op-lab
helm repo add postgres-operator-charts https://opensource.zalando.com/postgres-operator/charts/postgres-operator
helm install postgres-operator postgres-operator-charts/postgres-operator
```

- **Steps**: 1. `kubectl get pods -l app.kubernetes.io/name=postgres-operator` 2. `kubectl apply -f pgcluster.yaml` (Example 1) 3. `kubectl get postgresql` 4. `kubectl get pods -l cluster-name=acid-minimal-cluster` - **Verification**:

```
kubectl exec -it acid-minimal-cluster-0 -- psql -U postgres -c '\l'
```

- **Cleanup**: `kubectl delete postgresql acid-minimal-cluster && helm uninstall postgres-operator` - **Troubleshooting**: If Pods stuck Pending → check PVC/StorageClass. - **Bonus**: Scale from 3 → 5 replicas and watch operator reconcile.

Lab 58-B — Build a "Hello Operator" with Kubebuilder (Intermediate) - **Objective**: Scaffold a `Website` operator that creates a Deployment + Service. - **Prerequisites**: Go 1.22+, kubebuilder v3.14+, Docker, Kind. - **Steps**: 1. `mkdir website-op && cd website-op && kubebuilder init --domain=example.com --repo=github.com/me/website-op` 2. `kubebuilder create api --group=web --version=v1 --kind=Website` 3. Edit `api/v1/website_types.go` — add `GitRepo` and `Replicas` fields. 4. Edit `internal/controller/website_controller.go` — add reconcile logic (Example 2). 5. `make manifests && make install` 6. `make run` (runs controller against cluster) 7. In another terminal: `kubectl apply -f config/samples/web_v1_website.yaml` 8. `kubectl get deployments` - **Verification**: Deployment `my-blog` exists with 3 replicas. - **Cleanup**: `kubectl delete website my-blog && make uninstall` - **Bonus**: Add owner references so the Deployment is garbage-collected when the CR is deleted.

Lab 58-C — Package Operator with OLM & Ship a v2 Upgrade (Advanced) - **Objective**: Bundle the operator, publish to a catalog, install via Subscription, and upgrade to v2. - **Prerequisites**: OLM installed (`operator-sdk olm install`), image registry. - **Steps**: 1. `make bundle IMG=me/op:v1 VERSION=1.0.0` 2. `make bundle-build bundle-push` 3. Create CatalogSource + Subscription. 4. Update code → v2 → repeat bundle steps with `VERSION=2.0.0` and `--channels=stable` . 5. Trigger upgrade via `kubectl patch subscription` . - **Verification**: Operator Pod uses v2 image, CRD conversion webhook active. - **Cleanup**: `operator-sdk cleanup my-op` . - **Bonus**: Write conversion tests using envtest.

Interview Deliverable Answer (60-90s)

> "An Operator is a Kubernetes controller plus one or more CRDs that codify the operational knowledge for a specific application. The **what** is: I get a Kubernetes-native API — say `kind: PostgresCluster` — and the operator's reconciliation loop keeps the real Postgres cluster matching that spec, handling installs, upgrades, failover, and backups. The **why** is toil elimination — runbooks become code. The **how** is a control loop: watch, diff, act, update status, repeat. On EKS I've used ACK to manage RDS from Kubernetes and the AWS Load Balancer Controller for Ingress; on GKE, Config Connector plays a similar role. I built one myself with Kubebuilder — it exposed a `Tenant` CRD that provisioned a namespace, quota, and network policy in one shot, cutting onboarding from a runbook to a single YAML."

? Interview Questions

Basic 1. **Q**: What is an Operator? **A**: A controller that uses CRDs to manage a specific application's lifecycle inside Kubernetes.

1. **Q**: Difference between a Controller and an Operator? **A**: All operators are controllers; not all controllers are operators. Operators are domain-specific (Postgres, Kafka) and typically ship with CRDs.

Intermediate 3. **Q**: Explain the reconcile loop. **A**: Get desired state (CR), compare to actual, take action to close the gap, update status, requeue on error.

1. **Q**: What is leader election and why? **A**: Uses a Lease object so only one replica reconciles; prevents duplicate actions in HA setups.
2. **Q**: How do you prevent hot-loops? **A**: Compare `observedGeneration` to `generation` ; use `RequeueAfter` ; filter watch predicates.

Advanced 6. **Q**: What are Operator capability levels? **A**: 1 (install), 2 (seamless upgrade), 3 (full lifecycle), 4 (deep insights), 5 (auto-pilot).

1. **Q**: How would you test an operator? **A**: Unit-test the reconciler function; integration-test with `envtest` (fake apiserver); e2e-test with Kind.

Scenario 8. **Q**: Two operator pods are running and creating duplicate resources — what's wrong? **A**: Leader election disabled. Enable `--leader-elect=true` and verify with `kubectl get lease` .

GKE 9. **Q**: How does Config Connector integrate with Workload Identity? **A**: The Config Connector service account is bound to a GCP IAM SA via Workload Identity; the operator calls GCP APIs as that identity.

EKS 10. **Q**: How do you give an operator AWS API access on EKS? **A**: IRSA — create an IAM role with trust policy for the OIDC provider, annotate the ServiceAccount, mount the projected token.

Lab-based 11. **Q**: Walk me through building an operator with Kubebuilder. **A**: (Lab 58-B) init → create api → edit types → edit controller → make manifests/install/run → apply CR → verify.

🗄 Topic 59 — Helm (Charts, Templating, Releases)

Official Documentation References - https://helm.sh/docs/ - https://helm.sh/docs/topics/charts/ - https://helm.sh/docs/chart_template_guide/ - https://helm.sh/docs/topics/architecture/

🗄 Definition

Helm is the package manager for Kubernetes; it bundles YAML manifests into reusable, versioned units called **charts** and installs them as **releases** with templated, parameterized values.

📖 About

Managing hundreds of YAML files across environments (dev/stage/prod) creates duplication and drift. Helm solves this by treating a set of manifests as a chart, letting you template values, version the chart, and install/upgrade/rollback deterministically.

Core concepts - **Chart** — a directory of templates + `Chart.yaml` + `values.yaml` . - **Release** — an installed instance of a chart (name + namespace). - **Repository** — HTTP server (or OCI registry) hosting charts. - **Values** — parameters supplied at install time. - **Template** — Go text/template YAML rendered with values, Sprig functions, and Helm-specific helpers. - **Hooks** — templates that run at pre-install / post-install / pre-upgrade / etc. - **Release history** — Helm stores each revision in a Secret in the release's namespace.

Chart structure

```
mychart/
├── Chart.yaml
├── values.yaml
├── charts/               # sub-charts
├── templates/
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── NOTES.txt
│   └── tests/
└── crds/                 # applied before templates, never upgraded
```

Why Helm exists - DRY across environments. - Version-controlled application delivery. - Shareable community catalogs (Bitnami, Artifact Hub). - Atomic install/upgrade/rollback.

Helm 3 vs Helm 2 Helm 3 removed Tiller (server component); releases are stored as Secrets, and RBAC is enforced via the user's kubeconfig.

🔗 How It Integrates With Other Components

Component	Integration
kubectl / kubeconfig	Helm uses the current context
API Server	Applies rendered YAML
Secrets	Release history storage (default)
RBAC	User running helm must have permissions for the objects
OCI registries	Helm 3.8+ pushes/pulls charts as OCI artifacts
CI/CD	`helm upgrade --install` is the standard pipeline step
ArgoCD / Flux	Both consume Helm charts natively
CRDs	Placed in `crds/` and applied first (not templated)

🏗 Architecture / Flow Diagram

Diagram 1 — helm install flow

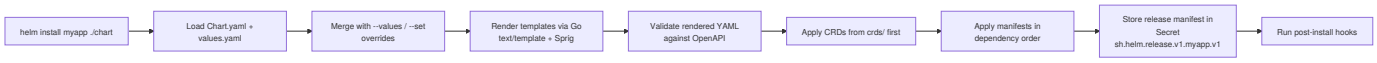


Diagram 2 — Release history / rollback

```
helm install foo ./chart      -> revision 1  (Secret: sh.helm.release.v1.foo.v1)
helm upgrade foo ./chart      -> revision 2  (Secret: sh.helm.release.v1.foo.v2)
helm upgrade foo ./chart      -> revision 3  (Secret: sh.helm.release.v1.foo.v3)
helm rollback foo 1           -> revision 4  (copy of revision 1, marked deployed)
```

💻 Real-World Examples

Example 1 — A minimal chart

`Chart.yaml` :

```
apiVersion: v2
name: hello
description: A minimal Helm chart
type: application
version: 0.1.0
appVersion: "1.0.0"
```

`values.yaml` :

```
replicaCount: 2
image:
  repository: nginx
  tag: "1.27"
service:
  type: ClusterIP
  port: 80
```

`templates/_helpers.tpl`:

```
{{- define "hello.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" -}}
{{- end -}}
```

`templates/deployment.yaml`:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "hello.name" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ include "hello.name" . }}
  template:
    metadata:
      labels:
        app: {{ include "hello.name" . }}
    spec:
      containers:
        - name: web
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
```

`templates/service.yaml`:

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "hello.name" . }}
spec:
  type: {{ .Values.service.type }}
  selector:
    app: {{ include "hello.name" . }}
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
```

Install:

```
helm install hello ./hello --set replicaCount=5
```

Example 2 — Environment-specific values

`values-prod.yaml`:

```
replicaCount: 10
image:
  tag: "1.27.1"
service:
  type: LoadBalancer
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1",    memory: "1Gi" }
```

```
helm upgrade --install hello ./hello \
  -f values.yaml -f values-prod.yaml \
  --namespace prod --create-namespace --atomic --timeout=5m
```

💻 Related Commands

Command	Flag	Purpose	Example
`helm create`	–	Scaffold new chart	`helm create mychart`
`helm lint`	–	Validate chart syntax	`helm lint ./mychart`
`helm template`	`--debug`	Render without installing	`helm template rel ./mychart --debug`
`helm install`	`--values -f`	Extra values file	`helm install app ./chart -f prod.yaml`
`helm install`	`--set`	Override single value	`helm install app ./chart --set replicaCount=3`
`helm install`	`--dry-run`	Simulate install	`helm install app ./chart --dry-run`
`helm install`	`--atomic`	Rollback on failure	`helm install app ./chart --atomic`
`helm install`	`--wait --timeout`	Wait for readiness	`helm install app ./chart --wait --timeout=10m`
`helm upgrade`	`--install`	Install if missing	`helm upgrade --install app ./chart`
`helm rollback`	`<REL> <REV>`	Roll back to revision	`helm rollback app 3`
`helm history`	–	Show revisions	`helm history app`
`helm uninstall`	`--keep-history`	Delete release	`helm uninstall app --keep-history`
`helm list`	`-A`	All namespaces	`helm list -A`
`helm repo add`	–	Add chart repo	`helm repo add bitnami https://charts.bitnami.com/bitnami`
`helm repo update`	–	Refresh index	`helm repo update`
`helm search repo`	–	Search charts	`helm search repo nginx`
`helm pull`	`--untar`	Download chart	`helm pull bitnami/nginx --untar`
`helm dependency update`	–	Refresh sub-charts	`helm dependency update`
`helm push`	–	Push to OCI registry	`helm push mychart-0.1.0.tgz oci://registry.example.com/charts`
`helm get manifest`	–	Show rendered manifest	`helm get manifest app`
`helm get values`	`--all`	Show effective values	`helm get values app --all`
`helm test`	–	Run chart tests	`helm test app`

🎬 Scenarios

Scenario 1 — Failed upgrade leaves release in `pending-upgrade` - **Problem**: `helm upgrade` fails mid-flight; subsequent runs error with "another operation in progress." - **Resolution**: 1. `helm history app` — identify the stuck revision. 2. `helm rollback app <last-good>` — force back to previous. 3. If rollback fails, delete the stuck release Secret manually: `kubectl delete secret sh.helm.release.v1.app.v<N>` and retry. 4. Always use `--atomic` in CI to auto-rollback on failure.

Scenario 2 — CRD updates not applied on upgrade - **Problem**: Chart in `crds/` gets installed once and never updated (by design). - **Resolution**: 1. Move CRDs to `templates/` with `helm.sh/hook: crd-install` (deprecated) or manage CRDs separately via a dedicated chart. 2. Best practice: apply CRDs out-of-band with `kubectl apply --server-side -f crds/`.

Scenario 3 — Secret leaks in Helm values in Git - **Problem**: `values.yaml` with plaintext secrets committed to Git. - **Resolution**: 1. Move secrets to Kubernetes `Secret` referenced by `secretKeyRef` in templates. 2. Use `helm-secrets` plugin + SOPS or Sealed Secrets or External Secrets Operator. 3. On GKE, use Secret Manager + `ExternalSecret`; on EKS, use AWS Secrets Manager + `SecretStore`.

☁ GCP GKE Perspective

- Helm works identically on GKE Standard & Autopilot.
- **Google Cloud Marketplace** publishes Helm-compatible apps.
- Store charts in **Artifact Registry OCI** (`helm push` supported).
- Autopilot note: hooks that use `hostPath` or privileged containers will be rejected — refactor chart.
- Command:

```
gcloud auth configure-docker <region>-docker.pkg.dev
helm registry login <region>-docker.pkg.dev
helm push mychart-0.1.0.tgz oci://<region>-docker.pkg.dev/<project>/charts
```

▩ AWS EKS Perspective

- Store charts in **ECR OCI** (helm 3.8+).

```
aws ecr get-login-password | helm registry login --username AWS --password-stdin <acct>.dkr.ecr.<region>.amazonaws.com
helm push mychart-0.1.0.tgz oci://<acct>.dkr.ecr.<region>.amazonaws.com/charts
```

- Common EKS Helm installs: `aws-load-balancer-controller`, `cluster-autoscaler`, `external-dns`, `metrics-server`, `karpenter`.
- On Fargate, Helm hooks (Jobs) must target a Fargate profile-matching namespace.

🧪 Hands-On Lab Exercises

🧪 **Lab 59-A — Package Your First Chart (Beginner)** - **Objective**: Create, lint, install, upgrade, rollback. - **Prerequisites**: Minikube, `helm` v3.14+. - **Steps**: 1. `helm create demo` 2. Edit `values.yaml`: set `replicaCount: 2`. 3. `helm lint ./demo` 4. `helm template test ./demo | less` 5. `helm install demo ./demo` 6. `kubectl get pods -l app.kubernetes.io/instance=demo` 7. Edit `values.yaml` → `replicaCount: 4` 8. `helm upgrade demo ./demo` 9. `helm history demo` 10. `helm rollback demo 1` - **Verification**: `kubectl get deploy demo-demo -w` - **Cleanup**: `helm uninstall demo` - **Bonus**: Add a NOTES.txt with a helpful post-install message.

🧪 **Lab 59-B — Multi-env Deployment with Values Files (Intermediate)** - **Objective**: Same chart, three environments. - **Prerequisites**: Kind cluster, 3

namespaces. - **Steps**: 1. Create `values-dev.yaml`, `values-stage.yaml`, `values-prod.yaml`. 2. Install to 3 namespaces:

```
for env in dev stage prod; do
  helm upgrade --install app ./demo -f values-${env}.yaml \
    --namespace ${env} --create-namespace --atomic
done
```

3. Compare with `helm get values app -n dev`. - **Verification**: Each ns has different replica count. - **Bonus**: Add ImagePullSecrets only for prod via conditional template.

Lab 59-C — Helm + OCI Registry + CI (Advanced) - **Objective**: Push chart to OCI and install from it. - **Prerequisites**: ECR or GAR registry, credentials. - **Steps**: 1. `helm package ./demo` 2. `helm push demo-0.1.0.tgz oci://<registry>/charts` 3. In another cluster: `helm install demo oci://<registry>/charts/demo --version 0.1.0` - **Verification**: Release installed from OCI. - **Bonus**: Wire into GitHub Actions on push to `main`.

Interview Deliverable Answer (60-90s)

> "Helm is the package manager for Kubernetes. It bundles YAML manifests into a versioned, parameterized package called a chart, and installs each instance as a release. **Why**: managing hundreds of raw YAMLs across dev/stage/prod becomes copy-paste hell; Helm lets me template with values, upgrade atomically, and roll back if something breaks. **How**: `helm install` merges values, renders Go templates, applies the result, and stores the release history as a Kubernetes Secret. I use `--atomic --wait` in CI so failed upgrades auto-revert. On EKS I push charts to ECR as OCI artifacts and pull them from ArgoCD; on GKE I use Artifact Registry. In one project we cut onboarding a new microservice from days to minutes by parameterizing a single 'workload' chart every team could inherit."

? Interview Questions

Basic 1. **Q**: What is a Helm chart? **A**: A packaged collection of templates + values that describes a Kubernetes application.

1. **Q**: Difference between chart and release? **A**: Chart is the package; release is an installed instance of that chart with specific values.

Intermediate 3. **Q**: How do you override values? **A**: `--set` for single values, `-f/--values` for files (multiple, right-most wins).

1. **Q**: Where does Helm 3 store release state? **A**: In a Secret named `sh.helm.release.v1.<release>.v<revision>` in the release's namespace.
2. **Q**: What does `--atomic` do? **A**: On failure, automatically rolls back to the prior revision.

Advanced 6. **Q**: How do you handle CRDs in a chart? **A**: Place them in `crds/` for install-only. For upgrades, manage CRDs separately via `kubectl apply --server-side` or a dedicated ops chart.

1. **Q**: Helm 2 vs Helm 3 major differences? **A**: No Tiller in v3, Secret-based storage, RBAC via kubeconfig, OCI registry support, three-way strategic merge → JSON merge patch during upgrade.

Scenario 8. **Q**: Release stuck in `pending-upgrade` — how to fix? **A**: Roll back to last good revision; if the state Secret is corrupt, delete it and retry.

GKE-specific 9. **Q**: How do you host Helm charts on GCP? **A**: Push as OCI artifacts to Artifact Registry; authenticate with `gcloud auth configure-docker` then `helm registry login`.

EKS-specific 10. **Q**: How do you deploy AWS Load Balancer Controller? **A**: Add the eks-charts Helm repo, create IRSA-bound ServiceAccount, `helm install aws-load-balancer-controller eks/aws-load-balancer-controller --set clusterName=<name>`.

Lab-based 11. **Q**: Walk me through parameterizing a chart for three environments. **A**: (Lab 59-B) Create `values-<env>.yaml` per environment, run `helm upgrade --install` with layered `-f` flags per namespace.

Topic 60 — Kustomize

Official Documentation References - https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/ - https://kubectl.docs.kubernetes.io/references/kustomize/ - https://kustomize.io/

Definition

Kustomize is a template-free, YAML-native customization tool built into `kubectl` (`-k`) that composes and overlays vanilla Kubernetes manifests using a `kustomization.yaml` file.

About

Where Helm uses Go templates and values files, Kustomize uses **base + overlay** layering. Bases are plain, valid Kubernetes YAML; overlays patch or transform them for each environment. Nothing is templated — every file remains a legal manifest.

Why Kustomize exists - Some teams find Go templating fragile and error-prone. - Kustomize preserves YAML validity; you can `kubectl apply -f` any base directly. - Purely declarative composition.

Core concepts - **Base** — a directory with a `kustomization.yaml` referencing resources. - **Overlay** — a directory that references a base and adds patches. - **Patches** — strategic merge patches or JSON6902 patches. - **Generators** — `configMapGenerator`, `secretGenerator` produce hashed ConfigMaps/Secrets. - **Transformers** — namespace, name prefix/suffix, labels, annotations, images, replicas. - **Components** — reusable slices of customization (v1beta1+).

Kustomize vs Helm | Aspect | Helm | Kustomize | |--------|------|-----------| | Templating | Go templates | None (pure YAML) | | Packaging | `.tgz` chart | Directory tree | | Distribution | Repos / OCI | Git / OCI | | Rollback | Built-in | External (Git) | | Learning curve | Steeper | Gentler | | Composability | Sub-charts | Bases + overlays + components |

How It Integrates With Other Components

Component	Integration
kubectl	`kubectl apply -k <dir>` renders + applies
ArgoCD / Flux	Native Kustomize support (Application spec has `kustomize:` block)
Helm	Kustomize can post-render Helm output
CRDs	Fully supported; use `configurations:` to teach Kustomize about custom name refs
CI	`kustomize build` → apply / dry-run

🏗 Architecture / Flow Diagram

Diagram 1 — base + overlays

```
base/
├── kustomization.yaml
├── deployment.yaml
├── service.yaml
overlays/
├── dev/kustomization.yaml   -> ../../base + replicas=1 + image tag dev
├── stage/kustomization.yaml -> ../../base + replicas=3 + tag stage
└── prod/kustomization.yaml  -> ../../base + replicas=10 + tag prod
```

Diagram 2 — build pipeline



💻 Real-World Examples

Example 1 — Base + prod overlay

`base/deployment.yaml` :

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports: [{ containerPort: 80 }]
```

`base/service.yaml` :

```yaml
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web }
  ports: [{ port: 80, targetPort: 80 }]
```

`base/kustomization.yaml` :

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
commonLabels:
  app.kubernetes.io/part-of: shop
```

`overlays/prod/kustomization.yaml` :

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: prod
namePrefix: prod-
resources:
  - ../../base
images:
  - name: nginx
    newTag: "1.27.1"
replicas:
  - name: web
    count: 10
patches:
  - target: { kind: Deployment, name: web }
    patch: |-
      - op: add
        path: /spec/template/spec/containers/0/resources
        value:
          requests: { cpu: "500m", memory: "512Mi" }
          limits:   { cpu: "1",    memory: "1Gi" }
configMapGenerator:
  - name: web-config
    literals:
      - LOG_LEVEL=INFO
      - REGION=us-east-1
```

Apply:

```
kubectl apply -k overlays/prod
```

Example 2 — Kustomize as post-renderer for Helm

`kustomization.yaml` :

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - all.yaml    # helm template output
patches:
  - target: { kind: Deployment, name: nginx }
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/imagePullPolicy
        value: Always
```

Script `post-renderer.sh` :

```
#!/usr/bin/env bash
cat > all.yaml
kustomize build .
```

Then:

```
helm install my-nginx bitnami/nginx --post-renderer ./post-renderer.sh
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl apply`	`-k`	Apply a kustomization dir	`kubectl apply -k overlays/prod`
`kubectl kustomize`	–	Render only	`kubectl kustomize overlays/prod`
`kustomize build`	–	Render (standalone binary)	`kustomize build overlays/prod`
`kustomize edit set image`	–	Update image in kustomization.yaml	`kustomize edit set image nginx=nginx:1.27.2`
`kustomize edit set replicas`	–	Update replica count	`kustomize edit set replicas web=5`
`kustomize edit add resource`	–	Append resource file	`kustomize edit add resource pdb.yaml`
`kustomize create`	`--resources`	Bootstrap kustomization.yaml	`kustomize create --resources deploy.yaml,svc.yaml`
`kustomize edit set namespace`	–	Set target namespace	`kustomize edit set namespace prod`
`kubectl delete`	`-k`	Delete resources from a kustomization	`kubectl delete -k overlays/prod`

🎬 Scenarios

Scenario 1 — Rolling out a hotfix image tag safely - **Problem**: Prod bug requires image bump from `1.27` → `1.27.1` immediately. - **Resolution**: 1. `cd overlays/prod && kustomize edit set image nginx=nginx:1.27.1` 2. `git diff` shows only `kustomization.yaml` changed. 3. Commit + PR + CI applies via `kubectl apply -k overlays/prod` . 4. Argo/Flux auto-syncs. Rollback = revert commit.

Scenario 2 — Generator name changes break references - **Problem**: `configMapGenerator` adds a hash suffix (`web-config-abc123`); a Deployment referencing `web-config` fails. - **Resolution**: 1. Kustomize auto-updates references *if* you set the name via `envFrom.configMapRef.name` . 2. Alternatively set `generatorOptions.disableNameSuffixHash: true` (loses immutability benefit).

Scenario 3 — Two overlays merge-conflict on the same field - **Problem**: A `component` overlay and an env overlay both patch container image. - **Resolution**: 1. Order matters — later patches win. Reorganize. 2. Use JSON6902 with distinct paths. 3. Use `patchesStrategicMerge` on separate containers.

☁ GCP GKE Perspective

- GKE **Config Sync** (Anthos) uses Kustomize + Git to reconcile fleet-wide config.
- GKE ships with `kubectl` that has Kustomize built-in (v1.14+).
- Best pattern: base + per-cluster overlay (`overlays/prod-us` , `overlays/prod-eu`) driven by Config Sync.

▩ AWS EKS Perspective

- AWS CDK for Kubernetes (cdk8s) can emit manifests consumed by Kustomize overlays.
- Common pattern: base workload + IRSA overlay adding `eks.amazonaws.com/role-arn` annotation via patch.
- Flux on EKS commonly renders Kustomize; no EKS-specific requirement beyond IAM/OIDC for the controller.

🧪 Hands-On Lab Exercises

🧪 **Lab 60-A — Your First Base + Overlays (Beginner)** - **Objective**: Build base with dev/prod overlays. - **Prerequisites**: `kubectl` v1.28+ (Kustomize built-in) or `kustomize` v5+, Minikube. - **Steps**: 1. Create `base/` with the files from Example 1. 2. Create `overlays/dev/kustomization.yaml` :

```
resources: [../../base]
namespace: dev
replicas: [{ name: web, count: 1 }]
```

3. Create `overlays/prod/` (Example 1). 4. `kubectl kustomize overlays/dev` 5. `kubectl create ns dev prod` 6. `kubectl apply -k overlays/dev` and `kubectl apply -k overlays/prod` - **Verification**: Different replica counts per ns. - **Cleanup**: `kubectl delete -k overlays/dev && kubectl delete -k overlays/prod` - **Bonus**: Add a `commonLabels` in base and confirm it appears in every resource.

Lab 60-B — Components & Multi-team Composition (Intermediate) - **Objective**: Extract a "monitoring sidecar" into a reusable Component. - **Prerequisites**: Kustomize v4+. - **Steps**: 1. Create `components/monitoring/kustomization.yaml` with `kind: Component`. 2. Add sidecar patch injecting a Prometheus exporter. 3. In `overlays/prod` add: `components: [../../components/monitoring]` 4. Build & inspect. - **Verification**: Only prod has the sidecar. - **Bonus**: Add a `security` component adding a `securityContext` and use it in both dev & prod.

Lab 60-C — Kustomize + ArgoCD Sync (Advanced) - **Objective**: ArgoCD auto-syncs an overlay from Git. - **Prerequisites**: ArgoCD installed on GKE/EKS/Kind. - **Steps**: 1. Push base+overlays to Git. 2. Create `Application` CR pointing to `overlays/prod` with `kustomize: {}`. 3. Change image tag; commit; ArgoCD syncs automatically. - **Verification**: Rollout observed in ArgoCD UI. - **Bonus**: Use `kustomize.namePrefix` from ArgoCD ApplicationSet template.

Interview Deliverable Answer (60-90s)

> "Kustomize is Kubernetes' native, template-free way to customize manifests. Instead of Go templates, you keep a plain YAML **base** that's a valid manifest on its own, and layer **overlays** for each environment that patch fields — namespace, replica count, image tag, labels — without touching the base. **Why**: templating is fragile; Kustomize preserves YAML validity end-to-end, and CI diffs are readable. **How**: `kubectl apply -k` renders and applies the overlay. On GKE, Config Sync uses Kustomize to fleet-manage clusters; on EKS, Flux consumes overlays from Git. I use it heavily with ArgoCD — one base, three overlays (dev, stage, prod), promotion is just `kustomize edit set image` and a Git push."

? Interview Questions

Basic 1. **Q**: What is Kustomize? **A**: A template-free customization tool built into kubectl that composes vanilla YAML using bases and overlays.

1. **Q**: Helm vs Kustomize? **A**: Helm templates + packages; Kustomize overlays + patches. Helm has releases/rollback; Kustomize relies on Git.

Intermediate 3. **Q**: What's the difference between `patchesStrategicMerge` and JSON6902? **A**: Strategic merge deep-merges YAML the same way `kubectl apply` does; JSON6902 uses precise ops (add/remove/replace) on specific paths.

1. **Q**: Why does `configMapGenerator` add a hash suffix? **A**: To make ConfigMaps immutable and trigger Pod rollouts when the data changes.

Advanced 5. **Q**: What is a Kustomize Component? **A**: A reusable customization slice (`kind: Component`) that can be included in any overlay to add optional features.

1. **Q**: How does Kustomize handle CRDs? **A**: Via the `configurations:` field pointing to a config that teaches Kustomize the ref paths inside custom resources.

Scenario 7. **Q**: Two overlays patch the same field — which wins? **A**: The one applied later in the resource list; reorder or split with distinct JSON6902 paths.

GKE-specific 8. **Q**: What is Anthos Config Sync? **A**: A GKE-native GitOps agent that syncs Kustomize/YAML from a Git repo to a fleet of clusters.

EKS-specific 9. **Q**: How would you inject an IRSA annotation across all overlays? **A**: Create a base ServiceAccount patched per overlay with the environment-specific IAM role ARN.

Lab-based 10. **Q**: Walk me through promoting an image tag from dev → prod. **A**: (Lab 60-A) `kustomize edit set image` in the prod overlay, commit, PR, apply / GitOps sync.

Topic 61 — GitOps (ArgoCD, Flux)

Official Documentation References - https://opengitops.dev/ - https://argo-cd.readthedocs.io/en/stable/ - https://fluxcd.io/flux/ - https://kubernetes.io/docs/concepts/cluster-administration/manage-deployment/

Definition

GitOps is an operating model where Git is the single source of truth for declarative infrastructure, and a controller in the cluster continuously reconciles the live state to match the Git-stored desired state.

About

Traditional CI/CD pushes to clusters from outside. GitOps flips this: the cluster **pulls** desired state from Git, so the cluster is always convergent, self-healing, and auditable.

Four GitOps principles (OpenGitOps) 1. **Declarative** — everything as code. 2. **Versioned & Immutable** — Git history is the audit log. 3. **Pulled Automatically** — agents fetch approved changes. 4. **Continuously Reconciled** — drift is detected and corrected.

Why GitOps - Auditability: every change is a Git commit. - Reversibility: revert = rollback. - Multi-cluster consistency at scale. - Reduced blast radius: no cluster credentials in CI runners. - Secure by default: cluster only needs pull access to Git.

ArgoCD vs Flux

Feature	ArgoCD	Flux
UI	Rich web UI	CLI-first, community UIs
App model	`Application` CRD	`Kustomization` + `HelmRelease` CRDs
Multi-tenancy	`AppProject`	Namespaced by default
Progressive delivery	Argo Rollouts	Flagger
Notifications	Argo Notifications	Notification Controller
ApplicationSet	Yes (templated multi-app)	Similar via `Kustomization` templating
SSO	OIDC, SAML, Dex	Via Kubernetes RBAC
CNCF status	Graduated	Graduated

How It Integrates With Other Components

Component	Integration
Git provider	Source of truth
Kustomize / Helm	Rendering engines
Kubernetes API	Reconcile target
RBAC	Agent gets a ServiceAccount with cluster-admin (scoped)
Secrets management	ExternalSecrets, Sealed Secrets, SOPS integration
CI (GitHub Actions, GitLab)	Build image → PR to config repo
Notifications	Slack, Teams, email on sync events
Prometheus	Sync metrics
SSO	ArgoCD via OIDC/SAML

🏗 Architecture / Flow Diagram

Diagram 1 — Classic GitOps flow

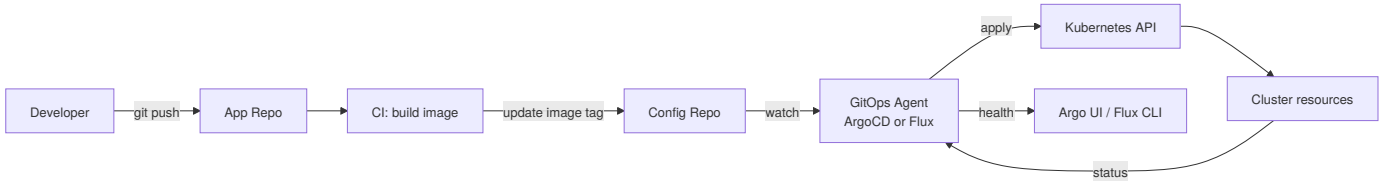


Diagram 2 — Multi-cluster fan-out with ApplicationSet

```
              +--------------------------+
              |  Config Git Repo (mono)   |
              +-----+---------+----------+
                    |         |
       +------------+         +-------------+
       |                                    |
   ApplicationSet                       ApplicationSet
   (generator: List of clusters)        (generator: Git)
       |                                    |
  +----+----+----+                    +----+----+
  |    |    |    |                    |    |    |
 EKS  GKE  Kind ...                   dev  stg  prod
```

💻 Real-World Examples

Example 1 — ArgoCD `Application` for a Helm chart

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: shop-prod
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/acme/shop-config
    targetRevision: main
    path: charts/shop
    helm:
      valueFiles:
        - values-prod.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: shop-prod
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true
    retry:
      limit: 5
      backoff:
        duration: 5s
        factor: 2
        maxDuration: 3m
```

Example 2 — Flux `Kustomization` + `HelmRelease`

`GitRepository`:

```yaml
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: shop-config
  namespace: flux-system
spec:
  interval: 1m
  url: https://github.com/acme/shop-config
  ref: { branch: main }
```

`Kustomization`:

```
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata:
  name: shop-prod
  namespace: flux-system
spec:
  interval: 5m
  path: ./overlays/prod
  prune: true
  sourceRef:
    kind: GitRepository
    name: shop-config
  targetNamespace: shop-prod
  wait: true
  timeout: 5m
```

💻 Related Commands

ArgoCD

Command	Flag	Purpose	Example
`argocd login`	`--sso`	Log into ArgoCD	`argocd login argocd.example.com --sso`
`argocd app list`	–	List apps	`argocd app list`
`argocd app get`	–	App details	`argocd app get shop-prod`
`argocd app sync`	`--prune --force`	Force sync	`argocd app sync shop-prod --prune`
`argocd app rollback`	`<APP> <HISTORY-ID>`	Roll back	`argocd app rollback shop-prod 3`
`argocd app diff`	–	Live vs Git diff	`argocd app diff shop-prod`
`argocd app history`	–	Sync history	`argocd app history shop-prod`
`argocd repo add`	`--username --password`	Register repo	`argocd repo add https://... --username x --password y`
`argocd proj create`	–	Create AppProject	`argocd proj create shop --dest ... --src ...`
`argocd cluster add`	–	Register external cluster	`argocd cluster add prod-context`

Flux

Command	Flag	Purpose	Example
`flux bootstrap github`	`--owner --repository`	Install Flux + push manifests to Git	`flux bootstrap github --owner=acme --repository=fleet`
`flux create source git`	`--url --branch`	Add GitRepository	`flux create source git shop --url=https://... --branch=main`
`flux create kustomization`	`--source --path`	Add Kustomization	`flux create kustomization shop --source=shop --path=./overlays/prod`
`flux reconcile ks`	`--with-source`	Force reconcile	`flux reconcile ks shop --with-source`
`flux suspend ks`	–	Pause reconciliation	`flux suspend ks shop`
`flux resume ks`	–	Resume	`flux resume ks shop`
`flux get kustomizations`	–	Status	`flux get kustomizations`
`flux logs`	`-f`	Controller logs	`flux logs -f`

🎪 Scenarios

Scenario 1 — Drift from manual `kubectl edit` - **Problem**: An engineer edits a Deployment directly; ArgoCD/Flux keeps reverting it (or shows OutOfSync). - **Resolution**: 1. **This is the point** of GitOps — drift is detected and healed. 2. Educate: change must go through Git PR. 3. Enforce with RBAC: only ArgoCD SA has write access to that namespace. 4. Enable `selfHeal: true` (ArgoCD) or leave `Kustomization.prune: true` (Flux) to auto-revert.

Scenario 2 — Secret rotation in a GitOps world - **Problem**: You can't commit plaintext secrets to Git. - **Resolution**: 1. Use **Sealed Secrets** (Bitnami) — encrypt with cluster public key. 2. Or **External Secrets Operator** — pull from GCP Secret Manager / AWS Secrets Manager on reconcile. 3. Or **SOPS + age/KMS** — encrypt files, decrypt in agent. 4. Never store plaintext; RBAC-restrict who can create Secrets outside GitOps.

Scenario 3 — Rolling out to 50 clusters - **Problem**: Copy-pasting Applications for 50 clusters is unmanageable. - **Resolution**: 1. ArgoCD **ApplicationSet** with `List`, `Git`, or `Cluster` generator. 2. Flux with a matrix of `Kustomizations` templated via `postBuild.substituteFrom`. 3. Fleet management: Argo `ClusterGenerator` + labels per cluster.

☁ GCP GKE Perspective

- **Anthos Config Management / Config Sync** is Google's first-party GitOps agent (uses Kustomize + Helm).
- **Backup for GKE** integrates with GitOps for full disaster recovery.
- Use **Workload Identity** so ArgoCD/Flux calls GCP APIs (e.g., pull from Artifact Registry) without keys.
- Command:

```
gcloud beta container fleet config-management apply \
  --membership=<cluster> --config=config-management.yaml
```

▩ AWS EKS Perspective

- Both ArgoCD and Flux are popular on EKS; no first-party GitOps product from AWS.
- Use **IRSA** for the ArgoCD repo-server or Flux source-controller to pull from CodeCommit / ECR privately.
- Common stack: ArgoCD + Sealed Secrets + AWS Load Balancer Controller for the ArgoCD UI.

- Command:

```
helm install argocd argo/argo-cd -n argocd --create-namespace \
  --set server.serviceAccount.annotations."eks\.amazonaws\.com/role-arn"=arn:aws:iam::...:role/argocd
```

Hands-On Lab Exercises

Lab 61-A — Install ArgoCD and Deploy Your First App (Beginner) - **Objective**: End-to-end GitOps on a Kind cluster. - **Prerequisites**: Kind, kubectl, GitHub account. - **Steps**: 1. `kind create cluster --name gitops` 2. `kubectl create ns argocd` 3. `kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml` 4. `kubectl -n argocd port-forward svc/argocd-server 8080:443` 5. Fetch initial password: `kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d` 6. Fork https://github.com/argoproj/argocd-example-apps 7. Create Application pointing to `guestbook/` . 8. Push a change and watch auto-sync. - **Verification**: UI shows `Synced` + `Healthy` ; guestbook pods running. - **Cleanup**: `kind delete cluster --name gitops` - **Bonus**: Turn on `selfHeal` and manually edit a Deployment — watch it revert.

Lab 61-B — Flux Bootstrap on EKS (Intermediate) - **Objective**: Bootstrap Flux with `flux bootstrap github` . - **Prerequisites**: EKS cluster, GitHub PAT with `repo` scope, `flux` CLI. - **Steps**: 1. Export `GITHUB_TOKEN=<pat>` and `GITHUB_USER=<user>` . 2. `flux bootstrap github --owner=$GITHUB_USER --repository=fleet --branch=main --path=clusters/prod --personal` 3. Add a `GitRepository` + `Kustomization` under `clusters/prod/` . 4. Commit; Flux reconciles. - **Verification**: `flux get kustomizations` shows Ready. - **Cleanup**: `flux uninstall --namespace=flux-system` - **Bonus**: Add a `HelmRelease` for ingress-nginx.

Lab 61-C — Progressive Delivery with Argo Rollouts (Advanced) - **Objective**: Canary a Deployment via Argo Rollouts + ArgoCD. - **Prerequisites**: ArgoCD, Argo Rollouts controller, Prometheus. - **Steps**: 1. Install Argo Rollouts: `kubectl apply -k github.com/argoproj/argo-rollouts/manifests/crds` 2. Convert Deployment to `Rollout` with canary steps: 20% → analysis → 40% → 100%. 3. Configure `AnalysisTemplate` querying Prometheus success rate. 4. Push image change; watch progressive rollout in Rollouts UI. - **Verification**: On success → full promotion; on failure → auto-abort. - **Bonus**: Add Slack notifications for rollout events.

Interview Deliverable Answer (60–90s)

> "GitOps makes Git the single source of truth for what's in the cluster, and a controller — ArgoCD or Flux — continuously pulls that state and reconciles it. **What**: declarative + versioned + pulled + continuously reconciled. **Why**: every change is a Git commit, so we get audit history, easy rollback via revert, and no cluster credentials in CI runners. **How**: developer PRs a manifest change, CI updates the image tag in the config repo, ArgoCD sees the change and applies it; drift is auto-healed. On EKS I run ArgoCD with an IRSA-bound ServiceAccount so it can pull from ECR; on GKE, Config Sync does the same. In one program we scaled to 30 clusters using an ArgoCD `ApplicationSet` — one YAML defined the app for every cluster and every env, so onboarding a new region was a two-line commit."

Interview Questions

Basic 1. **Q**: What is GitOps? **A**: An operating model where Git is the single source of truth and an in-cluster controller continuously reconciles state.

1. **Q**: Push vs pull CD — which is GitOps? **A**: Pull — the cluster pulls from Git; CI never touches the cluster directly.

Intermediate 3. **Q**: ArgoCD `Application` vs `ApplicationSet` ? **A**: `Application` is one app on one cluster; `ApplicationSet` templates many Applications from generators (list, cluster, git).

1. **Q**: What does `selfHeal: true` do? **A**: ArgoCD reverts any manual changes so cluster state always matches Git.
2. **Q**: Difference between ArgoCD and Flux? **A**: ArgoCD has a rich UI and app-centric model; Flux is CRD-driven with a lighter footprint. Both are CNCF Graduated.

Advanced 6. **Q**: How do you manage secrets in GitOps? **A**: Sealed Secrets, External Secrets Operator, or SOPS — never store plaintext in Git.

1. **Q**: What is progressive delivery? **A**: Gradual rollout (canary/blue-green) with automated analysis (Argo Rollouts / Flagger) that promotes or aborts based on metrics.

Scenario 8. **Q**: Someone bypassed GitOps with `kubectl edit` . How to prevent it? **A**: Enable `selfHeal` , restrict namespace RBAC so only the GitOps SA can write, add admission policies (Kyverno/OPA) to reject non-GitOps changes.

GKE-specific 9. **Q**: What is Config Sync? **A**: GCP's Anthos GitOps agent that syncs Kustomize/Helm from Git to fleets of GKE clusters.

EKS-specific 10. **Q**: How does ArgoCD authenticate to ECR on EKS? **A**: IRSA on the argocd-repo-server ServiceAccount + `AmazonEC2ContainerRegistryReadOnly` IAM policy.

Lab-based 11. **Q**: Walk me through installing ArgoCD and connecting a Git repo. **A**: (Lab 61-A) Install manifest, port-forward UI, fetch initial admin password, create Application pointing at repo/path, watch sync.

⏸ Pause Point — End of Batch 1 of 4

Completed in this batch (Topics 57–61): - ✅ Custom Resource Definitions (CRDs) - ✅ Operators and Operator Pattern - ✅ Helm - ✅ Kustomize - ✅ GitOps (ArgoCD, Flux)

Remaining across next batches: - **Batch 2 (Topics 62–66):** Admission Controllers, API Aggregation Layer, etcd deep dive, Multi-Cluster Management, Service Catalog & APF - **Batch 3 (Topics 67–70):** GKE Standard vs Autopilot, EKS node models, WLI vs IRSA, Version lifecycle - **Batch 4 (Topics 71–73):** Top 50 troubleshooting, Top 30 system design, Behavioral+K8s hybrid - **Batch 5 (Capstones 1–10):** End-to-end lab projects - **Final:** PDF compilation

Say **"continue"** to proceed with Batch 2 (Topics 62–66).

Module 10 — Advanced Topics (Part 2)

Topics 62 – 66

📄 Topic 62 — Admission Controllers (Mutating & Validating)

Official Documentation References - https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/ - https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/ - https://kubernetes.io/docs/reference/access-authn-authz/validating-admission-policy/ - https://open-policy-agent.github.io/gatekeeper/ - https://kyverno.io/docs/

📄 Definition

An admission controller is a piece of code in (or plugged into) the Kubernetes API server that intercepts requests **after** authentication and authorization but **before** the object is persisted to etcd, and can mutate or reject the request.

📖 About

Admission is the last line of defense (and shaping) for API objects. It runs *inside* the API request path, so it can enforce policy in real time — no reconciliation lag.

Two categories 1. **Mutating** admission controllers — modify the object (e.g., inject a sidecar, set default fields). 2. **Validating** admission controllers — accept or reject the request (e.g., "no `latest` tags allowed").

Built-in admission plugins (compiled into kube-apiserver) — a few key ones: - `NamespaceLifecycle` — blocks writes to terminating namespaces. - `LimitRanger` — applies default limits. - `ServiceAccount` — auto-mounts SA token. - `ResourceQuota` — enforces quotas. - `PodSecurity` — enforces Pod Security Standards. - `DefaultStorageClass` — sets default SC on PVCs. - `MutatingAdmissionWebhook` , `ValidatingAdmissionWebhook` — call external webhooks. - `ValidatingAdmissionPolicy` (v1.30 GA) — CEL-based validation without a webhook.

Ordering 1. Mutating phase (built-in mutators → mutating webhooks). 2. Object schema validation. 3. Validating phase (built-in validators → validating webhooks → validating admission policies). 4. Persist to etcd.

Common external solutions - **OPA/Gatekeeper** — policy-as-code via Rego, ConstraintTemplate + Constraint CRDs. - **Kyverno** — Kubernetes-native policies as YAML. - **Cert-manager, Istio, Linkerd** — install webhooks for injection. - **kube-mutating-webhook-tutorial**, **jsonpatch** libraries — DIY.

Why admission controllers exist - Enforce security invariants (no root containers, no `hostNetwork`). - Guarantee organizational conventions (labels, annotations, image registry allowlists). - Automate boilerplate (sidecar injection, resource defaults).

🔗 How It Integrates With Other Components

Component	Integration
kube-apiserver	Hosts the admission chain; loaded via `--enable-admission-plugins`
kubectl	User's request hits admission on every apply/create/update
ValidatingWebhookConfiguration / MutatingWebhookConfiguration	CRDs registering external webhook endpoints
Webhook server	TLS-served HTTPS endpoint responding with `AdmissionReview`
RBAC	Webhook needs its own SA + Service
CRDs	Custom policies (Gatekeeper `Constraint` , Kyverno `ClusterPolicy`) are CRDs
CI/CD	Policies enforce e.g. image scanning gates
Cert-manager	Auto-rotates webhook TLS certs

🏗 Architecture Diagrams

Diagram 1 — Admission phases



Diagram 2 — Webhook request/response

```
kube-apiserver                Webhook Server (Pod)
     |                                |
     |--- AdmissionReview (JSON) ------>|
     |     .request.uid               |
     |     .request.object            |
     |                                |-- policy eval / patch generation
     |<-- AdmissionReview (JSON) -------|
     |     .response.uid              |
     |     .response.allowed=true/false |
     |     .response.patch (base64)     |
     |     .response.warnings[]         |
     |                                |
```

💻 Real-World Examples

Example 1 — MutatingWebhookConfiguration injecting a sidecar

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: MutatingWebhookConfiguration
metadata:
  name: sidecar-injector.example.com
webhooks:
  - name: sidecar.example.com
    admissionReviewVersions: ["v1"]
    sideEffects: None
    timeoutSeconds: 5
    failurePolicy: Ignore   # Fail-open so cluster works if webhook is down
    reinvocationPolicy: IfNeeded
    clientConfig:
      service:
        name: sidecar-injector
        namespace: platform
        path: /mutate
        port: 443
      caBundle: <base64-ca>
    rules:
      - apiGroups: [""]
        apiVersions: ["v1"]
        resources: ["pods"]
        operations: ["CREATE"]
    namespaceSelector:
      matchLabels:
        sidecar-injection: enabled
    objectSelector:
      matchExpressions:
        - key: sidecar.example.com/inject
          operator: NotIn
          values: ["false"]
```

Example 2 — ValidatingAdmissionPolicy (CEL, no webhook needed)

```yaml
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata:
  name: "disallow-latest-tag"
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
      - apiGroups: ["apps"]
        apiVersions: ["v1"]
        operations: ["CREATE","UPDATE"]
        resources: ["deployments","statefulsets","daemonsets"]
  validations:
    - expression: >-
        object.spec.template.spec.containers.all(c,
          !c.image.endsWith(':latest') && c.image.contains(':'))
      message: "Container images must have an explicit non-'latest' tag."
      reason: Invalid
---
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicyBinding
metadata:
  name: "disallow-latest-tag-binding"
spec:
  policyName: "disallow-latest-tag"
  validationActions: [Deny, Audit, Warn]
  matchResources:
    namespaceSelector:
      matchLabels:
        environment: production
```

Example 3 — Kyverno ClusterPolicy (require resource limits)

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-resource-limits
spec:
  validationFailureAction: Enforce
  background: true
  rules:
    - name: check-limits
      match:
        any:
          - resources:
              kinds: [Pod]
      validate:
        message: "CPU and memory limits are required."
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    cpu: "?*"
                    memory: "?*"
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get validatingwebhookconfigurations`	–	List validating webhooks	`kubectl get validatingwebhookconfigurations`
`kubectl get mutatingwebhookconfigurations`	–	List mutating webhooks	`kubectl get mutatingwebhookconfigurations`
`kubectl get validatingadmissionpolicies`	–	List CEL policies	`kubectl get vap`
`kubectl describe vwc <name>`	–	Inspect config	`kubectl describe vwc sidecar-injector`
`kubectl auth can-i`	–	Check RBAC before admission	`kubectl auth can-i create pods`
`kubectl get --raw`	–	Debug apiserver response	`kubectl get --raw /apis/admissionregistration.k8s.io/v1/validatingwebhookconfigurations`
`kubectl -n <ns> logs <webhook-pod>`	`-f`	Tail webhook logs	`kubectl logs -f -n platform deploy/sidecar-injector`
`kubectl apply --dry-run=server`	–	Test admission without persistence	`kubectl apply -f pod.yaml --dry-run=server`
`kubectl explain validatingadmissionpolicy.spec`	–	Learn schema	`kubectl explain vap.spec.validations`
`kyverno test`	–	Test Kyverno policies locally	`kyverno test .`
`gator test`	–	Test Gatekeeper constraints	`gator test --filename=./policies`

🕸 Scenarios

Scenario 1 — Webhook down brings cluster to its knees - **Problem**: `failurePolicy: Fail` webhook Pod is unhealthy; every Pod creation is rejected → cluster unable to schedule anything. - **Resolution**: 1. For non-critical webhooks, set `failurePolicy: Ignore`. 2. Exclude `kube-system` and the webhook's own namespace via `namespaceSelector`. 3. Run at least 2 replicas of the webhook with a PDB. 4. Set aggressive `timeoutSeconds` (≤5s). 5. Add `matchConditions` (v1.30+) to reduce match volume.

Scenario 2 — Webhook certificate expired - **Problem**: `caBundle` no longer valid; kube-apiserver errors `x509: certificate has expired`. - **Resolution**: 1. Use **cert-manager** with `Certificate` + `CAInjector` annotation on the WebhookConfiguration. 2. Or manually rotate + `kubectl patch vwc <name> -p '{"webhooks":[{"name":"...","clientConfig":{"caBundle":"..."}}]}'`. 3. Monitor cert expiration with Prometheus `x509-certificate-exporter`.

Scenario 3 — Sidecar injected into system pods causing crash - **Problem**: Mutating webhook injected an Envoy sidecar into a DaemonSet in `kube-system`, breaking node bootstrap. - **Resolution**: 1. Add `namespaceSelector.matchExpressions` excluding `kube-system` and control-plane namespaces. 2. Use an opt-in label (e.g., `sidecar-injection: enabled`) rather than opt-out. 3. Test admission policies with `--dry-run=server` in CI.

☁ GCP GKE Perspective

- GKE ships with the standard admission plugins enabled.
- **Policy Controller** (Anthos add-on) is a managed distribution of OPA Gatekeeper.
- **Autopilot** enforces built-in security policies via admission — you cannot disable them. Attempts to run privileged containers, `hostNetwork: true`, or `hostPath` are rejected at admission.
- GKE binary authorization uses an admission webhook to verify image signatures.
- Command: `gcloud container fleet policycontroller enable --memberships=<cluster>`.

▩ AWS EKS Perspective

- Standard plugins enabled; you can extend `--enable-admission-plugins` only through EKS-supported flags.
- Common installs: **Gatekeeper** or **Kyverno** via Helm.
- Fargate Pods still pass through admission — sidecar injection works, but you must whitelist your Fargate profile namespace.
- IRSA can be used to give a webhook AWS API access (e.g., look up signed image metadata in Signer).
- Command: `aws eks describe-cluster --name <c> --query 'cluster.identity.oidc.issuer'` to find OIDC issuer for IRSA on the webhook SA.

🧪 Hands-On Lab Exercises

🧪 Lab 62-A — First Validating Admission Policy (Beginner) - **Objective**: Reject Pods without a `team` label using ValidatingAdmissionPolicy. - **Prerequisites**: Cluster v1.30+. - **Steps**: 1. Save policy:

```
apiVersion: admissionregistration.k8s.io/v1
kind: ValidatingAdmissionPolicy
metadata: { name: require-team-label }
spec:
  failurePolicy: Fail
  matchConstraints:
    resourceRules:
    - apiGroups: [""]
      apiVersions: ["v1"]
      operations: ["CREATE","UPDATE"]
      resources: ["pods"]
  validations:
    - expression: "has(object.metadata.labels) && has(object.metadata.labels.team)"
      message: "Pods must have a 'team' label."
```

2. Apply it + a binding scoped to `default` namespace. 3. `kubectl run bad --image=nginx` → rejected. 4. `kubectl run good --image=nginx --labels=team=platform` → accepted. - **Verification**: rejection message printed. - **Cleanup**: `kubectl delete vap require-team-label vapbinding require-team-label-binding`. - **Bonus**: Add `validationActions: [Warn]` for a soft-launch phase.

🧪 Lab 62-B — Kyverno Enforcing Image Registry Allowlist (Intermediate) - **Objective**: Only allow images from `gcr.io/my-project/*` or `<acct>.dkr.ecr.us-east-1.amazonaws.com/*`. - **Prerequisites**: Kyverno installed (`helm install kyverno kyverno/kyverno -n kyverno --create-namespace`). - **Steps**: 1. Apply a `ClusterPolicy` with `validate.deny` on non-allowlisted registries. 2. Attempt `kubectl run hacker --image=docker.io/evil:latest` → denied. 3. Attempt with an allowlisted image → allowed. - **Verification**: `kubectl get events` shows policy denial reason. - **Bonus**: Add `mutate` rule that auto-adds an `imagePullSecrets`

entry.

Lab 62-C — Sidecar Injection Webhook from Scratch (Advanced) - **Objective**: Build a mutating webhook in Go that injects a `busybox` sidecar into pods labeled `inject=true` . - **Prerequisites**: Go, cert-manager, kind, docker registry. - **Steps**: 1. Scaffold Go HTTP server serving `/mutate` returning `AdmissionReview` with a JSON patch. 2. Build & push image; deploy as Deployment + Service. 3. Create `Certificate` (cert-manager) + `MutatingWebhookConfiguration` with `cert-manager.io/inject-ca-from` annotation. 4. Create Pod with `labels: { inject: "true" }` . - **Verification**: Pod comes up with 2 containers. - **Bonus**: Add unit tests for patch generation + e2e test with envtest.

Interview Deliverable Answer (60–90s)

> "Admission controllers are the API server's gate between authorization and etcd. They come in two flavors: mutating (change the request — like injecting a sidecar or default resource limits) and validating (accept or reject — like blocking `:latest` tags). Built-in plugins ship in the apiserver; external ones plug in via `MutatingWebhookConfiguration` and `ValidatingWebhookConfiguration` , and since v1.30 we also have `ValidatingAdmissionPolicy` — pure CEL, no webhook. Why: enforce security and org standards in real time, not after the fact. How I've used it: on EKS with Kyverno we blocked untrusted registries and required team labels; on GKE Autopilot the platform enforces PSS via admission, so we shifted our custom checks to CEL-based policies. Key gotcha: `failurePolicy: Fail` on a webhook that goes down can wedge the whole cluster — always exclude `kube-system` and run multiple replicas with a PDB."

? Interview Questions

Basic 1. **Q**: What is an admission controller? **A**: Code in the apiserver that intercepts authorized requests before persistence, and can mutate or reject them.

1. **Q**: Mutating vs Validating? **A**: Mutating modifies the object; validating only accepts or rejects. Mutating runs first.

Intermediate 3. **Q**: What does `failurePolicy: Fail` mean? **A**: If the webhook is unreachable, admission is denied (fail-closed). `Ignore` is fail-open.

1. **Q**: When does the mutating chain re-run? **A**: If `reinvocationPolicy: IfNeeded` , later webhooks may cause earlier ones to be re-invoked once.
2. **Q**: How is `ValidatingAdmissionPolicy` different from a validating webhook? **A**: It's in-process CEL evaluation — no external server, no TLS, no network hop; introduced Beta in v1.28, GA v1.30.

Advanced 6. **Q**: What order do admission plugins run in? **A**: Mutating → schema validation → Validating; within each phase, built-ins first, then webhooks in an undefined but stable order.

1. **Q**: How do you keep webhook certs fresh? **A**: cert-manager issues + rotates a Certificate; `caBundle` on the webhook config is auto-injected via `cert-manager.io/inject-ca-from` .

Scenario 8. **Q**: All Pod creates in production are failing after installing a webhook — how do you unblock the cluster? **A**: Delete the webhook configuration (`kubectl delete vwc <name>`), fix, re-apply. Add `kube-system` exclusion.

GKE-specific 9. **Q**: How does Autopilot use admission? **A**: Google-managed policies reject privileged/hostPath/hostNetwork; you can't bypass them.

EKS-specific 10. **Q**: How to run a webhook that reads secrets from AWS Secrets Manager? **A**: IRSA-bind its SA + `secretsmanager:GetSecretValue` ; call SDK from the webhook process.

Lab-based 11. **Q**: Show me writing a policy that blocks `latest` tags. **A**: (Lab 62-A) VAP with CEL expression checking `!c.image.endsWith(':latest')` .

Topic 63 — API Aggregation Layer

Official Documentation References - https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/apiserver-aggregation/ - https://kubernetes.io/docs/tasks/extend-kubernetes/configure-aggregation-layer/ - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#apiservice-v1-apiregistration-k8s-io

Definition

The API Aggregation Layer lets you extend Kubernetes with additional, first-class APIs served by *your own* extension apiserver, exposed under the same `/apis/<group>/<version>` URL space as core APIs.

About

CRDs give you extension APIs backed by etcd; the **aggregation layer** gives you extension APIs backed by *any* custom logic — including in-memory data, external systems, or specialized storage.

CRD vs Aggregated APIServer

Aspect	CRD	Aggregated APIServer
Storage	etcd (managed)	Your choice — etcd, memory, external DB
Schema	OpenAPI + CEL	Full Go types
Discovery	Auto via apiserver	Registered via APIService
Watch/List	Free	You implement
Best for	Declarative CRs	Live/computed data, high-scale, protocol translation

How it works 1. The main kube-apiserver runs the `kube-aggregator` component. 2. You create an `APIService` object registering `v1beta1.metrics.k8s.io` (say) with a `Service` that points at your extension server. 3. Requests to `/apis/metrics.k8s.io/v1beta1/*` are transparently proxied to your Pod. 4. Your Pod runs a Go server built on the `k8s.io/apiserver` library, or any HTTP server implementing the discovery, list/watch, and CRUD verbs.

Canonical example: `metrics-server` — computes `PodMetrics` / `NodeMetrics` on demand rather than storing them.

When to use aggregation - Serving data too volatile for etcd (metrics, live event streams). - Wrapping an external system as Kubernetes API. - Very high QPS / throughput. - Needing custom auth flows or protocol adapters.

How It Integrates With Other Components

Component	Integration
kube-apiserver	Runs `kube-aggregator`; proxies matching paths
APIService (CRD-like)	Registers the extension
Service + Pod	Your extension apiserver runs here
Auth	Delegated: main apiserver forwards user identity via headers (`--requestheader-*` flags)
TLS	The extension server presents a cert; main apiserver validates it
Discovery	Your extension answers `/apis/<group>/<version>` for kubectl to auto-discover
kubectl	Same UX as core APIs

🏗 Architecture Diagrams

Diagram 1 — Request routing

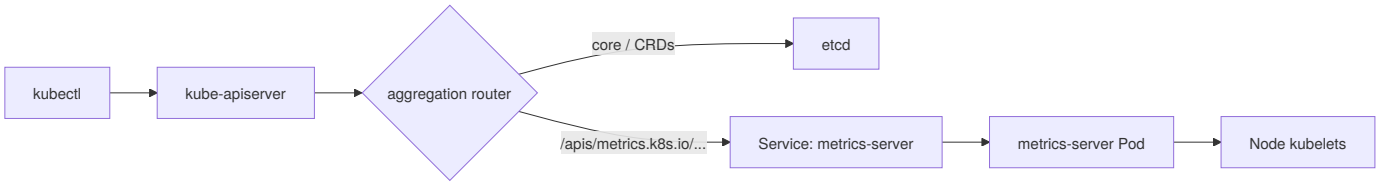
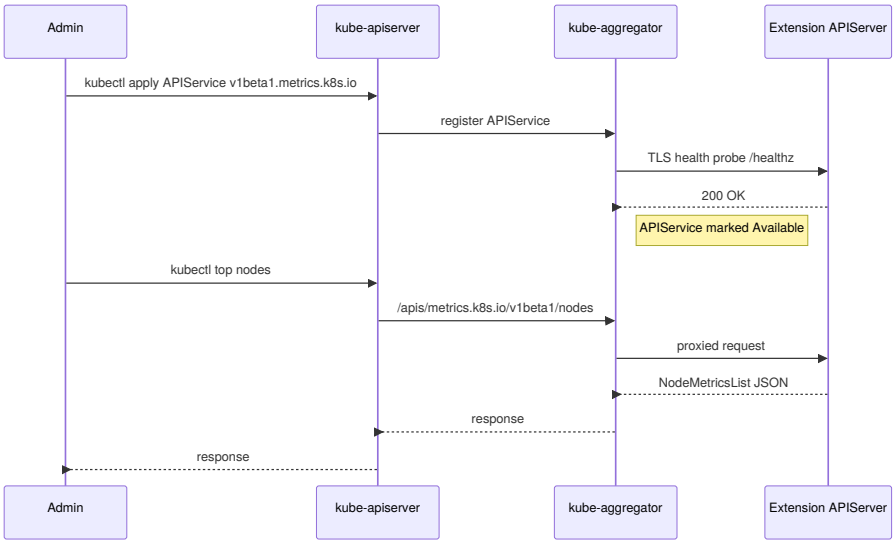


Diagram 2 — APIService registration



💻 Real-World Examples

Example 1 — APIService for a custom extension apiserver

```
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  name: v1alpha1.wardle.example.com
spec:
  group: wardle.example.com
  version: v1alpha1
  groupPriorityMinimum: 1000
  versionPriority: 15
  service:
    name: wardle-server
    namespace: wardle
    port: 443
  caBundle: <base64-CA>
  insecureSkipTLSVerify: false
```

Supporting Service:

```
apiVersion: v1
kind: Service
metadata:
  name: wardle-server
  namespace: wardle
spec:
  selector: { app: wardle-apiserver }
  ports:
    - name: https
      port: 443
      targetPort: 8443
```

Example 2 — metrics-server APIService (from upstream)

```
apiVersion: apiregistration.k8s.io/v1
kind: APIService
metadata:
  name: v1beta1.metrics.k8s.io
  labels:
    k8s-app: metrics-server
spec:
  service:
    name: metrics-server
    namespace: kube-system
  group: metrics.k8s.io
  version: v1beta1
  insecureSkipTLSVerify: true
  groupPriorityMinimum: 100
  versionPriority: 100
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get apiservices`	–	List all APIServices	`kubectl get apiservices`
`kubectl get apiservices`	`-o wide`	Show Available column	`kubectl get apiservices -o wide`
`kubectl describe apiservice`	–	See status & conditions	`kubectl describe apiservice v1beta1.metrics.k8s.io`
`kubectl get --raw`	–	Hit aggregated endpoint directly	`kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes`
`kubectl top nodes/pods`	–	Uses metrics.k8s.io	`kubectl top pods`
`kubectl api-resources`	`--api-group`	Filter to a group	`kubectl api-resources --api-group=metrics.k8s.io`
`kubectl logs -n kube-system deploy/metrics-server`	`-f`	Debug extension apiserver	`kubectl logs -n kube-system deploy/metrics-server -f`

📽 Scenarios

Scenario 1 — `kubectl top nodes` returns "error from server (ServiceUnavailable)" - **Problem**: metrics-server Pod is CrashLooping or its Service selector is wrong. - **Resolution**: 1. `kubectl get apiservice v1beta1.metrics.k8s.io -o yaml` → check `.status.conditions`. 2. Check Pod logs; common causes: kubelet TLS errors (`--kubelet-insecure-tls`), CNI blocking apiserver → metrics-server. 3. Verify `--requestheader-*` flags on apiserver.

Scenario 2 — Aggregated APIService blocks entire discovery - **Problem**: When an APIService is `Unavailable`, `kubectl api-resources` returns errors and Helm/ArgoCD may fail. - **Resolution**: 1. Delete the broken APIService: `kubectl delete apiservice <name>`. 2. Ensure health checks are quick; extension server must serve `/healthz`. 3. Set `groupPriorityMinimum` and `versionPriority` correctly — misordering causes routing issues.

Scenario 3 — TLS handshake failing between apiserver and extension - **Problem**: `caBundle` in APIService doesn't match cert served by extension. - **Resolution**: 1. Use `kubernetes.io/kube-apiserver-client` signer or cert-manager. 2. Automate CA injection with `caBundle` from a Secret via a small controller. 3. `openssl s_client -connect wardle-server.wardle:443 -showcerts` to inspect.

☁ GCP GKE Perspective

- `metrics-server` and `metrics.k8s.io` are managed by GKE — you don't install them.
- Custom aggregated APIs work; use a **NEG-backed Service** for the extension server if you need external access.
- On Autopilot, aggregated apiservers must satisfy Autopilot pod constraints (no privileged).
- **Managed Service for Prometheus** on GKE registers `external.metrics.k8s.io` and `custom.metrics.k8s.io` via aggregation to feed HPA.

▩ AWS EKS Perspective

- `metrics-server` **is not** installed by default — you install via Helm or the EKS add-on.
- On Fargate, the aggregation layer works, but the extension Pod must be on a Fargate profile.
- Use IRSA to give the extension apiserver AWS API access.
- Common aggregated APIs on EKS: `metrics-server`, `external-metrics-adapter` (Prometheus adapter for HPA).

🧪 Hands-On Lab Exercises

🧪 Lab 63-A — Install metrics-server and Explore Aggregation (Beginner) - **Objective**: See `metrics.k8s.io` in action. - **Prerequisites**: Kind or Minikube. - **Steps**: 1. `kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml` 2. For Kind, patch to add `--kubelet-insecure-tls`. 3. `kubectl get apiservice v1beta1.metrics.k8s.io -o wide` → wait for `Available: True`. 4. `kubectl top nodes` and `kubectl top pods -A`. 5. `kubectl get --raw /apis/metrics.k8s.io/v1beta1/nodes | jq`. - **Verification**: Nodes + Pods with CPU/mem values. - **Cleanup**: `kubectl delete -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml` - **Bonus**: Inspect logs and identify the `kubelet_summary_scrapes_total` metric.

🧪 Lab 63-B — Sample "Wardle" Aggregated APIServer (Intermediate) - **Objective**: Deploy the upstream sample-apiserver from https://github.com/kubernetes-sigs/apiserver-builder-alpha. - **Steps**: 1. Clone repo, build image. 2. Deploy Namespace, ServiceAccount, RBAC, Deployment, Service, APIService. 3. `kubectl api-resources --api-group=wardle.example.com`. 4. `kubectl create -f - <<EOF ... kind: Flunder ...` and list them. - **Verification**: Custom kind visible. - **Cleanup**: Delete namespace + APIService. - **Bonus**: Add TLS via cert-manager.

🧪 Lab 63-C — Prometheus Adapter for HPA (Advanced) - **Objective**: Serve `custom.metrics.k8s.io` from Prometheus to drive HPA on a custom metric (`http_requests_per_second`). - **Prerequisites**: Prometheus + prometheus-adapter Helm chart. - **Steps**: 1. Install kube-prometheus-stack. 2. Install `prometheus-adapter` with rules mapping HTTP metric. 3. Deploy sample app + Service. 4. `kubectl get --raw "/apis/custom.metrics.k8s.io/v1beta1/namespaces/default/pods/*/http_requests_per_second"`. 5. Create HPA `.spec.metrics.type: Pods`. 6. Load-test with `hey`. - **Verification**: HPA scales on custom metric. - **Bonus**: Scale on an external metric (SQS queue length on EKS or Pub/Sub on GKE).

🎯 Interview Deliverable Answer (60–90s)

> "The API Aggregation Layer lets me extend Kubernetes with new APIs that aren't backed by etcd. Where CRDs give me declarative objects stored in etcd, an

> aggregated apiserver serves any URL under `/apis/<group>/<version>` from a Pod I control. Why: some data — live pod metrics, external system views, high-volume streams — doesn't belong in etcd. The classic example is `metrics-server` : it registers `metrics.k8s.io` , and `kubectl top nodes` transparently proxies through kube-aggregator. On EKS you install it as an add-on; on GKE it's managed. To wire it in you create an `APIService` object pointing at a Service; the apiserver validates TLS with your CA bundle and forwards requests. In an HPA project I registered `custom.metrics.k8s.io` via prometheus-adapter to scale on requests-per-second — that only works because of aggregation."

? Interview Questions

Basic 1. **Q**: What's the difference between CRD and aggregation? **A**: CRDs are stored in etcd; aggregated APIs are served by your Pod and can back onto anything.

1. **Q**: Name a well-known aggregated API. **A**: `metrics.k8s.io` served by metrics-server.

Intermediate 3. **Q**: How does the apiserver route a request to an aggregated API? **A**: The kube-aggregator matches path prefix against registered APIServices and proxies over HTTPS to the target Service.

1. **Q**: What does `groupPriorityMinimum` do? **A**: Controls ordering of API groups in discovery and resolution when clients pick a version.

Advanced 5. **Q**: How is user identity passed to the extension apiserver? **A**: Via `--requestheader-*` flags: apiserver signs headers (`X-Remote-User` , `X-Remote-Group`) using a client cert the extension trusts.

1. **Q**: Trade-offs of aggregation vs CRD? **A**: Aggregation: more flexible, harder to build/operate. CRD: free storage, watch, RBAC; less flexible.

Scenario 7. **Q**: `kubectl api-resources` is slow — diagnose. **A**: Likely a broken APIService returning slowly; `kubectl get apiservices | grep False` to find it.

GKE-specific 8. **Q**: How is `metrics-server` handled on GKE? **A**: Managed by Google; installed and upgraded automatically.

EKS-specific 9. **Q**: How do you install metrics-server on EKS? **A**: `kubectl apply` the upstream manifest or use the EKS add-on / Helm chart.

Lab-based 10. **Q**: Show me making HPA scale on a custom metric. **A**: (Lab 63-C) install prometheus-adapter → verify `custom.metrics.k8s.io` → HPA with `type: Pods` .

🗄 Topic 64 — etcd Deep Dive (Backup, Restore, Disaster Recovery)

Official Documentation References - https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/ - https://etcd.io/docs/ - https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/setup-ha-etcd-with-kubeadm/ - https://kubernetes.io/docs/tasks/administer-cluster/backup-cluster/

🗄 Definition

etcd is the distributed, strongly-consistent key-value store used by Kubernetes as its sole source of truth — every object you `kubectl apply` lives here.

🕮 About

etcd is a Raft-based CP (consistent + partition-tolerant) datastore. If etcd loses quorum or its data is corrupted, the cluster's API becomes unavailable and no scheduling decisions can be made or preserved.

Key properties - **Raft consensus**: writes require majority ([N/2]+1) of members to ack. - **Cluster sizes**: 3, 5, 7 members. 3 tolerates 1 failure; 5 tolerates 2. Even sizes give no extra fault tolerance. - **Watch**: streaming API used heavily by kube-apiserver and controllers. - **MVCC**: versioned key-value; historical revisions kept until compaction. - **Compaction**: removes old revisions to reclaim space. - **Defragmentation**: reclaims disk fragmentation, must be run periodically (per member).

Data organization in Kubernetes etcd - All Kubernetes objects stored under `/registry/<resource>/<namespace>/<name>` . - Encoded in Protobuf by default. - Encryption-at-rest via `EncryptionConfiguration` on kube-apiserver (`aescbc` , `kms`).

Failure modes - Loss of quorum → API server writes fail. - Disk full → etcd goes read-only (alarm NOSPACE). - Slow disk → apply latency spikes (P99 > 100ms is bad). - Corruption → member refuses to start.

Best-practice hardware - SSD/NVMe with < 10ms fsync latency. - Dedicated disk (not shared with OS/logs). - 8+ GB RAM. - 4+ vCPU. - 1 Gbps+ network, low latency between members.

Backup strategies - **Snapshots** via `etcdctl snapshot save` . - Frequency: every 15–30 min for prod; retain 24 hours + daily for 30 days. - Store off-cluster (S3/GCS with immutability). - Test restores routinely.

🔗 How It Integrates With Other Components

Component	Integration
kube-apiserver	Sole client (all reads/writes go through it)
controllers / scheduler	Talk to apiserver, not etcd directly
kubelet	No direct etcd access
Cloud providers	Usually run etcd inside the control plane
Encryption	apiserver encrypts data before storing
Backup for GKE / Velero	Higher-level tools that snapshot workloads (not etcd directly on managed)

🏗 Architecture Diagrams

Diagram 1 — Raft quorum



Diagram 2 — Backup + restore flow

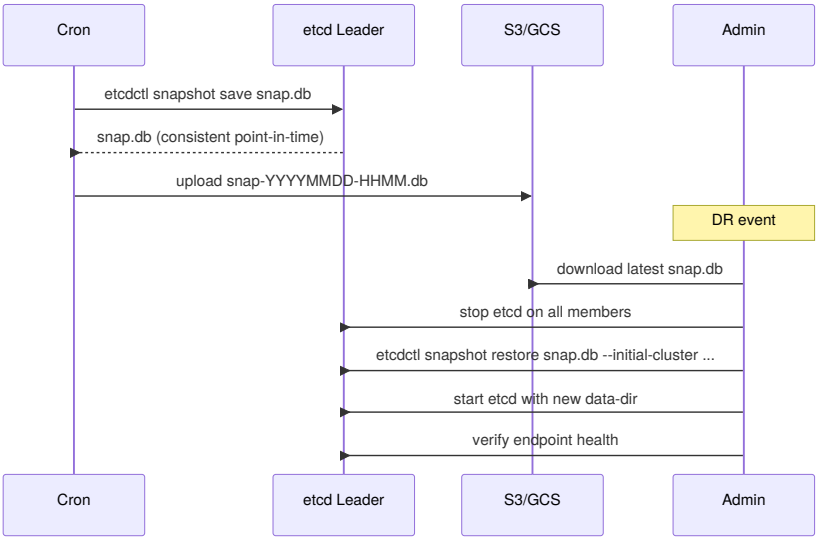


Diagram 3 — kubeadm 3-node etcd stacked topology

```
+--------------------+   +--------------------+   +--------------------+
|  control-plane-1   |   |  control-plane-2   |   |  control-plane-3   |
| apiserver          |   | apiserver          |   | apiserver          |
| scheduler          |   | scheduler          |   | scheduler          |
| controller-mgr     |   | controller-mgr     |   | controller-mgr     |
| etcd-1  <========> |   | etcd-2  <========> |   | etcd-3             |
+--------------------+   +--------------------+   +--------------------+
            |_____________ HAProxy / LB _______________|
                                |
                            kubelets
```

💻 Real-World Examples

Example 1 — Encryption at rest configuration

`/etc/kubernetes/encryption-config.yaml` :

```
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources: ["secrets", "configmaps"]
    providers:
      - aescbc:
          keys:
            - name: key1
              secret: <base64-32-bytes>
      - identity: {}
```

Add flag to kube-apiserver:

```
--encryption-provider-config=/etc/kubernetes/encryption-config.yaml
```

Example 2 — Cron-driven snapshot to S3

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: etcd-backup
  namespace: kube-system
spec:
  schedule: "*/30 * * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          hostNetwork: true
          nodeSelector:
            node-role.kubernetes.io/control-plane: ""
          tolerations:
            - key: node-role.kubernetes.io/control-plane
              operator: Exists
              effect: NoSchedule
          restartPolicy: OnFailure
          containers:
            - name: backup
              image: registry.k8s.io/etcd:3.5.15-0
              command:
                - /bin/sh
                - -c
                - >
                  TS=$(date +%Y%m%d-%H%M);
                  ETCDCTL_API=3 etcdctl
                  --endpoints=https://127.0.0.1:2379
                  --cacert=/etc/kubernetes/pki/etcd/ca.crt
                  --cert=/etc/kubernetes/pki/etcd/server.crt
                  --key=/etc/kubernetes/pki/etcd/server.key
                  snapshot save /backup/snap-$TS.db &&
                  aws s3 cp /backup/snap-$TS.db s3://etcd-backups/$TS.db
              volumeMounts:
                - name: etcd-pki
                  mountPath: /etc/kubernetes/pki/etcd
                  readOnly: true
                - name: backup
                  mountPath: /backup
          volumes:
            - name: etcd-pki
              hostPath: { path: /etc/kubernetes/pki/etcd }
            - name: backup
              emptyDir: {}
```

💻 Related Commands

Command	Flag	Purpose	Example
`etcdctl endpoint status`	`-w table`	Show leader, DB size, revision	`etcdctl --endpoints=... endpoint status -w table`
`etcdctl endpoint health`	`--cluster`	Cluster-wide health	`etcdctl endpoint health --cluster`
`etcdctl member list`	`-w table`	Cluster members	`etcdctl member list -w table`
`etcdctl snapshot save`	`<path>`	Take snapshot	`etcdctl snapshot save /tmp/snap.db`
`etcdctl snapshot restore`	`--data-dir`	Restore snapshot	`etcdctl snapshot restore snap.db --data-dir=/var/lib/etcd-new`
`etcdctl snapshot status`	`-w table`	Snapshot metadata	`etcdctl snapshot status snap.db -w table`
`etcdctl defrag`	`--command-timeout=`	Defrag DB	`etcdctl defrag --command-timeout=90s --cluster`
`etcdctl alarm list`	–	List alarms (NOSPACE, CORRUPT)	`etcdctl alarm list`
`etcdctl alarm disarm`	–	Clear alarm after fix	`etcdctl alarm disarm`
`etcdctl compaction`	`<rev>`	Compact old revisions	`etcdctl compaction $(etcdctl endpoint status -w json | jq -r '.[0].Status.header.revision')`
`etcdctl move-leader`	`<id>`	Migrate leadership	`etcdctl move-leader 12345abcd`
`etcdctl member add/remove`	–	Manage membership	`etcdctl member add etcd-4 --peer-urls=https://...`
`kubeadm certs check-expiration`	–	Check cert expiry	`kubeadm certs check-expiration`

🚨 Scenarios

Scenario 1 — etcd disk full (NOSPACE alarm) - **Problem**: DB exceeds `--quota-backend-bytes` (default 2 GB); writes fail. - **Resolution**: 1. `etcdctl alarm list` → NOSPACE. 2. Compact: `etcdctl compaction <current-rev>`. 3. Defrag: `etcdctl defrag --cluster`. 4. Disarm: `etcdctl alarm disarm`. 5. Increase quota to 8 GB via `--quota-backend-bytes=8589934592`. 6. Ensure regular auto-compaction: `--auto-compaction-retention=1h`.

Scenario 2 — Lost quorum (2 of 3 members down) - **Problem**: apiserver returns 503; cluster is read-only or fully unavailable. - **Resolution**: 1. Stop remaining etcd Pod on healthy control plane node. 2. Restore from snapshot on all planned members with `--force-new-cluster` on the first. 3. Or rebuild remaining members as a single-node cluster, then `member add` new ones. 4. Post-recovery: verify all Deployments and Services intact.

Scenario 3 — Corrupted DB after power outage - **Problem**: `etcd` crash loops with "walpb: crc mismatch". - **Resolution**: 1. Restore from latest snapshot; expect brief data loss window. 2. Ensure `sync` on the storage class (SSD with power-loss protection). 3. Improve backup cadence (every 10 min).

☁ GCP GKE Perspective

- **Fully managed etcd** — you have no access.

- **Backup for GKE** (BackupPlan / RestorePlan CRDs) backs up **workloads and volumes** — it does not expose raw etcd snapshots.
- Google internally snapshots etcd for their SLA-covered restores.
- Regional clusters run etcd across 3 zones with automatic failover.
- Cost & retention: `gcloud beta container backup-restore backup-plans create` — configure schedule and retention.

AWS EKS Perspective

- **Fully managed etcd** — you have no access.
- EKS control plane is HA across 3 AZs by default.
- For workload-level DR: **Velero** with S3 backend.

```
velero install --provider aws --bucket <bucket> \
  --backup-location-config region=us-east-1 \
  --plugins velero/velero-plugin-for-aws:v1.9.0
velero backup create daily-$(date +%F) --include-namespaces prod
```

- For self-managed etcd on EC2: EBS snapshots + cross-region copy.

Hands-On Lab Exercises

Lab 64-A — Snapshot & Restore on Minikube (Beginner) - **Objective**: Take etcd snapshot; break the cluster; restore. - **Prerequisites**: Minikube. - **Steps**: 1. `minikube ssh` → `sudo ETCDCTL_API=3 etcdctl snapshot save /tmp/snap.db --endpoints=127.0.0.1:2379 --cacert=... --cert=... --key=...` . 2. Create a Deployment `test-app` . 3. Take another snapshot. 4. Delete `test-app` . 5. Restore snapshot (see docs). 6. Verify `test-app` is back. - **Verification**: Deployment reappears after restart. - **Cleanup**: `minikube delete` . - **Bonus**: Automate with a CronJob.

Lab 64-B — kubeadm HA Cluster with External etcd (Intermediate) - **Objective**: Build a 3-node stacked etcd control plane. - **Prerequisites**: 3 VMs, HAProxy VIP. - **Steps**: 1. Configure HAProxy for apiserver. 2. `kubeadm init --control-plane-endpoint=<VIP>:6443 --upload-certs` . 3. Join two more control planes. 4. Verify `kubectl get pods -n kube-system` → 3 etcd pods. 5. `etcdctl member list -w table` . - **Verification**: 3 healthy etcd members. - **Cleanup**: `kubeadm reset` on each. - **Bonus**: Kill one control plane and confirm quorum survives.

Lab 64-C — Velero Backup + Cross-Region Restore on EKS (Advanced) - **Objective**: Backup a namespace in us-east-1 EKS, restore in us-west-2 EKS. - **Prerequisites**: 2 EKS clusters, S3 bucket, IRSA for Velero. - **Steps**: 1. Install Velero in each cluster with the same bucket. 2. Deploy sample app in `us-east-1` . 3. `velero backup create demo --include-namespaces=sample` . 4. Wait for `Completed` . 5. Switch context to `us-west-2` cluster. 6. `velero restore create --from-backup demo` . - **Verification**: App running in `us-west-2` . - **Cleanup**: Delete both clusters. - **Bonus**: Add schedule for daily backups with `velero schedule create` .

Interview Deliverable Answer (60-90s)

> "etcd is the single source of truth for Kubernetes — every object goes through the apiserver into etcd via Raft consensus. What matters operationally: it needs a majority for writes, so we run 3 or 5 members; it needs fast disks (low fsync latency), because everything Kubernetes does is a read or watch against etcd; and it needs backup discipline — regular snapshots stored off-cluster, tested restores, and monitoring for the NOSPACE alarm. On kubeadm clusters I run `etcdctl snapshot save` on a schedule and copy to S3; on GKE and EKS, Google and AWS manage etcd, so I rely on Backup for GKE or Velero for workload-level DR. The failure I've handled most is disk pressure: compact, defrag, raise the quota to 8 GB, and set `--auto-compaction-retention=1h` . Quorum loss is the scariest — you restore from a snapshot with `--force-new-cluster` and rebuild."

? Interview Questions

Basic 1. **Q**: What is etcd? **A**: A distributed, consistent key-value store used by Kubernetes for all state.

1. **Q**: Why 3 or 5 members and not even numbers? **A**: Raft needs a majority; even sizes don't add fault tolerance and increase quorum cost.

Intermediate 3. **Q**: How is etcd data encrypted at rest? **A**: apiserver-level `EncryptionConfiguration` with `aescbc` or KMS provider.

1. **Q**: What does compaction do? **A**: Discards old MVCC revisions so the DB doesn't grow unbounded.
2. **Q**: What does `--quota-backend-bytes` control? **A**: Max DB size before etcd raises NOSPACE alarm and rejects writes.

Advanced 6. **Q**: How do you restore etcd cluster-wide? **A**: Stop all etcd, `snapshot restore` on each with fresh `--data-dir` and matching `--initial-cluster` , restart.

1. **Q**: How would you monitor etcd health? **A**: Prometheus metrics — `etcd_disk_wal_fsync_duration_seconds` , `etcd_server_leader_changes_seen_total` , `etcd_mvcc_db_total_size_in_bytes` , alarms.

Scenario 8. **Q**: Cluster is 503-ing; apiserver logs show etcd context deadline. What now? **A**: Check etcd leader election, disk latency, alarms; failover to healthy member; scale etcd or upgrade disks.

GKE-specific 9. **Q**: How do you back up a GKE cluster? **A**: Backup for GKE (BackupPlan CRD) — snapshots workloads & PVs, restorable to any cluster.

EKS-specific 10. **Q**: How does Velero back up an EKS cluster? **A**: Velero calls the apiserver to snapshot object YAML to S3 and optionally EBS/EFS snapshots via CSI.

Lab-based 11. **Q**: Walk me through a Velero DR to another region. **A**: (Lab 64-C) Install Velero both sides, backup with `--include-namespaces` , switch context, `velero restore create --from-backup` .

Topic 65 — Multi-Cluster Management (GKE Fleet, EKS Anywhere)

Official Documentation References - https://kubernetes.io/docs/concepts/cluster-administration/networking/ - https://cloud.google.com/kubernetes-engine/fleet-management/docs - https://cloud.google.com/anthos/config-management/docs - https://anywhere.eks.amazonaws.com/docs/ - https://karmada.io/docs/ - https://open-cluster-management.io/

Definition

Multi-cluster management is the practice — and set of tools — for operating many Kubernetes clusters as a coordinated fleet, with unified identity, policy, config delivery, service discovery, and observability.

About

Real-world environments span dozens or hundreds of clusters (per region, per BU, per environment, per compliance boundary). Managing each cluster individually

doesn't scale. Multi-cluster tools solve three big problems:

1. **Config & policy delivery** — same YAML rolled out consistently.
2. **Cross-cluster service discovery & traffic** — a Service in cluster A talking to a Pod in cluster B.
3. **Observability & identity** — one pane of glass, unified auth.

Common patterns - **Hub-and-spoke**: management cluster + workload clusters. - **Federation**: KubeFed / Karmada distribute workloads across many clusters. - **Fleet**: GKE's first-class abstraction. - **Mesh-driven**: Istio multi-cluster or Cilium ClusterMesh.

Standards emerging - **Multi-Cluster Services (MCS)** — SIG-Multicluster API (`ServiceExport` , `ServiceImport`). - **Cluster API** — declarative cluster lifecycle. - **KubeStellar / Karmada** — CNCF projects for scheduling across clusters.

Products in the space - **GKE Fleet + Config Sync + Policy Controller + Multi-Cluster Ingress/Services**. - **EKS Anywhere / EKS Hybrid Nodes + AWS Controllers for Kubernetes**. - **Red Hat ACM (Advanced Cluster Management)**. - **Rancher Fleet**. - **Karmada, Open Cluster Management (open-source).** - **Cilium ClusterMesh, Istio multicluster, Linkerd multicluster**.

🔗 How It Integrates With Other Components

Component	Integration
kubeconfig	Multiple contexts, switchable via kubectx
GitOps (ArgoCD/Flux)	ApplicationSets / Kustomization matrices target many clusters
Service Mesh	mTLS + cross-cluster routing
DNS	Global DNS + ExternalDNS per cluster; MCS auto-provisions DNS
Identity	Fleet Workload Identity (GKE), IAM roles for SA (EKS), SPIFFE/SPIRE
Observability	Central Prometheus/Thanos, Managed Prometheus, CloudWatch cross-account
Certificate management	cert-manager per cluster; central CA

🏗 Architecture Diagrams

Diagram 1 — Fleet + GitOps

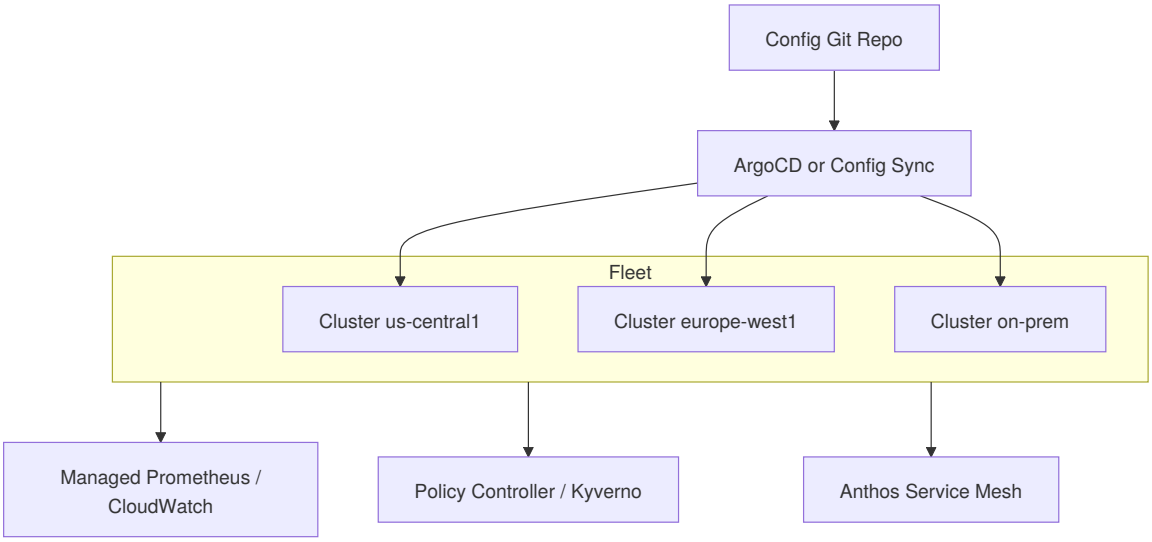


Diagram 2 — Cross-cluster service discovery (MCS)

```
Cluster A                        Cluster B
+-----------------+              +-----------------+
| Service: cart   |              | ServiceImport   |
| ServiceExport   |<------------>| for 'cart'      |
+-----------------+              +-----------------+
        |                                |
        +---- EndpointSlice sync ----------+
                (via MCS controller)
```

💻 Real-World Examples

Example 1 — Register a cluster to a GKE Fleet

```
gcloud container fleet memberships register prod-us-east \
  --gke-cluster=us-east1/prod-us-east \
  --enable-workload-identity
```

Enable Config Sync fleet-wide:

```
apiVersion: configmanagement.gke.io/v1
kind: ConfigManagement
metadata:
  name: config-management
spec:
  clusterName: prod-us-east
  git:
    syncRepo: https://github.com/acme/fleet-config
    syncBranch: main
    secretType: none
    policyDir: "clusters/prod-us-east"
```

Example 2 — MCS `ServiceExport` / `ServiceImport`

Cluster A (exports):

```
apiVersion: net.gke.io/v1
kind: ServiceExport
metadata:
  name: cart
  namespace: shop
```

Cluster B (imports automatically via MCS controller):

```
apiVersion: net.gke.io/v1
kind: ServiceImport
metadata:
  name: cart
  namespace: shop
spec:
  type: ClusterSetIP
  ports:
    - port: 80
      protocol: TCP
```

Access from cluster B:

```
curl http://cart.shop.svc.clusterset.local
```

Example 3 — ArgoCD ApplicationSet targeting all clusters

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: platform-baseline
  namespace: argocd
spec:
  generators:
    - clusters: {}          # every registered cluster
  template:
    metadata:
      name: 'baseline-{{name}}'
    spec:
      project: platform
      source:
        repoURL: https://github.com/acme/platform
        targetRevision: main
        path: baseline
      destination:
        server: '{{server}}'
        namespace: platform
      syncPolicy:
        automated: { prune: true, selfHeal: true }
```

💻 Related Commands

GKE Fleet | Command | Flag | Purpose | Example | |---------|------|---------|---------| | `gcloud container fleet memberships register` | `--gke-cluster` | Add cluster to fleet | See Example 1 | | `gcloud container fleet memberships list` | – | List members | `gcloud container fleet memberships list` | | `gcloud container fleet config-management enable` | – | Enable Config Sync | `gcloud container fleet config-management enable` | | `gcloud container fleet policycontroller enable` | – | Enable Policy Controller | `gcloud container fleet policycontroller enable` | | `gcloud container fleet mesh enable` | – | Enable Anthos Service Mesh | `gcloud container fleet mesh enable` | | `gcloud container fleet ingress enable` | – | Multi-Cluster Ingress | `gcloud container fleet ingress enable --config-membership=...` |

EKS Anywhere / General | Command | Flag | Purpose | Example | |---------|------|---------|---------| | `eksctl anywhere create cluster` | `-f` | Create EKS-A cluster | `eksctl anywhere create cluster -f cluster.yaml` | | `eksctl anywhere upgrade cluster` | – | Upgrade | `eksctl anywhere upgrade cluster -f cluster.yaml` | | `kubectx` | – | Switch context | `kubectx prod-us-east` | | `kubens` | – | Switch namespace | `kubens shop` | | `karmadactl join` | `--cluster-context` | Add cluster to Karmada | `karmadactl join member1 --cluster-context=member1` | | `clusteradm join` | `--hub-token` | OCM registration | `clusteradm join --hub-token <t> --hub-apiserver ...` |

🚨 Scenarios

Scenario 1 — Config drift across 30 clusters - **Problem**: Manual `kubectl` on one cluster caused drift; policy audits fail. - **Resolution**: 1. Enforce GitOps (Config Sync / ArgoCD) with `selfHeal`. 2. Kyverno / Policy Controller `Generate` rules to backfill missing objects. 3. RBAC: developers can't touch `kube-system` outside GitOps.

Scenario 2 — Cross-region failover - **Problem**: us-east-1 goes down; need to shift traffic to eu-west-1. - **Resolution**: 1. Multi-Cluster Ingress (GKE) or Global Accelerator (EKS) with health checks. 2. Data replication via database (Cloud SQL cross-region, Aurora Global). 3. Practice with quarterly game-day drills.

Scenario 3 — Cross-cluster mTLS trust - **Problem**: Two clusters can't mutually authenticate services. - **Resolution**: 1. Establish a shared root CA (SPIRE, cert-manager with a central issuer). 2. Use Anthos Service Mesh / Istio multi-primary with shared trust root. 3. Or Cilium ClusterMesh (WireGuard-encrypted node-to-node).

☁ GCP GKE Perspective

- **Fleet** is the primary abstraction (Anthos underneath).

- **Config Sync** delivers YAML from Git to every fleet member.
- **Policy Controller** = managed Gatekeeper across fleet.
- **Anthos Service Mesh** = managed Istio with multi-cluster support.
- **Multi-Cluster Services (MCS)** + **Multi-Cluster Ingress (MCI)** = native cross-cluster networking.
- **Fleet Workload Identity** = single trust root across clusters.
- **Backup for GKE** = fleet-wide backup.

AWS EKS Perspective

- **EKS Anywhere** — run EKS on-prem (bare metal, vSphere, Snow).
- **EKS Hybrid Nodes** — run on-prem nodes joined to a cloud EKS control plane.
- **AWS Controllers for Kubernetes (ACK)** — manage AWS from each cluster.
- **AWS Global Accelerator** + **Route 53 latency routing** for cross-cluster traffic.
- **Karpenter** can span multiple AZs; for cross-region, use multiple clusters.
- **Amazon VPC Lattice** — service-to-service across clusters/accounts.
- **Amazon Managed Grafana** for fleet observability.

Hands-On Lab Exercises

Lab 65-A — Two-Cluster GitOps with ArgoCD (Beginner) - **Objective**: One ArgoCD, two workload clusters, one repo. - **Prerequisites**: Two Kind clusters (`hub` and `worker`). - **Steps**: 1. Install ArgoCD in `hub` . 2. `argocd cluster add kind-worker` from hub. 3. Create `ApplicationSet` with a list generator naming both clusters. 4. Commit a Deployment YAML. - **Verification**: Deployment present in both clusters. - **Cleanup**: `kind delete cluster --name hub && kind delete cluster --name worker` .

Lab 65-B — GKE Fleet Register + Config Sync (Intermediate) - **Objective**: Register two GKE clusters into a fleet and enable Config Sync. - **Prerequisites**: Two GKE clusters (Autopilot fine), gcloud logged in. - **Steps**: 1. `gcloud container fleet memberships register cluster-a --gke-cluster=<region-a>/cluster-a --enable-workload-identity` 2. Repeat for cluster-b. 3. `gcloud container fleet config-management enable` . 4. Apply `ConfigManagement` CR with your Git repo (Example 1). 5. Push a Deployment to the repo. - **Verification**: Same Deployment in both clusters within a minute. - **Cleanup**: `gcloud container fleet memberships unregister ...` then delete clusters (very important for cost).

Lab 65-C — Cilium ClusterMesh Cross-cluster Service (Advanced) - **Objective**: Two Kind clusters, Cilium CNI, cross-cluster service call. - **Prerequisites**: Cilium CLI, Docker, Kind. - **Steps**: 1. Create two Kind clusters with non-overlapping Pod CIDRs. 2. Install Cilium on both. 3. `cilium clustermesh enable` on both. 4. `cilium clustermesh connect --context c1 --destination-context c2` . 5. Deploy `productcatalog` in c1, `frontend` in c2; annotate Service `io.cilium/global-service: "true"` . 6. `frontend` calls `productcatalog.default.svc.cluster.local` — resolved to c1 endpoints. - **Verification**: Successful cross-cluster HTTP calls. - **Bonus**: Add a NetworkPolicy that allows only frontend → productcatalog cross-cluster.

Interview Deliverable Answer (60–90s)

> "Multi-cluster management is how you keep a fleet of clusters coherent — same policies, same base workloads, cross-cluster service discovery, unified identity and observability. The **why** is regulatory boundaries, blast-radius reduction, and multi-region resilience. On GKE the Fleet API pulls it all together: Config Sync ships YAML from Git, Policy Controller enforces Gatekeeper policies, Multi-Cluster Services + Multi-Cluster Ingress handle east-west and north-south traffic across regions. On EKS I lean on ArgoCD ApplicationSets to fan config out, VPC Lattice or Cilium ClusterMesh for service-to-service, and IAM Identity Center for unified auth. In one project we ran 24 clusters across 4 regions with a single ArgoCD hub and a Kyverno baseline policy pack — onboarding a new cluster was: register in fleet, add the label, the baseline appeared within minutes."

? Interview Questions

Basic 1. **Q**: Why not one giant cluster? **A**: Blast radius, per-region latency, compliance boundaries, upgrade windows, API server scale limits.

1. **Q**: What is a fleet? **A**: A named group of clusters managed together with shared config, identity, and policy.

Intermediate 3. **Q**: How do you deliver config to many clusters? **A**: GitOps: Config Sync, ArgoCD ApplicationSet, Flux Kustomizations.

1. **Q**: What is Multi-Cluster Services (MCS)? **A**: A SIG-Multicluster API (ServiceExport/ServiceImport) for cross-cluster service discovery.

Advanced 5. **Q**: Differences between Karmada and OCM? **A**: Karmada focuses on workload propagation with a scheduler; OCM is a broader hub-agent model with subscriptions and policies.

1. **Q**: How do you unify identity across clusters? **A**: Fleet Workload Identity (GKE), IAM Identity Center + IRSA (EKS), SPIFFE/SPIRE for open-source.

Scenario 7. **Q**: Cluster A can't call Service in cluster B — where do you look? **A**: Network reachability, CNI (ClusterMesh/MCS status), DNS resolution, mTLS trust root, NetworkPolicies.

GKE-specific 8. **Q**: What does `gcloud container fleet memberships register` do? **A**: Adds a cluster to the fleet, installs the fleet agent (`gke-connect`), and gives it a stable identity.

EKS-specific 9. **Q**: What's the difference between EKS Anywhere and EKS Hybrid Nodes? **A**: EKS-A runs a full self-managed control plane on-prem; Hybrid Nodes join on-prem nodes to an existing cloud EKS control plane.

Lab-based 10. **Q**: Show me spreading a workload to two clusters via ArgoCD. **A**: (Lab 65-A) ApplicationSet with list generator naming both clusters + `destination.server: {{server}}` .

Topic 66 — Service Catalog & API Priority and Fairness (APF)

Official Documentation References - https://kubernetes.io/docs/concepts/extend-kubernetes/service-catalog/ *(archived — Service Catalog is deprecated)* - https://kubernetes.io/docs/concepts/cluster-administration/flow-control/ - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#flowschema-v1-flowcontrol-apiserver-k8s-io - https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.31/#prioritylevelconfiguration-v1-flowcontrol-apiserver-k8s-io

Definition

- **Service Catalog** was a Kubernetes extension (now deprecated) that let apps discover and provision external managed services via the Open Service Broker API.
- **API Priority and Fairness (APF)** is the built-in mechanism that classifies inbound apiserver requests into priority levels and fairly shares each level's capacity across users so no client can starve the apiserver.

📖 About

Service Catalog (historical context)

- Introduced circa 2017 to let workloads request managed services (MySQL, Redis, S3) via `ServiceInstance` and `ServiceBinding` CRs.
- Backed by Open Service Broker (OSB) API implementations from GCP, Azure, AWS, and OpenShift.
- **Deprecated** in 2020; replaced in practice by:
- **Crossplane** — declarative infrastructure via CRDs.
- **Config Connector** on GKE, **ACK** on EKS.
- **Kubernetes Service Bindings** (SIG proposal) for injecting connection details.
- You may still see it in interview questions or legacy platforms.

API Priority and Fairness (APF) — the modern, essential topic

APF replaces the old `--max-requests-inflight` global throttle with a fair, tenant-aware model. Every inbound API request is:

1. Matched by a **FlowSchema** (which requests are you?).
2. Assigned a **priority level** (how important are you?).
3. Placed into a **flow queue** (fair sharing within the level).
4. Executed if capacity allows; else queued (or rejected with 429).

Why APF exists - A single misbehaving controller used to be able to DoS the apiserver. - Different clients have different criticality (leader-election vs. bulk list). - APF isolates categories: system > leader-election > workload-high > workload-low > global-default > catch-all.

Default priority levels (v1.30+) | Level | Concurrency share | Use | |-------|-------------------|-----| | `system` | high | system:masters and control plane | | `node-high` | medium | kubelets | | `leader-election` | small | Lease writes | | `workload-high` | medium | in-cluster controllers | | `workload-low` | medium | user apps | | `global-default` | small | default catch-all | | `catch-all` | tiny | anything unmatched | | `exempt` | unlimited | truly critical (e.g. system:masters via FS) |

FlowSchema matches by: - Subject (user, group, ServiceAccount). - Resource / non-resource rules. - Namespace.

Distinguishing "flows" within a level: `distinguisherMethod: ByUser` or `ByNamespace`.

Metrics you monitor - `apiserver_flowcontrol_dispatched_requests_total` - `apiserver_flowcontrol_current_inqueue_requests` - `apiserver_flowcontrol_rejected_requests_total`

🔗 How It Integrates With Other Components

Component	Integration
kube-apiserver	Runs the APF filter; exposes flow-control metrics
Controllers	Get isolated priority — one bad controller can't starve others
Leader-election Leases	Own priority level to guarantee stability
Prometheus	Scrapes APF metrics for capacity & rejection tracking
Custom admission	Doesn't participate in APF (admission runs later)

🏗 Architecture Diagrams

Diagram 1 — Request path through APF

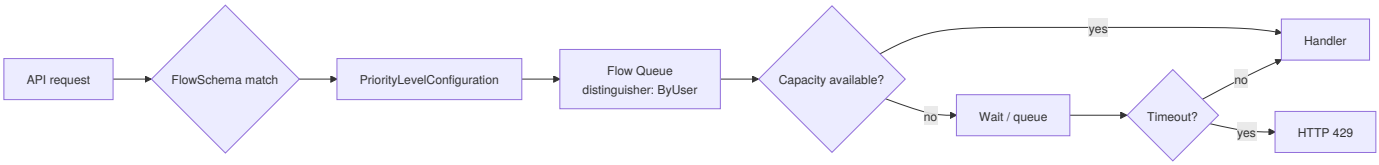


Diagram 2 — Priority level isolation

```
apiserver total inflight = 400
  |
  +--> system (25%)           = 100
  +--> leader-election (10%)  = 40
  +--> workload-high (30%)    = 120
  +--> workload-low (20%)     = 80
  +--> global-default (10%)   = 40
  +--> catch-all (5%)         = 20
```

💻 Real-World Examples

Example 1 — Custom FlowSchema for your controller

```yaml
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: FlowSchema
metadata:
  name: my-important-controller
spec:
  priorityLevelConfiguration:
    name: workload-high
  matchingPrecedence: 900
  distinguisherMethod:
    type: ByUser
  rules:
    - subjects:
        - kind: ServiceAccount
          serviceAccount:
            name: my-controller
            namespace: platform
      resourceRules:
        - verbs: ["*"]
          apiGroups: ["*"]
          resources: ["*"]
          namespaces: ["*"]
```

Example 2 — Custom PriorityLevelConfiguration

```yaml
apiVersion: flowcontrol.apiserver.k8s.io/v1
kind: PriorityLevelConfiguration
metadata:
  name: bulk-jobs
spec:
  type: Limited
  limited:
    nominalConcurrencyShares: 10
    lendablePercent: 50
    borrowingLimitPercent: 100
    limitResponse:
      type: Queue
      queuing:
        queues: 32
        handSize: 6
        queueLengthLimit: 50
```

Then a FlowSchema pointing at `bulk-jobs` matches Job controllers doing large list operations.

Example 3 — Legacy Service Catalog `ServiceInstance` (illustrative only)

```yaml
apiVersion: servicecatalog.k8s.io/v1beta1
kind: ServiceInstance
metadata:
  name: mydb
spec:
  clusterServiceClassExternalName: gcp-cloud-sql
  clusterServicePlanExternalName: mysql-8-basic
  parameters:
    region: us-east1
    tier: db-n1-standard-1
```

Modern equivalent with **Crossplane** or **Config Connector**:

```yaml
apiVersion: sql.cnrm.cloud.google.com/v1beta1
kind: SQLInstance
metadata: { name: mydb }
spec:
  databaseVersion: MYSQL_8_0
  region: us-east1
  settings:
    tier: db-n1-standard-1
```

💻 Related Commands

Command	Flag	Purpose	Example
`kubectl get flowschemas`	–	List FlowSchemas	`kubectl get flowschemas`
`kubectl get prioritylevelconfigurations`	–	List priority levels	`kubectl get prioritylevelconfigurations`
`kubectl describe flowschema <name>`	–	Inspect a schema	`kubectl describe flowschema my-important-controller`
`kubectl get --raw /debug/api_priority_and_fairness/dump_priority_levels`	–	Live state	–
`kubectl get --raw /debug/api_priority_and_fairness/dump_queues`	–	Queue details	–
`kubectl get --raw /metrics`	`| grep flowcontrol`	Metrics	–
`kubectl top`	–	Ensure metrics-server not in `catch-all`	–
`kubectl get --raw /readyz?verbose`	–	Confirms APF healthy	–

🎬 Scenarios

Scenario 1 — 429s spiking on the apiserver - **Problem**: A custom operator is being throttled (429 "TooManyRequests"); reconciliation slows. - **Resolution**: 1. Identify the priority level: `apiserver_flowcontrol_rejected_requests_total{priority_level="global-default"}` . 2. Create a FlowSchema mapping your controller's SA to `workload-high` . 3. Also fix the controller: add caching (client-go informers), avoid unbounded lists.

Scenario 2 — Leader-election flapping under load - **Problem**: `Lease` renewals miss deadlines; controllers keep re-electing. - **Resolution**: 1. Ensure the built-

in FlowSchema `system-leader-election` is not modified. 2. Increase Lease renewal deadline in controller configuration. 3. Scale apiserver replicas / raise `--max-requests-inflight`.

Scenario 3 — Legacy Service Catalog broker unavailable - **Problem**: `ServiceInstance` stuck `ProvisionCallFailed`. - **Resolution**: 1. Migrate off Service Catalog (deprecated) to Crossplane / Config Connector / ACK. 2. In the interim, restart the broker; check credentials + network.

☁ GCP GKE Perspective

- Modern replacement for Service Catalog on GKE: **Config Connector** — GCP resources as CRDs.
- APF is enabled on GKE; you can add custom FlowSchemas.
- **Autopilot** users can create FlowSchemas & PriorityLevels (they're cluster-scoped API objects), just like Standard.
- Monitor with **Cloud Monitoring →** `apiserver_flowcontrol_*` metrics via managed Prometheus.

▩ AWS EKS Perspective

- Modern Service Catalog replacement on EKS: **AWS Controllers for Kubernetes (ACK)** or **Crossplane**.
- APF works identically. EKS control plane scales its apiserver replicas horizontally; APF still prevents noisy-neighbor problems.
- Monitor via **CloudWatch Container Insights** or a Prometheus scrape of `/metrics`.

🧪 Hands-On Lab Exercises

🧪 **Lab 66-A — Explore Default APF (Beginner)** - **Objective**: See default FlowSchemas + PriorityLevels; observe metrics. - **Prerequisites**: Any cluster ≥ v1.29. - **Steps**: 1. `kubectl get flowschemas -o wide` 2. `kubectl get prioritylevelconfigurations -o wide` 3. `kubectl describe flowschema system-leader-election` 4. Port-forward apiserver metrics if accessible; otherwise `kubectl get --raw /metrics | grep flowcontrol_dispatched`. - **Verification**: Metrics visible. - **Bonus**: Draw the mapping between top 5 FlowSchemas and priority levels.

🧪 **Lab 66-B — Prioritize a Custom Controller (Intermediate)** - **Objective**: Move your controller from `global-default` to `workload-high`. - **Steps**: 1. Create a Deployment + SA representing your controller doing `list pods` in a loop. 2. Observe `apiserver_flowcontrol_dispatched_requests_total{flow_schema="global-default"}` climb. 3. Apply the FlowSchema from Example 1 pointing at your SA. 4. Watch metric shift to `workload-high`. - **Verification**: Priority level changes; no 429s under load. - **Bonus**: Add a custom PriorityLevel `bulk-lists` and route your controller there.

🧪 **Lab 66-C — Provision Cloud Resource with Crossplane (Advanced)** - **Objective**: Replace legacy Service Catalog with Crossplane provisioning an S3 bucket / GCS bucket. - **Prerequisites**: Kind, Crossplane installed, cloud creds. - **Steps**: 1. `helm install crossplane crossplane-stable/crossplane -n crossplane-system --create-namespace` 2. Install provider (`provider-aws` or `provider-gcp`). 3. Create `ProviderConfig` with credentials. 4. Apply a `Bucket` CR. - **Verification**: Bucket exists in cloud console. - **Cleanup**: Delete CR; provider deletes bucket. Uninstall Crossplane. - **Bonus**: Compose a `XPostgresCluster` CompositeResource that creates DB + user + secret.

🎯 Interview Deliverable Answer (60–90s)

> "Two topics that often get lumped together. **Service Catalog** was the older way to provision managed services from Kubernetes via the Open Service Broker API — `ServiceInstance` / `ServiceBinding` CRs. It's deprecated; the modern replacements are Crossplane, Config Connector on GKE, and ACK on EKS, all of which model cloud resources as CRDs. **API Priority and Fairness** is the essential one: APF classifies every API request into a FlowSchema, assigns it a PriorityLevel, and fair-queues it so a noisy controller can't starve the apiserver or preempt leader-election. Why it matters: pre-APF, one runaway controller could 429 the whole cluster. On GKE and EKS both, I've added FlowSchemas moving critical operators from `global-default` to `workload-high` and watched 429 rejections drop. When I see rising `apiserver_flowcontrol_rejected_requests_total`, that's my signal to look at both the workload behavior and its priority mapping."

? Interview Questions

Basic 1. **Q**: What is Service Catalog and what replaced it? **A**: Legacy CRD-based provisioning of managed services via OSB; replaced by Crossplane, Config Connector, ACK.

1. **Q**: What problem does APF solve? **A**: Fairly shares apiserver capacity so no client can DoS others.

Intermediate 3. **Q**: What's a FlowSchema? **A**: A CR matching requests (by subject + resource) and routing them to a PriorityLevelConfiguration.

1. **Q**: What's the difference between `Queue` and `Reject` limit responses? **A**: `Queue` waits with configured queue length; `Reject` returns 429 immediately.
2. **Q**: What's the `exempt` priority level? **A**: Unlimited concurrency — used for truly critical requests like health probes or system:masters.

Advanced 6. **Q**: What is `matchingPrecedence`? **A**: FlowSchemas evaluated lowest-precedence-first; first match wins. Use lower numbers for more specific schemas.

1. **Q**: What does `nominalConcurrencyShares` do? **A**: Sets this level's share of total apiserver concurrency proportional to the sum of all shares.

Scenario 8. **Q**: 429 storm from a controller — what steps do you take? **A**: Check `apiserver_flowcontrol_rejected_requests_total` by FlowSchema, add or adjust FlowSchema for that SA, and fix the controller to use informers.

GKE-specific 9. **Q**: What's the modern equivalent of Service Catalog on GKE? **A**: Config Connector (GCP resources as CRDs) and/or Crossplane.

EKS-specific 10. **Q**: How to provision an RDS instance from an EKS cluster? **A**: ACK RDS controller — apply a `DBInstance` CR; ACK creates it via AWS APIs (SA authenticated via IRSA).

Lab-based 11. **Q**: Show me raising the priority of my custom operator. **A**: (Lab 66-B) Create FlowSchema referencing the operator's SA and pointing to `workload-high`, verify with metrics.

⏸ Pause Point — End of Batch 2 of 5

Completed in this batch (Topics 62–66): - ✅ Admission Controllers (Mutating & Validating, VAP/CEL, Kyverno, Gatekeeper) - ✅ API Aggregation Layer (APIService, metrics-server, custom aggregators) - ✅ etcd Deep Dive (Raft, snapshots, restore, DR, Velero, Backup for GKE) - ✅ Multi-Cluster Management (Fleet, ArgoCD ApplicationSets, MCS, Cilium ClusterMesh, EKS Anywhere) - ✅ Service Catalog (legacy) & API Priority and Fairness (APF, FlowSchemas, PriorityLevels)

Remaining: - **Batch 3 (Topics 67–70):** GKE Standard vs Autopilot, EKS node models, Workload Identity vs IRSA, Version lifecycle - **Batch 4 (Topics 71–73):** Top 50 troubleshooting, Top 30 system design, Behavioral + K8s hybrid - **Batch 5 (Capstones 1–10):** End-to-end lab projects - **Final:** PDF compilation

Say **"continue"** to proceed with Batch 3 (Topics 67–70).

MODULE

Module 11 — Managed Kubernetes Deep Dive

Module 11 — Managed Kubernetes Deep Dive

Topics 67 – 70

🗐 Topic 67 — GKE Standard vs GKE Autopilot — Full Comparison

Official Documentation References - https://cloud.google.com/kubernetes-engine/docs/concepts/choose-cluster-mode - https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview - https://cloud.google.com/kubernetes-engine/docs/concepts/cluster-architecture - https://cloud.google.com/kubernetes-engine/pricing

🗐 Definition

GKE Standard is Google's traditional Kubernetes offering where you manage node pools; GKE Autopilot is a fully managed mode where Google runs and bills you per Pod for compute, while enforcing security and reliability best practices at admission.

🕮 About

GKE Standard — you configure everything below the API surface: node pools, machine types, autoscaling, node images, DaemonSets, and pay for VMs whether idle or not (plus a control-plane fee outside the free tier). You get maximum flexibility: privileged Pods, `hostNetwork`, custom DaemonSets, GPU node pools, TPU nodes, sole-tenant nodes, Windows nodes, spot VMs.

GKE Autopilot — Google manages the nodes; you never see them (no `kubectl get nodes -o wide` view of machine types you selected). You define workloads, Google right-sizes and provisions the underlying compute. You're billed **per Pod vCPU/memory/ephemeral-storage**, not per VM. Autopilot enforces a strict security baseline via admission: no privileged containers, no `hostPath` (with narrow exceptions), no `hostNetwork`, no `hostPID`. DaemonSets are allowed but restricted; system-level tools that need node access must use the supported extension mechanisms.

Why Autopilot exists - Eliminate node-ops toil. - Guarantee upgrade/security posture. - Bill only for workload consumption. - Give a "just deploy Pods" experience.

Key trade-offs | Aspect | GKE Standard | GKE Autopilot | |--------|--------------|---------------| | Node management | You (node pools, upgrades opt-in) | Google | | Billing model | Per-VM (node) + control plane | Per-Pod resources + control plane | | Minimum Pod cost | Any (share node) | Enforced minimum requests (e.g., 250m CPU / 512 Mi mem for general workloads) | | Privileged containers | Allowed | Blocked at admission | | `hostNetwork` / `hostPath` | Allowed | Blocked (with narrow exceptions) | | DaemonSets | Any | Only workloads that satisfy Autopilot constraints | | Custom kernel modules | Yes | No | | GPU support | Yes | Yes (from 1.24.2-gke.400) | | Spot VMs / Preemptible | Yes | Spot Pods (feature) | | Windows | Yes | No | | SSH to node | Yes | No | | SLA | 99.5%–99.95% | 99.9% Pod SLA + 99.95% control plane in regional | | Best for | Custom infra, kernel tweaks, cost tuning | Product teams, high compliance, low ops burden |

Autopilot resource classes Autopilot supports specialized compute classes: `general-purpose` (default), `balanced`, `scale-out`, `performance`, `accelerator` (GPUs), and `spot` — selectable via `nodeSelector` `cloud.google.com/compute-class`.

Autopilot minimums (v1.30+) - CPU: 50m minimum per container, 250m per Pod. - Memory: 52 Mi per container, 512 Mi per Pod (varies by compute class). - CPU:Memory ratios enforced (1 vCPU : 1–6.5 GiB for general-purpose).

Pricing note: Pricing changes; always verify at https://cloud.google.com/kubernetes-engine/pricing.

🔗 How It Integrates With Other Components

Component	Standard	Autopilot
Node pools	You create/scale	Managed; invisible
Cluster Autoscaler	Optional add-on	Always on
Node Auto-Provisioning (NAP)	Optional	Always on
Workload Identity	Optional	Always on (mandatory)
Shielded Nodes	Optional	Always on
GKE Sandbox (gVisor)	Optional	Optional per Pod
VPC-native (alias IP)	Optional (recommended)	Always
Private cluster	Optional	Optional
GKE Dataplane V2 (Cilium)	Optional	Always
Logging & Monitoring	Optional	Always on

🏗 Architecture Diagrams

Diagram 1 — GKE Standard

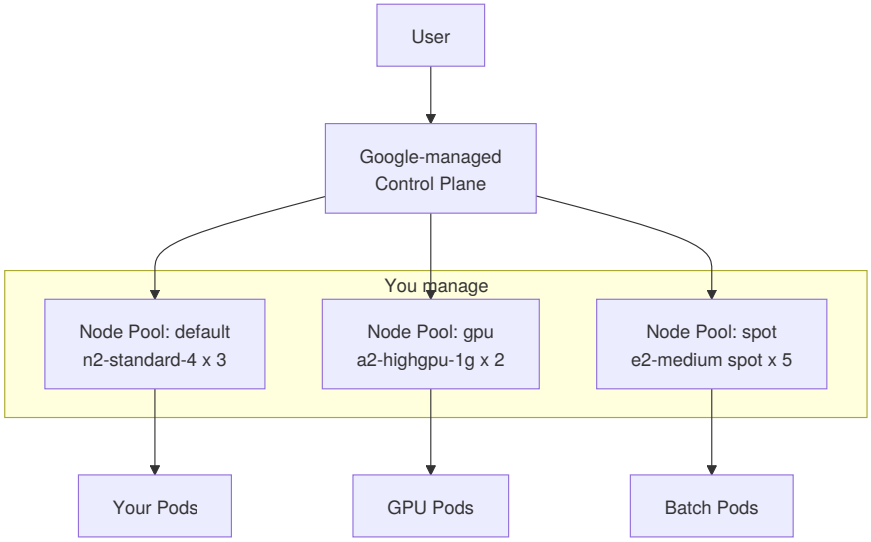


Diagram 2 — GKE Autopilot

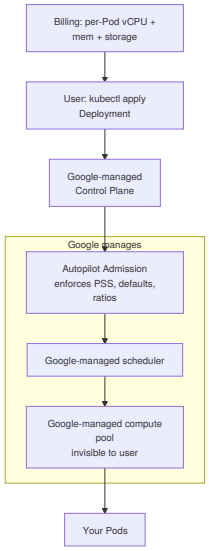


Diagram 3 — Decision tree

```
Need privileged / hostNetwork / kernel tweaks?
   yes ---> GKE Standard
   no
   |
Need Windows nodes?
   yes ---> GKE Standard
   no
   |
Highly variable/burst workload?
   yes ---> Autopilot (pay per Pod)
   no
   |
Predictable, high-utilization cluster?
   yes ---> Standard (reserved VMs may be cheaper)
   |
Team lacks node-ops expertise?
   yes ---> Autopilot
```

💻 Real-World Examples

Example 1 — Create Standard cluster

```
gcloud container clusters create shop-std \
  --region=us-central1 \
  --release-channel=regular \
  --enable-ip-alias \
  --enable-dataplane-v2 \
  --enable-shielded-nodes \
  --workload-pool=my-project.svc.id.goog \
  --num-nodes=1 \
  --machine-type=n2-standard-4 \
  --enable-autoscaling --min-nodes=1 --max-nodes=10 \
  --enable-autoupgrade --enable-autorepair
```

Add a spot node pool:

```
gcloud container node-pools create spot-pool \
  --cluster=shop-std --region=us-central1 \
  --spot --machine-type=e2-standard-4 \
  --num-nodes=0 --enable-autoscaling --min-nodes=0 --max-nodes=20 \
  --node-taints=cloud.google.com/gke-spot=true:NoSchedule
```

Example 2 — Create Autopilot cluster

```
gcloud container clusters create-auto shop-auto \
  --region=us-central1 \
  --release-channel=regular \
  --workload-pool=my-project.svc.id.goog
```

Deploy a Pod requesting the `performance` compute class:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: api }
spec:
  replicas: 3
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api } }
    spec:
      nodeSelector:
        cloud.google.com/compute-class: Performance
        cloud.google.com/machine-family: c3
      containers:
        - name: api
          image: gcr.io/my-project/api:1.2.3
          resources:
            requests: { cpu: "2", memory: "4Gi" }
            limits:   { cpu: "2", memory: "4Gi" }
```

Example 3 — Spot Pod on Autopilot

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: batch-worker }
spec:
  replicas: 10
  selector: { matchLabels: { app: worker } }
  template:
    metadata: { labels: { app: worker } }
    spec:
      terminationGracePeriodSeconds: 25
      nodeSelector:
        cloud.google.com/gke-spot: "true"
      tolerations:
        - key: cloud.google.com/gke-spot
          operator: Equal
          value: "true"
          effect: NoSchedule
      containers:
        - name: worker
          image: my-repo/worker:2.0
          resources:
            requests: { cpu: "500m", memory: "1Gi" }
```

💻 Related Commands

Command	Flag	Purpose	Example
`gcloud container clusters create`	`--region --num-nodes`	Standard cluster	See Ex. 1
`gcloud container clusters create-auto`	`--region --release-channel`	Autopilot cluster	See Ex. 2
`gcloud container clusters list`	–	List clusters	`gcloud container clusters list`
`gcloud container clusters describe`	`--region`	Detailed info	`gcloud container clusters describe shop-std --region=us-central1`
`gcloud container clusters get-credentials`	`--region`	Configure kubectl	`gcloud container clusters get-credentials shop-std --region=us-central1`
`gcloud container node-pools create`	`--spot --enable-autoscaling`	Add pool	See Ex. 1
`gcloud container node-pools list`	`--cluster`	List pools	`gcloud container node-pools list --cluster=shop-std --region=us-central1`
`gcloud container clusters update`	`--enable-autoscaling`	Modify options	`gcloud container clusters update shop-std --enable-vertical-pod-autoscaling`
`gcloud container clusters upgrade`	`--cluster-version`	Upgrade	`gcloud container clusters upgrade shop-std --master --cluster-version=1.31.1-gke.1846000`
`gcloud container operations list`	–	Track long-running ops	`gcloud container operations list --region=us-central1`
`gcloud container clusters delete`	`--quiet`	Delete	`gcloud container clusters delete shop-std --region=us-central1 --quiet`

🎬 Scenarios

Scenario 1 — Pod stuck Pending on Autopilot with "no compute available" - **Problem**: Pod requests violate compute-class rules or exceed max sizes. - **Resolution**: 1. `kubectl describe pod` shows Autopilot admission message ("resource request exceeds..."). 2. Adjust requests to fit ratios (1 vCPU : 1–6.5 GiB general). 3. If need >30 vCPU per Pod, use `Performance` class. 4. Split into smaller Pods where possible.

Scenario 2 — Bill on Autopilot larger than expected - **Problem**: Team over-requested resources; Autopilot bills the requests. - **Resolution**: 1. Right-size with VPA (`updateMode: "Off"` for recommendations). 2. Use Spot Pods for stateless batch. 3. Consider Standard with committed use discount for predictable, high-utilization workloads.

Scenario 3 — Custom DaemonSet rejected on Autopilot - **Problem**: Trying to install a monitoring agent that uses `hostPath` . - **Resolution**: 1. Use Google-managed alternative (Managed Prometheus, Cloud Ops Agent) — installed as DaemonSets by Google. 2. Convert agent to a sidecar model. 3. Or move to Standard.

☁ GCP GKE — Deep dive specifics

- **Release channels**: `rapid` , `regular` , `stable` . Autopilot requires channel enrollment; Standard can opt out.
- **Autopilot Extended Support**: pay-per-hour to stay on an older minor version.
- **Cost note**: Both modes charge $0.10/hour per cluster after the free tier (one zonal cluster free). Autopilot compute is priced per Pod resource-hour.
- **Node auto-provisioning (NAP)** in Standard grows/shrinks node pools automatically; in Autopilot it's the whole scheduling model.

▩ AWS EKS — Comparison Perspective

- Autopilot's closest EKS analogue is **Fargate**, but Fargate has stricter limitations (no DaemonSets, no privileged, one Pod per micro-VM). See Topic 68 for the full comparison.
- EKS with Managed Node Groups + Karpenter approximates Standard + NAP.
- IRSA (EKS) is the analogue of Workload Identity (Autopilot mandates it; Standard can enable it).

🧪 Hands-On Lab Exercises

🧪 **Lab 67-A — Provision Autopilot + Standard Side-by-Side (Beginner)** - **Objective**: Compare experience. - **Prerequisites**: GCP project with billing, `gcloud` . - **Steps**: 1. Create Autopilot: Example 2. 2. Create Standard: Example 1. 3. `kubectl get nodes` on both (Autopilot shows nodes but they're Google-managed). 4. Deploy an nginx Deployment (3 replicas) to each. 5. Compare `kubectl top pods` , cost after 1 hour. - **Verification**: Both serving traffic. - **Cleanup**: `gcloud container clusters delete shop-std shop-auto --region=us-central1 --quiet` . - **Troubleshooting**: If quotas denied, request more in IAM & Admin > Quotas. - **Bonus**: Deploy the same workload to both, port-forward, and note the DNS/service parity.

🧪 **Lab 67-B — Autopilot Compute Classes (Intermediate)** - **Objective**: Schedule workloads on `Performance` + `Spot` . - **Steps**: 1. Create Autopilot cluster. 2. Deploy an app with `nodeSelector: cloud.google.com/compute-class: Performance` . 3. Deploy a batch app with the spot nodeSelector (Example 3). 4. Watch the Pods land on separate underlying node types. - **Verification**: `kubectl get pod -o wide` shows different node names. - **Bonus**: Add a PDB for the spot workload.

🧪 **Lab 67-C — Migrate a Workload from Standard → Autopilot (Advanced)** - **Objective**: Move an existing 3-tier app. - **Steps**: 1. Export objects from Standard: `kubectl get -A -o yaml > backup.yaml` . 2. Clean up: remove privileged, hostPath, hostNetwork. 3. Add resource requests to every container. 4. Apply on Autopilot cluster. 5. Check `kubectl get events` for admission rejections; fix and re-apply. - **Verification**: All Pods Running. - **Cleanup**: Delete both clusters. - **Bonus**: Run the same workload for 24 h in each; compare billing.

🎯 Interview Deliverable Answer (60–90s)

> "GKE gives you two flavors. **Standard** is the classic — I own node pools, machine types, upgrade cadence, and I can run anything: privileged Pods, GPUs, spot VMs, Windows nodes. **Autopilot** is fully managed — Google runs the nodes, enforces PSS at admission (no privileged, no hostPath, no hostNetwork), and bills per Pod's vCPU and memory requests. Why: Autopilot eliminates node-ops and drives a strict security baseline, so it's my default for product teams and regulated workloads. Standard is my choice for platform layers that need low-level access, cost tuning with committed use discounts, or custom DaemonSets. Both share the same Kubernetes API, so migration is mostly about scrubbing privileged assumptions and setting resource requests. In practice I run a Standard 'platform' cluster for shared infra like ingress and Prometheus, and Autopilot 'workload' clusters for application teams."

? Interview Questions

Basic 1. **Q**: What's the biggest difference between GKE Standard and Autopilot? **A**: Standard: you manage nodes and pay per VM. Autopilot: Google manages nodes and you pay per Pod resources.

1. **Q**: Can you run privileged Pods on Autopilot? **A**: No — admission rejects them.

Intermediate 3. **Q**: What does Autopilot enforce out of the box that you'd have to opt into on Standard? **A**: Workload Identity, Shielded Nodes, Dataplane V2, Cluster Autoscaler, Node Auto-Provisioning, PSS admission, encrypted etcd, logging & monitoring.

1. **Q**: How is billing different? **A**: Standard bills VMs regardless of utilization. Autopilot bills only per Pod resource requests (plus cluster fee).
2. **Q**: What are Autopilot compute classes? **A**: Named profiles (general-purpose, balanced, scale-out, performance, accelerator, spot) selected via nodeSelector.

Advanced 6. **Q**: When would you *not* pick Autopilot? **A**: Windows nodes, custom kernel modules, low-cost long-running workloads with committed use discounts, workloads that require hostNetwork/hostPath.

1. **Q**: How does Autopilot enforce security? **A**: Via admission controllers that mutate/reject Pods violating Autopilot policy; the platform runs a hardened node OS with restricted node privileges.

Scenario 8. **Q**: Migrating an on-prem workload to Autopilot fails admission — walk me through debugging. **A**: `kubectl describe` the Pod for admission messages; remove privileged, hostPath (or use CSI), hostNetwork; add resource requests; adjust CPU:memory ratios.

GKE-specific 9. **Q**: What are release channels? **A**: `rapid` , `regular` , `stable` — pin the cluster's upgrade cadence.

EKS-specific 10. **Q**: EKS analogue of Autopilot? **A**: Fargate is closest but stricter (no DaemonSets, one Pod per micro-VM); many teams use EKS + Karpenter as an alternative "elastic" model.

Lab-based 11. **Q**: Show me creating an Autopilot cluster and deploying a Spot workload. **A**: (Lab 67-B) `gcloud container clusters create-auto ...` , then Deployment with `cloud.google.com/gke-spot: "true"` nodeSelector + toleration.

🗄 Topic 68 — EKS Managed Node Groups vs Self-Managed Nodes vs Fargate

Official Documentation References - https://docs.aws.amazon.com/eks/latest/userguide/eks-compute.html - https://docs.aws.amazon.com/eks/latest/userguide/managed-node-groups.html - https://docs.aws.amazon.com/eks/latest/userguide/fargate.html -

https://karpenter.sh/docs/ - https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html

📄 Definition

EKS offers three compute models for running Pods: 1. **Managed Node Groups (MNG)** — AWS-managed EC2 node groups with lifecycle automation. 2. **Self-Managed Nodes** — EC2 you build with the EKS-optimized AMI and register yourself (bring-your-own automation). 3. **Fargate** — serverless per-Pod micro-VMs; you don't see EC2 at all.

A modern fourth option is **Karpenter** — an open-source scheduler that provisions raw EC2 (not a node group) on demand; it replaces or augments MNG for many teams.

📖 About

Managed Node Groups - AWS creates an Auto Scaling Group and EC2 instances using the EKS-optimized AMI (AL2/Bottlerocket/Windows). - Handles rolling updates, cordon+drain, upgrade to a new AMI or K8s version with a single API call. - Supports Spot Instances, GPU AMIs, and custom launch templates. - IAM role attached; nodes register via bootstrap script.

Self-Managed Nodes - You author launch templates or use Terraform/eksctl. - Freedom to run custom AMIs, exotic instance types, or federated on-prem nodes (via EKS Hybrid Nodes). - You own upgrades, drain, and AMI rotation.

Fargate - No nodes visible to you. Each Pod runs in a dedicated Firecracker micro-VM. - Billed per Pod vCPU/memory/second (with a minimum billing time). - Restrictions: - No DaemonSets (workaround: sidecars). - No privileged containers. - No `hostPort`, `hostNetwork`, `hostPath` (only projected volumes, ConfigMaps, Secrets, EmptyDir up to 20 GiB, EFS). - Only one Pod per micro-VM. - Some CNI features restricted; uses **Fargate Pod networking** with pre-allocated ENIs. - Selected per Pod via **Fargate Profile** matching `namespace` + `labels`.

Karpenter (open source, now graduated for EKS) - Watches unschedulable Pods and provisions raw EC2 within seconds using diverse instance types. - Consolidates and terminates underutilized nodes automatically. - Requires **NodePool** and **EC2NodeClass** CRDs. - Often replaces Cluster Autoscaler + MNG for elasticity-heavy workloads.

Comparison table

Aspect	MNG	Self-Managed	Fargate	Karpenter
Who owns EC2	AWS	You	N/A	Karpenter
Upgrade rolling	AWS managed	You	N/A	Drift-triggered replace
Spot support	Yes	Yes	No (Fargate Spot exists separately)	Yes (built-in diversity)
GPU	Yes	Yes	No	Yes
DaemonSets	Yes	Yes	✗	Yes
Windows	Yes	Yes	✗	Yes
Privileged Pods	Yes	Yes	✗	Yes
Bin-packing	ASG-limited	Manual	Per-Pod	Excellent
Node startup	1–3 min	1–3 min	30–60 s (Pod-level)	40–60 s
Cost model	EC2 + EKS fee	EC2 + EKS fee	Per Pod (higher $/vCPU)	EC2 + EKS fee
Best for	Standard workloads	Special AMIs / hybrid	Bursty, low-density	Bursty, dense, mixed

🔗 How It Integrates With Other Components

Component	Integration
EKS control plane	Consumes all three via the same API server
VPC CNI (aws-node)	DaemonSet on MNG/self-managed; not on Fargate (uses per-Pod ENI)
kube-proxy	DaemonSet on MNG/self-managed; not on Fargate
CoreDNS	Runs anywhere; on Fargate-only clusters requires a Fargate Profile matching `kube-system`
CoreDNS/aws-load-balancer-controller	Works with all three
Karpenter	Watches unscheduled Pods; ignores MNG scale events
IRSA / EKS Pod Identity	Works on all three
CSI drivers (EBS/EFS)	EBS DaemonSet only on nodes; Fargate supports EFS via CSI

🏗 Architecture Diagrams

Diagram 1 — Three compute models coexisting

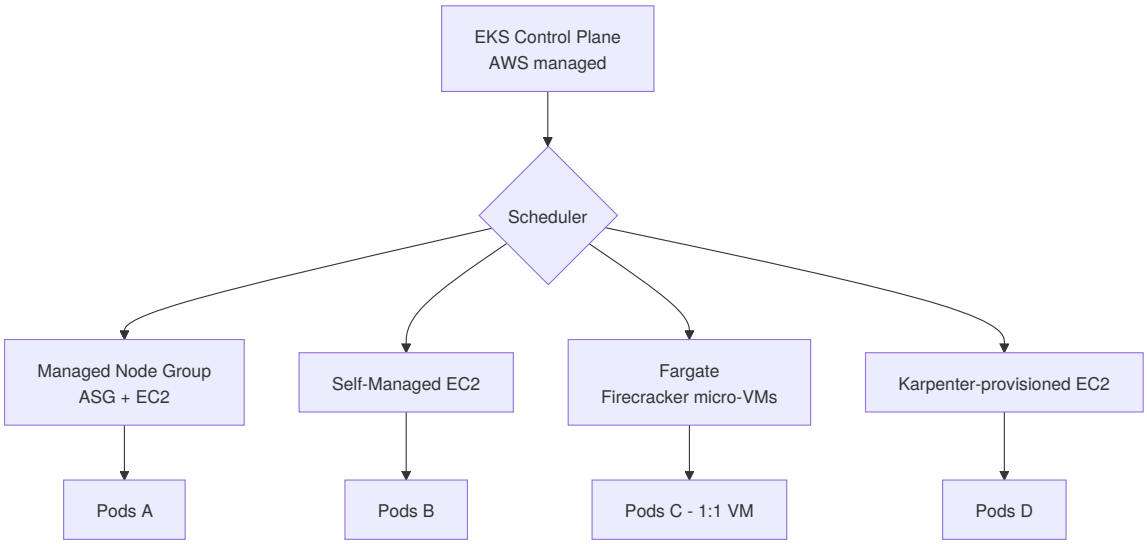
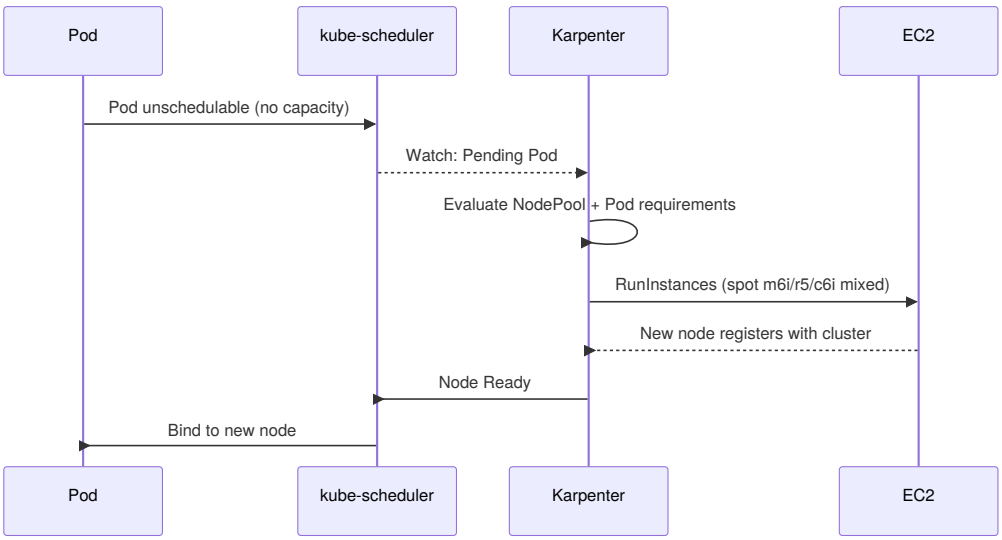


Diagram 2 — Fargate Pod networking

```
EKS control plane
     |
     v
Pod schedule -> Fargate Profile match (ns=serverless, label=tier=front)
     |
     v
Fargate provisions Firecracker micro-VM
     |
     +-- ENI in private subnet (Pod IP = ENI IP)
     +-- kubelet + containerd inside micro-VM
     +-- No host access (no DaemonSets)
     +-- Metrics via CloudWatch Container Insights (side-agent injected)
```

Diagram 3 — Karpenter provisioning loop



💻 Real-World Examples

Example 1 — Create EKS cluster + MNG with eksctl

```
# cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: shop
  region: us-east-1
  version: "1.31"
iam:
  withOIDC: true
managedNodeGroups:
  - name: mng-general
    instanceTypes: ["m6i.large", "m6a.large", "m5.large"]
    minSize: 2
    maxSize: 10
    desiredCapacity: 3
    volumeSize: 60
    labels: { role: general }
    tags: { team: platform }
    iam:
      withAddonPolicies:
        externalDNS: true
        certManager: true
  - name: mng-gpu
    instanceType: g4dn.xlarge
    minSize: 0
    maxSize: 5
    desiredCapacity: 0
    labels: { workload: ml }
    taints:
      - key: nvidia.com/gpu
        value: "true"
        effect: NoSchedule
```

```
eksctl create cluster -f cluster.yaml
```

Example 2 — Fargate Profile

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: shop
  region: us-east-1
fargateProfiles:
  - name: fp-serverless
    selectors:
      - namespace: serverless
      - namespace: kube-system
        labels: { k8s-app: kube-dns }
```

```
eksctl create fargateprofile -f cluster.yaml
```

Deploy a Pod:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: serverless
spec:
  replicas: 2
  selector: { matchLabels: { app: api } }
  template:
    metadata: { labels: { app: api } }
    spec:
      containers:
        - name: api
          image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/api:1.0
          resources:
            requests: { cpu: "500m", memory: "1Gi" }
            limits:   { cpu: "500m", memory: "1Gi" }
```

Example 3 — Karpenter NodePool + EC2NodeClass

```yaml
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: { name: default }
spec:
  template:
    spec:
      requirements:
        - key: kubernetes.io/arch
          operator: In
          values: ["amd64"]
        - key: karpenter.sh/capacity-type
          operator: In
          values: ["spot", "on-demand"]
        - key: karpenter.k8s.aws/instance-family
          operator: In
          values: ["m6i", "m6a", "m5", "c6i", "r6i"]
      nodeClassRef:
        group: karpenter.k8s.aws
        kind: EC2NodeClass
        name: default
      expireAfter: 720h
  disruption:
    consolidationPolicy: WhenUnderutilized
    expireAfter: 720h
---
apiVersion: karpenter.k8s.aws/v1
kind: EC2NodeClass
metadata: { name: default }
spec:
  amiFamily: Bottlerocket
  subnetSelectorTerms:
    - tags: { karpenter.sh/discovery: shop }
  securityGroupSelectorTerms:
    - tags: { karpenter.sh/discovery: shop }
  role: KarpenterNodeRole-shop
  tags:
    team: platform
```

💻 Related Commands

eksctl

Command	Flag	Purpose	Example
`eksctl create cluster`	`-f`	Cluster from config	`eksctl create cluster -f cluster.yaml`
`eksctl create nodegroup`	`--managed`	Add MNG	`eksctl create nodegroup --cluster=shop --managed --node-type=m6i.large`
`eksctl create fargateprofile`	`-f`	Add Fargate profile	See Ex. 2
`eksctl upgrade nodegroup`	`--kubernetes-version`	Upgrade MNG	`eksctl upgrade nodegroup --name=mng-general --cluster=shop --kubernetes-version=1.31`
`eksctl delete nodegroup`	`--wait`	Remove	`eksctl delete nodegroup --cluster=shop --name=mng-gpu --wait`
`eksctl utils update-cluster-vpc-config`	–	Toggle private endpoint	–

aws / kubectl

Command	Flag	Purpose	Example
`aws eks list-clusters`	`--region`	List clusters	`aws eks list-clusters --region us-east-1`
`aws eks describe-cluster`	`--name`	Cluster info	`aws eks describe-cluster --name shop`
`aws eks update-kubeconfig`	`--name`	Configure kubectl	`aws eks update-kubeconfig --name shop`
`aws eks list-fargate-profiles`	`--cluster-name`	List profiles	`aws eks list-fargate-profiles --cluster-name shop`
`aws eks list-nodegroups`	`--cluster-name`	List MNGs	–
`aws eks describe-addon-versions`	–	Available addons	–
`kubectl get nodes -L eks.amazonaws.com/compute-type`	–	Distinguish Fargate/EC2	–
`karpenter/kubectl get nodepools`	–	List Karpenter pools	–

🚨 Scenarios

Scenario 1 — Pod stuck Pending on Fargate - **Problem**: Namespace/label doesn't match any Fargate Profile. - **Resolution**: 1. `kubectl describe pod` → "no Fargate profile matches". 2. Add/adjust Fargate Profile selector. 3. Ensure resource requests fit Fargate sizing (0.25 vCPU / 0.5 GB → 4 vCPU / 30 GB).

Scenario 2 — MNG upgrade fails mid-rolling - **Problem**: New AMI missing a required kernel module for a DaemonSet. - **Resolution**: 1. Force rollback: `aws eks update-nodegroup-version --nodegroup-name mng --force`. 2. Pin AMI in launch template. 3. Use custom AMI or Bottlerocket for consistency.

Scenario 3 — Karpenter consolidates node with critical Pod - **Problem**: Pod evicted mid-request due to consolidation. - **Resolution**: 1. Add PDB (`minAvailable: 1`). 2. Set `karpenter.sh/do-not-disrupt: "true"` annotation for long-running jobs. 3. Use `consolidationPolicy: WhenEmpty` if drift is too aggressive.

☁ GCP GKE Perspective

- MNG ≈ GKE managed node pools.
- Fargate ≈ GKE Autopilot (but Autopilot supports DaemonSets, wider Pod variety).
- Karpenter ≈ GKE Node Auto-Provisioning.
- No direct hybrid analogue on GCP; GKE clusters can join a fleet with EKS Anywhere via Anthos on AWS.

▩ AWS EKS — Deep dive

- **Fargate Spot** — separate product (via AWS Batch); not a Fargate Profile flag.
- **EKS Pod Identity** — newer alternative to IRSA (association at the ServiceAccount level, no OIDC dependency in the pod).
- **Bottlerocket** — AWS's minimal, container-optimized OS. Recommended for Karpenter and MNG.

🧪 Hands-On Lab Exercises

🧪 **Lab 68-A — EKS Cluster with MNG + Fargate + Karpenter (Beginner–Intermediate)** - **Objective**: Cluster with all three compute options. - **Prerequisites**: aws CLI, eksctl, kubectl, helm, AWS account. - **Steps**: 1. `eksctl create cluster -f cluster.yaml` (Example 1 with `iam.withOIDC: true`). 2. `eksctl create fargateprofile -f cluster.yaml` (Example 2). 3. Install Karpenter via Helm (follow https://karpenter.sh/docs/getting-started/getting-started-with-karpenter/). 4. Apply NodePool + EC2NodeClass (Example 3). 5. Deploy 3 workloads: one to `serverless` ns (Fargate), one on `general` label (MNG), one large ScaleUp burst (Karpenter). - **Verification**:

```
kubectl get nodes -L eks.amazonaws.com/compute-type,karpenter.sh/nodepool
```

- **Cleanup**: `eksctl delete cluster --name shop` (deletes NGs, profiles, VPC). - **Troubleshooting**: Karpenter requires proper IAM role + SQS interruption queue. - **Bonus**: Add `topologySpreadConstraints` on Karpenter Pods across 3 AZs.

🧪 **Lab 68-B — Spot Diversification with Karpenter (Intermediate)** - **Objective**: Run 100 batch Pods across 6 spot instance families. - **Steps**: 1. Add a Karpenter NodePool restricted to spot with `instance-family: [m6i, m5, m6a, r6i, r5, c6i]`. 2. Deploy 100 Pods with `resources.requests.cpu: 250m`. 3. Karpenter picks the cheapest diverse pool. - **Verification**: `kubectl get nodes -L node.kubernetes.io/instance-type,karpenter.sh/capacity-type`. - **Bonus**: Simulate a spot interruption using the AWS FIS service.

🧪 **Lab 68-C — Fargate-only Cluster (Advanced)** - **Objective**: Run everything on Fargate including CoreDNS. - **Steps**: 1. Create EKS cluster with no node group. 2. Create Fargate Profile matching `default` and `kube-system` (with `k8s-app: kube-dns` selector). 3. Patch CoreDNS to remove EC2 restriction annotations. 4. Deploy an app. - **Verification**: All Pods running on Fargate; `kubectl get nodes` shows `fargate-*` names. - **Bonus**: Add an ALB Ingress and note that AWS Load Balancer Controller also runs on Fargate.

🎯 Interview Deliverable Answer (60–90s)

> "EKS gives me three compute models plus an increasingly popular fourth. **Managed Node Groups** are AWS-managed EC2 with rolling upgrades built in — my default for steady workloads. **Self-Managed** is EC2 I bootstrap myself when I need custom AMIs, exotic instance types, or hybrid nodes. **Fargate** is per-Pod Firecracker micro-VMs — no nodes, priced per Pod-second, but with strict limits: no DaemonSets, no privileged, no hostPath. It's my choice for bursty, low-density, security-sensitive workloads. **Karpenter** — the modern option — watches unschedulable Pods and provisions raw EC2 in ~40 seconds across a diverse spot pool, then consolidates. In one migration we replaced Cluster Autoscaler + MNG with Karpenter and cut compute cost about 40% by running 80% spot with instance-family diversity. Rule of thumb: MNG for stable baseline, Karpenter for elasticity, Fargate for compliance/burst, self-managed only when you truly need it."

? Interview Questions

Basic 1. **Q**: Three EKS compute models? **A**: Managed Node Groups, Self-Managed nodes, Fargate.

1. **Q**: What is Fargate? **A**: Per-Pod serverless compute using Firecracker micro-VMs; no EC2 visibility.

Intermediate 3. **Q**: What can't you do on Fargate? **A**: DaemonSets, privileged Pods, hostPath, hostNetwork, hostPort, Windows.

1. **Q**: How does Karpenter differ from Cluster Autoscaler + MNG? **A**: Provisions raw EC2 (not an ASG), faster scale-up, better instance diversity, built-in consolidation.
2. **Q**: What's Bottlerocket? **A**: AWS's minimal, container-optimized OS designed for K8s and Karpenter.

Advanced 6. **Q**: How is IAM for nodes managed? **A**: MNG uses a per-nodegroup IAM role; Karpenter uses a launch template role; Fargate uses a Pod execution role.

1. **Q**: What is EKS Pod Identity? **A**: A newer alternative to IRSA; SA↔IAM association without OIDC-in-pod. Simpler for cross-account and multi-cluster.

Scenario 8. **Q**: Migrating from CA+MNG to Karpenter — steps? **A**: Install Karpenter, add NodePool+EC2NodeClass, drain and cordon MNG nodes gradually, verify no scale-critical DaemonSet dependencies, delete MNG.

GKE-specific 9. **Q**: What's the GKE analogue of Karpenter? **A**: Node Auto-Provisioning (Standard) or Autopilot end-to-end.

EKS-specific 10. **Q**: How do you get spot on Fargate? **A**: Fargate Spot via AWS Batch — not a Fargate Profile flag; regular Fargate is on-demand only.

Lab-based 11. **Q**: Walk me through deploying a workload only on Fargate. **A**: (Lab 68-C) Create Fargate Profile matching namespace + labels; deploy; verify `kubectl get nodes` shows `fargate-*`.

🗄 Topic 69 — GKE Workload Identity vs EKS IRSA (& EKS Pod Identity)

Official Documentation References - https://cloud.google.com/kubernetes-engine/docs/how-to/workload-identity - https://cloud.google.com/kubernetes-engine/fleet-management/docs/use-workload-identity - https://docs.aws.amazon.com/eks/latest/userguide/iam-roles-for-service-accounts.html - https://docs.aws.amazon.com/eks/latest/userguide/pod-identities.html

🗄 Definition

GKE Workload Identity and **EKS IRSA (IAM Roles for Service Accounts)** let Kubernetes ServiceAccounts assume cloud IAM identities so Pods can call cloud APIs without long-lived credentials. **EKS Pod Identity** is AWS's newer, simpler mechanism that replaces IRSA's OIDC dance with an EKS-side association.

📖 About

The problem - Pods need cloud API access (write to S3/GCS, publish to Pub/Sub/SQS, read Secret Manager). - Storing long-lived access keys in Secrets is risky and hard to rotate. - Instance-role fallback (node IAM role) gives *all* Pods on a node the same permissions — violates least privilege.

The solution — per-workload cloud identity - Kubernetes ServiceAccount ↔ Cloud IAM identity. - Pods automatically get short-lived credentials scoped to that SA.

GKE Workload Identity (WLI) - Enable at cluster level: `--workload-pool=<project>.svc.id.goog`. - Kubernetes SA is annotated with a GCP Service Account email: `iam.gke.io/gcp-service-account: my-gsa@my-project.iam.gserviceaccount.com`. - The GCP SA gets IAM binding `roles/iam.workloadIdentityUser` for the

Kubernetes SA identity: `serviceAccount:<project>.svc.id.goog[<ns>/<ksa>]` . - Pods use the SA; the GKE metadata server intercepts calls to metadata IP `169.254.169.254` , returns short-lived tokens tied to the GCP SA. - On Autopilot, mandatory.

Fleet Workload Identity — extension for multi-cluster where all clusters in a fleet share a trust boundary.

EKS IRSA - Cluster must have an OIDC provider registered in IAM. - Kubernetes SA annotated: `eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/my-role` . - IAM role's trust policy trusts the cluster's OIDC provider with a `sub` condition matching `system:serviceaccount:<ns>:<sa>` . - The **mutating webhook** (deployed on control plane) projects a signed JWT into the Pod at `/var/run/secrets/eks.amazonaws.com/serviceaccount/token` . - AWS SDKs (`AWS_WEB_IDENTITY_TOKEN_FILE` env) call `sts:AssumeRoleWithWebIdentity` to get temporary creds.

EKS Pod Identity (newer, 2023+) - No OIDC needed in Pod path. - You create an **EKS Pod Identity Association** linking a cluster + namespace + SA → IAM role. - An `eks-pod-identity-agent` DaemonSet (or Fargate injection) issues credentials via a local endpoint. - Simpler cross-account and multi-cluster (single trust policy per role, not one-per-cluster-OIDC).

Comparison

Aspect	GKE WLI	EKS IRSA	EKS Pod Identity
Setup	Enable workload-pool + SA annotation + IAM binding	Register OIDC + create IAM role with trust policy + SA annotation	Install agent + create Pod Identity Association
Trust mechanism	Kubernetes SA identity via GKE metadata server	OIDC JWT → `AssumeRoleWithWebIdentity`	EKS-signed credentials via agent
Cross-account	Yes via impersonation	Yes	Yes (simplest)
Multi-cluster	Fleet WLI	Per-cluster OIDC	Single role trust policy per cluster set
SDK requirement	None (metadata IP)	AWS SDK v1/v2 with WebIdentity support	Standard SDK (env-based)
Fargate	Yes (WLI on Autopilot)	Yes	Yes

🔗 How It Integrates With Other Components

Component	Integration
Kubernetes SA	Anchor of identity
Cloud IAM	Backing role / GCP SA
kube-apiserver	Issues projected SA tokens
Cloud metadata server	(GKE) proxies token requests
Mutating webhook	(EKS IRSA) injects env + volume
eks-pod-identity-agent	(EKS PI) local credential provider
Cloud SDKs	Consumers of the credentials
Audit logs	Cloud IAM logs show which SA identity called

🏗 Architecture Diagrams

Diagram 1 — GKE Workload Identity flow

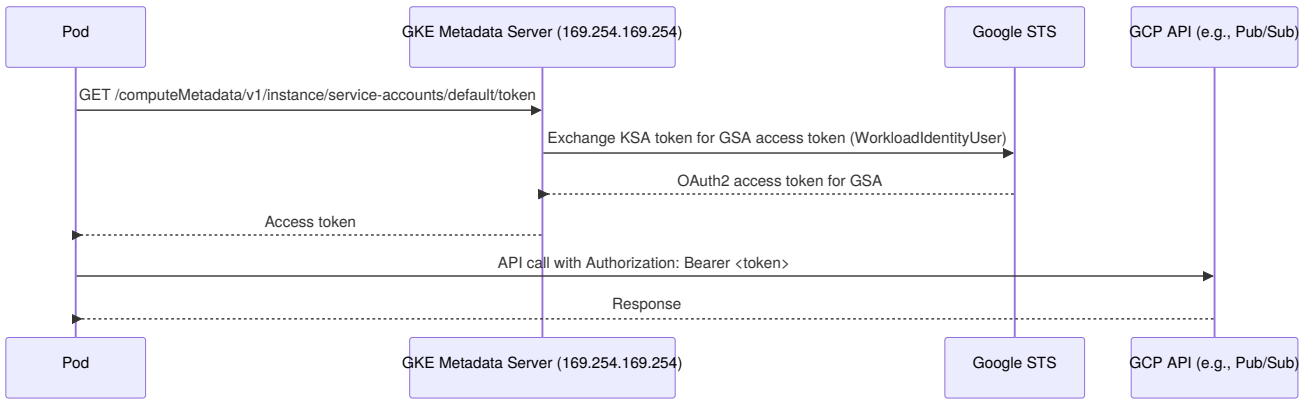


Diagram 2 — EKS IRSA flow

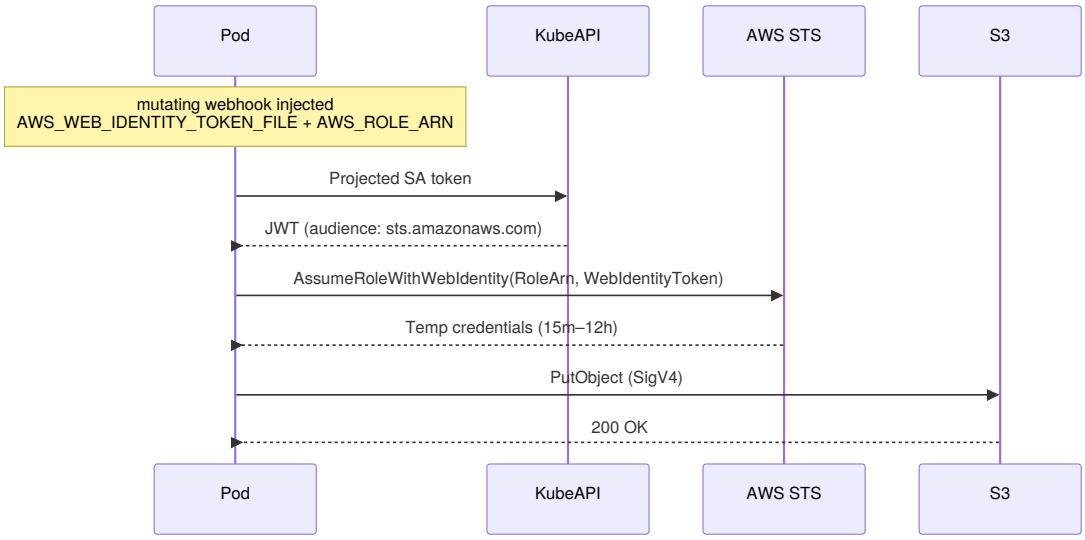
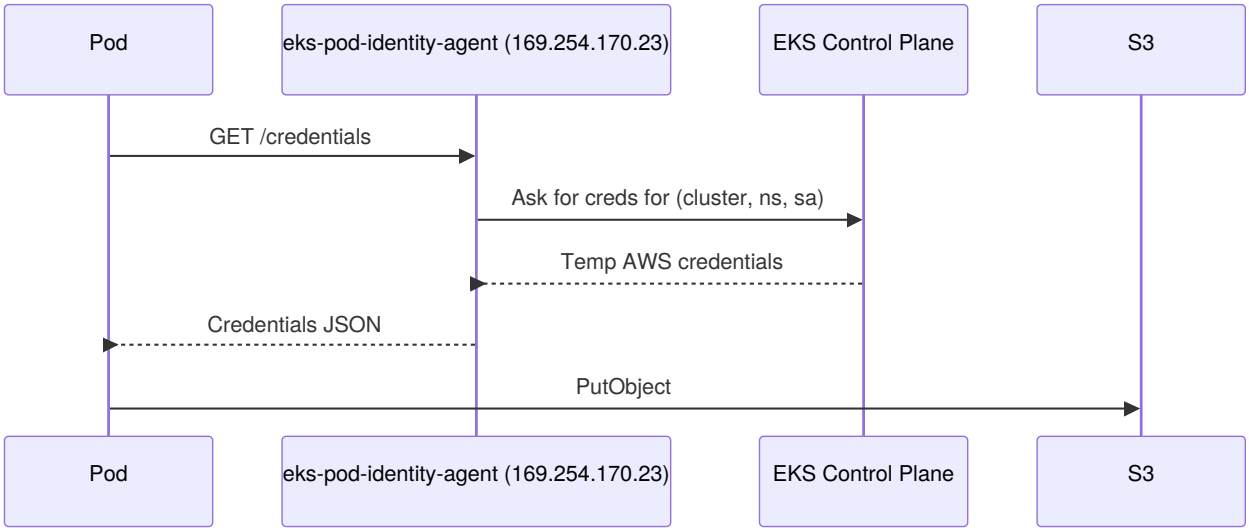


Diagram 3 — EKS Pod Identity flow



💻 Real-World Examples

Example 1 — GKE Workload Identity setup

```
# Enable on cluster
gcloud container clusters update shop-std \
  --workload-pool=my-project.svc.id.goog --region=us-central1

# Enable on node pool (Standard only)
gcloud container node-pools update default-pool --cluster=shop-std \
  --region=us-central1 --workload-metadata=GKE_METADATA

# Create GCP SA
gcloud iam service-accounts create shop-writer \
  --display-name="Shop writer"

# Grant GCP permissions
gcloud projects add-iam-policy-binding my-project \
  --member="serviceAccount:shop-writer@my-project.iam.gserviceaccount.com" \
  --role="roles/pubsub.publisher"

# Bind KSA to GSA
gcloud iam service-accounts add-iam-policy-binding \
  shop-writer@my-project.iam.gserviceaccount.com \
  --role=roles/iam.workloadIdentityUser \
  --member="serviceAccount:my-project.svc.id.goog[shop/shop-ksa]"

# Kubernetes SA
kubectl create ns shop
kubectl create serviceaccount shop-ksa -n shop
kubectl annotate serviceaccount shop-ksa -n shop \
  iam.gke.io/gcp-service-account=shop-writer@my-project.iam.gserviceaccount.com
```

Pod uses it:

```
apiVersion: v1
kind: Pod
metadata:
  name: publisher
  namespace: shop
spec:
  serviceAccountName: shop-ksa
  containers:
    - name: app
      image: gcr.io/google.com/cloudsdktool/cloud-sdk:slim
      command: ["gcloud", "pubsub", "topics", "publish", "orders", "--message=hi"]
```

Example 2 — EKS IRSA with eksctl

```
# Create IAM Role + SA in one command
eksctl create iamserviceaccount \
  --cluster=shop \
  --namespace=shop \
  --name=shop-ksa \
  --attach-policy-arn=arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess \
  --approve
```

Under the hood: creates IAM role, trust policy for OIDC, SA in cluster with annotation `eks.amazonaws.com/role-arn`.

Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: reader
  namespace: shop
spec:
  serviceAccountName: shop-ksa
  containers:
    - name: aws
      image: amazon/aws-cli
      command: ["aws", "s3", "ls"]
```

Example 3 — EKS Pod Identity Association

```
# Install add-on
aws eks create-addon --cluster-name shop --addon-name eks-pod-identity-agent

# Trust policy: pods.eks.amazonaws.com
cat > trust.json <<EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "pods.eks.amazonaws.com" },
      "Action": ["sts:AssumeRole","sts:TagSession"]
    }
  ]
}
EOF
aws iam create-role --role-name shop-role --assume-role-policy-document file://trust.json
aws iam attach-role-policy --role-name shop-role \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess

# Associate
aws eks create-pod-identity-association \
  --cluster-name shop --namespace shop --service-account shop-ksa \
  --role-arn arn:aws:iam::123456789012:role/shop-role
```

💻 Related Commands

GKE WLI

Command	Flag	Purpose	Example
`gcloud container clusters update`	`--workload-pool`	Enable WLI	See Ex.1
`gcloud container node-pools update`	`--workload-metadata=GKE_METADATA`	Enable on pool	See Ex.1
`gcloud iam service-accounts add-iam-policy-binding`	`--role=roles/iam.workloadIdentityUser`	Bind KSA↔GSA	See Ex.1
`kubectl annotate sa`	`iam.gke.io/gcp-service-account=...`	KSA annotation	See Ex.1
`gcloud container fleet workload-identity update`	–	Fleet WLI	–

EKS IRSA

Command	Flag	Purpose	Example
`eksctl utils associate-iam-oidc-provider`	`--approve`	Register OIDC	`eksctl utils associate-iam-oidc-provider --cluster=shop --approve`
`eksctl create iamserviceaccount`	`--attach-policy-arn`	IAM SA in one shot	See Ex.2
`aws iam create-role`	`--assume-role-policy-document`	Manual IAM role	–
`aws eks describe-cluster`	`--query 'cluster.identity.oidc.issuer'`	Find OIDC URL	–

EKS Pod Identity

Command	Flag	Purpose	Example
`aws eks create-addon`	`--addon-name eks-pod-identity-agent`	Install agent	See Ex.3
`aws eks create-pod-identity-association`	`--namespace --service-account --role-arn`	Bind	See Ex.3
`aws eks list-pod-identity-associations`	`--cluster-name`	Audit	–

🛎 Scenarios

Scenario 1 — Pod gets 403 calling cloud API despite annotations - **GKE**: `--workload-pool` not enabled cluster-wide, or pool doesn't have `GKE_METADATA`. - **EKS IRSA**: Trust policy `sub` doesn't match `system:serviceaccount:<ns>:<sa>`, or Pod doesn't use the SA (`serviceAccountName:` missing). - **EKS PI**: Association missing; check `aws eks list-pod-identity-associations`.

Scenario 2 — Cross-account access to a bucket - **EKS IRSA**: Add trust between IAM role in account B and IAM role in account A; Pod SA assumes A's role, which then assumes B's role (or a bucket policy in B trusts A's role). - **GKE WLI**: Cross-project — grant GCP SA in project A `roles/storage.objectViewer` on bucket in project B (bucket-level IAM).

Scenario 3 — Rotating an IAM role/policy - Change policy, wait ~15 min for STS credential expiry, or force Pod restart. IRSA credentials cache for up to 1 hour by default.

☁ GCP GKE — deep-dive notes

- **Autopilot** clusters have WLI mandatory.
- **Fleet Workload Identity** allows one GCP SA binding to serve many clusters in a fleet using `<fleet-project>.svc.id.goog[<ns>/<ksa>]`.
- Impersonate a GCP SA from a KSA using `gcloud auth application-default print-access-token` inside the Pod for CLI-based access.

▩ AWS EKS — deep-dive notes

- IRSA works on both MNG and Fargate.
- **EKS Pod Identity** is preferred for new setups (simpler, no OIDC in Pod), but IRSA remains widely used.
- **Session tags** via `sts:TagSession` allow cross-account attribute-based access control (ABAC).

🧪 Hands-On Lab Exercises

🧪 **Lab 69-A — GKE WLI Publishing to Pub/Sub (Beginner)** - **Objective**: Pod publishes messages using WLI. - **Prerequisites**: GKE Standard cluster with `--workload-pool`. - **Steps**: Follow Example 1 end-to-end. - **Verification**: `gcloud pubsub subscriptions pull` sees the message. - **Cleanup**: Delete cluster + GSA + IAM binding. - **Bonus**: Deny node metadata for other Pods to prove per-SA isolation.

🧪 **Lab 69-B — EKS IRSA Reading S3 (Intermediate)** - **Objective**: Pod lists an S3 bucket without keys. - **Steps**: Follow Example 2. - **Verification**: Pod's `aws s3 ls` succeeds. - **Cleanup**: `eksctl delete iamserviceaccount --cluster=shop --name=shop-ksa --namespace=shop`. - **Bonus**: Change SA and observe access denied.

🧪 **Lab 69-C — Migrate from IRSA to EKS Pod Identity (Advanced)** - **Objective**: Zero-downtime migration. - **Steps**: 1. Existing IRSA config. 2. Install eks-pod-identity-agent add-on. 3. Create new IAM role with `pods.eks.amazonaws.com` trust. 4. Add Pod Identity Association for same ns/sa. 5. Rollout restart; SDK now uses agent (Pod Identity precedence over IRSA env if agent creds present). 6. Remove IRSA annotation and OIDC-trust IAM role. - **Verification**: Pods still function; CloudTrail shows role assumed via `pods.eks.amazonaws.com`. - **Bonus**: Do a cross-account variant.

🎯 Interview Deliverable Answer (60–90s)

> "Both clouds solve the same problem: Pods need cloud API access without long-lived keys. **GKE Workload Identity** binds a Kubernetes ServiceAccount to a GCP Service Account — the GKE metadata server intercepts token requests and hands back short-lived GCP credentials scoped to that SA. **EKS IRSA** uses OIDC: the cluster is registered as an OIDC provider in IAM, a mutating webhook injects a projected JWT into Pods, and the AWS SDK calls `AssumeRoleWithWebIdentity` to swap it for temp creds. **EKS Pod Identity** is the newer, simpler alternative — you just create an association via the EKS API, and an agent DaemonSet serves creds locally. Why it matters: no keys in Secrets, per-workload least privilege, and clean audit trails in Cloud IAM / CloudTrail. Whenever I stand up a new cluster, WLI is on from day one on GKE — mandatory on Autopilot — and on EKS I default to Pod Identity for new services and IRSA for legacy."

? Interview Questions

Basic 1. **Q**: Why not store AWS/GCP keys in a Secret? **A**: Long-lived, hard to rotate, blast-radius on leak; violates least privilege because everyone in ns can read.

1. **Q**: What does IRSA stand for? **A**: IAM Roles for Service Accounts.

Intermediate 3. **Q**: How does the Pod get creds under IRSA? **A**: Mutating webhook injects a projected token file + env var; SDK exchanges via `AssumeRoleWithWebIdentity`.

1. **Q**: How does WLI hand out creds? **A**: GKE metadata server on the node exchanges the KSA identity for a GCP SA token via Workload Identity Federation.
2. **Q**: Trade-offs of Pod Identity vs IRSA? **A**: PI: simpler trust policies, no OIDC-in-pod, easier cross-account. IRSA: broader tooling maturity.

Advanced 6. **Q**: How would you audit which SA called `s3:PutObject`? **A**: CloudTrail `userIdentity.sessionContext.sessionIssuer` shows the assumed role; `sub` in the token maps to the K8s SA.

1. **Q**: How would you rotate the OIDC signing key? **A**: EKS rotates it automatically; keep the OIDC thumbprint updated in IAM provider.

Scenario 8. **Q**: SDK reports "no credentials" — debug on EKS. **A**: Check SA annotation, `eks.amazonaws.com/role-arn`, projected token file exists, AWS SDK version supports WebIdentity, trust policy `sub` matches.

GKE-specific 9. **Q**: What is Fleet Workload Identity? **A**: A single trust boundary across a fleet so KSA→GSA bindings use a fleet-scoped principal, avoiding per-cluster wiring.

EKS-specific 10. **Q**: How does EKS Pod Identity work on Fargate? **A**: Fargate injects the agent-equivalent; SDK still reads creds via the local endpoint URL.

Lab-based 11. **Q**: Show me giving a Pod S3 read-only access on EKS. **A**: (Lab 69-B) `eksctl create iamserviceaccount` with `AmazonS3ReadOnlyAccess`, reference SA in Pod, aws s3 ls.

🗄 Topic 70 — Upgrades and Version Lifecycle on GKE and EKS

Official Documentation References - https://kubernetes.io/releases/version-skew-policy/ - https://cloud.google.com/kubernetes-engine/versioning - https://cloud.google.com/kubernetes-engine/docs/concepts/release-channels - https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html - https://docs.aws.amazon.com/eks/latest/userguide/update-cluster.html

📄 Definition

Upgrade lifecycle management is the practice of moving a Kubernetes cluster through minor versions (1.30 → 1.31 → 1.32) while respecting version-skew rules and keeping workloads available. Both GKE and EKS offer managed control-plane upgrades and node-fleet upgrade tooling.

📖 About

Kubernetes upstream cadence - Minor releases: every ~4 months. - Support: ~14 months of patch support per minor version (as of 2024+; historically 12). - Version skew: `kubelet` ≤ 3 minors behind apiserver (v1.28+); `kube-proxy` same node; managed add-ons per docs.

Order of operations for a cluster upgrade 1. Control plane (apiserver, scheduler, controller-manager) — must go first. 2. Nodes (rolling with drain). 3. Add-ons (CoreDNS, kube-proxy, CNI, CSI, metrics-server).

Pre-upgrade checklist - Check deprecated API usage (`kubectl deprecations` plugin, `pluto` , `kubent` , GKE Insights). - Test in a staging cluster with your workloads and CI. - Verify PDBs so drain doesn't starve availability. - Take a snapshot / backup. - Note any add-on compatibility (Karpenter, Istio, cert-manager). - Bump CRDs bundled with the version. - Ensure `PriorityClass` and Cluster Autoscaler / Karpenter aren't set to disruption-block windows.

GKE-specific - **Release channels**: `rapid` (early), `regular` (default), `stable` (latest patch of an older minor). Enrolling a cluster ties both control plane and nodes to that channel's cadence. - **Auto-upgrades**: on by default in channels; Google chooses maintenance windows. - **Maintenance windows & exclusions**: define allowed upgrade times and blackout dates (up to 30 days). - **Surge upgrades**: node upgrades use configurable `max-surge` and `max-unavailable` . - **Blue-green upgrades** (Standard): create a new node pool at the target version, migrate workloads, decommission old — safer than in-place surges. - **Extended Support**: pay-per-hour to remain on a minor after standard support ends.

EKS-specific - **Cluster upgrade** is initiated per minor step (no skipping): `1.29 → 1.30 → 1.31` . - **Add-ons upgrade** via EKS Add-on API (VPC CNI, CoreDNS, kube-proxy, EBS CSI, EFS CSI, ADOT, Pod Identity Agent). - **Managed Node Group upgrades** — AWS handles cordon/drain/replace across ASG. - **Karpenter/self-managed** — you drift-terminate nodes; Karpenter honors `NodePool.disruption.expireAfter` . - **Extended Support** — pay-per-cluster-hour surcharge; caps at ~12 months of extended support. - **Version deprecation calendar** — AWS publishes end-of-standard-support and end-of-extended-support dates.

Comparison

Aspect	GKE	EKS
Auto-upgrade	Yes (channels)	Manual (per minor step)
Skip versions	No	No
Maintenance windows	Native	Use blueprints / Karpenter drift
Node upgrade	Surge or blue-green	MNG rolling; self-managed via ASG
Extended Support	Paid hourly	Paid hourly per cluster
CRD-based upgrade config	ReleaseChannel via API	UpdateClusterVersion API

🔗 How It Integrates With Other Components

Component	Integration
API deprecations	Blocked by version bump; test with `kubent`
PDBs	Prevent excessive disruption during drain
Cluster Autoscaler / Karpenter	Must be at compatible version
Ingress controllers	Deployed as workloads; may pin K8s API dependencies
Service Mesh (Istio)	Coupled to Kubernetes minor windows
Cloud CNI (VPC CNI, GKE Dataplane V2)	Version-tied add-ons
Storage CSI	Bug-fixes shipped per K8s version
Observability (metrics-server)	Version-tied

🏗 Architecture Diagrams

Diagram 1 — Upgrade order

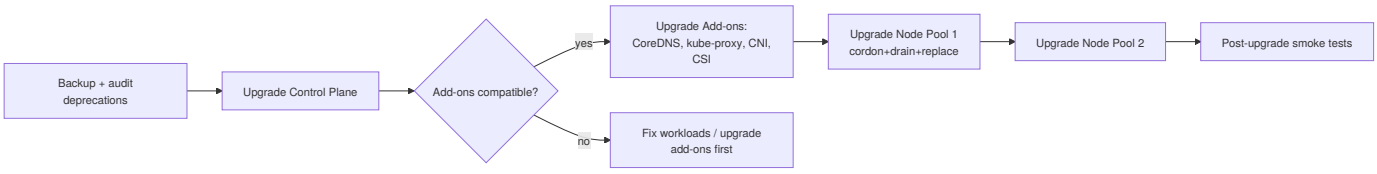
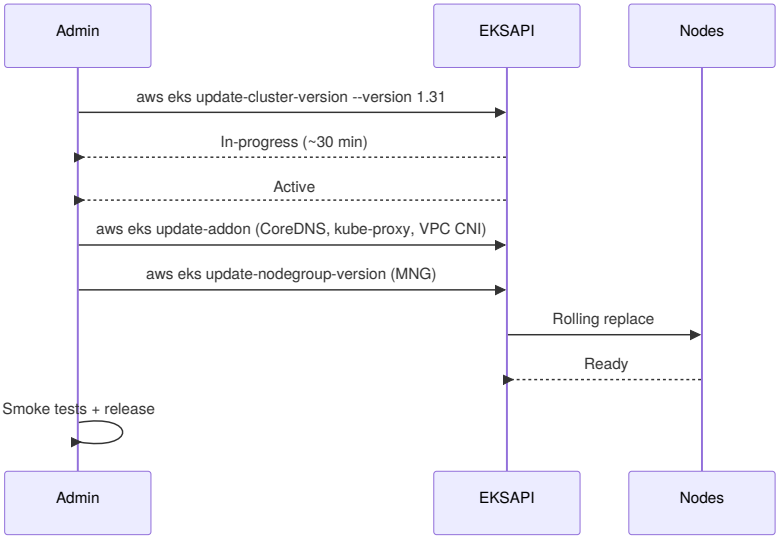


Diagram 2 — GKE surge vs blue-green

```
Surge (in-place):
+---------------+   +---------------+
|  Old pool     |   |  Old pool     |
|  4 nodes v1.30|-->|  3 nodes v1.30| ---> ...
|               |   |  1 node v1.31 |
+---------------+   +---------------+

Blue-green:
[ Old pool v1.30  Running workloads ]
        +
[ New pool v1.31  Empty  ]  --> shift workloads --> [ Old pool empty ] --> delete
```

Diagram 3 — EKS upgrade sequence



💻 Real-World Examples

Example 1 — GKE Standard upgrade with maintenance window

```
# Set release channel + maintenance window
gcloud container clusters update shop \
  --release-channel=regular \
  --maintenance-window-start=2026-01-01T09:00:00Z \
  --maintenance-window-end=2026-01-01T14:00:00Z \
  --maintenance-window-recurrence=FREQ=WEEKLY;BYDAY=SA \
  --region=us-central1

# Trigger control-plane upgrade (or wait for auto)
gcloud container clusters upgrade shop --master \
  --cluster-version=1.31.1-gke.1846000 --region=us-central1

# Upgrade node pool (blue-green style)
gcloud container node-pools update default-pool \
  --cluster=shop --region=us-central1 \
  --enable-blue-green-upgrade \
  --standard-rollout-policy=batch-node-count=3,batch-soak-duration=60s \
  --node-pool-soak-duration=1800s

gcloud container clusters upgrade shop \
  --node-pool=default-pool --cluster-version=1.31.1-gke.1846000 \
  --region=us-central1
```

Example 2 — EKS upgrade with eksctl

```
# Check current
eksctl get cluster --name shop

# Upgrade control plane
eksctl upgrade cluster --name shop --version 1.31 --approve

# Upgrade add-ons
for addon in vpc-cni coredns kube-proxy aws-ebs-csi-driver; do
  aws eks update-addon --cluster-name shop --addon-name $addon \
    --resolve-conflicts PRESERVE
done

# Upgrade MNGs
eksctl upgrade nodegroup --cluster=shop --name=mng-general --kubernetes-version=1.31
```

Example 3 — Audit deprecated APIs before upgrading

```
# Using kubent
kubent --context shop-prod

# Using pluto
pluto detect-helm --output json > deprecations.json

# Using kubectl deprecations plugin
kubectl deprecations
```

💻 Related Commands

GKE

Command	Flag	Purpose	Example
`gcloud container get-server-config`	`--region`	Available versions	`gcloud container get-server-config --region us-central1`
`gcloud container clusters update`	`--release-channel`	Enroll channel	See Ex.1
`gcloud container clusters update`	`--maintenance-window-*`	Windows/exclusions	See Ex.1
`gcloud container clusters upgrade`	`--master`	Control plane	See Ex.1
`gcloud container clusters upgrade`	`--node-pool`	Node pool	See Ex.1
`gcloud container node-pools update`	`--enable-blue-green-upgrade`	Blue-green	See Ex.1

EKS

Command	Flag	Purpose	Example
`aws eks describe-cluster`	`--query 'cluster.version'`	Current version	–
`aws eks update-cluster-version`	`--version`	Control plane	`aws eks update-cluster-version --name shop --version 1.31`
`aws eks list-updates`	`--name`	Track updates	`aws eks list-updates --name shop`
`aws eks describe-update`	`--name --update-id`	Progress	–
`aws eks update-addon`	`--resolve-conflicts PRESERVE`	Add-on upgrade	See Ex.2
`eksctl upgrade cluster`	`--version --approve`	eksctl wrapper	See Ex.2
`eksctl upgrade nodegroup`	`--kubernetes-version`	MNG upgrade	See Ex.2

Common audit tools

Command	Purpose	Example
`kubent`	Deprecated API scan	`kubent`
`pluto`	Chart deprecation	`pluto detect-helm`
`kubectl deprecations`	Plugin scanner	`kubectl deprecations`
`kubectl api-versions`	Available versions	–
`kubectl api-resources`	–	–

Scenarios

Scenario 1 — Ingress broken after upgrade due to removed API - **Problem**: `networking.k8s.io/v1beta1 Ingress` removed in 1.22; upgrade fails or Ingress objects don't work. - **Resolution**: 1. Pre-scan with kubent. 2. Migrate manifests to `networking.k8s.io/v1`. 3. Update Helm charts and re-render. 4. GKE/EKS may proactively block the upgrade with warnings.

Scenario 2 — PDB starves drain - **Problem**: `minAvailable: 100%` PDB blocks node drain forever. - **Resolution**: 1. Loosen to `maxUnavailable: 25%`. 2. On GKE, `--pdb-timeout-seconds` on the node pool (blue-green) will eventually force-drain. 3. On EKS MNG, `--force` on `update-nodegroup-version`.

Scenario 3 — Karpenter nodes stuck at old version during upgrade - **Problem**: Karpenter's `EC2NodeClass.amiSelectorTerms` pinned to old AMI. - **Resolution**: 1. Update `amiSelectorTerms` to `alias: bottlerocket@latest` or a newer AMI ID. 2. Karpenter drifts nodes and replaces them. 3. Add `disruption.expireAfter: 720h` so nodes rotate at least monthly.

GCP GKE Perspective (deep dive)

- **Insights** in Cloud Console proactively flags deprecated APIs.
- **Extended Support** billed hourly; use only during migration windows.
- **Node Auto-Upgrade** aligns with channel; disable during freeze weeks via maintenance exclusions.
- **Ingress-GCE + NEGs** — controller upgrades handled by GKE; no user action.

AWS EKS Perspective (deep dive)

- **Version support**: each minor supported for 14 months of standard, then extended.
- **Add-on version pinning** via `configurationValues` block for each add-on.
- **Karpenter drift** is the recommended way to roll node AMIs post-upgrade.
- **CloudTrail** records `UpdateClusterVersion` and `UpdateNodegroupVersion`.

Hands-On Lab Exercises

Lab 70-A — Deprecated API Audit + GKE Channel Upgrade (Beginner) - **Objective**: Detect deprecations and upgrade a Standard cluster. - **Steps**: 1. Create GKE Standard cluster at `1.30`. 2. Deploy sample workload with an intentionally old API version (e.g., `autoscaling/v2beta2`). 3. Run `kubent` → note detections. 4. Migrate manifests. 5. Trigger `gcloud container clusters upgrade`. - **Verification**: `gcloud container clusters describe` shows new version. - **Cleanup**: Delete cluster. - **Bonus**: Set a maintenance exclusion for the next weekend.

Lab 70-B — EKS Blue/Green Node Upgrade with Karpenter (Intermediate) - **Objective**: Upgrade control plane and rotate all Karpenter nodes. - **Steps**: 1. Cluster at 1.30; deploy Karpenter with `EC2NodeClass` pinned to Bottlerocket 1.30. 2. `aws eks update-cluster-version --version 1.31`. 3. Wait for `ACTIVE`. 4. `aws eks update-addon` for CoreDNS, VPC CNI, kube-proxy. 5. Update EC2NodeClass to Bottlerocket 1.31 AMI alias. 6. Watch Karpenter drift nodes. - **Verification**: `kubectl get nodes` shows `v1.31.x`. - **Bonus**: Add a PDB and observe safe rollout.

Lab 70-C — Two-step upgrade (1.29 → 1.30 → 1.31) with rollback plan (Advanced) - **Objective**: Practice multi-minor upgrade with staging validation. - **Steps**: 1. Two EKS clusters (staging + prod) at 1.29. 2. Upgrade staging first; run full smoke suite (`hey`, Postman). 3. Repeat on prod with maintenance window. 4. Prepare rollback plan: not possible in-place — use Velero restore to a 1.29 cluster. - **Verification**: Prod on 1.31; DR plan documented. - **Bonus**: Automate with GitHub Actions or Argo Workflows.

Interview Deliverable Answer (60–90s)

> "Kubernetes ships a new minor every ~4 months and supports each for 14 months, so upgrade discipline is a continuous job, not a project. My process: **audit** for deprecated APIs with kubent, **test** in staging, **backup**, then upgrade in order — control plane first, then add-ons (CoreDNS, kube-proxy, CNI, CSI), then nodes with drain-respecting PDBs. On GKE I enroll the cluster in the `regular` release channel and set maintenance windows and exclusions; for risky rolls I use blue-green node pool upgrades so I can pause and validate. On EKS I upgrade one minor at a time via `aws eks update-cluster-version`, then update add-ons with `--resolve-conflicts PRESERVE`, then MNGs or Karpenter drift. The two things that bite people: PDBs blocking drain and deprecated APIs breaking Helm charts. Both are solved by pre-flight scans and a proper staging cluster."

? Interview Questions

Basic 1. **Q**: Upstream Kubernetes release cadence? **A**: ~4 months per minor; ~14 months support.

1. **Q**: Order of upgrade? **A**: Control plane → add-ons → nodes.

Intermediate 3. **Q**: What are GKE release channels? **A**: `rapid`, `regular`, `stable` — pin cadence and version selection.

1. **Q**: Can you skip minors on EKS? **A**: No — one minor at a time.
2. **Q**: How do you find deprecated APIs before upgrading? **A**: `kubent`, `pluto`, GKE Insights, EKS Upgrade Insights.

Advanced 6. **Q**: What is blue-green node pool upgrade on GKE? **A**: Create a new node pool at target version, migrate workloads, then delete old pool.

1. **Q**: How does Karpenter handle version upgrades? **A**: Update the EC2NodeClass AMI reference; Karpenter drifts and replaces nodes based on disruption policy.

Scenario 8. **Q**: PDB set to 100% blocks upgrade — options? **A**: Loosen PDB, use `--force` (EKS MNG), blue-green (GKE), or manually adjust replicas.

GKE-specific 9. **Q**: How to avoid weekend upgrades? **A**: Maintenance exclusion for that date range in cluster config.

EKS-specific 10. **Q**: What does `--resolve-conflicts PRESERVE` do on add-on update? **A**: Keeps your custom values while applying the new add-on version.

Lab-based 11. **Q**: Walk me through a safe 1.30 → 1.31 upgrade on EKS. **A**: (Lab 70-B) audit APIs → upgrade CP → upgrade add-ons → rotate nodes via Karpenter drift → validate with smoke tests.

⏸ Pause Point — End of Batch 3 of 5

Completed in this batch (Topics 67–70) — Module 11 done: - ✅ GKE Standard vs Autopilot — full comparison, compute classes, decision tree - ✅ EKS MNG vs Self-Managed vs Fargate vs Karpenter — full comparison + Karpenter deep dive - ✅ GKE Workload Identity vs EKS IRSA vs EKS Pod Identity — flows, examples, migration - ✅ Upgrade & Version Lifecycle — GKE channels + blue-green, EKS step upgrades + Karpenter drift

Remaining: - **Batch 4 (Topics 71–73):** Interview-only content — 50 troubleshooting scenarios, 30 system design questions, behavioral + K8s hybrids - **Batch 5 (Capstones 1–10):** End-to-end labs - **Final:** PDF compilation

Say **"continue"** to proceed with Batch 4 (Topics 71–73 — this batch will be large; the troubleshooting catalog alone runs ~30 pages).

MODULE

Module 12 — Interview-Only Content

Module 12 — Interview-Only Content

Topics 71 – 73

🗐 Topic 71 — Top 50 Real-World Kubernetes Troubleshooting Scenarios

Reference material - https://kubernetes.io/docs/tasks/debug/ - https://kubernetes.io/docs/tasks/debug/debug-application/ - https://kubernetes.io/docs/tasks/debug/debug-cluster/ - https://cloud.google.com/kubernetes-engine/docs/troubleshooting - https://docs.aws.amazon.com/eks/latest/userguide/troubleshooting.html

Each scenario follows: **Symptoms → Root causes → Diagnostic commands → Resolution → Prevention.**

🚨 1. Pod stuck in `Pending`

Symptoms: `kubectl get pods` shows `Pending`; no container starts.

Common causes - Insufficient node capacity (CPU/mem/GPU). - PVC not bound. - No node matches nodeSelector / taints / affinity. - ResourceQuota exceeded in namespace. - Cluster autoscaler at max size.

Diagnose

```
kubectl describe pod <pod>
kubectl get events --sort-by=.lastTimestamp
kubectl top nodes
kubectl get pvc
kubectl get quota -n <ns>
```

Look for `FailedScheduling` and the exact reason.

Fix - Add nodes / scale node pool. - Fix nodeSelector or tolerations. - Create the referenced StorageClass / PV. - Raise or delete blocking ResourceQuota.

Prevent - Set sensible requests; monitor cluster utilization; alert on `unschedulable_pods_total`.

🚨 2. `CrashLoopBackOff`

Symptoms: Pod restarts repeatedly; `RESTARTS` climbs.

Causes - App crashes on startup (config, missing env, DB unreachable). - Failing liveness probe. - OOM (see #4). - Wrong command/args.

Diagnose

```
kubectl logs <pod> --previous
kubectl describe pod <pod>
kubectl get pod <pod> -o yaml | grep -A5 lastState
```

Fix - Fix bug or config; restart. - Raise `initialDelaySeconds` on livenessProbe. - Add missing ConfigMap/Secret.

Prevent - Add readiness probe to avoid routing during warmup. - Structured logging + startup diagnostics.

🚨 3. `ImagePullBackOff` / `ErrImagePull`

Symptoms: Pod shows `ImagePullBackOff`.

Causes - Wrong image name/tag. - Private registry, missing `imagePullSecret`. - Rate-limited by Docker Hub. - Node has no network route to registry.

Diagnose

```
kubectl describe pod <pod> | grep -A5 Events
```

Fix - Correct image / re-tag. - Create Secret + reference in Pod:

```
kubectl create secret docker-registry regcred --docker-server=... --docker-username=... --docker-password=... --docker-email=...
```

- Use ECR/GAR with IRSA/WLI so no secret needed.

Prevent - Mirror upstream images to your own registry. - Digest-pinning (`image@sha256:...`) for immutability.

🚨 4. `OOMKilled`

Symptoms: `kubectl describe pod` → `Last State: Terminated, Reason: OOMKilled, ExitCode: 137`.

Causes - Container's memory usage exceeded `resources.limits.memory`. - Memory leak. - Underestimated JVM/Go/Node heap.

Diagnose

```
kubectl top pod <pod> --containers
kubectl describe pod <pod>
```

Compare `limits.memory` to observed usage.

Fix - Raise limits (make sure node has capacity). - Tune GC / heap flags. - Fix leak.

Prevent - VPA in `Off` recommender mode for right-sizing. - Grafana alert on memory-usage / limit ratio > 80%.

🛎 5. Node `NotReady`

Symptoms: `kubectl get nodes` shows `NotReady`.

Causes - kubelet down or unhealthy. - CNI plugin failure. - Disk pressure / PID pressure. - Network partition. - Docker/containerd hung.

Diagnose

```
kubectl describe node <node>
# on node:
journalctl -u kubelet
crictl ps
df -h
```

Fix - Restart kubelet. - Reinstall CNI. - Reboot the node (GKE/EKS: replace it).

Prevent - Monitor kubelet + container-runtime health; auto-replace unhealthy nodes.

🛎 6. DNS resolution failures inside cluster

Symptoms: `nslookup kubernetes.default` fails; apps get `no such host`.

Causes - CoreDNS Pods down / OOM. - NetworkPolicy blocking DNS to `kube-system`. - Wrong `dnsPolicy` on Pod. - Node-local DNS cache issue.

Diagnose

```
kubectl -n kube-system get pods -l k8s-app=kube-dns
kubectl -n kube-system logs -l k8s-app=kube-dns
kubectl exec -it netshoot -- dig kubernetes.default
```

Fix - Scale CoreDNS; raise memory limits. - Add NetworkPolicy allowing UDP/TCP 53 to `kube-system`. - Restart node-local DNS DaemonSet.

Prevent - Run CoreDNS with PDB and topology spread; monitor `coredns_dns_request_duration_seconds`.

---

7. Service returns 503 / connection refused

Causes - Endpoints empty (no ready Pods). - Selector mismatch. - Pod readinessProbe failing. - kube-proxy dead.

Diagnose

```
kubectl get endpoints <svc>
kubectl get pods -l <selector> -o wide
kubectl describe pod <pod> | grep -A3 Readiness
```

Fix: Align labels; fix probe; scale up healthy Pods.

Prevent: CI test that the selector matches at least one label.

---

8. `Terminating` Pod stuck

Causes: Finalizer not removed, node lost, PV won't detach.

Diagnose

```
kubectl get pod <p> -o json | jq .metadata.finalizers
kubectl describe pod <p>
```

Fix - `kubectl patch pod <p> --type=merge -p '{"metadata":{"finalizers":[]}}'` - Force delete: `kubectl delete pod <p> --grace-period=0 --force` - Repair the underlying controller.

Prevent: Idempotent finalizer logic in operators.

---

9. PVC stuck `Pending`

Causes: No StorageClass, StorageClass missing provisioner, quota, no matching PV.

Diagnose

```
kubectl describe pvc <pvc>
kubectl get sc
kubectl get pv
```

Fix: Create SC; annotate default SC; ensure CSI driver Pods healthy.

Prevent: `is-default-class: true` annotation on cluster's default SC.

---

10. Pod can't mount PVC (multi-attach error)

Causes: RWO volume attached to another node; StatefulSet scaled down mid-rollout.

Diagnose

```
kubectl describe pod <p>
kubectl get volumeattachments
```

Fix: Wait for detach; force delete the old attachment; use RWX volume (EFS/Filestore).

Prevent: Only RWX for multi-Pod workloads.

---

11. HPA never scales

Causes: metrics-server not installed; missing CPU/memory requests; wrong target ref.

Diagnose

```
kubectl get hpa <name>
kubectl top pods -l app=<x>
kubectl describe hpa <name>
```

Fix: Install metrics-server; add requests; check `scaleTargetRef` kind/name.

Prevent: Alert on `hpa_scaler_metric_stale`.

---

12. Cluster Autoscaler never adds nodes

Causes: Node group at max; no matching instance type; Pod requests exceed largest node; missing tolerations on new nodes.

Diagnose: CA logs (`kubectl logs -n kube-system deploy/cluster-autoscaler`), events on Pending Pod.

Fix: Raise max; add larger instance type; fix nodeSelector.

Prevent: Regularly reconcile `max` size vs peak demand.

---

13. HPA and VPA fighting

Symptoms: Pod restarts loop, replicas thrash.

Fix: Use VPA in `Off` mode (recommendations only) with HPA on CPU; or VPA + HPA on custom metric.

Prevent: Documented autoscaling policy per workload.

---

14. `Unauthorized` from `kubectl`

Causes: Expired token, wrong context, kubeconfig missing `exec` plugin (`aws eks get-token` , `gke-gcloud-auth-plugin`).

Fix: `aws eks update-kubeconfig` , `gcloud container clusters get-credentials` , refresh token.

Prevent: SSO with short-lived tokens; rotate CI kubeconfigs.

---

15. `Forbidden` on a resource

Cause: RBAC missing.

Diagnose

```
kubectl auth can-i <verb> <resource> -n <ns>
kubectl auth can-i <verb> <resource> --as=<user>
```

Fix: Add Role/ClusterRole binding.

Prevent: RBAC lint in CI (e.g., `rbac-lookup` , `polaris`).

---

16. Ingress returns 404 or 502

Causes: Missing `IngressClass` , wrong `pathType` , backend Service selector mismatch, LB annotation typo, TLS cert missing.

Diagnose: `kubectl describe ingress` ; check controller logs; `curl -H "Host: ..."` to LB IP.

Fix: Set `ingressClassName` ; verify backend Service targets ready endpoints.

Prevent: Synthetic checks per Ingress route.

---

17. TLS cert not issued (cert-manager)

Causes: DNS-01 propagation slow, HTTP-01 firewall, ClusterIssuer misconfig.

Diagnose: `kubectl describe certificate/challenge/order` .

Fix: Fix DNS/records; open port 80 to LB; check ACME account key.

---

18. NetworkPolicy blocking wanted traffic

Symptoms: Timeouts within cluster.

Diagnose: `kubectl describe networkpolicy` ; test with `kubectl exec busybox -- nc -zv target 80` .

Fix: Add egress rule; whitelist DNS to `kube-system` .

Prevent: Default deny + explicit allow patterns; policy tests with `netpol-analyzer` .

---

19. StatefulSet Pod stuck `0/1 Ready`

Causes: PVC pending, ordinal peer down, headless Service missing.

Diagnose: `kubectl get pvc` ; verify headless Service.

Fix: Create Service `clusterIP: None` ; ensure peers healthy.

20. `evicted` Pods due to node pressure

Causes: Disk / memory / PID pressure.

Diagnose: `kubectl describe node`; look for `MemoryPressure=True`.

Fix: Increase node size; add ephemeral-storage requests; clean image cache.

Prevent: Node-local ephemeral-storage requests + eviction thresholds.

---

21. `too many open files`

Fix: Raise `ulimits` via node OS or LimitRange; on GKE/EKS use custom AMI/OS config.

---

22. Application drops requests during rolling update

Causes: No `preStop` hook + `terminationGracePeriodSeconds`; no readinessProbe.

Fix:

```
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 15"]
terminationGracePeriodSeconds: 60
```

Add readinessProbe so the Service removes the Pod before it exits.

---

23. Rolling update stuck

Causes: New Pods failing readiness; PDB blocking eviction.

Fix: `kubectl rollout status`; `kubectl rollout undo`; lower PDB minAvailable temporarily.

---

24. `deployment.apps` stuck at old ReplicaSet

Cause: Deployment paused or `progressDeadlineSeconds` exceeded.

Fix: `kubectl rollout resume deploy/<x>`; check logs of failing pods.

---

25. Node clock skew breaks TLS

Symptoms: `x509: certificate is not yet valid`.

Fix: Install/enable NTP; on EKS/GKE the nodes ship with time sync.

---

26. `Too many pods` on node

Cause: `--max-pods` limit hit (EKS VPC CNI computes based on ENI/IP limits).

Fix: Use larger instance; enable prefix delegation on VPC CNI (`ENABLE_PREFIX_DELEGATION=true`); or switch CNI.

---

27. Cluster suddenly slow to schedule

Causes: apiserver overloaded, etcd slow disk, huge list requests, watch storm.

Diagnose: `apiserver_request_duration_seconds`, `etcd_disk_wal_fsync_duration_seconds`, `apiserver_flowcontrol_dispatched_requests_total`.

Fix: Add FlowSchemas, tune informers, upgrade disk class.

---

28. Pod's outbound traffic times out

Causes: NetworkPolicy egress, NAT gateway saturated, cloud firewall.

Fix: Allow egress; add NAT capacity; check cloud VPC routes.

---

29. Node cannot join cluster (kubeadm)

Cause: Bad token, apiserver unreachable, wrong CA hash.

Fix: `kubeadm token create --print-join-command`.

---

30. Controller-manager not electing leader

Cause: Clock skew, Lease API blocked, etcd write latency.

Fix: Repair time sync; raise `--leader-elect-lease-duration`.

---

31. `no route to host` between Pods on different nodes

Cause: CNI misconfig, MTU mismatch, cloud firewall.

Fix: Ensure CNI matches cluster CIDR; lower Pod MTU (`1450` on GCP with encapsulation).

---

32. Container image huge, slow start

Fix: Multi-stage builds; distroless base; layer caching; use image streaming (GKE Image Streaming, EKS via lazy pulling).

---

33. Secrets not updated in running Pod

Cause: Env vars from Secret are captured at Pod start.

Fix: Mount as volume (auto-refresh) or use Reloader / Stakater.

---

34. Job never completes

Cause: Pod OOMKilled retried past `backoffLimit` ; wrong `restartPolicy` .

Fix: Set correct `restartPolicy: OnFailure` , adjust `backoffLimit` and `activeDeadlineSeconds` .

---

35. CronJob missing runs

Cause: `concurrencyPolicy: Forbid` + long-running job; `startingDeadlineSeconds` too small.

Fix: Adjust concurrency; use `Replace` if idempotent.

---

36. Persistent volume filled up

Fix: Expand volume (`kubectl edit pvc` → `resources.requests.storage` , needs `allowVolumeExpansion: true` on SC).

---

37. Autopilot rejects a Pod

Cause: Privileged, hostPath, missing requests, or violating compute-class ratios.

Fix: Remove offending fields; add requests; choose correct compute class.

---

38. Fargate Pod stuck Pending on EKS

Cause: No matching Fargate Profile.

Fix: Create Fargate Profile matching namespace + labels.

---

39. GKE Ingress not creating LB

Cause: `ingressClass` wrong (`gce` vs `nginx`); missing `NEG` annotation.

Fix: `kubernetes.io/ingress.class: gce` ; add `cloud.google.com/neg: '{"ingress": true}'` on Service.

---

40. EKS ALB Ingress: `no target group`

Cause: AWS Load Balancer Controller not installed or IRSA missing perms.

Fix: Install controller with correct IAM policy.

---

41. Metrics-server cannot reach kubelet

Fix: `--kubelet-insecure-tls` , `--kubelet-preferred-address-types=InternalIP` .

---

42. Prometheus scrape targets down

Cause: NetworkPolicy blocks Prometheus namespace.

Fix: Allow ingress from `monitoring` .

---

43. GKE node upgrade disrupts stateful app

Fix: Use PDB + blue-green upgrades; drain deadline `--pdb-timeout-seconds` .

---

44. Karpenter kills node hosting critical Pod

Fix: Annotate Pod `karpenter.sh/do-not-disrupt: "true"` ; add PDB.

---

45. IngressClass conflict

Cause: Two controllers watching same class.

Fix: Distinct `IngressClass` per controller; set `--ingress-class` flag.

---

46. `kubectl exec` hangs

Cause: Long-lived TCP through LB with idle timeout; SPDY blocked.

Fix: Use `--tty=false` for scripts; enable websocket support in LB.

---

47. Cluster upgrade blocked by deprecated API

Fix: Run `kubent`; migrate manifests to current API version.

---

48. `too many requests` (429) from apiserver

Cause: APF throttling; controller doing unbounded lists.

Fix: Add FlowSchema at `workload-high`; move to informer-based caching.

---

49. Pod gets wrong ServiceAccount

Cause: `automountServiceAccountToken: false` on SA/Pod.

Fix: Enable automount or use projected token.

---

50. Workload works in dev, fails in prod (same YAML)

Common differentiators - Different Kubernetes minor version (API deprecations). - Prod PDB / ResourceQuota / NetworkPolicy stricter. - Prod uses Autopilot / Fargate — enforced admission constraints. - Prod IRSA/WLI missing → SDK falls back to node role that fails. - Prod uses PSA `restricted` — dev uses `privileged`.

Fix: Use identical baseline (same CRDs, same admission policies, same node OS) via GitOps.

Prevent: Prod-parity dev environment; policy-as-code enforcement in CI.

📄 Topic 72 — Top 30 System Design Questions Using Kubernetes

Framework for any K8s system design question

1. **Clarify requirements** — scale, latency, availability, budget, regulation.
2. **Sketch high-level architecture** — clusters, network topology.
3. **Discuss compute strategy** — MNG/Autopilot/Fargate/Karpenter.
4. **Storage** — stateful vs stateless, database choice, PV strategy.
5. **Networking** — ingress, service mesh, cross-cluster, DNS.
6. **Security** — RBAC, network policies, secrets, image supply chain.
7. **Observability** — metrics, logs, traces, alerting.
8. **CI/CD & GitOps** — how do changes reach the cluster.
9. **Reliability** — HPA/VPA/CA, PDBs, multi-AZ/region, DR.
10. **Cost** — where the money goes, optimization levers.

Each question below sketches the answer around this frame.

🎯 1. Design a global e-commerce site on Kubernetes

Requirements: 50M DAU, ≤200 ms P95, active/active in 3 regions, 99.99% SLA.

High-level - 3 regional EKS/GKE clusters + edge CDN (CloudFront/Cloud CDN). - Global load balancer (Cloud LB / Route 53 latency + Global Accelerator). - Regional databases with cross-region replication (Aurora Global / Cloud Spanner). - Redis Cluster per region for cache. - Kafka MSK / Pub/Sub for events.

K8s - Compute: MNG for baseline + Karpenter for burst; separate node pool for GPUs (recommendations). - Ingress: ALB / GCE Ingress + WAF. - Service mesh (Istio) for retries, circuit breakers, mTLS. - HPA on RPS custom metrics; CA/Karpenter for infra scale.

Reliability: PDBs, topology spread across AZs, chaos GameDay.

Observability: Central Grafana + Thanos across regions; distributed tracing via Tempo/Jaeger.

Trade-offs: Cross-region consistency (eventual) vs latency.

🎯 2. Multi-tenant SaaS platform

- Tenant isolation via **namespaces** with strict `ResourceQuota` + `LimitRange` + NetworkPolicy per tenant.
- Optional per-tenant node pool with taints for hostile tenants.
- Ingress via wildcard `*.tenant.example.com` + TLS.
- OPA/Kyverno policies enforcing labels `tenant=<id>`.
- Central logging tagged by namespace.

Trade-off: Namespace vs cluster-per-tenant — namespaces cheaper but noisier neighbor risk; clusters are stronger boundaries but 10× cost.

🎯 3. Real-time chat backend (WebSocket + presence)

- Stateful frontends (WebSocket) on StatefulSet with headless Service.

- Sticky sessions via `sessionAffinity: ClientIP` or ALB with cookies.
- Redis for presence (pub/sub).
- HPA on active-connections custom metric.
- Graceful drain via `preStop: sleep` + connection draining on LB.

🎯 4. CI/CD pipeline running on Kubernetes

- **Tekton** or **Argo Workflows** for pipelines; **GitHub Actions self-hosted runners** on K8s for CI.
- Build with **Kaniko** / **BuildKit** in-cluster (no Docker daemon).
- Cache in S3/GCS; artifacts in ECR/GAR.
- CD via **ArgoCD ApplicationSets** targeting multiple environments.
- Progressive delivery via **Argo Rollouts** or **Flagger** with metrics-driven analysis.

🎯 5. ML inference platform

- GPU node pool (A10G, L4) with taints.
- **KServe** or **Seldon** as serving framework.
- HPA on QPS (custom metrics) + VPA for right-sizing.
- Model artifacts on S3/GCS via a MinIO-backed cache.
- Autoscale to zero via KServe / Knative for cost.

🎯 6. Batch data-processing platform

- **Argo Workflows** / **Volcano** for DAGs.
- Spot-only Karpenter NodePool for cost.
- Kubernetes Jobs with `activeDeadlineSeconds` and `backoffLimit`.
- Results in S3/GCS; metadata in Postgres.

🎯 7. Financial trading system with sub-ms tail latencies

- Bare-metal or dedicated nodes; disable CPU frequency scaling.
- **CPU manager** with `static` policy for exclusive CPUs.
- HugePages, SR-IOV NICs.
- Isolated node pool with `Guaranteed` QoS pods.
- Not a fit for Fargate/Autopilot.

🎯 8. IoT ingestion platform (1M devices)

- MQTT broker (EMQX) on StatefulSet with LoadBalancer.
- Kafka for buffering.
- Karpenter with c-family nodes for CPU-bound consumers.
- Regional edge clusters via EKS Anywhere / GKE on-prem.

🎯 9. Video streaming CDN origin

- Nginx/Origin StatefulSet with local NVMe (`local-path` StorageClass).
- Kubernetes-native autoscaling on network throughput metrics.
- Global LB fronting HLS/DASH.
- Object storage tier (S3/GCS) for cold segments.

🎯 10. Design an in-cluster monitoring stack

- **kube-prometheus-stack** with 2 Prometheus replicas + Thanos sidecar.
- **Loki** for logs; **Tempo** for traces; **Grafana** for UI.
- **Alertmanager** with PagerDuty/Slack.
- Long-term storage: S3/GCS via Thanos + Loki chunks.

🎯 11. Design fleet-wide config management

- GitOps: ArgoCD ApplicationSets or Anthos Config Sync.
- Bases (Kustomize) + per-cluster overlays.
- Policy Controller / Kyverno for baseline enforcement.

🎯 12. Multi-region active-active database on K8s

- CockroachDB / YugabyteDB / TiDB StatefulSets with per-region PV.
- Anti-affinity across AZ; TopologySpread across regions via multi-cluster.
- Backup: incremental snapshots to S3/GCS.

Note: For most orgs, managed DB (Aurora Global, Spanner) is safer than self-managed on K8s.

---

13. Serverless functions platform

- **Knative Serving** or **OpenFaaS**.
- Scale-to-zero; concurrency-based scaling.
- Ingress via Gateway API / Istio.

---

14. Zero-trust internal APIs

- **Istio ambient / sidecar** with strict mTLS.
- **SPIFFE/SPIRE** identities.
- AuthorizationPolicy per service.
- NetworkPolicy default-deny.

---

15. HA control plane on bare-metal

- **kubeadm** with stacked etcd, 3 control-plane nodes, HAProxy VIP.
- Cluster API for provisioning.
- Regular etcd snapshots to S3-compatible object store.

---

16. Blue-green deployment for a legacy monolith

- Two Deployments (`app-blue` , `app-green`) behind one Service.
- Switch selector to shift traffic.
- Argo Rollouts BlueGreen strategy automates it.

---

17. Canary rollout with automated analysis

- **Argo Rollouts** `Rollout` with `AnalysisTemplate` querying Prometheus for `success_rate > 99%` and `p95_latency < 300ms` .
- Steps: 5% → 25% → 50% → 100%.
- Auto-abort on failure.

---

18. Design an Ingress at 1M RPS

- Multiple **ingress-nginx** DaemonSets or NGINX Plus / Envoy Gateway with HPA.
- LB with cross-zone.
- Layer-7 caching via Envoy.
- Anti-affinity so 1 node loss doesn't kill throughput.

---

19. Migrate monolith to K8s

- Strangler pattern: front with API gateway routing to legacy + new services.
- Data: dual-write then cut over.
- Feature flags for rollback.

---

20. Design workload isolation for compliance (PCI/HIPAA)

- Dedicated node pool with taints; only PCI Pods tolerate.
- Encrypted etcd; encrypted disks; Sealed Secrets / KMS.
- Restricted PSS; strict egress NetworkPolicies.
- Audit logs shipped to SIEM.

---

21. Cost-optimize a 500-node cluster

- Right-size with VPA recommendations.
- Karpenter with 80% spot mix + on-demand fallback.
- Consolidate underutilized nodes.
- Bin-pack via `PodTopologySpread` + priority classes.
- Delete idle namespaces; enforce `LimitRange` .

---

22. Disaster recovery strategy

- Velero cross-region backups (daily + hourly).
- Backup for GKE (workload + PV).
- etcd snapshots (self-managed) every 15 min → immutable S3.
- Quarterly restore drills.

---

23. Design multi-cluster service discovery

- **MCS ServiceExport/ServiceImport** or **Cilium ClusterMesh**.
- Global DNS via ExternalDNS.

- Cross-cluster mTLS via shared root CA.

🎯 24. Migrating from VM-based deployment to K8s

- Containerize with Dockerfiles / Buildpacks.
- Wrap in Deployment/StatefulSet.
- Introduce HPA gradually.
- Migrate config to ConfigMap / Secret.
- Start with Kubernetes as compute; keep DB external.

🎯 25. Design an internal developer platform (IDP)

- Backstage as portal.
- Golden path templates → Argo CD apps.
- Crossplane for infra self-service.
- Kyverno policies enforce guardrails.

🎯 26. Message queue on Kubernetes

- RabbitMQ or Kafka via community operator.
- StatefulSet + PVC per broker.
- Anti-affinity across zones.
- PDB `maxUnavailable: 1`.
- Cross-zone replication for durability.

🎯 27. GPU sharing platform for research

- NVIDIA GPU Operator.
- **Time-slicing** or **MIG** (Multi-Instance GPU) for A100/H100.
- Queue via `Kueue` / Volcano for fairness.
- Users request `nvidia.com/gpu: 1`.

🎯 28. Edge cluster (retail store) design

- K3s / KubeEdge lightweight nodes.
- Managed from central hub via ACM / Fleet.
- Local-only workloads; cloud sync for aggregates.
- Offline resilience with local etcd or SQLite backend.

🎯 29. Security incident response readiness

- Falco or Cilium Tetragon for runtime alerts.
- Automated Pod isolation via label change + NetworkPolicy.
- Central log store (SIEM) with 90-day retention.
- Runbook: quarantine node, evict Pod, snapshot for forensics.

🎯 30. Design an ephemeral dev environment per PR

- ArgoCD ApplicationSet with `pull request` generator → 1 namespace per PR.
- TTL controller to garbage-collect after N days.
- Ingress: `pr-<num>.dev.example.com` via cert-manager wildcard.
- Cost gate via ResourceQuota per namespace.

📄 Topic 73 — Behavioral + Kubernetes Hybrid Questions

Interviews often blend "tell me about a time…" with a Kubernetes twist. Use the **STAR framework** (Situation → Task → Action → Result) for each.

🎯 1. "Tell me about a time you diagnosed a production Kubernetes incident."

STAR sketch - **S**: Payment service on EKS started returning 5xx during Black Friday peak. - **T**: Restore <10 min; root-cause without a rollback. - **A**: Checked HPA (maxed at 30), then `kubectl top pods` → memory near limit. `kubectl logs --previous` showed OOMKilled restart pattern. Scaled HPA `maxReplicas`, raised memory limit via hotfix Deployment patch, deployed. Followed up with VPA analysis + profiling to fix the leak. - **R**: 8-min recovery; no failed transactions; leak fixed within 48 h; VPA + alerting added. - **Lesson**: Ship VPA recommendations early; don't rely on average-case memory sizing.

🎯 2. "Describe a change you rolled back and why."

- Deployed a new ingress-nginx version; latency P99 spiked 2×.
- Rolled back via `helm rollback` in 3 min after Grafana alert.
- Traced to a default `worker_shutdown_timeout` change.

- Follow-up: added canary rollout with automated latency SLO analysis in Argo Rollouts.

🎯 3. "Tell me about a conflict with a teammate on architecture."

- Colleague wanted 1 giant cluster per environment; I argued for multiple by BU.
- Discussed blast radius, upgrade windows, quota conflicts.
- Compromise: one shared platform cluster + per-team workload clusters + fleet-wide GitOps.
- Result: onboarding time dropped from 2 weeks to 2 days.

🎯 4. "Time you influenced without authority."

- Champion for GitOps in a team using `kubectl apply` from laptops.
- Ran a lunch-and-learn, built a PoC ArgoCD instance in 1 day, wrote migration guide.
- Adopted org-wide within a quarter; audit trail from Git satisfied compliance.

🎯 5. "Time you made a mistake — how did you handle it?"

- Deleted a CRD without checking finalizers → cascading delete of prod DBs.
- Recovered from etcd snapshot within an hour; wrote a public postmortem.
- Introduced `kubectl` alias with confirmation prompt; added Kyverno policy blocking CRD deletes without label `sre-approved=true`.

🎯 6. "How do you keep up with Kubernetes changes?"

- KubeCon talks, Kubernetes changelog, KEPs, weekly release-team notes.
- Test each new minor in a Kind cluster before promoting.
- Run `kubent` in CI to detect deprecated APIs.

🎯 7. "Time you optimized cost."

- Migrated 500-node EKS from CA + MNG to Karpenter with 80/20 spot/on-demand.
- Introduced Namespace-level `ResourceQuota` and enforced default requests via LimitRange.
- Cut monthly compute by 42% ($180k → $104k) with no SLO regression.

🎯 8. "Time you had to onboard a junior."

- Structured 30/60/90 plan around Pods → Deployments → Services → Ingress → HPA.
- Paired on real tickets from day one; had them own a small operator.
- After 90 days they led a runbook for a common troubleshooting scenario.

🎯 9. "How do you approach security in Kubernetes?"

- Layered: image scanning (Trivy) → admission (Kyverno) → runtime (Falco) → network (NetworkPolicy) → identity (WLI/IRSA).
- Baseline PSS `restricted`; explicit exceptions via `pod-security.kubernetes.io/enforce=privileged` on tightly-controlled namespaces.
- Quarterly RBAC audits via `rakkess`.

🎯 10. "Time you designed for reliability under failure."

- Migrated stateful service to StatefulSet with 3 replicas across 3 AZs.
- Added PDB `maxUnavailable: 1`, topology spread, anti-affinity.
- Simulated zone loss with chaos experiment; hit SLO throughout.

🎯 11. "How do you decide between GKE and EKS for a new project?"

- Team's cloud skills, existing footprint, compliance boundaries.
- If ops maturity is low: GKE Autopilot.
- Deep AWS ecosystem dependence (SQS, Aurora, IAM ABAC): EKS.
- Multi-cloud strategy: use fleet abstractions to abstract differences.

🎯 12. "Time you communicated a complex K8s topic to non-technical stakeholders."

- Explained multi-region DR strategy to finance and legal using a car-insurance analogy.
- Got budget for cross-region backups and quarterly game-day.

🎯 13. "How do you handle disagreement on rollout speed?"

- Product wants weekly canary; SRE wants monthly.
- Meet in the middle: automated canary with SLO-based auto-abort.
- Data-driven review after 3 months; adjust cadence.

🎯 14. "Describe a time you missed a deadline."

- Underestimated ingress migration from ELB to ALB with cert-manager.
- Communicated early, replanned, delivered 5 days late.
- Built a checklist for future migrations to spot 3rd-party dependencies upfront.

🎯 15. "How do you decide when to use an Operator vs a Helm chart?"

- Helm for stateless, install-and-forget.
- Operator when you need day-2 behaviors: failover, backup, upgrade coordination.
- Prefer community-maintained operators; write custom only when domain-specific.

🎯 16. "Tell me about a K8s upgrade that went sideways."

- Upgrading GKE 1.24 → 1.25 broke a Helm chart using removed `PodSecurityPolicy` API.
- Discovered mid-upgrade; paused via blue-green; migrated chart to PSA + Kyverno; resumed.
- Now run `kubent` and canary each minor in dev.

🎯 17. "How do you drive operational excellence?"

- SLIs/SLOs per service tied to error budget.
- Weekly review of top toil sources; automate top 3 each quarter.
- Runbooks in Git, versioned with charts.

🎯 18. "How do you decide between Cluster Autoscaler and Karpenter on EKS?"

- CA is proven, works with ASG boundaries.
- Karpenter is faster, more flexible, cost-optimized, but younger.
- I default to Karpenter for new EKS clusters; retain CA on regulated clusters where change control is slower.

🎯 19. "Tell me about a time you led an on-call rotation improvement."

- On-call fatigue from 40+ pages/week.
- Categorized pages, killed 60% via better probes and alert tuning.
- Introduced SLO-based paging; page count dropped to <10/week.

🎯 20. "How do you approach production readiness reviews?"

- Checklist: probes, resources, PDB, HPA, RBAC, NetworkPolicy, image provenance, dashboards, runbook, chaos test result.
- Sign-off from SRE + Security before Ingress goes live.

🎯 21. "Handling ambiguity"

- Asked to 'move everything to Kubernetes' with no scope.
- Ran discovery: catalog apps, categorize by lift-and-shift vs refactor vs retire; produced a 6-month migration wave plan with metrics.
- Got exec buy-in; delivered wave 1 on time.

🎯 22. "Ownership under failure"

- Pager fires at 3 AM; feature I owned was root cause of DB overload.
- Woke myself, coordinated rollback, wrote postmortem, closed all action items within 2 sprints.

🎯 23. "Data-driven decision"

- Debated Istio vs Linkerd for mesh.
- Ran a bake-off measuring P99, memory footprint, incident count for 4 weeks; picked Linkerd for our latency profile with a documented list of features we gave up.

🎯 24. "How do you mentor engineers on K8s?"

- Curated learning path: CKAD → CKA → CKS.
- Weekly office hours; real-cluster labs for common scenarios.
- Rotate junior engineers into a shadow-on-call program.

🎯 25. "Long-term technical strategy"

- Started policy-as-code + progressive delivery + FinOps together as a 12-month platform bet.
- Measured DORA metrics quarterly; deploy frequency 4×, MTTR halved, cost/req dropped 30%.

⏸ Pause Point — End of Batch 4 of 5

Completed in this batch (Topics 71–73) — Module 12 done: - ✅ 50 real-world troubleshooting scenarios (CrashLoopBackOff, OOMKilled, DNS failures, upgrade issues, cloud-specific quirks) - ✅ 30 system design questions with framework and worked answers - ✅ 25 behavioral + Kubernetes hybrid questions with STAR responses

Remaining: - **Batch 5 (Capstones 1–10):** End-to-end lab projects - **Final:** PDF compilation

Say **"continue"** to proceed with Batch 5 — the 10 Capstone Lab Projects (3-tier app, CI/CD, autoscaling under load, Ingress+TLS, full observability stack, StatefulSet DB with backup, security stack, GitOps multi-env, custom Operator, DR with etcd/Velero/Backup-for-GKE).

MODULE

Module 13 — Capstone Hands-On Lab Projects

Module 13 — Capstone Hands-On Lab Projects

End-to-End Labs 1 – 10

Each capstone combines multiple topics from earlier modules. Follow the standard lab structure: **Objective → Prerequisites → Environment Setup → Step-by-Step → YAML → Verification → Cleanup → Troubleshooting → Bonus Challenge.**

⚠ **Cost-safety reminder**: All cloud labs incur charges. Run cleanup at the end of every session. Set budget alerts.

🏆 Capstone 1 — Deploy a 3-Tier Web Application (Frontend + Backend + MySQL) on Minikube → GKE Autopilot → EKS Fargate

Difficulty: Beginner–Intermediate **Estimated time**: 3 hours (all three targets)

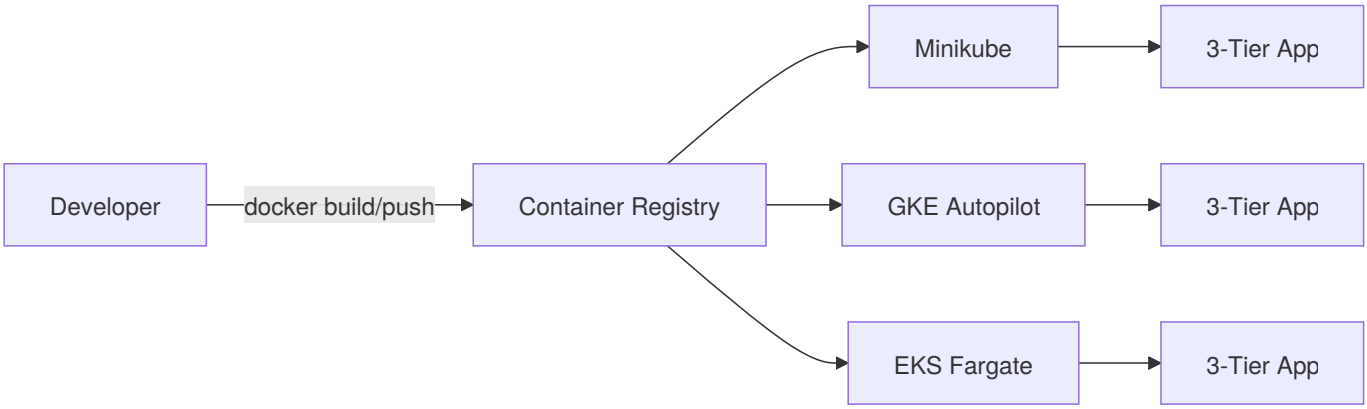
Objective

Build and deploy a canonical 3-tier app: React frontend → Node/Python backend → MySQL StatefulSet. Repeat the exact same manifests on Minikube, GKE Autopilot, and EKS Fargate to observe portability and platform-specific tweaks.

Prerequisites

- Docker, kubectl, git.
- Minikube v1.33+.
- gcloud SDK + a GCP project with billing.
- aws CLI + eksctl + an AWS account.
- A container registry (Docker Hub / ECR / GAR).

Workflow Diagram



Environment Setup

Sample app source layout

```
capstone1/
├── frontend/   (React, port 3000)
├── backend/    (Node.js Express, port 8080)
├── db/         (MySQL 8 via official image)
├── manifests/
│   ├── namespace.yaml
│   ├── mysql-statefulset.yaml
│   ├── backend-deployment.yaml
│   ├── frontend-deployment.yaml
│   ├── services.yaml
│   └── ingress.yaml
```

Step-by-Step

1. Build and push images

```
docker build -t myrepo/shop-frontend:1.0 frontend/
docker build -t myrepo/shop-backend:1.0 backend/
docker push myrepo/shop-frontend:1.0
docker push myrepo/shop-backend:1.0
```

2. Manifests

`namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata: { name: shop, labels: { pod-security.kubernetes.io/enforce: baseline } }
```

`mysql-statefulset.yaml`

```
apiVersion: v1
kind: Service
metadata: { name: mysql, namespace: shop }
spec:
  clusterIP: None
  selector: { app: mysql }
  ports: [{ port: 3306, name: mysql }]
---
apiVersion: v1
kind: Secret
metadata: { name: mysql-secret, namespace: shop }
stringData:
  MYSQL_ROOT_PASSWORD: changeme
  MYSQL_DATABASE: shopdb
  MYSQL_USER: shop
  MYSQL_PASSWORD: shoppass
---
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: mysql, namespace: shop }
spec:
  serviceName: mysql
  replicas: 1
  selector: { matchLabels: { app: mysql } }
  template:
    metadata: { labels: { app: mysql } }
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          envFrom: [{ secretRef: { name: mysql-secret } }]
          ports: [{ containerPort: 3306 }]
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "1",   memory: "1Gi" }
          readinessProbe:
            exec: { command: ["sh","-c","mysqladmin -uroot -p$MYSQL_ROOT_PASSWORD ping"] }
            initialDelaySeconds: 20
            periodSeconds: 10
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
  volumeClaimTemplates:
    - metadata: { name: data }
      spec:
        accessModes: ["ReadWriteOnce"]
        resources: { requests: { storage: 10Gi } }
```

`backend-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: backend, namespace: shop }
spec:
  replicas: 2
  selector: { matchLabels: { app: backend } }
  template:
    metadata: { labels: { app: backend } }
    spec:
      containers:
        - name: backend
          image: myrepo/shop-backend:1.0
          ports: [{ containerPort: 8080 }]
          env:
            - name: DB_HOST
              value: mysql.shop.svc.cluster.local
            - { name: DB_USER, valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_USER } } }
            - { name: DB_PASS, valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_PASSWORD } } }
            - { name: DB_NAME, valueFrom: { secretKeyRef: { name: mysql-secret, key: MYSQL_DATABASE } } }
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 5
```

`frontend-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: frontend, namespace: shop }
spec:
  replicas: 2
  selector: { matchLabels: { app: frontend } }
  template:
    metadata: { labels: { app: frontend } }
    spec:
      containers:
        - name: frontend
          image: myrepo/shop-frontend:1.0
          ports: [{ containerPort: 3000 }]
          env:
            - name: BACKEND_URL
              value: http://backend.shop.svc.cluster.local:8080
          resources:
            requests: { cpu: "250m", memory: "512Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
```

`services.yaml`

```
apiVersion: v1
kind: Service
metadata: { name: backend, namespace: shop }
spec:
  selector: { app: backend }
  ports: [{ port: 8080, targetPort: 8080 }]
---
apiVersion: v1
kind: Service
metadata: { name: frontend, namespace: shop }
spec:
  type: LoadBalancer
  selector: { app: frontend }
  ports: [{ port: 80, targetPort: 3000 }]
```

3. Deploy to Minikube

```
minikube start --cpus=4 --memory=8g --kubernetes-version=v1.31.0
minikube addons enable ingress
kubectl apply -f manifests/
kubectl -n shop rollout status statefulset/mysql
kubectl -n shop rollout status deploy/backend deploy/frontend
minikube service frontend -n shop
```

4. Deploy to GKE Autopilot

```
gcloud container clusters create-auto capstone1 --region=us-central1
gcloud container clusters get-credentials capstone1 --region=us-central1
kubectl apply -f manifests/
kubectl -n shop get svc frontend -w   # wait for EXTERNAL-IP
```

5. Deploy to EKS Fargate

```
# fargate-cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata: { name: capstone1, region: us-east-1, version: "1.31" }
iam: { withOIDC: true }
fargateProfiles:
  - name: fp-shop
    selectors:
      - namespace: shop
      - namespace: kube-system
        labels: { k8s-app: kube-dns }
```

```
eksctl create cluster -f fargate-cluster.yaml
kubectl apply -f manifests/
# NOTE: MySQL StatefulSet on Fargate needs a supported storage class
# For Fargate: switch to EFS via aws-efs-csi-driver or Amazon RDS as an external DB.
```

Verification

```
kubectl -n shop get pods -o wide
kubectl -n shop get svc,pvc,statefulset
curl http://<frontend-external-ip>/
kubectl -n shop exec -it mysql-0 -- mysql -uroot -p$MYSQL_ROOT_PASSWORD -e "SHOW DATABASES;"
```

Expected: `shopdb` present, frontend renders items fetched via backend.

Cleanup

```
# Minikube
minikube delete

# GKE
gcloud container clusters delete capstone1 --region=us-central1 --quiet

# EKS
eksctl delete cluster --name capstone1 --region=us-east-1
```

Troubleshooting

- **Frontend can't reach backend** — check Service DNS: `kubectl exec pod -- nslookup backend.shop`.
- **MySQL Pending on Fargate** — Fargate doesn't support EBS; switch StatefulSet PVC to `efs-sc` (EFS CSI) or use RDS externally.
- **LoadBalancer stuck** `<pending>` — cloud LB quota exceeded or IAM missing.

Bonus Challenges

1. Replace MySQL StatefulSet with RDS (EKS) / Cloud SQL (GKE) using Config Connector / ACK.
2. Add horizontal scaling with HPA on backend CPU 60%.
3. Store secrets in Secret Manager / Secrets Manager via External Secrets Operator.

🏆 Capstone 2 — CI/CD Pipeline: GitHub Actions → Docker → GKE and EKS with Rolling Updates + Rollback

Difficulty: Intermediate **Estimated time**: 2.5 hours

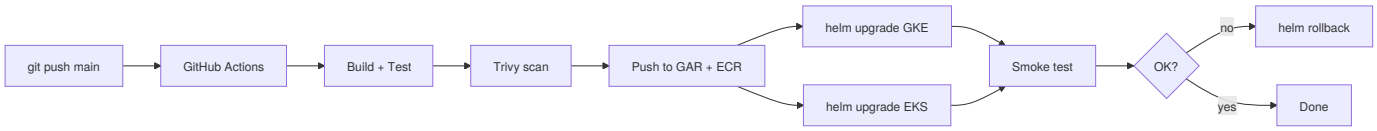
Objective

Every push to `main` builds a new image, pushes to GAR/ECR, and rolls out to GKE and EKS with automatic rollback on failure.

Prerequisites

- GitHub repo (fork of capstone1).
- GKE + EKS clusters from previous capstones (or fresh).
- WLI on GKE, IRSA on EKS for keyless registry auth.
- `helm` v3.14+.

Workflow Diagram



Environment Setup

GitHub OIDC federation - GKE: create a GCP Workload Identity Pool that trusts GitHub OIDC. - EKS: create IAM Role with GitHub OIDC trust policy.

Helm chart: put the manifests from Capstone 1 into `charts/shop/`.

Step-by-Step

1. Helm chart values (`charts/shop/values.yaml`)

```
image:
  registry: gcr.io/my-project/shop
  tag: latest
replicas:
  frontend: 3
  backend: 3
mysql:
  storage: 10Gi
```

2. GitHub Actions workflow (`.github/workflows/deploy.yaml`)

```
name: build-and-deploy
on:
  push: { branches: [main] }
permissions:
  id-token: write
  contents: read
jobs:
  build:
    runs-on: ubuntu-latest
    outputs: { tag: ${{ steps.meta.outputs.tag }} }
    steps:
      - uses: actions/checkout@v4
      - id: meta
        run: echo "tag=$(git rev-parse --short HEAD)" >> $GITHUB_OUTPUT
      - uses: docker/setup-buildx-action@v3
      - name: Auth GCP
        uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: projects/123/locations/global/workloadIdentityPools/gh/providers/gh
          service_account: ci@my-project.iam.gserviceaccount.com
      - name: Auth ECR
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111111111111:role/gha-oidc
          aws-region: us-east-1
      - run: gcloud auth configure-docker us-central1-docker.pkg.dev -q
      - run: aws ecr get-login-password | docker login --username AWS --password-stdin 111111111111.dkr.ecr.us-east-1.amazonaws.com
      - name: Build & push frontend
        run: |
          docker buildx build --push -t us-central1-docker.pkg.dev/my-project/shop/frontend:${{ steps.meta.outputs.tag }} \
              -t 111111111111.dkr.ecr.us-east-1.amazonaws.com/shop/frontend:${{ steps.meta.outputs.tag }} frontend/
      - name: Trivy scan
        uses: aquasecurity/trivy-action@0.20.0
        with: { image-ref: 'us-central1-docker.pkg.dev/my-project/shop/frontend:${{ steps.meta.outputs.tag }}', exit-code: '1', severity: 'CRITICAL,HIGH' }
  deploy-gke:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2
        with: { workload_identity_provider: ..., service_account: ci@... }
      - uses: google-github-actions/get-gke-credentials@v2
        with: { cluster_name: capstone1, location: us-central1 }
      - name: Helm upgrade
        run: |
          helm upgrade --install shop charts/shop \
            --namespace shop --create-namespace \
            --set image.tag=${{ needs.build.outputs.tag }} \
            --atomic --timeout 5m --wait
      - name: Smoke test
        run: kubectl -n shop rollout status deploy/frontend
  deploy-eks:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with: { role-to-assume: arn:aws:iam::111111111111:role/gha-oidc, aws-region: us-east-1 }
      - run: aws eks update-kubeconfig --name capstone1
      - name: Helm upgrade
        run: |
          helm upgrade --install shop charts/shop \
            --namespace shop --create-namespace \
            --set image.tag=${{ needs.build.outputs.tag }} \
            --set image.registry=111111111111.dkr.ecr.us-east-1.amazonaws.com/shop \
            --atomic --timeout 5m --wait
```

3. Rollback - `--atomic` auto-rolls-back on failure. - Manual: `helm rollback shop <REV> -n shop`.

Verification

- Push a change; watch Actions succeed; observe rolling update:

```
kubectl -n shop rollout history deploy/frontend
kubectl -n shop get pods -w
helm history shop -n shop
```

Cleanup

```
helm uninstall shop -n shop
# Delete clusters as in Capstone 1
```

Troubleshooting

- **OIDC federation fails** — recheck audience `sts.amazonaws.com` (AWS) / provider mapping (GCP).
- **Trivy fails** — patch base image or add ignore file.
- **`--atomic` rollback but pods stuck** — check readinessProbe path.

Bonus Challenges

- Add a `manual approval` gate for prod.
- Add SBOM (`syft`) upload as workflow artifact.
- Add `cosign` image signing + Kyverno verification policy.

🏆 Capstone 3 — HPA + Cluster Autoscaler on GKE and Karpenter on EKS Under Load Testing

(hey / k6)

Difficulty: Intermediate **Estimated time**: 2 hours

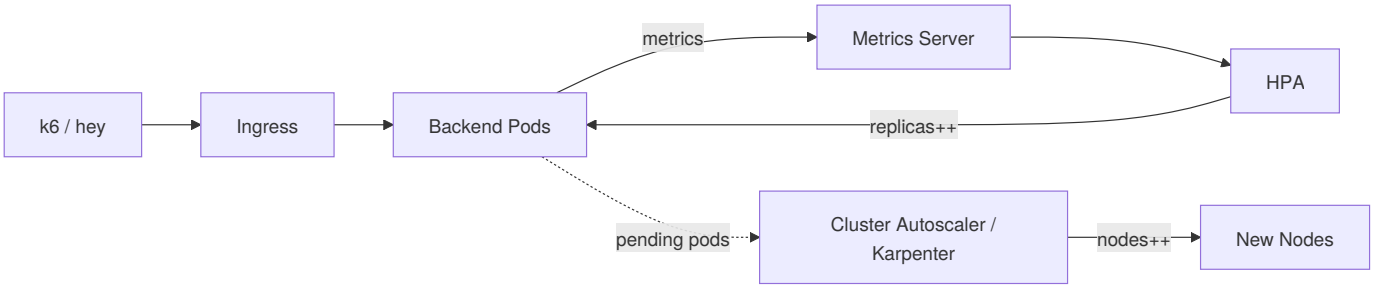
Objective

Load-test the shop backend, observe HPA scaling Pods and CA/Karpenter scaling nodes; produce a load-vs-latency plot.

Prerequisites

- GKE Standard (for CA) + EKS with Karpenter installed.
- `hey` or `k6`, `kubectl`, `helm`.

Workflow



Setup

GKE Standard cluster with CA

```
gcloud container clusters create loadtest --region=us-central1 \
  --enable-autoscaling --min-nodes=1 --max-nodes=10 \
  --machine-type=e2-standard-4 --release-channel=regular \
  --enable-ip-alias --enable-dataplane-v2
```

EKS + Karpenter — follow https://karpenter.sh/docs/getting-started/

Backend Deployment + HPA

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: backend, namespace: shop }
spec:
  replicas: 2
  selector: { matchLabels: { app: backend } }
  template:
    metadata: { labels: { app: backend } }
    spec:
      containers:
        - name: backend
          image: myrepo/shop-backend:1.0
          resources:
            requests: { cpu: "250m", memory: "256Mi" }
            limits:   { cpu: "500m", memory: "512Mi" }
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: backend-hpa, namespace: shop }
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: backend }
  minReplicas: 2
  maxReplicas: 50
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 60 } }
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies: [{ type: Percent, value: 100, periodSeconds: 30 }]
    scaleDown:
      stabilizationWindowSeconds: 300
```

Load Test

```
kubectl -n shop port-forward svc/backend 8080:8080 &
hey -z 5m -c 200 -q 50 http://localhost:8080/api/items

# k6 alternative
cat > script.js <<EOF
import http from 'k6/http';
export const options = { vus: 200, duration: '5m' };
export default () => http.get('http://backend.shop:8080/api/items');
EOF
k6 run script.js
```

Watch:

```
watch 'kubectl top pods -n shop; echo; kubectl get hpa -n shop; echo; kubectl get nodes'
```

Verification

- HPA `TARGETS` climbs, replicas grow.
- Nodes scale from 1 → many.
- Latency P95 stays within target after stabilization.

Cleanup

```
kubectl delete deploy backend -n shop
gcloud container clusters delete loadtest --region=us-central1 --quiet
eksctl delete cluster --name karp-lab
```

Troubleshooting

- **HPA `<unknown>` targets** — metrics-server not installed / no requests set.
- **CA logs "no matching NG"** — instance-family mismatch.
- **Karpenter picks huge nodes** — restrict `instance-family` and `instance-size`.

Bonus

- Add HPA on custom metric `http_requests_per_second` via Prometheus Adapter.
- Compare cost of CA vs Karpenter for the same load (use `kubectl-cost` or CloudZero/Kubecost).

🏆 Capstone 4 — Ingress with TLS: cert-manager + Let's Encrypt on GKE (GCE Ingress) and EKS (ALB Ingress)

Difficulty: Intermediate **Estimated time**: 2 hours

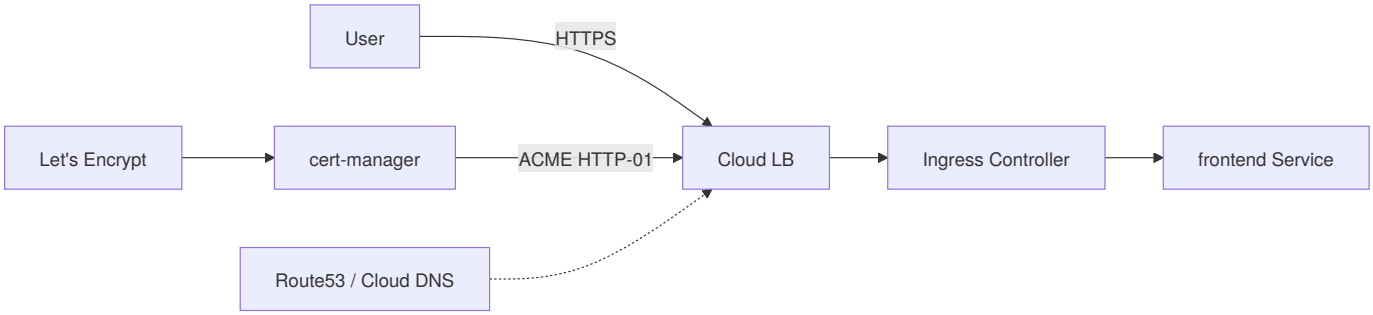
Objective

Expose the shop app on `shop.example.com` with automatic TLS certificates.

Prerequisites

- Owned domain with DNS access.
- GKE + EKS clusters.
- Helm.

Workflow



Setup

Install cert-manager (both clusters)

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager -n cert-manager --create-namespace \
  --version v1.15.0 --set installCRDs=true
```

ClusterIssuer

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: letsencrypt-prod }
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: you@example.com
    privateKeySecretRef: { name: letsencrypt-prod }
    solvers:
      - http01:
          ingress:
            class: gce         # for GKE; use "alb" for EKS
```

GKE Ingress (`ingress-gke.yaml`)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    kubernetes.io/ingress.class: "gce"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    kubernetes.io/ingress.global-static-ip-name: "shop-ip"
spec:
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /*
            pathType: ImplementationSpecific
            backend: { service: { name: frontend, port: { number: 80 } } }
```

Reserve static IP:

```
gcloud compute addresses create shop-ip --global
gcloud compute addresses describe shop-ip --global
# Update Cloud DNS: A record for shop.example.com → returned IP
```

EKS ALB Ingress

```
# Install AWS Load Balancer Controller with IRSA (see Capstone 7 for IRSA pattern)
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system --set clusterName=capstone1 --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  namespace: shop
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  tls:
    - hosts: [shop.example.com]
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: frontend, port: { number: 80 } } }
```

Verification

```
kubectl -n shop describe ingress shop
kubectl -n shop get certificate,order,challenge
curl -v https://shop.example.com/
```

Cleanup

```
kubectl delete -f ingress-*.yaml
helm uninstall cert-manager -n cert-manager
gcloud compute addresses delete shop-ip --global --quiet
```

Troubleshooting

- **Challenge stuck** `Pending` — DNS hasn't propagated; ensure record is public and correct.
- `ACME connection issue` — LB not routing; check `--cluster-issuer` events.
- **Rate limits from Let's Encrypt** — use staging issuer while testing.

Bonus

- Migrate to **DNS-01 challenge** (needed for wildcard certs) via Cloud DNS / Route53 provider.
- Add HTTP → HTTPS redirect.
- Add WAF (Cloud Armor / AWS WAFv2) via Ingress annotations.

🏆 Capstone 5 — Full Observability Stack: Prometheus + Grafana + Loki + Alertmanager on GKE and EKS

Difficulty: Intermediate–Advanced **Estimated time**: 3 hours

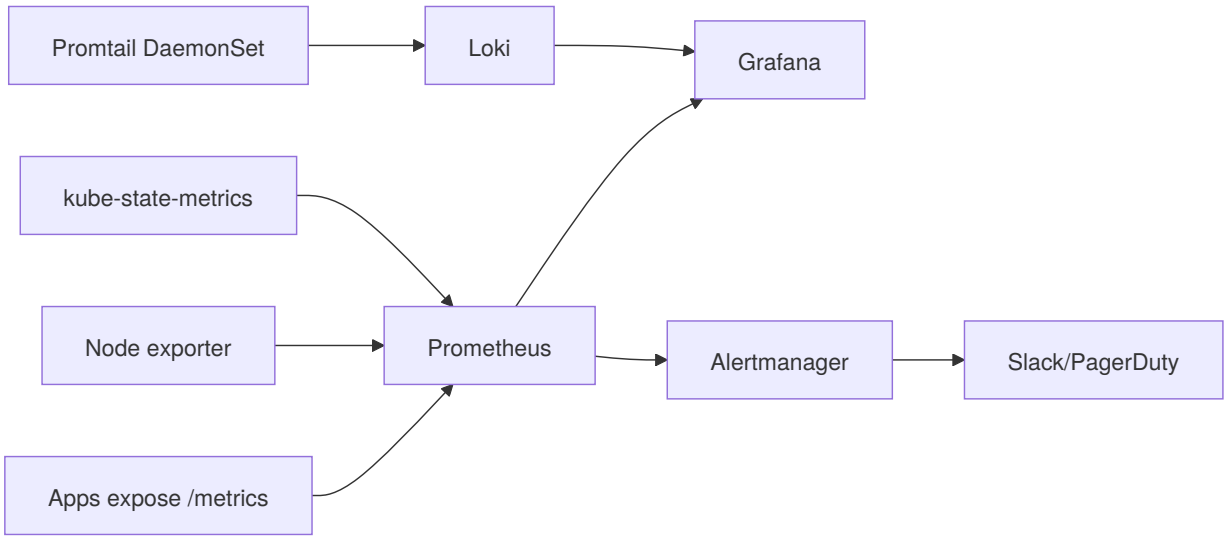
Objective

Deploy kube-prometheus-stack + Loki + Promtail + Alertmanager; build a dashboard for shop app SLIs; fire an alert on P95 latency breach.

Prerequisites

- GKE or EKS cluster with ≥3 nodes.
- Helm.
- (Optional) Slack webhook for Alertmanager.

Workflow



Setup

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

kubectl create ns monitoring

# Prometheus + Grafana + AM
helm install kube-prom prometheus-community/kube-prometheus-stack -n monitoring \
  --set grafana.adminPassword=admin123 \
  --set prometheus.prometheusSpec.retention=15d \
  --set prometheus.prometheusSpec.resources.requests.memory=1Gi

# Loki
helm install loki grafana/loki-stack -n monitoring \
  --set grafana.enabled=false --set promtail.enabled=true
```

Instrument the backend

Add `/metrics` endpoint (Prometheus format) or use an SDK; add a `ServiceMonitor`:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: backend
  namespace: monitoring
  labels: { release: kube-prom }
spec:
  selector: { matchLabels: { app: backend } }
  namespaceSelector: { matchNames: [shop] }
  endpoints:
    - port: http
      path: /metrics
      interval: 15s
```

Alert Rule

```
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: shop-slo
  namespace: monitoring
  labels: { release: kube-prom }
spec:
  groups:
    - name: shop.rules
      rules:
        - alert: BackendHighLatency
          expr: |
            histogram_quantile(0.95,
              sum by (le) (rate(http_request_duration_seconds_bucket{job="backend"}[5m]))
            ) > 0.5
          for: 5m
          labels: { severity: warning }
          annotations:
            summary: "Backend P95 latency > 500ms"
```

Grafana Dashboard

- Port-forward: `kubectl -n monitoring port-forward svc/kube-prom-grafana 3000:80`
- Login (admin/admin123).
- Import dashboard IDs: `315` (K8s), `13639` (Loki logs), and build custom shop SLI panels: request rate, error rate, P95.

Verification

- Prometheus targets: `kubectl -n monitoring port-forward svc/kube-prom-kube-prometheus 9090:9090` → `/targets` shows backend UP.
- Loki logs visible in Grafana Explore.
- Trigger a load spike; alert fires in Alertmanager UI.

Cleanup

```
helm uninstall kube-prom loki -n monitoring
kubectl delete ns monitoring
```

Troubleshooting

- **ServiceMonitor not scraped** — label mismatch with the release; add `release: kube-prom`.
- **Loki OOM** — raise memory, or use SingleBinary mode for demos.
- **Alertmanager silent** — verify receiver config; test with `amtool`.

Bonus

- Send traces via OpenTelemetry Collector → Tempo.
- Add SLO burn-rate alerts (multi-window multi-burn).
- Use **GKE Managed Prometheus** and **CloudWatch Container Insights** as managed alternatives; compare cost.

🏆 Capstone 6 — StatefulSet PostgreSQL with PVC on GKE Filestore + EKS EBS/EFS, Including Backup and Restore

Difficulty: Intermediate–Advanced **Estimated time**: 3 hours

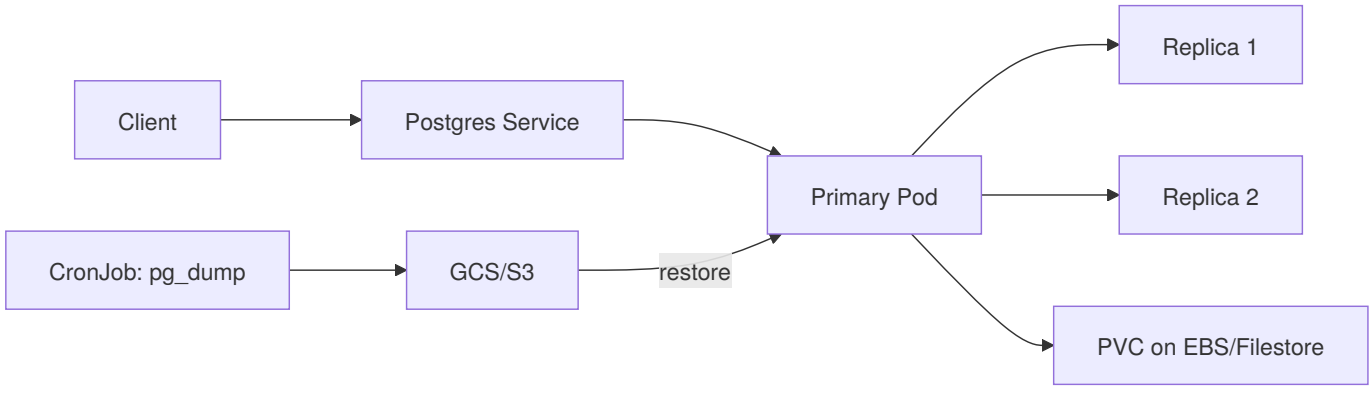
Objective

Run a 3-replica PostgreSQL StatefulSet with proper storage; take a logical backup to object storage; restore it.

Prerequisites

- GKE cluster with Filestore CSI driver enabled OR EKS with EBS CSI + EFS CSI.
- `pg_dump` client locally.
- Object bucket (GCS / S3).

Workflow



Storage class

GKE Filestore

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: filestore-sc }
provisioner: filestore.csi.storage.gke.io
parameters: { tier: BASIC_HDD, network: default }
allowVolumeExpansion: true
```

EKS EBS

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: gp3-sc }
provisioner: ebs.csi.aws.com
parameters: { type: gp3, iops: "3000", throughput: "125" }
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

PostgreSQL via Bitnami Helm

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install pg bitnami/postgresql -n db --create-namespace \
  --set architecture=replication \
  --set primary.persistence.storageClass=gp3-sc \
  --set primary.persistence.size=20Gi \
  --set readReplicas.replicaCount=2 \
  --set readReplicas.persistence.storageClass=gp3-sc \
  --set readReplicas.persistence.size=20Gi \
  --set auth.postgresPassword=changeme \
  --set auth.database=shopdb
```

Backup CronJob to S3

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: pg-backup, namespace: db }
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: pg-backup-sa   # IRSA / WLI SA
          restartPolicy: OnFailure
          containers:
            - name: dump
              image: postgres:16
              env:
                - name: PGPASSWORD
                  valueFrom: { secretKeyRef: { name: pg-postgresql, key: postgres-password } }
              command:
                - /bin/sh
                - -c
                - |
                  TS=$(date +%Y%m%d-%H%M)
                  pg_dump -h pg-postgresql -U postgres shopdb | gzip > /tmp/backup-$TS.sql.gz
                  aws s3 cp /tmp/backup-$TS.sql.gz s3://acme-pg-backups/shopdb/backup-$TS.sql.gz
```

Restore

```
aws s3 cp s3://acme-pg-backups/shopdb/backup-20260927-1200.sql.gz .
gunzip -c backup-20260927-1200.sql.gz | \
  kubectl exec -it -n db pg-postgresql-0 -- psql -U postgres shopdb
```

Verification

```
kubectl -n db get statefulset,pvc
kubectl -n db exec -it pg-postgresql-0 -- psql -U postgres -c "SELECT * FROM pg_stat_replication;"
kubectl -n db get cronjob,job
aws s3 ls s3://acme-pg-backups/shopdb/
```

Cleanup

```
helm uninstall pg -n db
kubectl -n db delete pvc --all
aws s3 rm s3://acme-pg-backups --recursive
```

Troubleshooting

- **PVC Pending** — CSI driver not installed; `kubectl get pods -n kube-system | grep csi`.
- **`multi-attach error`** — RWO on Filestore: use RWX Filestore tier.
- **Replication lag** — check `pg_stat_replication`, network throughput.

Bonus

- Replace with **Zalando Postgres Operator** (Capstone 9 preview).
- Add PITR via WAL archiving to S3 (`wal-g`).
- Multi-region: run standby cluster + `pg_basebackup` bootstrap.

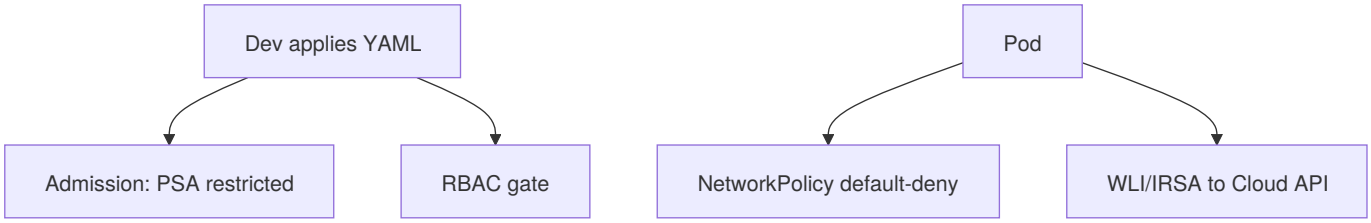
🏆 Capstone 7 — Enforce Security: RBAC + NetworkPolicy + Pod Security Admission + WLI (GKE) + IRSA (EKS)

Difficulty: Advanced **Estimated time**: 3 hours

Objective

Layer security controls on the shop app: least-privilege RBAC, default-deny NetworkPolicy, enforce `restricted` PSA, and use cloud identities for backend to reach a bucket.

Workflow



Prerequisites

- Cluster from prior capstones.
- Kyverno or built-in PSA.

Step 1 — Enforce PSA `restricted`

```
kubectl label ns shop \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted --overwrite
```

Then update Pods to comply:

```
securityContext:
  runAsNonRoot: true
  seccompProfile: { type: RuntimeDefault }
containers:
  - name: backend
    securityContext:
      allowPrivilegeEscalation: false
      capabilities: { drop: ["ALL"] }
      readOnlyRootFilesystem: true
```

Step 2 — RBAC

```
apiVersion: v1
kind: ServiceAccount
metadata: { name: shop-dev, namespace: shop }
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: shop-dev, namespace: shop }
rules:
  - apiGroups: ["", "apps"]
    resources: ["pods", "deployments", "services", "configmaps"]
    verbs: ["get","list","watch","create","update","patch","delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: shop-dev, namespace: shop }
subjects: [{ kind: User, name: dev@acme.io, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: shop-dev, apiGroup: rbac.authorization.k8s.io }
```

Test: `kubectl auth can-i delete nodes --as=dev@acme.io` → no.

Step 3 — NetworkPolicy default-deny + allow

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: default-deny, namespace: shop }
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: allow-dns, namespace: shop }
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to: [{ namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } } }]
      ports: [{ protocol: UDP, port: 53 }, { protocol: TCP, port: 53 }]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: frontend-to-backend, namespace: shop }
spec:
  podSelector: { matchLabels: { app: backend } }
  policyTypes: [Ingress]
  ingress:
    - from: [{ podSelector: { matchLabels: { app: frontend } } }]
      ports: [{ port: 8080 }]
```

Step 4 — Cloud identity

GKE WLI — see Topic 69 Example 1: bind `shop-ksa` to a GCP SA with `roles/storage.objectViewer`.

EKS IRSA —

```
eksctl create iamserviceaccount --cluster=capstone1 --namespace=shop --name=shop-ksa \
  --attach-policy-arn=arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess --approve
```

Reference the SA in the backend Deployment:

```
spec:
  serviceAccountName: shop-ksa
```

Verification

```
# PSA
kubectl -n shop run bad --image=nginx --dry-run=server -o yaml    # should be rejected
# RBAC
kubectl auth can-i delete pod --as=dev@acme.io -n shop            # yes
kubectl auth can-i delete pod --as=dev@acme.io -n kube-system     # no
# NetworkPolicy
kubectl -n shop exec -it deploy/frontend -- curl backend:8080/healthz   # 200
kubectl -n shop exec -it deploy/frontend -- curl google.com             # blocked
```

Cleanup

Revert labels and delete policies; delete IAM roles/GSAs.

Troubleshooting

- **PSA blocks legitimate Pod** — audit first (`warn` + `audit`), then fix.
- **NetworkPolicy has no effect** — CNI must support them (Cilium, Calico, GKE DPv2, AWS VPC CNI with policy add-on).

Bonus

- Add Kyverno policy requiring `runAsNonRoot: true`.
- Add Falco DaemonSet for runtime detection.
- Add image signature verification (cosign + Kyverno).

🏆 Capstone 8 — GitOps Deployment Using ArgoCD on GKE and EKS with Multi-Environment (dev/stage/prod) Sync

Difficulty: Advanced **Estimated time**: 3 hours

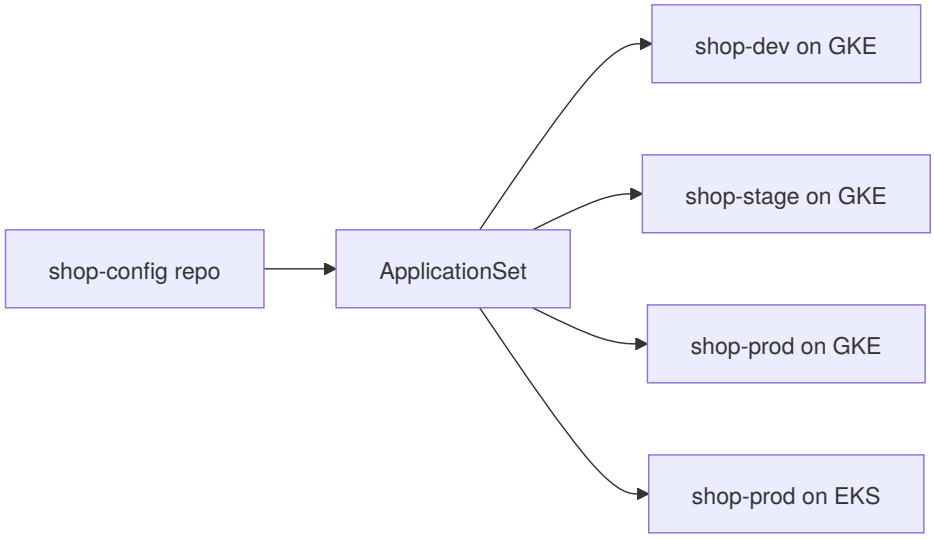
Objective

One config repo, three environments (dev/stage/prod), two clusters (GKE + EKS), one ArgoCD hub managing everything.

Prerequisites

- GKE + EKS clusters.
- GitHub repo `shop-config`.
- ArgoCD.

Workflow



Repo layout

```
shop-config/
├── charts/shop/               # from Capstone 2
├── envs/
│   ├── dev/values.yaml
│   ├── stage/values.yaml
│   └── prod/values.yaml
└── argocd/
    └── applicationset.yaml
```

Install ArgoCD (hub cluster = GKE)

```
kubectl create ns argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl -n argocd rollout status deploy/argocd-server
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d
```

Register EKS as a target

```
kubectl config use-context arn:aws:eks:us-east-1:111111111111:cluster/capstone1
argocd cluster add arn:aws:eks:us-east-1:111111111111:cluster/capstone1
```

ApplicationSet

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata: { name: shop, namespace: argocd }
spec:
  generators:
    - list:
        elements:
          - env: dev
            cluster: https://kubernetes.default.svc      # GKE hub
            valuesFile: envs/dev/values.yaml
          - env: stage
            cluster: https://kubernetes.default.svc
            valuesFile: envs/stage/values.yaml
          - env: prod-gke
            cluster: https://kubernetes.default.svc
            valuesFile: envs/prod/values.yaml
          - env: prod-eks
            cluster: https://<eks-endpoint>              # from argocd cluster list
            valuesFile: envs/prod/values.yaml
  template:
    metadata: { name: 'shop-{{env}}' }
    spec:
      project: default
      source:
        repoURL: https://github.com/acme/shop-config
        targetRevision: main
        path: charts/shop
        helm:
          valueFiles: ['../../{{valuesFile}}']
      destination:
        server: '{{cluster}}'
        namespace: 'shop-{{env}}'
      syncPolicy:
        automated: { prune: true, selfHeal: true }
        syncOptions: [CreateNamespace=true, ServerSideApply=true]
```

Verification

- ArgoCD UI shows 4 Applications, all `Synced/Healthy`.
- `kubectl -n shop-dev get pods` on GKE and `kubectl -n shop-prod get pods` on both clusters.

Cleanup

```
kubectl delete applicationset shop -n argocd
kubectl delete ns argocd
```

Troubleshooting

- **Application** `OutOfSync` — inspect diff in UI.
- **EKS cluster registration fails** — ensure argocd SA on hub has kubeconfig access.
- **Auto-sync causes drift war** — someone editing manually; add RBAC to lock out.

Bonus

- Add **Argo Rollouts** for canary in prod.
- Add **Sealed Secrets** for encrypted values.
- Add **ApplicationSet PR generator** for per-PR ephemeral envs.

🏆 Capstone 9 — Build and Deploy a Custom Operator Using Kubebuilder on GKE and EKS

Difficulty: Advanced **Estimated time**: 4 hours

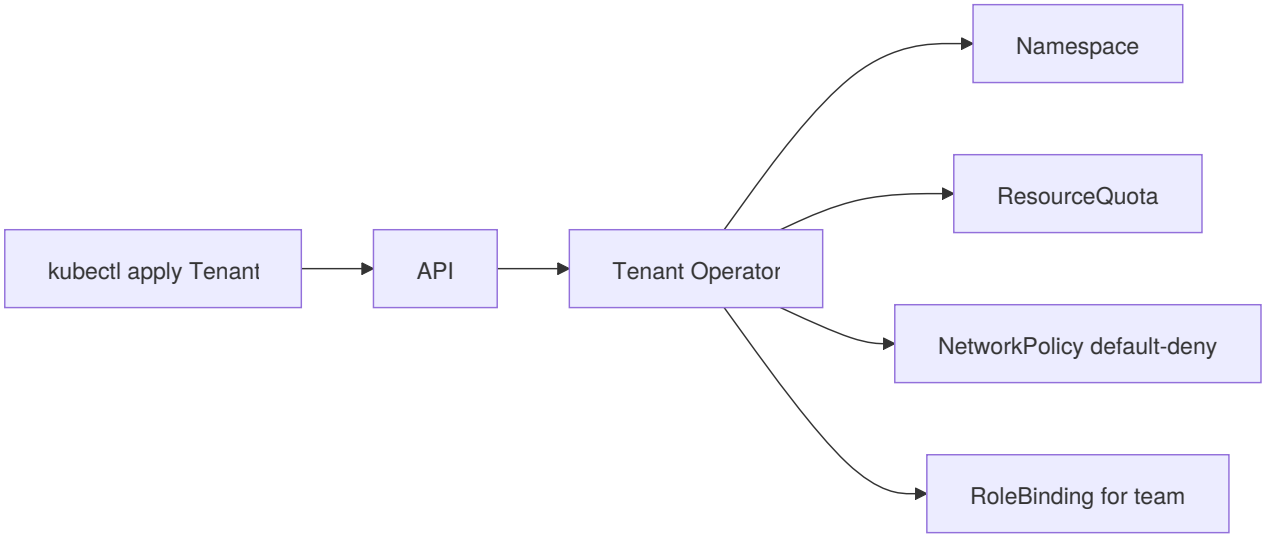
Objective

Create a `Tenant` CRD + controller that, when a `Tenant` object is applied, creates a Namespace + ResourceQuota + NetworkPolicy + baseline RoleBinding.

Prerequisites

- Go 1.22+, kubebuilder v3.14+, Docker, kubectl, kind (for local test), GKE + EKS clusters, container registry.

Workflow



Scaffold

```
mkdir tenant-op && cd tenant-op
kubebuilder init --domain acme.io --repo github.com/acme/tenant-op
kubebuilder create api --group platform --version v1 --kind Tenant --resource --controller
```

CRD types (`api/v1/tenant_types.go`)

```
type TenantSpec struct {
    Team    string `json:"team"`
    CPU     string `json:"cpu"`
    Memory  string `json:"memory"`
    Admins  []string `json:"admins"`
}
type TenantStatus struct {
    Namespace string `json:"namespace,omitempty"`
    Ready     bool   `json:"ready,omitempty"`
}
```

Reconciler sketch (`internal/controller/tenant_controller.go`)

```
func (r *TenantReconciler) Reconcile(ctx context.Context, req ctrl.Request) (ctrl.Result, error) {
    var t platformv1.Tenant
    if err := r.Get(ctx, req.NamespacedName, &t); err != nil {
        return ctrl.Result{}, client.IgnoreNotFound(err)
    }
    nsName := "team-" + t.Spec.Team

    // 1. Namespace
    ns := &corev1.Namespace{ObjectMeta: metav1.ObjectMeta{Name: nsName, Labels: map[string]string{"tenant": t.Name}}}
    _ = ctrl.SetControllerReference(&t, ns, r.Scheme)
    if err := r.Client.Patch(ctx, ns, client.Apply, client.ForceOwnership, client.FieldOwner("tenant-op")); err != nil { return ctrl.Result{}, err }

    // 2. ResourceQuota
    rq := &corev1.ResourceQuota{
        ObjectMeta: metav1.ObjectMeta{Name: "default", Namespace: nsName},
        Spec: corev1.ResourceQuotaSpec{ Hard: corev1.ResourceList{
            corev1.ResourceRequestsCPU:    resource.MustParse(t.Spec.CPU),
            corev1.ResourceRequestsMemory: resource.MustParse(t.Spec.Memory),
        }},
    }
    _ = ctrl.SetControllerReference(&t, rq, r.Scheme)
    if err := r.Client.Patch(ctx, rq, client.Apply, client.ForceOwnership, client.FieldOwner("tenant-op")); err != nil { return ctrl.Result{}, err }

    // 3. NetworkPolicy default-deny
    np := &netv1.NetworkPolicy{
        ObjectMeta: metav1.ObjectMeta{Name: "default-deny", Namespace: nsName},
        Spec: netv1.NetworkPolicySpec{
            PodSelector: metav1.LabelSelector{},
            PolicyTypes: []netv1.PolicyType{netv1.PolicyTypeIngress, netv1.PolicyTypeEgress},
        },
    }
    _ = ctrl.SetControllerReference(&t, np, r.Scheme)
    _ = r.Client.Patch(ctx, np, client.Apply, client.ForceOwnership, client.FieldOwner("tenant-op"))

    // 4. RoleBinding for admins
    if len(t.Spec.Admins) > 0 {
        rb := &rbacv1.RoleBinding{
            ObjectMeta: metav1.ObjectMeta{Name: "team-admin", Namespace: nsName},
            RoleRef: rbacv1.RoleRef{Kind: "ClusterRole", Name: "admin", APIGroup: "rbac.authorization.k8s.io"},
        }
        for _, u := range t.Spec.Admins {
            rb.Subjects = append(rb.Subjects, rbacv1.Subject{Kind: "User", Name: u, APIGroup: "rbac.authorization.k8s.io"})
        }
        _ = ctrl.SetControllerReference(&t, rb, r.Scheme)
        _ = r.Client.Patch(ctx, rb, client.Apply, client.ForceOwnership, client.FieldOwner("tenant-op"))
    }

    t.Status.Namespace = nsName
    t.Status.Ready = true
    return ctrl.Result{}, r.Status().Update(ctx, &t)
}
```

Build and deploy

```
make manifests generate fmt vet
make docker-build docker-push IMG=myrepo/tenant-op:0.1
make deploy IMG=myrepo/tenant-op:0.1
```

Sample Tenant

```
apiVersion: platform.acme.io/v1
kind: Tenant
metadata: { name: shop }
spec:
  team: shop
  cpu: "10"
  memory: 20Gi
  admins: [alice@acme.io, bob@acme.io]
```

```
kubectl apply -f tenant-shop.yaml
kubectl get ns team-shop
kubectl -n team-shop get quota,networkpolicy,rolebinding
```

Verification

- Namespace, quota, netpol, rolebinding all owned by the Tenant CR.
- Deleting the Tenant garbage-collects children.

Cleanup

```
kubectl delete tenant shop
make undeploy
```

Troubleshooting

- **Reconcile errors** — `kubectl -n tenant-op-system logs deploy/tenant-op-controller-manager`.
- **PSA blocks** — mark op namespace `enforce=privileged` if the SA needs it (usually not).
- **RBAC missing** — regenerate via `make manifests` after adding `//+kubebuilder:rbac` markers.

Bonus

- Add a validating webhook that rejects Tenants with too-large quotas.
- Publish as an OLM bundle.

- Ship a metric `tenant_ready_total` for observability.

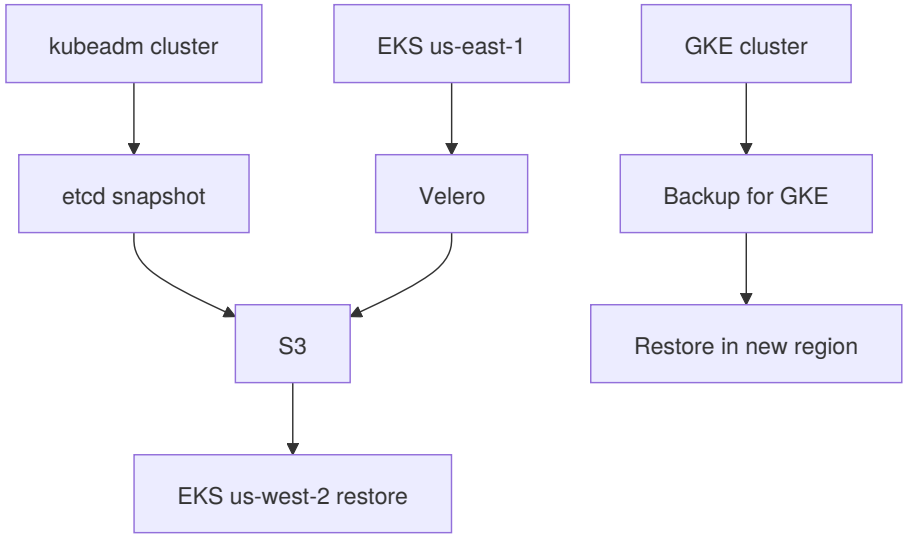
🏆 Capstone 10 — Disaster Recovery: etcd Backup/Restore on kubeadm + Backup for GKE + Velero Cross-Region on EKS

Difficulty: Advanced **Estimated time**: 4 hours

Objective

Practice three DR flows: raw etcd snapshot/restore, GKE workload backup and cross-region restore, EKS Velero cross-region restore.

Workflow



Part A — kubeadm etcd DR

Prerequisites: kubeadm HA cluster (or single-node lab).

Snapshot

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/snap.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
aws s3 cp /tmp/snap.db s3://etcd-dr/snap-$(date +%F-%H%M).db
```

Break — delete a namespace as chaos.

Restore

```
# Stop kubelet + move existing data
systemctl stop kubelet
mv /var/lib/etcd /var/lib/etcd.old

ETCDCTL_API=3 etcdctl snapshot restore /tmp/snap.db \
  --name=<node-name> \
  --initial-cluster=<node-name>=https://<node-ip>:2380 \
  --initial-advertise-peer-urls=https://<node-ip>:2380 \
  --data-dir=/var/lib/etcd

systemctl start kubelet
```

Verify namespace back.

Part B — Backup for GKE

Enable

```
gcloud services enable gkebackup.googleapis.com
gcloud beta container backup-restore backup-plans create shop-plan \
  --project=my-project --location=us-central1 \
  --cluster=projects/my-project/locations/us-central1/clusters/shop \
  --include-volume-data --include-secrets \
  --cron-schedule="0 */6 * * *" --backup-retain-days=30 \
  --all-namespaces
```

Take a backup manually

```
gcloud beta container backup-restore backups create shop-b1 \
  --location=us-central1 --backup-plan=shop-plan --wait-for-completion
```

Restore in another region

```
gcloud beta container backup-restore restore-plans create shop-restore \
  --location=us-west1 \
  --backup-plan=projects/my-project/locations/us-central1/backupPlans/shop-plan \
  --cluster=projects/my-project/locations/us-west1/clusters/shop-dr \
  --all-namespaces --namespaced-resource-restore-mode=DELETE_AND_RESTORE \
  --volume-data-restore-policy=RESTORE_VOLUME_DATA_FROM_BACKUP

gcloud beta container backup-restore restores create shop-r1 \
  --location=us-west1 --restore-plan=shop-restore \
  --backup=projects/my-project/locations/us-central1/backupPlans/shop-plan/backups/shop-b1
```

Part C — Velero on EKS Cross-Region

Install (in both clusters, same S3 bucket)

```
velero install \
  --provider aws --plugins velero/velero-plugin-for-aws:v1.9.0 \
  --bucket velero-dr --backup-location-config region=us-east-1 \
  --snapshot-location-config region=us-east-1 \
  --secret-file ./credentials-velero
```

(Prefer IRSA over static credentials.)

Backup

```
velero backup create shop-daily --include-namespaces=shop,db --wait
```

Restore in us-west-2 cluster

```
aws eks update-kubeconfig --name shop-dr --region us-west-2
velero restore create --from-backup shop-daily --wait
```

Verification

- kubeadm: deleted objects reappear post-restore.
- GKE: new region cluster has shop workloads + PV data.
- EKS: shop namespace up in us-west-2 with S3-backed PVs.

Cleanup

```
# kubeadm — keep or reset per your lab
gcloud beta container backup-restore backup-plans delete shop-plan --location=us-central1 --quiet
velero uninstall
```

Troubleshooting

- **Restore fails on immutable PVs** — use `--restore-volume-data-from-backup` and matching StorageClass in target.
- **CSI snapshots missing** — install VolumeSnapshot CRDs + external snapshotter.
- **Cross-region latency** — expect longer restore times; measure and document RTO.

Bonus

- Automate weekly restore game-days.
- Add Alertmanager alert on failed backups.
- Use **immutable S3 bucket** (Object Lock) to protect from ransomware.

🗓 Capstone Lab Index

#	Title	Difficulty	Time	Cloud
1	3-tier app on Minikube / Autopilot / Fargate	B–I	3h	Local + GCP + AWS
2	CI/CD via GitHub Actions to GKE + EKS	I	2.5h	GCP + AWS
3	HPA + CA + Karpenter under load	I	2h	GCP + AWS
4	Ingress + TLS (cert-manager)	I	2h	GCP + AWS
5	Observability stack (Prom/Grafana/Loki/AM)	I–A	3h	GCP + AWS
6	Postgres StatefulSet + backup/restore	I–A	3h	GCP + AWS
7	Security stack (RBAC/NP/PSA/WLI/IRSA)	A	3h	GCP + AWS
8	GitOps ArgoCD multi-env dev/stage/prod	A	3h	GCP + AWS
9	Custom Operator with Kubebuilder	A	4h	GCP + AWS
10	DR: etcd + Backup for GKE + Velero cross-region	A	4h	Local + GCP + AWS

⏸ Pause Point — End of Batch 5 of 5

Completed: - ✅ Batch 1 (57–61) - ✅ Batch 2 (62–66) - ✅ Batch 3 (67–70) - ✅ Batch 4 (71–73) - ✅ Batch 5 — **10 Capstone Labs**

Final step remaining: compile everything (Modules 1–13) into `Kubernetes_Complete_Interview_Study_Notes.pdf` with cover, TOC, indexes, and cheat-sheet

appendix.

Action needed from you: please attach your **Modules 1–9 files (Topics 1–56)**. Without them I can only compile Modules 10–13 (Topics 57–73 + Capstones). When you upload them, I'll: 1. Merge every module in order. 2. Render Mermaid diagrams inline. 3. Apply the color theme (blue headings, green sub-headings, dark code blocks, red warnings, orange tips, purple interview answers, teal scenarios, light-green labs). 4. Generate the cover page, clickable TOC, page numbers, Lab Index, kubectl command index, interview question index, and cheat-sheet appendix. 5. Deliver the final PDF.

Reply with your Modules 1–9 files (markdown or text is fine), or say **"compile with modules 10–13 only"** and I'll produce a partial PDF containing only what's been generated in this project.

Appendix — Cheat Sheet & Indexes

Top kubectl Commands

`kubectl get pods -A -o wide`
All Pods, all namespaces, with node+IP

`kubectl describe pod <p>`
Spec, events, status

`kubectl logs -f <p> --previous`
Follow logs incl. crashed container

`kubectl exec -it <p> -- sh`
Shell into a container

`kubectl debug -it <p> --image=busybox --target=<c>`
Ephemeral debug container

`kubectl top pods/nodes`
Live CPU/mem (needs metrics-server)

`kubectl rollout status/undo deploy/x`
Watch or roll back a Deployment

`kubectl scale deploy/x --replicas=5`
Manual scale

`kubectl set image deploy/x c=img:v2`
Trigger a rolling update

`kubectl auth can-i verb res --as=user`
RBAC test / impersonate

`kubectl apply -f dir/  / -k overlay/`
Declarative apply / Kustomize

`kubectl diff -f d.yaml`
Live vs proposed diff

`kubectl get events --sort-by=.lastTimestamp`
Recent cluster events

`kubectl drain/cordon/uncordon node`
Node maintenance lifecycle

`kubectl port-forward svc/x 8080:80`
Local tunnel to a Service

`kubectl explain pod.spec --recursive`
Inline schema reference

`kubectl config use-context / kubens`
Switch cluster / namespace

`kubectl label / annotate <obj> k=v`
Add metadata

`helm upgrade --install x ./c --atomic`
Idempotent, auto-rollback release

`argocd app sync x  / flux reconcile ks x`
Force a GitOps sync

Top 25 Troubleshooting Quick-Refs

1. **Pod Pending** → describe → resources / PVC / taints / affinity
2. **CrashLoopBackOff** → logs --previous → exit code
3. **ImagePullBackOff** → image name / registry auth / secret
4. **OOMKilled (137)** → raise memory limit / VPA recommend
5. **Node NotReady** → journalctl -u kubelet / CNI
6. **DNS fail** → CoreDNS pods + NetworkPolicy for :53
7. **Service 503** → get endpoints + readinessProbe
8. **Terminating stuck** → finalizers
9. **PVC Pending** → StorageClass / CSI driver / zone
10. **Multi-attach** → RWO on two nodes; use RWX or wait detach
11. **HPA no-scale** → metrics-server + requests set
12. **CA no nodes** → max size / instance type / tolerations
13. **Unauthorized** → refresh kubeconfig token
14. **Forbidden** → auth can-i + add RoleBinding
15. **Ingress 404/502** → ingressClass + backend endpoints
16. **TLS not issued** → describe certificate/order/challenge
17. **NetworkPolicy blocks** → allow-DNS egress first
18. **Rolling update stuck** → readinessProbe / PDB
19. **429 apiserver** → APF FlowSchema at workload-high

20. **Cluster slow** → etcd disk latency / defrag / compact
21. **Config not reloaded** → env=restart; volume=~60s
22. **Autopilot rejects pod** → drop privileged/hostPath; add requests
23. **Fargate Pending** → no matching Fargate profile
24. **Zone failure Pending** → zonal PD; use regional/app replication
25. **Dev works, prod fails** → PSA / IRSA / version differences

Capstone Lab Index

#	Title	Difficulty	Time	Cloud
1	3-tier app on Minikube / Autopilot / Fargate	B–I	3h	Local+GCP+AWS
2	CI/CD via GitHub Actions to GKE + EKS	I	2.5h	GCP+AWS
3	HPA + CA + Karpenter under load	I	2h	GCP+AWS
4	Ingress + TLS (cert-manager)	I	2h	GCP+AWS
5	Observability stack (Prom/Grafana/Loki/AM)	I–A	3h	GCP+AWS
6	Postgres StatefulSet + backup/restore	I–A	3h	GCP+AWS
7	Security stack (RBAC/NP/PSA/WLI/IRSA)	A	3h	GCP+AWS
8	GitOps ArgoCD multi-env dev/stage/prod	A	3h	GCP+AWS
9	Custom Operator with Kubebuilder	A	4h	GCP+AWS
10	DR: etcd + Backup for GKE + Velero cross-region	A	4h	Local+GCP+AWS

Module Map (Interview Question Index)

Module	Topics	Focus
Pre	Lab Setup	Minikube, Kind, kubeadm, GKE, EKS, cost safety
1	1–5	What/why K8s, VMs vs containers, architecture, cluster setup, kubectl
2	6–12	Pods, ReplicaSets, Deployments, StatefulSets, DaemonSets, Jobs, RCs
3	13–18	Labels, ConfigMaps, Secrets, Requests/Limits, Namespaces, Downward API
4	19–25	Networking model, Services, Ingress, NetworkPolicy, CNI, DNS, Mesh
5	26–31	Volumes, PV, PVC, StorageClasses, CSI, StatefulSet storage
6	32–37	Scheduler, affinity, taints, topology spread, priority, custom schedulers
7	38–45	AuthN/Z, RBAC, SAs, PSA, security contexts, secrets, netpol, admission
8	46–50	Logging, monitoring, events/debugging, probes, tracing
9	51–56	HPA, VPA, CA, Karpenter/NAP, PDB, multi-zone/region HA
10	57–66	CRDs, Operators, Helm, Kustomize, GitOps, admission, aggregation, etcd, multi-cluster, APF
11	67–70	GKE Std vs Autopilot, EKS node models, WLI vs IRSA, upgrades
12	71–73	50 troubleshooting + 30 system design + behavioral hybrids
13	Capstones	10 end-to-end lab projects

🎓 **End of Kubernetes Complete Interview Study Notes**

13 Modules · Pre-Module + 73 Topics + 10 Capstone Labs · ~500 interview Q&A

Compiled 27 September 2026 · Asia/Kolkata